EXPLO-STORY

A 2D adventure RPG for Sejarah KSSM Tingkatan 4
Bab 1: Warisan Negara Bangsa · Bab 2: Kebangkitan Nasionalisme

WORK COMPILATION

Game design, mechanics, story, source code and assets

OWNER	CAVAN VENTURES 202603046170 (AS0510258-T)
AUTHORS	1. AHMAD WILDAN BIN MAZANI (050722100499) (MALAYSIAN) 2. MIKAEL HAQIMI BIN NAHAR JUNAIDI (051020010557) (MALAYSIAN) 3. MUHAMMAD ADAM ASHRAFF BIN ZAMRI (051009040085) (MALAYSIAN) 4. MUHAMMAD KHAIRIL HAKIM BIN ISMAIL (050414011099) (MALAYSIAN)
VERSION	8003de7
DATE	27 October 2026
ENGINE	Godot Engine 4 (GDScript)
PLATFORM	Web (HTML5), Windows

Contents

Game Overview . . . 4

- Summary . . . 4
- Work details . . . 4
- Core loop . . . 4
- Game flow . . . 4

Game Mechanics . . . 5

- World structure: Arka, Zon and Misi . . . 5
- Exploration . . . 5
- Mission puzzles . . . 5
- Rewards and progression . . . 6
- Ujian Bab (chapter test) . . . 6
- Bilik Pertarungan and Gelanggang Pesta Silat . . . 6
- Arena Ulang Kaji . . . 7
- Profile, shop and skins . . . 7
- Accessibility and settings . . . 7
- Controls . . . 7

Story and Characters . . . 9

- Premise . . . 9
- Bab 1: Warisan Negara Bangsa . . . 9
- Bab 2: Kebangkitan Nasionalisme . . . 9
- Cast . . . 10

Learning Content . . . 12

- Bab 1: Bab 1 Warisan Negara Bangsa . . . 12
- Bab 2: Bab 2 Kebangkitan Nasionalisme . . . 13

World Map . . . 14

Project Structure . . . 15

- Autoloads . . . 15
- Code modules . . . 15
- Scenes . . . 16

Source Code . . . 17

- scripts/audio/ . . . 19
- scripts/autoload/ . . . 22
- scripts/battle/ . . . 33
- scripts/brawl/ . . . 38
- scripts/dev/ . . . 54

scripts/ . . . 68
scripts/puzzles/ . . . 84
scripts/ . . . 97
scripts/screens/ . . . 97
scripts/ . . . 126
scripts/ui/ . . . 127
scripts/world/ . . . 175
shaders/ . . . 216
Game Data . . . 218
Asset Catalogue . . . 252
assets/backgrounds/ (22) . . . 252
assets/backgrounds/tajuk/ (1) . . . 252
assets/characters/ (1) . . . 253
assets/icons/ (27) . . . 253
assets/items/ (10) . . . 254
assets/objects/ (80) . . . 254
assets/objects/bilik/ (1) . . . 257
assets/portraits/ (63) . . . 257
assets/proto/bg/ (8) . . . 260
assets/proto/objects/ (21) . . . 260
assets/proto/sprites/ (13) . . . 261
assets/tiles/ (6) . . . 261
assets/ui/ (4) . . . 262
assets/characters32/ (182 sprite sets, 1496 frames) . . . 262
Audio (21) . . . 266
Fonts and other media (8) . . . 267

CHAPTER 1

Game Overview

Summary

Explo-Story is a 2D top-down pixel-art adventure RPG for learning History (Sejarah KSSM, Form 4). The player is **Alif**, a Form 4 student who falls asleep the night before a History test and wakes up inside the Explo-Story world, where the villain **Nirbana** is spreading the *Kabus Lupa* (Fog of Forgetting) to make everyone forget their history.

Each chapter of the syllabus becomes a region of the world. Each subtopic becomes a mission (*Misi*) with its own place, guide character, puzzles and reward artifact. Finishing a mission clears the fog from that place and brings its colour back.

Work details

Item	Detail
Title	Explo-Story
Genre	2D adventure RPG / educational game
Subject	Sejarah KSSM Tingkatan 4
Chapters included	Bab 1: Warisan Negara Bangsa; Bab 2: Kebangkitan Nasionalisme
Target players	Secondary school students (age 15–16), including dyslexia-friendly reading options
Engine	Godot Engine 4 (GDScript), GL Compatibility renderer
Platforms	Web (HTML5) for desktop browsers and phones; Windows desktop
Languages	Bahasa Melayu (game content); BM / English (menus)
Online features	Optional login and cloud save; plays fully offline as a guest

Core loop

1. **Explore** the open world on a tile grid. Talk to villagers, read signs and follow the gold arrow to the active mission.
2. **Enter a mission area** (greyed out by Kabus Lupa) and meet its guide character.
3. **Solve the mission's puzzles**, which teach the subtopic through matching, sorting, investigation boards and objective questions. There is no timer and no penalty.
4. **Earn the reward artifact**, EXP and coins. The area regains its colour and a page is added to *Nota Alif*.
5. **Clear the chapter test** (*Ujian Bab*) by defeating the chapter boss, which opens the next chapter.
6. **Revise and compete** in the Bilik Pertarungan / Gelanggang Pesta Silat, where correct answers power up attacks.

Game flow

Title → Login / Register / Guest → Prologue (watch-only, skippable) → Alif's bedroom (wake-up and tutorial) → Kampung Jejak → Misi 1.1 to 1.4 → Ujian Bab 1 → Pintu Kabus → Lembah Kebangkitan → Misi 2.1 to 2.8 → Ujian Bab 2 → Ending.

CHAPTER 2

Game Mechanics

World structure: Arka, Zon and Misi

The syllabus is mapped onto three levels, shown on the **Arka & Zon** screen:

Level	Meaning	Example
Arka	School form (Tingkatan)	Sejarah Tingkatan 4
Zon	Chapter (Bab)	Bab 2: Kebangkitan Nasionalisme
Misi	Subtopic, played as one mission area	Misi 2.7: Maktab Perguruan Sultan Idris

Locked chapters are shown with their titles so players can see the whole path ahead.

Exploration

- **Grid movement:** the player walks one tile at a time (WASD / arrow keys, Shift to run) or taps a destination. A pathfinder routes Alif there.
- **Pau the companion** follows Alif and gives hints on request (H key or tapping Pau). Hints are logged for teachers.
- **Guidance:** an exclamation mark (!) marks the next task. A gold arrow on the minimap, the full map and the screen edge points to the active mission.
- **Kabus Lupa:** mission areas start grey under a thin white fog and regain colour once their mission is complete. The minimap and full map show the same fog.
- **Living world:** villagers wander. Chickens, ducks, fish, butterflies and birds animate the world, along with kitchen smoke and lamps.
- **Maps:** a minimap with three zoom levels, plus a full map with a legend of places, keywords and symbols. Keywords can be searched and tapped to show their location.
- **Jalan Pintas (fast travel):** jump straight to any unlocked place from the bag or the full map.
- **Area title cards** introduce each place on first visit and explain why Alif has come.

Mission puzzles

Each mission is a sequence of steps and puzzles defined in data. Teachers can edit questions, answers and text without touching code.

Puzzle type	How it plays
Seret dan Padankan (drag & match)	Drag items onto their matching targets, e.g. kingdoms onto their period
Papan Siasatan (investigation board)	Pin evidence cards to the correct question on a cork board
Objektif	Multiple-choice questions with an explanation after each answer
Suaikan	Pair related items from two columns
World puzzles	Act on the map itself, e.g. putting the five kingdom gates in order (Funan, Champa, Srivijaya, Melaka, Johor-Riau)

There is no timer and no failure penalty. A wrong answer gives a gentle explanation and the player tries again.

Rewards and progression

- **EXP and levels**, shown on the HUD with coins.
- **Artifacts:** each mission rewards a historical artifact, stored in **Beg Alif**. Tapping an artifact plays its story.
- **Kad Tokoh:** character cards for the historical figures met.
- **Nota Alif:** an illustrated notebook. One page unlocks per mission and summarises the subtopic.
- **Peta Jejak:** a stone monument that tracks chapter progress with flags.
- **Replay:** finished missions can be replayed for practice. Replays give no further rewards.

Ujian Bab (chapter test)

At the end of each chapter the player faces a boss: **Panglima Derhaka** (Bab 1) or **Bayang Nirbana** (Bab 2).

- Questions are drawn at random from the chapter's question bank (12 per test).
- Alif has **3 lives**. **7 correct answers** defeat the boss. There is no time limit.
- The test must be passed to open the next chapter. On failure the player can retry or revise, and the boss waits on the map.

Bilik Pertarungan and Gelanggang Pesta Silat

A revision arena where answering correctly powers up attacks. The question bank can be set to Bab 1, Bab 2 or Campur (mixed).

Quiz battle: answer correctly to attack a chosen opponent. The last one standing wins 20 coins. Bot opponents are four classmates: Aisyah, Ravi, Mei Ling and Daniel.

Gelanggang Pesta Silat is a side-view action brawl set in the school hall. Modes are 1 vs 1, 2 vs 2, 1 vs 2 and 1 vs 3, on Easy, Normal or Hard. Each round has three phases:

1. **SOALAN (15 s):** everyone answers the same question.
2. **KEPUTUSAN:** a correct answer gives *Semangat* (full-strength attacks) and drops a school item into the ring. The fastest correct answer earns *Jurus Kilat* (a special move). A wrong answer gives *Lemah* (30% damage).
3. **AKSI (10 s):** free fighting with punches, kicks, the J-J-K combo, flying kicks, blocking, and picking up and throwing items.

Effects are cartoon-only (stars, "PAK!", plasters, dizzy "PENGSAN"), with no blood.

Move data (damage × 1.0 with Semangat, × 0.3 when Lemah)

Move	Duration (s)	Damage	Effect
jab	0.26	4	“PAK”
jab2	0.26	4	“PAK”
straight	0.36	7	“DUSH”
kick	0.44	7	“BAM”
roundhouse	0.55	12	knock-down, “POW!”
flykick	0.5	10	knock-down, lunge, “WHAM”
dash	0.4	8	lunge, “DUSH”
special	0.85	18	knock-down, hits all round (140 px), “JURUS KILAT!”
swing	0.42	0	“PAP!”

Pick-up items

School item	Damage	Reach	Uses	Thrown damage
Pembaris Panjang	8	120	4	7
Buku Teks Tebal	10	92	3	9
Penyapu	7	130	5	6
Beg Sekolah	9	98	3	9
Bola Keranjang	6	84	6	9
Bantal	5	96	6	5

Brawl controls: WASD move, J punch, K kick, Space jump, L pick up / throw, I block, U special, Shift run. On phones these become a joystick / D-pad and action buttons.

Arena Ulang Kaji

A training yard on each island (Kampung Jejak, and beside the padang in Lembah Kebangkitan). It has a lobby, other players sparring in pairs, training dummies and a **Papan Markah** leaderboard (Overall / State / Friends, ranked by total EXP). Guru Jalal offers the side quest *Jurus Ingatan* after the first fragment is recovered.

Profile, shop and skins

Each player has a Player ID and a profile picture that matches the outfit worn. Coins buy outfits for Alif in the **Kedai** (shop):

Outfit	Price (coins)	Description
Baju Sekolah	Free	Kemeja putih, seluar biru. Alif yang biasa.
Baju Melayu	Free	Baju Melayu hijau, samping dan songkok.
Pengakap	120	Sedia selalu! Lengkap dengan skarf dan beret.
Baju Sukan	150	Jersi rumah sukan biru dan ikat kepala kuning.
Kadet Bomba	200	Pakaian kadet bomba dengan jalur pemantul.
Pahlawan Melaka	300	Tengkolok merah, samping songket dan keris kecil.

Accessibility and settings

- **Mod Mudah Baca (easy-read mode):** a choice of Lexend, Atkinson Hyperlegible or OpenDyslexic fonts, adjustable text size and text speed, and read-aloud voice.
- **Momen Tenang:** a calm-down overlay opened by holding Pau.
- Separate Music, Sound and Voice volumes. Menus in Bahasa Melayu or English.
- Interactive music that changes with the area and the situation.

Controls

Action	Keyboard	Phone / mouse
Walk one tile	WASD / arrow keys, Shift to run	D-pad, or tap anywhere
Talk / use	E, Space, Enter	A button, or tap a character or object
Pau's hint	H	Tap Pau
Calm moment	Pau button in menu bar	Long-press Pau
Full map and legend	M	Tap the minimap

Action	Keyboard	Phone / mouse
Settings	Esc	Gear button

CHAPTER 3

Story and Characters

Premise

It is 11:47 pm, the night before a History test. Alif's class group chat is busy while he sits in his room, half-studying. As he reads, keywords drift in and out of "Ingatan Alif" (Alif's memory) and the letters scatter. He falls asleep.

In his dream the port is in chaos: classmates no longer recognise each other inside the **Kabus Lupa**. In the **Gua Warisan** (Cave of Heritage), **Nirbana**, the hooded Penghulu of the Kabus Lupa, shatters the **Inskripsi Telaga Batu** into four fragments (*Serpihan*). He wants everyone to forget their history and give up.

Alif wakes in his bedroom inside the Explo-Story world. His companion **Pau** explains the artifact, the four fragments and the mission. Alif picks up his school bag, and Misi 1.1 begins.

Bab 1: Warisan Negara Bangsa

Alif travels across the first island, from Kampung Jejak to the four mission areas. Each mission recovers one fragment of the inscription, and each fragment restores colour to its area. Nirbana taunts Alif along the way. When all four fragments are back, Alif faces **Panglima Derhaka** in Ujian Bab 1. The chapter closes with the first flag raised on the Peta Jejak, a new page in Nota Alif, and a teaser for Bab 2.

Misi	Place	Subtopic and guiding question	Guide	Puzzles	Reward
1.1	Gerbang Masa	**1.1 Negara Bangsa Kerajaan Alam Melayu** *Dari mana asal negara bangsa kita?*	Penjaga Gerbang Masa	drag match, suaikan	Serpihan 1: Batu Asas Telaga Batu
1.2	Pelabuhan Melaka	**1.2 Ciri-ciri Negara Bangsa Kesultanan Melayu Melaka** *Apa yang menjadikan Melaka sebuah negara bangsa yang lengkap?*	Syahbandar	drag match, drag match, objektif, drag match, drag match, suaikan	Serpihan 2: Cap Mohor
1.3	Balai Penghadapan	**1.3 Keunggulan Sistem Pentadbiran dan Perundangan** *Bagaimana Melaka kekal aman dan makmur?*	Bendahara Tun Perak	drag match, board	Serpihan 3: Naskhah Hukum Kanun
1.4	Bukit Siguntang	**1.4 Peranan Pemerintah dan Rakyat dalam Pembentukan Negara Bangsa** *Apa yang berlaku jika pemerintah atau rakyat mungkir janji?*	Demang Lebar Daun	drag match, objektif, board, choice	Serpihan 4: Warkah Waadat

Bab 2: Kebangkitan Nasionalisme

After Bab 1, the **Pintu Kabus** (Fog Gate) west of Kampung Jejak opens onto a second island: **Lembah Kebangkitan** (Valley of Awakening). It is an orderly 1920s colonial town with a Jalan Besar, a padang with a wakaf and a clock tower, rows of shophouses, and a steamship at the Dermaga.

The setting runs from the 1900s to 1941. The Kabus Lupa greys out every area until its mission is done. The eight missions follow one route, with a numbered sign in front of each place. Nirbana mocks Alif every two missions. In Misi 2.8, **Gabenor Sekatan** guards the Menara Pentadbiran. The chapter ends with Ujian Bab 2 against **Bayang Nirbana**.

Story interiors include the **Jelutong Press** printing house and the **Maktab SITC** (Sultan Idris Training College).

Misi	Place	Subtopic and guiding question	Guide	Puzzles	Reward
2.1	Dewan Kedaulatan Rakyat	**2.1 Maksud Nasionalisme** *Apakah maksud nasionalisme?*	Profesor Sejarawan	objektif, suaikan	Kitab Tafsiran Nasionalisme
2.2	Rumah Falsafah Sejagat	**2.2 Perkembangan Idea Nasionalisme di Barat** *Dari mana datangnya idea kedaulatan rakyat?*	Profesor Sejarawan	drag match, suaikan, objektif	Bill of Rights & The Social Contract
2.3	Dermaga Asia	**2.3 Perkembangan Nasionalisme di Asia** *Bagaimana bangsa Asia bangkit menentang kuasa Barat?*	Nakhoda Warkah	drag match, objektif	Warkah al-Urwah al-Wuthqa
2.4	Balai Serantau	**2.4 Perkembangan Nasionalisme di Asia Tenggara** *Bagaimana jiran-jiran kita memperjuangkan kemerdekaan?*	Nakhoda Warkah	drag match, objektif	Akhbar Kalayan
2.5	Tugu Pahlawan	**2.5 Kesedaran Nasionalisme di Negara Kita** *Bagaimana kesedaran nasionalisme bermula di negara kita?*	Ketua Pengarang	drag match, objektif	Pena Kebangkitan
2.6	Kompleks Intelektual & Jelutong Press	**2.6 Faktor Kemunculan Gerakan Nasionalisme** *Apakah faktor yang mencetuskan gerakan nasionalisme?*	Syed Sheikh al-Hadi	drag match, drag match, objektif	Akhbar Jawi Peranakan
2.7	Maktab Perguruan Sultan Idris	**2.7 Perkembangan Nasionalisme** *Bagaimana nasionalisme berkembang di negara kita?*	Za'ba	objektif, drag match, suaikan, suaikan	Naskhah Al-Imam (1906)
2.8	Menara Pentadbiran Kolonial	**2.8 Kesan Perkembangan Nasionalisme** *Apakah kesan perkembangan nasionalisme?*	Ibu Zain	drag match, objektif	Majalah Bulan Melayu

Cast

Character	Role	Description
Alif	Main character	Form 4 student; the player.
Pau	Companion	Follows Alif, explains missions and gives hints.
Penjaga Gerbang Masa	Pemandu Misi 1.1 (Misi 1.1)	Watak rekaan. Berpakaian gabungan lima zaman dan menjaga benang warisan kerajaan awal Alam Melayu.
Syahbandar	Pemandu Misi 1.2 (Misi 1.2)	Watak rekaan. Pegawai pelabuhan Melaka yang sibuk mengurus kapal dan pedagang asing.
Laksamana Cheng Ho	Utusan Dinasti Ming (Misi 1.2)	Kunjungannya menjadi bukti kedaulatan Melaka diiktiraf oleh kuasa luar.
Bendahara Tun Perak	Bendahara Melaka (Misi 1.3)	Pembesar yang bijaksana dan tenang. Membantu sultan mentadbir Melaka.
Sang Sapurba	Pihak pemerintah dalam Waadat (Misi 1.4)	Tokoh dalam Sejarah Melayu. Berjanji memerintah dengan adil.
Demang Lebar Daun	Pihak rakyat dalam Waadat (Misi 1.4)	Tokoh dalam Sejarah Melayu. Berjanji rakyat akan taat setia.
Sultan		
Pedagang		
Mak		

Character	Role	Description
Aisyah		
Ravi		
Mei Ling		
Daniel		
Mak Som		
Pak Mat		
Profesor Sejarawan	Pemandu Misi 2.1 dan 2.2 (Misi 2.1)	Watak rekaan. Sejarawan yang menjaga Dewan Kedaulatan Rakyat dan suka bercerita tentang idea besar.
Nakhoda Warkah	Pemandu Misi 2.3 dan 2.4 (Misi 2.3)	Watak rekaan. Nakhoda yang membawa warkah perjuangan dari seluruh Asia.
Ketua Pengarang	Pemandu Misi 2.5 (Misi 2.5)	Watak rekaan. Ketua pengarang akhbar Melayu tahun 1930-an yang percaya pena lebih tajam daripada pedang.
Syed Sheikh al-Hadi	Pelopor Islah (Misi 2.6)	Tokoh Kaum Muda dan penggerak Al-Imam (1906) serta pendidikan madrasah.
Za'ba	Pendeta Pena (Misi 2.7)	Ketua Penterjemah Pejabat Karang Mengarang di SITC. Kerap mengkritik dasar pendidikan British.
Mohammad Eunos Abdullah	Pengasas KMS (Misi 2.7)	Menubuhkan Kesatuan Melayu Singapura pada 1926 untuk memperjuangkan kepentingan orang Melayu.
Ibrahim Haji Yaakob	Pengasas KMM (Misi 2.7)	Menubuhkan Kesatuan Melayu Muda pada Mei 1937 dan memperjuangkan gagasan Melayu Raya melalui akhbar Majlis.
Ibu Zain	Srikandi Ilmu (Misi 2.8)	Hajah Zainon Munshi Sulaiman. Penggerak majalah Bulan Melayu (1930) dan Tabung Derma Pelajaran Tinggi untuk anak perempuan Melayu.
Nirbana	Fictional	Watak rekaan. Penghulu Kabus Lupa. Dia mahu semua orang lupa sejarah dan putus asa.
Panglima Derhaka	Fictional	
Gabenor Sekatan	Fictional	
Pak Hamid	Fictional	
Pelatih SITC	Fictional	
Guru Jalal	Fictional	
Amir	Fictional	
Hafiz	Fictional	

CHAPTER 4

Learning Content

Each mission adds a page to **Nota Alif**, and each chapter has a question bank used by the Ujian Bab and the Bilik Pertarungan (Bab 1, Bab 2 or Campur).

Bab 1: Bab 1 Warisan Negara Bangsa

Nota Alif pages (6)

Misi	Page title	Key points
1.1	Garis Masa Kerajaan	Funan: abad pertama; Champa: abad kedua; Srivijaya: abad ke-7, Palembang; Kesultanan Melayu Melaka: abad ke-15
1.1	Empat Tiang Negara Bangsa	Raja: Raja Champa dikaitkan dengan Dewa Siva; Undang-undang: Kutara Manawa (Majapahit); Wilayah Pengaruh: Funan: Sungai Mekong hingga Segenting Kra; Rakyat: Sumpah setia Prasasti Telaga Batu
1.2	Enam Panji Melaka	Kerajaan: Sultan + Sistem Pembesar Empat Lipatan; Rakyat: Pemerintah dan diperintah, timbal balik; Kedaulatan: Diiktiraf Dinasti Ming; Wilayah Pengaruh: Taklukan, naungan, perkahwinan
1.3	Pembesar Melaka	Bendahara: Memangku sultan; Penghulu Bendahari: Perbendaharaan dan cukai; Syahbandar: Pelabuhan dan cukai kapal; Temenggung: Keamanan dalam kota
1.3	Undang-undang Melaka	UULM Fasal 9: Malim lalai » pelayaran selamat, pedagang yakin; HKM Fasal 30: Riba diharamkan » perdagangan adil; UULM Fasal 13: Nakhoda bermesyuarat » kurang pertikaian; HKM: bahasa istana: Titah hanya untuk raja » kedaulatan terpelihara
1.4	Pohon dan Akar (Waadat)	Sang Sapurba (pemerintah): Adil, tidak zalim, jaga kebajikan; Demang Lebar Daun (rakyat): Taat setia, tidak derhaka, patuh; Rantaian kukuh: Raja adil » rakyat setia » serah & kerah » kerajaan stabil; Rantaian runtuh: Mungkir janji » kepercayaan hilang » kerajaan goyah

Question bank: 32 questions. By subtopic: 1.1 (9), 1.2 (10), 1.3 (9), 1.4 (4)

Sample questions

Kerajaan manakah yang paling awal?
A. Kesultanan Johor-Riau
B. Srivijaya
C. **Funan** (answer)
D. Kesultanan Melayu Melaka

Yang manakah BUKAN Pembesar Berempat?
A. Laksamana
B. **Syahbandar** (answer)
C. Bendahara
D. Temenggung

Pahang ialah wilayah ___ Melaka.
A. **Taklukan** (answer)
B. Perkahwinan
C. Naungan
D. Pajakan

Bab 2: Bab 2 Kebangkitan Nasionalisme

Nota Alif pages (8)

Misi	Page title	Key points
2.1	Tafsiran Nasionalisme	Maksud: Cinta mendalam pada bangsa dan negara; menentang penjajahan; Hans Kohn: Kesetiaan ulung hanya kepada negara atau bangsa; William R. Roff: Ancaman hak Melayu, ekonomi, sekularisme dan imperialisme Barat; Zainal Abidin Abdul Wahid: Wujud sejak Kesultanan Melayu Melaka
2.2	Revolusi di Barat	England 1688: Revolusi Keagungan, Bill of Rights, John Locke; Amerika 1776: "Tiada cukai tanpa perwakilan", Thomas Jefferson; Perancis 1789: Rousseau, The Social Contract, penjara Bastile
2.3	Nasionalisme di Asia	India: Satyagraha (Gandhi); Tilak dan Pal; China: San Min Chu-i, Revolusi 1911; Mesir: Islah, al-Urwah al-Wuthqa (1883); Jepun: Pemulihan Meiji 1868, Bushido
2.4	Asia Tenggara	Filipina: Jose Rizal, Katipunan, akhbar Kalayan; Burma: Isu Kasut (1906), golongan Thakin; Vietnam: Tribune Indige, Cloche Felee; Indonesia: Kartini, Sumpah Pemuda 1928
2.5	Dari Senjata ke Pena	Sultan Abdullah Mukarram Shah: Kedah, mahu dapatkan kembali Pulau Pinang; Pahlawan setempat: Dol Said, Maharaja Lela, Bahaman, Rentap, Mat Salleh, Tok Janggut, Abdul Rahman Limbong; Strategi baharu: Penulisan dan persatuan, lebih sederhana tetapi tersusun
2.6	Enam Faktor	Agama: Islah dari Mesir; madrasah 1914, 1916, 1926; Penulisan kreatif: Putera Gunung Tahan (Pak Sako); Media cetak: Jawi Peranakan, Fajar Sarawak; Pendidikan: SITC, MCKK, Raffles Institution
2.7	Tiga Landasan Perkembangan	1902 Khalsa Diwan: Masyarakat Sikh, Taiping; 1906 Al-Imam: Kaum Muda, Singapura; 1926 KMS: Mohammad Eunos Abdullah; 1934 PASPAM: Pulau Pinang
2.8	Kesan Nasionalisme	Politik: Pegawai Melayu MCS: 7 (1930) ke 20 (1937); Ekonomi: al-Jamiah al-Khairiah, semangat berniaga; Sosial: Guru SITC 58 ke 141; Bulan Melayu 1930

Question bank: 41 questions. By subtopic: 2.1 (5), 2.2 (4), 2.3 (5), 2.4 (8), 2.5 (2), 2.6 (3), 2.7 (10), 2.8 (4)

Sample questions

2.1 Apakah maksud asas nasionalisme dalam konteks perjuangan sesebuah bangsa?
A. Usaha meluaskan tanah jajahan ke benua lain melalui kekuatan tentera
B. Dasar menutup sempadan negara daripada sebarang hubungan perdagangan luar
C. Sistem pentadbiran yang menyerahkan kuasa mutlak sepenuhnya kepada golongan bangsawan
D. **Perasaan cinta yang mendalam terhadap bangsa dan negara serta perjuangan menentang penjajahan** (answer)

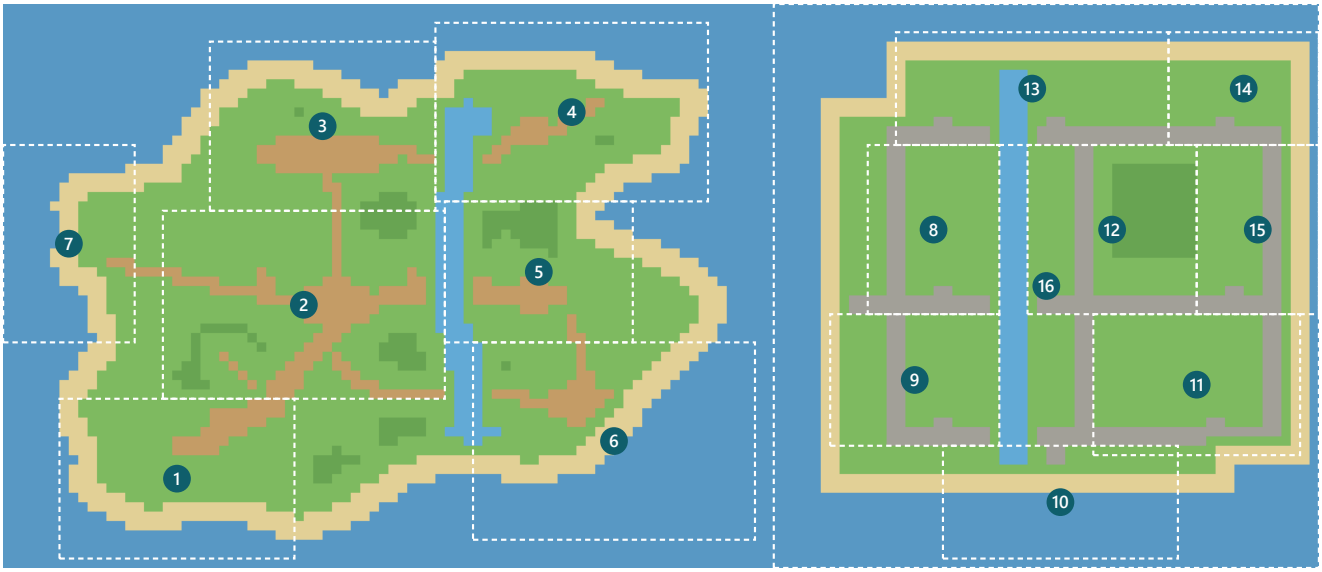
2.3 Semasa era Pemulihan Meiji (1868), apakah prinsip yang dijadikan asas membina nasionalisme Jepun yang bermaksud sikap sanggup menanggung risiko dan sedia berkorban demi ketua dan negara?
A. Satyagraha
B. **Semangat Bushido** (answer)
C. Islah
D. San Min Chu-i

2.6 Akhbar manakah yang diterajui oleh angkatan intelektual Melayu di Sarawak seperti Muhammad Rakawi Yusuf sebagai wadah mengkritik dasar pemerintahan Brooke yang mengabaikan hak bumiputera?
A. Lembaga Melayu
B. Utusan Sarawak
C. Lidah Teruna
D. **Fajar Sarawak** (answer)

CHAPTER 5

World Map

The world is a 140 × 60 tile grid with two islands. Bab 1 is on the left island (Kampung Jejak and its four mission areas). Bab 2, Lembah Kebangkitan, is on the right island, reached through the Pintu Kabus portal. Numbered boxes mark the named areas.



Sea · Grass · Meadow · Sand · Gravel road (Bab 2) · Dirt road · River

#	Area	Tiles (x0, y0 – x1, y1)
1	Bukit Permulaan	6, 42 – 30, 58
2	Kampung Jejak	17, 22 – 46, 41
3	Misi 1.1: Gerbang Masa	22, 4 – 45, 21
4	Misi 1.4: Bukit Siguntang	46, 2 – 74, 20
5	Misi 1.3: Kota Melaka	47, 21 – 66, 35
6	Misi 1.2: Pelabuhan Melaka	50, 36 – 79, 56
7	Kabus Masa	0, 15 – 13, 35
8	Misi 2.1: Dewan Kedaulatan Rakyat	92, 15 – 105, 32
9	Misi 2.2: Rumah Falsafah Sejagat	88, 33 – 105, 46
10	Misi 2.3: Dermaga Asia	100, 47 – 124, 58
11	Misi 2.4: Balai Serantau	116, 33 – 137, 47
12	Misi 2.5: Tugu Pahlawan	109, 15 – 126, 32
13	Misi 2.6: Kompleks Intelektual	95, 3 – 123, 14
14	Misi 2.7: Maktab SITC	124, 3 – 139, 14
15	Misi 2.8: Menara Pentadbiran	127, 15 – 139, 32
16	Lembah Kebangkitan	82, 0 – 139, 59

CHAPTER 6

Project Structure

Folder	Contents	Files
scripts/	GDScript game code	72
scenes/	Godot scenes	11
shaders/	Shaders	3
data/	Game content (JSON)	37
assets/	Art, audio and fonts	1793

Autoloads

Singleton	Script
GameState	res://scripts/game_state.gd
Backend	res://scripts/autoload/backend.gd
Game	res://scripts/autoload/game.gd
Tts	res://scripts/autoload/tts.gd
Dialogue	res://scripts/dialogue.gd
Sfx	res://scripts/audio/sfx.gd
MusicDirector	res://scripts/audio/music_director.gd
Main	res://scripts/main.gd

Code modules

Module	Files	Lines
scripts/ (root)	8	1,372
scripts/audio/	2	246
scripts/autoload/	3	942
scripts/battle/	3	397
scripts/brawl/	6	1,250
scripts/dev/	1	1,177
scripts/puzzles/	6	1,085
scripts/screens/	10	2,336
scripts/ui/	21	3,958
scripts/world/	12	3,417
Total	72	**16,180**

Scenes

Scene	Root node	Nodes
scenes/interior/balai.tscn	Balai (Node2D)	9
scenes/interior/bilik_alif.tscn	BilikAlif (Node2D)	9
scenes/interior/jelutong.tscn	Jelutong (Node2D)	9
scenes/interior/sitc.tscn	Sitc (Node2D)	9
scenes/pau.tscn	Pau (Node2D)	2
scenes/player.tscn	Player (Node2D)	3
scenes/prologue.tscn	Prologue (Control)	15
scenes/screen_host.tscn	ScreenHost (Control)	2
scenes/title.tscn	Title (Control)	6
scenes/ui/peta_jejak.tscn	PetaJejak (CanvasLayer)	6
scenes/world/world.tscn	World (Node2D)	16

APPENDIX A

Source Code

Complete, unmodified listings of all 75 source files (GDScript and shaders), grouped by folder, with line numbers.

File	Lines
scripts/audio/music_director.gd	161
scripts/audio/sfx.gd	85
scripts/autoload/backend.gd	307
scripts/autoload/game.gd	592
scripts/autoload/tts.gd	43
scripts/battle/battle_rules.gd	120
scripts/battle/local_battle.gd	181
scripts/battle/online_battle.gd	96
scripts/brawl/brawl_ai.gd	110
scripts/brawl/brawl_fighter.gd	498
scripts/brawl/brawl_fx.gd	82
scripts/brawl/brawl_match.gd	446
scripts/brawl/brawl_rules.gd	58
scripts/brawl/weapon_art.gd	56
scripts/dev/autotest.gd	1177
scripts/dialogue.gd	44
scripts/game_state.gd	83
scripts/main.gd	286
scripts/pau.gd	62
scripts/player.gd	180
scripts/prologue.gd	648
scripts/puzzles/board.gd	241
scripts/puzzles/choice.gd	38
scripts/puzzles/drag_match.gd	257
scripts/puzzles/objektif.gd	238
scripts/puzzles/puzzle_base.gd	148
scripts/puzzles/suaikan.gd	163
scripts/screen_host.gd	17
scripts/screens/arka.gd	162
scripts/screens/gelanggang.gd	495
scripts/screens/lobby.gd	303
scripts/screens/login.gd	132
scripts/screens/penutup.gd	191
scripts/screens/pertarungan.gd	417

File	Lines
scripts/screens/register.gd	157
scripts/screens/ujian_bab.gd	192
scripts/screens/zone_prep.gd	132
scripts/screens/zone_summary.gd	155
scripts/title.gd	52
scripts/ui/beg_panel.gd	186
scripts/ui/brawl_guide.gd	136
scripts/ui/calm_overlay.gd	76
scripts/ui/dialogue_box.gd	270
scripts/ui/hud.gd	225
scripts/ui/map_overlay.gd	396
scripts/ui/map_view.gd	379
scripts/ui/minimap.gd	82
scripts/ui/modal.gd	55
scripts/ui/nota_panel.gd	167
scripts/ui/objective_arrow.gd	74
scripts/ui/papan_panel.gd	222
scripts/ui/pau_button.gd	94
scripts/ui/peta_jejak.gd	84
scripts/ui/phone_chat.gd	131
scripts/ui/profile_panel.gd	264
scripts/ui/settings_panel.gd	281
scripts/ui/start_prompt.gd	71
scripts/ui/title_backdrop.gd	98
scripts/ui/touch_controls.gd	267
scripts/ui/ui.gd	400
scripts/world/ambience.gd	259
scripts/world/arena_yard.gd	152
scripts/world/bilik_alif.gd	234
scripts/world/bilik_art.gd	367
scripts/world/character_frames.gd	91
scripts/world/critters.gd	116
scripts/world/interior.gd	377
scripts/world/npc.gd	96
scripts/world/pathfind.gd	41
scripts/world/terrain_builder.gd	138
scripts/world/world.gd	1267
scripts/world/zone_runner.gd	279
shaders/kabus_lupa.gdshader	34

File	Lines
shaders/sway.gdshader	13
shaders/water.gdshader	24

scripts/audio/

music_director.gd · scripts/audio/music_director.gd · 161 lines

```
extends Node
## Autoload "MusicDirector": interactive music (tools/gen_music.py renders the
## gamelan loops). Each area has two stems that loop in sync:
##   base   - always on
##   melody - rises while Alif explores, settles when he stands still
## Changing area crossfades on the next bar line. Puzzles add the "fokus"
## layer. Dialogue ducks everything. Stingers play on top.

const DIR := "res://assets/audio/music/"
const BPM := {
    "tajuk": 76, "bilik": 64, "permulaan": 84, "kampung": 92, "gerbang": 70,
    "pelabuhan": 108, "melaka": 80, "siguntang": 66,
}
const AREA_TRACK := {  # world area id -> track
    "permulaan": "permulaan", "kampung": "kampung", "gerbang": "gerbang",
    "pelabuhan": "pelabuhan", "melaka": "melaka", "siguntang": "siguntang",
    "kabus": "siguntang", "balai": "melaka",
    # Lembah Kebangkitan (Bab 2) borrows the island's tracks for now
    "lembah": "kampung", "dewan": "melaka", "falsafah": "gerbang", "dermaga": "pelabuhan",
    "serantau": "pelabuhan", "tugu": "siguntang", "kompleks": "kampung", "sitc": "melaka", "menara": "gerbang",
}
const MELODY_ON := -3.0
const MELODY_OFF := -22.0

var current := ""
var _a: AudioStreamPlayer
var _b: AudioStreamPlayer
var _fokus: AudioStreamPlayer
var _sting: AudioStreamPlayer
var _melody_db := MELODY_ON
var _ducked := false
var _switch_id := 0


func _ready() -> void:
    process_mode = Node.PROCESS_MODE_ALWAYS
    _a = _player()
    _b = _player()
    _fokus = _player()
    _sting = _player()
    _sting.bus = "SFX" if AudioServer.get_bus_index("SFX") >= 0 else "Master"


func _player() -> AudioStreamPlayer:
    var p := AudioStreamPlayer.new()
    p.bus = "Music" if AudioServer.get_bus_index("Music") >= 0 else "Master"
    p.volume_db = -80
    add_child(p)
    return p


func _loop(path: String) -> AudioStream:
    var s: AudioStreamWAV = load(path)
    if s == null:
        return null
    s.loop_mode = AudioStreamWAV.LOOP_FORWARD
    s.loop_begin = 0
    s.loop_end = int(s.get_length() * s.mix_rate)
    return s


func _track(name: String) -> AudioStream:
    var sync := AudioStreamSynchronized.new()
    sync.stream_count = 2
    sync.set_sync_stream(0, _loop(DIR + name + "_base.wav"))
    sync.set_sync_stream(1, _loop(DIR + name + "_melody.wav"))
    sync.set_sync_stream_volume(1, _melody_db)
    return sync


## Play a track ("tajuk", "bilik") or a world area id ("kampung", "balai").
```

```
func play(name: String, fade := 1.6) -> void:
    name = AREA_TRACK.get(name, name)
    if name == current or not BPM.has(name):
        return
    _switch_id += 1
    var my_id := _switch_id
    # wait for the next bar of the current track so the change feels musical
    if current != "" and _a.playing:
        var bar := 4.0 * 60.0 / float(BPM[current])
        var wait := bar - fmod(_a.get_playback_position(), bar)
        if wait < 2.5:
            await get_tree().create_timer(wait).timeout
        if my_id != _switch_id:
            return
    current = name
    var old := _a
    _a = _b
    _b = old
    _a.stream = _track(name)
    _a.volume_db = -40
    _a.play()
    var t := create_tween().set_parallel()
    t.tween_property(_a, "volume_db", _target_db(), fade)
    if old.playing:
        t.tween_property(old, "volume_db", -60.0, fade)
        t.chain().tween_callback(old.stop)


func stop(fade := 1.2) -> void:
    current = ""
    var t := create_tween().set_parallel()
    for p in [_a, _b, _fokus]:
        if p.playing:
            t.tween_property(p, "volume_db", -60.0, fade)
    t.chain().tween_callback(func():
        for p in [_a, _b, _fokus]:
            p.stop())


## melody layer up (exploring) or down (standing still)
func set_melody(on: bool) -> void:
    var target := MELODY_ON if on else MELODY_OFF
    if is_equal_approx(target, _melody_db):
        return
    var t := create_tween()
    t.tween_method(_set_melody_db, _melody_db, target, 2.5 if on else 4.0)


func _set_melody_db(v: float) -> void:
    _melody_db = v
    var s = _a.stream
    if s is AudioStreamSynchronized:
        s.set_sync_stream_volume(1, v)


## puzzles: a quiet focused pulse joins the music
func puzzle(on: bool) -> void:
    if on and not _fokus.playing:
        _fokus.stream = _loop(DIR + "fokus.wav")
        _fokus.volume_db = -40
        _fokus.play()
    var t := create_tween()
    t.tween_property(_fokus, "volume_db", -8.0 if on else -60.0, 1.5)
    if not on:
        t.tween_callback(_fokus.stop)
    set_melody(not on)


func duck(on: bool) -> void:
    _ducked = on
    var t := create_tween()
    t.tween_property(_a, "volume_db", _target_db(), 0.4)


func _target_db() -> float:
    return -9.0 if _ducked else -2.0


func stinger(name: String) -> void:
    var s: AudioStream = load(DIR + name + ".wav")
    if s == null:
        return
    _sting.stream = s
    _sting.volume_db = 0.0
    _sting.play()
```

```
    # let the stinger breathe
    var t := create_tween()
    t.tween_property(_a, "volume_db", -14.0, 0.2)
    t.tween_interval(maxf(s.get_length() - 1.5, 0.5))
    t.tween_property(_a, "volume_db", _target_db(), 1.2)
```

sfx.gd · scripts/audio/sfx.gd · 85 lines

```
extends Node
## Autoload "Sfx": retro sound effects synthesised in code (no audio files),
## so they cost nothing to download on the web. Sfx.play("tick").

const RATE := 22050
const POOL := 8

var _sounds := {}
var _players: Array[AudioStreamPlayer] = []
var _next := 0


func _ready() -> void:
    for i in POOL:
        var p := AudioStreamPlayer.new()
        p.bus = "SFX" if AudioServer.get_bus_index("SFX") >= 0 else "Master"
        add_child(p)
        _players.append(p)
    _sounds = {
        "tick": _make(0.05, func(t, _i): return _noise() * exp(-t * 120.0) * 0.5 + sin(TAU * 2200 * t) * exp(-t * 90.0) * 0.4),
        "tock": _make(0.06, func(t, _i): return _noise() * exp(-t * 110.0) * 0.4 + sin(TAU * 1500 * t) * exp(-t * 80.0) * 0.4),
        "heartbeat": _make(0.5, func(t, _i): return _thump(t, 0.0) + _thump(t, 0.18) * 0.7),
        "squish": _make(0.22, func(t, _i): return sin(TAU * (520.0 - 380.0 * t / 0.22) * t) * exp(-t * 9.0) * 0.5),
        "whoosh": _make(1.2, func(t, _i): return _noise() * sin(PI * t / 1.2) * 0.35),
        "chime": _make(1.6, func(t, _i): return (sin(TAU * 880 * t) + sin(TAU * 1320 * t) * 0.6 + sin(TAU * 1760 * t) * 0.3) * exp(-t * 2.4) * 0.25),
        "sparkle": _make(0.9, func(t, _i): return sin(TAU * (1200.0 + 1400.0 * fmod(t * 6.0, 1.0)) * t) * exp(-t * 3.5) * 0.18),
        "pop": _make(0.12, func(t, _i): return sin(TAU * (300.0 + 900.0 * t / 0.12) * t) * exp(-t * 25.0) * 0.5),
        "bump": _make(0.1, func(t, _i): return sin(TAU * 90 * t) * exp(-t * 30.0) * 0.7),
        "blip": _make(0.04, func(t, _i): return signf(sin(TAU * 620 * t)) * exp(-t * 60.0) * 0.12),
        "select": _make(0.08, func(t, _i): return signf(sin(TAU * (880.0 if t < 0.04 else 1320.0) * t)) * 0.12 * (1.0 - t / 0.08)),
        "step_grass": _make(0.07, func(t, _i): return _noise() * exp(-t * 60.0) * 0.18),
        "step_sand": _make(0.09, func(t, _i): return _noise() * exp(-t * 40.0) * 0.14),
        "step_stone": _make(0.05, func(t, _i): return (_noise() * 0.5 + sin(TAU * 180 * t)) * exp(-t * 90.0) * 0.2),
        "step_wood": _make(0.08, func(t, _i): return (sin(TAU * 140 * t) + _noise() * 0.3) * exp(-t * 50.0) * 0.25),
        # Gelanggang (brawl): cartoon hits, never gory
        "punch": _make(0.12, func(t, _i): return (sin(TAU * (180.0 - 600.0 * t) * t) * 0.8 + _noise() * 0.5) * exp(-t * 38.0) * 0.6),
        "kick": _make(0.16, func(t, _i): return (sin(TAU * (120.0 - 300.0 * t) * t) + _noise() * 0.6) * exp(-t * 26.0) * 0.6),
        "swish": _make(0.14, func(t, _i): return _noise() * sin(PI * t / 0.14) * 0.22),
        "thwack": _make(0.14, func(t, _i): return (signf(sin(TAU * 330 * t)) * 0.4 + _noise() * 0.6) * exp(-t * 34.0) * 0.45),
        "fall": _make(0.3, func(t, _i): return _thump(t, 0.0) * 0.9 + _noise() * exp(-t * 18.0) * 0.25),
        "bell": _make(1.4, func(t, _i): return (sin(TAU * 660 * t) + sin(TAU * 1650 * t) * 0.5 + sin(TAU * 2310 * t) * 0.25) * exp(-t * 3.0) * 0.28),
    }


## bus: "SFX", or "Voice" for the dialogue voice blips (own volume slider)
func play(name: String, volume_db := 0.0, pitch := 1.0, bus := "SFX") -> void:
    if not _sounds.has(name):
        return
    var p := _players[_next]
    _next = (_next + 1) % POOL
    p.bus = bus if AudioServer.get_bus_index(bus) >= 0 else "Master"
    p.stream = _sounds[name]
    p.volume_db = volume_db
    p.pitch_scale = pitch
    p.play()


## same sound with a little random pitch so repeats never sound looped
func play_varied(name: String, volume_db := 0.0, spread := 0.08) -> void:
    play(name, volume_db, 1.0 + randf_range(-spread, spread))


func _make(seconds: float, fn: Callable) -> AudioStreamWAV:
    var n := int(seconds * RATE)
    var data := PackedByteArray()
    data.resize(n * 2)
```

```
    for i in n:
        var v: float = clampf(fn.call(float(i) / RATE, i), -1.0, 1.0)
        data.encode_s16(i * 2, int(v * 32767.0))
    var s := AudioStreamWAV.new()
    s.format = AudioStreamWAV.FORMAT_16_BITS
    s.mix_rate = RATE
    s.data = data
    return s


func _noise() -> float:
    return randf() * 2.0 - 1.0


func _thump(t: float, start: float) -> float:
    var u := t - start
    if u < 0.0:
        return 0.0
    return sin(TAU * (60.0 + 40.0 * exp(-u * 30.0)) * u) * exp(-u * 14.0) * 0.9
```

scripts/autoload/

backend.gd · scripts/autoload/backend.gd · 307 lines

```
extends Node
## Backend (autoload "Backend") for Module 1 (accounts) and cloud sync.
##
## Online: Supabase (Auth + PostgREST). Fill res://config/backend.json or let
## CI write it from the SUPABASE_URL / SUPABASE_ANON_KEY secrets.
## Offline (no config): accounts are kept on this device only, so the game
## always works, including on itch.io without a backend.

const CONFIG_PATH := "res://config/backend.json"
const LOCAL_ACCOUNTS := "user://akaun.json"

var url: String = ""
var anon_key: String = ""
var access_token: String = ""
var user_id: String = ""
var _push_pending: bool = false
var _push_timer: Timer


func _ready() -> void:
    if FileAccess.file_exists(CONFIG_PATH):
        var cfg = JSON.parse_string(FileAccess.get_file_as_string(CONFIG_PATH))
        if cfg is Dictionary:
            url = str(cfg.get("supabase_url", "")).strip_edges().trim_suffix("/")
            anon_key = str(cfg.get("supabase_anon_key", "")).strip_edges()
    _push_timer = Timer.new()
    _push_timer.one_shot = true
    _push_timer.wait_time = 3.0
    _push_timer.timeout.connect(_do_push)
    add_child(_push_timer)


func is_online() -> bool:
    return url != "" and anon_key != ""


func is_signed_in() -> bool:
    return access_token != ""


func mode_text() -> String:
    if is_online():
        return "Dalam talian: kemajuan disimpan di awan."
    return "Luar talian: akaun disimpan pada peranti ini sahaja."


# ---------------------------------------------------------------- HTTP
func _headers(auth: bool = false) -> PackedStringArray:
    var h := PackedStringArray(["Content-Type: application/json", "apikey: " + anon_key])
    if auth and access_token != "":
        h.append("Authorization: Bearer " + access_token)
    else:
        h.append("Authorization: Bearer " + anon_key)
    return h


func _request(path: String, method: int, body: Variant = null, auth: bool = false, extra: PackedStringArray = PackedStr
```

```
ingArray()) -> Dictionary:
    var http := HTTPRequest.new()
    http.timeout = 15.0
    add_child(http)
    var headers := _headers(auth)
    headers.append_array(extra)
    var payload := "" if body == null else JSON.stringify(body)
    var err := http.request(url + path, headers, method, payload)
    if err != OK:
        http.queue_free()
        return {"ok": false, "code": 0, "data": null}
    var res: Array = await http.request_completed
    http.queue_free()
    var code: int = res[1]
    var text: String = (res[3] as PackedByteArray).get_string_from_utf8()
    var data = null
    if text != "":
        data = JSON.parse_string(text)
    return {"ok": res[0] == HTTPRequest.RESULT_SUCCESS and code >= 200 and code < 300, "code": code, "data": data}


func _err_text(r: Dictionary, fallback: String) -> String:
    if r.get("code", 0) == 0:
        return "Tiada sambungan internet. Cuba lagi atau main sebagai tetamu."
    var d = r.get("data")
    if d is Dictionary:
        var m := str(d.get("msg", d.get("error_description", d.get("message", ""))))
        if m.to_lower().contains("invalid login"):
            return "E-mel atau kata laluan tidak betul."
        if m.to_lower().contains("already registered"):
            return "E-mel ini sudah didaftarkan."
        if m != "":
            return m
    return fallback


# ---------------------------------------------------------------- accounts (FR-AUTH-02 / 03)
## Returns {"ok": bool, "error": String, "username": String, "role": String, "avatar": String, "email": String}
func sign_up(username: String, email: String, password: String, role: String, avatar: String) -> Dictionary:
    if is_online():
        var r := await _request("/auth/v1/signup", HTTPClient.METHOD_POST, {
            "email": email, "password": password,
            "data": {"username": username, "role": role, "avatar": avatar},
        })
        if not r["ok"]:
            return {"ok": false, "error": _err_text(r, "Pendaftaran gagal.")}
        var d: Dictionary = r["data"] if r["data"] is Dictionary else {}
        if d.has("access_token"):
            _take_session(d)
            return {"ok": true, "error": "", "username": username, "role": role, "avatar": avatar, "email": email}
        return {"ok": true, "error": "", "username": username, "role": role, "avatar": avatar, "email": email,
            "confirm": "Akaun dicipta. Sila sahkan e-mel anda, kemudian log masuk."}
    var accts := _local_accounts()
    var key := username.to_lower()
    if accts.has(key):
        return {"ok": false, "error": "Nama pengguna sudah digunakan pada peranti ini."}
    for a in accts.values():
        if str(a.get("email", "")).to_lower() == email.to_lower():
            return {"ok": false, "error": "E-mel ini sudah didaftarkan pada peranti ini."}
    accts[key] = {"username": username, "email": email, "hash": _hash(username, password), "role": role, "avatar": avat
ar}
    Game.write_json_user(LOCAL_ACCOUNTS, accts)
    return {"ok": true, "error": "", "username": username, "role": role, "avatar": avatar, "email": email}


func sign_in(login_id: String, password: String) -> Dictionary:
    if is_online():
        if not login_id.contains("@"):
            return {"ok": false, "error": "Dalam talian, sila log masuk dengan e-mel."}
        var r := await _request("/auth/v1/token?grant_type=password", HTTPClient.METHOD_POST, {"email": login_id, "pass
word": password})
        if not r["ok"]:
            return {"ok": false, "error": _err_text(r, "Log masuk gagal.")}
        var d: Dictionary = r["data"]
        _take_session(d)
        var meta: Dictionary = d.get("user", {}).get("user_metadata", {})
        var result := {"ok": true, "error": "", "email": login_id,
            "username": str(meta.get("username", login_id.get_slice("@", 0))),
            "role": str(meta.get("role", "student")), "avatar": str(meta.get("avatar", "pelajar_1"))}
        var cloud = await pull_progress()
        if cloud is Dictionary and not cloud.is_empty():
            result["cloud_save"] = cloud
        return result
    var accts := _local_accounts()
    var found: Dictionary = {}
```

```
    for a in accts.values():
        if str(a.get("username", "")).to_lower() == login_id.to_lower() or str(a.get("email", "")).to_lower() == login_id.to_lower():
            found = a
    if found.is_empty():
        return {"ok": false, "error": "Akaun tidak dijumpai pada peranti ini. Daftar dahulu."}
    if found.get("hash", "") != _hash(str(found["username"]), password):
        return {"ok": false, "error": "Kata laluan tidak betul."}
    return {"ok": true, "error": "", "username": found["username"], "role": found.get("role", "student"),
        "avatar": found.get("avatar", "pelajar_1"), "email": found.get("email", "")}


func reset_password(email: String) -> String:
    if not is_online():
        return "Mod luar talian: kata laluan tidak boleh ditetapkan semula. Daftar akaun baharu atau main sebagai tetamu."
    var r := await _request("/auth/v1/recover", HTTPClient.METHOD_POST, {"email": email})
    if r["ok"]:
        return "Pautan tetapan semula telah dihantar ke e-mel anda."
    return _err_text(r, "Tidak dapat menghantar e-mel.")


func sign_out() -> void:
    access_token = ""
    user_id = ""


func _take_session(d: Dictionary) -> void:
    access_token = str(d.get("access_token", ""))
    user_id = str(d.get("user", {}).get("id", ""))


func _local_accounts() -> Dictionary:
    var a = Game.load_json_user(LOCAL_ACCOUNTS)
    return a if a is Dictionary else {}


func _hash(username: String, password: String) -> String:
    return ("explostory:" + username.to_lower() + ":" + password).sha256_text()


# ---------------------------------------------------------------------- progress sync (FR-PROG-04)
func push_progress(_save: Dictionary) -> void:
    if not (is_online() and is_signed_in()):
        return
    _push_pending = true
    if _push_timer.is_stopped():
        _push_timer.start()


func _do_push() -> void:
    if not _push_pending:
        return
    _push_pending = false
    var s: Dictionary = Game.save
    await _request("/rest/v1/player_progress", HTTPClient.METHOD_POST, {
        "user_id": user_id,
        "current_level": int(s.get("level", 1)),
        "total_coins": int(s.get("coins", 0)),
        "total_exp": int(s.get("exp", 0)),
        "save_data": s,
    }, true, PackedStringArray(["Prefer: resolution=merge-duplicates,return=minimal"]))
    var rows: Array = []
    for z in s.get("zones", {}).keys():
        var st: Dictionary = s["zones"][z]
        if int(st.get("total", 0)) > 0:
            rows.append({"user_id": user_id, "topic_id": "t4_b1_" + str(z),
                "correct_answers": int(st.get("first_try", 0)), "incorrect_answers": int(st.get("retries", 0)),
                "mastery_percentage": Game.mastery_percent(z)})
    if rows.size() > 0:
        await _request("/rest/v1/syllabus_mastery?on_conflict=user_id,topic_id", HTTPClient.METHOD_POST, rows, true,
            PackedStringArray(["Prefer: resolution=merge-duplicates,return=minimal"]))
    await push_stats()


func pull_progress() -> Variant:
    if not (is_online() and is_signed_in()):
        return null
    var r := await _request("/rest/v1/player_progress?select=save_data&user_id=eq." + user_id, HTTPClient.METHOD_GET, null, true)
    if r["ok"] and r["data"] is Array and r["data"].size() > 0:
        return r["data"][0].get("save_data", null)
    return null


```

```
## Anonymous hint usage for teacher analytics (no user id stored).
func log_hint(puzzle_id: String, tier: int) -> void:
    if not is_online():
        return
    _request("/rest/v1/hint_usage", HTTPClient.METHOD_POST, {"puzzle_id": puzzle_id, "tier": tier}, false,
        PackedStringArray(["Prefer: return=minimal"]))


# ---------------------------------------------------------------- Bilik Pertarungan
## Friends and battle rooms (supabase/multiplayer.sql). All need a signed-in
## online account; offline the game offers the bot battle instead.
## NOTE: not tested against a live Supabase project yet.

func can_play_online() -> bool:
    return is_online() and is_signed_in()


## Calls a Postgres function. {"ok", "code", "data"}
func db_call(fn: String, args: Dictionary = {}) -> Dictionary:
    if not can_play_online():
        return {"ok": false, "code": 0, "data": null}
    return await _request("/rest/v1/rpc/" + fn, HTTPClient.METHOD_POST, args, true)


## The server's Player ID replaces the local one so friends can find you.
func sync_player_code() -> void:
    var r := await db_call("ensure_player_code")
    if r["ok"] and r["data"] is String and str(r["data"]) != "":
        Game.save["player_id"] = str(r["data"])
        Game.persist()


func add_friend(code: String) -> String:
    var r := await db_call("add_friend", {"p_code": code.strip_edges().to_upper()})
    return str(r["data"]) if r["ok"] else "error"


func my_friends() -> Array:
    var r := await db_call("my_friends")
    return r["data"] if r["ok"] and r["data"] is Array else []


func _me_args() -> Dictionary:
    var p: Dictionary = Game.save["profile"]
    return {"p_name": str(p.get("display_name", p["username"])), "p_avatar": str(p.get("avatar", "pelajar_1")), "p_skin": Game.skin_folder()}


func create_room() -> String:
    var r := await db_call("create_room", _me_args())
    return str(r["data"]) if r["ok"] else ""


func join_room(code: String) -> String:
    var args := _me_args()
    args["p_code"] = code.strip_edges().to_upper()
    var r := await db_call("join_room", args)
    return str(r["data"]) if r["ok"] else "error"


func room_state(code: String) -> Dictionary:
    var r := await db_call("battle_state", {"p_code": code})
    return r["data"] if r["ok"] and r["data"] is Dictionary else {}


# ---------------------------------------------------------------- Papan Markah
## supabase/leaderboard.sql. NOTE: not tested against a live project yet.

## Sends name, outfit, negeri, total EXP and wins (rides on the debounced
## progress push, so at most every few seconds).
func push_stats() -> void:
    var p: Dictionary = Game.save["profile"]
    await db_call("update_my_stats", {"p_name": str(p.get("display_name", p["username"])), "p_skin": Game.skin_folder(),
        "p_negeri": str(p.get("negeri", "")), "p_exp": int(Game.save.get("exp", 0)),
        "p_wins": int(Game.save.get("battle", {}).get("wins", 0))})


## scope: "all" | "negeri" | "friends". {} when offline or on error, else
## {rows: [{rank, name, skin, negeri, exp, wins, me}], me: {...}}
func leaderboard(scope: String) -> Dictionary:
    var r := await db_call("leaderboard", {"p_scope": scope})
    return r["data"] if r["ok"] and r["data"] is Dictionary else {}
```

```
func start_battle(code: String) -> String:
    var r := await db_call("start_battle", {"p_code": code})
    return str(r["data"]) if r["ok"] else "error"
```

game.gd · scripts/autoload/game.gd · 592 lines

```
extends Node
## Global game state (autoload "Game"): content data, save file, settings,
## progression (Module 5), screen navigation and the shared theme.

const UI = preload("res://scripts/ui/ui.gd")

signal settings_changed
signal stats_changed
signal skin_changed
signal pau_pressed
signal hint_requested(tier: int)

## full scenes (the explorable world and cinematic prologue)
const SCENES := {
    "title": "res://scenes/title.tscn",
    "prolog": "res://scenes/prologue.tscn",
    "peta": "res://scenes/world/world.tscn",
    "world": "res://scenes/world/world.tscn",
}
## menu-style screens from the first prototype, hosted by scenes/screen_host.tscn
const SCREENS := {
    "login": "res://scripts/screens/login.gd",
    "register": "res://scripts/screens/register.gd",
    "zone_summary": "res://scripts/screens/zone_summary.gd",
    "penutup": "res://scripts/screens/penutup.gd",
    "ujian": "res://scripts/screens/ujian_bab.gd",
    "lobby": "res://scripts/screens/lobby.gd",
    "arka": "res://scripts/screens/arka.gd",
    "gelanggang": "res://scripts/screens/gelanggang.gd",
    "pertarungan": "res://scripts/screens/pertarungan.gd",
}

## every zone of every open chapter, in play order (filled from data/chapters.json)
var ZONES: Array = []
const EXP_PER_LEVEL := 200
const SETTINGS_PATH := "user://tetapan.json"
## for Papan Markah (leaderboard by negeri)
const NEGERI := [
    "Johor", "Kedah", "Kelantan", "Melaka", "Negeri Sembilan", "Pahang", "Perak", "Perlis",
    "Pulau Pinang", "Sabah", "Sarawak", "Selangor", "Terengganu",
    "WP Kuala Lumpur", "WP Labuan", "WP Putrajaya",
]

const DEFAULT_SETTINGS := {
    "font": "lexend", "text_size": 1, "text_speed": 1, "auto_read": false,
    "muted": false, "music": 0.8, "sfx": 0.8, "voice": 0.8, "lang": "ms",
    "controls": "auto", "pad_size": 1, "pad_alpha": 0.7, "left_handed": false,
}
const I18N_PATH := "res://data/i18n/en.json"  ## BM menu text -> English

var main: Node = null
var theme: Theme
var settings: Dictionary = {}
var save: Dictionary = {}
var characters: Dictionary = {}
var chapters: Dictionary = {}
var story: Dictionary = {}
var _zone_cache: Dictionary = {}
var _skins: Array = []
var _tr_en: Translation
var input_lock: int = 0
## screen waiting to be shown by scenes/screen_host.tscn
var pending_screen: String = ""
var pending_params: Dictionary = {}
## Set by the active screen so Pau can give a context hint outside puzzles.
var context_hint: String = ""
## Puzzle currently on screen (receives Pau hints), or null.
var active_puzzle: Node = null


func _ready() -> void:
    _setup_input()
    settings = DEFAULT_SETTINGS.duplicate()
    var s = load_json_user(SETTINGS_PATH)
    if s is Dictionary:
        for k in s.keys():
            settings[k] = s[k]
```

```
    theme = UI.build_theme(settings)
    _load_translations()
    characters = load_json("res://data/characters.json")
    chapters = load_json("res://data/chapters.json")
    for ch in chapter_list():
        ZONES.append_array(ch.get("zones", []))
    story = load_json("res://data/story.json")
    new_save("Tetamu", "guest")
    apply_audio.call_deferred()


# ---------------------------------------------------------------- data
func load_json(path: String) -> Dictionary:
    if not FileAccess.file_exists(path):
        push_error("Fail tidak dijumpai: " + path)
        return {}
    var txt := FileAccess.get_file_as_string(path)
    var data = JSON.parse_string(txt)
    if data is Dictionary:
        return data
    push_error("JSON rosak: " + path)
    return {}


func load_json_user(path: String) -> Variant:
    if not FileAccess.file_exists(path):
        return null
    return JSON.parse_string(FileAccess.get_file_as_string(path))


func write_json_user(path: String, data: Variant) -> void:
    var f := FileAccess.open(path, FileAccess.WRITE)
    if f:
        f.store_string(JSON.stringify(data, "  "))
        f.close()


func zone_data(id: String) -> Dictionary:
    if not _zone_cache.has(id):
        _zone_cache[id] = load_json("res://data/zon_%s.json" % id)
    return _zone_cache[id]


func lines(key: String) -> Array:
    var l = story.get(key, [])
    return l if l is Array else []


func char_name(id: String) -> String:
    return str(characters.get(id, {}).get("name", id))


func portrait_path(id: String, expr: String = "") -> String:
    var c: Dictionary = characters.get(id, {})
    var base: String = str(c.get("portrait", ""))
    if base == "":
        return ""
    if base.ends_with("_"):
        var e := expr if expr != "" else str(c.get("default_expr", "neutral"))
        if id == "alif" and not save.is_empty():  # Alif talks in the outfit he wears
            var sk := "res://assets/portraits/alif_%s_%s.png" % [str(save.get("skin", "sekolah")), e]
            if ResourceLoader.exists(sk):
                return sk
        var p := "res://assets/portraits/%s%s.png" % [base, e]
        if ResourceLoader.exists(p):
            return p
        return "res://assets/portraits/%s%s.png" % [base, c.get("default_expr", "neutral")]
    var own := "res://assets/portraits/%s.png" % base
    if not ResourceLoader.exists(own) and c.has("stand_in"):  # new character, art not dropped in yet
        return "res://assets/portraits/%s.png" % str(c["stand_in"].get("portrait", base))
    return own


func sprite_path(id: String) -> String:
    return "res://assets/proto/sprites/%s.png" % id


# ---------------------------------------------------------------- settings
func set_setting(key: String, value: Variant) -> void:
    settings[key] = value
    write_json_user(SETTINGS_PATH, settings)
    UI.apply_settings(theme, settings)
    apply_audio()
    if key == "lang":
        _set_language(str(value))
```

```
    settings_changed.emit()


## Menus can be in English; history content stays in BM (see data/i18n/en.json).
func _load_translations() -> void:
    _tr_en = Translation.new()
    _tr_en.locale = "en"
    var d = JSON.parse_string(FileAccess.get_file_as_string(I18N_PATH))
    if d is Dictionary:
        for k in d:
            if not str(k).begins_with("_"):
                _tr_en.add_message(str(k), str(d[k]))
    _set_language(str(settings.get("lang", "ms")))


## The English table is only registered while English is chosen: Godot falls
## back to English for any locale without its own table, so BM would show English.
func _set_language(lang: String) -> void:
    if lang == "en":
        TranslationServer.add_translation(_tr_en)
    else:
        TranslationServer.remove_translation(_tr_en)
    TranslationServer.set_locale(lang)


func apply_audio() -> void:
    AudioServer.set_bus_mute(0, bool(settings.get("muted", false)))
    for bus in [["Music", "music"], ["SFX", "sfx"], ["Ambient", "sfx"], ["Voice", "voice"]]:
        var i := AudioServer.get_bus_index(bus[0])
        if i >= 0:
            AudioServer.set_bus_volume_db(i, linear_to_db(maxf(float(settings.get(bus[1], 0.8)), 0.001)))


func type_speed() -> float:
    ## characters per second; 0 = instant
    return [28.0, 55.0, 0.0][clampi(int(settings.get("text_speed", 1)), 0, 2)]


# ---------------------------------------------------------------- save / profile
func new_save(username: String, mode: String, role: String = "student", avatar: String = "pelajar_1", email: String = "
") -> void:
    save = {
        "version": 1,
        "profile": {"username": username, "mode": mode, "role": role, "avatar": avatar, "email": email},
        "level": 1, "exp": 0, "coins": 0,
        "prolog_done": false,
        "zones": {},
        "chapters_done": [],
        "artifacts": [],
        "figures": [],
        "hint_log": {},
        "seen": [],
        "map_pos": [],
        "reflection": "",
    }
    _ensure_fields()


## Fields added after the first release; old saves get them on load.
func _ensure_fields() -> void:
    var defaults := {
        "player_id": "", "skins_owned": ["sekolah", "melayu"], "skin": "sekolah",
        "battle": {"played": 0, "wins": 0}, "ujian": [],
    }
    for k in defaults:
        if not save.has(k):
            save[k] = defaults[k].duplicate() if defaults[k] is Array or defaults[k] is Dictionary else defaults[k]
    if str(save["player_id"]) == "":
        save["player_id"] = _new_player_id()
    for k in ["display_name", "motto", "negeri"]:
        if not save["profile"].has(k):
            save["profile"][k] = save["profile"]["username"] if k == "display_name" else ""


## 6 characters, no look-alikes (0/O, 1/I) so it is easy to read out to a friend
func _new_player_id() -> String:
    const CH := "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"
    var s := ""
    for i in 6:
        s += CH[randi() % CH.length()]
    return s


func save_path_for(username: String) -> String:
    var safe := username.to_lower().validate_filename().replace(" ", "_")
```

```
    return "user://simpan_%s.json" % safe


func load_profile(username: String, mode: String, role: String = "student", avatar: String = "pelajar_1", email: String = "") -> void:
    var data = load_json_user(save_path_for(username))
    if data is Dictionary and data.has("profile"):
        save = data
        save["profile"]["mode"] = mode
        if email != "":
            save["profile"]["email"] = email
    else:
        new_save(username, mode, role, avatar, email)
    _ensure_fields()
    persist()


func persist() -> void:
    write_json_user(save_path_for(str(save["profile"]["username"])), save)
    Backend.push_progress(save)


func reset_progress() -> void:
    # Bab 1 starts again; the player keeps their ID, profile and bought skins
    var keep := {"profile": save["profile"], "player_id": save["player_id"], "skins_owned": save["skins_owned"], "skin": save["skin"], "battle": save["battle"]}
    var p: Dictionary = save["profile"]
    new_save(str(p["username"]), str(p["mode"]), str(p["role"]), str(p["avatar"]), str(p.get("email", "")))
    for k in keep:
        save[k] = keep[k]
    persist()


# ---------------------------------------------------------------- skins and shop
func skins() -> Array:
    if _skins.is_empty():
        _skins = load_json("res://data/skins.json").get("skins", [])
    return _skins


func skin_def(id: String) -> Dictionary:
    for s in skins():
        if str(s["id"]) == id:
            return s
    return skins()[0]


## Character folder for the equipped skin (falls back to plain Alif).
func skin_folder() -> String:
    var f := str(skin_def(str(save.get("skin", "sekolah")))["folder"])
    return f if ResourceLoader.exists("res://assets/characters32/%s/%s_south.png" % [f, f]) else "alif"


func spend(amount: int) -> bool:
    if int(save["coins"]) < amount:
        return false
    save["coins"] = int(save["coins"]) - amount
    stats_changed.emit()
    return true


## "" when bought, otherwise why not
func buy_skin(id: String) -> String:
    if save["skins_owned"].has(id):
        return "Sudah dimiliki"
    var price := int(skin_def(id)["price"])
    if not spend(price):
        return tr("Syiling tak cukup. Perlu %d lagi.") % (price - int(save["coins"]))
    save["skins_owned"].append(id)
    persist()
    return ""


func equip_skin(id: String) -> void:
    if save["skins_owned"].has(id):
        save["skin"] = id
        persist()
        skin_changed.emit()
    stats_changed.emit()


# ---------------------------------------------------------------- progression (Module 5)
func level_for(exp_total: int) -> int:
    return 1 + exp_total / EXP_PER_LEVEL

```

```
330
331 func add_reward(exp_gain: int, coin_gain: int) -> bool:
332     ## returns true when the player levels up
333     var before := int(save["level"])
334     save["exp"] = int(save["exp"]) + exp_gain
335     save["coins"] = int(save["coins"]) + coin_gain
336     save["level"] = level_for(int(save["exp"]))
337     stats_changed.emit()
338     return int(save["level"]) > before
339
340
341 func zone_state(id: String) -> Dictionary:
342     if not save["zones"].has(id):
343         save["zones"][id] = {"done": false, "steps": [], "first_try": 0, "retries": 0, "hints": 0, "total": 0}
344     return save["zones"][id]
345
346
347 func zone_done(id: String) -> bool:
348     return bool(zone_state(id).get("done", false))
349
350
351 func zone_unlocked(id: String) -> bool:
352     var ch := chapter_of(id)
353     var list := chapter_zones(ch)
354     var i := list.find(id)
355     if i < 0:
356         return false
357     if i == 0:
358         return ch <= 1 or chapter_done(ch - 1)
359     return zone_done(list[i - 1])
360
361
362 ## the mission before zid in the same chapter ("" for a chapter's first one)
363 func prev_zone(zid: String) -> String:
364     var list := chapter_zones(chapter_of(zid))
365     var i := list.find(zid)
366     return list[i - 1] if i > 0 else ""
367
368
369 func next_zone() -> String:
370     for z in ZONES:
371         if not zone_done(z):
372             return z
373     return ""
374
375
376 func chapter_done(no: int) -> bool:
377     return save["chapters_done"].has(no) or save["chapters_done"].has(float(no))
378
379
380 ## every mission of the chapter is finished (its Ujian Bab may still be ahead)
381 func chapter_zones_done(no: int) -> bool:
382     var list := chapter_zones(no)
383     if list.is_empty():
384         return false
385     for z in list:
386         if not zone_done(z):
387             return false
388     return true
389
390
391 ## the Ujian Bab is waiting: missions done, chapter not closed yet
392 func ujian_pending(no: int) -> bool:
393     return chapter_zones_done(no) and not chapter_done(no) and not ujian_def(no).is_empty()
394
395
396 var _ujian := {}
397
398
399 func ujian_def(no: int) -> Dictionary:
400     if _ujian.is_empty():
401         _ujian = load_json("res://data/ujian.json")
402     return _ujian.get(str(no), {})
403
404
405 ## n random questions from the chapter's bank, answers shuffled (only_zones:
406 ## just the questions of those missions, e.g. the ones already finished)
407 func ujian_questions(no: int, n: int, only_zones: Array = []) -> Array:
408     var bank: Array = BattleRules.bank_of(no).duplicate(true)
409     if not only_zones.is_empty():
410         bank = bank.filter(func(q): return only_zones.has(str(q.get("zon", ""))))
411     bank.shuffle()
412     var out: Array = []
413     for q in bank.slice(0, n):
414         var opts: Array = q["options"]
```

```
        var right: String = opts[int(q["answer"])]
        var mixed := opts.duplicate()
        mixed.shuffle()
        out.append({"q": q["q"], "options": mixed, "answer": mixed.find(right), "explain": q.get("explain", ""), "zon":
 q.get("zon", "")})
    return out


## Tingkatan 4 chapters from data/chapters.json (dictionaries with no, title, zones)
func chapter_list() -> Array:
    var arcs: Array = chapters.get("arcs", [])
    return arcs[0].get("chapters", []) if arcs.size() > 0 else []


func chapter_info(no: int) -> Dictionary:
    for ch in chapter_list():
        if int(ch["no"]) == no:
            return ch
    return {}


func chapter_zones(no: int) -> Array:
    return chapter_info(no).get("zones", [])


## "2_5" -> 2
func chapter_of(zid: String) -> int:
    return int(zid.get_slice("_", 0))


## true when zid is the last mission of its chapter
func is_last_zone(zid: String) -> bool:
    var list := chapter_zones(chapter_of(zid))
    return list.size() > 0 and list[-1] == zid


## highest chapter the player may play now (1 + finished chapters in a row)
func current_chapter() -> int:
    var n := 1
    while chapter_done(n) and not chapter_zones(n + 1).is_empty():
        n += 1
    return n


func add_figure(id: String) -> void:
    if characters.get(id, {}).get("tokoh", false) and not save["figures"].has(id):
        save["figures"].append(id)


func add_artifact(id: String) -> void:
    if not save["artifacts"].has(id):
        save["artifacts"].append(id)


func log_hint(puzzle_id: String, tier: int) -> void:
    ## anonymous count of Pau hint use, for teachers (no personal data)
    var hl: Dictionary = save["hint_log"]
    if not hl.has(puzzle_id):
        hl[puzzle_id] = {"1": 0, "2": 0, "3": 0}
    hl[puzzle_id][str(tier)] = int(hl[puzzle_id].get(str(tier), 0)) + 1
    Backend.log_hint(puzzle_id, tier)


func mastery_percent(zone_id: String) -> float:
    var z := zone_state(zone_id)
    var total := int(z.get("total", 0))
    if total == 0:
        return 0.0
    return 100.0 * float(z.get("first_try", 0)) / float(total)


# ---------------------------------------------------------------- navigation
func goto(screen: String, params: Dictionary = {}) -> void:
    input_lock = 0
    if SCENES.has(screen):
        GameState.go_to(SCENES[screen])
        return
    pending_screen = screen
    pending_params = params
    GameState.go_to("res://scenes/screen_host.tscn")


func lock_input() -> void:
    input_lock += 1

```

```
499
500 func unlock_input() -> void:
501     input_lock = max(0, input_lock - 1)
502
503
504 func is_locked() -> bool:
505     return input_lock > 0
506
507
508 ## on-screen controller for walking (Tetapan > Kawalan): "dpad", "joystick" or
509 ## "none". Automatik = D-pad on phones/tablets, nothing on a laptop.
510 func touch_mode() -> String:
511     var m := str(settings.get("controls", "auto"))
512     if m == "auto":
513         return "dpad" if DisplayServer.is_touchscreen_available() else "none"
514     return m if m in ["dpad", "joystick"] else "none"
515
516
517 func controls_on_screen() -> bool:
518     return touch_mode() != "none"
519
520
521 func toast(text: String, color: Color = UI.GOLD) -> void:
522     if main and main.has_method("toast"):
523         main.toast(text, color)
524
525
526 func say(key_or_lines: Variant) -> void:
527     ## Show a dialogue (array of lines or story key) and wait for it to finish.
528     if main and main.has_method("dialogue"):
529         await main.dialogue(key_or_lines)
530
531
532 # ---------------------------------------------------------------- input
533 const DEFAULT_KEYS := {
534     "move_left": [KEY_A, KEY_LEFT], "move_right": [KEY_D, KEY_RIGHT],
535     "move_up": [KEY_W, KEY_UP], "move_down": [KEY_S, KEY_DOWN],
536     "interact": [KEY_E, KEY_SPACE, KEY_ENTER, KEY_Z], "pau": [KEY_H], "menu": [KEY_ESCAPE], "map": [KEY_M],
537     "run": [KEY_SHIFT, KEY_X],
538     # Gelanggang (brawl)
539     "attack_punch": [KEY_J], "attack_kick": [KEY_K], "jump": [KEY_SPACE], "grab": [KEY_L],
540     "block": [KEY_I], "special": [KEY_U],
541 }
542 ## Gelanggang keyboard layouts (Jeda > Kekunci). "wasd": left hand walks,
543 ## right hand fights. "panah": right hand on the arrows, every fight key in
544 ## one block under the left hand (Z X C / A S D).
545 const BRAWL_KEYS := {
546     "wasd": {
547         "move_left": [KEY_A], "move_right": [KEY_D], "move_up": [KEY_W], "move_down": [KEY_S],
548         "attack_punch": [KEY_J], "attack_kick": [KEY_K], "jump": [KEY_SPACE], "grab": [KEY_L],
549         "block": [KEY_I], "special": [KEY_U], "run": [KEY_SHIFT],
550     },
551     "panah": {
552         "move_left": [KEY_LEFT], "move_right": [KEY_RIGHT], "move_up": [KEY_UP], "move_down": [KEY_DOWN],
553         "attack_punch": [KEY_Z], "attack_kick": [KEY_X], "jump": [KEY_C, KEY_SPACE], "grab": [KEY_A],
554         "block": [KEY_S], "special": [KEY_D], "run": [KEY_SHIFT],
555     },
556 }
557
558
559 func _setup_input() -> void:
560     for action in DEFAULT_KEYS.keys():
561         if not InputMap.has_action(action):
562             InputMap.add_action(action)
563         _bind(action, DEFAULT_KEYS[action])
564
565
566 func _bind(action: String, keys: Array) -> void:
567     InputMap.action_erase_events(action)
568     for k in keys:
569         var ev := InputEventKey.new()
570         ev.physical_keycode = k
571         InputMap.action_add_event(action, ev)
572
573
574 ## the Gelanggang layout picked in its pause menu ("wasd" or "panah")
575 func brawl_scheme() -> String:
576     var s := str(settings.get("brawl_keys", "wasd"))
577     return s if BRAWL_KEYS.has(s) else "wasd"
578
579
580 ## on = use the Gelanggang layout; off = back to the normal keys
581 func use_brawl_keys(on: bool) -> void:
582     var scheme: Dictionary = BRAWL_KEYS[brawl_scheme()]
583     for action in scheme:
```

```
        _bind(action, scheme[action] if on else DEFAULT_KEYS[action])


## readable key names for an action in the current Gelanggang layout
func brawl_key_names(action: String) -> String:
    var names: Array = []
    for k in BRAWL_KEYS[brawl_scheme()].get(action, []):
        names.append({KEY_SPACE: "Space", KEY_SHIFT: "Shift", KEY_LEFT: tr("Kiri"), KEY_RIGHT: tr("Kanan"), KEY_UP: tr("Atas"), KEY_DOWN: tr("Bawah")}.get(k, OS.get_keycode_string(k)))
    return " / ".join(names)
```

`tts.gd` · scripts/autoload/tts.gd · 43 lines

```
extends Node
## Read-aloud (autoload "Tts"). Uses the platform text-to-speech
## (Web Speech API in browsers). Prefers a Malay voice, then Indonesian.

var voice_id: String = ""
var available: bool = true


func _ready() -> void:
    if DisplayServer.get_name() == "headless" or not DisplayServer.has_feature(DisplayServer.FEATURE_TEXT_TO_SPEECH):
        available = false
        return
    _pick_voice()
    # desktop without a speech engine: switch read-aloud off quietly
    if voice_id == "" and OS.get_name() != "Web":
        available = false


func _pick_voice() -> void:
    for lang in ["ms", "id"]:
        var v: PackedStringArray = DisplayServer.tts_get_voices_for_language(lang)
        if v.size() > 0:
            voice_id = v[0]
            return
    var all: Array = DisplayServer.tts_get_voices()
    if all.size() > 0:
        voice_id = str(all[0].get("id", ""))


func speak(text: String) -> void:
    if not available or bool(Game.settings.get("muted", false)):
        return
    if voice_id == "":
        _pick_voice()
    if voice_id == "" and OS.get_name() != "Web":
        return
    DisplayServer.tts_stop()
    DisplayServer.tts_speak(text, voice_id, 90, 1.0, 0.95)


func stop() -> void:
    if available:
        DisplayServer.tts_stop()
```

scripts/battle/

`battle_rules.gd` · scripts/battle/battle_rules.gd · 120 lines

```
class_name BattleRules
extends RefCounted
## Rules of the Bilik Pertarungan, shared by the offline battle (bots) and the
## online one (supabase/multiplayer.sql implements the same numbers).
##
## Each round: every player still standing picks a target and answers the
## same question. A correct answer hits the target for DAMAGE (the fastest
## correct answer adds BONUS). A wrong answer does nothing, no punishment.
## At 0 HP a player is out. The last one standing wins REWARD syiling.

const HP := 100
const DAMAGE := 25
const BONUS := 5
const ROUND_TIME := 30.0     ## seconds to answer (generous: read-aloud is there)
const REVEAL_TIME := 4.0     ## seconds showing the answer and the hits
const REWARD_COINS := 20
const REWARD_EXP := 30       ## everyone who plays gets this much EXP
const MIN_FRIENDS := 2       ## room = you + 2 to 4 friends (or bots)
```

```
const MAX_FRIENDS := 4
const BANK := "res://data/soalan/bab%d.json"
## which chapter the next battle asks about (0 = every finished chapter mixed)
static var chapter := 1

static var _bank: Array = []
static var _bank_for := -1


## Questions for the chosen chapter. The online room always uses Bab 1, the
## answer key the server holds (supabase/multiplayer.sql).
static func bank() -> Array:
    if _bank_for != chapter:
        _bank = []
        var nos: Array = [chapter]
        if chapter == 0:
            nos = []
            for n in range(1, 11):
                if n == 1 or Game.chapter_done(n) or Game.chapter_done(n - 1):
                    nos.append(n)
        for n in nos:
            _bank.append_array(bank_of(n))
        _bank_for = chapter
    return _bank


static func chapter_name() -> String:
    return tr_safe("semua bab") if chapter == 0 else tr_safe("Bab %d") % chapter


static func tr_safe(s: String) -> String:
    return TranslationServer.translate(s)


static func bank_of(no: int) -> Array:
    var path := BANK % no
    if not FileAccess.file_exists(path):
        return []
    var d = JSON.parse_string(FileAccess.get_file_as_string(path))
    return d.get("questions", []) if d is Dictionary else []


## chapters that have a question bank and are open to this player
static func chapters_open() -> Array:
    var out: Array = []
    for n in range(1, 11):
        if FileAccess.file_exists(BANK % n) and Game.zone_unlocked(str(Game.chapter_zones(n)[0]) if not Game.chapter_zo
nes(n).is_empty() else "%d_1" % n):
            out.append(n)
    return out


static func alive(players: Array) -> Array:
    return players.filter(func(p): return p["alive"])


## Default target: the weakest other player still standing.
static func auto_target(players: Array, from_id: String) -> String:
    var best := ""
    var best_hp := 99999
    for p in players:
        if p["alive"] and p["id"] != from_id and int(p["hp"]) < best_hp:
            best = p["id"]
            best_hp = int(p["hp"])
    return best


static func find(players: Array, id: String) -> Dictionary:
    for p in players:
        if p["id"] == id:
            return p
    return {}


## Applies one round. players[i] needs: id, hp, alive, answer (-1 = none),
## answered_at (seconds, smaller = faster), target. Returns the hits.
static func resolve(players: Array, correct: int) -> Array:
    var hits: Array = []
    var fastest := ""
    var fastest_t := INF
    for p in players:
        p["correct"] = p["alive"] and int(p.get("answer", -1)) == correct
        if p["correct"] and float(p.get("answered_at", INF)) < fastest_t:
            fastest_t = float(p["answered_at"])
            fastest = p["id"]
```

```
    for p in players:
        if not p["correct"]:
            continue
        var t := find(players, str(p.get("target", "")))
        if t.is_empty() or not t["alive"] or t["id"] == p["id"]:
            t = find(players, auto_target(players, p["id"]))
        if t.is_empty():
            continue
        var dmg := DAMAGE + (BONUS if p["id"] == fastest else 0)
        hits.append({"from": p["id"], "to": t["id"], "dmg": dmg})
    # damage lands at the same moment for everyone
    for h in hits:
        var t := find(players, h["to"])
        t["hp"] = maxi(0, int(t["hp"]) - int(h["dmg"]))
    for p in players:
        if p["alive"] and int(p["hp"]) <= 0:
            p["alive"] = false
    return hits
```

`local_battle.gd` · scripts/battle/local_battle.gd · 181 lines

```
class_name LocalBattle
extends Node
## Offline Bilik Pertarungan: you against 2-4 classmate bots on this device.
## Works without any server. Same state shape as OnlineBattle, so the battle
## screen (scripts/screens/pertarungan.gd) does not care which one it has.
##
## state = {status: "question"|"reveal"|"over", round, q: {q, options},
##          answer (reveal only), explain, time_left, players: [...], hits: [...],
##          winner: id ("" = draw)}

signal state_changed(state: Dictionary)

const BOTS := [
    {"name": "Aisyah", "skin": "rakan_aisyah"},
    {"name": "Ravi", "skin": "rakan_ravi"},
    {"name": "Mei Ling", "skin": "rakan_meiling"},
    {"name": "Daniel", "skin": "rakan_daniel"},
]

var state: Dictionary = {}
var my_id := "me"
var _questions: Array = []
var _qi := -1
var _t := 0.0
var _bot_plan := {}   ## bot id -> {at: seconds, answer: int}
var _reveal_left := 0.0


func setup(bot_count: int) -> void:
    bot_count = clampi(bot_count, BattleRules.MIN_FRIENDS, BattleRules.MAX_FRIENDS)
    var prof: Dictionary = Game.save["profile"]
    var players: Array = [{
        "id": my_id, "name": str(prof.get("display_name", prof["username"])),
        "skin": Game.skin_folder(), "me": true, "bot": false, "acc": 1.0,
    }]
    var bots := BOTS.duplicate()
    bots.shuffle()
    for i in bot_count:
        var b: Dictionary = bots[i].duplicate()
        b["id"] = "bot%d" % i
        b["me"] = false
        b["bot"] = true
        b["acc"] = randf_range(0.45, 0.7)  # good classmates, not perfect
        players.append(b)
    for p in players:
        p["hp"] = BattleRules.HP
        p["alive"] = true
        p["target"] = ""
    for p in players:
        p["target"] = BattleRules.auto_target(players, p["id"])
    _questions = BattleRules.bank().duplicate()
    _questions.shuffle()
    state = {"status": "", "round": 0, "players": players, "hits": [], "winner": ""}


func start() -> void:
    _next_round()


func me() -> Dictionary:
    return BattleRules.find(state["players"], my_id)

```

```

## --- player actions (same API as OnlineBattle)
func set_target(id: String) -> void:
    var t := BattleRules.find(state["players"], id)
    if not t.is_empty() and t["alive"] and id != my_id:
        me()["target"] = id
        state_changed.emit(state)


func answer(i: int) -> void:
    var m := me()
    if state["status"] != "question" or not m["alive"] or int(m["answer"]) >= 0:
        return
    m["answer"] = i
    m["answered_at"] = _t
    m["answered"] = true
    state_changed.emit(state)
    _check_round_end()


func leave() -> void:
    queue_free()


# ---------------------------------------------------------------- rounds
func _next_round() -> void:
    _qi = (_qi + 1) % _questions.size()
    var q: Dictionary = _questions[_qi]
    _t = 0.0
    state["round"] = int(state["round"]) + 1
    state["status"] = "question"
    state["q"] = {"q": q["q"], "options": q["options"]}
    state.erase("answer")
    state["explain"] = ""
    state["hits"] = []
    state["time_left"] = BattleRules.ROUND_TIME
    _bot_plan.clear()
    for p in state["players"]:
        p["answer"] = -1
        p["answered"] = false
        p["answered_at"] = INF
        p["correct"] = false
        if p["alive"] and p["bot"]:
            var right := randf() < float(p["acc"])
            var wrong := (int(q["answer"]) + randi_range(1, 3)) % 4
            _bot_plan[p["id"]] = {"at": randf_range(4.0, 18.0), "answer": int(q["answer"]) if right else wrong}
            # bots spread their attacks (no ganging up on the player)
            p["target"] = _random_target(p["id"])
        var t := BattleRules.find(state["players"], str(p["target"]))
        if t.is_empty() or not t["alive"]:
            p["target"] = BattleRules.auto_target(state["players"], p["id"])
    state_changed.emit(state)


func _random_target(from_id: String) -> String:
    var opts: Array = BattleRules.alive(state["players"]).filter(func(p): return p["id"] != from_id)
    return opts.pick_random()["id"] if not opts.is_empty() else ""


func _process(delta: float) -> void:
    match str(state.get("status", "")):
        "question":
            _t += delta
            state["time_left"] = maxf(0.0, BattleRules.ROUND_TIME - _t)
            for id in _bot_plan:
                var b := BattleRules.find(state["players"], id)
                if not b["answered"] and _t >= float(_bot_plan[id]["at"]):
                    b["answer"] = _bot_plan[id]["answer"]
                    b["answered_at"] = _t
                    b["answered"] = true
                    state_changed.emit(state)
            if state["time_left"] <= 0.0:
                _reveal()
            else:
                _check_round_end()
        "reveal":
            _reveal_left -= delta
            if _reveal_left <= 0.0:
                _after_reveal()


func _check_round_end() -> void:
    if state["status"] != "question":
        return
    for p in BattleRules.alive(state["players"]):
```

```
        if not p["answered"]:
            return
    _reveal()


func _reveal() -> void:
    var q: Dictionary = _questions[_qi]
    state["answer"] = int(q["answer"])
    state["explain"] = str(q.get("explain", ""))
    state["hits"] = BattleRules.resolve(state["players"], int(q["answer"]))
    state["status"] = "reveal"
    _reveal_left = BattleRules.REVEAL_TIME
    state_changed.emit(state)


func _after_reveal() -> void:
    var standing := BattleRules.alive(state["players"])
    if standing.size() > 1:
        _next_round()
        return
    state["status"] = "over"
    state["winner"] = standing[0]["id"] if standing.size() == 1 else ""
    _reward()
    state_changed.emit(state)


func _reward() -> void:
    var b: Dictionary = Game.save["battle"]
    b["played"] = int(b.get("played", 0)) + 1
    var won: bool = state["winner"] == my_id
    if won:
        b["wins"] = int(b.get("wins", 0)) + 1
    state["level_up"] = Game.add_reward(BattleRules.REWARD_EXP, BattleRules.REWARD_COINS if won else 0)
    Game.persist()
```

online_battle.gd · scripts/battle/online_battle.gd · 96 lines

```
class_name OnlineBattle
extends Node
## Online Bilik Pertarungan with friends, through Supabase (see
## supabase/multiplayer.sql). The server holds the room and applies the
## damage (BattleRules numbers), so nobody can cheat by editing their game.
## The client polls battle_state once a second (works in the web build).
##
## NOTE: written and documented but not tested yet: it needs a Supabase
## project (config/backend.json). The offline LocalBattle is the demo path.

signal state_changed(state: Dictionary)
signal failed(message: String)

const POLL := 1.0

var state: Dictionary = {}
var my_id := ""
var code := ""
var _poll_t := 0.0
var _busy := false
var _rewarded := false


func join(room_code: String) -> void:
    code = room_code
    my_id = Backend.user_id
    await _poll()


func me() -> Dictionary:
    return BattleRules.find(state.get("players", []), my_id)


func set_target(id: String) -> void:
    if id == my_id:
        return
    me()["target"] = id  # show it at once; the server keeps the real one
    state_changed.emit(state)
    await Backend.db_call("battle_target", {"p_code": code, "p_target": id})


func answer(i: int) -> void:
    var m := me()
    if m.is_empty() or int(m.get("answer", -1)) >= 0:
        return
    m["answer"] = i
```

```
    state_changed.emit(state)
    await Backend.db_call("battle_answer", {"p_code": code, "p_round": int(state.get("round", 0)), "p_answer": i, "p_target": m.get("target") if str(m.get("target", "")) != "" else null})


func leave() -> void:
    Backend.db_call("battle_leave", {"p_code": code})
    queue_free()


func _process(delta: float) -> void:
    if state.has("time_left"):
        state["time_left"] = maxf(0.0, float(state["time_left"]) - delta)
    _poll_t += delta
    if _poll_t >= POLL and not _busy:
        _poll_t = 0.0
        _poll()


func _poll() -> void:
    _busy = true
    var r: Dictionary = await Backend.db_call("battle_state", {"p_code": code})
    _busy = false
    if not r["ok"] or not (r["data"] is Dictionary):
        failed.emit(tr("Sambungan ke bilik terputus."))
        return
    var s: Dictionary = r["data"]
    # the bank lives in the game; the server sends only the question number
    if s.has("qid"):
        var online_bank := BattleRules.bank_of(1)
        var q: Dictionary = online_bank[int(s["qid"]) % online_bank.size()]
        s["q"] = {"q": q["q"], "options": q["options"]}
        s["explain"] = str(q.get("explain", ""))
    for p in s.get("players", []):
        p["me"] = str(p["id"]) == my_id
        p["answered"] = bool(p.get("answered", false))
        if p["me"] and state.has("players") and int(s.get("round", 0)) == int(state.get("round", -1)):
            var old := me()
            if int(p.get("answer", -1)) < 0 and int(old.get("answer", -1)) >= 0:
                p["answer"] = old["answer"]  # our answer may not have arrived yet
    state = s
    if str(s.get("status", "")) == "over" and not _rewarded:
        _rewarded = true
        var b: Dictionary = Game.save["battle"]
        b["played"] = int(b.get("played", 0)) + 1
        var won: bool = str(s.get("winner", "")) == my_id
        if won:
            b["wins"] = int(b.get("wins", 0)) + 1
        Game.add_reward(BattleRules.REWARD_EXP, BattleRules.REWARD_COINS if won else 0)
        Game.persist()
    state_changed.emit(state)
```

scripts/brawl/

brawl_ai.gd · scripts/brawl/brawl_ai.gd · 110 lines

```
class_name BrawlAI
extends RefCounted
## A classmate bot in the Gelanggang. Picks an enemy, walks to stand beside
## it, throws short combos, sometimes blocks, grabs the school item that
## dropped for it. Fair play: at most 2 bots attack the same fighter at once
## (the others hover around), so 1 vs 3 stays winnable.

var lv: Dictionary
var target: BrawlFighter
var _retarget := 0.0
var _cd := 0.6
var _seq: Array = []
var _seq_t := 0.0
var _block_left := 0.0
var _reacted := {}     ## attacker -> the attack we already reacted to
var _hover := Vector2.ZERO


func _init(level: String) -> void:
    lv = BrawlRules.LEVELS.get(level, BrawlRules.LEVELS["biasa"])
    _cd = randf_range(0.3, 0.9)
    _hover = Vector2(randf_range(160, 230), randf_range(-60, 60))


func think(f: BrawlFighter, delta: float, m: BrawlMatch) -> void:
```

```
    f.clear_intent()
    if not f.alive() or f.state in ["down", "hurt", "getup", "pickup", "dizzy"]:
        _seq.clear()
        return
    _cd -= delta
    _retarget -= delta
    _seq_t -= delta
    if _block_left > 0.0:
        _block_left -= delta
        f.intent["block"] = true
        return
    if _react(f, m):
        return
    if not _seq.is_empty():
        if _seq_t <= 0.0:
            f.intent[_seq.pop_front()] = true
            _seq_t = 0.17
        return
    if _retarget <= 0.0 or target == null or not target.alive():
        target = _pick_target(f, m)
        _retarget = randf_range(1.0, 1.8)
    if target == null:
        return
    # a school item lying close by (ours, or nobody's): go and take it
    if f.weapon_id == "":
        var w = m.weapon_near(f, 240.0)
        if w:
            var d: Vector2 = w.position - f.position
            if d.length() < BrawlFighter.PICK_RANGE - 10:
                f.intent["grab"] = true
            else:
                f.intent["move"] = d.normalized() * lv["speed"]
            return
    var side := -1 if f.position.x < target.position.x else 1
    var crowded := m.attackers_on(target, f) >= 2 and absf(f.position.x - target.position.x) > 120
    var spot := target.position + Vector2(side * 76.0, 0)
    if crowded:
        spot = target.position + Vector2(side * _hover.x, _hover.y)   # wait for a turn
    var d := spot - f.position
    if absf(d.x) > 20 or absf(d.y) > 10:
        f.intent["move"] = d.normalized() * float(lv["speed"])
        f.intent["run"] = absf(d.x) > 320
        return
    f.facing = -side
    if crowded or _cd > 0.0:
        return
    _cd = randf_range(lv["cd"][0], lv["cd"][1])
    if f.special_ready and randf() < 0.7:
        f.intent["special"] = true
    elif f.weapon_id != "":
        _seq = ["punch"]
    elif randf() < float(lv["combo"]):
        _seq = [["punch", "punch", "kick"], ["punch", "punch", "punch"], ["jump", "kick"]].pick_random()
    else:
        _seq = [["punch"], ["kick"], ["punch", "punch"]].pick_random()
    _seq_t = 0.0


## see a punch coming? sometimes block it
func _react(f: BrawlFighter, m: BrawlMatch) -> bool:
    for e in m.enemies_of(f):
        if e.state != "attack" or _reacted.get(e, -1) == e.attack_no:
            continue
        _reacted[e] = e.attack_no
        var dx: float = e.position.x - f.position.x
        if absf(dx) < 140 and absf(e.position.y - f.position.y) < 30 and signf(-dx) == float(e.facing):
            if randf() < float(lv["block"]):
                f.facing = 1 if dx > 0 else -1
                _block_left = 0.45
                f.intent["block"] = true
                return true
    return false


func _pick_target(f: BrawlFighter, m: BrawlMatch) -> BrawlFighter:
    var best: BrawlFighter = null
    var best_score := INF
    for e in m.enemies_of(f):
        if not e.alive():
            continue
        var score: float = f.position.distance_to(e.position) + e.hp + m.attackers_on(e, f) * 260.0
        if score < best_score:
            best_score = score
            best = e
    return best
```

brawl_fighter.gd · scripts/brawl/brawl_fighter.gd · 498 lines

```
class_name BrawlFighter
extends Node2D
## One fighter in the Gelanggang: moves on a 2.5D floor (x, depth) and jumps
## (z), punches / kicks with combos, blocks, picks up and swings or throws
## school items, gets hurt, knocked down and finally PENGSAN (fainted, dizzy
## stars). Driven by `intent` (filled by the player's keys or BrawlAI), so
## people and bots use exactly the same rules. No blood: hits show stars and
## comic words, low HP shows a plaster and a bruise.

signal hit(target: BrawlFighter, dmg: int, info: Dictionary)

const WALK := Vector2(250, 150)
const RUN_MUL := 1.65
const JUMP_V := 560.0
const GRAVITY := 1700.0
const CHAIN := 0.42        ## seconds to press the next combo button
const DOWN_TIME := 0.9
const GETUP_TIME := 0.5
const BLOCK_MAX := 2.2     ## hold block longer than this and you get dizzy
const PICK_RANGE := 70.0

const TEAM_COLORS := [Color("f0c040"), Color("e05a4a")]  # your team gold, theirs red
const ALLY_COLOR := Color("6aa7e8")

var id := ""
var display_name := ""
var team := 0
var is_me := false
var skin := "alif"
var arena: Node            ## BrawlMatch: enemies, weapons, fx
var hp := BrawlRules.HP
var mult := BrawlRules.LEMAH
var special_ready := false
var frozen := true
var facing := 1
var z := 0.0
var vz := 0.0
var vel := Vector2.ZERO    ## knockback / lunge, fades out
var state := "idle"
var state_t := 0.0
var weapon_id := ""
var weapon_uses := 0
var intent := {}
var hitpause := 0.0
var invuln := 0.0
var answered_right := false
var attack_no := 0         ## counts attacks (bots react once per attack)

var _move := {}
var _move_name := ""
var _hit_ids := {}
var _last_attack := ""
var _last_attack_end := -10.0
var _queued := ""
var _clock := 0.0
var _stun := 0.0
var _block_t := 0.0
var _flash := 0.0
var _sprite: AnimatedSprite2D
var _held: Node2D
var _tag: Label
var _has := {}             ## anim -> has real frames
var _feet := 0.0


func setup(p: Dictionary) -> void:
    id = str(p["id"])
    display_name = str(p["name"])
    team = int(p["team"])
    is_me = bool(p.get("me", false))
    skin = str(p.get("skin", "alif"))
    clear_intent()


func _ready() -> void:
    _sprite = AnimatedSprite2D.new()
    _sprite.sprite_frames = CharacterFrames.build_fight(skin)
    _sprite.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    _sprite.scale = Vector2.ONE * BrawlRules.SCALE
    add_child(_sprite)
    for a in CharacterFrames.FIGHT_ANIMS:
```

```
        _has[a] = CharacterFrames.has_fight_anim(skin, a)
    # feet on the node origin
    var tex: Texture2D = load("%s%s/%s_east.png" % [CharacterFrames.BASE, skin, skin])
    _feet = CharacterFrames.visible_bottom_offset(tex).y * BrawlRules.SCALE
    _held = Node2D.new()
    _held.draw.connect(_draw_weapon)
    add_child(_held)
    # only you get a tag ("KAU"); team rings + the HUD name everyone else, so a
    # crowded fight doesn't turn into a pile of names
    _tag = Label.new()
    _tag.text = tr("KAU")
    _tag.visible = is_me
    _tag.add_theme_font_size_override("font_size", 20)
    _tag.add_theme_color_override("font_color", _team_color())
    _tag.add_theme_color_override("font_outline_color", Color(0.08, 0.05, 0.03))
    _tag.add_theme_constant_override("outline_size", 6)
    _tag.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    _tag.size = Vector2(180, 28)
    add_child(_tag)
    _play("stance")


func _team_color() -> Color:
    if is_me:
        return TEAM_COLORS[0]
    return ALLY_COLOR if team == 0 else TEAM_COLORS[1]


func clear_intent() -> void:
    intent = {"move": Vector2.ZERO, "run": false, "punch": false, "kick": false, "jump": false,
        "grab": false, "block": false, "special": false}


func alive() -> bool:
    return state != "pengsan"


func can_be_hit() -> bool:
    return alive() and invuln <= 0.0 and state != "down"


## in the match's own coordinates (the arena is drawn scaled)
func head_pos() -> Vector2:
    return position + Vector2(0, -118 - z)


# ---------------------------------------------------------------------- loop
func _physics_process(delta: float) -> void:
    _clock += delta
    if hitpause > 0.0:
        hitpause -= delta
        _sprite.speed_scale = 0.0
        return
    if _sprite.speed_scale == 0.0:
        _sprite.speed_scale = 1.0
    state_t += delta
    invuln = maxf(0.0, invuln - delta)
    _flash = maxf(0.0, _flash - delta)
    var move_v := Vector2.ZERO
    if not frozen:
        move_v = _think_state(delta)
    elif state in ["walk", "block", "attack"] and z == 0.0:
        _set_state("idle")
    # physics: knockback + walking, jump arc, stay on the floor
    position += (move_v + vel) * delta
    vel = vel.move_toward(Vector2.ZERO, 1400.0 * delta)
    var f := BrawlRules.FLOOR
    position.x = clampf(position.x, f.position.x, f.end.x)
    position.y = clampf(position.y, f.position.y, f.end.y)
    if z > 0.0 or vz != 0.0:
        vz -= GRAVITY * delta
        z += vz * delta
        if z <= 0.0:
            z = 0.0
            vz = 0.0
            _landed()
    _update_visual()


func _think_state(delta: float) -> Vector2:
    match state:
        "idle", "walk":
            return _free(delta)
        "attack":
            _attack_tick()
```

```
        "hurt":
            if state_t >= _stun:
                _set_state("idle")
        "down":
            if state_t >= DOWN_TIME and z == 0.0:
                _set_state("getup")
                invuln = GETUP_TIME + 0.4
        "getup":
            if state_t >= GETUP_TIME:
                _set_state("idle")
        "block":
            _block_t += delta
            if not intent["block"]:
                _set_state("idle")
            elif _block_t > BLOCK_MAX:
                _stun = 1.0
                _set_state("hurt")
                arena.fx_word(head_pos(), "PENING", Color("c9a0ff"))
        "pickup":
            if state_t >= 0.32:
                _set_state("idle")
        "dizzy":
            if state_t >= 1.0:
                _set_state("idle")
    return Vector2.ZERO


## standing / walking: read the intent
func _free(_delta: float) -> Vector2:
    var m: Vector2 = intent["move"]
    if intent["block"] and z == 0.0:
        _block_t = 0.0
        _set_state("block")
        return Vector2.ZERO
    if intent["special"] and special_ready:
        special_ready = false
        _start("special")
        return Vector2.ZERO
    if intent["grab"]:
        if weapon_id != "":
            _start("throw")
        else:
            var w = arena.weapon_near(self)
            if w:
                arena.take_weapon(w, self)
                _set_state("pickup")
                Sfx.play("pop", -6.0, 1.1)
        return Vector2.ZERO
    if intent["jump"] and z == 0.0:
        vz = JUMP_V
        Sfx.play("swish", -8.0, 0.8)
    if intent["punch"] or _queued == "punch":
        _queued = ""
        _start(_pick_punch(m))
        return Vector2.ZERO
    if intent["kick"] or _queued == "kick":
        _queued = ""
        _start(_pick_kick())
        return Vector2.ZERO
    if m.length() > 0.1:
        if absf(m.x) > 0.1:
            facing = 1 if m.x > 0 else -1
        var run: bool = intent["run"]
        if state != "walk" and z == 0.0:
            _set_state("walk")
        return m.limit_length(1.0) * WALK * (RUN_MUL if run else 1.0)
    if state != "idle" and z == 0.0:
        _set_state("idle")
    return Vector2.ZERO


func _pick_punch(m: Vector2) -> String:
    if weapon_id != "":
        return "swing"
    var chained := _clock - _last_attack_end < CHAIN
    if intent["run"] and m.length() > 0.1 and z == 0.0:
        return "dash"
    if chained and _last_attack == "jab":
        return "jab2"
    if chained and _last_attack == "jab2":
        return "straight"
    return "jab"


func _pick_kick() -> String:
```

```
    if z > 0.0:
        return "flykick"
    if _clock - _last_attack_end < CHAIN and _last_attack == "jab2":
        return "roundhouse"   # P, P, K
    return "kick"


func _start(name: String) -> void:
    _move = BrawlRules.move(name)
    _move_name = name
    attack_no += 1
    _hit_ids.clear()
    _set_state("attack")
    var anim: String = _move.get("anim", "jab")
    _play(anim)
    var n := _sprite.sprite_frames.get_frame_count(anim)
    _sprite.speed_scale = maxf(0.2, float(n) / (float(_move["time"]) * 10.0))
    if _move.has("lunge"):
        vel.x += float(_move["lunge"]) * facing
    Sfx.play("swish", -10.0, randf_range(0.9, 1.2))


func _attack_tick() -> void:
    var t: float = state_t / float(_move["time"])
    if _move_name == "throw":
        if t >= float(_move["release"]) and weapon_id != "":
            arena.throw_weapon(self)
            weapon_id = ""
    elif _move.has("active"):
        var a: Array = _move["active"]
        if t >= float(a[0]) and t <= float(a[1]):
            _hit_test()
        # a button pressed late in the move is remembered for the combo
        if t > float(a[1]):
            if intent["punch"]:
                _queued = "punch"
            elif intent["kick"]:
                _queued = "kick"
    if state_t >= float(_move["time"]):
        _last_attack = _move_name
        _last_attack_end = _clock
        _set_state("idle")


func _hit_test() -> void:
    var w := BrawlRules.weapon(weapon_id) if _move_name == "swing" else {}
    var reach: Array = _move.get("reach", [0, 80])
    var far: float = float(reach[1]) + (float(w.get("reach", 0)) if not w.is_empty() else 0.0)
    var sweep: bool = w.get("sweep", false)
    var landed := false
    for e in arena.enemies_of(self):
        if _hit_ids.has(e) or not e.can_be_hit():
            continue
        if sweep and _hit_ids.size() >= 2:
            break
        var dx: float = (e.position.x - position.x) * facing
        var dy: float = absf(e.position.y - position.y)
        var ok := false
        if _move.has("radius"):
            ok = Vector2(dx, dy * 2.0).length() <= float(_move["radius"])
        else:
            ok = dx >= float(reach[0]) and dx <= far and dy <= BrawlRules.DEPTH_BAND
        if ok and absf(e.z - z) > 70.0 and not _move.get("air", false):
            ok = false   # jumped over it
        if not ok:
            continue
        _hit_ids[e] = true
        var info := _move.duplicate()
        if not w.is_empty():
            info["dmg"] = w["dmg"]
            info["kb"] = w.get("kb", _move["kb"])
            info["word"] = "PAP!"
            info["stun"] = w.get("stun", _move.get("stun", 0.35))
            info["down"] = w.get("down", false)
        e.take_hit(self, info)
        landed = true
        if not w.is_empty() and (not sweep or _hit_ids.size() == 1):
            weapon_uses -= 1
    if landed and weapon_id != "" and weapon_uses <= 0:
        arena.fx_poof(position + Vector2(facing * 50, -70), BrawlRules.weapon(weapon_id).get("color", "ffffff"))
        weapon_id = ""


## damage = move damage x the attacker's Semangat (1.0) or Lemah (0.3)
func take_hit(by: BrawlFighter, info: Dictionary) -> void:
```

```
    var from := signi(int(by.position.x - position.x))
    if from == 0:
        from = -by.facing
    var blocked := state == "block" and from == facing
    var dmg := float(info.get("dmg", 4)) * by.mult
    if blocked:
        dmg *= BrawlRules.BLOCK_TAKE
    var d := maxi(1, roundi(dmg))
    hp = maxi(0, hp - d)
    facing = from
    var kb := float(info.get("kb", 100)) * (0.3 if blocked else 1.0)
    vel.x = -from * kb
    hitpause = 0.07
    by.hitpause = 0.05
    _flash = 0.14
    if hp <= 0:
        _set_state("pengsan")
        vz = 280.0
        frozen = true
    elif blocked:
        pass
    elif info.get("down", false):
        _set_state("down")
        vz = 300.0
    else:
        _stun = float(info.get("stun", 0.3))
        _set_state("hurt")
    by.hit.emit(self, d, {"blocked": blocked, "word": info.get("word", "PAK"), "big": info.get("big", false),
        "sfx": info.get("sfx", "punch"), "ko": hp <= 0})


func _landed() -> void:
    if state in ["down", "pengsan"]:
        Sfx.play("fall", -4.0)
        arena.fx_dust(position)
    elif state != "attack":
        arena.fx_dust(position, 0.5)


func _set_state(s: String) -> void:
    if state == "pengsan":
        return
    state = s
    state_t = 0.0
    match s:
        "idle", "block", "dizzy":
            _play("stance")
        "walk":
            _play("walk")
        "hurt":
            _play("hurt")
        "down", "pengsan":
            _play("fall")
        "getup":
            _play("getup")
        "pickup":
            _play("pickup")


func _play(anim: String) -> void:
    if _sprite.animation != anim or not _sprite.is_playing():
        _sprite.play(anim)
    _sprite.speed_scale = 1.0


# ---------------------------------------------------------------- look
func _update_visual() -> void:
    _sprite.flip_h = facing < 0
    _sprite.position = Vector2(0, _feet - z)
    _sprite.rotation = 0.0
    _sprite.modulate = Color.WHITE
    # cartoon fallbacks while an animation has no PixelLab frames yet
    match state:
        "attack":
            var anim: String = _move.get("anim", "")
            if not _has.get(anim, false):
                var t: float = clampf(state_t / float(_move.get("time", 0.3)), 0.0, 1.0)
                _sprite.position.x = facing * sin(t * PI) * 18.0
                _sprite.rotation = facing * sin(t * PI) * (0.25 if anim in ["kick", "roundhouse", "flykick"] else 0.08)
                if anim == "special":
                    _sprite.rotation = state_t * 18.0 * facing
        "hurt":
            if not _has.get("hurt", false):
                _sprite.position.x = -facing * 6.0
        "down", "pengsan":
```

```
            if not _has.get("fall", false):
                _sprite.rotation = -facing * PI / 2.0
                _sprite.position.y = -20.0 - z
        "getup":
            if not _has.get("getup", false):
                _sprite.rotation = -facing * PI / 2.0 * (1.0 - state_t / GETUP_TIME)
        "block":
            _sprite.modulate = Color(0.75, 0.85, 1.0)
        "dizzy":
            _sprite.rotation = sin(state_t * 12.0) * 0.08
    if _flash > 0.0:
        _sprite.modulate = Color(1.6, 1.6, 1.6)
    if state == "pengsan":
        _sprite.modulate = Color(0.7, 0.7, 0.75)
    if invuln > 0.0 and state != "pengsan":
        _sprite.modulate.a = 0.55 if int(_clock * 20.0) % 2 == 0 else 1.0
    _tag.position = Vector2(-90, -170 - z)
    _tag.modulate.a = 0.45 if state == "pengsan" else 1.0
    _held.queue_redraw()
    queue_redraw()


func _draw() -> void:
    # shadow + team ring on the floor
    var s := 1.0 - minf(z / 300.0, 0.5)
    draw_set_transform(Vector2.ZERO, 0, Vector2(1, 0.3))
    draw_circle(Vector2.ZERO, 46 * s, Color(0, 0, 0, 0.35))
    if alive():
        draw_arc(Vector2.ZERO, 54 * s, 0, TAU, 40, Color(_team_color(), 0.85), 5.0, true)
    draw_set_transform(Vector2.ZERO)
    var head := Vector2(4 * facing, -106 - z)
    if state == "pengsan":
        head = Vector2(-facing * 70, -26 - z)
    # low HP: plaster, then a bruise and sweat (cartoon, never blood)
    if hp < BrawlRules.HP * 0.4:
        var p := head + Vector2(-14 * facing, -6)
        draw_set_transform(p, 0.5 * facing, Vector2.ONE)
        draw_rect(Rect2(-13, -5, 26, 10), Color("f3d2a2"))
        draw_rect(Rect2(-4, -5, 8, 10), Color("e2b98a"))
        draw_rect(Rect2(-13, -5, 26, 10), Color("b98a5a"), false, 1.5)
        draw_set_transform(Vector2.ZERO)
    if hp < BrawlRules.HP * 0.2:
        draw_circle(head + Vector2(12 * facing, 8), 7, Color(0.55, 0.35, 0.75, 0.55))
        if alive():
            var sy := fmod(_clock * 30.0, 18.0)
            draw_circle(head + Vector2(26 * facing, -8 + sy), 3.5, Color(0.6, 0.85, 1.0, 0.8))
    # dizzy stars over fainted / dizzy heads
    if state in ["pengsan", "dizzy"] or (state == "hurt" and _stun >= 0.9):
        for i in 3:
            var a := _clock * 4.0 + i * TAU / 3.0
            var c := head + Vector2(cos(a) * 26, -24 + sin(a) * 8)
            _star(c, 7.0, Color("ffe066"))
    if state == "pengsan":
        draw_string(ThemeDB.fallback_font, head + Vector2(20, -46), "Zzz", HORIZONTAL_ALIGNMENT_LEFT, -1, 20, Color(1,
1, 1, 0.8))


func _star(c: Vector2, r: float, col: Color) -> void:
    var pts := PackedVector2Array()
    for i in 10:
        var rr := r if i % 2 == 0 else r * 0.45
        pts.append(c + Vector2.from_angle(-PI / 2 + i * PI / 5) * rr)
    draw_colored_polygon(pts, col)


## the school item in Alif's hand (drawn, so any skin can hold any item)
func _draw_weapon() -> void:
    if weapon_id == "" or not alive():
        return
    var w := BrawlRules.weapon(weapon_id)
    var col := Color(str(w.get("color", "ffffff")))
    var hand := Vector2(22 * facing, -66 - z)
    var ang := -0.9
    if state == "attack" and _move_name == "swing":
        ang = lerpf(-2.2, 0.9, clampf(state_t / float(_move["time"]), 0.0, 1.0))
    _held.draw_set_transform(hand, ang * facing + (0.0 if facing > 0 else PI), Vector2.ONE)
    WeaponArt.draw_item(_held, weapon_id, col)
    _held.draw_set_transform(Vector2.ZERO)
```

brawl_fx.gd · scripts/brawl/brawl_fx.gd · 82 lines

```
class_name BrawlFx
extends Node2D
## Cartoon hit effects for the Gelanggang, all drawn in code: impact stars,
## comic words (PAK! DUSH! POW!), dust puffs, paper bits when an item breaks,
## damage numbers. Never blood.

const WORD_FONT := "res://assets/fonts/pixelify-sans-latin-700-normal.woff2"

var _items: Array = []   ## {kind, pos, t, life, ...}
var _font: Font


func _ready() -> void:
    z_index = 100
    _font = load(WORD_FONT) if ResourceLoader.exists(WORD_FONT) else ThemeDB.fallback_font


func star(pos: Vector2, big := false) -> void:
    _items.append({"kind": "star", "pos": pos, "t": 0.0, "life": 0.22 if not big else 0.32, "r": 34.0 if not big else 56.0, "rot": randf() * TAU})


func word(pos: Vector2, text: String, col := Color("ffe066"), big := false) -> void:
    _items.append({"kind": "word", "pos": pos + Vector2(randf_range(-14, 14), 0), "t": 0.0, "life": 0.7,
        "text": text, "col": col, "size": 30 if big else 20, "rot": randf_range(-0.25, 0.25)})


func number(pos: Vector2, n: int, col: Color) -> void:
    _items.append({"kind": "word", "pos": pos + Vector2(randf_range(-10, 10), -20), "t": 0.0, "life": 0.9,
        "text": "-%d" % n, "col": col, "size": 18, "rot": 0.0, "rise": 70.0})


func dust(pos: Vector2, amount := 1.0) -> void:
    for i in int(6 * amount) + 2:
        _items.append({"kind": "dust", "pos": pos + Vector2(randf_range(-30, 30), randf_range(-4, 4)), "t": 0.0,
            "life": randf_range(0.35, 0.6), "vel": Vector2(randf_range(-70, 70), randf_range(-50, -10)), "r": randf_range(6, 12)})


func poof(pos: Vector2, col: Color) -> void:
    for i in 12:
        _items.append({"kind": "bit", "pos": pos, "t": 0.0, "life": randf_range(0.5, 0.9),
            "vel": Vector2.from_angle(randf() * TAU) * randf_range(80, 220), "col": col.lerp(Color.WHITE, randf() * 0.5), "rot": randf() * TAU})
    word(pos + Vector2(0, -20), "PUF!", Color(0.95, 0.95, 1.0))


func _process(delta: float) -> void:
    for it in _items:
        it["t"] += delta
        if it.has("vel"):
            it["pos"] += it["vel"] * delta
            it["vel"].y += 380.0 * delta
    _items = _items.filter(func(it): return it["t"] < it["life"])
    queue_redraw()


func _draw() -> void:
    for it in _items:
        var k: float = it["t"] / it["life"]
        match it["kind"]:
            "star":
                var r: float = it["r"] * (0.5 + k * 0.7)
                var pts := PackedVector2Array()
                for i in 16:
                    var rr := r if i % 2 == 0 else r * 0.42
                    pts.append(it["pos"] + Vector2.from_angle(it["rot"] + i * PI / 8) * rr)
                draw_colored_polygon(pts, Color(1, 1, 0.85, 1.0 - k))
                draw_circle(it["pos"], r * 0.3, Color(1, 1, 1, 1.0 - k))
            "word":
                var a := 1.0 - maxf(0.0, (k - 0.6) / 0.4)
                var p: Vector2 = it["pos"] + Vector2(0, -it.get("rise", 40.0) * k)
                var s: float = 1.0 + (0.4 * (1.0 - minf(k * 6.0, 1.0)))
                draw_set_transform(p, it["rot"], Vector2(s, s))
                var size: int = it["size"]
                var w := _font.get_string_size(it["text"], HORIZONTAL_ALIGNMENT_LEFT, -1, size).x
                draw_string_outline(_font, Vector2(-w / 2.0, 0), it["text"], HORIZONTAL_ALIGNMENT_LEFT, -1, size, 8, Color(0.1, 0.05, 0.05, a))
                draw_string(_font, Vector2(-w / 2.0, 0), it["text"], HORIZONTAL_ALIGNMENT_LEFT, -1, size, Color(it["col"], a))
                draw_set_transform(Vector2.ZERO)
            "dust":
                draw_circle(it["pos"], it["r"] * (1.0 + k), Color(0.85, 0.8, 0.7, 0.55 * (1.0 - k)))
```

```
            "bit":
                draw_set_transform(it["pos"], it["rot"] + k * 8.0, Vector2.ONE)
                draw_rect(Rect2(-5, -3, 10, 6), Color(it["col"], 1.0 - k))
                draw_set_transform(Vector2.ZERO)
```

`brawl_match.gd` · scripts/brawl/brawl_match.gd · 446 lines

```
class_name BrawlMatch
extends Node2D
## Gelanggang Pesta Silat, the match itself (offline, you + classmate bots).
## Phases: "soalan" (everyone answers, arena frozen) -> "keputusan" (who got
## Semangat, school items drop, fastest gets Jurus Kilat) -> "aksi" (live
## brawl for FIGHT_TIME) -> next round ... -> "tamat". The screen
## (scripts/screens/gelanggang.gd) draws the question card and HUD from the
## signals; this node owns the fighters, items and effects.

signal phase_changed(phase: String)
signal changed
signal hit_landed(by: BrawlFighter, target: BrawlFighter, dmg: int)

var format := "1v1"
var level := "biasa"
var fighters: Array[BrawlFighter] = []
var me: BrawlFighter
var me_autopilot := false    ## autotest: a bot plays for you
var phase := ""
var phase_t := 0.0
var round_no := 0
var q: Dictionary = {}       ## {q, options} (no answer until keputusan)
var answer_index := -1       ## right answer, shown in keputusan
var explain := ""
var answers := {}            ## fighter id -> {i, at}
var results: Array = []      ## [{id, name, right, fastest}] for the banner
var winner_team := -1
var level_up := false

var stage: Node2D            ## y-sorted: fighters + items
var fx: BrawlFx
var _ais := {}               ## fighter id -> BrawlAI
var _bot_plan := {}          ## fighter id -> {at, i}
var _questions: Array = []
var _qi := -1
var _items: Array = []       ## FloorItem


func setup(fmt: String, lvl: String) -> void:
    format = fmt if BrawlRules.FORMATS.has(fmt) else "1v1"
    level = lvl if BrawlRules.LEVELS.has(lvl) else "biasa"
    scale = Vector2.ONE * BrawlRules.VIEW_SCALE
    position = BrawlRules.SCREEN_FLOOR.position
    stage = Node2D.new()
    stage.y_sort_enabled = true
    add_child(stage)
    fx = BrawlFx.new()
    add_child(fx)
    var sizes: Array = BrawlRules.FORMATS[format]
    var prof: Dictionary = Game.save["profile"]
    var bots := LocalBattle.BOTS.duplicate()   # same order as the Lobby shows
    var people: Array = [{"id": "me", "name": str(prof.get("display_name", prof["username"])), "skin": Game.skin_folder
(), "team": 0, "me": true}]
    var bi := 0
    for t in 2:
        for k in int(sizes[t]) - (1 if t == 0 else 0):
            var b: Dictionary = bots[bi % bots.size()].duplicate()
            b["id"] = "bot%d" % bi
            b["team"] = t
            people.append(b)
            bi += 1
    for p in people:
        var f := BrawlFighter.new()
        f.setup(p)
        f.arena = self
        var mates := people.filter(func(o): return o["team"] == p["team"])
        var k := mates.find(p)
        var fl := BrawlRules.FLOOR
        var x := fl.position.x + fl.size.x * 0.16 - k * 50.0 if p["team"] == 0 else fl.end.x - fl.size.x * 0.16 + k * 5
0.0
        var y := BrawlRules.FLOOR.position.y + BrawlRules.FLOOR.size.y * (0.5 + (k - (mates.size() - 1) / 2.0) * 0.32)
        f.position = Vector2(x, y)
        f.facing = 1 if p["team"] == 0 else -1
        f.hit.connect(_on_hit.bind(f))
        stage.add_child(f)
        fighters.append(f)
```

```
        if f.is_me:
            me = f
        else:
            _ais[f.id] = BrawlAI.new(level)
    _questions = BattleRules.bank().duplicate()
    _questions.shuffle()


func start() -> void:
    _next_round()


# ---------------------------------------------------------------- queries (fighters use these)
func enemies_of(f: BrawlFighter) -> Array:
    return fighters.filter(func(o): return o.team != f.team and o.alive())


func team_alive(t: int) -> Array:
    return fighters.filter(func(o): return o.team == t and o.alive())


func attackers_on(target: BrawlFighter, except: BrawlFighter) -> int:
    var n := 0
    for id in _ais:
        var ai: BrawlAI = _ais[id]
        var f := find(id)
        if f and f != except and f.alive() and ai.target == target and f.position.distance_to(target.position) < 170:
            n += 1
    return n


func find(id: String) -> BrawlFighter:
    for f in fighters:
        if f.id == id:
            return f
    return null


## closest floor item a fighter may take (dropped for them, or for nobody)
func weapon_near(f: BrawlFighter, within := BrawlFighter.PICK_RANGE) -> Node2D:
    var best: Node2D = null
    var bd := within
    for it in _items:
        if it.flying or it.z > 4.0 or (it.for_id != "" and it.for_id != f.id and it.age < 4.0):
            continue
        var d := Vector2(it.position.x - f.position.x, (it.position.y - f.position.y) * 1.6).length()
        if d < bd:
            bd = d
            best = it
    return best


func take_weapon(it: Node2D, f: BrawlFighter) -> void:
    f.weapon_id = it.id
    f.weapon_uses = int(BrawlRules.weapon(it.id)["uses"])
    _items.erase(it)
    it.queue_free()


func throw_weapon(f: BrawlFighter) -> void:
    var it := FloorItem.new()
    it.id = f.weapon_id
    it.position = f.position
    it.z = 64.0
    it.flying = true
    it.vx = 760.0 * f.facing
    it.thrower = f
    stage.add_child(it)
    _items.append(it)
    Sfx.play("swish", -4.0, 0.8)


func fx_word(pos: Vector2, text: String, col := Color("ffe066")) -> void:
    fx.word(pos, text, col)


func fx_dust(pos: Vector2, amount := 1.0) -> void:
    fx.dust(pos, amount)


func fx_poof(pos: Vector2, col_hex: String) -> void:
    fx.poof(pos, Color(col_hex))
    Sfx.play("pop", -2.0, 0.8)


```

```
# ---------------------------------------------------------------- phases
func _set_phase(p: String) -> void:
    phase = p
    phase_t = 0.0
    phase_changed.emit(p)
    changed.emit()


func _next_round() -> void:
    round_no += 1
    if round_no > BrawlRules.MAX_ROUNDS:
        _finish(_team_by_hp())
        return
    for f in fighters:
        f.frozen = true
        f.clear_intent()
    _qi = (_qi + 1) % _questions.size()
    var item: Dictionary = _questions[_qi]
    q = {"q": item["q"], "options": item["options"]}
    answer_index = int(item["answer"])
    explain = str(item.get("explain", ""))
    answers.clear()
    results.clear()
    _bot_plan.clear()
    for f in fighters:
        if f.alive() and not f.is_me:
            var acc := randf_range(BrawlRules.LEVELS[level]["acc"][0], BrawlRules.LEVELS[level]["acc"][1])
            var right := randf() < acc
            _bot_plan[f.id] = {"at": randf_range(3.0, 11.0), "i": answer_index if right else (answer_index + randi_rang
e(1, 3)) % 4}
    if me_autopilot and me.alive():
        _bot_plan[me.id] = {"at": 1.0, "i": answer_index}
    _set_phase("soalan")


## the player's answer (A-D = 0-3)
func answer(i: int) -> void:
    if phase != "soalan" or not me.alive() or answers.has(me.id):
        return
    answers[me.id] = {"i": i, "at": phase_t}
    changed.emit()


func _keputusan() -> void:
    var fastest := ""
    var best := INF
    for f in fighters:
        if not f.alive():
            continue
        var a: Dictionary = answers.get(f.id, {})
        var right: bool = not a.is_empty() and int(a["i"]) == answer_index
        f.answered_right = right
        f.mult = BrawlRules.SEMANGAT if right else BrawlRules.LEMAH
        f.special_ready = false
        if right and float(a["at"]) < best:
            best = float(a["at"])
            fastest = f.id
        results.append({"id": f.id, "name": f.display_name, "right": right, "fastest": false, "team": f.team, "me": f.i
s_me})
    for r in results:
        if r["id"] == fastest:
            r["fastest"] = true
            find(fastest).special_ready = true
    # a school item drops near everyone who got it right (a few on the floor at most)
    for f in fighters:
        if f.alive() and f.answered_right and f.weapon_id == "" and _items.size() < BrawlRules.MAX_FLOOR_WEAPONS:
            _drop_item(f)
    _set_phase("keputusan")


func _drop_item(f: BrawlFighter) -> void:
    var it := FloorItem.new()
    it.id = BrawlRules.weapon_ids().pick_random()
    it.for_id = f.id
    var x := clampf(f.position.x + f.facing * 60.0, BrawlRules.FLOOR.position.x + 20, BrawlRules.FLOOR.end.x - 20)
    it.position = Vector2(x, f.position.y + 6)
    it.z = 520.0   # falls from the sky in a little crate
    it.crate = true
    stage.add_child(it)
    _items.append(it)


func _team_by_hp() -> int:
    var hp := [0, 0]
    for f in fighters:
```

```
        hp[f.team] += f.hp
    return 0 if hp[0] >= hp[1] else 1


func _finish(team: int) -> void:
    winner_team = team
    for f in fighters:
        f.frozen = true
        f.clear_intent()
    _reward()
    _set_phase("tamat")


func _reward() -> void:
    var b: Dictionary = Game.save["battle"]
    b["played"] = int(b.get("played", 0)) + 1
    var won := winner_team == 0
    if won:
        b["wins"] = int(b.get("wins", 0)) + 1
    level_up = Game.add_reward(BrawlRules.REWARD_EXP, BrawlRules.REWARD_COINS if won else 0)
    Game.persist()


func _process(delta: float) -> void:
    if phase == "" or phase == "tamat":
        return
    phase_t += delta
    match phase:
        "soalan":
            for id in _bot_plan:
                if not answers.has(id) and phase_t >= float(_bot_plan[id]["at"]):
                    answers[id] = {"i": int(_bot_plan[id]["i"]), "at": phase_t}
                    changed.emit()
            var all := true
            for f in fighters:
                if f.alive() and not answers.has(f.id):
                    all = false
            if all or phase_t >= BrawlRules.ASK_TIME:
                _keputusan()
        "keputusan":
            if phase_t >= BrawlRules.BANNER_TIME:
                for f in fighters:
                    f.frozen = not f.alive()
                Sfx.play("bell", -4.0)
                _set_phase("aksi")
        "aksi":
            if team_alive(1).is_empty():
                _finish(0)
            elif team_alive(0).is_empty():
                _finish(1)
            elif phase_t >= BrawlRules.FIGHT_TIME:
                Sfx.play("bell", -6.0, 0.8)
                _next_round()


func _physics_process(delta: float) -> void:
    # intents first (this node runs before its children: the fighters)
    if phase != "aksi":
        return
    for f in fighters:
        if f.is_me and not me_autopilot:
            _read_player(f)
        elif _ais.has(f.id):
            _ais[f.id].think(f, delta, self)
        elif f.is_me:
            if not _ais.has("me"):
                _ais["me"] = BrawlAI.new("sukar")
            _ais["me"].think(f, delta, self)
    for it in _items.duplicate():
        _update_item(it, delta)


func _read_player(f: BrawlFighter) -> void:
    f.intent = {
        "move": Input.get_vector("move_left", "move_right", "move_up", "move_down"),
        "run": Input.is_action_pressed("run"),
        "punch": Input.is_action_just_pressed("attack_punch"),
        "kick": Input.is_action_just_pressed("attack_kick"),
        "jump": Input.is_action_just_pressed("jump"),
        "grab": Input.is_action_just_pressed("grab"),
        "block": Input.is_action_pressed("block"),
        "special": Input.is_action_just_pressed("special"),
    }


```

```
func _update_item(it: FloorItem, delta: float) -> void:
    it.age += delta
    if it.flying:
        it.position.x += it.vx * delta
        for e in fighters:
            if it.thrower and e.team != it.thrower.team and e.can_be_hit() \
                    and absf(e.position.x - it.position.x) < 44 and absf(e.position.y - it.position.y) < BrawlRules.DEPTH_BAND:
                var w := BrawlRules.weapon(it.id)
                e.take_hit(it.thrower, {"dmg": w.get("throw_dmg", 8), "kb": 300, "stun": 0.45, "word": "BUK!", "sfx": "thwack",
                    "down": w.get("down", false)})
                _land_thrown(it, true)
                return
        var f := BrawlRules.FLOOR
        if it.position.x < f.position.x - 40 or it.position.x > f.end.x + 40:
            _land_thrown(it, false)
    elif it.z > 0.0:
        it.vz -= 1700.0 * delta
        it.z = maxf(0.0, it.z + it.vz * delta)
        if it.z == 0.0:
            it.vz = 0.0
            if it.crate:
                it.crate = false
                fx.dust(it.position, 0.8)
                Sfx.play("bump", -6.0, 1.3)
    it.queue_redraw()


func _land_thrown(it: FloorItem, hit_someone: bool) -> void:
    if BrawlRules.weapon(it.id).get("bounce", false):
        it.flying = false
        it.vx = 0.0
        it.thrower = null
        it.for_id = ""
        it.vz = 260.0 if hit_someone else 0.0
        it.position.x = clampf(it.position.x, BrawlRules.FLOOR.position.x, BrawlRules.FLOOR.end.x)
        return
    fx.poof(it.position + Vector2(0, -it.z), Color(str(BrawlRules.weapon(it.id).get("color", "ffffff"))))
    _items.erase(it)
    it.queue_free()


func _on_hit(target: BrawlFighter, dmg: int, info: Dictionary, by: BrawlFighter) -> void:
    var p := target.head_pos() + Vector2(0, 40)
    if info["blocked"]:
        fx.word(p, "TANGKIS", Color("9fd4ff"))
        Sfx.play("tock", -4.0)
    else:
        fx.star(p + Vector2(-target.facing * 10, 10), info["big"])
        # one word per attack, even when it hits several people
        if by.get_meta("word_attack", -1) != by.attack_no:
            by.set_meta("word_attack", by.attack_no)
            fx.word(p + Vector2(0, -30), str(info["word"]), Color("ffe066") if by.team == 0 else Color("ffb0a0"), info["big"])
        Sfx.play_varied(str(info["sfx"]), -2.0, 0.12)
        if info["big"] or info["ko"]:
            _slowmo()
    fx.number(target.head_pos(), dmg, Color("ffd24a") if by.is_me else (Color("ff7a6a") if target.is_me else Color.WHITE))
    if target.is_me or by.is_me:
        _shake(7.0 if info["big"] else 4.0)
    if info["ko"]:
        fx.word(target.head_pos() + Vector2(0, -20), "PENGSAN!", Color("c9a0ff"), true)
    hit_landed.emit(by, target, dmg)
    changed.emit()


func _slowmo() -> void:
    if Engine.time_scale != 1.0:
        return   # autotest runs fast, leave it alone
    Engine.time_scale = 0.35
    # a connection, not an await: if the match is freed mid slow-mo (Keluar)
    # the call is dropped instead of resuming on a freed node
    get_tree().create_timer(0.15, true, false, true).timeout.connect(_end_slowmo)


func _end_slowmo() -> void:
    Engine.time_scale = 1.0


func _exit_tree() -> void:
    Engine.time_scale = 1.0


```

```
func _shake(s: float) -> void:
    var home := BrawlRules.SCREEN_FLOOR.position
    var t := create_tween()
    for i in 4:
        t.tween_property(self, "position", home + Vector2(randf_range(-s, s), randf_range(-s, s)), 0.03)
    t.tween_property(self, "position", home, 0.03)


## A school item on the floor (or flying, after a throw). Drops in a crate.
class FloorItem extends Node2D:
    var id := ""
    var for_id := ""     ## dropped for this fighter (others can take it after 4 s)
    var z := 0.0
    var vz := 0.0
    var vx := 0.0
    var flying := false
    var crate := false
    var age := 0.0
    var thrower: BrawlFighter

    func _draw() -> void:
        draw_set_transform(Vector2.ZERO, 0, Vector2(1, 0.3))
        draw_circle(Vector2.ZERO, 22, Color(0, 0, 0, 0.3))
        draw_set_transform(Vector2.ZERO)
        var p := Vector2(0, -12 - z)
        if crate:
            draw_rect(Rect2(p + Vector2(-22, -22), Vector2(44, 40)), Color("b07a3a"))
            draw_rect(Rect2(p + Vector2(-22, -22), Vector2(44, 40)), Color(0.2, 0.12, 0.05), false, 3.0)
            draw_line(p + Vector2(-22, -2), p + Vector2(22, -2), Color(0.2, 0.12, 0.05), 2.0)
            draw_arc(p + Vector2(0, -40), 26, PI, TAU, 16, Color.WHITE, 3.0)   # little parachute
            return
        var w := BrawlRules.weapon(id)
        var spin := age * 14.0 if flying else 0.0
        if not flying and for_id != "" and age < 4.0:
            draw_arc(Vector2.ZERO + Vector2(0, -2), 30, 0, TAU, 24, Color(1, 0.9, 0.4, 0.5 + 0.4 * sin(age * 6.0)), 3.0
)
        draw_set_transform(p + Vector2(-20, 0), spin, Vector2(0.8, 0.8))
        WeaponArt.draw_item(self, id, Color(str(w.get("color", "ffffff"))))
        draw_set_transform(Vector2.ZERO)
```

`brawl_rules.gd` · scripts/brawl/brawl_rules.gd · 58 lines

```
class_name BrawlRules
extends RefCounted
## Gelanggang Pesta Silat: the numbers. Every round is SOALAN (everyone
## answers one Bab 1 question) -> KEPUTUSAN (who got Semangat) -> AKSI (a
## short live brawl). Right answer = full damage + a school weapon drops for
## you; fastest right = Jurus Kilat; wrong = Lemah (30% damage). Last team
## standing wins. Frame data lives in data/brawl/moves.json.

const HP := 140
const ASK_TIME := 15.0
const BANNER_TIME := 2.2
const FIGHT_TIME := 10.0
const MAX_ROUNDS := 8
const SEMANGAT := 1.0
const LEMAH := 0.3
const BLOCK_TAKE := 0.2
const REWARD_COINS := 20
const REWARD_EXP := 30
const MAX_FLOOR_WEAPONS := 4

## team sizes [yours, theirs]
const FORMATS := {"1v1": [1, 1], "2v2": [2, 2], "1v2": [1, 2], "1v3": [1, 3]}
## bot answer accuracy range and fighting style
const LEVELS := {
    "mudah": {"name": "Mudah", "acc": [0.4, 0.55], "speed": 0.72, "block": 0.12, "cd": [1.0, 1.7], "combo": 0.25},
    "biasa": {"name": "Biasa", "acc": [0.55, 0.72], "speed": 0.88, "block": 0.3, "cd": [0.6, 1.2], "combo": 0.5},
    "sukar": {"name": "Sukar", "acc": [0.72, 0.88], "speed": 1.0, "block": 0.5, "cd": [0.35, 0.8], "combo": 0.75},
}

## The arena is simulated in "fight units" (fighters drawn 4x) and shown
## VIEW_SCALE bigger on the floor of the hall background (SCREEN_FLOOR).
const VIEW_SCALE := 1.25
const SCREEN_FLOOR := Rect2(120, 515, 1040, 175)
## the floor fighters stand on, in fight units: x left-right, y = depth
const FLOOR := Rect2(0, 0, 832, 140)
const DEPTH_BAND := 28.0   ## how far apart in depth a hit still connects
const SCALE := 4.0         ## sprites are drawn 4x

const MOVES_PATH := "res://data/brawl/moves.json"
static var _data := {}

```

```
42
43  static func data() -> Dictionary:
44      if _data.is_empty():
45          _data = JSON.parse_string(FileAccess.get_file_as_string(MOVES_PATH))
46      return _data
47
48
49  static func move(name: String) -> Dictionary:
50      return data()["moves"].get(name, {})
51
52
53  static func weapon(id: String) -> Dictionary:
54      return data()["weapons"].get(id, {})
55
56
57  static func weapon_ids() -> Array:
58      return data()["weapons"].keys()
```

weapon_art.gd · scripts/brawl/weapon_art.gd · 56 lines

```
1   class_name WeaponArt
2   extends RefCounted
3   ## Draws the Gelanggang school items. Uses the PixelLab picture in
4   ## assets/brawl/<id>.png when there is one, otherwise simple cartoon shapes.
5   ## Items are drawn pointing along +x from the hand (handle at the origin).
6
7   const DIR := "res://assets/brawl/"
8   static var _tex := {}
9
10
11  static func texture(id: String) -> Texture2D:
12      if not _tex.has(id):
13          var p := DIR + id + ".png"
14          _tex[id] = load(p) if ResourceLoader.exists(p) else null
15      return _tex[id]
16
17
18  static func draw_item(ci: CanvasItem, id: String, col: Color) -> void:
19      var t := texture(id)
20      if t:
21          var s := 2.6
22          ci.draw_texture_rect(t, Rect2(Vector2(-6, -t.get_height() * s / 2.0), t.get_size() * s), false)
23          return
24      var ink := Color(0.12, 0.08, 0.05)
25      match id:
26          "pembaris":
27              ci.draw_rect(Rect2(0, -6, 92, 12), col)
28              ci.draw_rect(Rect2(0, -6, 92, 12), ink, false, 2.0)
29              for i in range(8, 90, 10):
30                  ci.draw_line(Vector2(i, -6), Vector2(i, -1 if i % 20 else 1), ink, 1.5)
31          "buku":
32              ci.draw_rect(Rect2(0, -18, 40, 36), col)
33              ci.draw_rect(Rect2(34, -18, 6, 36), Color(0.95, 0.92, 0.85))
34              ci.draw_rect(Rect2(0, -18, 40, 36), ink, false, 2.0)
35              ci.draw_line(Vector2(8, -6), Vector2(28, -6), Color(1, 0.9, 0.5), 3.0)
36          "penyapu":
37              ci.draw_line(Vector2(0, 0), Vector2(84, 0), Color("8a5a2a"), 6.0)
38              ci.draw_colored_polygon(PackedVector2Array([Vector2(80, -6), Vector2(118, -20), Vector2(122, 20), Vector2(8
    0, 6)]), col)
39              for i in 5:
40                  ci.draw_line(Vector2(84, 0), Vector2(120, -16 + i * 8), ink, 1.2)
41          "beg":
42              ci.draw_rect(Rect2(10, -20, 40, 40), col)
43              ci.draw_rect(Rect2(10, -20, 40, 40), ink, false, 2.0)
44              ci.draw_line(Vector2(0, 0), Vector2(14, -14), ink, 4.0)
45              ci.draw_rect(Rect2(18, -4, 24, 14), col.lightened(0.25))
46          "bola":
47              ci.draw_circle(Vector2(26, 0), 22, col)
48              ci.draw_arc(Vector2(26, 0), 22, 0, TAU, 24, ink, 2.0)
49              ci.draw_line(Vector2(4, 0), Vector2(48, 0), ink, 1.5)
50              ci.draw_arc(Vector2(26, 0), 12, -PI / 2, PI / 2, 12, ink, 1.5)
51          "bantal":
52              ci.draw_rect(Rect2(0, -16, 56, 32), col)
53              ci.draw_rect(Rect2(0, -16, 56, 32), Color(0.6, 0.6, 0.65), false, 2.0)
54              ci.draw_line(Vector2(6, 0), Vector2(50, 0), Color(0.8, 0.8, 0.85), 1.5)
55          _:
56              ci.draw_rect(Rect2(0, -6, 60, 12), col)
```

scripts/dev/

autotest.gd · scripts/dev/autotest.gd · 1177 lines

```
extends Node
## End-to-end play-through used by CI and for screenshots.
## Run:  godot --path . -- --autotest [--shots=/abs/dir]
## title -> guest login -> prologue -> wake-up in Bilik Alif (bag, first
## mission, map arrow) -> every Bab 1 puzzle at
## its place in the world (one wrong try + Pau hints each) -> summaries ->
## penutup -> Beg Alif -> lobby, then prints AUTOTEST PASS (exit 0, 1 on fail).

const UI = preload("res://scripts/ui/ui.gd")
const TIMEOUT := 1200.0

var shots_dir: String = ""
var _t: float = 0.0
var _adv_t: float = 0.0
var _shot_n: int = 0


func _ready() -> void:
    for a in OS.get_cmdline_user_args():
        if a.begins_with("--shots="):
            shots_dir = a.trim_prefix("--shots=")
            DirAccess.make_dir_recursive_absolute(shots_dir)
    process_mode = Node.PROCESS_MODE_ALWAYS
    var d := DirAccess.open("user://")  # start from a clean guest profile
    if d:
        for f in d.get_files():
            if f.begins_with("simpan_"):
                d.remove(f)
    _run()


func _process(delta: float) -> void:
    _t += delta
    if _t > TIMEOUT:
        _fail("timeout after %d s" % int(TIMEOUT))
    _adv_t += delta
    # tap through every dialogue (--watch: hands off, cutscenes must play by themselves)
    if Main.dialog_box.visible and _adv_t > 0.08 and not "--watch" in OS.get_cmdline_user_args():
        _adv_t = 0.0
        Main.dialog_box._advance()


func _fail(msg: String) -> void:
    printerr("AUTOTEST FAIL: " + msg)
    get_tree().quit(1)


func wait(s: float) -> void:
    await get_tree().create_timer(s).timeout


func shot(name: String) -> void:
    if shots_dir == "":
        return
    await RenderingServer.frame_post_draw
    var img := get_viewport().get_texture().get_image()
    if img == null or img.is_empty():
        return
    _shot_n += 1
    img.save_png("%s/%02d_%s.png" % [shots_dir, _shot_n, name])
    print("shot ", name)


func scene() -> Node:
    return get_tree().current_scene


## name of what's on screen: scene file name, or the hosted prototype screen
func screen_name() -> String:
    var s := scene()
    if s == null:
        return ""
    var n := s.scene_file_path.get_file().get_basename()
    return Game.pending_screen if n == "screen_host" else n


func hosted() -> Node:
    return scene().get_child(scene().get_child_count() - 1)

```

```
80
81  func wait_screen(name: String, limit: float = 30.0) -> void:
82      var t := 0.0
83      while screen_name() != name:
84          await wait(0.1)
85          t += 0.1
86          if t > limit:
87              _fail("screen %s did not appear (at %s)" % [name, screen_name()])
88              return
89      await wait(0.6)
90
91
92  ## zone briefings ("Persiapan Misi") wait for MULA MENEROKA: shoot and close them
93  func _close_preps() -> void:
94      for c in Main.overlay_root().get_children():
95          if c.has_method("_close") and not c.is_queued_for_deletion() and str(c.get_script().resource_path).ends_with("z
    one_prep.gd"):
96              await shot("prep_" + str(c.get("params").get("zone", "")))
97              if is_instance_valid(c) and not c.is_queued_for_deletion():
98                  c._close()
99
100
101 func wait_idle(limit: float = 60.0) -> void:
102     var t := 0.0
103     while true:
104         await _close_preps()
105         var s := scene()
106         var busy: bool = Main.dialog_box.visible or Game.is_locked()
107         if s and "busy" in s and s.busy:
108             busy = true
109         if s and "runner" in s and s.runner.busy:
110             busy = true
111         if not busy:
112             break
113         await wait(0.1)
114         t += 0.1
115         if t > limit:
116             _fail("still busy after %d s on %s" % [int(limit), screen_name()])
117             return
118
119
120 func find_button(root: Node, prefix: String) -> Button:
121     for c in root.get_children():
122         if c is Button and (c as Button).visible and (c as Button).text.begins_with(prefix):
123             return c
124         var r := find_button(c, prefix)
125         if r:
126             return r
127     return null
128
129
130 func close_top_overlay() -> void:
131     var root: Control = Main.overlay_root()
132     var top := root.get_child(root.get_child_count() - 1)
133     if top.has_method("close"):
134         top.close()
135
136
137 # ---------------------------------------------------------------- the run
138 ## group chat, room, reading, then the dream: classmates at the port, Nirbana
139 ## in Gua Warisan. It is a cutscene: the dialogue taps above just hurry it.
140 ## after Bab 1: the Arena yards on both islands (sparring players, dummies,
141 ## board) and the Jalan Pintas in Beg Alif (locked places refuse)
142 func _check_arena_travel() -> bool:
143     var w := scene()
144     if w.entities().get("arenas", []).size() != 2:
145         _fail("there should be an Arena yard on each island")
146         return false
147     var beg: Node = Main.open_beg(w)
148     await wait(0.3)
149     if find_button(beg, "Jalan Pintas") == null:
150         _fail("Beg Alif in the world should have a Jalan Pintas tab")
151         return false
152     beg._show_travel()
153     await wait(0.3)
154     await shot("beg_jalan_pintas")
155     close_top_overlay()
156     var by_id := {}
157     for t in w.travel_places():
158         by_id[str(t["place"]["id"])] = t
159     if not by_id["pelabuhan"]["ok"] or not by_id["arena_lembah"]["ok"]:
160         _fail("after Bab 1, Misi 1.2 and Lembah should be open for Jalan Pintas")
161         return false
162     if by_id["serantau"]["ok"]:
163         _fail("Misi 2.4 should still be locked for Jalan Pintas")
```

```
        return false
    # Kampung Jejak arena yard
    w.fast_travel({"at": [40, 38], "name": "Arena Ulang Kaji"})
    await _wait_reload(w)
    await wait_idle()
    await shot("arena_yard")
    # across the Pintu Kabus to the Lembah arena, then back to the port
    w = scene()
    w.fast_travel(by_id["arena_lembah"]["place"])
    await _wait_reload(w)
    await wait_idle()
    if scene().player.cell.x < 82:
        _fail("Jalan Pintas to Arena Lembah should land in Lembah, at %s" % str(scene().player.cell))
        return false
    await shot("arena_lembah")
    w = scene()
    w.fast_travel(by_id["pelabuhan"]["place"])
    await _wait_reload(w)
    await wait_idle()
    if (scene().player.cell - Vector2i(61, 41)).length() > 4.0:
        _fail("Jalan Pintas to Misi 1.2 landed at %s" % str(scene().player.cell))
        return false
    return true


## a world that reloads itself (Jalan Pintas, Pintu Kabus): wait for the new one
func _wait_reload(old: Node) -> void:
    var t := 0.0
    while scene() == old or scene() == null:
        await wait(0.1)
        t += 0.1
        if t > 15.0:
            _fail("the world did not reload")
            return
    await wait_screen("world")


func _check_prologue() -> bool:
    await wait_screen("prologue")
    var phases := {}
    var dream_shots := {"bg_pelabuhan_huru": false, "rakan_aisyah_north": false, "nirbana_south": false}
    var tw := 0.0
    while screen_name() == "prologue" and tw < 480.0:
        var ph := str(scene().phase)
        if ph != "" and not phases.has(ph):
            phases[ph] = true
            await wait(1.2)
            await shot("prologue_" + ph)
            if screen_name() != "prologue":
                break
        if ph == "reading" and not phases.has("ingatan") and int(scene()._words_read) >= 3:
            phases["ingatan"] = true  # key words in the jar, the oldest falling out
            await shot("prologue_ingatan")
        for c in scene().get_children():
            if c is TextureRect and c.texture:
                for k in dream_shots:
                    if not dream_shots[k] and c.texture.resource_path.get_file().get_basename() == k and c.modulate.a >
 0.9:
                        dream_shots[k] = true
                        await shot("prologue_" + k)
        await wait(0.1)
        tw += 0.1
    for ph in ["phone", "room", "reading", "doze", "port", "gua"]:
        if not phases.has(ph):
            _fail("prologue never reached '%s' (got %s)" % [ph, str(phases.keys())])
            return false
    if dream_shots.values().has(false):
        _fail("prologue dream scenes not shown: %s" % str(dream_shots))
        return false
    return true


func _run() -> void:
    if "--only=brawl" in OS.get_cmdline_user_args():
        # quick loop while working on the Gelanggang
        await wait_screen("title")
        Game.load_profile("Tetamu", "guest")
        Game.goto("lobby")
        await wait_screen("lobby")
        await shot("lobby")
        await _check_brawl()
        print("AUTOTEST PASS (brawl only)")
        get_tree().quit(0)
        return
    if "--only=ronda3" in OS.get_cmdline_user_args():
```

```
        await _check_ronda3()
        return
    await wait_screen("title")
    await shot("title")
    Game.goto("login")
    await wait_screen("login")
    await shot("login")
    Game.goto("register")
    await wait_screen("register")
    await shot("register")
    Game.goto("login")
    await wait_screen("login")
    hosted()._guest()

    # ---- prologue + the wake-up in Alif's room, played in full
    if not await _check_prologue():
        return
    if not await _check_bilik():
        return
    if "--only=prologue" in OS.get_cmdline_user_args():
        # quick loop while working on the introduction
        await wait_idle(240)
        await shot("world_start")
        print("AUTOTEST PASS (prologue only) in %d s" % int(_t))
        get_tree().quit(0)
        return
    await wait_idle(120)
    await wait(0.5)
    await shot("world_start")
    # first visit to Kampung Jejak: title card + why we are here (once only)
    scene().player.place_at(Vector2i(31, 31))
    scene()._enter_area_at(Vector2i(31, 31))
    await wait(0.9)
    await shot("area_card_kampung")
    await wait_idle()
    var seen_n: int = Game.save["seen"].size()
    scene()._enter_area_at(Vector2i(31, 31))
    await wait_idle()
    if Game.save["seen"].size() != seen_n:
        _fail("area intro should only play once")
        return

    var sp: Node = Main.open_settings()
    await wait(0.4)
    await shot("settings")
    sp._show(1)
    await wait(0.3)
    await shot("settings_audio")
    close_top_overlay()
    Game.set_setting("lang", "en")
    await wait(0.4)
    await shot("world_english")
    if scene().hud._level.text.begins_with("TAHAP"):
        _fail("HUD not translated to English")
        return
    Game.set_setting("lang", "ms")
    await _check_map()
    await _check_controls()
    Main._on_pau_tap()
    await wait(0.3)
    await shot("pau_hint_bubble")

    # ---- Bab 1: the four zones, each puzzle at its spot in the world
    if not await _play_zones(Game.chapter_zones(1)):
        return
    if not await _play_ujian(1, true):
        return
    if not await _finish_penutup("bab2"):
        return
    await wait_screen("world")
    await wait_idle()
    await shot("world_after_bab1")
    scene().hud.set_menu_open(true)
    await wait(0.3)
    await shot("hud_menu")
    if not scene().hud.is_menu_open():
        _fail("HUD menu did not open")
        return
    scene().hud.set_menu_open(false)
    scene().open_map()
    await wait(0.5)
    var mo: Node = Main.overlay_root().get_child(Main.overlay_root().get_child_count() - 1)
    mo._select_tab(1, "_show_keywords")
    await wait(0.3)
    await shot("map_keywords_bab1")
```

```
    var before_center: Vector2 = mo.view.center
    mo._focus_zone("1_1")
    if mo.view.center == before_center:
        _fail("tapping a keyword should move the map")
        return
    mo._kw_filter = "melaka"
    mo._fill_keywords()
    await wait(0.3)
    await shot("map_keywords_search")
    mo._kw_filter = "zzz"
    mo._fill_keywords()
    await wait(0.1)
    if mo._kw_box.get_child_count() != 1:
        _fail("keyword search with no hit should show one 'tiada' line")
        return
    mo.close()
    await wait(0.2)
    Main.open_beg()
    await wait(0.4)
    await shot("beg")
    close_top_overlay()
    Main.open_nota()
    await wait(0.4)
    await shot("nota")
    close_top_overlay()
    if not await _check_arena_travel():
        return
    await _check_shop()
    Game.goto("arka")
    await wait_screen("arka")
    await shot("arka")
    hosted()._enter({"no": 2}, false)
    hosted()._show_arc(1)
    await wait(0.3)
    await shot("arka_t5_locked")
    Game.goto("lobby")
    await wait_screen("lobby")
    await shot("lobby")
    await _check_papan()
    await _check_battle()
    await _check_brawl()

    # ---- Bab 2: through the Pintu Kabus into Lembah Kebangkitan, eight missions
    var ent: Dictionary = JSON.parse_string(FileAccess.get_file_as_string("res://data/world/entities.json"))
    GameState.spawn_cell = Vector2i(int(ent["arrive2"][0]), int(ent["arrive2"][1]))
    Game.goto("world")
    await wait_screen("world")
    await wait_idle()
    await shot("lembah_arrive")
    if scene().entities().get("misi_signs", []).size() != 8:
        _fail("Lembah should have 8 numbered mission signposts")
        return
    if scene().fog_rects().size() != 8:
        _fail("all 8 Bab 2 places should start under the Kabus, got %d" % scene().fog_rects().size())
        return
    scene().open_map()
    await wait(0.5)
    await shot("map_lembah_fog")
    close_top_overlay()
    if scene().portals.is_empty():
        _fail("Bab 2 portal should be open after Bab 1")
        return
    for c in scene().portals:
        if not scene().is_walkable(c):
            _fail("portal cell %s should be walkable" % str(c))
            return
    if not await _play_zones(Game.chapter_zones(2)):
        return
    if not await _play_ujian(2, false):
        return
    if not await _finish_penutup("bab3"):
        return
    await wait_screen("world")
    await wait_idle()
    await shot("world_after_bab2")
    if not scene().fog_rects().is_empty():
        _fail("the Kabus should be gone after Bab 2, left: %d" % scene().fog_rects().size())
        return
    if scene()._gabenor != null:
        _fail("Gabenor Sekatan should be gone after Misi 2.8")
        return
    Main.open_beg()
    await wait(0.4)
    await shot("beg_bab2")
    close_top_overlay()
```

```
    Main.open_nota()
    await wait(0.4)
    await shot("nota_bab2")
    close_top_overlay()
    BattleRules.chapter = 2
    await _check_battle()
    if not await _check_rooms():
        return

    # ---- checks
    if not Game.chapter_done(2):
        _fail("Bab 2 not marked done")
        return
    if not Game.chapter_done(1):
        _fail("Bab 1 not marked done")
        return
    for z in Game.ZONES:
        if not Game.zone_done(z):
            _fail("zone %s not done" % z)
            return
    var old_skin: String = Game.save["skin"]
    Game.save["skin"] = "melayu"
    if not Game.portrait_path("alif", "happy").ends_with("alif_melayu_happy.png"):
        _fail("Alif's dialogue portrait should follow the skin")
        return
    Game.save["skin"] = old_skin
    if not Game.save["seen"].has("area_kampung"):
        _fail("area intro for Kampung Jejak never shown")
        return
    if Game.save["artifacts"].size() < 12:
        _fail("artifacts missing: %s" % str(Game.save["artifacts"]))
        return
    print("AUTOTEST PASS level=%d exp=%d coins=%d figures=%s" % [Game.save["level"], Game.save["exp"], Game.save["coins
"], str(Game.save["figures"])])
    get_tree().quit(0)


## Plays every zone in the list: walks to each step's spot, solves its puzzle,
## then continues from the summary. False (after _fail) if something broke.
func _play_zones(zones: Array) -> bool:
    for z in zones:
        if z == "1_3":
            GameState.go_to("res://scenes/interior/balai.tscn")
            await wait_screen("balai")
            await wait_idle()
        elif screen_name() != "world":
            await wait_screen("world")
            await wait_idle()
        await shot("zone_" + z)
        if z == "2_8" and screen_name() == "world" and scene()._gabenor == null:
            _fail("Gabenor Sekatan should guard the Menara during Misi 2.8")
            return false
        for s in Game.zone_data(z)["steps"]:
            if not _walk_to(str(s["at"])):
                _fail("no spot for %s in %s" % [s["at"], screen_name()])
                return false
            var tt := 0.0
            while Game.active_puzzle == null:
                await wait(0.1)
                tt += 0.1
                if tt > 30:
                    _fail("puzzle %s never opened" % s["puzzle"])
                    return false
            await wait(0.4)
            await _solve(Game.active_puzzle, "%s_%s" % [z, s["puzzle"]])
            await wait_idle(60)
            await wait(0.3)
            if s == Game.zone_data(z)["steps"][0]:
                await shot("world_after_%s" % s["puzzle"])
        await wait_screen("zone_summary", 40)
        await shot("summary_" + z)
        hosted()._continue()
        if not Game.is_last_zone(z):
            await wait_screen("world")
            await wait_idle()
    return true


## Ujian Bab after a chapter's last summary. With fail_first: lose all hearts,
## go back to revise, check the boss blocks the way on the map, talk to him and
## pass. Passing = 7 right answers.
func _play_ujian(ch: int, fail_first: bool) -> bool:
    await wait_screen("ujian", 30)
    if fail_first:
        var p = await _ujian_puzzle()
```

```
        if p == null:
            return false
        await shot("ujian_%d" % ch)
        for i in 3:
            p._pick((int(p._q()["answer"]) + 1) % 4)
            await wait(0.2)
            if i < 2:
                p._advance()
        await wait(3.0)
        await wait_idle()
        var back := find_button(hosted(), "Ulang kaji")
        if back == null:
            _fail("Ujian fail card missing")
            return false
        await shot("ujian_%d_fail" % ch)
        back.pressed.emit()
        await wait_screen("world")
        await wait_idle()
        if Game.chapter_done(ch) or not scene().portals.is_empty() and ch == 1:
            _fail("chapter %d should stay closed until the Ujian is passed" % ch)
            return false
        if not _walk_to(str(Game.ujian_def(ch)["npc"])):
            _fail("Ujian boss %d not on the map" % ch)
            return false
        await shot("ujian_%d_boss_on_map" % ch)
        await wait_screen("ujian", 30)
    var q = await _ujian_puzzle()
    if q == null:
        return false
    for i in int(Game.ujian_def(ch)["need"]):
        q._pick(int(q._q()["answer"]))
        await wait(0.2)
        if i == 2:
            await shot("ujian_%d_hp" % ch)
        if q.get("_next").visible:
            q._advance()
    await wait(2.0)
    var go := find_button(q, "Teruskan")
    if go == null:
        _fail("Ujian %d not won" % ch)
        return false
    go.pressed.emit()
    if not Game.save["ujian"].has(ch):
        _fail("Ujian %d not saved" % ch)
        return false
    return true


func _ujian_puzzle():
    var t := 0.0
    while Game.active_puzzle == null:
        await wait(0.1)
        t += 0.1
        if t > 60:
            _fail("Ujian puzzle never opened")
            return null
    await wait(0.4)
    return Game.active_puzzle


## Pau wakes Alif in his room and explains; the start prompt sits on the bag,
## the door waits for it, then Misi 1.1 starts and the map arrow points there.
func _check_bilik() -> bool:
    await wait_screen("bilik_alif", 120)
    await wait(2.0)
    await shot("bilik_wake")
    var t := 0.0
    while not scene()._waiting_bag:
        await wait(0.2)
        t += 0.2
        if t > 240.0:
            _fail("the wake-up never got to the bag")
            return false
    await wait(0.5)
    await shot("bilik_start_prompt")
    if scene()._prompt == null:
        _fail("no start prompt over the bag")
        return false
    if scene().player._look != "alif_tanpa_beg":
        _fail("before he picks up his bag, Alif should not wear one")
        return false
    scene()._leave()
    await wait(0.5)
    if screen_name() != "bilik_alif":
        _fail("Alif left his room without the bag")
```

```
        return false
    scene().player.place_at(Vector2i(12, 8), "north")
    scene()._interact(Vector2i(12, 7))
    await wait_idle(120)
    if not Game.save["seen"].has("bilik_tutorial") or scene()._prompt != null:
        _fail("taking the bag should clear the prompt and start Misi 1.1")
        return false
    if scene().player._look != "":
        _fail("after picking it up, Alif should wear his bag again")
        return false
    await shot("bilik_mission_start")
    scene()._leave()
    await wait_screen("world", 60)
    var goal: Dictionary = scene().objective()
    if str(goal.get("name", "")) != "Misi 1.1":
        _fail("the map arrow should point at Misi 1.1, not %s" % str(goal))
        return false
    await shot("world_arrow")
    return true


## The story rooms: enter from the map door, read every exhibit,
## walk out again.
func _check_rooms() -> bool:
    for id in ["rumah_alif", "sitc", "jelutong_press"]:
        Game.goto("world")
        await wait_screen("world")
        await wait_idle()
        scene()._talk_or_act(id)
        var room: String = scene().ROOMS[id][0].get_file().get_basename()
        await wait_screen(room, 30)
        await wait_idle()
        await shot("room_" + room)
        var s := scene()
        if s._exhibits.is_empty():
            _fail("room %s has no exhibits" % room)
            return false
        for ex in s._exhibits:
            s._interact(s._exhibits[ex]["cells"][0])
            await wait(0.2)
            for c in Main.overlay_root().get_children():
                if c.has_method("close"):
                    c.close()
            await wait_idle()
        if s.npc:  # Bilik Alif has nobody but Pau
            s._interact(s.npc.cell)
            await wait_idle()
        s._on_step(s.exit_cells[0])
        await wait_screen("world", 30)
        await wait_idle()
    return true


## Clicks through the chapter ending: flag, Nota Alif, wellbeing card, teaser.
func _finish_penutup(teaser: String) -> bool:
    await wait_screen("penutup", 30)
    var t := 0.0
    var shot_flag := false
    while screen_name() == "penutup":
        var nota: Node = null
        for c in Main.overlay_root().get_children():
            if c.has_method("_read_page"):
                nota = c
        if nota:
            await wait(0.5)
            await shot("nota_alif_" + teaser)
            nota.close()
        elif not Main.dialog_box.visible:
            var b := find_button(hosted(), "Teruskan")
            if b == null:
                b = find_button(hosted(), "KEMBALI")
                if b:
                    await shot("teaser_" + teaser)
            if b:
                b.pressed.emit()
        elif not shot_flag:
            shot_flag = true
            await shot("penutup_flag_" + teaser)
        await wait(0.2)
        t += 0.2
        if t > 120:
            _fail("penutup stuck")
            return false
    return true

```

```
672
673 ## Bilik Pertarungan vs 2 bots, sped up: answer right, hit the weakest,
674 ## play until one is left. The winner gets +20 syiling.
675 func _check_battle() -> void:
676     var coins := int(Game.save["coins"])
677     var played := int(Game.save["battle"]["played"])
678     Game.goto("pertarungan", {"mode": "bot", "bots": 2})
679     await wait_screen("pertarungan")
680     var b: Node = hosted().battle
681     Engine.time_scale = 6.0
682     var shot_q := false
683     var shot_hit := false
684     var shot_ko := false
685     var t := 0.0
686     while str(b.state.get("status", "")) != "over":
687         var st := str(b.state.get("status", ""))
688         if st == "question" and b.me()["alive"] and int(b.me()["answer"]) < 0:
689             var q: Dictionary = b._questions[b._qi]
690             hosted()._pick_target(BattleRules.auto_target(b.state["players"], b.my_id))
691             if not shot_q:
692                 shot_q = true
693                 Engine.time_scale = 1.0
694                 await shot("battle_question")
695                 Engine.time_scale = 6.0
696             hosted()._answer(int(q["answer"]))
697         elif st == "reveal" and not shot_hit and not b.state["hits"].is_empty():
698             shot_hit = true
699             Engine.time_scale = 1.0
700             await wait(0.3)
701             await shot("battle_hits")
702             Engine.time_scale = 6.0
703         if not shot_ko and st == "question" and BattleRules.alive(b.state["players"]).size() < b.state["players"].size(
):
704             shot_ko = true
705             Engine.time_scale = 1.0
706             await wait(0.6)
707             await shot("battle_knocked_out")
708             Engine.time_scale = 6.0
709         await wait(0.1)
710         t += 0.1
711         if t > 120.0:
712             Engine.time_scale = 1.0
713             _fail("battle never finished (round %d)" % int(b.state.get("round", 0)))
714             return
715     Engine.time_scale = 1.0
716     await wait(0.5)
717     await shot("battle_result")
718     if int(Game.save["battle"]["played"]) != played + 1:
719         _fail("battle not counted")
720         return
721     var won: bool = b.state["winner"] == b.my_id
722     if won and int(Game.save["coins"]) != coins + BattleRules.REWARD_COINS:
723         _fail("winner did not get %d syiling (%d -> %d)" % [BattleRules.REWARD_COINS, coins, int(Game.save["coins"])])
724         return
725     print("battle: rounds=%d winner=%s won=%s" % [int(b.state["round"]), str(b.state["winner"]), str(won)])
726
727
728 ## Gelanggang Pesta Silat in every format, a bot playing for you (autopilot).
729 ## Damage only lands in AKSI, Lemah hits weaker than Semangat, someone
730 ## faints, the match ends and the winners get their syiling.
731 func _check_brawl() -> void:
732     for fmt in ["1v1", "2v2", "1v2", "1v3"]:
733         var coins := int(Game.save["coins"])
734         Game.goto("gelanggang", {"format": fmt, "level": "mudah", "autopilot": true})
735         await wait_screen("gelanggang")
736         var s := hosted()
737         var m: BrawlMatch = s.match_node
738         var sizes: Array = BrawlRules.FORMATS[fmt]
739         if m.team_alive(0).size() != sizes[0] or m.team_alive(1).size() != sizes[1]:
740             _fail("brawl %s: wrong team sizes" % fmt)
741             return
742         Engine.time_scale = 1.0 if fmt == "1v1" and shots_dir != "" else 4.0
743         var shots_done := {}
744         var hp_at_question := -1
745         var t := 0.0
746         var hits := {"weak": 0, "full": 0}   # lambdas capture by value: count in a dictionary
747         m.hit_landed.connect(func(by, _target, dmg):
748             if m.phase != "aksi":
749                 _fail("brawl: damage outside AKSI")
750             if by.mult < 1.0:
751                 hits["weak"] += 1
752                 if dmg > 4:
753                     _fail("brawl: Lemah hit did %d" % dmg)
754             else:
755                 hits["full"] += 1)
```

```
        while m.phase != "tamat":
            if fmt == "1v1":
                if m.phase == "soalan" and not shots_done.has("q") and m.phase_t > 0.5:
                    shots_done["q"] = true
                    await shot("brawl_soalan")
                elif m.phase == "keputusan" and not shots_done.has("k") and m.phase_t > 0.4:
                    shots_done["k"] = true
                    await shot("brawl_keputusan")
                elif m.phase == "aksi" and not shots_done.has("a") and m.phase_t > 2.5:
                    shots_done["a"] = true
                    await shot("brawl_aksi")
                elif m.phase == "aksi" and not shots_done.has("a2") and m.phase_t > 6.0:
                    shots_done["a2"] = true
                    await shot("brawl_aksi_2")
                    Engine.time_scale = 4.0
            elif m.phase == "aksi" and not shots_done.has("a") and m.phase_t > 3.0:
                shots_done["a"] = true
                Engine.time_scale = 1.0
                await shot("brawl_%s" % fmt)
                Engine.time_scale = 4.0
            if m.phase == "soalan" and hp_at_question < 0:
                hp_at_question = 0
                for f in m.fighters:
                    hp_at_question += f.hp
            await wait(0.1)
            t += 0.1
            if t > 200.0:
                Engine.time_scale = 1.0
                _fail("brawl %s never finished (round %d, phase %s)" % [fmt, m.round_no, m.phase])
                return
        Engine.time_scale = 1.0
        await wait(0.6)
        await shot("brawl_result_%s" % fmt)
        if hits["full"] == 0:
            _fail("brawl %s: no full-power hits landed" % fmt)
            return
        var won := m.winner_team == 0
        if won and int(Game.save["coins"]) != coins + BrawlRules.REWARD_COINS:
            _fail("brawl %s: winners should get %d syiling" % [fmt, BrawlRules.REWARD_COINS])
            return
        print("brawl %s: rounds=%d won=%s hits full=%d weak=%d" % [fmt, m.round_no, str(won), hits["full"], hits["weak"
]])
    Game.goto("lobby")
    await wait_screen("lobby")


## Papan Markah offline: sample classmates + you, ranked by EXP then wins.
func _check_papan() -> void:
    var rows: Array = load("res://scripts/ui/papan_panel.gd").sample_rows("all")
    var mine: Dictionary = rows.filter(func(r): return r["me"])[0]
    var above := rows.filter(func(r): return not r["me"] and (r["exp"] > mine["exp"] or (r["exp"] == mine["exp"] and r[
"wins"] > mine["wins"]))).size()
    if int(mine["rank"]) != above + 1:
        _fail("papan: my rank %d, expected %d" % [int(mine["rank"]), above + 1])
        return
    var pp: Node = Main.open_papan()
    await wait(0.4)
    await shot("papan_seluruh")
    pp._show(1)
    await wait(0.3)
    await shot("papan_negeri")
    pp._show(2)
    await wait(0.3)
    await shot("papan_rakan")
    pp.close()
    await wait(0.2)


## Profil & Kedai: buy and wear a skin with the syiling earned in Bab 1.
func _check_shop() -> void:
    var before := int(Game.save["coins"])
    Game.save["profile"]["negeri"] = "Melaka"
    var pp: Node = Main.open_profile()
    await wait(0.5)
    await shot("profile")
    pp._show(1)
    await wait(0.3)
    var err := Game.buy_skin("pengakap")
    if err != "" or int(Game.save["coins"]) != before - 120:
        _fail("buying a skin failed: %s (coins %d -> %d)" % [err, before, int(Game.save["coins"])])
        return
    Game.equip_skin("pengakap")
    pp._show(1)
    await wait(0.3)
    await shot("shop")
```

```
    pp.close()
    await wait(0.3)
    await shot("world_new_skin")
    if scene().hud._avatar.texture != UI.head("alif_pengakap"):
        _fail("HUD picture should follow the outfit")
        return


## Mini map in the corner (Pau button hidden), zoom levels, full map + legend.
func _check_map() -> void:
    var w := scene()
    if not w.minimap.visible or Main.pau_button.visible:
        _fail("mini map should replace the Pau button in the world")
        return
    await shot("minimap")
    w.minimap._set_zoom(2)
    await wait(0.3)
    await shot("minimap_whole")
    w.minimap._set_zoom(0)
    w.open_map()
    await wait(0.5)
    var m: Node = Main.overlay_root().get_child(Main.overlay_root().get_child_count() - 1)
    if not m.has_method("_show_keywords"):
        _fail("full map did not open")
        return
    await shot("map_places")
    m._zoom(1.5)
    m._select_tab(1, "_show_keywords")
    await wait(0.3)
    await shot("map_keywords")
    m._select_tab(2, "_show_symbols")
    m.view.set_zoom(0.0)
    await wait(0.3)
    await shot("map_symbols_whole")
    m.close()
    await wait(0.2)


## Tetapan > Kawalan: D-pad, joystick (fake thumb drag walks Alif east),
## left-handed + big, tap only; then back to Automatik.
func _check_controls() -> void:
    Game.set_setting("controls", "joystick")
    var sp: Node = Main.open_settings()
    sp._show(2)
    await wait(0.3)
    await shot("settings_kawalan")
    close_top_overlay()
    var w := scene()
    var tc: Node = w.get_node("TouchControls")
    Game.set_setting("controls", "dpad")
    await wait(0.3)
    if tc.get_child_count() != 6 or not tc.visible:
        _fail("D-pad should have 6 buttons (has %d)" % tc.get_child_count())
        return
    if w.minimap.position.y > 300:
        _fail("mini map should move up when controls are on screen")
        return
    await shot("controls_dpad")
    Game.set_setting("controls", "joystick")
    await wait(0.3)
    var st: Node = tc._stick
    if st == null:
        _fail("joystick not built")
        return
    await shot("controls_joystick_idle")
    var down := InputEventScreenTouch.new()
    down.index = 3
    down.pressed = true
    down.position = st.rest
    st._input(down)
    var drag := InputEventScreenDrag.new()
    drag.index = 3
    drag.position = st.rest + Vector2(st.radius * 0.6, 4)
    st._input(drag)
    if not Input.is_action_pressed("move_right") or Input.is_action_pressed("run"):
        _fail("joystick push right should press move_right only")
        return
    await wait(0.5)
    await shot("controls_joystick_push")
    if w.player.facing != "east":
        _fail("joystick did not turn Alif east (%s)" % w.player.facing)
        return
    if not tc.owns(st.rest) or tc.owns(Vector2(900, 200)):
        _fail("joystick zone should block tap-to-walk only in its corner")
        return
```

```
    var up := InputEventScreenTouch.new()
    up.index = 3
    up.pressed = false
    up.position = drag.position
    st._input(up)
    if Input.is_action_pressed("move_right"):
        _fail("joystick release should let go of move_right")
        return
    Game.set_setting("left_handed", true)
    Game.set_setting("pad_size", 2)
    Game.set_setting("controls", "dpad")
    await wait(0.3)
    await shot("controls_dpad_lefty_big")
    if w.minimap.position.x > 640:
        _fail("left-handed: mini map should move to the left side")
        return
    Game.set_setting("controls", "tap")
    await wait(0.3)
    if tc.get_child_count() != 0 or tc.visible:
        _fail("Ketik sahaja should show no buttons")
        return
    Game.set_setting("left_handed", false)
    Game.set_setting("pad_size", 1)
    Game.set_setting("controls", "auto")
    await wait(0.4)  # let the player stop walking before the zones start


## Put Alif next to a spot (NPC, object or prop), face it and interact.
func _walk_to(id: String) -> bool:
    var s := scene()
    var target := Vector2i(-999, -999)
    if s.has_method("spot_ids") and "spots" in s and s.spots.has(id):
        var sp: Dictionary = s.spots[id]
        target = Vector2i(sp["pos"][0], sp["pos"][1])
    elif "npcs" in s:
        for at in s.npcs:
            if s.npcs[at].id == id:
                target = at
    if "npc" in s and s.npc and s.npc_id == id:
        target = s.npc.cell
    if "_props" in s and s._props.has(id):
        target = s._props[id]["cells"][0]
    if target.x == -999:
        return false
    var stand := Vector2i.ZERO
    var found := false
    for r in range(1, 4):
        for d in [Vector2i(0, r), Vector2i(-r, 0), Vector2i(r, 0), Vector2i(0, -r)]:
            if s.is_walkable(target + d):
                stand = target + d
                found = true
                break
        if found:
            break
    if not found:
        return false
    # multi-cell objects: step next to the nearest footprint cell
    var face := target
    if "spots" in s and s.spots.has(id):
        var sp: Dictionary = s.spots[id]
        for dy in sp["size"][1]:
            for dx in sp["size"][0]:
                var c := Vector2i(sp["pos"][0] + dx, sp["pos"][1] - dy)
                if (c - stand).length_squared() == 1:
                    face = c
    s.player.place_at(stand)
    s.player.face_cell(face)
    s._interact(face)
    return true


func _solve(p: Node, label: String) -> void:
    var path: String = p.get_script().resource_path
    if path.ends_with("choice.gd"):
        await shot("puzzle_" + label)
        p._pick(p.def["options"][0])
    elif path.ends_with("objektif.gd"):
        for qi in p.questions.size():
            var ans := int(p.questions[qi]["answer"])
            p._pick((ans + 1) % 4)  # one wrong try, then a hint that removes an answer
            p.request_hint()
            p.request_hint()
            await wait(0.2)
            if qi == 0:
                await shot("puzzle_" + label)
```

```
            p._pick(ans)
            await wait(0.2)
            if qi + 1 < p.questions.size():
                p._advance()
    elif path.ends_with("suaikan.gd"):
        p._tap_left(0)
        p.try_match(0, (p._right_of(0) + 1) % p.pairs.size())  # one wrong pair
        p.request_hint()
        p.request_hint()
        p.request_hint()  # tier 3 joins a pair
        await wait(0.2)
        for i in p.pairs.size():
            if i == p.pairs.size() / 2:
                await shot("puzzle_" + label)
            if not p._done.has(i):
                p._tap_left(i)
                p.try_match(i, p._right_of(i))
                await wait(0.05)
    elif path.ends_with("board.gd"):
        var links: Array = p.links
        var ids: Array = p.cards.keys()
        var a = p.cards[links[0][0]]
        for id in ids:  # one wrong attempt, one Pau hint
            if p._key(links[0][0], id) == "" and id != links[0][0]:
                p.try_link(a, p.cards[id])
                break
        p.request_hint()
        await wait(0.2)
        for i in links.size():
            if i == links.size() / 2:
                await shot("puzzle_" + label)
            var l: Array = links[i]
            if not p._is_made("%s|%s" % [l[0], l[1]]):
                p.try_link(p.cards[l[0]], p.cards[l[1]])
                await wait(0.05)
    else:
        var cards: Array = p.cards
        var c0 = cards[0]
        for tid in p.targets.keys():
            if tid != str(c0.item["target"]):
                p.try_place(c0, p.targets[tid])
                break
        p.request_hint()
        p.request_hint()
        await wait(0.2)
        await shot("puzzle_" + label)
        for c in cards:
            if not c.placed:
                p.try_place(c, p.targets[str(c.item["target"])])
                await wait(0.05)
    var tt := 0.0
    while not p._finished:
        await wait(0.1)
        tt += 0.1
        if tt > 10:
            _fail("puzzle %s did not finish" % label)
            return
    await wait(0.4)
    await shot("done_" + label)
    p.completed.emit(p.stats)


## Playtest round 3 (2026-10-24): room exit, mission replay, Nota huraian,
## Ujian Bab feedback, Gelanggang Cara Main / Jeda / key layouts.
func _check_ronda3() -> void:
    await wait_screen("title")
    Game.load_profile("Tetamu", "guest")
    Game.save["prolog_done"] = true
    for z in Game.chapter_zones(1):
        var st := Game.zone_state(z)
        st["done"] = true
        st["steps"] = []
        for step in Game.zone_data(z).get("steps", []):
            st["steps"].append(step["puzzle"])
    Game.persist()
    # room: Keluar arrow + HUD button take Alif back outside
    GameState.go_to("res://scenes/interior/balai.tscn")
    await wait_screen("balai")
    await wait_idle()
    await shot("r3_balai_exit")
    var out := find_button(scene(), "Keluar")
    if out == null:
        _fail("no Keluar button in the Balai")
        return
    out.pressed.emit()
```

```
    await wait_screen("world")
    print("ronda3: Keluar ok")
    # replay a finished mission from its NPC
    GameState.go_to("res://scenes/interior/balai.tscn")
    await wait_screen("balai")
    await wait_idle()
    var exp_before := int(Game.save["exp"])
    scene()._interact(scene().npc.cell)
    await wait(0.8)
    await shot("r3_replay_ask")
    var yes := find_button(Main.overlay_root(), "Main semula")
    if yes == null:
        _fail("no replay prompt")
        return
    yes.pressed.emit()
    var t := 0.0
    while Game.active_puzzle == null and t < 20.0:
        await wait(0.2)
        t += 0.2
    if not Game.zone_state("1_3").has("replay") or Game.active_puzzle == null:
        _fail("replay did not start")
        return
    await shot("r3_replay_puzzle")
    print("ronda3: replay started, exp before ", exp_before)
    # Nota opens on the mission page with the huraian
    var n: Control = Main.open_nota("1_3")
    await wait(0.6)
    await shot("r3_nota_top")
    n._scroll.scroll_vertical = 600
    await wait(0.3)
    await shot("r3_nota_huraian")
    n.close()
    Game.active_puzzle.queue_free()
    Game.active_puzzle = null
    Game.unlock_input()
    # Ujian Bab: lose every heart, then the feedback card
    Game.goto("ujian", {"chapter": 1})
    await wait_screen("ujian")
    t = 0.0
    while Game.active_puzzle == null and t < 30.0:
        await wait(0.2)
        t += 0.2
    for k in 3:
        var p = Game.active_puzzle
        if p == null:
            break
        p._pick((int(p._q()["answer"]) + 1) % 4)
        await wait(0.4)
        if p._next.visible:
            p._advance()
            await wait(0.3)
    await wait(4.0)
    await shot("r3_ujian_fail")
    if find_button(scene(), "Cuba lagi") == null:
        _fail("no fail card")
        return
    print("ronda3: ujian feedback ok")
    # Gelanggang: first visit shows Cara Main, then Jeda works
    Game.save["seen"].erase("gelanggang_cara")
    Game.goto("gelanggang", {"format": "1v1", "level": "mudah"})
    await wait_screen("gelanggang", 30.0)
    await wait(0.5)
    await shot("r3_brawl_guide")
    var ok := find_button(scene(), "Mula")
    if ok == null:
        _fail("no Cara Main before the first match")
        return
    ok.pressed.emit()
    await wait(1.0)
    hosted()._open_pause()
    await wait(0.4)
    await shot("r3_brawl_pause")
    find_button(scene(), "Cara main").pressed.emit()
    await wait(0.3)
    find_button(scene(), "Anak panah").pressed.emit()
    await wait(0.3)
    await shot("r3_brawl_keys_panah")
    if Game.brawl_scheme() != "panah":
        _fail("layout switch")
        return
    Game.set_setting("brawl_keys", "wasd")
    hosted()._close_pause()
    print("AUTOTEST PASS (ronda3)")
    get_tree().quit(0)
```

scripts/

dialogue.gd · scripts/dialogue.gd · 44 lines

```
extends Node
## Autoload "Dialogue": world chatter in data/dialogue/*.json (natural,
## repeatable conversations). Talking to someone again moves to their next
## variant; after the last one it picks at random so it never feels scripted.
## Lines are returned in the story format used by Game.say():
##   {"s": speaker, "t": text, "e": expression, "rekaan": bool}

var _data := {}
var _seen := {}  ## id -> times played


func _ready() -> void:
    var dir := DirAccess.open("res://data/dialogue")
    if dir == null:
        push_error("data/dialogue missing")
        return
    for file in dir.get_files():
        if file.ends_with(".json"):
            var parsed = JSON.parse_string(FileAccess.get_file_as_string("res://data/dialogue/" + file))
            if parsed is Dictionary:
                _data.merge(parsed, true)


func has(id: String) -> bool:
    return _data.has(id)


func lines(id: String) -> Array:
    if not _data.has(id):
        push_warning("No dialogue for '%s'" % id)
        return []
    var entry: Dictionary = _data[id]
    var variants: Array = entry["variants"]
    var n: int = _seen.get(id, 0)
    _seen[id] = n + 1
    var pick: Array = variants[n] if n < variants.size() else variants[randi_range(1 if variants.size() > 1 else 0, var
iants.size() - 1)]
    var out := []
    for l in pick:
        var who: String = l["who"]
        var line := {"s": who, "t": l["text"], "e": l.get("e", "")}
        if entry.get("rekaan", false) and Game.characters.get(who, {}).get("tokoh", false):
            line["rekaan"] = true
        out.append(line)
    return out
```

game_state.gd · scripts/game_state.gd · 83 lines

```
extends Node
## Autoload: scene transitions (fade), warp spawn point and session flags.
## Saved progress lives in Game.save (scripts/autoload/game.gd).

const CHAPTERS := [
    "Warisan Negara Bangsa",
    "Kebangkitan Nasionalisme",
    "Konflik Dunia dan Pendudukan Jepun",
    "Era Peralihan Kuasa British",
    "Persekutuan Tanah Melayu 1948",
    "Ancaman Komunis dan Darurat",
    "Usaha ke Arah Kemerdekaan",
    "Pilihan Raya",
    "Perlembagaan Persekutuan 1957",
    "Pemasyhuran Kemerdekaan",
]

## how many flags can be entered: chapters finished so far + the one in play
var unlocked_chapters: int:
    get:
        return Game.current_chapter()
var flags := {}              # session flags, e.g. flags["bilik_intro"] = true
var spawn_cell = null        # Vector2i to place Alif at after a warp
var spawn_facing := "south"

var _fade: ColorRect
var _card: VBoxContainer
var _card_title: Label
var _card_sub: Label
```

```
30
31
32  func _ready() -> void:
33      var layer := CanvasLayer.new()
34      layer.layer = 100
35      add_child(layer)
36      _fade = ColorRect.new()
37      _fade.color = Color(0, 0, 0, 0)
38      _fade.mouse_filter = Control.MOUSE_FILTER_IGNORE
39      _fade.set_anchors_preset(Control.PRESET_FULL_RECT)
40      layer.add_child(_fade)
41      # title card shown on the black between two maps ("Lembah Kebangkitan / 1900-an")
42      _card = VBoxContainer.new()
43      _card.set_anchors_preset(Control.PRESET_FULL_RECT)
44      _card.alignment = BoxContainer.ALIGNMENT_CENTER
45      _card.mouse_filter = Control.MOUSE_FILTER_IGNORE
46      _card.modulate.a = 0.0
47      _card_title = Label.new()
48      _card_title.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
49      _card_title.add_theme_font_size_override("font_size", 52)
50      _card_title.add_theme_color_override("font_color", Color("ffe39a"))
51      _card.add_child(_card_title)
52      _card_sub = Label.new()
53      _card_sub.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
54      _card_sub.add_theme_font_size_override("font_size", 26)
55      _card_sub.add_theme_color_override("font_color", Color(1, 1, 1, 0.85))
56      _card.add_child(_card_sub)
57      layer.add_child(_card)
58
59
60  func go_to(scene_path: String, color := Color.BLACK, title := "", subtitle := "") -> void:
61      _fade.color = Color(color, _fade.color.a)
62      _fade.mouse_filter = Control.MOUSE_FILTER_STOP
63      var tween := create_tween()
64      tween.tween_property(_fade, "color:a", 1.0, 0.4 if title == "" else 0.6)
65      await tween.finished
66      if title != "":
67          _card_title.text = title
68          _card_sub.text = subtitle
69          _card_title.theme = Game.theme
70          _card_sub.theme = Game.theme
71          var light := color.get_luminance() > 0.5
72          _card_title.add_theme_color_override("font_color", Color("5a381e") if light else Color("ffe39a"))
73          _card_sub.add_theme_color_override("font_color", Color("5a381e") if light else Color(1, 1, 1, 0.85))
74          tween = create_tween()
75          tween.tween_property(_card, "modulate:a", 1.0, 0.4)
76          tween.tween_interval(1.3)
77          tween.tween_property(_card, "modulate:a", 0.0, 0.4)
78          await tween.finished
79      get_tree().change_scene_to_file(scene_path)
80      tween = create_tween()
81      tween.tween_property(_fade, "color:a", 0.0, 0.4)
82      await tween.finished
83      _fade.mouse_filter = Control.MOUSE_FILTER_IGNORE
```

`main.gd` · scripts/main.gd · 286 lines

```
1   extends Node
2   ## Autoload "Main": global UI layers that live above every scene (from the
3   ## first prototype's root node): dialogue (20), Pau button + hint bubble (25)
4   ## and overlays (30: settings, Nota Alif, Beg Alif, calm moment, toasts).
5   ## Game.main points here, so Game.say(), pau_say(), toast() work anywhere.
6
7   const UI = preload("res://scripts/ui/ui.gd")
8   const DialogueBox = preload("res://scripts/ui/dialogue_box.gd")
9   const PauButton = preload("res://scripts/ui/pau_button.gd")
10  const SettingsPanel = preload("res://scripts/ui/settings_panel.gd")
11  const NotaPanel = preload("res://scripts/ui/nota_panel.gd")
12  const BegPanel = preload("res://scripts/ui/beg_panel.gd")
13  const CalmOverlay = preload("res://scripts/ui/calm_overlay.gd")
14  const MapOverlay = preload("res://scripts/ui/map_overlay.gd")
15  const ProfilePanel = preload("res://scripts/ui/profile_panel.gd")
16  const PapanPanel = preload("res://scripts/ui/papan_panel.gd")
17
18  var dialog_layer: CanvasLayer
19  var pau_layer: CanvasLayer
20  var overlay_layer: CanvasLayer
21  var dialog_box: Control
22  var pau_button: Control
23  var bubble: PanelContainer
24  var bubble_label: Label
25  var toast_box: VBoxContainer
26  var _bubble_timer: Timer
```

```

func _ready() -> void:
    Game.main = self
    get_tree().root.theme = Game.theme  # dyslexia-friendly fonts everywhere
    dialog_layer = _layer(20)
    pau_layer = _layer(25)
    overlay_layer = _layer(30)

    dialog_box = DialogueBox.new()
    dialog_box.theme = Game.theme
    dialog_layer.add_child(dialog_box)

    var pau_root := Control.new()
    UI.full(pau_root)
    pau_root.mouse_filter = Control.MOUSE_FILTER_IGNORE
    pau_root.theme = Game.theme
    pau_layer.add_child(pau_root)
    pau_button = PauButton.new()
    pau_root.add_child(pau_button)
    pau_button.tapped.connect(_on_pau_tap)
    pau_button.held.connect(open_calm)
    bubble = UI.panel("Bubble")
    bubble.visible = false
    bubble.custom_minimum_size = Vector2(420, 0)
    var bb := UI.hbox(10)
    bubble_label = UI.label("", "Ink", true)
    bubble_label.custom_minimum_size.x = 340
    bubble_label.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    bb.add_child(bubble_label)
    var speak := UI.icon_button("res://assets/icons/suara.png", "Blue", Vector2(60, 56))
    speak.tooltip_text = "Baca dengan suara"
    speak.pressed.connect(func(): Tts.speak(bubble_label.text))
    bb.add_child(speak)
    bubble.add_child(bb)
    bubble.gui_input.connect(func(e: InputEvent):
        if e is InputEventMouseButton and e.pressed:
            bubble.visible = false)
    pau_root.add_child(bubble)
    _bubble_timer = Timer.new()
    _bubble_timer.one_shot = true
    _bubble_timer.timeout.connect(func(): bubble.visible = false)
    add_child(_bubble_timer)

    var ov := Control.new()
    UI.full(ov)
    ov.mouse_filter = Control.MOUSE_FILTER_IGNORE
    ov.theme = Game.theme
    overlay_layer.add_child(ov)
    toast_box = UI.vbox(8)
    toast_box.position = Vector2(390, 96)
    toast_box.custom_minimum_size = Vector2(500, 0)
    toast_box.mouse_filter = Control.MOUSE_FILTER_IGNORE
    ov.add_child(toast_box)

    _place_pau()
    set_pau_visible(false)
    Game.settings_changed.connect(func():
        get_tree().root.theme = Game.theme
        _place_pau())
    if "--autotest" in OS.get_cmdline_user_args():
        add_child(load("res://scripts/dev/autotest.gd").new())


## with on-screen controls (Tetapan > Kawalan) the bottom corners belong to
## the thumbs, so Pau moves up (left side for left-handed: the pad is right)
func _place_pau() -> void:
    if not Game.controls_on_screen():
        pau_button.position = Vector2(1280 - 130, 720 - 140)
    elif Game.settings.get("left_handed", false):
        pau_button.position = Vector2(16, 166)
    else:
        pau_button.position = Vector2(1280 - 130, 150)


func _layer(n: int) -> CanvasLayer:
    var l := CanvasLayer.new()
    l.layer = n
    add_child(l)
    return l


func overlay_root() -> Control:
    return overlay_layer.get_child(0) as Control

```

```
112
113 # ---------------------------------------------------------------- dialogue
114 func dialogue(key_or_lines: Variant) -> void:
115     var arr: Array = []
116     if key_or_lines is String:
117         arr = Game.lines(key_or_lines)
118         if not Game.save["seen"].has(key_or_lines):
119             Game.save["seen"].append(key_or_lines)
120     elif key_or_lines is Array:
121         arr = key_or_lines
122     if arr.is_empty():
123         return
124     for l in arr:
125         Game.add_figure(str(l.get("s", "")))
126     Game.lock_input()
127     MusicDirector.duck(true)
128     await dialog_box.play(arr)
129     MusicDirector.duck(false)
130     Game.unlock_input()
131
132
133 func is_dialogue_open() -> bool:
134     return dialog_box.visible
135
136
137 # ---------------------------------------------------------------- Pau
138 func set_pau_visible(v: bool) -> void:
139     pau_button.visible = v
140     bubble.visible = false  # every screen calls this on entry: no leftover bubble
141
142
143 func pau_say(text: String, seconds: float = 7.0) -> void:
144     pau_button.happy()
145     Sfx.play("pop", -8.0, 1.3)
146     if Game.active_puzzle != null and is_instance_valid(Game.active_puzzle) and Game.active_puzzle.has_method("show_pau
    _hint"):
147         Game.active_puzzle.show_pau_hint(text)
148         return
149     bubble_label.text = text
150     bubble.visible = true
151     bubble.reset_size()
152     var bx := pau_button.size.x + 20.0 if pau_button.position.x < 640 else -20.0 - bubble.size.x  # Pau on the left: bu
    bble opens right
153     bubble.position = pau_button.position + Vector2(bx, 10 - (bubble.size.y if pau_button.position.y > 360 else 0.0))
154     _bubble_timer.start(seconds)
155     if bool(Game.settings.get("auto_read", false)):
156         Tts.speak(text)
157
158
159 func _on_pau_tap() -> void:
160     Game.pau_pressed.emit()
161     if Game.active_puzzle != null and is_instance_valid(Game.active_puzzle):
162         Game.active_puzzle.call("request_hint")
163         return
164     if is_dialogue_open():
165         return
166     var t := Game.context_hint
167     if t == "":
168         t = "Aku kat sini! Ketik aku untuk petunjuk. Tekan lama kalau nak bertenang."
169     pau_say(t)
170
171
172 func open_calm() -> void:
173     overlay_root().add_child(CalmOverlay.new())
174
175
176 # ---------------------------------------------------------------- overlays
177 func open_settings() -> Control:
178     var p := SettingsPanel.new()
179     overlay_root().add_child(p)
180     return p
181
182
183 ## zone: open on that mission's first page (e.g. "1_3"), else the newest Bab
184 func open_nota(zone: String = "") -> Control:
185     var p := NotaPanel.new()
186     p.start_zone = zone
187     overlay_root().add_child(p)
188     return p
189
190
191 ## A yes/no card over everything. Returns true for yes.
192 func ask(text: String, yes_text: String = "Ya", no_text: String = "Tak") -> bool:
193     Game.lock_input()
194     var root := Control.new()
```

```
    UI.full(root)
    root.theme = Game.theme
    root.mouse_filter = Control.MOUSE_FILTER_STOP
    root.add_child(UI.dim(0.5))
    var card := UI.panel("Paper")
    card.custom_minimum_size = Vector2(620, 0)
    var col := UI.vbox(16)
    var l := UI.label(tr(text), "Ink", true)
    l.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    col.add_child(l)
    var row := UI.hbox(16)
    row.alignment = BoxContainer.ALIGNMENT_CENTER
    var no := UI.button(tr(no_text), "", 200)
    var yes := UI.button(tr(yes_text), "Primary", 200)
    row.add_child(no)
    row.add_child(yes)
    col.add_child(row)
    card.add_child(col)
    root.add_child(UI.center(card))
    overlay_root().add_child(root)
    yes.grab_focus()
    var pick := [false]
    var done := func(v: bool):
        pick[0] = v
        root.set_meta("done", true)
    yes.pressed.connect(done.bind(true))
    no.pressed.connect(done.bind(false))
    while is_instance_valid(root) and not root.has_meta("done"):
        await get_tree().process_frame
    Sfx.play("select", -6.0)
    root.queue_free()
    Game.unlock_input()
    return pick[0]


## Beg Alif: the artifacts found so far and the Kad Tokoh; with the world
## given, also the Jalan Pintas between places
func open_beg(world: Node = null) -> Control:
    var p := BegPanel.new()
    p.world = world
    overlay_root().add_child(p)
    return p


## Profil & Kedai (tab 0 = profile, 1 = skin shop)
func open_profile(tab: int = 0) -> Control:
    var p := ProfilePanel.new()
    p.start_tab = tab
    overlay_root().add_child(p)
    return p


## Papan Markah (leaderboard): Seluruh / Negeri / Rakan.
func open_papan(tab: int = 0) -> Control:
    var p := PapanPanel.new()
    p.start_tab = tab
    overlay_root().add_child(p)
    return p


## Full world map with legend (world scene only).
func open_map(world: Node) -> Control:
    var p := MapOverlay.new()
    p.world = world
    overlay_root().add_child(p)
    return p


func toast(text: String, color: Color = UI.GOLD) -> void:
    var p := UI.panel()
    p.mouse_filter = Control.MOUSE_FILTER_IGNORE
    var l := UI.label(text, "", true)
    l.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    l.add_theme_color_override("font_color", color)
    p.add_child(l)
    toast_box.add_child(p)
    p.modulate.a = 0.0
    var tw := create_tween()
    tw.tween_property(p, "modulate:a", 1.0, 0.2)
    tw.tween_interval(2.4)
    tw.tween_property(p, "modulate:a", 0.0, 0.4)
    tw.tween_callback(p.queue_free)


func shake(strength: float = 8.0, duration: float = 0.4) -> void:
```

```
    var cam := get_viewport().get_camera_2d()
    if cam == null:
        return
    var tw := create_tween()
    for i in int(duration / 0.04):
        tw.tween_property(cam, "offset", Vector2(randf_range(-strength, strength), randf_range(-strength, strength)) / cam.zoom.x, 0.04)
    tw.tween_property(cam, "offset", Vector2.ZERO, 0.04)
```

pau.gd · scripts/pau.gd · 62 lines

```
class_name Pau
extends Node2D
## Pau follows Alif one tile behind (HG/SS style): whenever Alif starts a
## step, Pau hops into the tile Alif just left. Squishy bounce on every hop.

const TEX := preload("res://assets/characters32/pau/pau_small.png")

var cell := Vector2i.ZERO
var _t := 0.0
var _hop := 0.0
var _squash := 0.0

@onready var sprite: Sprite2D = $Sprite


func _ready() -> void:
    sprite.texture = TEX
    sprite.offset = CharacterFrames.visible_bottom_offset(TEX)


func follow(player: Player) -> void:
    player.step_started.connect(_on_leader_step)


func place_at(c: Vector2i) -> void:
    cell = c
    position = Player.cell_to_pos(c)


func _on_leader_step(from: Vector2i, _to: Vector2i, duration: float) -> void:
    if from == cell:
        return
    var dir := Vector2(from - cell)
    cell = from
    sprite.flip_h = dir.x < 0
    var tween := create_tween().set_parallel()
    tween.tween_property(self, "position", Player.cell_to_pos(from), duration)
    tween.tween_method(func(v: float): _hop = sin(v * PI) * 5.0, 0.0, 1.0, duration)
    tween.chain().tween_property(self, "_squash", 1.0, 0.0)
    tween.chain().tween_property(self, "_squash", 0.0, 0.18)


func _process(delta: float) -> void:
    _t += delta
    var idle := absf(sin(_t * 3.0)) * 1.5  # gentle breathing bounce when still
    sprite.position.y = -_hop - idle
    var s := _squash * 0.25 + (1.0 - absf(sin(_t * 3.0))) * 0.05
    sprite.scale = Vector2(1.0 + s, 1.0 - s)


## happy hop when tapped or talked to
func hop() -> void:
    var tween := create_tween()
    tween.tween_property(self, "_hop", 9.0, 0.12).set_ease(Tween.EASE_OUT)
    tween.tween_property(self, "_hop", 0.0, 0.2).set_ease(Tween.EASE_IN)
    tween.tween_property(self, "_squash", 1.0, 0.0)
    tween.tween_property(self, "_squash", 0.0, 0.18)


func _draw() -> void:
    draw_set_transform(Vector2(0, -1), 0, Vector2(1, 0.45))
    draw_circle(Vector2.ZERO, 5, Color(0, 0, 0, 0.25))
```

player.gd · scripts/player.gd · 180 lines

```
class_name Player
extends Node2D
## Alif on the overworld grid, Pokémon-style: one 16 px tile per step,
## a quick tap only turns him, hold B (Shift / X) to run.

signal step_started(from: Vector2i, to: Vector2i, duration: float)
signal step_finished(cell: Vector2i)
signal bumped(cell: Vector2i)

const TILE := 16
const WALK_TIME := 0.22
const RUN_TIME := 0.13
const TURN_GRACE := 0.09  ## a tap shorter than this only turns
const DIRS := {
    "south": Vector2i(0, 1), "north": Vector2i(0, -1),
    "east": Vector2i(1, 0), "west": Vector2i(-1, 0),
}

var cell := Vector2i.ZERO
var facing := "south"
var can_move := true
var moving := false
var world: Node  ## must provide is_walkable(cell) -> bool

var _hold_time := 0.0
var _held_dir := ""
var _turned := false
var _bump_cooldown := 0.0
var _path: Array = []
var _on_arrive: Callable
var _look := ""  ## set_look(): a sprite folder worn instead of the outfit

@onready var sprite: AnimatedSprite2D = $Sprite


func _ready() -> void:
    _apply_skin()
    Game.skin_changed.connect(_apply_skin)
    _idle()


## Alif's outfit from the shop (Game.save.skin), unless a scene dresses him
## for a moment (set_look: Bilik Alif before he picks up his bag)
func _apply_skin() -> void:
    var folder := Game.skin_folder()
    if _look != "" and ResourceLoader.exists("%s%s/%s_south.png" % [CharacterFrames.BASE, _look, _look]):
        folder = _look
    sprite.sprite_frames = CharacterFrames.build(folder, true)
    sprite.offset = CharacterFrames.feet_offset(folder)
    sprite.play("idle_" + facing)


## wear this sprite folder instead of the shop outfit ("" = back to the outfit)
func set_look(folder: String) -> void:
    _look = folder
    if sprite:
        _apply_skin()


func place_at(c: Vector2i, face := "south") -> void:
    cell = c
    position = cell_to_pos(c)
    facing = face
    _idle()


static func cell_to_pos(c: Vector2i) -> Vector2:
    return Vector2(c.x * TILE + TILE / 2, c.y * TILE + TILE - 1)


func facing_cell() -> Vector2i:
    return cell + DIRS[facing]


func face_cell(c: Vector2i) -> void:
    var d := c - cell
    if absi(d.x) > absi(d.y):
        facing = "east" if d.x > 0 else "west"
    else:
        facing = "south" if d.y > 0 else "north"
    _idle()


```

```
## tap-to-walk: follow a list of cells, then call on_arrive
func walk_path(path: Array, on_arrive := Callable()) -> void:
    _path = path.duplicate()
    _on_arrive = on_arrive


func _process(delta: float) -> void:
    _bump_cooldown = maxf(_bump_cooldown - delta, 0)
    if moving or not can_move or Game.is_locked():
        return
    var dir := _input_dir()
    if dir != "":
        _path.clear()  # keyboard / D-pad always wins over a tap path
    elif not _path.is_empty():
        var next: Vector2i = _path.pop_front()
        var d := next - cell
        var pdir := "east" if d.x > 0 else "west" if d.x < 0 else "south" if d.y > 0 else "north"
        facing = pdir
        if world and not world.is_walkable(next):
            _path.clear()
            _idle()
            return
        _try_step(pdir)
        if _path.is_empty() and _on_arrive.is_valid():
            var cb := _on_arrive
            _on_arrive = Callable()
            await step_finished
            cb.call()
        return
    if dir == "":
        _held_dir = ""
        _hold_time = 0.0
        _idle()
        return
    if dir != _held_dir:  # new press
        _held_dir = dir
        _hold_time = 0.0
        _turned = dir != facing
        facing = dir
        _idle()
    _hold_time += delta
    if _turned and _hold_time < TURN_GRACE:
        return  # tap = turn on the spot
    _try_step(dir)


func _input_dir() -> String:
    # last-pressed wins would be nicer; vertical first is the classic feel
    if Input.is_action_pressed("move_up"): return "north"
    if Input.is_action_pressed("move_down"): return "south"
    if Input.is_action_pressed("move_left"): return "west"
    if Input.is_action_pressed("move_right"): return "east"
    return ""


func _try_step(dir: String) -> void:
    var target: Vector2i = cell + DIRS[dir]
    if world and not world.is_walkable(target):
        _idle()
        if _bump_cooldown <= 0:
            _bump_cooldown = 0.35
            bumped.emit(target)
            _bump_wobble()
        return
    var running := Input.is_action_pressed("run")
    var t := RUN_TIME if running else WALK_TIME
    var from := cell
    cell = target
    moving = true
    sprite.play("walk_" + dir)
    sprite.speed_scale = WALK_TIME / t
    step_started.emit(from, target, t)
    var tween := create_tween()
    tween.tween_property(self, "position", cell_to_pos(target), t)
    await tween.finished
    moving = false
    step_finished.emit(cell)
    if _input_dir() == "" and _path.is_empty():
        _idle()


func _idle() -> void:
    var anim := "idle_" + facing
    if sprite.animation != anim:
        sprite.play(anim)
```

```

func _bump_wobble() -> void:
    var tween := create_tween()
    var nudge := Vector2(DIRS[facing]) * 1.5
    tween.tween_property(sprite, "position", nudge, 0.06)
    tween.tween_property(sprite, "position", Vector2.ZERO, 0.08)


func _draw() -> void:
    draw_set_transform(Vector2(0, -1), 0, Vector2(1, 0.45))
    draw_circle(Vector2.ZERO, 6, Color(0, 0, 0, 0.28))
```

`prologue.gd` · scripts/prologue.gd · 648 lines

```
extends Control
## Prologue (GDS section 2): a cutscene, nothing to press (lines move on by
## themselves, Langkau skips it all), told so players see their own nights:
##  1. cold open: the class group chat at 11:47 PM (scripts/ui/phone_chat.gd)
##  2. Alif's room: he looks at the calendar, his trial paper, Mak's message,
##     the window (Tok's stories) and Pau, each glowing in turn
##  3. reading: key words fly into "Ingatan Alif", but it only holds two and
##     the letters keep scrambling: "satu pun tak melekat" (dyslexia, and
##     anyone drowning in facts)
##  4. he dozes off; golden pages carry us into a dream: his classmates at an
##     old port forget each other in the Kabus Lupa, then Gua Warisan where
##     Nirbana breaks the Telaga Batu inscription into four Serpihan (the goal
##     of Bab 1). Then Alif wakes up in his room in Dunia Explo-Story, where
##     Pau explains everything (scripts/world/bilik_alif.gd).
## Text: data/prologue_room.json and data/dialogue/prologue.json.

const UI = preload("res://scripts/ui/ui.gd")
const PhoneChat = preload("res://scripts/ui/phone_chat.gd")
const BILIK := "res://scenes/interior/bilik_alif.tscn"
const DATA := "res://data/prologue_room.json"
## the Malaysian remake of Alif's night room (PixelLab, same layout as the old
## picture so the glow rects in prologue_room.json still fit); old one until then
const ROOM_V2 := "res://assets/backgrounds/bg_bilik_alif_malam_v2.png"
const MIRROR := {"b": "d", "d": "b", "p": "q", "q": "p", "m": "w", "w": "m", "n": "u", "u": "n"}
const FRIENDS := ["aisyah", "ravi", "meiling", "daniel"]

var phase := ""  ## what is on screen: phone, room, reading, doze, port, gua

var _data: Dictionary
var _skipping := false
var _tick_gap := 1.0
var _ticking := false
var _heart_gap := 0.0  ## 0 = no heartbeat
var _tokens: Array = []  ## paragraph pieces {text, kw}
var _memory: Array = []  ## key words Alif still remembers (token index), oldest first
var _words_read := 0
var _amount := 0.0       ## how scrambled the page is, 0..1
var _jar: HBoxContainer

@onready var room: TextureRect = $Room
@onready var lamp: ColorRect = $LampGlow
@onready var vignette: TextureRect = $Vignette
@onready var caption: Label = $Caption
@onready var book: PanelContainer = $Book
@onready var book_text: RichTextLabel = $Book/Text
@onready var pau: TextureRect = $Pau
@onready var lid_top: ColorRect = $LidTop
@onready var lid_bottom: ColorRect = $LidBottom
@onready var pages: CPUParticles2D = $Pages
@onready var flash: ColorRect = $Flash
@onready var skip: Button = $SkipLayer/Skip


func _ready() -> void:
    Main.set_pau_visible(false)
    if ResourceLoader.exists(ROOM_V2):
        room.texture = load(ROOM_V2)
    room.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    MusicDirector.play("bilik", 2.5)
    skip.pressed.connect(_skip)
    pages.texture = _page_texture()
    _data = Game.load_json(DATA)
    Main.dialog_box.auto_advance = true
    book.modulate.a = 0.0
    caption.modulate.a = 0.0
    _run()


```

```
# --- the sequence ------------------------------------------------------------
func _run() -> void:
    _lamp_loop()
    _desk_phone()
    await _phone_open()
    if _skipping: return
    _clock_loop()
    _heart_loop()
    caption.text = str(_data.get("caption", ""))
    await _fade(caption, 1.0, 1.0)
    await _wait(1.6)
    await _fade(caption, 0.0, 0.8)
    await _room_tour()
    await _reading()
    await _doze()
    await _dream()
    if _skipping: return
    await _fade(flash, 1.0, 0.8)
    _go_to_world()


## 11:47 PM, the class group chat: everyone is as lost as Alif
func _phone_open() -> void:
    phase = "phone"
    var chat := PhoneChat.new()
    add_child(chat)
    move_child(chat, flash.get_index())
    chat.setup(_data.get("chat", {}))
    await chat.done
    if _skipping: return
    var t := create_tween()
    t.tween_property(chat, "modulate:a", 0.0, 0.6)
    await t.finished
    chat.queue_free()


## the room picture has no phone: one lies on the desk, screen still glowing
func _desk_phone() -> void:
    var p := Panel.new()
    p.add_theme_stylebox_override("panel", UI.sb(Color("1b1d24"), Color("3a3d48"), 2, 4, 0))
    p.position = Vector2(318, 444)
    p.size = Vector2(54, 22)
    p.rotation = -0.08
    p.mouse_filter = Control.MOUSE_FILTER_IGNORE
    var screen := ColorRect.new()
    screen.color = Color("8fd8ff")
    screen.position = Vector2(5, 4)
    screen.size = Vector2(44, 14)
    p.add_child(screen)
    add_child(p)
    move_child(p, vignette.get_index())
    var t := screen.create_tween().set_loops()
    t.tween_property(screen, "modulate:a", 0.45, 1.2).set_trans(Tween.TRANS_SINE)
    t.tween_property(screen, "modulate:a", 1.0, 1.2).set_trans(Tween.TRANS_SINE)


## Alif looks around his room: each thing glows while he talks about it
func _room_tour() -> void:
    if _skipping: return
    phase = "room"
    await _say("pro_room")
    for s in _data.get("room", []):
        if _skipping: return
        var glow := _glow(s)
        await _wait(0.6)
        _spot_fx(str(s["id"]))
        await _say(str(s.get("say", "")))
        if _skipping: return
        var t := create_tween()
        t.tween_property(glow, "modulate:a", 0.0, 0.3)
        t.tween_callback(glow.queue_free)
        await _wait(0.4)


## a soft gold outline (and its name) around the thing Alif is looking at
func _glow(s: Dictionary) -> Control:
    var r: Array = s["rect"]
    var p := Panel.new()
    p.add_theme_stylebox_override("panel", UI.sb(Color(UI.GOLD, 0.12), UI.GOLD, 4, 14, 0))
    p.position = Vector2(r[0], r[1])
    p.size = Vector2(r[2], r[3])
    p.mouse_filter = Control.MOUSE_FILTER_IGNORE
    var tag := UI.label(str(s.get("label", "")))
    tag.add_theme_color_override("font_outline_color", Color(0.1, 0.06, 0.15))
    tag.add_theme_constant_override("outline_size", 8)
```

```
    tag.position = Vector2(4, -36)
    p.add_child(tag)
    add_child(p)
    move_child(p, flash.get_index())
    p.modulate.a = 0.0
    create_tween().tween_property(p, "modulate:a", 1.0, 0.3)
    var pulse := p.create_tween().set_loops()
    pulse.tween_property(p, "self_modulate:a", 0.5, 0.6).set_trans(Tween.TRANS_SINE)
    pulse.tween_property(p, "self_modulate:a", 1.0, 0.6).set_trans(Tween.TRANS_SINE)
    return p


func _spot_fx(id: String) -> void:
    match id:
        "pau":
            _squish_pau()
        "kalendar":
            _heart_gap = 1.4  # and it stays with him
            Sfx.play("heartbeat", -2.0)
        "telefon":
            Sfx.play("pop", -4.0)
        _:
            Sfx.play("select", -6.0)


## Reading: key words fly into Alif's memory, but it only holds two and the
## page keeps scrambling, so they fall out again as fast as they go in
func _reading() -> void:
    if _skipping: return
    phase = "reading"
    _tokens = _parse(str(_data.get("paragraph", "")))
    _render_book()
    var jar := _build_jar()
    await _fade(book, 1.0, 0.8)
    await _say("pro_read")
    _tick_gap = 0.7
    _heart_gap = 1.1
    var keys: Array = []
    for i in _tokens.size():
        if _tokens[i]["kw"]:
            keys.append(i)
    var words := mini(int(_data.get("read_words", 6)), keys.size())
    var every := float(_data.get("word_every", 1.3))
    var thoughts: Dictionary = _data.get("thoughts", {})
    for n in words:
        if _skipping: return
        _remember(keys[n])
        for k in 4:  # the letters keep moving while he reads
            _amount = minf(1.0, 0.1 + n * 0.12 + k * 0.03)
            _render_book()
            await _wait(every / 4.0)
        if thoughts.has(str(n + 1)):
            await _say(str(thoughts[str(n + 1)]))
    if _skipping: return

    # everything slides off the page
    while not _memory.is_empty():
        _forget(_memory.pop_front())
    _amount = 1.0
    _render_book()
    _tick_gap = 0.45
    _heart_gap = 0.7
    _shake_room(true)
    await _say("pro_scramble")
    _shake_room(false)
    _fade(jar, 0.0, 0.6)
    await _fade(book, 0.0, 0.6)
    jar.queue_free()


func _parse(p: String) -> Array:
    var out := []
    var re := RegEx.new()
    re.compile("\\{([^}]*)\\}")
    var at := 0
    for m in re.search_all(p):
        if m.get_start() > at:
            out.append({"text": p.substr(at, m.get_start() - at), "kw": false})
        out.append({"text": m.get_string(1), "kw": true})
        at = m.get_end()
    if at < p.length():
        out.append({"text": p.substr(at), "kw": false})
    return out


```

```
func _render_book() -> void:
    var out := ""
    for i in _tokens.size():
        var tk: Dictionary = _tokens[i]
        if not tk["kw"]:
            out += _scramble(tk["text"], _amount * 0.6)
        elif _memory.has(i):
            out += "[color=#c9a040]%s[/color]" % tk["text"]
        else:
            out += "[u][color=#a0521d]%s[/color][/u]" % _scramble(tk["text"], _amount * 0.4)
    book_text.text = "[shake rate=%d level=%d]%s[/shake]" % [10 + int(_amount * 16), int(_amount * 8), out]


func _build_jar() -> PanelContainer:
    var p := UI.panel("Paper")
    p.position = Vector2(200, 430)
    p.size = Vector2(880, 0)
    p.mouse_filter = Control.MOUSE_FILTER_IGNORE
    _jar = UI.hbox(10)
    _jar.add_child(UI.label("Ingatan Alif:", "InkHeading"))
    p.add_child(_jar)
    add_child(p)
    move_child(p, flash.get_index())
    p.modulate.a = 0.0
    _fade(p, 1.0, 0.8)
    return p


## the word flies from the page into Alif's memory
func _remember(i: int) -> void:
    _words_read += 1
    _memory.append(i)
    Sfx.play("chime", -8.0, 1.0 + _memory.size() * 0.1)
    var word := str(_tokens[i]["text"])
    var chip := UI.panel()
    chip.add_theme_stylebox_override("panel", UI.sb(Color("fff1c4"), UI.GOLD, 2, 8, 6))
    chip.set_meta("word", i)
    chip.add_child(UI.label(word, "Ink"))
    chip.modulate.a = 0.0
    _jar.add_child(chip)
    var ghost := UI.label(word, "Title")
    ghost.position = book.global_position + Vector2(randf_range(60, book.size.x - 220), randf_range(40, book.size.y - 80))
    add_child(ghost)
    var t := create_tween()
    t.tween_property(ghost, "position", _jar.global_position + Vector2(180 + _jar.get_child_count() * 60, 0), 0.4).set_ease(Tween.EASE_IN)
    t.parallel().tween_property(ghost, "modulate:a", 0.0, 0.4)
    t.tween_callback(ghost.queue_free)
    t.tween_property(chip, "modulate:a", 1.0, 0.2)
    if _memory.size() > int(_data.get("memory_size", 2)):
        _forget(_memory.pop_front())


## ...and the oldest one falls out again
func _forget(i: int) -> void:
    for chip in _jar.get_children():
        if chip.has_meta("word") and int(chip.get_meta("word")) == i:
            chip.reparent(self)
            var t := create_tween().set_parallel()
            t.tween_property(chip, "position:y", chip.position.y + 90, 0.7).set_ease(Tween.EASE_IN)
            t.tween_property(chip, "rotation", randf_range(-0.6, 0.6), 0.7)
            t.tween_property(chip, "modulate:a", 0.0, 0.7)
            t.chain().tween_callback(chip.queue_free)
            Sfx.play("swish", -6.0)
            return


## Alif gives up, squeezes Pau and dozes off; golden pages carry him away
func _doze() -> void:
    if _skipping: return
    phase = "doze"
    for i in 2:
        _squish_pau(1.0 + i * 0.1)
        await _wait(0.5)
    _tick_gap = 0.9
    _heart_gap = 1.4
    await _say("pro_pau")

    # drowsy: the clock slows, eyelids close... silence
    _tick_gap = 1.6
    _heart_gap = 0.0
    await _blink(0.55, 0.7)
    await _blink(0.85, 1.0)
    _ticking = false
```

```
    await _lids(1.0, 1.4)
    await _wait(1.2)

    if _skipping: return
    MusicDirector.stop(2.0)
    Sfx.play("whoosh")
    Sfx.play("chime", -2.0)
    pages.emitting = true
    var t2 := create_tween()
    t2.tween_property(lid_top, "color", Color(1, 0.93, 0.7), 1.2)
    t2.parallel().tween_property(lid_bottom, "color", Color(1, 0.93, 0.7), 1.2)
    await _wait(1.6)
    Sfx.play("sparkle")
    await _fade(flash, 1.0, 0.8)


func _squish_pau(pitch := 1.0) -> void:
    Sfx.play("squish", -4.0, pitch)
    var t := create_tween()
    t.tween_property(pau, "scale", Vector2(1.25, 0.75), 0.12)
    t.tween_property(pau, "scale", Vector2(1, 1), 0.25).set_trans(Tween.TRANS_ELASTIC).set_ease(Tween.EASE_OUT)


## The dream: his classmates at an old port forget each other in the Kabus
## Lupa, then the cave, Nirbana and the four Serpihan.
func _dream() -> void:
    if _skipping: return
    phase = "port"
    var bg := _scene_image("res://assets/backgrounds/bg_pelabuhan_huru.png")
    bg.modulate = Color(1.0, 0.75, 0.7)
    var alif := _sprite("res://assets/characters32/%s/%s_south.png" % [Game.skin_folder(), Game.skin_folder()], Vector2
(640, 610), 6)
    var feet := [Vector2(250, 560), Vector2(430, 650), Vector2(850, 650), Vector2(1030, 560)]
    var pals: Array[TextureRect] = []
    for i in FRIENDS.size():
        pals.append(_sprite(_rakan(FRIENDS[i], "south"), feet[i], 5))
    MusicDirector.play("siguntang", 1.0)
    flash.color = Color(1, 1, 1)
    await _fade(flash, 0.0, 0.8)
    room.position = Vector2.ZERO
    await _say("pro_friends_1")
    if _skipping: return

    # the Kabus Lupa rolls in: they turn away from him...
    var fog := ColorRect.new()
    fog.color = Color(0.35, 0.15, 0.5, 0.0)
    fog.mouse_filter = Control.MOUSE_FILTER_IGNORE
    fog.size = Vector2(1280, 720)
    add_child(fog)
    move_child(fog, bg.get_index() + 1)
    Sfx.play("whoosh", -4.0, 0.7)
    var k := create_tween().set_parallel()
    k.tween_property(fog, "color:a", 0.4, 1.4)
    for i in pals.size():
        k.tween_property(pals[i], "modulate", Color(0.8, 0.65, 1.0), 1.4)
        pals[i].texture = UI.tex(_rakan(FRIENDS[i], "north"))
    await k.finished
    await _say("pro_friends_2")
    if _skipping: return
    # ...then on each other
    var faces := ["east", "west", "east", "west"]
    for i in pals.size():
        pals[i].texture = UI.tex(_rakan(FRIENDS[i], faces[i]))
        _hop(pals[i])
    Sfx.play("bump", -4.0)
    await _say("pro_friends_3")
    if _skipping: return
    _shake_node(bg, 1.6)
    Sfx.play("bump", -2.0)
    await _say("pro_huru")
    if _skipping: return

    # run into the cave
    var run := create_tween()
    run.tween_property(alif, "position:x", 1300.0, 0.7).set_ease(Tween.EASE_IN)
    Sfx.play("whoosh")
    flash.color = Color.BLACK
    await _fade(flash, 1.0, 0.6)
    if run.is_running():
        await run.finished
    phase = "gua"
    for p in pals:
        p.queue_free()
    fog.queue_free()
    bg.texture = UI.tex("res://assets/backgrounds/bg_gua_warisan.png")
```

```
    bg.modulate = Color.WHITE
    alif.position = Vector2(300 - alif.size.x / 2.0, 640 - alif.size.y)
    alif.texture = UI.tex("res://assets/characters32/%s/%s_east.png" % [Game.skin_folder(), Game.skin_folder()])
    await _fade(flash, 0.0, 0.8)
    await _say("pro_gua_1")
    if _skipping: return
    # Nirbana gathers out of the mist
    var nir := _sprite("res://assets/characters32/nirbana/nirbana_south.png", Vector2(900, 640), 8)
    nir.modulate = Color(0.6, 0.4, 0.9, 0.0)
    Sfx.play("whoosh", -4.0, 0.6)
    var t := create_tween()
    t.tween_property(nir, "modulate", Color.WHITE, 1.2)
    await t.finished
    var bob := create_tween().set_loops()
    bob.tween_property(nir, "position:y", nir.position.y - 8, 1.0).set_trans(Tween.TRANS_SINE)
    bob.tween_property(nir, "position:y", nir.position.y, 1.0).set_trans(Tween.TRANS_SINE)
    await _say("pro_gua_2")
    if _skipping: return
    # the inscription cracks into four shards that fly out of the cave
    Sfx.play("bump", 0.0, 0.6)
    _shake_node(bg, 0.6)
    flash.color = Color(1, 1, 1)
    flash.modulate.a = 0.0
    var f := create_tween()
    f.tween_property(flash, "modulate:a", 0.8, 0.08)
    f.tween_property(flash, "modulate:a", 0.0, 0.4)
    await _shards()
    await _say("pro_gua_3")
    bob.kill()


func _rakan(who: String, facing: String) -> String:
    return "res://assets/characters32/rakan_%s/rakan_%s_%s.png" % [who, who, facing]


func _hop(n: Control) -> void:
    var t := create_tween()
    t.tween_property(n, "position:y", n.position.y - 12, 0.1)
    t.tween_property(n, "position:y", n.position.y, 0.15)


func _scene_image(path: String) -> TextureRect:
    var r := TextureRect.new()
    r.texture = UI.tex(path)
    r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    r.size = Vector2(1280, 720)
    add_child(r)
    move_child(r, flash.get_index())
    return r


## a character sprite standing with its feet at `feet`, `sc` times bigger
func _sprite(path: String, feet: Vector2, sc: float) -> TextureRect:
    var r := TextureRect.new()
    r.texture = UI.tex(path)
    r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    r.size = r.texture.get_size() * sc
    r.position = feet - Vector2(r.size.x / 2.0, r.size.y)
    add_child(r)
    move_child(r, flash.get_index())
    return r


func _shards() -> void:
    var centre := Vector2(640, 330)
    var targets := [Vector2(-100, 100), Vector2(1380, 120), Vector2(-100, 640), Vector2(1380, 660)]
    var last: Tween
    for i in 4:
        # the same picture the player later earns for this Serpihan (zone ganjaran)
        var icon := str(Game.zone_data("1_%d" % (i + 1)).get("ganjaran", {}).get("icon", "bintang"))
        var s := _shard(icon)
        s.position = centre - s.size / 2.0
        s.rotation = randf_range(-0.6, 0.6)
        add_child(s)
        move_child(s, flash.get_index())
        last = create_tween()
        last.tween_property(s, "position", centre + Vector2(randf_range(-60, 60), randf_range(-60, 20)), 0.4).set_trans
(Tween.TRANS_BACK).set_ease(Tween.EASE_OUT)
        last.tween_property(s, "position", targets[i], 0.8).set_ease(Tween.EASE_IN)
        Sfx.play("sparkle", -6.0, 1.0 + i * 0.15)
        await _wait(0.15)
    if last:
        await last.finished
```

```

## one flying Serpihan: its reward icon on a soft gold glow
func _shard(icon: String) -> Control:
    var box := Control.new()
    box.size = Vector2(72, 72)
    box.pivot_offset = box.size / 2.0
    box.mouse_filter = Control.MOUSE_FILTER_IGNORE
    box.draw.connect(func():
        for k in 4:
            box.draw_circle(box.size / 2.0, 36.0 - k * 7.0, Color(1, 0.86, 0.4, 0.12 + k * 0.06)))
    var r := TextureRect.new()
    r.texture = load("res://assets/icons/%s.png" % icon)
    r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    r.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    r.size = Vector2(56, 56)
    r.position = Vector2(8, 8)
    r.mouse_filter = Control.MOUSE_FILTER_IGNORE
    box.add_child(r)
    return box


func _shake_node(n: Control, seconds: float) -> void:
    var t := 0.0
    while t < seconds and not _skipping and is_inside_tree():
        n.position = Vector2(randf_range(-5, 5), randf_range(-5, 5))
        await get_tree().create_timer(0.04).timeout
        t += 0.04
    n.position = Vector2.ZERO


# --- helpers ----------------------------------------------------------------
func _say(id: String) -> void:
    if _skipping: return
    await Game.say(Dialogue.lines(id))


func _wait(seconds: float) -> void:
    if _skipping: return
    await get_tree().create_timer(seconds).timeout


func _fade(node: CanvasItem, alpha: float, seconds: float) -> void:
    if _skipping: return
    var t := create_tween()
    t.tween_property(node, "modulate:a", alpha, seconds)
    await t.finished


func _show_caption(text: String) -> void:
    caption.text = text
    caption.modulate.a = 0.0
    _fade(caption, 1.0, 0.5)


func _clock_loop() -> void:
    _ticking = true
    var tock := false
    while _ticking and not _skipping:
        Sfx.play("tock" if tock else "tick", -6.0)
        tock = not tock
        await get_tree().create_timer(_tick_gap).timeout


func _heart_loop() -> void:
    while not _skipping and is_inside_tree():
        if _heart_gap > 0.0:
            Sfx.play("heartbeat", -3.0)
            var t := create_tween()
            t.tween_property(vignette, "modulate:a", 1.0, 0.08)
            t.tween_property(vignette, "modulate:a", 0.55, 0.4)
            await get_tree().create_timer(_heart_gap).timeout
        else:
            await get_tree().create_timer(0.2).timeout


func _lamp_loop() -> void:
    while not _skipping and is_inside_tree():
        lamp.modulate.a = randf_range(0.85, 1.0)
        await get_tree().create_timer(randf_range(0.05, 0.25)).timeout


var _shaking := false
func _shake_room(on: bool) -> void:
```

```
    _shaking = on
    while _shaking and not _skipping and is_inside_tree():
        room.position = Vector2(randf_range(-3, 3), randf_range(-3, 3))
        await get_tree().create_timer(0.04).timeout
    room.position = Vector2.ZERO


func _blink(close_to: float, seconds: float) -> void:
    await _lids(close_to, seconds)
    await _lids(0.25, seconds * 0.6)


## 0 = eyes open, 1 = fully closed
func _lids(closed: float, seconds: float) -> void:
    if _skipping: return
    var h := 360.0 * closed
    var t := create_tween().set_parallel()
    t.tween_property(lid_top, "size:y", h, seconds).set_ease(Tween.EASE_IN_OUT)
    t.tween_property(lid_bottom, "position:y", 720.0 - h, seconds).set_ease(Tween.EASE_IN_OUT)
    t.tween_property(lid_bottom, "size:y", h, seconds).set_ease(Tween.EASE_IN_OUT)
    await t.finished


## dyslexia-style scramble: mirrored letters and shuffled word middles
func _scramble(text: String, amount: float) -> String:
    var words := text.split(" ")
    for i in words.size():
        if randf() > amount:
            continue
        var w: String = words[i]
        var chars := w.split("")
        for c in chars.size():
            if MIRROR.has(chars[c]) and randf() < 0.6:
                chars[c] = MIRROR[chars[c]]
        if chars.size() > 3:
            var a := randi_range(1, chars.size() - 2)
            var b := randi_range(1, chars.size() - 2)
            var tmp := chars[a]
            chars[a] = chars[b]
            chars[b] = tmp
        words[i] = "".join(chars)
    return " ".join(words)


func _page_texture() -> Texture2D:
    var img := Image.create(6, 8, false, Image.FORMAT_RGBA8)
    img.fill(Color(1, 0.96, 0.82))
    for x in 6:
        img.set_pixel(x, 0, Color(0.85, 0.75, 0.5))
    for y in [3, 5]:
        for x in range(1, 5):
            img.set_pixel(x, y, Color(0.7, 0.6, 0.45))
    return ImageTexture.create_from_image(img)


func _skip() -> void:
    if _skipping:
        return
    _skipping = true
    Sfx.play("select")
    Main.dialog_box.cancel()
    _go_to_world()


var _leaving := false
func _go_to_world() -> void:
    if _leaving:
        return
    _leaving = true
    Main.dialog_box.auto_advance = false  # the wake-up in the room is read at your own pace
    Game.save["prolog_done"] = true
    Game.persist()
    GameState.flags["bilik_intro"] = true
    GameState.go_to(BILIK, Color(1, 0.97, 0.88))
```

scripts/puzzles/

board.gd · scripts/puzzles/board.gd · 241 lines

```
extends "res://scripts/puzzles/puzzle_base.gd"
## Engine 2: Papan Siasatan (cause-and-effect investigation board).
## Cards sit in columns; the player links them with red yarn. Tap a card,
## then tap the related card (or drag from one card to the other).

const UIB = preload("res://scripts/ui/ui.gd")


class BoardCard extends PanelContainer:
    var engine: Node
    var cid: String
    var data: Dictionary

    func _init(e: Node, d: Dictionary) -> void:
        engine = e
        data = d
        cid = str(d.get("id", ""))
        theme_type_variation = "Card"
        mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
        custom_minimum_size = Vector2(0, 84)
        size_flags_horizontal = Control.SIZE_EXPAND_FILL
        var row := HBoxContainer.new()
        row.add_theme_constant_override("separation", 8)
        row.mouse_filter = Control.MOUSE_FILTER_IGNORE
        var ip := str(d.get("icon", ""))
        if ip != "":
            row.add_child(UIB.icon(ip, 40))
        var l := Label.new()
        l.text = str(d.get("text", ""))
        l.theme_type_variation = "Ink"
        l.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART
        l.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        l.custom_minimum_size.x = 160
        l.mouse_filter = Control.MOUSE_FILTER_IGNORE
        row.add_child(l)
        add_child(row)

    func anchor() -> Vector2:
        return get_global_rect().get_center()

    func _gui_input(event: InputEvent) -> void:
        if event is InputEventMouseButton and event.button_index == MOUSE_BUTTON_LEFT:
            if event.pressed:
                engine.card_down(self)
            else:
                engine.card_up(self)
            accept_event()


class Yarn extends Control:
    var engine: Node

    func _draw() -> void:
        var origin := get_global_rect().position
        for l in engine.made:
            var a: Vector2 = engine.cards[l[0]].anchor() - origin
            var b: Vector2 = engine.cards[l[1]].anchor() - origin
            draw_line(a, b, Color(0.55, 0.08, 0.08), 7.0)
            draw_line(a, b, Color(0.85, 0.2, 0.2), 4.0)
            for p in [a, b]:
                draw_circle(p, 8, Color(0.25, 0.25, 0.3))
                draw_circle(p, 5, Color(0.95, 0.85, 0.3))
        if engine.bad.size() == 2:
            var a2: Vector2 = engine.cards[engine.bad[0]].anchor() - origin
            var b2: Vector2 = engine.cards[engine.bad[1]].anchor() - origin
            draw_dashed_line(a2, b2, Color(0.5, 0.5, 0.55), 4.0, 12.0)
        if engine.selected != null and engine.dragging:
            var a3: Vector2 = engine.selected.anchor() - origin
            draw_line(a3, get_global_mouse_position() - origin, Color(0.85, 0.2, 0.2, 0.6), 4.0)


var cards: Dictionary = {}
var links: Array = []
var made: Array = []
var bad: Array = []
var selected: BoardCard = null
var dragging: bool = false
var missed: Dictionary = {}
var hinted: Dictionary = {}
```

```
var yarn: Yarn
var tree_icon: TextureRect


func _build_puzzle() -> void:
    links = def.get("links", [])
    stats["total"] = links.size()
    var board := UI.panel()
    board.add_theme_stylebox_override("panel", UI.sb(Color("b98a58"), UI.WOOD_D, 6, 6, 14))
    board.size_flags_vertical = Control.SIZE_EXPAND_FILL
    var row := UI.hbox(70)
    var cols: Array = def.get("columns", [])
    var col_boxes: Array = []
    for i in cols.size():
        var c := UI.vbox(10)
        c.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        var h := UI.label(str(cols[i]), "InkHeading")
        h.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        c.add_child(h)
        col_boxes.append(c)
        row.add_child(c)
    var cdefs: Array = def.get("cards", [])
    for d in cdefs:
        var card := BoardCard.new(self, d)
        cards[card.cid] = card
        var ci := clampi(int(d.get("col", 0)), 0, col_boxes.size() - 1)
        col_boxes[ci].add_child(card)
    yarn = Yarn.new()
    yarn.engine = self
    yarn.mouse_filter = Control.MOUSE_FILTER_IGNORE
    board.add_child(yarn)
    board.add_child(row)
    body.add_child(board)
    if def.has("grow_on_links"):
        tree_icon = UI.icon("res://assets/proto/objects/pokok_1.png", 110)
        tree_icon.position = Vector2(1090, 470)
        tree_icon.size = Vector2(110, 110)
        tree_icon.pivot_offset = Vector2(55, 110)
        add_child(tree_icon)


func _process(_d: float) -> void:
    if yarn:
        yarn.queue_redraw()


func _key(a: String, b: String) -> String:
    for l in links:
        if (l[0] == a and l[1] == b) or (l[0] == b and l[1] == a):
            return "%s|%s" % [l[0], l[1]]
    return ""


func _is_made(k: String) -> bool:
    for m in made:
        if "%s|%s" % [m[0], m[1]] == k:
            return true
    return false


func _set_sel(c: BoardCard) -> void:
    if selected and is_instance_valid(selected):
        selected.theme_type_variation = "Card"
    selected = c
    if c:
        c.theme_type_variation = "CardSel"
        say_try("Dipilih: \"%s\". Ketik kad yang berkaitan." % c.data.get("text", ""))


func card_down(c: BoardCard) -> void:
    if _finished:
        return
    if selected != null and selected != c:
        var a := selected
        _set_sel(null)
        try_link(a, c)
        return
    _set_sel(c)
    dragging = true


func card_up(c: BoardCard) -> void:
    dragging = false
    if selected != c:
        return
```

```
    var mp := get_global_mouse_position()
    for k in cards.values():
        if k != c and (k as Control).get_global_rect().has_point(mp):
            _set_sel(null)
            try_link(c, k)
            return


func try_link(a: BoardCard, b: BoardCard) -> void:
    var k := _key(a.cid, b.cid)
    if k == "":
        missed[a.cid] = true
        missed[b.cid] = true
        stats["retries"] = int(stats["retries"]) + 1
        bad = [a.cid, b.cid]
        shake(b)
        say_try("Hmm, dua kad ini tidak bersambung secara langsung. Cuba lagi.")
        get_tree().create_timer(0.8).timeout.connect(func(): bad = [])
        return
    if _is_made(k):
        say_try("Sambungan itu sudah ada.")
        return
    var pair: PackedStringArray = k.split("|")
    made.append([pair[0], pair[1]])
    var first := not missed.has(a.cid) and not missed.has(b.cid) and not hinted.has(k)
    if first:
        stats["first_try"] = int(stats["first_try"]) + 1
    for c in [a, b]:
        (c as BoardCard).theme_type_variation = "CardGood"
        var cc: BoardCard = c
        get_tree().create_timer(0.5).timeout.connect(func():
            if is_instance_valid(cc):
                cc.theme_type_variation = "Card")
    var facts: Dictionary = def.get("link_facts", {})
    say_good("Bersambung! " + str(facts.get(k, "")))
    _update_tree()
    if made.size() >= links.size():
        await get_tree().create_timer(1.0).timeout
        finish(str(def.get("conclusion", "Papan siasatan lengkap!")))


func _update_tree() -> void:
    if tree_icon == null:
        return
    var grow: Array = def.get("grow_on_links", [])
    var n := 0
    for g in grow:
        if _is_made(str(g)):
            n += 1
    var stage := 1 if n < grow.size() else 2
    tree_icon.texture = UI.tex("res://assets/proto/objects/pokok_%d.png" % stage)
    pulse(tree_icon)


func _first_missing() -> Array:
    for l in links:
        if not _is_made("%s|%s" % [l[0], l[1]]):
            return l
    return []


func _hint(tier: int) -> void:
    var l := _first_missing()
    if l.is_empty():
        return
    match tier:
        1:
            Game.main.pau_say(str(def.get("hint", "Mulakan dari lajur pertama. Apa akibatnya?")))
        2:
            Game.main.pau_say("Tengok dua kad yang bercahaya. Adakah ia bersambung?")
            pulse(cards[l[0]])
            pulse(cards[l[1]])
        _:
            var k := "%s|%s" % [l[0], l[1]]
            hinted[k] = true
            Game.main.pau_say("Ini satu sambungan: \"%s\" dengan \"%s\"." % [cards[l[0]].data.get("text", ""), cards[l[
1]].data.get("text", "")])
            try_link(cards[l[0]], cards[l[1]])
```

choice.gd · scripts/puzzles/choice.gd · 38 lines

```
extends "res://scripts/puzzles/puzzle_base.gd"
## Reflection ("Pilihan Alif"): not a quiz, every choice is accepted.


func _build_puzzle() -> void:
    feedback.text = "Tiada jawapan salah."
    var row := UI.hbox(20)
    row.alignment = BoxContainer.ALIGNMENT_CENTER
    row.size_flags_vertical = Control.SIZE_EXPAND_FILL
    for opt in def.get("options", []):
        var b := UI.button("", "")
        b.custom_minimum_size = Vector2(340, 300)
        var c := UI.vbox(12)
        c.mouse_filter = Control.MOUSE_FILTER_IGNORE
        c.set_anchors_preset(Control.PRESET_FULL_RECT)
        c.offset_left = 16
        c.offset_right = -16
        c.offset_top = 20
        c.alignment = BoxContainer.ALIGNMENT_CENTER
        c.add_child(UI.icon(str(opt.get("icon", "")), 120))
        var l := UI.label(str(opt.get("text", "")), "", true)
        l.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        l.mouse_filter = Control.MOUSE_FILTER_IGNORE
        c.add_child(l)
        b.add_child(c)
        var o: Dictionary = opt
        b.pressed.connect(func(): _pick(o))
        row.add_child(b)
    body.add_child(row)


func _pick(opt: Dictionary) -> void:
    Game.save["reflection"] = str(opt.get("id", ""))
    finish(str(opt.get("result", "")))


func _hint(_tier: int) -> void:
    Game.main.pau_say("Pilih apa sahaja yang paling dekat dengan hidup kau. Semua pilihan diterima.")
```

drag_match.gd · scripts/puzzles/drag_match.gd · 257 lines

```
extends "res://scripts/puzzles/puzzle_base.gd"
## Engine 1: Seret dan Padankan (drag and match).
## One engine covers ordering, sorting into bins and pairing: every item
## names its correct target in JSON. Drag a card onto a target, or tap the
## card then tap the target.

const UIX = preload("res://scripts/ui/ui.gd")


class DragCard extends PanelContainer:
    var engine: Node
    var item: Dictionary
    var placed: bool = false
    var mistakes: int = 0
    var hinted: bool = false
    var _label: Label
    var _icon: TextureRect

    func _init(e: Node, it: Dictionary) -> void:
        engine = e
        item = it
        theme_type_variation = "Card"
        mouse_default_cursor_shape = Control.CURSOR_DRAG
        var row := HBoxContainer.new()
        row.add_theme_constant_override("separation", 8)
        row.mouse_filter = Control.MOUSE_FILTER_IGNORE
        var ip := str(it.get("icon", ""))
        if ip != "":
            _icon = UIX.icon(ip, 44)
            row.add_child(_icon)
        _label = Label.new()
        _label.text = str(it.get("label", ""))
        _label.theme_type_variation = "Ink"
        _label.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART
        _label.custom_minimum_size.x = 150
        _label.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        _label.mouse_filter = Control.MOUSE_FILTER_IGNORE
        row.add_child(_label)
        add_child(row)
        custom_minimum_size = Vector2(230, 56)
```

```
    func compact() -> void:
        custom_minimum_size = Vector2(0, 0)
        _label.custom_minimum_size.x = 0
        _label.theme_type_variation = "InkSmall"
        if _icon:
            _icon.custom_minimum_size = Vector2(28, 28)

    func _get_drag_data(_at: Vector2) -> Variant:
        if placed:
            return null
        var prev := PanelContainer.new()
        prev.theme = Game.theme
        prev.theme_type_variation = "CardSel"
        var l := Label.new()
        l.text = _label.text
        l.theme_type_variation = "Ink"
        prev.add_child(l)
        prev.modulate.a = 0.9
        set_drag_preview(prev)
        engine.select(self)
        return {"card": self}

    func _gui_input(event: InputEvent) -> void:
        if placed:
            return
        if event is InputEventMouseButton and event.button_index == MOUSE_BUTTON_LEFT and not event.pressed:
            if not get_viewport().gui_is_dragging():
                engine.select(self)


class DropTarget extends PanelContainer:
    var engine: Node
    var tid: String
    var box: VBoxContainer
    var _base_var: String = "Slot"

    func _init(e: Node, t: Dictionary, min_w: float) -> void:
        engine = e
        tid = str(t.get("id", ""))
        theme_type_variation = "Slot"
        custom_minimum_size = Vector2(min_w, 150)
        size_flags_horizontal = Control.SIZE_EXPAND_FILL
        mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
        var col := VBoxContainer.new()
        col.add_theme_constant_override("separation", 6)
        col.mouse_filter = Control.MOUSE_FILTER_IGNORE
        var head := HBoxContainer.new()
        head.add_theme_constant_override("separation", 6)
        head.alignment = BoxContainer.ALIGNMENT_CENTER
        head.mouse_filter = Control.MOUSE_FILTER_IGNORE
        var ip := str(t.get("icon", ""))
        if ip != "":
            head.add_child(UIX.icon(ip, 40))
        var l := Label.new()
        l.text = str(t.get("label", ""))
        l.theme_type_variation = "Heading"
        l.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART
        l.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        l.custom_minimum_size.x = min_w - (70 if ip != "" else 30)
        if min_w < 300:
            l.add_theme_font_size_override("font_size", 20)
        l.mouse_filter = Control.MOUSE_FILTER_IGNORE
        head.add_child(l)
        col.add_child(head)
        if t.has("sub"):
            var s := Label.new()
            s.text = str(t["sub"])
            s.theme_type_variation = "Small"
            s.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
            s.mouse_filter = Control.MOUSE_FILTER_IGNORE
            col.add_child(s)
        box = VBoxContainer.new()
        box.add_theme_constant_override("separation", 4)
        box.mouse_filter = Control.MOUSE_FILTER_IGNORE
        col.add_child(box)
        add_child(col)

    func set_state(v: String) -> void:
        theme_type_variation = v

    func _can_drop_data(_at: Vector2, data: Variant) -> bool:
        var ok: bool = data is Dictionary and data.has("card")
        if ok:
            set_state("SlotHot")
```

```
        return ok

    func _drop_data(_at: Vector2, data: Variant) -> void:
        set_state(_base_var)
        engine.try_place(data["card"], self)

    func _notification(what: int) -> void:
        if what == NOTIFICATION_MOUSE_EXIT or what == NOTIFICATION_DRAG_END:
            if theme_type_variation == "SlotHot":
                set_state(_base_var)

    func _gui_input(event: InputEvent) -> void:
        if event is InputEventMouseButton and event.pressed and event.button_index == MOUSE_BUTTON_LEFT:
            engine.target_tapped(self)


var selected: DragCard = null
var cards: Array = []
var targets: Dictionary = {}
var pool: HFlowContainer


func _build_puzzle() -> void:
    var tdefs: Array = def.get("targets", [])
    var items: Array = def.get("items", [])
    stats["total"] = items.size()
    var trow := UI.hbox(10)
    trow.size_flags_vertical = Control.SIZE_EXPAND_FILL
    var min_w := (1160.0 - 10.0 * (tdefs.size() - 1)) / maxf(1.0, tdefs.size())
    for t in tdefs:
        var dt := DropTarget.new(self, t, min_w)
        targets[dt.tid] = dt
        trow.add_child(dt)
    body.add_child(trow)
    var pool_panel := UI.panel("Slot")
    pool_panel.add_theme_stylebox_override("panel", UI.sb(Color(0.3, 0.22, 0.14, 0.9), UI.WOOD_D, 3, 8, 10))
    var pcol := UI.vbox(4)
    var pl := UI.label("Kad untuk diseret", "Small")
    pcol.add_child(pl)
    pool = HFlowContainer.new()
    pool.add_theme_constant_override("h_separation", 10)
    pool.add_theme_constant_override("v_separation", 8)
    pcol.add_child(pool)
    pool_panel.add_child(pcol)
    body.add_child(pool_panel)
    var shuffled := items.duplicate()
    shuffled.shuffle()
    for it in shuffled:
        var c := DragCard.new(self, it)
        cards.append(c)
        pool.add_child(c)


func select(c: DragCard) -> void:
    if selected and is_instance_valid(selected) and not selected.placed:
        selected.theme_type_variation = "Card"
    selected = c
    c.theme_type_variation = "CardSel"
    say_try("Dipilih: %s. Sekarang ketik atau seret ke tempat yang betul." % c.item.get("label", ""))


func target_tapped(t: DropTarget) -> void:
    if selected and is_instance_valid(selected) and not selected.placed:
        try_place(selected, t)


func try_place(c: DragCard, t: DropTarget) -> void:
    if c.placed:
        return
    if str(c.item.get("target", "")) == t.tid:
        _place(c, t)
    else:
        c.mistakes += 1
        stats["retries"] = int(stats["retries"]) + 1
        t.set_state("SlotBad")
        shake(t)
        get_tree().create_timer(0.5).timeout.connect(func():
            if is_instance_valid(t):
                t.set_state("Slot"))
        c.theme_type_variation = "Card"
        selected = null
        var tip := str(c.item.get("try", ""))
        say_try("Hmm, belum tepat. " + (tip if tip != "" else "Kad itu kembali ke tempatnya. Cuba lagi, atau ketik Pau
untuk petunjuk."))

```

```
210
211 func _place(c: DragCard, t: DropTarget) -> void:
212     c.placed = true
213     if c.mistakes == 0 and not c.hinted:
214         stats["first_try"] = int(stats["first_try"]) + 1
215     c.get_parent().remove_child(c)
216     t.box.add_child(c)
217     c.compact()
218     c.theme_type_variation = "CardGood"
219     c.mouse_default_cursor_shape = Control.CURSOR_ARROW
220     selected = null
221     t.set_state("SlotGood")
222     get_tree().create_timer(0.4).timeout.connect(func():
223         if is_instance_valid(t):
224             t.set_state("Slot"))
225     say_good("Betul! " + str(c.item.get("fact", "")))
226     for k in cards:
227         if not k.placed:
228             return
229     await get_tree().create_timer(0.9).timeout
230     finish(str(def.get("done_text", "Semua kad di tempat yang betul!")))
231
232
233 func _first_unplaced() -> DragCard:
234     for k in cards:
235         if not k.placed:
236             return k
237     return null
238
239
240 func _hint(tier: int) -> void:
241     var c := _first_unplaced()
242     if c == null:
243         return
244     var t: DropTarget = targets.get(str(c.item.get("target", "")))
245     match tier:
246         1:
247             Game.main.pau_say(str(def.get("hint", "Baca setiap kad perlahan-lahan. Fikir kenapa ia sesuai.")))
248         2:
249             Game.main.pau_say("Tengok kad dan tempat yang bercahaya tu.")
250             pulse(c)
251             if t:
252                 pulse(t)
253         _:
254             c.hinted = true
255             Game.main.pau_say("Ini satu sambungan yang betul: %s." % c.item.get("label", ""))
256             if t:
257                 _place(c, t)
```

objektif.gd · scripts/puzzles/objektif.gd · 238 lines

```
1  extends "res://scripts/puzzles/puzzle_base.gd"
2  ## Soalan objektif: one question at a time, four answers (A-D), one correct.
3  ## A wrong answer just greys out and the player tries again (no timer, no
4  ## penalty). After the right answer a short explanation shows the why.
5  ## Pau's hints: 1 = nudge, 2 and 3 = remove one wrong answer.
6  ##
7  ## def: {"type": "objektif", "title", "instruction", "hint", "done_text",
8  ##       "questions": [{"q", "options": [4 strings], "answer": index, "explain", "hint"}],
9  ##       "boss"?: name}   with a boss, every right answer knocks its HP bar down
10 ## Ujian Bab mode ("hearts": 3, "need": 7): a wrong answer costs a heart and
11 ## shows the right one with its explanation, then the test moves on. Boss HP 0
12 ## = passed; no hearts left = stats.passed false (the host offers a retry).
13
14 const LETTERS := ["A", "B", "C", "D"]
15
16 var questions: Array = []
17 var qi: int = 0
18 var _buttons: Array[Button] = []
19 var _wrong_here: bool = false
20 var _q_label: Label
21 var _count: Label
22 var _next: Button
23 var _answered: bool = false
24 var _boss_hp: ProgressBar
25 var _hearts: Hearts
26 var _need := 0
27 var _hits := 0
28
29
30 func _build_puzzle() -> void:
31     questions = def.get("questions", [])
32     stats["total"] = questions.size()
```

```
    stats["passed"] = true
    stats["wrong"] = []  ## Ujian mode: {q, picked, right, explain, zon} per lost heart
    _need = mini(int(def.get("need", questions.size())), questions.size())
    var top := UI.hbox(10)
    _count = UI.label("", "Small")
    top.add_child(_count)
    if def.has("hearts"):
        _hearts = Hearts.new()
        _hearts.count = int(def["hearts"])
        _hearts.custom_minimum_size = Vector2(26 * _hearts.count + 8, 26)
        top.add_child(_hearts)
    if def.has("boss"):
        var gap := Control.new()
        gap.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        top.add_child(gap)
        var bn := UI.label(str(def["boss"]).to_upper(), "Heading")
        bn.add_theme_color_override("font_color", Color("ff8a7a"))
        top.add_child(bn)
        _boss_hp = ProgressBar.new()
        _boss_hp.show_percentage = false
        _boss_hp.max_value = _need
        _boss_hp.value = _need
        _boss_hp.custom_minimum_size = Vector2(320, 22)
        _boss_hp.modulate = Color(1.0, 0.55, 0.5)
        top.add_child(_boss_hp)
    body.add_child(top)
    var qp := UI.panel("Paper")
    var qrow := UI.hbox(10)
    _q_label = UI.label("", "InkHeading", true)
    _q_label.custom_minimum_size.x = 1040
    _q_label.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    qrow.add_child(_q_label)
    var sp := UI.icon_button("res://assets/icons/suara.png", "Blue", Vector2(60, 52))
    sp.tooltip_text = "Baca soalan"
    sp.pressed.connect(func(): Tts.speak(_q_text()))
    qrow.add_child(sp)
    qp.add_child(qrow)
    body.add_child(qp)
    var grid := GridContainer.new()
    grid.columns = 2
    grid.add_theme_constant_override("h_separation", 14)
    grid.add_theme_constant_override("v_separation", 14)
    grid.size_flags_vertical = Control.SIZE_EXPAND_FILL
    for i in 4:
        var b := UI.button("")
        b.custom_minimum_size = Vector2(570, 92)
        b.alignment = HORIZONTAL_ALIGNMENT_LEFT
        b.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART
        b.pressed.connect(_pick.bind(i))
        grid.add_child(b)
        _buttons.append(b)
    body.add_child(grid)
    _next = UI.button("Soalan seterusnya »", "Primary", 280)
    _next.size_flags_horizontal = Control.SIZE_SHRINK_END
    _next.visible = false
    _next.pressed.connect(_advance)
    body.add_child(_next)
    _show_question()


func _q() -> Dictionary:
    return questions[qi]


func _q_text() -> String:
    var t := str(_q()["q"])
    var opts: Array = _q()["options"]
    for i in opts.size():
        t += ". %s, %s" % [LETTERS[i], opts[i]]
    return t


func _show_question() -> void:
    _answered = false
    _wrong_here = false
    hint_presses = 0
    _next.visible = false
    if _hearts:
        _count.text = tr("Soalan %d") % (qi + 1)
    else:
        _count.text = tr("Soalan %d / %d") % [qi + 1, questions.size()]
    _q_label.text = str(_q()["q"])
    var opts: Array = _q()["options"]
    for i in _buttons.size():
        var b := _buttons[i]
```

```
        b.visible = i < opts.size()
        if b.visible:
            b.text = "%s.  %s" % [LETTERS[i], opts[i]]
            b.disabled = false
            b.theme_type_variation = ""
            b.modulate = Color.WHITE
    say_try("Pilih satu jawapan. Tiada had masa.")
    if bool(Game.settings.get("auto_read", false)):
        Tts.speak(_q_text())


func _pick(i: int) -> void:
    if _answered:
        return
    if i == int(_q()["answer"]):
        _answered = true
        if not _wrong_here:
            stats["first_try"] = int(stats["first_try"]) + 1
        _buttons[i].theme_type_variation = "Primary"
        for j in _buttons.size():
            if j != i:
                _buttons[j].disabled = true
        Sfx.play("chime", -6.0)
        MusicDirector.stinger("st_correct")
        pulse(_buttons[i])
        _hits += 1
        if _boss_hp:
            Sfx.play("punch", -4.0)
            create_tween().tween_property(_boss_hp, "value", _need - _hits, 0.4)
            shake(_boss_hp)
        say_good(tr("Betul! ") + str(_q().get("explain", "")))
        if _hearts and _hits >= _need:
            await get_tree().create_timer(1.6).timeout
            finish(str(def.get("done_text", "")))
        elif qi + 1 < questions.size():
            _next.visible = true
            _next.grab_focus()
        else:
            await get_tree().create_timer(1.6).timeout
            finish(str(def.get("done_text", "")))
    elif _hearts:
        _lose_heart(i)
    else:
        _wrong_here = true
        stats["retries"] = int(stats["retries"]) + 1
        _buttons[i].disabled = true
        _buttons[i].modulate = Color(1, 1, 1, 0.5)
        shake(_buttons[i])
        Sfx.play("bump", -4.0)
        say_try(["Hampir! Cuba jawapan lain.", "Belum tepat. Baca soalan sekali lagi.", "Tak apa, cuba lagi."].pick_ran
dom())


## Ujian mode: one wrong pick costs a heart; the right answer is shown with its
## reason so the mistake still teaches something.
func _lose_heart(i: int) -> void:
    _answered = true
    stats["retries"] = int(stats["retries"]) + 1
    _hearts.count -= 1
    _hearts.queue_redraw()
    shake(_hearts)
    Sfx.play("bump", -2.0)
    var right := int(_q()["answer"])
    var opts: Array = _q()["options"]
    stats["wrong"].append({"q": _q()["q"], "picked": opts[i], "right": opts[right],
        "explain": _q().get("explain", ""), "zon": _q().get("zon", "")})
    for j in _buttons.size():
        _buttons[j].disabled = true
        if j == i:
            _buttons[j].modulate = Color(1, 0.6, 0.6)
    _buttons[right].theme_type_variation = "Primary"
    _buttons[right].modulate = Color.WHITE
    say_try(tr("Jawapan betul: %s. ") % LETTERS[right] + str(_q().get("explain", "")))
    if _hearts.count <= 0:
        stats["passed"] = false
        await get_tree().create_timer(2.4).timeout
        completed.emit(stats)
    elif qi + 1 < questions.size() and questions.size() - qi - 1 >= _need - _hits:
        _next.visible = true
        _next.grab_focus()
    else:  # not enough questions left to win
        stats["passed"] = false
        await get_tree().create_timer(2.4).timeout
        completed.emit(stats)

```

```
202
203 func _advance() -> void:
204     qi += 1
205     Sfx.play("select")
206     _show_question()
207
208
209 func _hint(tier: int) -> void:
210     if _answered:
211         Game.main.pau_say("Dah betul! Tekan Soalan seterusnya.")
212         return
213     if tier == 1:
214         Game.main.pau_say(str(_q().get("hint", def.get("hint", "Baca soalan perlahan-lahan. Cari kata kunci."))))
215         return
216     # remove one wrong answer that is still on screen
217     for i in _buttons.size():
218         if i != int(_q()["answer"]) and _buttons[i].visible and not _buttons[i].disabled:
219             _buttons[i].disabled = true
220             _buttons[i].modulate = Color(1, 1, 1, 0.35)
221             Game.main.pau_say(tr("Aku buang satu jawapan yang salah: %s.") % LETTERS[i])
222             return
223     Game.main.pau_say("Tinggal satu je lagi. Kau boleh!")
224
225
226 ## Row of pixel hearts for the Ujian Bab.
227 class Hearts extends Control:
228     var count := 3
229     var total := -1
230
231     func _draw() -> void:
232         if total < 0:
233             total = count
234         for h in total:
235             var o := Vector2(4 + h * 26, 3)
236             var c := Color(0.92, 0.25, 0.3) if h < count else Color(0.35, 0.3, 0.32)
237             for r in [[2, 0, 6, 2], [12, 0, 6, 2], [0, 2, 20, 8], [2, 10, 16, 3], [5, 13, 10, 3], [8, 16, 4, 3]]:
238                 draw_rect(Rect2(o + Vector2(r[0], r[1]), Vector2(r[2], r[3])), c)
```

puzzle_base.gd · scripts/puzzles/puzzle_base.gd · 148 lines

```
1  extends Control
2  ## Shared frame for both puzzle engines: header with title, instruction and
3  ## read-aloud, feedback line, Pau's 3-level hints, and the finish card.
4  ## No timer, no penalty: wrong answers just return the card.
5
6  const UI = preload("res://scripts/ui/ui.gd")
7
8  signal completed(stats: Dictionary)
9
10 var def: Dictionary = {}
11 var puzzle_id: String = ""
12 var stats := {"total": 0, "first_try": 0, "retries": 0, "hints": 0}
13 var hint_presses: int = 0
14 var body: VBoxContainer
15 var feedback: Label
16 var _feedback_panel: PanelContainer
17 var _frame: PanelContainer
18 var _finished: bool = false
19
20
21 func _ready() -> void:
22     UI.full(self)
23     mouse_filter = Control.MOUSE_FILTER_STOP
24     add_child(UI.dim(0.72))
25     _frame = UI.panel()
26     _frame.position = Vector2(30, 24)
27     _frame.size = Vector2(1220, 672)
28     _frame.custom_minimum_size = Vector2(1220, 672)
29     add_child(_frame)
30     var col := UI.vbox(10)
31     _frame.add_child(col)
32     var head := UI.hbox(12)
33     var tcol := UI.vbox(2)
34     tcol.size_flags_horizontal = Control.SIZE_EXPAND_FILL
35     tcol.add_child(UI.label(str(def.get("title", "")), "Heading"))
36     var ins := UI.label(str(def.get("instruction", "")), "", true)
37     ins.custom_minimum_size.x = 900
38     tcol.add_child(ins)
39     head.add_child(tcol)
40     var speak := UI.icon_button("res://assets/icons/suara.png", "Blue", Vector2(64, 56))
41     speak.tooltip_text = "Baca arahan"
42     speak.pressed.connect(func(): Tts.speak(str(def.get("instruction", ""))))
43     head.add_child(speak)
```

```
    col.add_child(head)
    body = UI.vbox(10)
    body.size_flags_vertical = Control.SIZE_EXPAND_FILL
    col.add_child(body)
    _feedback_panel = UI.panel("Paper")
    _feedback_panel.add_theme_stylebox_override("panel", UI.sb(UI.CREAM, UI.WOOD, 3, 6, 8))
    feedback = UI.label("Tiada had masa. Cuba seberapa banyak kali yang kau mahu.", "Ink", true)
    feedback.custom_minimum_size.x = 1100
    _feedback_panel.add_child(feedback)
    col.add_child(_feedback_panel)
    _build_puzzle()
    if bool(Game.settings.get("auto_read", false)):
        Tts.speak(str(def.get("instruction", "")))


func _build_puzzle() -> void:
    pass


func say_good(text: String) -> void:
    feedback.text = text
    _feedback_panel.add_theme_stylebox_override("panel", UI.sb(Color("dff0d4"), UI.GREEN_D, 3, 6, 8))
    if bool(Game.settings.get("auto_read", false)):
        Tts.speak(text)


func say_try(text: String) -> void:
    feedback.text = text
    _feedback_panel.add_theme_stylebox_override("panel", UI.sb(Color("fff0d8"), Color("c08a30"), 3, 6, 8))


func show_pau_hint(text: String) -> void:
    feedback.text = "Pau: " + text
    _feedback_panel.add_theme_stylebox_override("panel", UI.sb(Color("e4f0fb"), UI.BLUE, 3, 6, 8))
    if bool(Game.settings.get("auto_read", false)):
        Tts.speak(text)


func request_hint() -> void:
    if _finished:
        Game.main.pau_say("Dah siap! Ketik Teruskan.")
        return
    hint_presses += 1
    var tier := mini(hint_presses, 3)
    stats["hints"] = int(stats["hints"]) + 1
    Game.log_hint(puzzle_id, tier)
    _hint(tier)


## Override: tier 1 = general nudge, 2 = highlight, 3 = show one connection.
func _hint(tier: int) -> void:
    Game.main.pau_say(str(def.get("hint", "Cuba tengok artifak yang bercahaya tu.")))


func pulse(c: CanvasItem) -> void:
    var tw := create_tween().set_loops(3)
    tw.tween_property(c, "modulate", Color(1.6, 1.4, 0.6), 0.35)
    tw.tween_property(c, "modulate", Color.WHITE, 0.35)


func shake(c: Control) -> void:
    var orig := c.position
    var tw := create_tween()
    for i in 4:
        tw.tween_property(c, "position", orig + Vector2(8 if i % 2 == 0 else -8, 0), 0.04)
    tw.tween_property(c, "position", orig, 0.04)


func finish(done_text: String) -> void:
    _finished = true
    var cover := UI.dim(0.5)
    cover.mouse_filter = Control.MOUSE_FILTER_STOP
    add_child(cover)
    var card := UI.panel("Paper")
    card.custom_minimum_size = Vector2(720, 0)
    var c := UI.vbox(14)
    var star := UI.icon("res://assets/icons/bintang.png", 96)
    c.add_child(star)
    var t := UI.label("SYABAS!", "InkHeading")
    t.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(t)
    var d := UI.label(done_text, "Ink", true)
    d.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(d)
    var s := UI.label("Betul cubaan pertama: %d / %d    Petunjuk Pau: %d" % [stats["first_try"], stats["total"], stats[
```

```
"hints"]], "InkSmall", true)
    s.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    s.visible = int(stats["total"]) > 0
    c.add_child(s)
    var row := UI.hbox(12)
    row.alignment = BoxContainer.ALIGNMENT_CENTER
    var sp := UI.icon_button("res://assets/icons/suara.png", "Blue", Vector2(64, 56))
    sp.pressed.connect(func(): Tts.speak(done_text))
    row.add_child(sp)
    var go := UI.button("Teruskan »", "Primary", 220)
    go.pressed.connect(func(): completed.emit(stats))
    row.add_child(go)
    c.add_child(row)
    card.add_child(c)
    add_child(UI.center(card))
    card.scale = Vector2(0.8, 0.8)
    card.pivot_offset = Vector2(360, 160)
    create_tween().tween_property(card, "scale", Vector2.ONE, 0.25).set_trans(Tween.TRANS_BACK).set_ease(Tween.EASE_OUT
)
    go.grab_focus()
    if bool(Game.settings.get("auto_read", false)):
        Tts.speak(done_text)
```

suaikan.gd · scripts/puzzles/suaikan.gd · 163 lines

```
extends "res://scripts/puzzles/puzzle_base.gd"
## Suaikan (matching): tap an item on the left, then its partner on the
## right. A correct pair is joined by a gold line and shows its fact; a wrong
## one shakes and nothing is lost. Tapping works on phones (no dragging).
## Pau's hints: 1 = nudge, 2 = glow one pair, 3 = join one pair.
##
## def: {"type": "suaikan", "title", "instruction", "hint", "done_text",
##       "left_title", "right_title",
##       "pairs": [{"left", "right", "fact", "icon"?}]}

var pairs: Array = []
var _left: Array[Button] = []
var _right: Array[Button] = []   ## shuffled; _right_pair[i] = pair index
var _right_pair: Array[int] = []
var _sel: int = -1               ## selected left pair index
var _done := {}                  ## pair index -> true
var _missed := {}                ## pair index -> true (had a wrong try)
var _lines: Control


func _build_puzzle() -> void:
    pairs = def.get("pairs", [])
    stats["total"] = pairs.size()
    var row := UI.hbox(0)
    row.size_flags_vertical = Control.SIZE_EXPAND_FILL
    var lcol := UI.vbox(10)
    var rcol := UI.vbox(10)
    for c in [lcol, rcol]:
        c.custom_minimum_size.x = 470
        c.alignment = BoxContainer.ALIGNMENT_CENTER
    var lt := UI.label(str(def.get("left_title", "")), "Heading")
    lcol.add_child(lt)
    var rt := UI.label(str(def.get("right_title", "")), "Heading")
    rcol.add_child(rt)
    for i in pairs.size():
        var b := _item_button(str(pairs[i]["left"]), str(pairs[i].get("icon", "")))
        b.pressed.connect(_tap_left.bind(i))
        lcol.add_child(b)
        _left.append(b)
    var order := range(pairs.size())
    order.shuffle()
    if order == range(pairs.size()) and pairs.size() > 1:  # never already solved
        order.reverse()
    for k in order.size():
        var i: int = order[k]
        var b := _item_button(str(pairs[i]["right"]), "")
        b.pressed.connect(_tap_right.bind(k))
        rcol.add_child(b)
        _right.append(b)
        _right_pair.append(i)
    row.add_child(lcol)
    var gap := Control.new()
    gap.custom_minimum_size.x = 220
    row.add_child(gap)
    row.add_child(rcol)
    body.add_child(UI.center(row))
    _lines = Lines.new()
    _lines.owner_puzzle = self
```

```
    _lines.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(_lines)
    UI.full(_lines)
    say_try("Ketik satu di kiri, kemudian pasangannya di kanan.")


func _item_button(text: String, icon_path: String) -> Button:
    var b := UI.button(text)
    b.custom_minimum_size = Vector2(470, 64)
    b.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART
    b.toggle_mode = false
    if icon_path != "":
        b.icon = UI.tex(icon_path)
        b.expand_icon = false
        b.add_theme_constant_override("icon_max_width", 44)
    return b


func _tap_left(i: int) -> void:
    if _done.has(i):
        return
    _sel = i
    Sfx.play("select", -6.0)
    for j in _left.size():
        if not _done.has(j):
            _left[j].theme_type_variation = "Blue" if j == i else ""
    say_try(tr("Sekarang ketik pasangan untuk: %s") % str(pairs[i]["left"]))


func _tap_right(k: int) -> void:
    var i := _right_pair[k]
    if _done.has(i):
        return
    if _sel < 0:
        say_try("Pilih dulu satu di sebelah kiri.")
        pulse(_left[_first_open()])
        return
    try_match(_sel, k)


## used by taps, Pau's tier-3 hint and the autotest
func try_match(left_i: int, right_k: int) -> void:
    if _right_pair[right_k] == left_i:
        _done[left_i] = true
        if not _missed.has(left_i):
            stats["first_try"] = int(stats["first_try"]) + 1
        for b in [_left[left_i], _right[right_k]]:
            b.theme_type_variation = "Primary"  # stays green; taps on it are ignored
            b.focus_mode = Control.FOCUS_NONE
            pulse(b)
        _sel = -1
        Sfx.play("chime", -8.0)
        say_good(str(pairs[left_i].get("fact", "Betul!")))
        _lines.queue_redraw()
        if _done.size() == pairs.size():
            MusicDirector.stinger("st_correct")
            await get_tree().create_timer(1.4).timeout
            finish(str(def.get("done_text", "")))
    else:
        _missed[left_i] = true
        stats["retries"] = int(stats["retries"]) + 1
        shake(_right[right_k])
        Sfx.play("bump", -4.0)
        say_try(["Bukan pasangan itu. Cuba lagi.", "Hampir! Fikir sekali lagi.", "Tak apa, cuba yang lain."].pick_rando
m())


func _first_open() -> int:
    for i in pairs.size():
        if not _done.has(i):
            return i
    return 0


func _right_of(i: int) -> int:
    return _right_pair.find(i)


func _hint(tier: int) -> void:
    var i := _sel if _sel >= 0 else _first_open()
    match tier:
        1: Game.main.pau_say(str(def.get("hint", "Cari kata yang sama maksud di kedua-dua belah.")))
        2:
            pulse(_left[i])
            pulse(_right[_right_of(i)])
```

```
            Game.main.pau_say("Tengok dua yang bercahaya tu. Mereka sepasang.")
        _:
            Game.main.pau_say(tr("Aku tolong satu: %s.") % str(pairs[i]["left"]))
            try_match(i, _right_of(i))


## Gold lines between matched pairs (drawn over the buttons' edges).
class Lines extends Control:
    var owner_puzzle: Node

    func _draw() -> void:
        var p = owner_puzzle
        for i in p._done:
            var a: Button = p._left[i]
            var b: Button = p._right[p._right_of(i)]
            var from := a.get_global_rect().position + Vector2(a.size.x, a.size.y / 2.0) - global_position
            var to := b.get_global_rect().position + Vector2(0, b.size.y / 2.0) - global_position
            draw_line(from, to, Color(0.1, 0.08, 0.05), 7.0)
            draw_line(from, to, UI.GOLD, 4.0)
            draw_circle(from, 6, UI.GOLD)
            draw_circle(to, 6, UI.GOLD)
```

scripts/

screen_host.gd · scripts/screen_host.gd · 17 lines

```
extends Control
## Hosts one menu screen from the first prototype (login, register, lobby,
## zone summary, penutup). Game.goto(name, params) sets Game.pending_screen
## and changes to this scene.


func _ready() -> void:
    theme = Game.theme
    var screen := Game.pending_screen
    if not Game.SCREENS.has(screen):
        push_error("Unknown screen: " + screen)
        return
    var node: Node = load(Game.SCREENS[screen]).new()
    node.set("params", Game.pending_params)
    if node is Control:
        (node as Control).theme = Game.theme
    add_child(node)
```

scripts/screens/

arka.gd · scripts/screens/arka.gd · 162 lines

```
extends Control
## Arka & Zon: each Arka is a Tingkatan (Form), each Zon is a Bab (chapter).
## Open zon show their misi progress; locked ones are greyed with a padlock
## and the word BERKUNCI (never colour only). Data: data/chapters.json.

const UI = preload("res://scripts/ui/ui.gd")

var params: Dictionary = {}
var _arc: int = 0
var _grid: GridContainer
var _tabs: HBoxContainer
var _info: Label


func _ready() -> void:
    UI.full(self)
    Game.main.set_pau_visible(true)
    MusicDirector.play("kampung")
    Game.context_hint = "Pilih Zon yang terbuka. Zon lain akan terbuka bila bab sebelumnya selesai."
    add_child(UI.bg_image("res://assets/backgrounds/bg_peta_jejak_pembelajaran.png", Color(0.55, 0.55, 0.62)))
    var root := UI.vbox(8)
    UI.full(root)
    root.add_child(UI.header_bar("ARKA & ZON", tr("Setiap Arka ialah satu Tingkatan. Setiap Zon ialah satu Bab.")))

    var body := UI.vbox(10)
    _tabs = UI.hbox(10)
    _tabs.alignment = BoxContainer.ALIGNMENT_CENTER
    var arcs: Array = Game.chapters.get("arcs", [])
    for i in arcs.size():
```

```
        var a: Dictionary = arcs[i]
        var b := UI.button(str(a["name"]).replace("Sejarah ", ""), "", 260)
        if not a.get("open", false):
            b.icon = UI.tex("res://assets/icons/kunci.png")
            b.expand_icon = false
        b.pressed.connect(_show_arc.bind(i))
        _tabs.add_child(b)
    body.add_child(_tabs)
    _grid = GridContainer.new()
    _grid.columns = 5
    _grid.add_theme_constant_override("h_separation", 14)
    _grid.add_theme_constant_override("v_separation", 10)
    body.add_child(UI.center(_grid))
    _grid.get_parent().size_flags_vertical = Control.SIZE_EXPAND_FILL
    _info = UI.label("", "", true)
    _info.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    body.add_child(_info)

    var bar := UI.hbox(12)
    bar.alignment = BoxContainer.ALIGNMENT_CENTER
    var prof := UI.button("Profil & Kedai", "Blue", 230)
    prof.pressed.connect(func(): if Game.main.has_method("open_profile"): Game.main.open_profile())
    bar.add_child(prof)
    var battle := UI.button("Bilik Pertarungan", "", 260)
    battle.pressed.connect(func(): Game.goto("lobby"))
    bar.add_child(battle)
    var papan := UI.button("Papan Markah", "", 220)
    papan.pressed.connect(func(): Game.main.open_papan())
    bar.add_child(papan)
    var st := UI.button("Tetapan", "", 180)
    st.pressed.connect(func(): Game.main.open_settings())
    bar.add_child(st)
    body.add_child(bar)
    root.add_child(UI.margin(body, 40, 0, 40, 14))
    root.get_child(1).size_flags_vertical = Control.SIZE_EXPAND_FILL
    add_child(root)
    _show_arc(0)


func _show_arc(i: int) -> void:
    _arc = i
    for j in _tabs.get_child_count():
        (_tabs.get_child(j) as Button).theme_type_variation = "Primary" if j == i else ""
    for c in _grid.get_children():
        c.queue_free()
    var arc: Dictionary = Game.chapters["arcs"][i]
    if not arc.get("open", false):
        _info.text = tr("%s belum dibuka. Habiskan Tingkatan 4 dulu.") % str(arc["name"])
        for n in 10:
            _grid.add_child(_card({"no": n + 1, "title": "???", "icon": "kunci"}, false))
        Sfx.play("bump", -6.0)
        return
    _info.text = tr("Ketik Zon yang terbuka untuk masuk ke dunianya.")
    for ch in arc.get("chapters", []):
        var open := bool(ch.get("open", false)) and int(ch["no"]) <= GameState.unlocked_chapters
        _grid.add_child(_card(ch, open))
    Sfx.play("select", -8.0)


func _progress(ch: Dictionary) -> Vector2i:
    var zones: Array = ch.get("zones", [])
    var done := 0
    for z in zones:
        if Game.zone_done(str(z)):
            done += 1
    return Vector2i(done, zones.size())


func _card(ch: Dictionary, open: bool) -> Control:
    var no := int(ch["no"])
    var p := _progress(ch)
    var finished := open and p.y > 0 and p.x >= p.y
    var card := PanelContainer.new()
    card.theme_type_variation = ("SlotGood" if finished else "SlotHot") if open else "CardLocked"
    card.custom_minimum_size = Vector2(222, 186)
    card.mouse_filter = Control.MOUSE_FILTER_STOP
    card.mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
    var col := UI.vbox(4)
    var top := UI.hbox(6)
    var zl := UI.label(tr("ZON %d") % no, "Heading")
    zl.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    top.add_child(zl)
    if not open:
        top.add_child(UI.icon("res://assets/icons/kunci.png", 30))
    col.add_child(top)
```

```
    var ic := UI.icon("res://assets/icons/%s.png" % str(ch.get("icon", "bintang")), 44)
    if not open:
        ic.modulate = Color(0.5, 0.5, 0.55, 0.8)
    col.add_child(ic)
    var t := UI.label(str(ch.get("title", "")), "Small", true)
    t.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    t.custom_minimum_size.x = 190
    col.add_child(t)
    col.add_child(UI.spacer(false))
    var status := UI.label("", "Small")
    status.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    if open:
        var bar := ProgressBar.new()
        bar.theme_type_variation = "Good" if finished else ""
        bar.show_percentage = false
        bar.max_value = maxi(p.y, 1)
        bar.value = p.x
        bar.custom_minimum_size = Vector2(0, 14)
        col.add_child(bar)
        status.text = tr("SIAP") if finished else tr("%d / %d misi") % [p.x, p.y]
        status.add_theme_color_override("font_color", Color("8fe08a") if finished else UI.GOLD)
    else:
        status.text = tr("BERKUNCI")
        status.add_theme_color_override("font_color", UI.GREY)
    col.add_child(status)
    card.add_child(col)
    card.gui_input.connect(func(e: InputEvent):
        if e is InputEventMouseButton and e.pressed and e.button_index == MOUSE_BUTTON_LEFT:
            _enter(ch, open))
    return card


func _enter(ch: Dictionary, open: bool) -> void:
    if not open:
        Sfx.play("bump", -4.0)
        Game.main.pau_say(tr("Belum lagi. Zon %d akan terbuka lepas kita habiskan bab sebelumnya.") % int(ch["no"]), 4.
0)
        return
    Sfx.play("select")
    MusicDirector.stinger("st_correct")
    # each Bab has its own place in the world: Zon 2 starts in Lembah Kebangkitan
    var ent = JSON.parse_string(FileAccess.get_file_as_string("res://data/world/entities.json"))
    var pos: Array = Game.save.get("map_pos", [])
    var in_lembah := pos.size() == 2 and int(pos[0]) >= 82
    if int(ch["no"]) == 2 and not in_lembah and ent is Dictionary and ent.has("arrive2"):
        GameState.spawn_cell = Vector2i(int(ent["arrive2"][0]), int(ent["arrive2"][1]))
    elif int(ch["no"]) == 1 and in_lembah and ent is Dictionary:
        GameState.spawn_cell = Vector2i(11, 27)
    Game.goto("world" if bool(Game.save.get("prolog_done", false)) else "prolog")
```

gelanggang.gd · scripts/screens/gelanggang.gd · 495 lines

```
extends Control
## Gelanggang Pesta Silat: the side-view brawl screen. Each round: a question
## card (arena frozen) -> who got Semangat -> AKSI! (live fight, 10 s).
## params: {"format": "1v1"|"2v2"|"1v2"|"1v3", "level": "mudah"|"biasa"|"sukar"}
## The rules live in scripts/brawl/ (BrawlMatch, BrawlFighter, BrawlAI).
## Jeda (button, Esc or P) pauses the match and has the Cara Main guide and
## the keyboard layout switch; the guide also opens before the first match.

const UI = preload("res://scripts/ui/ui.gd")
const TouchControls = preload("res://scripts/ui/touch_controls.gd")
const BrawlGuide = preload("res://scripts/ui/brawl_guide.gd")
const LETTERS := ["A", "B", "C", "D"]
const BG := "res://assets/backgrounds/bg_gelanggang.png"
const BG_FALLBACK := "res://assets/backgrounds/bg_zon1_3_balai_penghadapan.png"

var params: Dictionary = {}
var match_node: BrawlMatch
var _rows := {}            ## fighter id -> {hp: ProgressBar, badge: Label, row}
var _round: Label
var _phase: Label
var _clock: Label
var _card: PanelContainer
var _q: Label
var _q_timer: ProgressBar
var _q_status: Label
var _buttons: Array[Button] = []
var _banner: PanelContainer
var _banner_box: VBoxContainer
var _hint: Label
var _big: Label
var _touch: CanvasLayer
```

```
var _result: Control
var _pause: Control        ## the Jeda overlay (null when playing)
var _leaving := false      ## Keluar pressed: the match is frozen until the scene changes

signal _pause_closed


func _ready() -> void:
    UI.full(self)
    # this screen keeps running while paused (for the Jeda menu); the match and
    # the touch buttons stop
    process_mode = Node.PROCESS_MODE_ALWAYS
    Game.use_brawl_keys(true)
    Game.main.set_pau_visible(false)
    MusicDirector.play("melaka")
    MusicDirector.puzzle(true)
    var bg := UI.bg_image(BG if ResourceLoader.exists(BG) else BG_FALLBACK, Color(1, 1, 1) if ResourceLoader.exists(BG) else Color(0.55, 0.5, 0.55))
    add_child(bg)
    add_child(Mat.new())
    match_node = BrawlMatch.new()
    match_node.setup(str(params.get("format", "1v1")), str(params.get("level", "biasa")))
    match_node.me_autopilot = bool(params.get("autopilot", false))
    match_node.process_mode = Node.PROCESS_MODE_PAUSABLE
    add_child(match_node)
    _build_hud()
    _build_card()
    _build_banner()
    _touch = TouchControls.new()
    _touch.brawl = true
    _touch.process_mode = Node.PROCESS_MODE_PAUSABLE
    add_child(_touch)
    match_node.phase_changed.connect(_on_phase)
    match_node.changed.connect(_refresh)
    if Game.save["seen"].has("gelanggang_cara") or match_node.me_autopilot:
        match_node.start()
    else:  # first time here: how to play, then fight
        Game.save["seen"].append("gelanggang_cara")
        Game.persist()
        await _open_pause(true)
        match_node.start()


## every way out (Keluar, Main lagi, Kembali, Log keluar) ends here: nothing
## of the match may stay paused, slowed or ducked in the next scene
func _exit_tree() -> void:
    get_tree().paused = false
    Engine.time_scale = 1.0
    MusicDirector.puzzle(false)
    Game.use_brawl_keys(false)


# ---------------------------------------------------------------- HUD
func _build_hud() -> void:
    var top := UI.hbox(12)
    top.position = Vector2(16, 10)
    top.size = Vector2(1248, 0)
    for t in 2:
        var panel := UI.panel()
        panel.custom_minimum_size = Vector2(430, 0)
        var col := UI.vbox(4)
        var title := UI.label("PASUKAN KAU" if t == 0 else "LAWAN", "Small")
        title.add_theme_color_override("font_color", UI.GOLD if t == 0 else Color("ff8a7a"))
        col.add_child(title)
        for f in match_node.fighters:
            if f.team != t:
                continue
            var row := UI.hbox(8)
            var head := UI.head_icon(f.skin, 34)
            row.add_child(head)
            var nm := UI.label(f.display_name + (tr(" (kau)") if f.is_me else ""), "Small")
            nm.custom_minimum_size.x = 130
            nm.clip_text = true
            row.add_child(nm)
            var hp := ProgressBar.new()
            hp.theme_type_variation = "Good"
            hp.max_value = BrawlRules.HP
            hp.value = f.hp
            hp.show_percentage = false
            hp.custom_minimum_size = Vector2(150, 16)
            hp.size_flags_vertical = Control.SIZE_SHRINK_CENTER
            row.add_child(hp)
            var badge := UI.label("", "Small")
            badge.custom_minimum_size.x = 90
            row.add_child(badge)
```

```
            col.add_child(row)
            _rows[f.id] = {"hp": hp, "badge": badge, "row": row}
        panel.add_child(col)
        if t == 1:
            var mid := UI.vbox(0)
            mid.alignment = BoxContainer.ALIGNMENT_CENTER
            var mid_panel := UI.panel()
            mid_panel.size_flags_horizontal = Control.SIZE_EXPAND_FILL
            mid_panel.size_flags_vertical = Control.SIZE_SHRINK_BEGIN
            mid_panel.add_child(mid)
            _round = UI.label("", "Heading")
            _round.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
            mid.add_child(_round)
            _clock = UI.label("", "Title")
            _clock.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
            mid.add_child(_clock)
            _phase = UI.label("", "Small")
            _phase.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
            mid.add_child(_phase)
            top.add_child(mid_panel)
        top.add_child(panel)
    var pause := UI.button(tr("Jeda"), "", 110)
    pause.size_flags_vertical = Control.SIZE_SHRINK_BEGIN
    pause.tooltip_text = tr("Jeda dan cara main (Esc)")
    pause.pressed.connect(_open_pause)
    top.add_child(pause)
    add_child(top)
    _hint = UI.label("", "Small", true)
    _hint.position = Vector2(20, 682)
    _hint.size = Vector2(1100, 30)
    add_child(_hint)
    _big = UI.label("", "Title")
    _big.add_theme_font_size_override("font_size", 96)
    _big.add_theme_color_override("font_color", UI.GOLD)
    _big.add_theme_color_override("font_outline_color", Color(0.1, 0.05, 0.03))
    _big.add_theme_constant_override("outline_size", 14)
    _big.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    _big.position = Vector2(0, 250)
    _big.size = Vector2(1280, 130)
    _big.pivot_offset = Vector2(640, 65)
    _big.modulate.a = 0.0
    _big.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(_big)


func _process(_d: float) -> void:
    if match_node == null:
        return
    var left := 0.0
    match match_node.phase:
        "soalan":
            left = BrawlRules.ASK_TIME - match_node.phase_t
            _q_timer.value = left
        "aksi":
            left = BrawlRules.FIGHT_TIME - match_node.phase_t
    _clock.text = "%d" % ceili(maxf(left, 0.0)) if match_node.phase in ["soalan", "aksi"] else ""
    _clock.add_theme_color_override("font_color", Color("ff6a5a") if match_node.phase == "aksi" and left < 3.0 else UI.
GOLD)
    if _touch:
        _touch.suspended = match_node.phase != "aksi" or not match_node.me.alive()


func _refresh() -> void:
    for f in match_node.fighters:
        var r: Dictionary = _rows.get(f.id, {})
        if r.is_empty():
            continue
        var hp: ProgressBar = r["hp"]
        create_tween().tween_property(hp, "value", float(f.hp), 0.25)
        hp.theme_type_variation = "Good" if f.hp > BrawlRules.HP / 2 else ("Warn" if f.hp > BrawlRules.HP / 4 else "")
        var badge: Label = r["badge"]
        if not f.alive():
            badge.text = tr("PENGSAN")
            badge.add_theme_color_override("font_color", Color("c9a0ff"))
            (r["row"] as Control).modulate.a = 0.55
        elif match_node.phase == "soalan":
            badge.text = tr("sudah jawab") if match_node.answers.has(f.id) else ""
            badge.add_theme_color_override("font_color", Color("9fe8a0"))
        elif match_node.phase in ["keputusan", "aksi"]:
            badge.text = (tr("JURUS!") if f.special_ready else tr("SEMANGAT")) if f.answered_right else tr("LEMAH")
            badge.add_theme_color_override("font_color", UI.GOLD if f.answered_right else Color("a8b0bc"))
    _round.text = tr("Pusingan %d") % match_node.round_no
    if match_node.phase == "soalan" and match_node.answers.has(match_node.me.id):
        _q_status.text = tr("Jawapan dihantar. Menunggu yang lain...")

```

```

# ---------------------------------------------------------------- question card
func _build_card() -> void:
    _card = UI.panel("Paper")
    _card.custom_minimum_size = Vector2(1100, 0)
    var col := UI.vbox(10)
    _q_timer = ProgressBar.new()
    _q_timer.show_percentage = false
    _q_timer.max_value = BrawlRules.ASK_TIME
    _q_timer.custom_minimum_size = Vector2(0, 14)
    col.add_child(_q_timer)
    var qrow := UI.hbox(10)
    _q = UI.label("", "InkHeading", true)
    _q.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    qrow.add_child(_q)
    var sp := UI.icon_button("res://assets/icons/suara.png", "Blue", Vector2(60, 52))
    sp.tooltip_text = "Baca soalan"
    sp.pressed.connect(func(): Tts.speak(_q.text))
    qrow.add_child(sp)
    col.add_child(qrow)
    var grid := GridContainer.new()
    grid.columns = 2
    grid.add_theme_constant_override("h_separation", 10)
    grid.add_theme_constant_override("v_separation", 8)
    for i in 4:
        var b := UI.button("")
        b.custom_minimum_size = Vector2(535, 58)
        b.alignment = HORIZONTAL_ALIGNMENT_LEFT
        b.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART
        b.pressed.connect(_answer.bind(i))
        grid.add_child(b)
        _buttons.append(b)
    col.add_child(grid)
    _q_status = UI.label("", "InkSmall", true)
    col.add_child(_q_status)
    _card.add_child(col)
    _card.position = Vector2(90, 150)
    _card.visible = false
    add_child(_card)


func _answer(i: int) -> void:
    if match_node.phase != "soalan" or match_node.answers.has(match_node.me.id) or not match_node.me.alive():
        return
    match_node.answer(i)
    for j in _buttons.size():
        _buttons[j].disabled = true
        _buttons[j].theme_type_variation = "Blue" if j == i else ""
    Sfx.play("select")


func _unhandled_input(e: InputEvent) -> void:
    var pause_key: bool = e.is_action_pressed("menu") or (e is InputEventKey and e.pressed and not e.echo and e.physica
l_keycode == KEY_P)
    if _leaving:
        get_viewport().set_input_as_handled()
        return
    if pause_key and _result == null:
        get_viewport().set_input_as_handled()
        if _pause:
            _close_pause()
        else:
            _open_pause()
        return
    if _pause:
        return
    if match_node and match_node.phase == "soalan" and e is InputEventKey and e.pressed and not e.echo:
        var k: int = e.keycode - KEY_1
        if k >= 0 and k < 4:
            _answer(k)


# ---------------------------------------------------------------- banner (who got Semangat)
func _build_banner() -> void:
    _banner = UI.panel()
    _banner.custom_minimum_size = Vector2(700, 0)
    _banner_box = UI.vbox(6)
    _banner.add_child(_banner_box)
    _banner.position = Vector2(290, 170)
    _banner.visible = false
    add_child(_banner)


## long questions or big text make the card taller: lift it so the bottom
## row of answers never runs off the screen
```

```
func _fit_card() -> void:
    _card.reset_size()
    var h := get_viewport_rect().size.y
    _card.position.y = clampf(h - _card.size.y - 16.0, 16.0, 150.0)


func _on_phase(p: String) -> void:
    _card.visible = p == "soalan"
    _banner.visible = p == "keputusan"
    match p:
        "soalan":
            _q.text = str(match_node.q["q"])
            for i in _buttons.size():
                var b := _buttons[i]
                b.text = "%s.  %s" % [LETTERS[i], match_node.q["options"][i]]
                b.disabled = not match_node.me.alive()
                b.theme_type_variation = ""
                b.modulate = Color.WHITE
            _q_status.text = tr("Jawab betul untuk dapat SEMANGAT (serangan penuh) dan senjata sekolah!") if match_node.me.alive() else tr("Kau pengsan. Tengok rakan sepasukan berjuang!")
            _phase.text = tr("SOALAN")
            _hint.text = ""
            Sfx.play("pop", -6.0)
            if bool(Game.settings.get("auto_read", false)):
                Tts.speak(_q.text)
            _fit_card.call_deferred()
        "keputusan":
            for c in _banner_box.get_children():
                c.queue_free()
            var head := UI.label(tr("Jawapan: %s. %s") % [LETTERS[match_node.answer_index], match_node.q["options"][match_node.answer_index]], "Heading", true)
            _banner_box.add_child(head)
            if match_node.explain != "":
                _banner_box.add_child(UI.label(match_node.explain, "Small", true))
            for r in match_node.results:
                var txt: String = "%s%s:  %s" % [r["name"], tr(" (kau)") if r["me"] else "", tr("BETUL, SEMANGAT!") if r["right"] else tr("salah, lemah")]
                if r["fastest"]:
                    txt += "  +" + (tr("JURUS KILAT (butang Jurus)") if BrawlGuide.on_touch() else tr("JURUS KILAT (tekan %s)") % Game.brawl_key_names("special"))
                var l := UI.label(txt, "")
                l.add_theme_color_override("font_color", UI.GOLD if r["right"] else Color("a8b0bc"))
                _banner_box.add_child(l)
            _phase.text = tr("KEPUTUSAN")
            if match_node.me.answered_right:
                MusicDirector.stinger("st_correct")
        "aksi":
            _phase.text = tr("AKSI!")
            _flash_big(tr("AKSI!"))
            _hint.text = _controls_hint()
        "tamat":
            _hint.text = ""
            _show_result()
    _refresh()


func _flash_big(text: String) -> void:
    _big.text = text
    _big.scale = Vector2(1.6, 1.6)
    _big.modulate.a = 1.0
    var t := create_tween()
    t.tween_property(_big, "scale", Vector2.ONE, 0.25).set_trans(Tween.TRANS_BACK).set_ease(Tween.EASE_OUT)
    t.tween_interval(0.5)
    t.tween_property(_big, "modulate:a", 0.0, 0.3)


# ---------------------------------------------------------------- result
func _show_result() -> void:
    MusicDirector.puzzle(false)
    var won := match_node.winner_team == 0
    _result = Control.new()
    UI.full(_result)
    _result.add_child(UI.dim(0.55))
    var card := UI.panel("Paper")
    card.custom_minimum_size = Vector2(640, 0)
    var c := UI.vbox(12)
    c.add_child(UI.center(UI.icon("res://assets/icons/%s.png" % ("mahkota" if won else "bendera"), 96)))
    var t := UI.label(tr("PASUKAN KAU MENANG!") if won else tr("PASUKAN LAWAN MENANG"), "InkHeading")
    t.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(t)
    var reward := "+%d EXP" % BrawlRules.REWARD_EXP + (("   +%d " % BrawlRules.REWARD_COINS) + tr("Syiling") if won else "")
    var r := UI.label(reward, "Ink")
    r.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(r)
```

```
    var note := UI.label(tr("Syabas! Setiap soalan yang kau jawab ialah ulang kaji %s.") % BattleRules.chapter_name() if won else tr("Takpe, cuba lagi! Lagi banyak jawapan betul, lagi kuat serangan kau."), "InkSmall", true)
    note.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(note)
    var row := UI.hbox(12)
    row.alignment = BoxContainer.ALIGNMENT_CENTER
    var again := UI.button("Main lagi", "Primary", 200)
    again.pressed.connect(func(): Game.goto("gelanggang", params))
    row.add_child(again)
    var back := UI.button("Kembali", "", 200)
    back.pressed.connect(func(): Game.goto("lobby"))
    row.add_child(back)
    c.add_child(row)
    card.add_child(c)
    _result.add_child(UI.center(card))
    add_child(_result)
    MusicDirector.stinger("st_levelup" if won else "st_zone")
    if won:
        Game.toast("+%d Syiling" % BrawlRules.REWARD_COINS)
    _flash_big(tr("MENANG!") if won else tr("KALAH"))


## one line of controls under the arena while fighting
func _controls_hint() -> String:
    if BrawlGuide.on_touch():
        return tr("Guna butang di kanan bawah. Tekan Jeda untuk cara main.")
    var k := func(a: String) -> String: return Game.brawl_key_names(a)
    return tr("%s Tumbuk · %s Sepak · %s Lompat · %s Ambil/Baling · %s Tangkis · %s Jurus · %s Lari · Esc Jeda") % [
        k.call("attack_punch"), k.call("attack_kick"), k.call("jump"), k.call("grab"),
        k.call("block"), k.call("special"), k.call("run")]


# ---------------------------------------------------------------- Jeda (pause)
## first = the intro before the first match: just the guide and "Mula"
func _open_pause(first := false) -> void:
    if _pause or _leaving:
        return
    if not first:  # (the intro comes before the match: nothing to stop, and
        get_tree().paused = true  # pausing would freeze the screen fade-in)
    Sfx.play("pop", -6.0)
    _pause = Control.new()
    UI.full(_pause)
    _pause.mouse_filter = Control.MOUSE_FILTER_STOP
    _pause.add_child(UI.dim(0.6))
    add_child(_pause)
    if first:
        _show_guide("Mula bertarung!")
    else:
        _show_menu()
    await _pause_closed  # never comes if the player leaves: the wait ends with the screen


func _close_pause() -> void:
    if _pause:
        _pause.queue_free()
        _pause = null
    get_tree().paused = false
    _pause_closed.emit()
    if match_node and match_node.phase == "aksi":
        _hint.text = _controls_hint()  # the layout may have changed


func _clear_pause() -> void:
    for c in _pause.get_children():
        if not c is ColorRect:
            c.queue_free()


func _show_menu() -> void:
    _clear_pause()
    var card := UI.panel("Paper")
    card.custom_minimum_size = Vector2(460, 0)
    var c := UI.vbox(14)
    var t := UI.label(tr("JEDA"), "InkHeading")
    t.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(t)
    var go := UI.button(tr("Sambung"), "Primary", 380)
    go.pressed.connect(_close_pause)
    c.add_child(go)
    var how := UI.button(tr("Cara main & kawalan"), "Blue", 380)
    how.pressed.connect(func(): _show_guide("« Kembali", true))
    c.add_child(how)
    var out := UI.button(tr("Keluar gelanggang"), "", 380)
    out.pressed.connect(_leave)
    c.add_child(out)
```

```
    card.add_child(c)
    _pause.add_child(UI.center(card))
    go.grab_focus()


func _show_guide(button_text: String, back_to_menu := false) -> void:
    _clear_pause()
    var card := BrawlGuide.card(button_text, true)
    var ok: Button = card.find_child("Ok", true, false)
    ok.pressed.connect(_show_menu if back_to_menu else _close_pause)
    _pause.add_child(UI.center(card))
    ok.grab_focus()


func _leave() -> void:
    if _leaving:
        return
    _leaving = true
    # freeze the match itself (not the tree: the fade to the lobby must run)
    match_node.process_mode = Node.PROCESS_MODE_DISABLED
    _touch.process_mode = Node.PROCESS_MODE_DISABLED
    for b in _pause.find_children("*", "Button", true, false):
        (b as Button).disabled = true
    get_tree().paused = false
    MusicDirector.puzzle(false)
    Engine.time_scale = 1.0
    Game.goto("lobby")


## the silat mat on the floor (a trapezoid, so the floor reads as depth)
class Mat extends Control:
    func _ready() -> void:
        set_anchors_preset(Control.PRESET_FULL_RECT)
        mouse_filter = Control.MOUSE_FILTER_IGNORE

    func _draw() -> void:
        var f := BrawlRules.SCREEN_FLOOR
        var top := f.position.y - 38
        var bot := f.end.y + 24
        var pts := PackedVector2Array([Vector2(f.position.x + 60, top), Vector2(f.end.x - 60, top), Vector2(f.end.x + 40, bot), Vector2(f.position.x - 40, bot)])
        draw_colored_polygon(pts, Color(0.62, 0.16, 0.14, 0.92))
        var inner := PackedVector2Array([Vector2(f.position.x + 110, top + 18), Vector2(f.end.x - 110, top + 18), Vector2(f.end.x - 20, bot - 18), Vector2(f.position.x + 20, bot - 18)])
        draw_colored_polygon(inner, Color(0.16, 0.3, 0.55, 0.95))
        draw_polyline(pts + PackedVector2Array([pts[0]]), Color(0.95, 0.85, 0.55), 4.0)
        var mid := Vector2((f.position.x + f.end.x) / 2.0, (top + bot) / 2.0)
        draw_set_transform(mid, 0, Vector2(1, 0.32))
        draw_arc(Vector2.ZERO, 120, 0, TAU, 48, Color(0.95, 0.85, 0.55, 0.8), 4.0, true)
        draw_set_transform(Vector2.ZERO)
```

lobby.gd · scripts/screens/lobby.gd · 303 lines

```
extends Control
## Bilik Pertarungan hub (Arena Ulang Kaji). Two ways to play:
##  - Lawan Bot: you against 2-4 classmate bots, on this device, no internet.
##  - Main dengan Rakan: add friends by Player ID, open a room, share its code;
##    2-4 friends join and the host starts. Needs the online server
##    (Supabase, supabase/multiplayer.sql); offline it explains that.

const UI = preload("res://scripts/ui/ui.gd")
const BrawlGuide = preload("res://scripts/ui/brawl_guide.gd")

var params: Dictionary = {}
var _format := "1v1"
var _level := "mudah"  ## new players start easy
var _faces: HBoxContainer
var _right: VBoxContainer
var _room_code := ""
var _room_box: VBoxContainer
var _poll_t := 0.0
var _polling := false


func _ready() -> void:
    UI.full(self)
    Game.main.set_pau_visible(true)
    MusicDirector.play("melaka")
    Game.context_hint = "Jawab betul untuk serang pemain yang kau pilih. Yang terakhir berdiri menang 20 syiling!"
    add_child(UI.bg_image("res://assets/backgrounds/bg_zon1_3_balai_penghadapan.png", Color(0.6, 0.55, 0.58)))
    var root := UI.vbox(12)
    UI.full(root)
    root.add_child(UI.header_bar("BILIK PERTARUNGAN", tr("Jawab betul, serang lawan. Yang terakhir berdiri menang %d sy
```

```
iling.") % BattleRules.REWARD_COINS))
    var row := UI.hbox(20)
    row.alignment = BoxContainer.ALIGNMENT_CENTER
    row.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    row.size_flags_vertical = Control.SIZE_EXPAND_FILL
    row.add_child(_bot_card())
    _right = UI.vbox(10)
    var friends := UI.panel("Glass")
    friends.custom_minimum_size = Vector2(560, 0)
    friends.add_child(_right)
    row.add_child(friends)
    # the two cards scroll when big text makes them taller than the screen;
    # the button row below stays put so KEMBALI is always reachable
    var scroll := ScrollContainer.new()
    scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
    scroll.size_flags_vertical = Control.SIZE_EXPAND_FILL
    scroll.add_child(row)
    root.add_child(scroll)
    var btns := UI.hbox(16)
    btns.alignment = BoxContainer.ALIGNMENT_CENTER
    var back := UI.button("« KEMBALI", "", 240)
    back.pressed.connect(func(): Game.goto("arka"))
    btns.add_child(back)
    var prof := UI.button("Profil & Kedai", "Blue", 240)
    prof.pressed.connect(func(): Game.main.open_profile())
    btns.add_child(prof)
    var papan := UI.button("Papan Markah", "", 220)
    papan.pressed.connect(func(): Game.main.open_papan())
    btns.add_child(papan)
    var st := UI.icon_button("res://assets/icons/tetapan.png", "", Vector2(200, 60), " TETAPAN")
    st.pressed.connect(func(): Game.main.open_settings())
    btns.add_child(st)
    root.add_child(UI.margin(btns, 0, 0, 0, 14))
    add_child(root)
    _build_friends()


## Gelanggang Pesta Silat vs classmate bots: pick the teams and how tough
func _bot_card() -> Control:
    var card := UI.panel("Glass")
    card.custom_minimum_size = Vector2(580, 0)
    var c := UI.vbox(8)
    var head := UI.hbox(10)
    var ht := UI.label("GELANGGANG PESTA SILAT", "InkHeading")
    ht.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    head.add_child(ht)
    var how := UI.button(tr("Cara main"), "Blue", 0)
    how.pressed.connect(_show_guide)
    head.add_child(how)
    c.add_child(head)
    var d := UI.label("Jawab betul untuk dapat semangat dan senjata sekolah, kemudian bertarung! Tiada internet diperlu
kan.", "Ink", true)
    d.custom_minimum_size.x = 530
    c.add_child(d)
    _faces = UI.hbox(4)
    c.add_child(_faces)
    var fmt := UI.hbox(6)
    fmt.add_child(_row_label("Pasukan"))
    for f in BrawlRules.FORMATS:
        var b := UI.button(f.replace("v", " lwn "), "", 100)
        b.pressed.connect(func():
            _format = f
            _pick(fmt, b)
            _show_faces())
        b.theme_type_variation = "Primary" if f == _format else ""
        fmt.add_child(b)
    c.add_child(fmt)
    var lv := UI.hbox(6)
    lv.add_child(_row_label("Tahap lawan"))
    for l in BrawlRules.LEVELS:
        var b := UI.button(BrawlRules.LEVELS[l]["name"], "", 130)
        b.pressed.connect(func():
            _level = l
            _pick(lv, b))
        b.theme_type_variation = "Primary" if l == _level else ""
        lv.add_child(b)
    c.add_child(lv)
    # which chapter the questions come from (only chapters the player has opened)
    var open := BattleRules.chapters_open()
    if not open.has(BattleRules.chapter) and BattleRules.chapter != 0:
        BattleRules.chapter = open[-1] if not open.is_empty() else 1
    if open.size() > 1:
        var ch_row := UI.hbox(6)
        ch_row.add_child(_row_label("Soalan"))
        for n in open + [0]:
```

```
            var b := UI.button(tr("Campur") if n == 0 else tr("Bab %d") % n, "", 110)
            b.pressed.connect(func():
                BattleRules.chapter = n
                _pick(ch_row, b))
            b.theme_type_variation = "Primary" if n == BattleRules.chapter else ""
            ch_row.add_child(b)
        c.add_child(ch_row)
    c.add_child(UI.spacer(false))
    # both ways in on one row so they stay on screen without scrolling
    var go_row := UI.hbox(10)
    var go := UI.button("MASUK GELANGGANG", "Primary", 290)
    go.pressed.connect(func():
        Sfx.play("select")
        Game.goto("gelanggang", {"format": _format, "level": _level}))
    go_row.add_child(go)
    # the plain quiz battle (no fighting) is still there for anyone who prefers it
    var quiz := UI.button("Kuiz sahaja", "", 0)
    quiz.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    quiz.tooltip_text = tr("Kuiz pertarungan tanpa aksi")
    quiz.pressed.connect(func(): Game.goto("pertarungan", {"mode": "bot", "bots": maxi(BattleRules.MIN_FRIENDS, BrawlRu
les.FORMATS[_format][1])}))
    go_row.add_child(quiz)
    c.add_child(go_row)
    card.add_child(c)
    _show_faces()
    go.call_deferred("grab_focus")
    return card


## the name of a row of choices, beside it (not above) so the card stays short
func _row_label(text: String) -> Label:
    var l := UI.label(text, "InkSmall", true)
    l.custom_minimum_size.x = 100
    l.vertical_alignment = VERTICAL_ALIGNMENT_CENTER
    return l


## the same Cara Main as the Gelanggang pause menu
func _show_guide() -> void:
    Sfx.play("pop", -6.0)
    var cover := Control.new()
    UI.full(cover)
    cover.mouse_filter = Control.MOUSE_FILTER_STOP
    cover.add_child(UI.dim(0.6))
    var card := BrawlGuide.card("Faham!")
    var ok: Button = card.find_child("Ok", true, false)
    ok.pressed.connect(cover.queue_free)
    cover.add_child(UI.center(card))
    add_child(cover)
    ok.grab_focus()


func _pick(row: HBoxContainer, chosen: Button) -> void:
    Sfx.play("tick", -8.0)
    for b in row.get_children():
        if b is Button:  # skip the row's name label
            b.theme_type_variation = "Primary" if b == chosen else ""


## your team (you + allies) vs the other team, as heads
func _show_faces() -> void:
    for ch in _faces.get_children():
        ch.queue_free()
    var sizes: Array = BrawlRules.FORMATS[_format]
    _faces.add_child(UI.head_icon(Game.skin_folder(), 60))
    for i in int(sizes[0]) - 1:
        _faces.add_child(UI.head_icon(str(LocalBattle.BOTS[i]["skin"]), 60))
    var vs := UI.label(" LWN ", "InkHeading")
    _faces.add_child(vs)
    for i in int(sizes[1]):
        _faces.add_child(UI.head_icon(str(LocalBattle.BOTS[(i + int(sizes[0]) - 1) % LocalBattle.BOTS.size()]["skin"]),
 60))


# ---------------------------------------------------------------- friends (online)
func _build_friends() -> void:
    for c in _right.get_children():
        c.queue_free()
    _right.add_child(UI.label("MAIN DENGAN RAKAN", "InkHeading"))
    var id_row := UI.hbox(8)
    id_row.add_child(UI.label("ID Pemain kau:", "Ink"))
    var idl := UI.label(str(Game.save["player_id"]), "InkHeading")
    id_row.add_child(idl)
    _right.add_child(id_row)
    if not Backend.can_play_online():
```

```
        var off := UI.label(tr("Mod ini perlukan pelayan dalam talian dan akaun yang log masuk dengan e-mel. Buat masa ini, cuba Lawan Bot dulu!") + "\n\n" + tr("Untuk guru: jalankan supabase/schema.sql dan supabase/multiplayer.sql di Supabase, kemudian isi config/backend.json."), "Ink", true)
        off.custom_minimum_size.x = 500
        _right.add_child(off)
        return
    # add a friend
    var add_row := UI.hbox(8)
    var code := LineEdit.new()
    code.placeholder_text = tr("ID rakan (6 aksara)")
    code.max_length = 6
    code.custom_minimum_size.x = 260
    add_row.add_child(code)
    var add := UI.button("Tambah rakan", "Blue")
    var status := UI.label("", "InkSmall", true)
    add.pressed.connect(func():
        var res: String = await Backend.add_friend(code.text)
        status.text = tr({"accepted": "Kini berkawan!", "pending": "Permintaan dihantar. Minta rakan tambah ID kau juga.",
            "not_found": "ID tidak dijumpai.", "self": "Itu ID kau sendiri."}.get(res, "Gagal. Cuba lagi."))
        _load_friends())
    add_row.add_child(add)
    _right.add_child(add_row)
    _right.add_child(status)
    var list := UI.vbox(4)
    list.name = "List"
    _right.add_child(list)
    _load_friends()
    # rooms
    var rooms := UI.hbox(8)
    var create := UI.button("Buka bilik", "Primary")
    create.pressed.connect(_create_room)
    rooms.add_child(create)
    var join_code := LineEdit.new()
    join_code.placeholder_text = tr("Kod bilik")
    join_code.max_length = 6
    join_code.custom_minimum_size.x = 160
    rooms.add_child(join_code)
    var join := UI.button("Sertai")
    join.pressed.connect(func(): _join_room(join_code.text))
    rooms.add_child(join)
    _right.add_child(rooms)
    _room_box = UI.vbox(4)
    _right.add_child(_room_box)


func _load_friends() -> void:
    var list: VBoxContainer = _right.get_node_or_null("List")
    if list == null:
        return
    for c in list.get_children():
        c.queue_free()
    var friends: Array = await Backend.my_friends()
    if friends.is_empty():
        list.add_child(UI.label(tr("Belum ada rakan. Tukar ID dengan kawan sekelas."), "InkSmall"))
    for f in friends:
        var st: String = {"accepted": "", "outgoing": tr("  (menunggu)"), "incoming": tr("  (tambah ID dia untuk terima)")}.get(str(f["status"]), "")
        list.add_child(UI.label("%s  ·  %s%s" % [str(f["name"]), str(f["code"]), st], "Ink"))


func _create_room() -> void:
    _room_code = await Backend.create_room()
    _show_room()


func _join_room(code: String) -> void:
    var res: String = await Backend.join_room(code)
    if res != "ok":
        Game.toast(tr({"not_found": "Bilik tidak dijumpai.", "started": "Pertarungan sudah bermula.", "not_friends": "Kau perlu berkawan dengan tuan bilik dulu.", "full": "Bilik penuh."}.get(res, "Gagal sertai bilik.")), Color("ff9a8a"))
        return
    _room_code = code.strip_edges().to_upper()
    _show_room()


func _show_room() -> void:
    if _room_code == "":
        return
    for c in _room_box.get_children():
        c.queue_free()
    _room_box.add_child(UI.label(tr("Kod bilik: %s  (kongsi dengan rakan)") % _room_code, "InkHeading"))


func _process(delta: float) -> void:
```

```
    if _room_code == "":
        return
    _poll_t += delta
    if _poll_t < 1.5 or _polling:
        return
    _poll_t = 0.0
    _polling = true
    var s: Dictionary = await Backend.room_state(_room_code)
    _polling = false
    if s.is_empty():
        return
    if str(s.get("status", "")) != "lobby":
        Game.goto("pertarungan", {"mode": "online", "code": _room_code})
        _room_code = ""
        return
    for c in _room_box.get_children():
        if c != _room_box.get_child(0):
            c.queue_free()
    var players: Array = s.get("players", [])
    _room_box.add_child(UI.label(tr("Pemain: %d / %d") % [players.size(), BattleRules.MAX_FRIENDS + 1], "Ink"))
    for p in players:
        _room_box.add_child(UI.label("·  " + str(p["name"]), "Ink"))
    if str(s.get("host", "")) == Backend.user_id:
        var start := UI.button("Mula!", "Primary")
        start.disabled = players.size() < BattleRules.MIN_FRIENDS + 1
        start.pressed.connect(func(): await Backend.start_battle(_room_code))
        _room_box.add_child(start)
```

login.gd · scripts/screens/login.gd · 132 lines

```
extends Control
## Module 1: Log masuk (FR-AUTH-01, FR-AUTH-03) + Guest mode (UR-LO-01).

const UI = preload("res://scripts/ui/ui.gd")
const TitleBackdrop = preload("res://scripts/ui/title_backdrop.gd")

var params: Dictionary = {}
var _user: LineEdit
var _pass: LineEdit
var _login: Button
var _status: Label


func _ready() -> void:
    UI.full(self)
    Game.main.set_pau_visible(false)
    MusicDirector.play("tajuk")
    var bd := TitleBackdrop.new()
    bd.tint = Color(0.75, 0.78, 0.8)
    add_child(bd)
    var frame := UI.panel()
    frame.custom_minimum_size = Vector2(520, 0)
    var col := UI.vbox(12)
    var title := UI.label("LOG MASUK", "Heading")
    title.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    col.add_child(title)

    col.add_child(UI.label("E-MEL ATAU NAMA PENGGUNA"))
    var r1 := UI.hbox(8)
    r1.add_child(UI.icon("res://assets/icons/orang.png", 40))
    _user = LineEdit.new()
    _user.placeholder_text = "cth: alif@sekolah.edu.my"
    _user.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    _user.text_changed.connect(func(_t): _validate())
    r1.add_child(_user)
    col.add_child(r1)

    col.add_child(UI.label("KATA LALUAN"))
    var r2 := UI.hbox(8)
    r2.add_child(UI.icon("res://assets/icons/kunci.png", 40))
    _pass = LineEdit.new()
    _pass.secret = true
    _pass.secret_character = "*"
    _pass.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    _pass.text_changed.connect(func(_t): _validate())
    _pass.text_submitted.connect(func(_t): _do_login())
    r2.add_child(_pass)
    col.add_child(r2)

    var forgot := LinkButton.new()
    forgot.text = "Lupa kata laluan?"
    forgot.size_flags_horizontal = Control.SIZE_SHRINK_END
    forgot.pressed.connect(_forgot)
```

```
    col.add_child(forgot)

    _status = UI.label("", "Bad", true)
    _status.custom_minimum_size.x = 480
    col.add_child(_status)

    _login = UI.button("LOG MASUK", "Primary")
    _login.disabled = true
    _login.pressed.connect(_do_login)
    col.add_child(_login)
    var reg := UI.button("DAFTAR AKAUN BAHARU")
    reg.pressed.connect(func(): Game.goto("register"))
    col.add_child(reg)
    var guest := UI.button("Main sebagai Tetamu", "Flat")
    guest.pressed.connect(_guest)
    col.add_child(guest)
    var mode := UI.label(Backend.mode_text(), "Small", true)
    mode.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    mode.custom_minimum_size.x = 480
    col.add_child(mode)
    frame.add_child(col)
    add_child(UI.center(frame))
    var logo := UI.label("EXPLO-STORY", "Logo")
    logo.position = Vector2(30, 20)
    add_child(logo)
    _user.grab_focus()


func _validate() -> void:
    _login.disabled = _user.text.strip_edges() == "" or _pass.text == ""
    _user.theme_type_variation = ""
    _pass.theme_type_variation = ""
    _status.text = ""


func _do_login() -> void:
    if _login.disabled:
        return
    _login.disabled = true
    _status.theme_type_variation = "Small"
    _status.text = "Sedang log masuk..."
    var r: Dictionary = await Backend.sign_in(_user.text.strip_edges(), _pass.text)
    if not r.get("ok", false):
        _status.theme_type_variation = "Bad"
        _status.text = str(r.get("error", "Log masuk gagal."))
        _user.theme_type_variation = "Invalid"
        _pass.theme_type_variation = "Invalid"
        _login.disabled = false
        return
    var mode := "online" if Backend.is_online() else "local"
    Game.load_profile(str(r["username"]), mode, str(r.get("role", "student")), str(r.get("avatar", "pelajar_1")), str(r
.get("email", "")))
    if r.has("cloud_save") and r["cloud_save"] is Dictionary and r["cloud_save"].has("profile"):
        Game.save = r["cloud_save"]
        Game._ensure_fields()
    if Backend.can_play_online():
        Backend.sync_player_code()  # friends add you by the server's Player ID
    _enter_game()


func _guest() -> void:
    Game.load_profile("Tetamu", "guest")
    _enter_game()


func _enter_game() -> void:
    if bool(Game.save.get("prolog_done", false)):
        Game.goto("arka")
    else:
        Game.goto("prolog")


func _forgot() -> void:
    var e := _user.text.strip_edges()
    if not e.contains("@"):
        _status.theme_type_variation = "Small"
        _status.text = "Taip e-mel anda dahulu, kemudian ketik \"Lupa kata laluan?\"."
        return
    _status.theme_type_variation = "Small"
    _status.text = await Backend.reset_password(e)
```

penutup.gd · scripts/screens/penutup.gd · 191 lines

```
extends Control
## Penutup of a chapter (params.chapter, default 1): its flag waves, the road
## to the next flag lights up, Nota Alif opens, then a teaser for the next Bab.

const UI = preload("res://scripts/ui/ui.gd")
const ORIGIN := Vector2(40, 140)  # map-pixel crop origin (map is drawn x4)

var params: Dictionary = {}
var _road: Line2D
var _ch := 1


## story key for this chapter: Bab 1 keeps its original keys (penutup_a ...)
func _key(part: String) -> String:
    return "penutup_" + part if _ch == 1 else "penutup_%d_%s" % [_ch, part]


func _ready() -> void:
    UI.full(self)
    Game.main.set_pau_visible(false)
    MusicDirector.play("tajuk")
    var bg := TextureRect.new()
    bg.texture = UI.tex("res://assets/backgrounds/bg_peta_jejak_pembelajaran.png")
    bg.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    bg.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    UI.full(bg)
    add_child(bg)
    _ch = int(params.get("chapter", 1))
    var flags: Array = load("res://scripts/ui/peta_jejak.gd").FLAGS
    var f1: Vector2 = flags[_ch - 1] * 4.0
    var f2: Vector2 = flags[mini(_ch, flags.size() - 1)] * 4.0
    _road = Line2D.new()
    _road.width = 26
    _road.default_color = Color(1, 0.85, 0.3, 0.0)
    _road.add_point(f1)
    _road.add_point(f2)
    add_child(_road)
    var flag2 := _img("res://assets/ui/bendera_berkunci.png", f2, 2)
    var flag1 := _img("res://assets/ui/bendera_berkibar.png", f1, 2.5)
    flag1.pivot_offset = Vector2(20, flag1.size.y)
    _img("res://assets/characters32/alif/alif_south.png", f1 + Vector2(-120, 60), 3)
    var pau := _img_pau(f1 + Vector2(-50, 64))
    _run(flag1, flag2, pau)


func _map_to_screen(p: Array) -> Vector2:
    return (Vector2(p[0], p[1]) / 4.0 - ORIGIN) * 4.0


func _img(path: String, feet: Vector2, sc: float) -> TextureRect:
    var r := TextureRect.new()
    r.texture = UI.tex(path)
    r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    r.size = r.texture.get_size() * sc
    r.position = feet - Vector2(r.size.x / 2.0, r.size.y)
    r.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(r)
    return r


func _img_frame(path: String, feet: Vector2, sc: float) -> TextureRect:
    var r := TextureRect.new()
    r.texture = UI.sheet_frame(path, 0, 0)
    r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    r.size = Vector2(16, 24) * sc
    r.position = feet - Vector2(r.size.x / 2.0, r.size.y)
    r.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(r)
    return r


func _img_pau(feet: Vector2) -> TextureRect:
    var r := TextureRect.new()
    r.texture = UI.tex("res://assets/characters32/pau/pau_small.png")
    r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    r.size = Vector2(96, 96)
    r.position = feet - Vector2(48, 96)
    r.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(r)
    return r
```

```
84
85
86  func _run(flag1: TextureRect, flag2: TextureRect, pau: TextureRect) -> void:
87      # chapter complete
88      if not Game.chapter_done(_ch):
89          Game.save["chapters_done"].append(_ch)
90          Game.add_reward(150, 100)
91          Game.persist()
92      await get_tree().create_timer(0.6).timeout
93      var wave := create_tween().set_loops()
94      wave.tween_property(flag1, "scale", Vector2(1.08, 1.0), 0.3)
95      wave.tween_property(flag1, "scale", Vector2(0.94, 1.0), 0.3)
96      var hop := create_tween().set_loops()
97      hop.tween_property(pau, "position:y", pau.position.y - 24, 0.25)
98      hop.tween_property(pau, "position:y", pau.position.y, 0.25)
99      var road := create_tween()
100     road.tween_property(_road, "default_color:a", 0.75, 1.5)
101     create_tween().tween_property(flag2, "modulate", Color(1.3, 1.2, 0.8), 1.5)
102     var banner := UI.label(tr("BENDERA %d BERKIBAR!") % _ch, "Title")
103     banner.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
104     banner.position = Vector2(0, 40)
105     banner.size = Vector2(1280, 100)
106     add_child(banner)
107     MusicDirector.stinger("st_zone")
108     await Game.say(_key("a"))
109     hop.kill()
110     # Nota Alif
111     var nota_panel = load("res://scripts/ui/nota_panel.gd").new()
112     Game.main.overlay_root().add_child(nota_panel)
113     await nota_panel.closed
114     # gentle wellbeing card
115     await _card("Satu nota kecil", "Jika rasa tertekan menjelang peperiksaan, bercakaplah dengan guru, kaunselor atau k
    eluarga. Kau tidak keseorangan.", "res://assets/icons/rakan.png")
116     # teaser for the next Bab
117     banner.queue_free()
118     await _teaser()
119     Game.save["map_pos"] = []
120     Game.persist()
121     Game.goto("peta")
122
123
124 func _card(title: String, text: String, icon_path: String) -> void:
125     var cover := UI.dim(0.55)
126     cover.mouse_filter = Control.MOUSE_FILTER_STOP
127     add_child(cover)
128     var p := UI.panel("Paper")
129     p.custom_minimum_size = Vector2(700, 0)
130     var c := UI.vbox(14)
131     c.add_child(UI.icon(icon_path, 96))
132     var t := UI.label(title, "InkHeading")
133     t.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
134     c.add_child(t)
135     var b := UI.label(text, "Ink", true)
136     b.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
137     c.add_child(b)
138     var row := UI.hbox(12)
139     row.alignment = BoxContainer.ALIGNMENT_CENTER
140     var sp := UI.icon_button("res://assets/icons/suara.png", "Blue", Vector2(64, 56))
141     sp.pressed.connect(func(): Tts.speak(text))
142     row.add_child(sp)
143     var ok := UI.button("Teruskan »", "Primary", 220)
144     row.add_child(ok)
145     c.add_child(row)
146     p.add_child(c)
147     var cc := UI.center(p)
148     add_child(cc)
149     ok.grab_focus()
150     await ok.pressed
151     cover.queue_free()
152     cc.queue_free()
153
154
155 func _teaser() -> void:
156     var dark := ColorRect.new()
157     dark.color = Color(0.05, 0.06, 0.1, 0.0)
158     UI.full(dark)
159     add_child(dark)
160     var tw := create_tween()
161     tw.tween_property(dark, "color:a", 0.94, 1.0)
162     await tw.finished
163     var col := UI.vbox(10)
164     col.alignment = BoxContainer.ALIGNMENT_CENTER
165     var next := Game.chapter_info(_ch + 1)
166     var icon := "res://assets/icons/%s.png" % str(next.get("icon", "bendera"))
167     col.add_child(UI.icon(icon if ResourceLoader.exists(icon) else "res://assets/icons/bendera.png", 120))
```

```
    var a := UI.label("BAB %d" % (_ch + 1), "Title")
    a.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    col.add_child(a)
    var b := UI.label(str(next.get("title", "")).to_upper(), "Heading")
    b.add_theme_font_size_override("font_size", 44 if b.text.length() <= 30 else 30)
    b.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART
    b.custom_minimum_size.x = 1100
    b.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    col.add_child(b)
    var c := UI.label(tr("Sudah dibuka!") if not Game.chapter_zones(_ch + 1).is_empty() else tr("Akan Datang"), "Small"
)
    c.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    col.add_child(c)
    var cc := UI.center(col)
    cc.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(cc)
    col.modulate.a = 0.0
    create_tween().tween_property(col, "modulate:a", 1.0, 1.2)
    await get_tree().create_timer(1.4).timeout
    await Game.say(_key("teaser"))
    var btn := UI.button("KEMBALI MENEROKA", "Primary", 320)
    btn.position = Vector2(480, 600)
    add_child(btn)
    btn.grab_focus()
    await btn.pressed
```

pertarungan.gd · scripts/screens/pertarungan.gd · 417 lines

```
extends Control
## Bilik Pertarungan: the battle. The players stand on a stage at the top,
## big, with their HP under them (tap one to make it your target), the shared question below. Answer right to hit your
target;
## the last player standing wins. params: {"mode": "bot", "bots": 2..4} or
## {"mode": "online", "code": "ABC123"} (online needs Supabase, see Backend).

const UI = preload("res://scripts/ui/ui.gd")
const LETTERS := ["A", "B", "C", "D"]

var params: Dictionary = {}
var battle: Node
var _cards := {}        ## player id -> Fighter
var _cards_row: HBoxContainer
var _round: Label
var _timer: ProgressBar
var _q: Label
var _buttons: Array[Button] = []
var _info: Label
var _last_status := ""
var _last_round := -1
var _result: Control


func _ready() -> void:
    UI.full(self)
    Game.main.set_pau_visible(false)
    MusicDirector.play("melaka")
    MusicDirector.puzzle(true)
    add_child(UI.bg_image("res://assets/backgrounds/bg_zon1_3_balai_penghadapan.png", Color(0.45, 0.42, 0.5)))
    var root := UI.vbox(10)
    UI.full(root)
    var top := UI.hbox(14)
    _round = UI.label("", "Heading")
    top.add_child(_round)
    _timer = ProgressBar.new()
    _timer.show_percentage = false
    _timer.max_value = BattleRules.ROUND_TIME
    _timer.custom_minimum_size = Vector2(0, 18)
    _timer.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    _timer.size_flags_vertical = Control.SIZE_SHRINK_CENTER
    top.add_child(_timer)
    var out := UI.button("Keluar")
    out.pressed.connect(_leave)
    top.add_child(out)
    root.add_child(UI.margin(top, 24, 14, 24, 0))
    _cards_row = UI.hbox(24)
    _cards_row.alignment = BoxContainer.ALIGNMENT_CENTER
    root.add_child(_cards_row)

    # question, answers and the explanation share whatever height the stage
    # leaves; with big text or a long explanation this scrolls instead of
    # pushing the bottom row of answers off the screen
    var scroll := ScrollContainer.new()
    scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
```

```
    scroll.size_flags_vertical = Control.SIZE_EXPAND_FILL
    var lower := UI.vbox(8)
    lower.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    scroll.add_child(lower)
    root.add_child(scroll)
    var qp := UI.panel("Paper")
    var qrow := UI.hbox(10)
    _q = UI.label("", "InkHeading", true)
    _q.custom_minimum_size.x = 1080
    _q.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    qrow.add_child(_q)
    var sp := UI.icon_button("res://assets/icons/suara.png", "Blue", Vector2(60, 52))
    sp.tooltip_text = "Baca soalan"
    sp.pressed.connect(func(): Tts.speak(_q.text))
    qrow.add_child(sp)
    qp.add_child(qrow)
    lower.add_child(UI.margin(qp, 24, 0, 24, 0))
    var grid := GridContainer.new()
    grid.columns = 2
    grid.add_theme_constant_override("h_separation", 12)
    grid.add_theme_constant_override("v_separation", 10)
    for i in 4:
        var b := UI.button("")
        b.custom_minimum_size = Vector2(610, 52)
        b.alignment = HORIZONTAL_ALIGNMENT_LEFT
        b.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART
        b.pressed.connect(_answer.bind(i))
        grid.add_child(b)
        _buttons.append(b)
    lower.add_child(UI.center(grid))
    _info = UI.label("", "", true)
    _info.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    _info.custom_minimum_size.x = 1200
    lower.add_child(UI.margin(UI.center(_info), 0, 0, 0, 12))
    add_child(root)

    if str(params.get("mode", "bot")) == "online":
        battle = OnlineBattle.new()
        add_child(battle)
        battle.state_changed.connect(_on_state)
        battle.join(str(params.get("code", "")))
    else:
        battle = LocalBattle.new()
        add_child(battle)
        battle.setup(int(params.get("bots", BattleRules.MIN_FRIENDS)))
        battle.state_changed.connect(_on_state)
        battle.start()


func _process(_d: float) -> void:
    if battle and battle.state.has("time_left"):
        _timer.value = float(battle.state["time_left"])


# ---------------------------------------------------------------- fighters
func _build_cards(players: Array) -> void:
    for c in _cards_row.get_children():
        c.queue_free()
    _cards.clear()
    for p in players:
        var f := Fighter.new()
        f.setup(p)
        var pid := str(p["id"])
        f.gui_input.connect(func(e: InputEvent):
            if e is InputEventMouseButton and e.pressed and e.button_index == MOUSE_BUTTON_LEFT:
                _pick_target(pid))
        if not p.get("me", false):
            f.mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
            f.tooltip_text = tr("Ketik untuk jadikan sasaran")
        _cards_row.add_child(f)
        _cards[pid] = f


func _pick_target(id: String) -> void:
    if id == battle.my_id:
        return
    battle.set_target(id)
    Sfx.play("select", -6.0)


func _refresh_cards(s: Dictionary) -> void:
    var me: Dictionary = battle.me()
    for p in s["players"]:
        var f: Fighter = _cards.get(str(p["id"]))
        if f == null:
```

```
            continue
        var is_target: bool = me.get("alive", false) and str(me.get("target", "")) == str(p["id"])
        var status := ""
        if not p["alive"]:
            status = tr("TERSINGKIR")
        elif s["status"] == "question" and p.get("answered", false):
            status = tr("sudah jawab")
        f.show_state(int(p["hp"]), p["alive"], is_target, status)


## One player on the battle stage: a name tag, Alif or a classmate drawn big
## and standing on a ring (gold = you, red = your target), the HP bar under
## the feet and a status line. Hits flash red and knock back; knocked-out
## players fall over.
class Fighter extends VBoxContainer:
    const UI = preload("res://scripts/ui/ui.gd")
    const SPRITE := 112.0

    var me := false
    var alive := true
    var target := false
    var _name: Label
    var _tag: Label
    var _stage: Control
    var _sprite: TextureRect
    var _hp: ProgressBar
    var _hp_label: Label
    var _status: Label
    var _t := randf() * TAU
    var _hp_value := -1

    func setup(p: Dictionary) -> void:
        me = p.get("me", false)
        custom_minimum_size = Vector2(210, 0)
        alignment = BoxContainer.ALIGNMENT_CENTER
        add_theme_constant_override("separation", 2)
        mouse_filter = Control.MOUSE_FILTER_STOP
        _tag = UI.label("", "Small")
        _tag.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        _tag.add_theme_color_override("font_color", Color("ff8a7a"))
        _tag.add_theme_color_override("font_outline_color", Color(0.1, 0.06, 0.03))
        _tag.add_theme_constant_override("outline_size", 5)
        add_child(_tag)
        _name = UI.label(str(p["name"]) + (tr(" (kau)") if me else ""), "Heading")
        _name.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        _name.clip_text = true
        _name.custom_minimum_size.x = 200
        _name.add_theme_font_size_override("font_size", 24)
        _name.add_theme_color_override("font_color", UI.GOLD if me else Color.WHITE)
        _name.add_theme_color_override("font_outline_color", Color(0.1, 0.06, 0.03))
        _name.add_theme_constant_override("outline_size", 6)
        add_child(_name)
        _stage = Control.new()
        _stage.custom_minimum_size = Vector2(200, SPRITE + 4)
        _stage.draw.connect(_draw_ring)
        add_child(_stage)
        var folder := str(p.get("skin", "alif"))
        _sprite = UI.icon("res://assets/characters32/%s/%s_south.png" % [folder, folder], SPRITE)
        _sprite.size = Vector2(SPRITE, SPRITE)
        _sprite.pivot_offset = Vector2(SPRITE / 2.0, SPRITE * 0.9)
        _stage.add_child(_sprite)
        _hp = ProgressBar.new()
        _hp.theme_type_variation = "Good"
        _hp.max_value = BattleRules.HP
        _hp.value = BattleRules.HP
        _hp.show_percentage = false
        _hp.custom_minimum_size = Vector2(180, 20)
        _hp.size_flags_horizontal = Control.SIZE_SHRINK_CENTER
        add_child(_hp)
        _hp_label = UI.label("", "Small")
        _hp_label.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        add_child(_hp_label)
        _status = UI.label(" ", "Small")
        _status.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        add_child(_status)
        for c in [_tag, _name, _stage, _sprite, _hp, _hp_label, _status]:
            (c as Control).mouse_filter = Control.MOUSE_FILTER_PASS

    func show_state(hp: int, is_alive: bool, is_target: bool, status: String) -> void:
        if hp != _hp_value:
            _hp_value = hp
            create_tween().tween_property(_hp, "value", float(hp), 0.4)
            _hp.theme_type_variation = "Good" if hp > 50 else ("Warn" if hp > 25 else "")
            _hp_label.text = "HP %d" % hp
        target = is_target and is_alive
```

```
        _tag.text = tr("SASARAN") if target else " "
        _status.text = status if status != "" else " "
        _status.add_theme_color_override("font_color", Color("ff8a7a") if not is_alive else Color("9fe8a0"))
        if alive and not is_alive:
            _fall()
        alive = is_alive
        _stage.queue_redraw()

    ## where the damage number pops up (screen space)
    func head_position() -> Vector2:
        return _sprite.global_position + Vector2(SPRITE / 2.0, 10)

    func hit() -> void:
        _sprite.modulate = Color(1, 0.3, 0.3)
        create_tween().tween_property(_sprite, "modulate", Color.WHITE if alive else Color(0.5, 0.5, 0.5, 0.6), 0.35)
        var k := create_tween()
        for i in 4:
            k.tween_property(_sprite, "position:x", _base_x() + (10.0 if i % 2 == 0 else -10.0), 0.04)
        k.tween_property(_sprite, "position:x", _base_x(), 0.05)

    func _fall() -> void:
        var t := create_tween().set_parallel()
        t.tween_property(_sprite, "rotation_degrees", 90.0, 0.45).set_trans(Tween.TRANS_BACK).set_ease(Tween.EASE_IN)
        t.tween_property(_sprite, "modulate", Color(0.5, 0.5, 0.5, 0.6), 0.45)
        t.tween_property(_name, "modulate:a", 0.5, 0.45)

    func _base_x() -> float:
        return (_stage.size.x - SPRITE) / 2.0

    func _process(delta: float) -> void:
        _t += delta
        if alive:  # breathing
            _sprite.position.y = 2.0 - absf(sin(_t * 2.2)) * 3.0
        else:
            _sprite.position.y = 16.0
        if _sprite.rotation_degrees == 0.0 and not _shaking():
            _sprite.position.x = _base_x()
        if target:
            _stage.queue_redraw()

    func _shaking() -> bool:
        return absf(_sprite.position.x - _base_x()) > 0.5 and _sprite.modulate.g < 0.99

    func _draw_ring() -> void:
        var c := Vector2(_stage.size.x / 2.0, SPRITE * 0.9)
        _stage.draw_set_transform(c, 0, Vector2(1, 0.32))
        _stage.draw_circle(Vector2.ZERO, SPRITE * 0.39, Color(0, 0, 0, 0.35))
        var col := Color(0, 0, 0, 0)
        if not alive:
            col = Color(0.5, 0.5, 0.5, 0.5)
        elif target:
            col = Color(1, 0.35, 0.3, 0.6 + 0.4 * absf(sin(_t * 5.0)))
        elif me:
            col = UI.GOLD
        if col.a > 0:
            _stage.draw_arc(Vector2.ZERO, SPRITE * 0.43, 0, TAU, 48, col, 5.0, true)
        _stage.draw_set_transform(Vector2.ZERO)


# ---------------------------------------------------------------- state
func _on_state(s: Dictionary) -> void:
    if _cards.is_empty() and s.has("players"):
        _build_cards(s["players"])
    var status := str(s.get("status", ""))
    var me: Dictionary = battle.me()
    if status == "question":
        if int(s["round"]) != _last_round:
            _last_round = int(s["round"])
            _new_question(s)
        var answered: bool = int(me.get("answer", -1)) >= 0
        if not me.get("alive", false):
            _info.text = tr("Kau sudah tersingkir. Tengok siapa yang menang!")
        elif answered:
            _info.text = tr("Jawapan dihantar. Menunggu pemain lain...")
        else:
            _info.text = tr("Ketik kad pemain untuk pilih sasaran, kemudian jawab.")
    elif status == "reveal" and _last_status != "reveal":
        _show_reveal(s)
    elif status == "over" and _last_status != "over":
        _show_result(s)
    _round.text = tr("Pusingan %d") % int(s.get("round", 0))
    _refresh_cards(s)
    _last_status = status


```

```
func _new_question(s: Dictionary) -> void:
    var q: Dictionary = s["q"]
    _q.text = str(q["q"])
    var alive: bool = battle.me().get("alive", false)
    for i in _buttons.size():
        var b := _buttons[i]
        b.text = "%s.  %s" % [LETTERS[i], q["options"][i]]
        b.disabled = not alive
        b.theme_type_variation = ""
        b.modulate = Color.WHITE
    Sfx.play("pop", -6.0)
    if bool(Game.settings.get("auto_read", false)):
        Tts.speak(_q.text)


func _answer(i: int) -> void:
    if str(battle.state.get("status", "")) != "question" or int(battle.me().get("answer", -1)) >= 0:
        return
    battle.answer(i)
    for j in _buttons.size():
        _buttons[j].disabled = true
        _buttons[j].theme_type_variation = "Blue" if j == i else ""
    Sfx.play("select")


func _show_reveal(s: Dictionary) -> void:
    var ans := int(s["answer"])
    var mine := int(battle.me().get("answer", -1))
    for i in _buttons.size():
        # the right answer stays bright green (a disabled button would turn grey)
        _buttons[i].disabled = i != ans
        _buttons[i].theme_type_variation = "Primary" if i == ans else ""
        _buttons[i].modulate = Color.WHITE if i == ans or i == mine else Color(1, 1, 1, 0.45)
    var me: Dictionary = battle.me()
    if me.get("alive", false) or mine >= 0:
        _info.text = (tr("Betul! ") if mine == ans else tr("Jawapan betul: %s. ") % LETTERS[ans]) + str(s.get("explain"
, ""))
    else:
        _info.text = str(s.get("explain", ""))
    if mine == ans:
        MusicDirector.stinger("st_correct")
    for h in s.get("hits", []):
        _hit(str(h["to"]), int(h["dmg"]), str(h["from"]) == battle.my_id)


func _hit(id: String, dmg: int, by_me: bool) -> void:
    var f: Fighter = _cards.get(id)
    if f == null:
        return
    var l := UI.label("-%d" % dmg, "Title")
    l.add_theme_color_override("font_color", Color("ff6a5a") if not by_me else UI.GOLD)
    l.add_theme_color_override("font_outline_color", Color(0.1, 0.06, 0.03))
    l.add_theme_constant_override("outline_size", 8)
    add_child(l)
    l.position = f.head_position() - Vector2(30, 20)
    var t := create_tween().set_parallel()
    t.tween_property(l, "position:y", l.position.y - 60, 1.0)
    t.tween_property(l, "modulate:a", 0.0, 1.0).set_delay(0.5)
    t.chain().tween_callback(l.queue_free)
    f.hit()
    Sfx.play_varied("bump", -2.0)
    if id == battle.my_id:
        Main.shake(6.0, 0.3)


func _show_result(s: Dictionary) -> void:
    MusicDirector.puzzle(false)
    var won: bool = s["winner"] == battle.my_id
    var winner := BattleRules.find(s["players"], str(s["winner"]))
    _result = Control.new()
    UI.full(_result)
    _result.add_child(UI.dim(0.6))
    var card := UI.panel("Paper")
    card.custom_minimum_size = Vector2(640, 0)
    var c := UI.vbox(12)
    c.add_child(UI.icon("res://assets/icons/%s.png" % ("mahkota" if won else "bendera"), 96))
    var title := tr("KAU MENANG!") if won else (tr("SERI! Tiada pemenang.") if winner.is_empty() else tr("%s MENANG!")
% str(winner["name"]).to_upper())
    var t := UI.label(title, "InkHeading")
    t.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(t)
    var reward := "+%d EXP" % BattleRules.REWARD_EXP + (("   +%d " % BattleRules.REWARD_COINS) + tr("Syiling") if won e
lse "")
    var r := UI.label(reward, "Ink")
    r.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
```

```
    c.add_child(r)
    var note := UI.label((tr("Bagus! Setiap soalan yang kau jawab ialah ulang kaji %s.") if not won else tr("Syabas! Fakta %s memang melekat dalam kepala kau.")) % BattleRules.chapter_name(), "InkSmall", true)
    note.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(note)
    var row := UI.hbox(12)
    row.alignment = BoxContainer.ALIGNMENT_CENTER
    var again := UI.button("Main lagi", "Primary", 200)
    again.pressed.connect(func(): Game.goto("pertarungan", params))
    row.add_child(again)
    var back := UI.button("Kembali", "", 200)
    back.pressed.connect(func(): Game.goto("lobby"))
    row.add_child(back)
    c.add_child(row)
    card.add_child(c)
    _result.add_child(UI.center(card))
    add_child(_result)
    MusicDirector.stinger("st_levelup" if won else "st_zone")
    if won:
        Game.toast("+%d Syiling" % BattleRules.REWARD_COINS)


func _leave() -> void:
    MusicDirector.puzzle(false)
    if battle:
        battle.leave()
    Game.goto("lobby")
```

register.gd · scripts/screens/register.gd · 157 lines

```
extends Control
## Module 1: Daftar akaun (FR-AUTH-02) with role and starting avatar.

const UI = preload("res://scripts/ui/ui.gd")
const TitleBackdrop = preload("res://scripts/ui/title_backdrop.gd")

var params: Dictionary = {}
var _name: LineEdit
var _email: LineEdit
var _pass: LineEdit
var _pass2: LineEdit
var _go: Button
var _status: Label
var _avatar: String = "pelajar_1"
var _role: String = "student"
var _negeri: OptionButton


func _ready() -> void:
    UI.full(self)
    Game.main.set_pau_visible(false)
    var bd := TitleBackdrop.new()
    bd.tint = Color(0.75, 0.78, 0.8)
    add_child(bd)
    var frame := UI.panel()
    frame.custom_minimum_size = Vector2(640, 0)
    var col := UI.vbox(8)
    var title := UI.label("DAFTAR AKAUN", "Heading")
    title.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    col.add_child(title)
    var grid := GridContainer.new()
    grid.columns = 2
    grid.add_theme_constant_override("h_separation", 14)
    grid.add_theme_constant_override("v_separation", 8)
    _name = _field(grid, "NAMA PENGGUNA", "cth: Alif")
    _email = _field(grid, "E-MEL", "cth: alif@sekolah.edu.my")
    _pass = _field(grid, "KATA LALUAN", "sekurang-kurangnya 6 aksara")
    _pass.secret = true
    _pass.secret_character = "*"
    _pass2 = _field(grid, "SAHKAN KATA LALUAN", "")
    _pass2.secret = true
    _pass2.secret_character = "*"
    col.add_child(grid)

    # the profile picture now comes from Alif's outfit; ask for the negeri
    # instead (Papan Markah ranks by negeri too)
    var neg_row := UI.hbox(10)
    neg_row.alignment = BoxContainer.ALIGNMENT_CENTER
    neg_row.add_child(UI.label("NEGERI", "Small"))
    _negeri = OptionButton.new()
    _negeri.custom_minimum_size = Vector2(320, 48)
    _negeri.add_item(tr("(Pilih negeri)"))
    for n in Game.NEGERI:
```

```
        _negeri.add_item(n)
    neg_row.add_child(_negeri)
    col.add_child(neg_row)

    var roles := UI.hbox(30)
    roles.alignment = BoxContainer.ALIGNMENT_CENTER
    var g := ButtonGroup.new()
    var s := CheckBox.new()
    s.text = "PELAJAR"
    s.button_group = g
    s.button_pressed = true
    s.pressed.connect(func(): _role = "student")
    roles.add_child(s)
    var t := CheckBox.new()
    t.text = "GURU"
    t.button_group = g
    t.pressed.connect(func(): _role = "teacher")
    roles.add_child(t)
    col.add_child(roles)

    _status = UI.label("", "Bad", true)
    _status.custom_minimum_size.x = 600
    col.add_child(_status)
    var row := UI.hbox(12)
    var back := UI.button("« KEMBALI")
    back.pressed.connect(func(): Game.goto("login"))
    row.add_child(back)
    _go = UI.button("DAFTAR", "Primary")
    _go.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    _go.disabled = true
    _go.pressed.connect(_submit)
    row.add_child(_go)
    col.add_child(row)
    var mode := UI.label(Backend.mode_text(), "Small", true)
    mode.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    col.add_child(mode)
    frame.add_child(col)
    add_child(UI.center(frame))
    _name.grab_focus()


func _field(grid: GridContainer, label_text: String, ph: String) -> LineEdit:
    grid.add_child(UI.label(label_text, "Small"))
    var e := LineEdit.new()
    e.placeholder_text = ph
    e.custom_minimum_size.x = 340
    e.text_changed.connect(func(_t): _validate())
    grid.add_child(e)
    return e


func _errors() -> Dictionary:
    var e := {}
    if _name.text.strip_edges().length() < 3:
        e["name"] = "Nama pengguna sekurang-kurangnya 3 aksara."
    var em := _email.text.strip_edges()
    if not (em.contains("@") and em.contains(".") and em.length() >= 6):
        e["email"] = "Format e-mel tidak betul."
    if _pass.text.length() < 6:
        e["pass"] = "Kata laluan sekurang-kurangnya 6 aksara."
    if _pass2.text != _pass.text:
        e["pass2"] = "Kata laluan tidak sepadan."
    return e


func _validate() -> void:
    var e := _errors()
    _go.disabled = not e.is_empty()
    _name.theme_type_variation = "Invalid" if e.has("name") and _name.text != "" else ""
    _email.theme_type_variation = "Invalid" if e.has("email") and _email.text != "" else ""
    _pass.theme_type_variation = "Invalid" if e.has("pass") and _pass.text != "" else ""
    _pass2.theme_type_variation = "Invalid" if e.has("pass2") and _pass2.text != "" else ""
    var shown := ""
    for k in ["name", "email", "pass", "pass2"]:
        var le: LineEdit = {"name": _name, "email": _email, "pass": _pass, "pass2": _pass2}[k]
        if e.has(k) and le.text != "":
            shown = e[k]
            break
    _status.theme_type_variation = "Bad"
    _status.text = shown


func _submit() -> void:
    if not _errors().is_empty():
        return
```

```
    _go.disabled = true
    _status.theme_type_variation = "Small"
    _status.text = "Sedang mendaftar..."
    var r: Dictionary = await Backend.sign_up(_name.text.strip_edges(), _email.text.strip_edges(), _pass.text, _role, _
avatar)
    if not r.get("ok", false):
        _status.theme_type_variation = "Bad"
        _status.text = str(r.get("error", "Pendaftaran gagal."))
        _go.disabled = false
        return
    if r.has("confirm"):
        _status.theme_type_variation = "Good"
        _status.text = str(r["confirm"])
        return
    var mode := "online" if Backend.is_online() else "local"
    Game.load_profile(str(r["username"]), mode, _role, _avatar, str(r.get("email", "")))
    if _negeri.selected > 0:
        Game.save["profile"]["negeri"] = Game.NEGERI[_negeri.selected - 1]
        Game.persist()
    Game.goto("prolog")
```

ujian_bab.gd · scripts/screens/ujian_bab.gd · 192 lines

```
extends Control
## Ujian Bab: the final test of a chapter (params.chapter). The chapter's
## boss stands between Alif and the next map. Random questions from
## data/soalan/bab<n>.json, no timer, 3 hearts; knock the boss's HP to 0 to
## pass. Passing goes on to the penutup (which marks the chapter done);
## failing offers a retry with new questions or a trip back to revise.

const UI = preload("res://scripts/ui/ui.gd")
const Objektif = preload("res://scripts/puzzles/objektif.gd")

var params: Dictionary = {}
var _ch := 1
var _cfg: Dictionary = {}
var _layer: Control
var _tries := 0


func _ready() -> void:
    UI.full(self)
    _ch = int(params.get("chapter", 1))
    _cfg = Game.ujian_def(_ch)
    Game.main.set_pau_visible(true)
    MusicDirector.play("tajuk")
    Game.context_hint = "Ujian Bab %d. Tiada had masa. Ketik aku kalau perlukan petunjuk." % _ch
    add_child(UI.bg_image(str(_cfg.get("background", "")), Color(0.45, 0.4, 0.5)))
    var root := UI.vbox(10)
    UI.full(root)
    var info := Game.chapter_info(_ch)
    root.add_child(UI.header_bar(tr("UJIAN BAB %d") % _ch, str(info.get("title", ""))))
    var mid := CenterContainer.new()
    mid.size_flags_vertical = Control.SIZE_EXPAND_FILL
    var col := UI.vbox(10)
    var boss := TextureRect.new()
    boss.texture = UI.tex("res://assets/characters32/%s/%s_south.png" % [_cfg.get("npc", ""), _cfg.get("npc", "")])
    boss.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    boss.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    boss.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    boss.custom_minimum_size = Vector2(240, 240)
    col.add_child(boss)
    var bn := UI.label(str(_cfg.get("boss", "")).to_upper(), "Title")
    bn.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    bn.add_theme_color_override("font_color", Color("ffb0a0"))
    col.add_child(bn)
    var rule := UI.label(tr("Jawab dengan betul untuk tumbangkan dia. Kau ada 3 nyawa. Tiada had masa."), "", true)
    rule.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    rule.custom_minimum_size.x = 760
    col.add_child(rule)
    mid.add_child(col)
    root.add_child(mid)
    add_child(root)
    _layer = Control.new()
    UI.full(_layer)
    _layer.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(_layer)
    var t := create_tween().set_loops()
    t.tween_property(boss, "modulate", Color(1.25, 0.9, 1.1), 0.8)
    t.tween_property(boss, "modulate", Color.WHITE, 0.8)
    _start.call_deferred()


```

```
func _start() -> void:
    await get_tree().create_timer(0.4).timeout
    await Game.say(str(_cfg.get("before", "")))
    _round()


func _round() -> void:
    _tries += 1
    var def := {
        "type": "objektif",
        "title": tr("Ujian Bab %d: Lawan %s") % [_ch, _cfg.get("boss", "")],
        "instruction": tr("Setiap jawapan betul melemahkan %s. Jawapan salah = hilang satu nyawa, tapi kau akan tahu jawapan betulnya.") % _cfg.get("boss", ""),
        "boss": _cfg.get("boss", ""),
        "hearts": int(_cfg.get("hearts", 3)),
        "need": int(_cfg.get("need", 7)),
        "hint": "Baca semua pilihan dulu. Buang yang kau pasti salah.",
        "done_text": tr("%s tumbang! Bab %d kau dah kuasai.") % [_cfg.get("boss", ""), _ch],
        "questions": Game.ujian_questions(_ch, int(_cfg.get("questions", 12))),
    }
    var p := Objektif.new()
    p.theme = Game.theme
    p.set("def", def)
    p.set("puzzle_id", "ujian_%d" % _ch)
    MusicDirector.puzzle(true)
    _layer.add_child(p)
    Game.active_puzzle = p
    var stats: Dictionary = await p.completed
    Game.active_puzzle = null
    MusicDirector.puzzle(false)
    p.queue_free()
    if stats.get("passed", false):
        await _passed(stats)
    else:
        await _failed(stats)


func _passed(stats: Dictionary) -> void:
    Sfx.play("whoosh")
    MusicDirector.stinger("st_zone")
    var ft := int(stats.get("first_try", 0))
    Game.add_reward(80 + ft * 10, 40)
    Game.toast("+%d EXP   +40 Syiling" % (80 + ft * 10))
    if not Game.save["ujian"].has(_ch):
        Game.save["ujian"].append(_ch)
    Game.persist()
    await Game.say(str(_cfg.get("win", "")))
    Game.goto("penutup", {"chapter": _ch})


func _failed(stats: Dictionary) -> void:
    await Game.say(str(_cfg.get("fail", "")))
    var cover := UI.dim(0.6)
    cover.mouse_filter = Control.MOUSE_FILTER_STOP
    _layer.add_child(cover)
    var card := UI.panel("Paper")
    card.custom_minimum_size = Vector2(940, 0)
    var c := UI.vbox(12)
    var t := UI.label(tr("Belum kali ini"), "InkHeading")
    t.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(t)
    var wrong: Array = stats.get("wrong", [])
    if not wrong.is_empty():
        c.add_child(_weak_topics(wrong))
        c.add_child(_wrong_list(wrong))
    var b := UI.label(tr("Tak apa. Setiap cubaan dapat soalan baru. Baca balik topik di atas dalam Nota Alif, kemudian cuba lagi."), "InkSmall", true)
    b.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(b)
    var row := UI.hbox(12)
    row.alignment = BoxContainer.ALIGNMENT_CENTER
    var back := UI.button(tr("Ulang kaji dulu"), "", 240)
    back.pressed.connect(func(): Game.goto("peta"))
    row.add_child(back)
    var again := UI.button(tr("Cuba lagi »"), "Primary", 240)
    row.add_child(again)
    c.add_child(row)
    card.add_child(c)
    var cc := UI.center(card)
    _layer.add_child(cc)
    again.grab_focus()
    await again.pressed
    cover.queue_free()
    cc.queue_free()
    _round()
```

```

## "Topik yang perlu dibaiki": each mission the player missed questions in,
## most mistakes first, with a button that opens Nota Alif on that page.
func _weak_topics(wrong: Array) -> Control:
    var count := {}
    for w in wrong:
        var z := str(w.get("zon", ""))
        count[z] = int(count.get(z, 0)) + 1
    var zones := count.keys()
    zones.sort_custom(func(a, b): return count[a] > count[b] or (count[a] == count[b] and a < b))
    var box := UI.vbox(6)
    box.add_child(UI.label(tr("Topik yang perlu dibaiki:"), "Ink"))
    for z in zones:
        var row := UI.hbox(12)
        var topic := str(Game.zone_data(z).get("subtopik", tr("Ulang kaji umum"))) if z != "" else tr("Ulang kaji umum"
)
        var l := UI.label("• %s  (%s)" % [topic, tr("%d salah") % count[z]], "Ink", true)
        l.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        row.add_child(l)
        if z != "":
            var nb := UI.icon_button("res://assets/icons/nota.png", "", Vector2(150, 48), tr(" Baca nota"))
            nb.pressed.connect(func(): Game.main.open_nota(z))
            row.add_child(nb)
        box.add_child(row)
    return box


## every question answered wrong: your answer, the right one and why
func _wrong_list(wrong: Array) -> Control:
    var scroll := ScrollContainer.new()
    scroll.custom_minimum_size = Vector2(0, 230)
    scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
    var list := UI.vbox(10)
    list.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    for i in wrong.size():
        var w: Dictionary = wrong[i]
        var item := UI.vbox(2)
        item.add_child(UI.label("%d. %s" % [i + 1, w.get("q", "")], "Ink", true))
        var mine := UI.label(tr("Jawapan kau: %s") % w.get("picked", ""), "InkSmall", true)
        mine.add_theme_color_override("font_color", Color("b03a2e"))
        item.add_child(mine)
        var right := UI.label(tr("Jawapan betul: %s") % w.get("right", ""), "InkSmall", true)
        right.add_theme_color_override("font_color", Color("2e7d32"))
        item.add_child(right)
        if str(w.get("explain", "")) != "":
            item.add_child(UI.label(str(w["explain"]), "InkSmall", true))
        list.add_child(item)
    scroll.add_child(list)
    return scroll
```

zone_prep.gd · scripts/screens/zone_prep.gd · 132 lines

```
extends Control
## "Persiapan Misi": a calm briefing the first time Alif walks into a zone
## (no time pressure, UR-LO-02) plus the Kad Tokoh collection. Shown as an
## overlay in the world; emits `closed`.

signal closed

const UI = preload("res://scripts/ui/ui.gd")

var params: Dictionary = {}
var _zone: Dictionary
var _guide: TextureRect
var _t: float = 0.0


func _ready() -> void:
    UI.full(self)
    mouse_filter = Control.MOUSE_FILTER_STOP
    Game.lock_input()
    Sfx.play("whoosh", -10.0)
    var zid: String = params.get("zone", "1_1")
    _zone = Game.zone_data(zid)
    Game.context_hint = "Baca persoalan misi ini. Tiada had masa. Ketik Mula Meneroka bila sedia."
    add_child(UI.bg_image(str(_zone.get("background", "")), Color(0.55, 0.55, 0.62)))
    var root := UI.vbox(0)
    UI.full(root)
    var ch_no := Game.chapter_of(zid)
    var ch_title := str(Game.chapter_info(ch_no).get("title", "")).to_upper()
    root.add_child(UI.header_bar("PERSIAPAN MISI", "TOPIK: SEJARAH TINGKATAN 4 - BAB %d (%s)" % [ch_no, ch_title]))
    var mid := UI.hbox(30)
```

```
    mid.size_flags_vertical = Control.SIZE_EXPAND_FILL
    var left := Control.new()
    left.custom_minimum_size = Vector2(420, 0)
    var glow := Control.new()
    glow.position = Vector2(210, 330)
    glow.draw.connect(func():
        glow.draw_set_transform(Vector2.ZERO, 0, Vector2(1, 0.35))
        glow.draw_circle(Vector2.ZERO, 110, Color(0.4, 0.85, 1.0, 0.35))
        glow.draw_arc(Vector2.ZERO, 110, 0, TAU, 48, Color(0.5, 0.9, 1.0), 4.0))
    left.add_child(glow)
    var guide_id: String = _zone.get("pemandu", "pau")
    _guide = TextureRect.new()
    _guide.texture = UI.tex("res://assets/characters32/%s/%s_south.png" % [guide_id, guide_id])
    _guide.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    _guide.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    _guide.size = Vector2(192, 192)
    _guide.position = Vector2(114, 140)
    left.add_child(_guide)
    mid.add_child(left)

    var info := UI.panel("Glass")
    info.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    info.size_flags_vertical = Control.SIZE_SHRINK_CENTER
    var ic := UI.vbox(10)
    _row(ic, "MISI:", "Misi %s: %s" % [_zone["no"], _zone["title"]])
    _row(ic, "SUBTOPIK:", str(_zone.get("subtopik", "")))
    _row(ic, "PERSOALAN:", str(_zone.get("persoalan", "")))
    _row(ic, "PEMANDU:", Game.char_name(guide_id))
    var g: Dictionary = _zone.get("ganjaran", {})
    _row(ic, "GANJARAN:", str(g.get("nama", "")))
    ic.get_child(ic.get_child_count() - 1).add_child(UI.icon("res://assets/icons/%s.png" % g.get("icon", "bintang"), 48
))
    var speak := UI.icon_button("res://assets/icons/suara.png", "Blue", Vector2(64, 56), "  Dengar")
    speak.size_flags_horizontal = Control.SIZE_SHRINK_BEGIN
    speak.pressed.connect(func(): Tts.speak("Misi %s, %s. Persoalan: %s" % [_zone["no"], _zone["title"], _zone["persoal
an"]]))
    ic.add_child(speak)
    if Game.zone_done(zid):
        ic.add_child(UI.label("Misi ini sudah selesai. Kau boleh main semula untuk ulang kaji.", "InkSmall", true))
    info.add_child(ic)
    mid.add_child(info)
    root.add_child(UI.margin(mid, 30, 10, 40, 0))

    # Kad Tokoh carousel
    var cards := UI.hbox(12)
    cards.alignment = BoxContainer.ALIGNMENT_CENTER
    for id in Game.characters.keys():
        var ch: Dictionary = Game.characters[id]
        if not ch.get("tokoh", false) or Game.chapter_of(str(ch.get("zon", "1_1"))) != ch_no:
            continue  # only this Bab's figures
        var met: bool = Game.save["figures"].has(id)
        var here: bool = str(ch.get("zon", "")) == zid
        var card := UI.panel("CardSel" if here else ("Card" if met else "CardLocked"))
        card.custom_minimum_size = Vector2(150 if ch_no == 1 else 128, 0)
        var cc := UI.vbox(2)
        var pic := UI.icon(Game.portrait_path(id), 64)  # 64 px portraits: 1:1, crisp
        if not met and not here:
            pic.modulate = Color(0, 0, 0, 0.85)
        cc.add_child(pic)
        var nm := UI.label(str(ch["name"]) if (met or here) else "BERKUNCI", "InkSmall" if (met or here) else "Small",
true)
        nm.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        cc.add_child(nm)
        var st := UI.label("DI MISI INI" if here else ("DITEMUI" if met else "Misi " + str(ch.get("zon", "")).replace("
_", ".")), "Small")
        st.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        st.add_theme_color_override("font_color", Color("2f6a2c") if here or met else UI.CREAM)
        cc.add_child(st)
        card.add_child(cc)
        card.tooltip_text = str(ch.get("lore", ""))
        cards.add_child(card)
    root.add_child(UI.margin(cards, 20, 0, 20, 8))

    var btns := UI.hbox(30)
    btns.alignment = BoxContainer.ALIGNMENT_CENTER
    var go := UI.button("MULA MENEROKA!", "Primary", 320)
    go.pressed.connect(_close)
    btns.add_child(go)
    root.add_child(UI.margin(btns, 0, 4, 0, 20))
    add_child(root)
    go.grab_focus()


func _row(parent: VBoxContainer, k: String, v: String) -> void:
    var r := UI.hbox(10)
```

```
    var kl := UI.label(k, "InkHeading")
    kl.custom_minimum_size.x = 190
    r.add_child(kl)
    var vl := UI.label(v, "Ink", true)
    vl.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    vl.custom_minimum_size.x = 420
    r.add_child(vl)
    parent.add_child(r)


func _close() -> void:
    Sfx.play("select")
    Game.unlock_input()
    closed.emit()
    queue_free()


func _process(delta: float) -> void:
    _t += delta
    if _guide:
        _guide.position.y = 140 - absf(sin(_t * 2.0)) * 8.0
```

zone_summary.gd · scripts/screens/zone_summary.gd · 155 lines

```
extends Control
## Module 5: Ringkasan Misi (post-zone summary, GDS "Post-Match Summary").
## Level, EXP bar, coins, artifact earned and syllabus mastery for the subtopic.

const UI = preload("res://scripts/ui/ui.gd")
const BegPanel = preload("res://scripts/ui/beg_panel.gd")

var params: Dictionary = {}
var _exp_bar: ProgressBar
var _coin_label: Label


func _ready() -> void:
    UI.full(self)
    Game.main.set_pau_visible(true)
    var zid: String = params.get("zone", "1_1")
    var zone := Game.zone_data(zid)
    var st := Game.zone_state(zid)
    Game.context_hint = "Tahniah! Ketik Teruskan untuk kembali ke peta."
    add_child(UI.bg_image(str(zone.get("background", "")), Color(0.5, 0.5, 0.58)))
    var root := UI.vbox(0)
    UI.full(root)
    root.add_child(UI.header_bar("RINGKASAN MISI", "Misi %s: %s" % [zone["no"], zone["title"]]))
    var card := UI.panel("Glass")
    card.custom_minimum_size = Vector2(1000, 0)
    var row := UI.hbox(40)

    # left: level, exp, coins
    var left := UI.vbox(10)
    left.custom_minimum_size.x = 360
    var lvl_panel := UI.panel()
    lvl_panel.add_theme_stylebox_override("panel", UI.sb(UI.WOOD, UI.WOOD_D, 6, 90, 20))
    lvl_panel.custom_minimum_size = Vector2(170, 170)
    lvl_panel.size_flags_horizontal = Control.SIZE_SHRINK_CENTER
    var lc := UI.vbox(0)
    lc.alignment = BoxContainer.ALIGNMENT_CENTER
    var lt := UI.label("TAHAP", "Small")
    lt.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    lc.add_child(lt)
    var ln := UI.label(str(Game.save["level"]), "Title")
    ln.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    lc.add_child(ln)
    lvl_panel.add_child(lc)
    left.add_child(lvl_panel)
    var er := UI.hbox(8)
    er.add_child(UI.label("EXP:", "InkHeading"))
    var ev := UI.label("%d / %d" % [int(Game.save["exp"]) % Game.EXP_PER_LEVEL, Game.EXP_PER_LEVEL], "Ink")
    ev.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    ev.horizontal_alignment = HORIZONTAL_ALIGNMENT_RIGHT
    er.add_child(ev)
    left.add_child(er)
    _exp_bar = ProgressBar.new()
    _exp_bar.theme_type_variation = "Good"
    _exp_bar.show_percentage = false
    _exp_bar.custom_minimum_size = Vector2(0, 24)
    _exp_bar.max_value = Game.EXP_PER_LEVEL
    _exp_bar.value = 0
    left.add_child(_exp_bar)
    if params.get("level_up", false):
```

```
        var up := UI.label("NAIK TAHAP!", "InkHeading")
        up.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        left.add_child(up)
    var cr := UI.hbox(10)
    cr.add_child(UI.icon("res://assets/icons/syiling.png", 64))
    var cc := UI.vbox(0)
    cc.add_child(UI.label("JUMLAH SYILING:", "InkSmall"))
    _coin_label = UI.label("0", "InkHeading")
    _coin_label.add_theme_font_size_override("font_size", 44)
    cc.add_child(_coin_label)
    cr.add_child(cc)
    left.add_child(cr)
    row.add_child(left)

    # right: mastery + artifact
    var right := UI.vbox(12)
    right.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var mt := UI.label("PENGUASAAN SUKATAN KSSM", "InkHeading")
    right.add_child(mt)
    right.add_child(UI.label(str(zone.get("subtopik", "")), "Ink", true))
    var total := maxi(1, int(st.get("total", 0)))
    _bar(right, "Betul cubaan pertama", int(st.get("first_try", 0)), total, "Good")
    _bar(right, "Cubaan semula", int(st.get("retries", 0)), maxi(total, int(st.get("retries", 0))), "Warn")
    right.add_child(UI.label("Petunjuk Pau digunakan: %d" % int(st.get("hints", 0)), "InkSmall"))
    right.add_child(UI.label("Penguasaan: %d%%" % int(Game.mastery_percent(zid)), "Ink"))
    var g: Dictionary = zone.get("ganjaran", {})
    var art := UI.hbox(12)
    art.add_child(UI.icon("res://assets/icons/%s.png" % g.get("icon", "bintang"), 72))
    var at := UI.vbox(2)
    at.add_child(UI.label("GANJARAN: " + str(g.get("nama", "")), "InkHeading"))
    at.add_child(UI.label(str(g.get("info", "")), "InkSmall", true))
    at.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    art.add_child(at)
    right.add_child(art)
    if not params.get("first_time", true):
        right.add_child(UI.label("Main semula untuk ulang kaji: tiada ganjaran kali ini.", "InkSmall"))
    row.add_child(right)
    card.add_child(row)
    var mid := CenterContainer.new()
    mid.size_flags_vertical = Control.SIZE_EXPAND_FILL
    mid.add_child(card)
    root.add_child(mid)

    var btns := UI.hbox(16)
    btns.alignment = BoxContainer.ALIGNMENT_CENTER
    var back := UI.button("« KEMBALI MENEROKA", "", 300)
    back.pressed.connect(_continue)
    btns.add_child(back)
    var nota := UI.icon_button("res://assets/icons/nota.png", "", Vector2(200, 64), " NOTA")
    nota.pressed.connect(func(): Game.main.open_nota())
    btns.add_child(nota)
    var beg := UI.icon_button("", "", Vector2(200, 64), " BEG ALIF")
    beg.icon = BegPanel.icon()
    beg.pressed.connect(func(): Game.main.open_beg())
    btns.add_child(beg)
    var st_b := UI.icon_button("res://assets/icons/tetapan.png", "", Vector2(64, 64))
    st_b.pressed.connect(func(): Game.main.open_settings())
    btns.add_child(st_b)
    var go := UI.button("TERUSKAN »", "Primary", 260)
    go.pressed.connect(_continue)
    btns.add_child(go)
    root.add_child(UI.margin(btns, 0, 0, 0, 24))
    add_child(root)
    go.grab_focus()

    var tw := create_tween()
    tw.tween_property(_exp_bar, "value", float(int(Game.save["exp"]) % Game.EXP_PER_LEVEL), 1.2).set_trans(Tween.TRANS_
QUAD)
    tw.parallel().tween_method(func(v: float): _coin_label.text = str(int(v)), 0.0, float(Game.save["coins"]), 1.2)


func _bar(parent: VBoxContainer, text: String, v: int, mx: int, variation: String) -> void:
    var r := UI.hbox(10)
    var l := UI.label(text, "Ink")
    l.custom_minimum_size.x = 260
    r.add_child(l)
    var b := ProgressBar.new()
    b.theme_type_variation = variation
    b.show_percentage = false
    b.max_value = mx
    b.value = v
    b.custom_minimum_size = Vector2(200, 22)
    b.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    b.size_flags_vertical = Control.SIZE_SHRINK_CENTER
    r.add_child(b)
```

```
    r.add_child(UI.label(str(v), "InkHeading"))
    parent.add_child(r)


func _continue() -> void:
    var zid: String = params.get("zone", "1_1")
    var ch := Game.chapter_of(zid)
    if Game.is_last_zone(zid) and params.get("first_time", true) and not Game.chapter_done(ch):
        # the chapter's final test stands between Alif and the next map
        Game.goto("ujian" if not Game.ujian_def(ch).is_empty() else "penutup", {"chapter": ch})
    else:
        Game.goto("peta")
```

scripts/

`title.gd` · scripts/title.gd · 52 lines

```
extends Control
## Title screen: tap / click / any key to start. The backdrop is
## scripts/ui/title_backdrop.gd (parallax PixelLab layers, or the old picture).

const TitleBackdrop = preload("res://scripts/ui/title_backdrop.gd")

var _t := 0.0
var _leaving := false

@onready var start_label: Label = $StartLabel
@onready var pau: TextureRect = $Pau


func _ready() -> void:
    $Background.visible = false
    var bd := TitleBackdrop.new()
    add_child(bd)
    move_child(bd, 0)
    Main.set_pau_visible(false)
    MusicDirector.play("tajuk")
    var gear := Button.new()
    gear.icon = load("res://assets/icons/tetapan.png")
    gear.expand_icon = true
    gear.custom_minimum_size = Vector2(64, 60)
    gear.position = Vector2(1280 - 84, 20)
    gear.tooltip_text = "Tetapan"
    gear.pressed.connect(func(): Main.open_settings())
    add_child(gear)
    var ver := Label.new()
    ver.text = "Prototaip v0.2  |  Mod Mudah Baca dalam Tetapan"
    ver.theme_type_variation = "Small"
    ver.position = Vector2(20, 684)
    add_child(ver)


func _process(delta: float) -> void:
    _t += delta
    start_label.modulate.a = 0.8 + 0.2 * sin(_t * 3.0)  # stays readable on the bright art
    pau.position.y = 560 - absf(sin(_t * 4.0)) * 24


func _input(event: InputEvent) -> void:
    if _leaving or Game.is_locked():
        return
    if event is InputEventMouseButton and event.position.x > 1180 and event.position.y < 90:
        return  # the settings gear
    var pressed: bool = (event is InputEventMouseButton and event.pressed) \
        or (event is InputEventKey and event.pressed and not event.echo)
    if pressed:
        _leaving = true
        Sfx.play("select")
        Game.goto("login")
```

scripts/ui/

beg_panel.gd · scripts/ui/beg_panel.gd · 186 lines

```
extends "res://scripts/ui/modal.gd"
## Beg Alif: every artifact Alif has found rides along in his school bag.
## One row of pockets per Bab (an artifact slot for each mission) plus the
## Kad Tokoh met in that Bab. Tap a found artifact and Pau tells its story
## (zone "ganjaran.cerita", else its "info"). Tabs switch between open chapters.
## Opened from the world, the bag also holds the Jalan Pintas: tap a place
## already unlocked and Alif is there (world.fast_travel).

const ICON := "res://assets/icons/beg.png"
const TRAVEL := "Jalan Pintas"

var world: Node  ## set when opened in the world: shows the Jalan Pintas tab
var _body: VBoxContainer
var _tabs: HBoxContainer


func _init() -> void:
    title_text = "BEG ALIF"
    panel_size = Vector2(1060, 620)


## the bag icon for menus: assets/icons/beg.png when it exists, else a small
## school bag drawn in pixels (24x24, same size as the other menu icons)
static func icon() -> Texture2D:
    if ResourceLoader.exists(ICON):
        return load(ICON)
    const ROWS := [
        "....####....", "...#....#...", "..########..", ".#BBBBBBBB#.",
        ".#BBBBBBBB#.", ".#BBBBBBBB#.", ".#bbbGGbbb#.", ".#bbbbbbbb#.",
        ".#bbbbbbbb#.", ".#bbbbbbbb#.", ".#bbbbbbbb#.", "..########..",
    ]
    var cols := {"#": Color("1b2a44"), "B": Color("5a8fd6"), "b": Color("3b6fb6"), "G": Color("ffd24a")}
    var img := Image.create(24, 24, false, Image.FORMAT_RGBA8)
    for y in ROWS.size():
        for x in ROWS[y].length():
            if cols.has(ROWS[y][x]):
                img.fill_rect(Rect2i(x * 2, y * 2, 2, 2), cols[ROWS[y][x]])
    return ImageTexture.create_from_image(img)


func _build() -> void:
    var open: Array = []
    for n in range(1, Game.current_chapter() + 1):
        if not Game.chapter_zones(n).is_empty():
            open.append(n)
    if open.size() > 1 or world:
        _tabs = UI.hbox(8)
        for n in open:
            var b := UI.button(tr("Bab %d") % n, "", 130)
            b.pressed.connect(_show.bind(n))
            _tabs.add_child(b)
        if world:
            var t := UI.button(tr(TRAVEL), "Blue", 200)
            t.pressed.connect(_show_travel)
            _tabs.add_child(t)
        content.add_child(_tabs)
    _body = UI.vbox(10)
    _body.size_flags_vertical = Control.SIZE_EXPAND_FILL
    content.add_child(_body)
    _show(open[-1] if not open.is_empty() else 1)


## light up the tab with this text, clear the page
func _page(tab: String) -> void:
    if _tabs:
        for b in _tabs.get_children():
            (b as Button).theme_type_variation = "Primary" if b.text == tab else ("Blue" if b.text == tr(TRAVEL) else "
")
    for c in _body.get_children():
        c.queue_free()


func _show(ch: int) -> void:
    _page(tr("Bab %d") % ch)
    var zones := Game.chapter_zones(ch)
    var big := zones.size() <= 4
    var shelf := GridContainer.new()
    shelf.columns = 4
    shelf.add_theme_constant_override("h_separation", 16)
```

```
    shelf.add_theme_constant_override("v_separation", 10)
    for z in zones:
        var zd := Game.zone_data(z)
        var g: Dictionary = zd.get("ganjaran", {})
        var have: bool = Game.save["artifacts"].has(str(g.get("id", "")))
        var slot := UI.panel("SlotGood" if have else "Slot")
        slot.custom_minimum_size = Vector2(230, 250 if big else 150)
        slot.tooltip_text = ((str(g.get("info", "")) + "
" if not big else "") + tr("Ketik untuk dengar ceritanya")) if have else ""
        if have:
            slot.mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
            slot.gui_input.connect(func(e: InputEvent):
                if e is InputEventMouseButton and e.pressed and e.button_index == MOUSE_BUTTON_LEFT:
                    _tell(g))
        var c := UI.vbox(4 if big else 2)
        var ic := UI.icon("res://assets/icons/%s.png" % g.get("icon", "kunci") if have else "res://assets/icons/kunci.p
ng", 110 if big else 56)
        if not have:
            ic.modulate = Color(1, 1, 1, 0.5)
        c.add_child(ic)
        var n := UI.label(str(g.get("nama", "?")) if have else "???", "Heading" if have and big else "Small", true)
        n.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        c.add_child(n)
        if big or not have:
            var info := UI.label(str(g.get("info", "")) if have else tr("Selesaikan Misi %s") % str(zd.get("no", "")),
"Small", true)
            info.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
            c.add_child(info)
        slot.add_child(c)
        shelf.add_child(slot)
    _body.add_child(UI.center(shelf))
    _body.add_child(UI.label("Kad Tokoh", "Heading"))
    var row := UI.hbox(10)
    for id in Game.characters.keys():
        var ch_def: Dictionary = Game.characters[id]
        if not ch_def.get("tokoh", false) or Game.chapter_of(str(ch_def.get("zon", "1_1"))) != ch:
            continue
        var met: bool = Game.save["figures"].has(id)
        var card := UI.panel("Card" if met else "CardLocked")
        card.custom_minimum_size = Vector2(118 if not big else 140, 0)
        card.tooltip_text = str(ch_def.get("lore", "")) if met else tr("Temui tokoh ini di Misi %s") % str(ch_def.get("
zon", "")).replace("_", ".")
        var cc := UI.vbox(2)
        var pic := UI.icon(Game.portrait_path(id), 64)  # 64 px portraits: 1:1, crisp
        if not met:
            pic.modulate = Color(0, 0, 0, 0.85)
        cc.add_child(pic)
        var nm := UI.label(str(ch_def["name"]) if met else "Berkunci", "InkSmall" if met else "Small", true)
        nm.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        cc.add_child(nm)
        card.add_child(cc)
        row.add_child(card)
    _body.add_child(row)


## the artifact's story, told over the bag
func _tell(g: Dictionary) -> void:
    if Main.is_dialogue_open():
        return
    Sfx.play("chime", -8.0)
    var lines: Array = g.get("cerita", [])
    if lines.is_empty():
        lines = [{"s": "pau", "t": str(g.get("info", ""))}]
    await Game.say(lines)


## Jalan Pintas: a card per place on the map (data/map_legend.json); the ones
## unlocked have PERGI, the rest say what to finish first
func _show_travel() -> void:
    _page(tr(TRAVEL))
    _body.add_child(UI.label("Pilih tempat yang dah dibuka. Alif terus sampai ke sana.", "Small", true))
    var scroll := ScrollContainer.new()
    scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
    scroll.size_flags_vertical = Control.SIZE_EXPAND_FILL
    var grid := GridContainer.new()
    grid.columns = 3
    grid.add_theme_constant_override("h_separation", 12)
    grid.add_theme_constant_override("v_separation", 10)
    for t in world.travel_places():
        var p: Dictionary = t["place"]
        var ok: bool = t["ok"]
        var card := UI.panel("SlotGood" if ok else "Slot")
        card.custom_minimum_size = Vector2(320, 0)
        var row := UI.hbox(10)
        var col := MapView.status_color(MapView.zone_status(str(p["zone"]))) if p.has("zone") else UI.CREAM
```

```
        var sym: Control = MapView.Symbol.new("flag", col) if p.has("zone") else MapView.Symbol.new("badge", Color.WHITE, UI.tex(str(p["icon"])))
        sym.custom_minimum_size = Vector2(36, 36)
        row.add_child(sym)
        var txt := UI.vbox(0)
        txt.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        txt.add_child(UI.label(str(p["name"]), "", true))
        txt.add_child(UI.label(str(p.get("info", "")) if ok else str(t["why"]), "Small", true))
        row.add_child(txt)
        if ok:
            var go := UI.button("PERGI", "Primary", 96)
            go.size_flags_vertical = Control.SIZE_SHRINK_CENTER
            go.pressed.connect(_travel.bind(p))
            row.add_child(go)
        else:
            card.modulate = Color(1, 1, 1, 0.55)
        card.add_child(row)
        grid.add_child(card)
    scroll.add_child(grid)
    _body.add_child(scroll)


func _travel(p: Dictionary) -> void:
    var w := world
    close()
    if not w.fast_travel(p):
        Game.toast(tr("Tak boleh pindah sekarang. Cuba lagi sekejap."))
```

brawl_guide.gd · scripts/ui/brawl_guide.gd · 136 lines

```
extends RefCounted
## "Cara Main" for the Gelanggang Pesta Silat: how a round works, the tips,
## and the controls for whatever the player is using (keyboard layout picked
## in the pause menu, or the on-screen fight buttons on a phone).
## Used by the pause menu, the first-match intro and the lobby.

const UI = preload("res://scripts/ui/ui.gd")

## [action, what it does]
const MOVES := [
    ["attack_punch", "Tumbuk"],
    ["attack_kick", "Sepak"],
    ["jump", "Lompat (tekan Sepak masa di udara = tendangan terbang)"],
    ["grab", "Ambil senjata / baling senjata"],
    ["block", "Tangkis (tahan) - kurangkan kesakitan"],
    ["special", "Jurus Kilat (hanya bila dapat JURUS!)"],
    ["run", "Lari (tahan sambil bergerak)"],
]
## the on-screen buttons, same names as touch_controls.gd
const TOUCH := [
    ["Tumbuk", "Butang kuning paling besar"],
    ["Sepak", "Butang oren"],
    ["Lompat", "Butang biru"],
    ["Ambil", "Butang hijau: ambil / baling senjata"],
    ["Tangkis", "Butang kelabu: tahan untuk menangkis"],
    ["Jurus", "Butang ungu: Jurus Kilat (bila dapat JURUS!)"],
]


static func on_touch() -> bool:
    return Game.controls_on_screen()


## the whole guide; scroll it if the space is short
static func build() -> Control:
    var col := UI.vbox(10)
    col.add_child(_heading("Setiap pusingan ada 3 langkah"))
    for s in [
        ["1. SOALAN (15 saat)", "Semua orang jawab soalan yang sama. Gelanggang berhenti masa ni. Tekan 1, 2, 3 atau 4, atau ketik jawapan."],
        ["2. KEPUTUSAN", "Jawapan betul = SEMANGAT: serangan penuh dan sebuah senjata sekolah jatuh dekat kau. Salah = LEMAH: serangan kau tinggal 30% sahaja. Paling cepat betul = JURUS KILAT."],
        ["3. AKSI! (10 saat)", "Bertarung! Pasukan yang semua ahlinya pengsan, kalah. Selepas 8 pusingan, pasukan dengan nyawa paling banyak menang."],
    ]:
        col.add_child(_point(s[0], s[1]))
    col.add_child(_heading("Kawalan skrin sentuh" if on_touch() else "Kawalan papan kekunci"))
    if on_touch():
        col.add_child(_point("Bergerak", "D-pad atau joystick di sebelah kiri"))
        for t in TOUCH:
            col.add_child(_point(t[0], t[1]))
    else:
        col.add_child(_point(Game.brawl_key_names("move_up") + " " + Game.brawl_key_names("move_left") + " " + Game.bra
```

```
wl_key_names("move_down") + " " + Game.brawl_key_names("move_right"), "Bergerak"))
        for m in MOVES:
            col.add_child(_point(Game.brawl_key_names(m[0]), m[1]))
        col.add_child(_point("Esc / P", "Jeda permainan"))
    col.add_child(_heading("Tip"))
    for t in [
        "Jawab dengan betul dulu. Semangat buat serangan kau 3 kali lebih kuat.",
        "Kutip senjata yang jatuh (Ambil). Pukul dengan senjata atau baling ke arah lawan.",
        "Lawan menyerang? Tahan Tangkis. Kau hanya rasa sedikit sakit.",
        "Kombo: Tumbuk, Tumbuk, Sepak dengan cepat.",
        "Baru main? Pilih tahap lawan Mudah dan pasukan 1 lwn 1.",
    ]:
        var l := UI.label("•  " + _t(t), "Ink", true)
        l.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        col.add_child(l)
    return col


static func _heading(t: String) -> Label:
    var h := UI.label(_t(t), "InkHeading")
    h.add_theme_color_override("font_color", Color("8a4b14"))
    return h


## a key chip (or step name) on the left, what it does on the right
static func _point(key: String, what: String) -> Control:
    var row := UI.hbox(12)
    var chip := PanelContainer.new()
    chip.add_theme_stylebox_override("panel", UI.sb(Color("3a2a1c"), Color("c99a4a"), 2, 6, 6))
    chip.custom_minimum_size.x = 230
    var k := UI.label(_t(key), "Small", true)
    k.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    k.add_theme_color_override("font_color", Color("ffe7a8"))
    chip.add_child(k)
    row.add_child(chip)
    var l := UI.label(_t(what), "Ink", true)
    l.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    row.add_child(l)
    return row


## a Paper card with the guide in a scroll box, a button named "Ok" for the
## caller to connect, and (on a keyboard) the layout switch. live = the fight
## keys are bound right now (in the Gelanggang), so a switch applies at once.
static func card(button_text: String, live := false) -> PanelContainer:
    var card := UI.panel("Paper")
    card.custom_minimum_size = Vector2(980, 0)
    var c := UI.vbox(12)
    var t := UI.label(_t("CARA MAIN: GELANGGANG PESTA SILAT"), "InkHeading")
    t.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    c.add_child(t)
    var scroll := ScrollContainer.new()
    scroll.custom_minimum_size = Vector2(0, 470)
    scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
    var holder := UI.vbox(0)
    holder.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    holder.add_child(build())
    scroll.add_child(holder)
    c.add_child(scroll)
    var row := UI.hbox(12)
    row.alignment = BoxContainer.ALIGNMENT_CENTER
    if not on_touch():
        row.add_child(UI.label(_t("Kekunci:"), "Ink"))
        for s in [["wasd", "W A S D + J K L"], ["panah", "Anak panah + Z X C"]]:
            var b := UI.button(s[1], "Primary" if Game.brawl_scheme() == s[0] else "", 230)
            b.pressed.connect(func():
                Game.set_setting("brawl_keys", s[0])
                if live:
                    Game.use_brawl_keys(true)
                for x in row.get_children():
                    if x is Button and x.has_meta("scheme"):
                        x.theme_type_variation = "Primary" if x.get_meta("scheme") == s[0] else ""
                for ch in holder.get_children():
                    ch.queue_free()
                holder.add_child(build()))
            b.set_meta("scheme", s[0])
            row.add_child(b)
    var ok := UI.button(_t(button_text), "Primary", 220)
    ok.name = "Ok"
    row.add_child(ok)
    c.add_child(row)
    card.add_child(c)
    return card


```

```
static func _t(s: String) -> String:
    return TranslationServer.translate(s)
```

calm_overlay.gd · scripts/ui/calm_overlay.gd · 76 lines

```
extends Control
## Calm moment: Pau slowly grows and shrinks; the player can breathe along.
## A comfort feature in the game only, not a treatment.

const UI = preload("res://scripts/ui/ui.gd")
const CYCLE := 8.0  # 4 s in, 4 s out

var _pau: TextureRect
var _text: Label
var _t: float = 0.0
var _cycles: int = 0


func _ready() -> void:
    UI.full(self)
    mouse_filter = Control.MOUSE_FILTER_STOP
    Game.lock_input()
    var bg := ColorRect.new()
    bg.color = Color(0.1, 0.13, 0.22, 0.92)
    UI.full(bg)
    bg.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(bg)
    _pau = TextureRect.new()
    _pau.texture = UI.sheet_frame("res://assets/proto/sprites/pau.png", 3, 0, 16, 16)
    _pau.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    _pau.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    _pau.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    _pau.size = Vector2(240, 240)
    _pau.pivot_offset = Vector2(120, 120)
    _pau.position = Vector2(640 - 120, 300 - 120)
    _pau.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(_pau)
    _text = UI.label("Tarik nafas...", "Title")
    _text.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    _text.position = Vector2(0, 470)
    _text.size = Vector2(1280, 80)
    _text.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(_text)
    var sub := UI.label("Bernafas ikut Pau. Ketik di mana-mana untuk kembali.", "Small")
    sub.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    sub.position = Vector2(0, 570)
    sub.size = Vector2(1280, 40)
    sub.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(sub)
    Tts.speak("Tarik nafas perlahan-lahan bersama Pau.")


func _process(delta: float) -> void:
    _t += delta
    var phase := fmod(_t, CYCLE) / CYCLE
    var s: float
    if phase < 0.5:
        s = lerpf(0.8, 1.3, ease(phase * 2.0, -2.0))
        _text.text = "Tarik nafas..."
    else:
        s = lerpf(1.3, 0.8, ease((phase - 0.5) * 2.0, -2.0))
        _text.text = "Hembus perlahan..."
    _pau.scale = Vector2(s, s)


func _gui_input(event: InputEvent) -> void:
    if event is InputEventMouseButton and event.pressed:
        _close()
        accept_event()


func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("menu") or event.is_action_pressed("interact"):
        _close()
        get_viewport().set_input_as_handled()


func _close() -> void:
    Game.unlock_input()
    Tts.stop()
    queue_free()
```

dialogue_box.gd · scripts/ui/dialogue_box.gd · 270 lines

```
extends Control
## NPC / Pau dialogue box with portrait, typewriter text and a read-aloud
## button on every line. Cream background, short lines, no time limit.

const UI = preload("res://scripts/ui/ui.gd")
## name plate colour per speaker (others: wood brown)
const PLATE := {
    "alif": Color("3b6fb6"), "pau": Color("d98a2b"), "nirbana": Color("5b3a7a"),
    "panglima": Color("9c2f2f"), "gabenor": Color("6b6b6b"),
}
const PORTRAIT_BOX := 144.0
## speakers drawn on the left (the player's side)
const LEFT_SIDE := ["alif", "pau"]

signal finished

## Animal Crossing-style voice blips while text types: pitch per speaker
const VOICE := {
    "alif": 1.0, "pau": 1.7, "penjaga": 0.62, "tunperak": 0.66, "syahbandar": 1.18,
    "sapurba": 0.78, "demang": 0.74, "makcik": 1.3, "nelayan": 0.88, "chengho": 0.84,
    "mak": 1.22, "aisyah": 1.4, "ravi": 0.94, "meiling": 1.5, "daniel": 1.08,
    "guru_silat": 0.7, "pesilat_amir": 1.12, "pesilat_hafiz": 1.02,
}

## cutscenes (the intro): each line moves on by itself once there has been
## time to read it; a tap still goes faster
var auto_advance := false
var _hold := 0.0

var _lines: Array = []
var _i: int = 0
var _typing: bool = false
var _shown: float = 0.0
var _panel: PanelContainer
var _portrait: TextureRect
var _portrait_frame: PanelContainer
var _portrait_r: TextureRect
var _portrait_r_frame: PanelContainer
var _plate: PanelContainer
var _more: Control
var _name: Label
var _text: Label
var _next: Button
var _speak: Button
var _tag: Label
var _who := ""
var _last_who := ""
var _blip_at := 0


func _ready() -> void:
    UI.full(self)
    visible = false
    mouse_filter = Control.MOUSE_FILTER_STOP
    var shade := ColorRect.new()
    shade.color = Color(0, 0, 0, 0.18)
    UI.full(shade)
    shade.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(shade)

    _panel = UI.panel("Paper")
    _panel.set_anchors_preset(Control.PRESET_BOTTOM_WIDE)
    _panel.offset_left = 40
    _panel.offset_right = -150
    _panel.offset_top = -230
    _panel.offset_bottom = -20
    add_child(_panel)
    var row := UI.hbox(18)
    _panel.add_child(row)

    # portrait on the left for Alif and Pau, on the right for everyone else,
    # so it is always clear who is talking
    _portrait_frame = _make_frame()
    _portrait = _portrait_frame.get_child(0)
    row.add_child(_portrait_frame)
    _portrait_r_frame = _make_frame()
    _portrait_r = _portrait_r_frame.get_child(0)

    var col := UI.vbox(6)
    col.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var name_row := UI.hbox(12)
    _plate = PanelContainer.new()
    _name = UI.label("", "Heading")
```

```
    _name.add_theme_color_override("font_color", Color.WHITE)
    _plate.add_child(_name)
    name_row.add_child(_plate)
    _tag = UI.label("", "InkSmall")
    name_row.add_child(_tag)
    col.add_child(name_row)
    _text = UI.label("", "Ink", true)
    _text.size_flags_vertical = Control.SIZE_EXPAND_FILL
    _text.custom_minimum_size = Vector2(560, 110)
    col.add_child(_text)
    _more = MoreArrow.new()
    _more.custom_minimum_size = Vector2(20, 14)
    _more.size_flags_horizontal = Control.SIZE_SHRINK_END
    col.add_child(_more)
    row.add_child(col)
    row.add_child(_portrait_r_frame)

    var btns := UI.vbox(10)
    _speak = UI.icon_button("res://assets/icons/suara.png", "Blue")
    _speak.tooltip_text = "Baca dengan suara"
    _speak.pressed.connect(_on_speak)
    btns.add_child(_speak)
    _next = UI.button("»", "Primary")
    _next.custom_minimum_size = Vector2(72, 64)
    _next.tooltip_text = "Seterusnya"
    _next.pressed.connect(_advance)
    btns.add_child(_next)
    row.add_child(btns)


func _enter_tree() -> void:
    if not Game.skin_changed.is_connected(_on_skin):
        Game.skin_changed.connect(_on_skin)


func _on_skin() -> void:
    if visible and _who == "alif":
        _set_portrait(_portrait, UI.tex(Game.portrait_path("alif", str(_lines[_i].get("e", "")))))


func play(lines_arr: Array) -> void:
    _lines = lines_arr
    _i = 0
    visible = true
    _show_line()
    await finished


func _show_line() -> void:
    var line: Dictionary = _lines[_i]
    var who: String = str(line.get("s", "narator"))
    _who = who
    _blip_at = 0
    _name.text = Game.char_name(who)
    _plate.visible = _name.text != ""
    _plate.add_theme_stylebox_override("panel", UI.sb(PLATE.get(who, UI.WOOD), PLATE.get(who, UI.WOOD).darkened(0.35),
3, 6, 4))
    _tag.text = "(dialog rekaan)" if line.get("rekaan", false) else ""
    var p := Game.portrait_path(who, str(line.get("e", "")))
    var left := who in LEFT_SIDE
    var frame := _portrait_frame if left else _portrait_r_frame
    _set_portrait(_portrait if left else _portrait_r, UI.tex(p))
    _portrait_frame.visible = p != "" and left
    _portrait_r_frame.visible = p != "" and not left
    if p != "" and (who != _last_who or _i == 0):
        frame.pivot_offset = frame.size / 2.0
        frame.scale = Vector2(0.9, 0.9)
        create_tween().tween_property(frame, "scale", Vector2.ONE, 0.18).set_trans(Tween.TRANS_BACK).set_ease(Tween.EAS
E_OUT)
    _last_who = who
    _text.text = str(line.get("t", ""))
    if who == "narator":
        _text.theme_type_variation = "InkSmall"
        _text.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    else:
        _text.theme_type_variation = "Ink"
        _text.horizontal_alignment = HORIZONTAL_ALIGNMENT_LEFT
    var spd := Game.type_speed()
    if spd <= 0.0:
        _text.visible_characters = -1
        _typing = false
    else:
        _text.visible_characters = 0
        _shown = 0.0
        _typing = true
```

```
    if bool(Game.settings.get("auto_read", false)):
        Tts.speak(_text.text)
    _hold = 1.2 + _text.text.length() * 0.045  # time to read it after it has typed out
    _next.grab_focus()


func _process(delta: float) -> void:
    _more.visible = visible and not _typing
    if not _typing:
        if auto_advance and visible:
            _hold -= delta
            if _hold <= 0.0:
                _advance()
        return
    _shown += delta * Game.type_speed()
    _text.visible_characters = int(_shown)
    if VOICE.has(_who) and _text.visible_characters >= _blip_at + 3:
        _blip_at = _text.visible_characters
        var ch := _text.text.substr(_text.visible_characters - 1, 1)
        if ch.strip_edges() != "" and not bool(Game.settings.get("auto_read", false)):
            Sfx.play("blip", -10.0, VOICE[_who] * randf_range(0.94, 1.06), "Voice")
    if _text.visible_characters >= _text.text.length():
        _text.visible_characters = -1
        _typing = false


func _advance() -> void:
    if _typing:
        _typing = false
        _text.visible_characters = -1
        return
    _i += 1
    if _i >= _lines.size():
        visible = false
        Tts.stop()
        finished.emit()
    else:
        _show_line()


## close immediately (used by Skip buttons)
func cancel() -> void:
    if not visible:
        return
    _typing = false
    visible = false
    Tts.stop()
    finished.emit()


func _on_speak() -> void:
    Tts.speak(_text.text)


func _gui_input(event: InputEvent) -> void:
    if event is InputEventMouseButton and event.pressed and event.button_index == MOUSE_BUTTON_LEFT:
        _advance()
        accept_event()


func _unhandled_input(event: InputEvent) -> void:
    if not visible:
        return
    if event.is_action_pressed("interact"):
        _advance()
        get_viewport().set_input_as_handled()


## portraits keep whole-number pixels: the largest integer scale that fits
## the 144 px slot (64 px art -> 128, 48 px art -> 144)
func _set_portrait(r: TextureRect, tex: Texture2D) -> void:
    r.texture = tex
    if tex:
        var side := maxf(tex.get_width(), tex.get_height())
        r.custom_minimum_size = Vector2.ONE * side * maxf(1.0, floorf(PORTRAIT_BOX / side))


func _make_frame() -> PanelContainer:
    var f := UI.panel("Slot")
    f.custom_minimum_size = Vector2(160, 160)
    var t := TextureRect.new()
    t.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    t.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    t.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    t.custom_minimum_size = Vector2(PORTRAIT_BOX, PORTRAIT_BOX)
```

```
    t.size_flags_horizontal = Control.SIZE_SHRINK_CENTER
    t.size_flags_vertical = Control.SIZE_SHRINK_CENTER
    f.add_child(t)
    return f


## Blinking down arrow: the line is fully shown, tap to go on.
class MoreArrow extends Control:
    var _t := 0.0

    func _process(delta: float) -> void:
        _t += delta
        queue_redraw()

    func _draw() -> void:
        if fmod(_t, 0.9) > 0.6:
            return
        var y := sin(_t * 6.0) * 2.0
        draw_colored_polygon(PackedVector2Array([Vector2(2, y + 2), Vector2(18, y + 2), Vector2(10, y + 12)]), Color(0.
45, 0.27, 0.12))
```

hud.gd · scripts/ui/hud.gd · 225 lines

```
extends Control
## HUD (Module 2): top-left location + a Menu button that drops down a list
## (Peta, Nota, Beg Alif + Jalan Pintas, Tenang, Papan Markah, Tetapan), top-right player card
## with the head of Alif's current outfit, level, EXP bar and coins.

const UI = preload("res://scripts/ui/ui.gd")
const BegPanel = preload("res://scripts/ui/beg_panel.gd")

var location_title: String = ""
var location_sub: String = ""
var show_map_button: bool = false
var world: Node = null  ## overworld: adds the Peta (map) and Tenang (calm) buttons
var exit_action: Callable  ## rooms: shows a Keluar button that calls this

var _level: Label
var _exp: ProgressBar
var _coins: Label
var _title: Label
var _name: Label
var _avatar: TextureRect
var _sub: Label
var _menu_btn: Button
var _menu: PanelContainer   ## the expanded list under the Menu button
var _catcher: Control       ## full-screen, closes the menu when tapping outside


func _ready() -> void:
    UI.full(self)
    theme = Game.theme
    mouse_filter = Control.MOUSE_FILTER_IGNORE

    # top-left: location + menu
    var left := UI.vbox(8)
    left.position = Vector2(16, 14)
    var loc := UI.panel()
    var lc := UI.vbox(0)
    _title = UI.label(location_title, "Heading")
    lc.add_child(_title)
    _sub = UI.label(location_sub, "Small")
    _sub.visible = location_sub != ""
    lc.add_child(_sub)
    loc.add_child(lc)
    left.add_child(loc)
    _menu_btn = UI.button("  Menu", "", 150)
    _menu_btn.icon = _bars_icon()
    _menu_btn.custom_minimum_size.y = 56
    _menu_btn.size_flags_horizontal = Control.SIZE_SHRINK_BEGIN
    _menu_btn.tooltip_text = tr("Buka menu")
    _menu_btn.pressed.connect(func(): set_menu_open(not _menu.visible))
    if exit_action.is_valid():  # rooms: a Keluar button next to Menu
        var btn_row := UI.hbox(10)
        btn_row.add_child(_menu_btn)
        var out := UI.button(tr("Keluar"), "", 150)
        out.custom_minimum_size.y = 56
        out.tooltip_text = tr("Keluar dari bilik ini")
        out.pressed.connect(func():
            if not Game.is_locked():
                set_menu_open(false)
                exit_action.call())
        btn_row.add_child(out)
```

```
        left.add_child(btn_row)
    else:
        left.add_child(_menu_btn)
    add_child(left)
    _build_menu(left)

    # top-right: player card
    var card := UI.panel()
    card.position = Vector2(1280 - 16 - 360, 14)
    # tap the card: Profil & Kedai
    card.mouse_filter = Control.MOUSE_FILTER_STOP
    card.mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
    card.tooltip_text = tr("Profil & Kedai")
    card.gui_input.connect(func(e: InputEvent):
        if e is InputEventMouseButton and e.pressed and e.button_index == MOUSE_BUTTON_LEFT and not Game.is_locked():
            Game.main.open_profile())
    card.custom_minimum_size = Vector2(360, 0)
    var row := UI.hbox(12)
    var av := UI.panel("Slot")
    av.add_theme_stylebox_override("panel", UI.sb(UI.NAVY2, UI.GOLD, 3, 6, 4))
    _avatar = UI.head_icon(Game.skin_folder(), 76)
    av.add_child(_avatar)
    row.add_child(av)
    var info := UI.vbox(4)
    info.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var name_l := UI.label("", "Small")
    _name = name_l
    name_l.clip_text = true
    info.add_child(name_l)
    _level = UI.label("", "Heading")
    info.add_child(_level)
    var exp_row := UI.hbox(6)
    var el := UI.label("EXP", "Small")
    el.add_theme_color_override("font_color", Color("7cc4ff"))
    exp_row.add_child(el)
    _exp = ProgressBar.new()
    _exp.show_percentage = false
    _exp.custom_minimum_size = Vector2(150, 16)
    _exp.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    _exp.size_flags_vertical = Control.SIZE_SHRINK_CENTER
    _exp.max_value = Game.EXP_PER_LEVEL
    exp_row.add_child(_exp)
    info.add_child(exp_row)
    var coin_row := UI.hbox(6)
    coin_row.add_child(UI.icon("res://assets/icons/syiling.png", 26))
    _coins = UI.label("", "Small")
    _coins.add_theme_color_override("font_color", UI.GOLD)
    coin_row.add_child(_coins)
    info.add_child(coin_row)
    row.add_child(info)
    card.add_child(row)
    _pass_clicks(row)
    add_child(card)
    Game.stats_changed.connect(_refresh)
    Game.skin_changed.connect(_refresh)
    Game.settings_changed.connect(_refresh)  # language switch
    _refresh()


func _pass_clicks(n: Node) -> void:
    if n is Control:
        (n as Control).mouse_filter = Control.MOUSE_FILTER_PASS
    for c in n.get_children():
        _pass_clicks(c)


func set_location(t: String, s: String = "") -> void:
    location_title = t
    location_sub = s
    if _title:
        _title.text = t
        _sub.text = s
        _sub.visible = s != ""


func _refresh() -> void:
    var prof: Dictionary = Game.save["profile"]
    _name.text = str(prof.get("display_name", prof["username"]))
    _avatar.texture = UI.head(Game.skin_folder())  # always matches the outfit
    _level.text = tr("TAHAP %d") % int(Game.save["level"])
    _exp.value = int(Game.save["exp"]) % Game.EXP_PER_LEVEL
    _coins.text = tr("SYILING: %d") % int(Game.save["coins"])


# ---------------------------------------------------------------- menu
```

```
func _build_menu(_left: Control) -> void:
    _catcher = Control.new()
    UI.full(_catcher)
    _catcher.visible = false
    _catcher.gui_input.connect(func(e: InputEvent):
        if e is InputEventMouseButton and e.pressed:
            set_menu_open(false))
    add_child(_catcher)
    move_child(_catcher, 0)
    _menu = UI.panel()
    _menu.visible = false
    _menu.z_index = 5
    var col := UI.vbox(6)
    if show_map_button:
        col.add_child(_item(UI.tex("res://assets/icons/peta.png"), "Kembali ke Peta", func(): Game.goto("peta")))
    if world:
        col.add_child(_item(UI.tex("res://assets/icons/peta.png"), "Peta Dunia  (M)", func(): Game.main.open_map(world)
))
    col.add_child(_item(UI.tex("res://assets/icons/nota.png"), "Nota Alif", func(): Game.main.open_nota()))
    col.add_child(_item(BegPanel.icon(), "Beg Alif", func(): Game.main.open_beg(world)))
    if world:
        col.add_child(_item(UI.tex("res://assets/portraits/pau_happy.png"), "Tenang dengan Pau", func(): Game.main.open
_calm()))
    col.add_child(_item(UI.tex("res://assets/icons/mahkota.png"), "Papan Markah", func(): Game.main.open_papan()))
    col.add_child(_item(UI.tex("res://assets/icons/tetapan.png"), "Tetapan", func(): Game.main.open_settings()))
    _menu.add_child(col)
    add_child(_menu)


func _item(icon: Texture2D, text: String, action: Callable) -> Button:
    var b := UI.button("  " + tr(text), "", 300)
    b.icon = icon
    b.expand_icon = true
    b.add_theme_constant_override("icon_max_width", 40)
    b.alignment = HORIZONTAL_ALIGNMENT_LEFT
    b.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    b.custom_minimum_size.y = 58
    b.pressed.connect(func():
        set_menu_open(false)
        Sfx.play("select", -6.0)
        action.call())
    return b


func set_menu_open(on: bool) -> void:
    if on and Game.is_locked():
        return
    _catcher.visible = on
    _menu.visible = on
    _menu_btn.theme_type_variation = "Primary" if on else ""
    if on:
        Sfx.play("pop", -8.0)
        _menu.modulate.a = 0.0
        var y: float = _menu_btn.global_position.y + _menu_btn.size.y + 6  # under the button (the location box can gro
w)
        _menu.position.x = 16
        _menu.position.y = y - 12
        var t := create_tween().set_parallel()
        t.tween_property(_menu, "modulate:a", 1.0, 0.15)
        t.tween_property(_menu, "position:y", y, 0.15)


func is_menu_open() -> bool:
    return _menu != null and _menu.visible


func _process(_d: float) -> void:
    if _menu and _menu.visible and Game.is_locked():
        set_menu_open(false)


func _unhandled_input(event: InputEvent) -> void:
    if is_menu_open() and (event.is_action_pressed("menu") or event.is_action_pressed("ui_cancel")):
        set_menu_open(false)
        get_viewport().set_input_as_handled()


## three bars (the "hamburger"), drawn so it needs no font glyph
static func _bars_icon() -> Texture2D:
    var img := Image.create(26, 22, false, Image.FORMAT_RGBA8)
    for bar in 3:
        img.fill_rect(Rect2i(0, bar * 9, 26, 4), Color(1, 0.95, 0.85))
    return ImageTexture.create_from_image(img)
```

map_overlay.gd · scripts/ui/map_overlay.gd · 396 lines

```
extends "res://scripts/ui/modal.gd"
## Full map of Dunia Explo-Story: drag to move, + / - or mouse wheel to zoom.
## Legend on the right: places (tap = show on map, Pergi = Jalan Pintas),
## keywords learned in each zone (from data/nota.json, hidden until the zone
## is done) and symbols.

var world: Node
var view: MapView
var _list: VBoxContainer
var _tabs: Array[Button] = []
var _drag_from := Vector2(-1, -1)
var _touches := {}  ## index -> position (two-finger pinch)
var _pinch_d := 0.0
var _kw_box: VBoxContainer
var _kw_filter := ""
var _open_zone := "?"   ## the Misi open in Kata Kunci ("?" = pick the latest done)


func _init() -> void:
    title_text = "PETA DUNIA EXPLO-STORY"
    panel_size = Vector2(1250, 690)


func _build() -> void:
    var row := UI.hbox(14)
    row.size_flags_vertical = Control.SIZE_EXPAND_FILL

    # map + zoom bar
    var left := UI.vbox(8)
    left.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var holder := PanelContainer.new()
    holder.add_theme_stylebox_override("panel", UI.sb(UI.NAVY, UI.GOLD, 3, 8, 3))
    holder.size_flags_vertical = Control.SIZE_EXPAND_FILL
    view = MapView.new()
    view.world = world
    view.follow_player = false
    view.show_names = true
    view.icon_scale = 1.4
    view.custom_minimum_size = Vector2(820, 520)
    view.mouse_filter = Control.MOUSE_FILTER_STOP
    view.mouse_default_cursor_shape = Control.CURSOR_DRAG
    view.gui_input.connect(_on_map_input)
    holder.add_child(view)
    left.add_child(holder)
    var bar := UI.hbox(8)
    var out := UI.button("-  Zum keluar")
    out.pressed.connect(func(): _zoom(1.0 / 1.5))
    bar.add_child(out)
    var inn := UI.button("+  Zum masuk")
    inn.pressed.connect(func(): _zoom(1.5))
    bar.add_child(inn)
    var whole := UI.button("Seluruh peta")
    whole.pressed.connect(func(): view.set_zoom(0.0))
    bar.add_child(whole)
    var me := UI.button("Cari Alif")
    me.pressed.connect(_find_me)
    bar.add_child(me)
    left.add_child(bar)
    var tip := UI.label("Seret peta untuk gerak. Roda tetikus atau jepit untuk zum.", "Small")
    left.add_child(tip)
    row.add_child(left)

    # legend
    var right := UI.vbox(8)
    right.custom_minimum_size = Vector2(360, 0)
    var tabs := UI.hbox(6)
    for t in [["Tempat", "_show_places"], ["Kata Kunci", "_show_keywords"], ["Simbol", "_show_symbols"]]:
        var b := UI.button(t[0])
        b.toggle_mode = true
        b.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        b.pressed.connect(_select_tab.bind(_tabs.size(), t[1]))
        tabs.add_child(b)
        _tabs.append(b)
    right.add_child(tabs)
    var scroll := ScrollContainer.new()
    scroll.size_flags_vertical = Control.SIZE_EXPAND_FILL
    scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
    _list = UI.vbox(8)
    _list.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    scroll.add_child(_list)
    right.add_child(scroll)
    row.add_child(right)
    content.add_child(row)
```

```
    Sfx.play("whoosh", -10.0, 1.3)
    _select_tab(0, "_show_places")
    await get_tree().process_frame
    view.set_zoom(view.fit_scale() * 1.6)
    _find_me()


func _select_tab(i: int, method: String) -> void:
    for j in _tabs.size():
        _tabs[j].button_pressed = j == i
    for c in _list.get_children():
        c.queue_free()
    call(method)


func _find_me() -> void:
    view.center = Vector2(world.player.cell) + Vector2(0.5, 0.5)
    view.highlight = view.center
    view.clamp_center()


func _zoom(f: float) -> void:
    view.set_zoom(view.px_per_tile * f)
    Sfx.play("tick", -10.0, 1.4)


# ---------------------------------------------------------------- legend tabs
func _row(sym: Control, title: String, sub: String, sub_col: Color = UI.CREAM_D) -> HBoxContainer:
    var r := UI.hbox(10)
    sym.custom_minimum_size = Vector2(40, 40)
    r.add_child(sym)
    var c := UI.vbox(0)
    c.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    c.add_child(UI.label(title, "", true))
    if sub != "":
        var s := UI.label(sub, "Small", true)
        s.add_theme_color_override("font_color", sub_col)
        c.add_child(s)
    r.add_child(c)
    return r


func _show_places() -> void:
    var nz := Game.next_zone()
    var can_go := {}  ## place id -> true when Jalan Pintas can take Alif there
    if world.has_method("travel_places"):
        for t in world.travel_places():
            can_go[str(t["place"]["id"])] = t["ok"]
    for p in MapView.legend()["places"]:
        var sym: Control
        var sub := str(p.get("info", ""))
        var col := UI.CREAM_D
        if p.has("zone"):
            var st := MapView.zone_status(str(p["zone"]))
            sym = MapView.Symbol.new("flag", MapView.status_color(st))
            col = MapView.status_color(st)
            sub = tr(MapView.status_text(st)) + (tr("  - SETERUSNYA") if str(p["zone"]) == nz else "") + ("\n" + tr(sub
) if sub != "" else "")
        else:
            sym = MapView.Symbol.new("badge", Color.WHITE, UI.tex(str(p["icon"])))
        # the whole row is a button: tap = show this place on the map
        var b := PanelContainer.new()
        var normal := UI.sb(Color(1, 1, 1, 0.04), Color(1, 1, 1, 0.12), 1, 8, 6)
        var hover := UI.sb(Color(1, 0.85, 0.3, 0.14), UI.GOLD, 2, 8, 6)
        b.add_theme_stylebox_override("panel", normal)
        b.mouse_filter = Control.MOUSE_FILTER_STOP
        b.mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
        b.tooltip_text = "Tunjuk di peta"
        b.mouse_entered.connect(func(): b.add_theme_stylebox_override("panel", hover))
        b.mouse_exited.connect(func(): b.add_theme_stylebox_override("panel", normal))
        var at := Vector2(p["at"][0], p["at"][1]) + Vector2(0.5, 0.5)
        b.gui_input.connect(func(e: InputEvent):
            if e is InputEventMouseButton and e.pressed and e.button_index == MOUSE_BUTTON_LEFT:
                _focus(at))
        var r := _row(sym, str(p["name"]), sub, col)
        _ignore_mouse(r)
        if can_go.get(str(p["id"]), false):
            var go := UI.button("Pergi", "Primary", 0)
            go.tooltip_text = tr("Jalan Pintas ke sini")
            go.pressed.connect(func():
                close()
                world.fast_travel(p))
            r.add_child(go)
        b.add_child(r)
```

```
        _list.add_child(b)


func _ignore_mouse(n: Node) -> void:
    if n is Control:
        (n as Control).mouse_filter = Control.MOUSE_FILTER_IGNORE
    for c in n.get_children():
        _ignore_mouse(c)


func _focus(tile: Vector2) -> void:
    Sfx.play("select", -6.0)
    view.set_zoom(maxf(view.px_per_tile, 9.0))
    view.center = tile
    view.highlight = tile
    view.clamp_center()


## Kata Kunci: a search box, then one folding card per Misi (only one open
## at a time). Inside, the keywords are small cards in two columns: icon +
## the word, with the detail under it. Tap a keyword = show on the map where
## it was learned. Locked Misi stay folded with a padlock.
func _show_keywords() -> void:
    if _open_zone == "?":
        _open_zone = ""
        for z in Game.ZONES:
            if Game.zone_done(z):
                _open_zone = z  # the latest finished Misi starts open
    var search := LineEdit.new()
    search.placeholder_text = tr("Cari kata kunci...")
    search.text = _kw_filter
    search.clear_button_enabled = true
    search.text_changed.connect(func(t: String):
        _kw_filter = t.strip_edges()
        _fill_keywords())
    _list.add_child(search)
    _kw_box = UI.vbox(8)
    _kw_box.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    _list.add_child(_kw_box)
    _fill_keywords()


func _fill_keywords() -> void:
    for c in _kw_box.get_children():
        c.queue_free()
    var nota: Dictionary = JSON.parse_string(FileAccess.get_file_as_string("res://data/nota.json"))
    if FileAccess.file_exists("res://data/nota_bab2.json"):
        nota["pages"].append_array(JSON.parse_string(FileAccess.get_file_as_string("res://data/nota_bab2.json"))["pages
"])
    var q := _kw_filter.to_lower()
    var shown := 0
    for z in Game.ZONES:
        if Game.chapter_of(z) > Game.current_chapter():
            continue
        var done: bool = Game.zone_done(z)
        var pages: Array = nota["pages"].filter(func(pg): return str(pg["zone"]) == z)
        var count := 0
        var matches := 0
        for pg in pages:
            for it in pg.get("items", []):
                count += 1
                if q == "" or (str(it.get("text", "")) + " " + str(it.get("sub", ""))).to_lower().contains(q):
                    matches += 1
        if q != "" and (not done or matches == 0):
            continue  # searching: only Misi with a hit
        shown += 1
        var open: bool = done and (q != "" or _open_zone == z)
        _kw_box.add_child(_zone_header(z, done, count, open))
        if not open:
            continue
        for pg in pages:
            var items: Array = pg.get("items", []).filter(func(it): return q == "" or (str(it.get("text", "")) + " " +
str(it.get("sub", ""))).to_lower().contains(q))
            if items.is_empty():
                continue
            var pt := UI.label(str(pg["title"]).to_upper(), "Small")
            pt.add_theme_color_override("font_color", UI.GOLD)
            _kw_box.add_child(UI.margin(pt, 4, 4, 0, 0))
            var grid := GridContainer.new()
            grid.columns = 2
            grid.add_theme_constant_override("h_separation", 6)
            grid.add_theme_constant_override("v_separation", 6)
            for it in items:
                grid.add_child(_keyword_chip(it, z))
            _kw_box.add_child(grid)
```

```
    if shown == 0:
        var l := UI.label(tr("Tiada kata kunci \"%s\" dalam misi yang sudah siap.") % _kw_filter, "Small", true)
        l.add_theme_color_override("font_color", UI.GREY)
        _kw_box.add_child(l)


func _zone_header(z: String, done: bool, count: int, open: bool) -> Control:
    var zd := Game.zone_data(z)
    var st := MapView.zone_status(z)
    var col := MapView.status_color(st)
    var card := PanelContainer.new()
    card.add_theme_stylebox_override("panel", UI.sb(Color(col, 0.18) if done else Color(1, 1, 1, 0.04), col if open els
e Color(col, 0.5), 2, 8, 6))
    var h := UI.hbox(10)
    var sym: Control = MapView.Symbol.new("flag", col) if done else MapView.Symbol.new("lock")
    sym.custom_minimum_size = Vector2(34, 34)
    h.add_child(sym)
    var c := UI.vbox(0)
    c.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    c.add_child(UI.label("Misi %s: %s" % [str(zd.get("no", z.replace("_", "."))), str(zd.get("title", ""))], "", true))
    var sub := UI.label(tr("%d kata kunci") % count if done else tr("Siapkan misi untuk buka"), "Small")
    sub.add_theme_color_override("font_color", col if done else UI.GREY)
    c.add_child(sub)
    h.add_child(c)
    if done:
        h.add_child(Arrow.new(open))
    card.add_child(h)
    _ignore_mouse(h)
    if done:
        card.mouse_filter = Control.MOUSE_FILTER_STOP
        card.mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
        card.gui_input.connect(func(e: InputEvent):
            if e is InputEventMouseButton and e.pressed and e.button_index == MOUSE_BUTTON_LEFT:
                Sfx.play("tick", -8.0, 1.2)
                _open_zone = "" if _open_zone == z else z
                _fill_keywords())
    return card


func _keyword_chip(it: Dictionary, z: String) -> Control:
    var chip := PanelContainer.new()
    var normal := UI.sb(Color(1, 1, 1, 0.06), Color(1, 1, 1, 0.14), 1, 8, 6)
    var hover := UI.sb(Color(1, 0.85, 0.3, 0.16), UI.GOLD, 2, 8, 6)
    chip.add_theme_stylebox_override("panel", normal)
    chip.custom_minimum_size.x = 168
    chip.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var v := UI.vbox(2)
    var top := UI.hbox(6)
    top.add_child(UI.icon(str(it.get("icon", "")), 28))
    var word := UI.label(str(it.get("text", "")), "", true)
    word.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    word.add_theme_font_size_override("font_size", 19)
    top.add_child(word)
    v.add_child(top)
    if str(it.get("sub", "")) != "":
        var s := UI.label(str(it["sub"]), "Small", true)
        s.add_theme_font_size_override("font_size", 15)
        s.add_theme_color_override("font_color", UI.CREAM_D)
        v.add_child(s)
    chip.add_child(v)
    _ignore_mouse(v)
    chip.mouse_filter = Control.MOUSE_FILTER_STOP
    chip.mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
    chip.tooltip_text = tr("Tunjuk di peta tempat kata kunci ini dipelajari")
    chip.mouse_entered.connect(func(): chip.add_theme_stylebox_override("panel", hover))
    chip.mouse_exited.connect(func(): chip.add_theme_stylebox_override("panel", normal))
    chip.gui_input.connect(func(e: InputEvent):
        if e is InputEventMouseButton and e.pressed and e.button_index == MOUSE_BUTTON_LEFT:
            _focus_zone(z))
    return chip


## centre the map on the place of a Misi (data/map_legend.json)
func _focus_zone(z: String) -> void:
    for p in MapView.legend()["places"]:
        if str(p.get("zone", "")) == z:
            _focus(Vector2(p["at"][0], p["at"][1]) + Vector2(0.5, 0.5))
            return


## the little open/closed triangle on a Misi card
class Arrow extends Control:
    var open := false

    func _init(is_open: bool) -> void:
```

```
        open = is_open
        custom_minimum_size = Vector2(24, 24)
        size_flags_vertical = Control.SIZE_SHRINK_CENTER

    func _draw() -> void:
        var c := size / 2.0
        var pts := PackedVector2Array([c + Vector2(-7, -4), c + Vector2(7, -4), c + Vector2(0, 5)]) if open \
            else PackedVector2Array([c + Vector2(-4, -7), c + Vector2(5, 0), c + Vector2(-4, 7)])
        draw_colored_polygon(pts, Color(1, 0.95, 0.8))


func _show_symbols() -> void:
    _list.add_child(_row(MapView.Symbol.new("me"), "Alif (kau di sini)", "Titik merah berkelip"))
    _list.add_child(_row(MapView.Symbol.new("bang"), "Tugasan seterusnya", "Pergi ke tanda seru"))
    _list.add_child(_row(MapView.Symbol.new("flag", MapView.COL_DONE), "Misi siap", "Bendera hijau"))
    _list.add_child(_row(MapView.Symbol.new("flag", MapView.COL_OPEN), "Misi terbuka", "Bendera emas"))
    _list.add_child(_row(MapView.Symbol.new("flag", MapView.COL_LOCKED), "Misi berkunci", "Bendera kelabu: siapkan misi sebelumnya"))
    _list.add_child(_row(MapView.Symbol.new("npc"), "Orang", "Ketik untuk bercakap"))
    for id in MapView.legend()["landmarks"]:
        var lm: Dictionary = MapView.legend()["landmarks"][id]
        _list.add_child(_row(MapView.Symbol.new("badge", Color.WHITE, UI.tex(str(lm["icon"]))), str(lm["name"]), ""))
    _list.add_child(_row(MapView.Symbol.new("lock"), "Gerbang bab berkunci", "Bab 2 dan seterusnya"))
    _list.add_child(_row(MapView.Symbol.new("mist"), "Kabus Masa", "Belum boleh diterokai"))


# ---------------------------------------------------------------- input
func _on_map_input(e: InputEvent) -> void:
    if e is InputEventScreenTouch:
        if e.pressed:
            _touches[e.index] = e.position
        else:
            _touches.erase(e.index)
        _pinch_d = 0.0
    elif e is InputEventScreenDrag and _touches.size() >= 2:
        _touches[e.index] = e.position
        var pts: Array = _touches.values()
        var d: float = (pts[0] as Vector2).distance_to(pts[1])
        if _pinch_d > 0.0:
            view.set_zoom(view.px_per_tile * d / _pinch_d, ((pts[0] as Vector2) + pts[1]) / 2.0)
        _pinch_d = d
        _drag_from = Vector2(-1, -1)
    elif e is InputEventMouseButton:
        if e.button_index == MOUSE_BUTTON_LEFT:
            _drag_from = e.position if e.pressed else Vector2(-1, -1)
        elif e.pressed and e.button_index == MOUSE_BUTTON_WHEEL_UP:
            view.set_zoom(view.px_per_tile * 1.2, e.position)
        elif e.pressed and e.button_index == MOUSE_BUTTON_WHEEL_DOWN:
            view.set_zoom(view.px_per_tile / 1.2, e.position)
        accept_event()
    elif e is InputEventMouseMotion and _drag_from.x >= 0 and _touches.size() < 2:
        view.center -= (e.position - _drag_from) / view.px_per_tile
        view.clamp_center()
        _drag_from = e.position
        accept_event()
    elif e is InputEventMagnifyGesture:
        view.set_zoom(view.px_per_tile * e.factor, e.position)


func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("menu") or event.is_action_pressed("map"):
        close()
        get_viewport().set_input_as_handled()
```

map_view.gd · scripts/ui/map_view.gd · 379 lines

```
class_name MapView
extends Control
## Draws Dunia Explo-Story from above: terrain at 1 px per tile, scaled up,
## with icons on top (Alif, next task "!", zone flags, people, landmarks).
## Used by the corner mini map and the full map overlay. Icons keep their
## screen size at every zoom so they stay easy to read.

const UI = preload("res://scripts/ui/ui.gd")
const LEGEND_PATH := "res://data/map_legend.json"

const COL_SEA := Color("3b8fc4")
const COL_SEA_D := Color("3685b9")
const COL_SAND := Color("e8d49a")
const COL_GRASS := Color("6dbb4f")
const COL_PATH := Color("c79a62")
const COL_RIVER := Color("4a8fe0")
const COL_MEADOW := Color("5aa845")
```

```
const COL_TREE := Color("2f7a35")
const COL_ROOF := Color("8a4f33")
const COL_WOOD := Color("9c6a3c")
const COL_MIST := Color(0.55, 0.4, 0.75, 0.55)
const COL_DONE := Color("5fd068")
const COL_OPEN := Color("f0c24a")
const COL_LOCKED := Color("8a8a96")
const ARRIVED := 2.5  ## tiles: this close to the objective, the arrows hide

var world: Node                ## world.gd
var px_per_tile: float = 5.0   ## zoom
var center: Vector2 = Vector2(40, 30)  ## view centre, in tiles
var follow_player: bool = true
var show_names: bool = false
var icon_scale: float = 1.0
var highlight: Vector2 = Vector2(-1, -1)  ## pulsing ring (legend tap)

var _t: float = 0.0
var _map_size: Vector2 = Vector2(80, 60)
var _icons := {}  ## path -> Texture2D

static var _legend: Dictionary = {}


static func legend() -> Dictionary:
    if _legend.is_empty():
        _legend = JSON.parse_string(FileAccess.get_file_as_string(LEGEND_PATH))
    return _legend


## 1 px per tile picture of the world: terrain colours, trees and buildings
## darker, jetty planks, the mist area tinted purple.
static func build_texture(w: Node) -> Texture2D:
    var tw: int = w.terrain.width
    var th: int = w.terrain.height
    var img := Image.create(tw, th, false, Image.FORMAT_RGBA8)
    var cols := {
        TerrainBuilder.SEA: COL_SEA, TerrainBuilder.SAND: COL_SAND, TerrainBuilder.GRASS: COL_GRASS,
        TerrainBuilder.PATH: COL_PATH, TerrainBuilder.JALAN: Color("b8b0a2"), TerrainBuilder.RIVER: COL_RIVER, TerrainB
uilder.MEADOW: COL_MEADOW,
    }
    for y in th:
        for x in tw:
            var c := Vector2i(x, y)
            var t: int = w.terrain.terrain_at(c)
            var col: Color = cols.get(t, COL_GRASS)
            if t == TerrainBuilder.SEA and (x + y) % 7 == 0:
                col = COL_SEA_D
            if w.walk_over.has(c):
                col = COL_WOOD
            elif w.blocked.has(c) and not w.terrain.is_water(c):
                col = COL_TREE if t == TerrainBuilder.GRASS or t == TerrainBuilder.MEADOW else COL_ROOF
            img.set_pixelv(c, col)
    for a in w.areas:
        if a["id"] == "kabus":
            var r: Array = a["rect"]
            for y in range(r[1], r[3] + 1):
                for x in range(r[0], r[2] + 1):
                    if x < tw and y < th:
                        img.set_pixel(x, y, img.get_pixel(x, y).blend(COL_MIST))
    return ImageTexture.create_from_image(img)


func _ready() -> void:
    clip_contents = true
    mouse_filter = Control.MOUSE_FILTER_PASS
    if world:
        _map_size = Vector2(world.terrain.width, world.terrain.height)


func _process(delta: float) -> void:
    _t += delta
    if follow_player and world and is_instance_valid(world.player):
        center = Vector2(world.player.cell) + Vector2(0.5, 0.5)
    clamp_center()
    queue_redraw()


func fit_scale() -> float:
    return minf(size.x / _map_size.x, size.y / _map_size.y)


## keep the view on the map; when the map is smaller than the view, centre it
func clamp_center() -> void:
    var half := size / (2.0 * px_per_tile)
```

```
    for i in 2:
        if half[i] * 2.0 >= _map_size[i]:
            center[i] = _map_size[i] / 2.0
        else:
            center[i] = clampf(center[i], half[i], _map_size[i] - half[i])


func to_screen(tile: Vector2) -> Vector2:
    return size / 2.0 + (tile - center) * px_per_tile


func to_tile(screen: Vector2) -> Vector2:
    return center + (screen - size / 2.0) / px_per_tile


func set_zoom(ppt: float, anchor: Vector2 = Vector2(-1, -1)) -> void:
    var before := to_tile(anchor) if anchor.x >= 0 else center
    px_per_tile = clampf(ppt, fit_scale(), 32.0)
    if anchor.x >= 0:  # keep the point under the cursor still
        center += before - to_tile(anchor)
    clamp_center()


# ---------------------------------------------------------------- status
## "done" / "open" / "locked" for a Bab 1 zone
static func zone_status(zid: String) -> String:
    if Game.zone_done(zid):
        return "done"
    return "open" if Game.zone_unlocked(zid) else "locked"


static func status_color(st: String) -> Color:
    return {"done": COL_DONE, "open": COL_OPEN}.get(st, COL_LOCKED)


static func status_text(st: String) -> String:
    return {"done": "SIAP", "open": "TERBUKA", "locked": "BERKUNCI"}.get(st, "")


## Where the next task is: the visible "!" markers, else the next zone.
func objectives() -> Array:
    var out: Array = []
    if world == null:
        return out
    for id in world.runner.markers:
        if world.runner.markers[id].visible:
            out.append(spot_tile(id))
    if out.is_empty():
        var nz := Game.next_zone()
        for p in legend()["places"]:
            if str(p.get("zone", "")) == nz and nz != "":
                out.append(Vector2(p["at"][0], p["at"][1]) + Vector2(0.5, 0.5))
    return out


func spot_tile(id: String) -> Vector2:
    if world.spots.has(id):
        var sp: Dictionary = world.spots[id]
        return Vector2(sp["pos"][0] + sp["size"][0] / 2.0, sp["pos"][1] + 0.5)
    for at in world.npcs:
        if world.npcs[at].id == id:
            return Vector2(at) + Vector2(0.5, 0.5)
    return Vector2(-1, -1)


func _icon(path: String) -> Texture2D:
    if not _icons.has(path):
        _icons[path] = UI.tex(path)
    return _icons[path]


# ---------------------------------------------------------------- drawing
func _draw() -> void:
    draw_rect(Rect2(Vector2.ZERO, size), world.map_sea if world else COL_SEA)
    if world == null or world.map_texture == null:
        return
    # the world's own picture (photographed when it loads); smooth when shrunk,
    # crisp pixels when zoomed in close
    texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST if px_per_tile >= 14.0 else CanvasItem.TEXTURE_FILTER_LINEAR_WIT
H_MIPMAPS
    var origin := to_screen(Vector2.ZERO)
    draw_texture_rect(world.map_texture, Rect2(origin, _map_size * px_per_tile), false)
    var s := icon_scale
    var ent: Dictionary = world.entities()
    var close := show_names and px_per_tile >= 10.0  # zoomed in: name every place and person
```

```
186
187     # Kabus Lupa: the same grey, misty places as in the world (cleared when a
188     # mission is done), soft-edged with a light drifting haze on top
189     if world.has_method("fog_rects"):
190         for r in world.fog_rects():
191             var tl := to_screen(Vector2(r.x, r.y) / 16.0)
192             var wh := Vector2(r.z, r.w) / 16.0 * px_per_tile
193             if wh.x <= 0.0:
194                 continue
195             draw_rect(Rect2(tl, wh), Color(0.78, 0.80, 0.85, 0.55))
196             draw_rect(Rect2(tl - Vector2(2, 2), wh + Vector2(4, 4)), Color(0.85, 0.87, 0.9, 0.18))
197             var step := maxf(wh.x, wh.y) / 4.0
198             for i in 4:
199                 var c := tl + wh * Vector2(0.2 + 0.2 * i, 0.5 + 0.3 * sin(_t * 0.4 + i * 1.7))
200                 draw_circle(c, step * 0.45, Color(0.96, 0.97, 1.0, 0.16))
201
202     for a in world.areas:  # the Mist of Time stays purple and mysterious
203         if a["id"] == "kabus":
204             var r: Array = a["rect"]
205             # soft drifting puffs instead of a hard box
206             for y in range(r[1], r[3] + 1, 3):
207                 for x in range(r[0], r[2] + 1, 3):
208                     var wob := Vector2(sin(_t * 0.6 + x * 0.7), cos(_t * 0.5 + y * 0.9)) * 0.6
209                     draw_circle(to_screen(Vector2(x + 1.5, y + 1.5) + wob), 2.6 * px_per_tile, Color(0.62, 0.48, 0.85,
    0.13))
210
211     if show_names:
212         for a in world.areas:
213             var r: Array = a["rect"]
214             var name_s := str(a["name"]).replace("Misi ", "").to_upper()
215             _label(name_s, to_screen(Vector2((r[0] + r[2] + 1) / 2.0, r[1] + 1.2)), int((16 if close else 14) * s), UI.
    GOLD if close else UI.CREAM)
216
217     # chapter gates (locked = padlock)
218     for g in ent["gates"]:
219         var gp := to_screen(Vector2(g["pos"][0] + 2, g["pos"][1] + 0.5))
220         if not g["open"]:
221             draw_lock(self, gp, 7 * s)
222
223     # landmarks (signs with an entry in the legend, plus spots like the arena)
224     var lm: Dictionary = legend()["landmarks"]
225     for sg in ent["signs"]:
226         if lm.has(sg["id"]):
227             var size_a: Array = sg.get("size", [1, 1])
228             var c := Vector2(sg["pos"][0] + size_a[0] / 2.0, sg["pos"][1] - size_a[1] / 2.0 + 1)
229             draw_badge(self, to_screen(c), _icon(lm[sg["id"]]["icon"]), 18 * s)
230             if close:
231                 _label(str(lm[sg["id"]]["name"]), to_screen(c) + Vector2(0, 22 * s), int(12 * s), UI.CREAM)
232     for id in lm:
233         if world.spots.has(id):
234             draw_badge(self, to_screen(spot_tile(id)), _icon(lm[id]["icon"]), 18 * s)
235             if close:
236                 _label(str(lm[id]["name"]), to_screen(spot_tile(id)) + Vector2(0, 22 * s), int(12 * s), UI.CREAM)
237
238     # people
239     for at in world.npcs:
240         var np := to_screen(Vector2(at) + Vector2(0.5, 0.5))
241         draw_circle(np, 4.0 * s, Color(0.1, 0.1, 0.12))
242         draw_circle(np, 3.0 * s, UI.CREAM)
243         if close:
244             _label(Game.char_name(world.npcs[at].id), np + Vector2(0, 16 * s), int(11 * s), Color("cfe8ff"))
245
246     # zone flags
247     for p in legend()["places"]:
248         if p.has("zone"):
249             var st := zone_status(str(p["zone"]))
250             draw_flag(self, to_screen(Vector2(p["at"][0], p["at"][1]) + Vector2(0.5, 0.5)), status_color(st), 9 * s)
251
252     # next task
253     for o in objectives():
254         var op := to_screen(o)
255         op = Vector2(clampf(op.x, 10, size.x - 10), clampf(op.y, 16, size.y - 4))  # stays on the edge when off view
256         draw_bang(self, op + Vector2(0, -sin(_t * 4.0) * 2.0), 8 * s)
257
258     # legend tap highlight
259     if highlight.x >= 0:
260         var hp := to_screen(highlight)
261         var r := (10.0 + fmod(_t * 18.0, 14.0)) * s
262         draw_arc(hp, r, 0, TAU, 32, Color(1, 0.9, 0.4, 1.0 - fmod(_t * 18.0, 14.0) / 14.0), 3.0)
263
264     # navigation arrow: a gold arrow beside Alif points at the objective, which
265     # gets a pulsing ring (or, off this view, an arrow on the map's edge)
266     var me := to_screen(Vector2(world.player.cell) + Vector2(0.5, 0.5))
267     var goal: Dictionary = world.objective() if world.has_method("objective") else {}
268     if not goal.is_empty() and Vector2(world.player.cell).distance_to(goal["tile"]) > ARRIVED:
```

```
        var gp := to_screen(goal["tile"])
        var dir := (gp - me).angle()
        var inner := Rect2(Vector2.ZERO, size).grow(-14 * s)
        if inner.has_point(gp):
            var k := fmod(_t * 1.2, 1.0)
            draw_arc(gp, (8.0 + k * 10.0) * s, 0, TAU, 28, Color(1, 0.86, 0.3, 1.0 - k), 2.5)
        else:
            draw_nav_arrow(self, _edge_point(me, gp, inner), dir, 11 * s)
        draw_nav_arrow(self, me + Vector2.from_angle(dir) * (15 + sin(_t * 5.0) * 2.0) * s, dir, 8 * s)

    # Alif (blinks so it is easy to find)
    var on := fmod(_t, 1.0) < 0.7
    draw_circle(me, 6.0 * s, Color(0.08, 0.08, 0.12))
    draw_circle(me, 4.5 * s, Color("ff5a4a") if on else Color("ffd0c8"))
    draw_arc(me, 6.5 * s, 0, TAU, 20, Color.WHITE, 1.5)


## where the line from a (inside r) towards b leaves the rect r
func _edge_point(a: Vector2, b: Vector2, r: Rect2) -> Vector2:
    var d := b - a
    var t := 1.0
    for i in 2:
        if d[i] > 0.0:
            t = minf(t, (r.end[i] - a[i]) / d[i])
        elif d[i] < 0.0:
            t = minf(t, (r.position[i] - a[i]) / d[i])
    return a + d * maxf(t, 0.0)


## text with a dark outline, centred on p (baseline)
func _label(text: String, p: Vector2, fs: int, col: Color) -> void:
    var font := get_theme_default_font()
    var w := font.get_string_size(text, HORIZONTAL_ALIGNMENT_LEFT, -1, fs).x
    var at := p - Vector2(w / 2.0, 0)
    draw_string_outline(font, at, text, HORIZONTAL_ALIGNMENT_LEFT, -1, fs, 5, Color(0.08, 0.08, 0.12, 0.95))
    draw_string(font, at, text, HORIZONTAL_ALIGNMENT_LEFT, -1, fs, col)


# ---------------------------------------------------------------- symbols
## These also draw the symbols in the legend (any CanvasItem).
static func draw_flag(ci: CanvasItem, p: Vector2, col: Color, h: float) -> void:
    var base := p + Vector2(-h * 0.3, h * 0.5)
    ci.draw_rect(Rect2(base + Vector2(-1, -h * 1.6), Vector2(3, h * 1.6)), Color(0.1, 0.08, 0.06))
    var tri := PackedVector2Array([base + Vector2(1, -h * 1.6), base + Vector2(1 + h * 1.1, -h * 1.25), base + Vector2(
1, -h * 0.9)])
    ci.draw_colored_polygon(tri, col)
    ci.draw_polyline(PackedVector2Array([tri[0], tri[1], tri[2]]), Color(0.1, 0.08, 0.06), 1.5)


static func draw_bang(ci: CanvasItem, p: Vector2, h: float) -> void:
    var r := Rect2(p + Vector2(-h * 0.45, -h * 1.6), Vector2(h * 0.9, h * 1.6))
    ci.draw_rect(r.grow(1.5), Color(0.17, 0.11, 0.06))
    ci.draw_rect(r, Color(1, 0.86, 0.3))
    ci.draw_rect(Rect2(r.position + Vector2(r.size.x * 0.33, r.size.y * 0.12), Vector2(r.size.x * 0.34, r.size.y * 0.5)
), Color(0.35, 0.2, 0.05))
    ci.draw_rect(Rect2(r.position + Vector2(r.size.x * 0.33, r.size.y * 0.72), Vector2(r.size.x * 0.34, r.size.y * 0.16
)), Color(0.35, 0.2, 0.05))


## the gold navigation arrow (anak panah penunjuk), tip at p + h along angle,
## dark outline like the "!" so it reads on grass, sand and sea
static func draw_nav_arrow(ci: CanvasItem, p: Vector2, angle: float, h: float) -> void:
    var pts := PackedVector2Array([Vector2(h, 0), Vector2(-h * 0.7, -h * 0.8), Vector2(-h * 0.3, 0), Vector2(-h * 0.7,
h * 0.8)])
    var tf := Transform2D(angle, p)
    var shape := tf * pts
    ci.draw_colored_polygon(shape, Color(1, 0.86, 0.3))
    ci.draw_polyline(shape + PackedVector2Array([shape[0]]), Color(0.17, 0.11, 0.06), maxf(2.0, h * 0.18), true)


static func draw_lock(ci: CanvasItem, p: Vector2, h: float) -> void:
    ci.draw_arc(p + Vector2(0, -h * 0.35), h * 0.45, PI, TAU, 10, Color(0.15, 0.12, 0.2), 2.5)
    ci.draw_rect(Rect2(p + Vector2(-h * 0.7, -h * 0.35), Vector2(h * 1.4, h * 1.1)), Color(0.55, 0.4, 0.75))
    ci.draw_rect(Rect2(p + Vector2(-h * 0.7, -h * 0.35), Vector2(h * 1.4, h * 1.1)), Color(0.15, 0.12, 0.2), false, 1.5
)


static func draw_badge(ci: CanvasItem, p: Vector2, tex: Texture2D, d: float) -> void:
    ci.draw_circle(p, d * 0.62, Color(0.12, 0.14, 0.2, 0.9))
    ci.draw_arc(p, d * 0.62, 0, TAU, 20, UI.GOLD, 1.5)
    if tex:
        ci.draw_texture_rect(tex, Rect2(p - Vector2(d, d) * 0.4, Vector2(d, d) * 0.8), false)


## Small Control that draws one map symbol (for the legend list).
```

```
class Symbol extends Control:
    var kind := ""
    var color := Color.WHITE
    var tex: Texture2D

    func _init(k: String, c: Color = Color.WHITE, t: Texture2D = null) -> void:
        kind = k
        color = c
        tex = t
        custom_minimum_size = Vector2(36, 36)
        mouse_filter = Control.MOUSE_FILTER_IGNORE

    func _draw() -> void:
        var p := size / 2.0
        match kind:
            "me":
                draw_circle(p, 9, Color(0.08, 0.08, 0.12))
                draw_circle(p, 7, Color("ff5a4a"))
                draw_arc(p, 9.5, 0, TAU, 20, Color.WHITE, 1.5)
            "bang": MapView.draw_bang(self, p + Vector2(0, 12), 15)
            "flag": MapView.draw_flag(self, p + Vector2(-2, 2), color, 16)
            "lock": MapView.draw_lock(self, p + Vector2(0, 2), 11)
            "npc":
                draw_circle(p, 6, Color(0.1, 0.1, 0.12))
                draw_circle(p, 4.5, UI.CREAM)
            "badge": MapView.draw_badge(self, p, tex, 28)
            "mist":
                draw_rect(Rect2(p - Vector2(14, 10), Vector2(28, 20)), COL_MIST.lerp(Color.WHITE, 0.2))
            _:
                if tex:
                    draw_texture_rect(tex, Rect2(p - Vector2(14, 14), Vector2(28, 28)), false)
```

minimap.gd · scripts/ui/minimap.gd · 82 lines

```
extends Control
## Corner mini map in the world (takes the Pau button's place; Pau's hint and
## calm moment live on the Pau follower and the HUD toolbar there).
## + / - zoom through 3 levels, up to the whole map. Tap the map or press M
## for the full map with the legend.

const UI = preload("res://scripts/ui/ui.gd")
const MAP := 156.0
const ZOOMS := [7.0, 3.5, 0.0]  ## px per tile; 0 = whole map

signal open_requested

var world: Node
var view: MapView
var _zoom_i: int = 0
var _zoom_label: Label


func _ready() -> void:
    theme = Game.theme
    mouse_filter = Control.MOUSE_FILTER_IGNORE
    var frame := PanelContainer.new()
    frame.add_theme_stylebox_override("panel", UI.sb(Color(0.12, 0.14, 0.2, 0.88), UI.GOLD, 3, 14, 6))
    var col := UI.vbox(4)
    view = MapView.new()
    view.world = world
    view.custom_minimum_size = Vector2(MAP, MAP)
    view.icon_scale = 0.8
    view.mouse_filter = Control.MOUSE_FILTER_STOP
    view.mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
    view.tooltip_text = "Ketik untuk buka peta penuh (M)"
    view.gui_input.connect(_on_map_input)
    col.add_child(view)
    var row := UI.hbox(4)
    var minus := _small_button("-", "Zum keluar")
    minus.pressed.connect(func(): _set_zoom(_zoom_i + 1))
    row.add_child(minus)
    _zoom_label = UI.label("", "Small")
    _zoom_label.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    _zoom_label.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    _zoom_label.add_theme_font_size_override("font_size", 15)
    row.add_child(_zoom_label)
    var plus := _small_button("+", "Zum masuk")
    plus.pressed.connect(func(): _set_zoom(_zoom_i - 1))
    row.add_child(plus)
    col.add_child(row)
    frame.add_child(col)
    add_child(frame)
    frame.reset_size()
```

```
    size = frame.size
    _set_zoom(int(Game.settings.get("minimap_zoom", 0)))


func _small_button(t: String, tip: String) -> Button:
    var b := UI.button(t)
    b.custom_minimum_size = Vector2(44, 36)
    b.add_theme_font_size_override("font_size", 22)
    b.tooltip_text = tip
    b.focus_mode = Control.FOCUS_NONE
    return b


func _set_zoom(i: int) -> void:
    _zoom_i = clampi(i, 0, ZOOMS.size() - 1)
    Game.settings["minimap_zoom"] = _zoom_i
    await get_tree().process_frame  # view has its size after the first layout
    var z: float = ZOOMS[_zoom_i]
    view.px_per_tile = view.fit_scale() if z == 0.0 else z
    view.follow_player = z != 0.0
    view.icon_scale = 0.55 if z == 0.0 else 0.8
    view.clamp_center()
    _zoom_label.text = ["DEKAT", "SEDERHANA", "SELURUH"][_zoom_i]
    Sfx.play("tick", -10.0, 1.4)


func _on_map_input(e: InputEvent) -> void:
    if e is InputEventMouseButton and e.pressed and e.button_index == MOUSE_BUTTON_LEFT:
        accept_event()
        open_requested.emit()
    elif e is InputEventMouseButton and e.pressed and e.button_index in [MOUSE_BUTTON_WHEEL_UP, MOUSE_BUTTON_WHEEL_DOWN
]:
        accept_event()
        _set_zoom(_zoom_i + (-1 if e.button_index == MOUSE_BUTTON_WHEEL_UP else 1))
```

`modal.gd` · scripts/ui/modal.gd · 55 lines

```
extends Control
## Base for full-screen modal panels (settings, notes, museum).
## Subclasses set `title_text` / `panel_size` and fill `content` in _build().

const UI = preload("res://scripts/ui/ui.gd")

signal closed

var title_text: String = ""
var panel_size: Vector2 = Vector2(900, 600)
var panel_variation: String = ""
var content: VBoxContainer
var frame: PanelContainer


func _ready() -> void:
    UI.full(self)
    theme = Game.theme
    mouse_filter = Control.MOUSE_FILTER_STOP
    Game.lock_input()
    add_child(UI.dim(0.6))
    frame = UI.panel(panel_variation)
    frame.custom_minimum_size = panel_size
    var col := UI.vbox(14)
    var head := UI.hbox(12)
    var t := UI.label(title_text, "InkHeading" if panel_variation == "Paper" else "Heading")
    t.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    head.add_child(t)
    var x := UI.button("Tutup", "", 0)
    x.pressed.connect(close)
    head.add_child(x)
    col.add_child(head)
    content = UI.vbox(12)
    content.size_flags_vertical = Control.SIZE_EXPAND_FILL
    col.add_child(content)
    frame.add_child(col)
    add_child(UI.center(frame))
    _build()
    x.grab_focus()


func _build() -> void:
    pass


func close() -> void:
```

```
    Game.unlock_input()
    closed.emit()
    queue_free()


func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("menu"):
        close()
        get_viewport().set_input_as_handled()
```

nota_panel.gd · scripts/ui/nota_panel.gd · 167 lines

```
extends "res://scripts/ui/modal.gd"
## Nota Alif: visual summary of every open Bab (data/nota.json for Bab 1,
## data/nota_bab<n>.json after that), built from the zones the player has
## completed. Can be opened any time for revision.

var _data: Dictionary
var _page: int = 0
var _page_box: VBoxContainer
var _scroll: ScrollContainer
var _page_label: Label
var _prev: Button
var _next: Button
var start_zone := ""  ## open on this mission's first page


func _init() -> void:
    title_text = "NOTA ALIF"
    panel_size = Vector2(1000, 600)
    panel_variation = "Paper"


func _build() -> void:
    _data = Game.load_json("res://data/nota.json")
    var start := 0
    for n in range(2, Game.current_chapter() + 1):
        var path := "res://data/nota_bab%d.json" % n
        if FileAccess.file_exists(path):
            start = _data["pages"].size()  # open on the newest chapter
            _data["pages"].append_array(Game.load_json(path).get("pages", []))
    # the page scrolls: icon summary on top, the Huraian (textbook notes) under it
    _scroll = ScrollContainer.new()
    _scroll.size_flags_vertical = Control.SIZE_EXPAND_FILL
    _scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
    _page_box = UI.vbox(12)
    _page_box.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    _scroll.add_child(_page_box)
    content.add_child(_scroll)
    var nav := UI.hbox(12)
    _prev = UI.button("« Sebelum")
    _prev.pressed.connect(func(): _show(_page - 1))
    nav.add_child(_prev)
    _page_label = UI.label("", "InkSmall")
    _page_label.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    _page_label.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    nav.add_child(_page_label)
    var read := UI.icon_button("res://assets/icons/suara.png", "Blue", Vector2(64, 56))
    read.pressed.connect(_read_page)
    nav.add_child(read)
    _next = UI.button("Seterus »")
    _next.pressed.connect(func(): _show(_page + 1))
    nav.add_child(_next)
    content.add_child(nav)
    if start_zone != "":
        for i in _pages().size():
            if str(_pages()[i].get("zone", "")) == start_zone:
                start = i
                break
    _show(start)


func _pages() -> Array:
    return _data.get("pages", [])


func _show(i: int) -> void:
    var pages := _pages()
    _page = clampi(i, 0, pages.size() - 1)
    for c in _page_box.get_children():
        c.queue_free()
    _scroll.scroll_vertical = 0
    var p: Dictionary = pages[_page]
```

```
    var unlocked := Game.zone_done(str(p.get("zone", "")))
    _page_label.text = tr("Bab %d") % Game.chapter_of(str(p.get("zone", "1_1"))) + "  ·  " + tr("Halaman %d / %d") % [_page + 1, pages.size()]
    _prev.disabled = _page == 0
    _next.disabled = _page >= pages.size() - 1
    var head := UI.label(str(p.get("title", "")), "InkHeading")
    _page_box.add_child(head)
    if not unlocked:
        var lock := UI.hbox(16)
        lock.add_child(UI.icon("res://assets/icons/kunci.png", 72))
        lock.add_child(UI.label("Selesaikan Misi %s untuk membuka halaman ini." % str(p["zone"]).replace("_", "."), "Ink", true))
        _page_box.add_child(lock)
        return
    var items: Array = p.get("items", [])
    var style := str(p.get("style", "list"))
    if style == "timeline":
        var row := UI.hbox(8)
        row.alignment = BoxContainer.ALIGNMENT_CENTER
        for k in items.size():
            var it: Dictionary = items[k]
            var c := UI.vbox(4)
            c.custom_minimum_size.x = 160
            c.add_child(UI.icon(str(it.get("icon", "")), 110))
            var l := UI.label(str(it["text"]), "Ink", true)
            l.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
            c.add_child(l)
            var s := UI.label(str(it.get("sub", "")), "InkSmall", true)
            s.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
            c.add_child(s)
            row.add_child(c)
            if k < items.size() - 1:
                var arrow := UI.label("»", "InkHeading")
                row.add_child(arrow)
        _page_box.add_child(row)
    else:
        var grid := GridContainer.new()
        grid.columns = 2 if style == "grid" else 1
        grid.add_theme_constant_override("h_separation", 24)
        grid.add_theme_constant_override("v_separation", 10)
        for it in items:
            var r := UI.hbox(14)
            r.custom_minimum_size.x = 440 if style == "grid" else 900
            r.add_child(UI.icon(str(it.get("icon", "")), 64))
            var tc := UI.vbox(2)
            tc.size_flags_horizontal = Control.SIZE_EXPAND_FILL
            tc.add_child(UI.label(str(it["text"]), "Ink", true))
            tc.add_child(UI.label(str(it.get("sub", "")), "InkSmall", true))
            r.add_child(tc)
            grid.add_child(r)
        _page_box.add_child(grid)
    _add_huraian(p)


## the simplified textbook notes: titled sections of short points, then an
## "Ingat!" box with the one line worth remembering for the exam
func _add_huraian(p: Dictionary) -> void:
    var secs: Array = p.get("huraian", [])
    if secs.is_empty():
        return
    var line := ColorRect.new()
    line.color = Color(0.45, 0.32, 0.2, 0.4)
    line.custom_minimum_size = Vector2(0, 2)
    _page_box.add_child(UI.margin(line, 0, 6, 0, 0))
    _page_box.add_child(UI.label(tr("HURAIAN"), "InkHeading"))
    for sec in secs:
        var h := UI.label(str(sec.get("h", "")), "Ink")
        h.add_theme_color_override("font_color", Color("8a4b14"))
        _page_box.add_child(h)
        for t in sec.get("t", []):
            var l := UI.label("•  " + str(t), "Ink", true)
            l.size_flags_horizontal = Control.SIZE_EXPAND_FILL
            _page_box.add_child(UI.margin(l, 14, 0, 8, 0))
    if p.has("ingat"):
        var box := PanelContainer.new()
        box.add_theme_stylebox_override("panel", UI.sb(Color("fff1c2"), UI.GOLD, 3, 8, 12))
        var row := UI.hbox(12)
        row.add_child(UI.icon("res://assets/icons/bintang.png", 40))
        var l := UI.label(tr("Ingat! ") + str(p["ingat"]), "Ink", true)
        l.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        row.add_child(l)
        box.add_child(row)
        _page_box.add_child(UI.margin(box, 0, 8, 8, 8))


```

```
func _read_page() -> void:
    var p: Dictionary = _pages()[_page]
    var parts: PackedStringArray = [str(p.get("title", ""))]
    for it in p.get("items", []):
        parts.append("%s. %s." % [it.get("text", ""), it.get("sub", "")])
    if Game.zone_done(str(p.get("zone", ""))):
        for sec in p.get("huraian", []):
            parts.append(str(sec.get("h", "")) + ".")
            for t in sec.get("t", []):
                parts.append(str(t))
        if p.has("ingat"):
            parts.append(tr("Ingat! ") + str(p["ingat"]))
    Tts.speak(" ".join(parts).replace("»", ","))
```

`objective_arrow.gd` · scripts/ui/objective_arrow.gd · 74 lines

```
extends Control
## Anak panah penunjuk in the world: when the objective (world.objective())
## is off screen, a gold arrow sits on the screen edge pointing at it, with
## the mission and how many steps away. On screen, a small arrow bobs over
## the place (the "!" marker is there too when it is a mission step). Hidden
## while there is no objective, Alif has arrived, or a cutscene / puzzle /
## dialogue is on.

const UI = preload("res://scripts/ui/ui.gd")
const MARGIN := 70.0

var world: Node
var _t := 0.0
var _label: Label


func _ready() -> void:
    UI.full(self)
    mouse_filter = Control.MOUSE_FILTER_IGNORE
    _label = UI.label("", "Small")
    _label.add_theme_color_override("font_color", UI.GOLD)
    _label.add_theme_color_override("font_outline_color", Color(0.1, 0.06, 0.03))
    _label.add_theme_constant_override("outline_size", 6)
    _label.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    _label.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(_label)


func _process(delta: float) -> void:
    _t += delta
    queue_redraw()


func _draw() -> void:
    _label.visible = false
    if world == null or not world.hud.visible or world.busy or world.runner.busy \
            or Game.active_puzzle != null or Main.is_dialogue_open():
        return
    var goal: Dictionary = world.objective()
    if goal.is_empty():
        return
    var steps := int(round(Vector2(world.player.cell).distance_to(goal["tile"])))
    if steps <= MapView.ARRIVED:
        return
    var ct := get_viewport().get_canvas_transform()
    var me: Vector2 = ct * world.player.global_position
    var gp: Vector2 = ct * (goal["tile"] * 16.0)  # tiles -> world px
    var view := Rect2(Vector2.ZERO, get_viewport_rect().size)
    var inner := view.grow(-MARGIN)
    var bob := sin(_t * 5.0) * 4.0
    if view.grow(-16).has_point(gp):  # on screen: bob over the place itself
        MapView.draw_nav_arrow(self, gp + Vector2(0, -60 + bob), PI / 2.0, 16)
        return
    var dir := (gp - me).angle()
    var at := _edge(me, gp, inner) - Vector2.from_angle(dir) * bob
    MapView.draw_nav_arrow(self, at, dir, 22)
    _label.visible = true
    _label.text = "%s · %s" % [goal["name"], tr("%d langkah") % steps]
    _label.reset_size()
    var lp := at - Vector2.from_angle(dir) * 46 - _label.size / 2.0
    _label.position = Vector2(clampf(lp.x, 8, view.size.x - _label.size.x - 8), clampf(lp.y, 8, view.size.y - _label.si
ze.y - 8))


## where the line from a towards b leaves the rect r (a may be outside it)
func _edge(a: Vector2, b: Vector2, r: Rect2) -> Vector2:
    a = a.clamp(r.position, r.end)
```

```
    var d := b - a
    var t := 1.0
    for i in 2:
        if d[i] > 0.0:
            t = minf(t, (r.end[i] - a[i]) / d[i])
        elif d[i] < 0.0:
            t = minf(t, (r.position[i] - a[i]) / d[i])
    return a + d * maxf(t, 0.0)
```

papan_panel.gd · scripts/ui/papan_panel.gd · 222 lines

```
extends "res://scripts/ui/modal.gd"
## Papan Markah (leaderboard): Seluruh (everyone on the server), Negeri (your
## state, chosen in Profil) and Rakan (your friends). Ranked by total EXP,
## then battle wins. Online it asks the server (supabase/leaderboard.sql);
## offline it shows sample classmates (data/leaderboard_sample.json) with a
## clear "Contoh" banner, ranked together with your real numbers.

const TABS := ["Seluruh", "Negeri", "Rakan"]
const SCOPES := ["all", "negeri", "friends"]
const SAMPLE_PATH := "res://data/leaderboard_sample.json"

var start_tab: int = 0
var _tab: int = 0
var _tab_row: HBoxContainer
var _body: VBoxContainer
var _req := 0  ## ignores answers from an older tab when switching fast


func _init() -> void:
    title_text = "PAPAN MARKAH"
    panel_size = Vector2(1180, 660)


func _build() -> void:
    var top := UI.hbox(8)
    _tab_row = UI.hbox(8)
    for i in TABS.size():
        var b := UI.button(TABS[i], "", 170)
        b.pressed.connect(_show.bind(i))
        _tab_row.add_child(b)
    top.add_child(_tab_row)
    top.add_child(UI.spacer())
    var how := UI.label("Kedudukan ikut jumlah EXP, kemudian menang pertarungan.", "Small")
    how.size_flags_vertical = Control.SIZE_SHRINK_CENTER
    top.add_child(how)
    content.add_child(top)
    _body = UI.vbox(10)
    _body.size_flags_vertical = Control.SIZE_EXPAND_FILL
    content.add_child(_body)
    _show(start_tab)


func _show(i: int) -> void:
    _tab = i
    _req += 1
    for j in _tab_row.get_child_count():
        (_tab_row.get_child(j) as Button).theme_type_variation = "Primary" if j == i else ""
    _clear()
    var negeri := str(Game.save["profile"].get("negeri", ""))
    if SCOPES[i] == "negeri" and negeri == "":
        _ask_negeri()
        return
    if Backend.can_play_online():
        _body.add_child(UI.label("Memuatkan...", "Small"))
        var req := _req
        var data: Dictionary = await Backend.leaderboard(SCOPES[i])
        if req != _req or not is_inside_tree():
            return
        _clear()
        if data.is_empty():
            _body.add_child(UI.label("Tak dapat sambung ke pelayan. Cuba lagi nanti.", "", true))
            return
        _fill(data.get("rows", []), data.get("me"), false)
    else:
        var rows := sample_rows(SCOPES[i])
        var me: Dictionary = {}
        for r in rows:
            if r["me"]:
                me = r
        _fill(rows, me, true)


```

```
func _clear() -> void:
    for c in _body.get_children():
        c.queue_free()


func _ask_negeri() -> void:
    var box := UI.vbox(14)
    box.alignment = BoxContainer.ALIGNMENT_CENTER
    box.size_flags_vertical = Control.SIZE_EXPAND_FILL
    var l := UI.label("Kau belum pilih negeri. Pilih dalam Profil, lepas tu tengok siapa paling hebat di negeri kau!", "", true)
    l.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    l.custom_minimum_size.x = 900
    box.add_child(UI.center(l))
    var b := UI.button("Pilih negeri di Profil", "Primary", 340)
    b.pressed.connect(func():
        close()
        Game.main.open_profile())
    box.add_child(UI.center(b))
    _body.add_child(box)


## Offline: sample classmates + you, ranked the same way as the server.
static func sample_rows(scope: String) -> Array:
    var data = JSON.parse_string(FileAccess.get_file_as_string(SAMPLE_PATH))
    var prof: Dictionary = Game.save["profile"]
    var negeri := str(prof.get("negeri", ""))
    var rows: Array = []
    for p in (data.get("players", []) if data is Dictionary else []):
        if scope == "negeri" and str(p["negeri"]) != negeri:
            continue
        if scope == "friends" and not p.get("friend", false):
            continue
        rows.append({"name": str(p["name"]), "skin": str(p["skin"]), "negeri": str(p["negeri"]),
            "exp": int(p["exp"]), "wins": int(p["wins"]), "me": false})
    rows.append({"name": str(prof.get("display_name", prof["username"])), "skin": Game.skin_folder(), "negeri": negeri,
        "exp": int(Game.save.get("exp", 0)), "wins": int(Game.save.get("battle", {}).get("wins", 0)), "me": true})
    rows.sort_custom(func(a, b): return a["exp"] > b["exp"] or (a["exp"] == b["exp"] and (a["wins"] > b["wins"] or (a["wins"] == b["wins"] and a["me"]))))
    var rank := 0
    for i in rows.size():
        if i == 0 or rows[i]["exp"] != rows[i - 1]["exp"] or rows[i]["wins"] != rows[i - 1]["wins"]:
            rank = i + 1
        rows[i]["rank"] = rank
    return rows


func _fill(rows: Array, me: Variant, sample: bool) -> void:
    if sample:
        var banner := UI.label("CONTOH: ini pemain contoh sebab permainan belum disambung ke pelayan. Markah kau sendiri adalah betul.", "Small", true)
        banner.add_theme_color_override("font_color", UI.GOLD)
        banner.custom_minimum_size.x = 1100
        _body.add_child(banner)
    if rows.is_empty():
        _body.add_child(UI.label("Belum ada pemain di sini.", ""))
        return
    var row := UI.hbox(18)
    row.size_flags_vertical = Control.SIZE_EXPAND_FILL
    row.add_child(_podium(rows))
    var right := UI.vbox(8)
    right.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var scroll := ScrollContainer.new()
    scroll.size_flags_vertical = Control.SIZE_EXPAND_FILL
    scroll.horizontal_scroll_mode = ScrollContainer.SCROLL_MODE_DISABLED
    var list := UI.vbox(6)
    list.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    for r in rows.slice(3):
        list.add_child(_row(r))
    if rows.size() <= 3:
        list.add_child(UI.label("Itu sahaja buat masa ni. Ajak kawan main!", "Small"))
    scroll.add_child(list)
    right.add_child(scroll)
    # your own place, always visible at the bottom
    if me is Dictionary and not (me as Dictionary).is_empty():
        var mine := _row(me)
        right.add_child(mine)
    row.add_child(right)
    _body.add_child(row)


## top 3 on a podium: 2nd | 1st | 3rd, with their heads big
func _podium(rows: Array) -> Control:
    var p := UI.hbox(8)
    p.alignment = BoxContainer.ALIGNMENT_CENTER
```

```
    p.custom_minimum_size = Vector2(450, 0)
    var order := [1, 0, 2]
    var heights := {0: 150.0, 1: 110.0, 2: 80.0}
    var colors := {0: UI.GOLD, 1: Color("c9d3dc"), 2: Color("d49a6a")}
    for i in order:
        if i >= rows.size():
            continue
        var r: Dictionary = rows[i]
        var col := UI.vbox(4)
        col.alignment = BoxContainer.ALIGNMENT_END
        col.custom_minimum_size.x = 140
        if i == 0:
            col.add_child(UI.center(UI.icon("res://assets/icons/mahkota.png", 44)))
        var head := UI.panel("Slot")
        head.add_child(UI.head_icon(str(r["skin"]), 96 if i == 0 else 80))
        col.add_child(UI.center(head))
        var n := UI.label(str(r["name"]), "Heading")  # you = gold name
        n.add_theme_font_size_override("font_size", 22)
        n.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        n.clip_text = true
        n.custom_minimum_size.x = 140
        if r.get("me", false):
            n.add_theme_color_override("font_color", UI.GOLD)
        col.add_child(n)
        var e := UI.label("%d EXP" % int(r["exp"]), "Small")
        e.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        col.add_child(e)
        var block := PanelContainer.new()
        block.add_theme_stylebox_override("panel", UI.sb(Color(colors[i], 0.85), Color(colors[i]).darkened(0.4), 3, 6,
4))
        block.custom_minimum_size = Vector2(130, heights[i])
        var num := UI.label("%d" % int(r["rank"]), "Title")
        num.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        num.add_theme_color_override("font_color", Color(0.17, 0.11, 0.06))
        block.add_child(num)
        col.add_child(UI.center(block))
        p.add_child(col)
    return p


func _row(r: Dictionary) -> Control:
    var me: bool = r.get("me", false)
    var panel := UI.panel("SlotGood" if me else "Slot")
    panel.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var h := UI.hbox(12)
    var rank := UI.label("#%d" % int(r["rank"]), "Heading")
    rank.custom_minimum_size.x = 70
    h.add_child(rank)
    h.add_child(UI.head_icon(str(r["skin"]), 48))
    var nc := UI.vbox(0)
    nc.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var n := UI.label(str(r["name"]) + (tr(" (kau)") if me else ""), "")
    n.clip_text = true
    if me:
        n.add_theme_color_override("font_color", UI.GOLD)
    nc.add_child(n)
    var neg := str(r.get("negeri", ""))
    nc.add_child(UI.label(neg if neg != "" else tr("(tiada negeri)"), "Small"))
    h.add_child(nc)
    var stats := UI.vbox(0)
    var lvl := UI.label(tr("Tahap %d  ·  %d EXP") % [Game.level_for(int(r["exp"])), int(r["exp"])], "")
    lvl.horizontal_alignment = HORIZONTAL_ALIGNMENT_RIGHT
    stats.add_child(lvl)
    var w := UI.label(tr("%d menang") % int(r["wins"]), "Small")
    w.horizontal_alignment = HORIZONTAL_ALIGNMENT_RIGHT
    stats.add_child(w)
    h.add_child(stats)
    panel.add_child(h)
    return panel
```

pau_button.gd · scripts/ui/pau_button.gd · 94 lines

```
extends Control
## Pau, always in the bottom-right corner. Tap = hint (3 levels inside a
## puzzle). Press and hold = calm moment.

const UI = preload("res://scripts/ui/ui.gd")
const HOLD_TIME := 0.6

signal tapped
signal held

var _sprite: TextureRect
```

```
var _frames: Array[Texture2D] = []
var _t: float = 0.0
var _press_time: float = -1.0
var _held_fired: bool = false
var _happy_t: float = 0.0
var _label: Label


func _ready() -> void:
    set_anchors_preset(Control.PRESET_BOTTOM_RIGHT)
    custom_minimum_size = Vector2(120, 130)
    size = custom_minimum_size
    position = Vector2(1280 - 130, 720 - 140)
    mouse_filter = Control.MOUSE_FILTER_STOP
    mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
    tooltip_text = "Ketik: petunjuk. Tekan lama: bertenang."
    var glow := PanelContainer.new()
    var gs := UI.sb(Color(0.12, 0.14, 0.2, 0.75), UI.GOLD, 3, 60, 0)
    glow.add_theme_stylebox_override("panel", gs)
    glow.position = Vector2(8, 4)
    glow.size = Vector2(104, 104)
    glow.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(glow)
    for i in 4:
        _frames.append(UI.sheet_frame("res://assets/proto/sprites/pau.png", i, 0, 16, 16))
    _sprite = TextureRect.new()
    _sprite.texture = _frames[0]
    _sprite.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    _sprite.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    _sprite.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    _sprite.size = Vector2(88, 88)
    _sprite.position = Vector2(16, 12)
    _sprite.pivot_offset = Vector2(44, 88)
    _sprite.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(_sprite)
    _label = UI.label("PAU", "Heading")
    _label.position = Vector2(36, 100)
    _label.add_theme_font_size_override("font_size", 20)
    _label.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(_label)


func happy() -> void:
    _happy_t = 1.2


func _process(delta: float) -> void:
    _t += delta
    var bounce := absf(sin(_t * 3.0))
    _sprite.position.y = 12 - bounce * 6.0
    if _press_time >= 0.0:
        var held_for := _t - _press_time
        _sprite.scale = Vector2.ONE * (1.0 + minf(held_for / HOLD_TIME, 1.0) * 0.18)
        _sprite.texture = _frames[1]
        if held_for >= HOLD_TIME and not _held_fired:
            _held_fired = true
            held.emit()
    else:
        _sprite.scale = Vector2.ONE
        if _happy_t > 0.0:
            _happy_t -= delta
            _sprite.texture = _frames[3]
        else:
            _sprite.texture = _frames[0] if bounce < 0.8 else _frames[2]


func _gui_input(event: InputEvent) -> void:
    if event is InputEventMouseButton and event.button_index == MOUSE_BUTTON_LEFT:
        if event.pressed:
            _press_time = _t
            _held_fired = false
        elif _press_time >= 0.0:
            var short := not _held_fired
            _press_time = -1.0
            if short:
                tapped.emit()
        accept_event()


func _unhandled_input(event: InputEvent) -> void:
    if visible and event.is_action_pressed("pau"):
        tapped.emit()
        get_viewport().set_input_as_handled()
```

peta_jejak.gd · scripts/ui/peta_jejak.gd · 84 lines

```
extends CanvasLayer
## Peta Jejak Pembelajaran overlay (GDS section 4), opened from the stone
## monument in Kampung Jejak. 10 flags along the path; unlocked ones wave,
## locked ones are grey. Tap a flag to read its chapter.

signal closed

## flag pole bases on the 320x180 illustrated map, in chapter order
const FLAGS := [
    Vector2(276, 118), Vector2(255, 92), Vector2(222, 86), Vector2(180, 96), Vector2(150, 104),
    Vector2(115, 112), Vector2(78, 112), Vector2(45, 100), Vector2(35, 72), Vector2(55, 48),
]
const OPEN := preload("res://assets/ui/bendera_berkibar.png")
const LOCKED := preload("res://assets/ui/bendera_berkunci.png")

var _flags: Array[TextureRect] = []
var _t := 0.0

@onready var map: TextureRect = $Map
@onready var info: Label = $InfoPanel/Info


func _ready() -> void:
    layer = 28
    Game.lock_input()
    $Close.pressed.connect(_close)
    Sfx.play("whoosh", -10.0)
    for i in FLAGS.size():
        var unlocked: bool = i < GameState.unlocked_chapters
        var f := TextureRect.new()
        f.texture = OPEN if unlocked else LOCKED
        f.size = Vector2(72, 96)
        f.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
        f.pivot_offset = Vector2(20, 96)
        f.position = FLAGS[i] * 4.0 - Vector2(20, 92)
        f.modulate = Color.WHITE if unlocked else Color(0.75, 0.75, 0.8)
        f.mouse_filter = Control.MOUSE_FILTER_STOP
        f.gui_input.connect(_on_flag_input.bind(i))
        map.add_child(f)
        _flags.append(f)
    _show_info(0)
    map.modulate.a = 0.0
    map.scale = Vector2(0.96, 0.96)
    map.pivot_offset = Vector2(640, 360)
    var t := create_tween().set_parallel()
    t.tween_property(map, "modulate:a", 1.0, 0.35)
    t.tween_property(map, "scale", Vector2.ONE, 0.35).set_trans(Tween.TRANS_BACK).set_ease(Tween.EASE_OUT)


func _process(delta: float) -> void:
    _t += delta
    for i in _flags.size():
        if i < GameState.unlocked_chapters:  # waving + glow on open chapters
            _flags[i].rotation = sin(_t * 3.0 + i) * 0.05
            _flags[i].self_modulate = Color(1, 1, 1).lerp(Color(1.35, 1.25, 0.9), (sin(_t * 3.0) + 1.0) * 0.25)


func _on_flag_input(event: InputEvent, i: int) -> void:
    if event is InputEventMouseButton and event.pressed:
        Sfx.play("select")
        _show_info(i)


func _show_info(i: int) -> void:
    var status := "Sedang diterokai" if i < GameState.unlocked_chapters else "Berkunci"
    if Game.chapter_done(i + 1):
        status = "Selesai!"
    info.text = "Bab %d: %s   ·   %s" % [i + 1, GameState.CHAPTERS[i], status]


func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("interact") or event.is_action_pressed("ui_cancel"):
        get_viewport().set_input_as_handled()
        _close()


func _close() -> void:
    Sfx.play("select")
    var t := create_tween()
    t.tween_property(map, "modulate:a", 0.0, 0.2)
    await t.finished
    Game.unlock_input()
    closed.emit()
```

```
    queue_free()
```

phone_chat.gd · scripts/ui/phone_chat.gd · 131 lines

```
extends Control
## A phone showing a group chat, messages popping in one by one (the prologue
## cold open). It plays and closes by itself; tap / E / Space only hurries it.
##   setup({title, time, messages: [{from, text}]})  from = characters.json id,
##   "alif" is the player's own message (right side, green)

const UI = preload("res://scripts/ui/ui.gd")
const BUBBLE := Color("fbf7ec")
const MINE := Color("d6f2c4")
const NAME_COL := {"aisyah": Color("b0457a"), "ravi": Color("2f7a4a"), "meiling": Color("3f68b8"), "daniel": Color("b5651d")}

signal done

var finished := false
var _msgs: Array = []
var _i := 0
var _gap := 0.8      ## seconds until the next message pops in
var _last_at := 0.0  ## when the last message appeared (no accidental double-tap close)
var _list: VBoxContainer


func setup(d: Dictionary) -> void:
    _msgs = d.get("messages", [])
    UI.full(self)
    mouse_filter = Control.MOUSE_FILTER_STOP
    var shade := ColorRect.new()
    shade.color = Color(0.03, 0.02, 0.07, 0.94)
    UI.full(shade)
    shade.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(shade)

    var phone := PanelContainer.new()
    phone.add_theme_stylebox_override("panel", UI.sb(Color("15171f"), Color("2c2f3a"), 10, 40, 14))
    phone.position = Vector2(370, 24)
    phone.size = Vector2(540, 672)
    phone.mouse_filter = Control.MOUSE_FILTER_IGNORE
    add_child(phone)
    var col := UI.vbox(0)
    phone.add_child(col)

    var head := PanelContainer.new()
    head.add_theme_stylebox_override("panel", UI.sb(Color("1f6f5c"), Color(0, 0, 0, 0), 0, 18, 14))
    var hrow := UI.hbox(12)
    head.add_child(hrow)
    var title := UI.label(str(d.get("title", "")), "Heading")
    title.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    hrow.add_child(title)
    hrow.add_child(UI.label(str(d.get("time", "")), "Small"))
    col.add_child(head)

    var chat := PanelContainer.new()
    chat.add_theme_stylebox_override("panel", UI.sb(Color("e9e2d2"), Color(0, 0, 0, 0), 0, 18, 14))
    chat.size_flags_vertical = Control.SIZE_EXPAND_FILL
    chat.clip_contents = true
    col.add_child(chat)
    _list = UI.vbox(10)
    _list.alignment = BoxContainer.ALIGNMENT_END
    chat.add_child(_list)


func _process(delta: float) -> void:
    if finished:
        return
    _gap -= delta
    if _gap <= 0.0:
        if _i < _msgs.size():
            _show_next()
        else:
            _close()


## next message now, or close once they have all been shown
func tap() -> void:
    if finished:
        return
    if _i < _msgs.size():
        _show_next()
    elif Time.get_ticks_msec() / 1000.0 - _last_at > 0.4:
```

```
        _close()


func _close() -> void:
    finished = true
    done.emit()


func _gui_input(event: InputEvent) -> void:
    if (event is InputEventMouseButton and event.pressed and event.button_index == MOUSE_BUTTON_LEFT) \
            or (event is InputEventScreenTouch and event.pressed):
        accept_event()
        tap()


func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("interact") or event.is_action_pressed("ui_accept"):
        get_viewport().set_input_as_handled()
        tap()


func _show_next() -> void:
    var m: Dictionary = _msgs[_i]
    _i += 1
    var who := str(m.get("from", ""))
    var text := str(m.get("text", ""))
    var mine := who == "alif"
    var row := UI.hbox(8)
    row.alignment = BoxContainer.ALIGNMENT_END if mine else BoxContainer.ALIGNMENT_BEGIN
    var bubble := PanelContainer.new()
    bubble.add_theme_stylebox_override("panel", UI.sb(MINE if mine else BUBBLE, Color(0, 0, 0, 0.12), 1, 14, 10))
    bubble.custom_minimum_size.x = 120
    var inner := UI.vbox(2)
    bubble.add_child(inner)
    if not mine:
        row.add_child(UI.head_icon("rakan_" + who, 44))
        var sender := UI.label(Game.char_name(who), "InkSmall")
        sender.add_theme_color_override("font_color", NAME_COL.get(who, UI.WOOD_D))
        inner.add_child(sender)
    var body := UI.label(text, "Ink", true)
    body.custom_minimum_size.x = mini(360, 40 + text.length() * 12)
    inner.add_child(body)
    row.add_child(bubble)
    _list.add_child(row)
    row.modulate.a = 0.0
    create_tween().tween_property(row, "modulate:a", 1.0, 0.2)
    Sfx.play("pop", -8.0, 1.25 if mine else 1.0)
    if bool(Game.settings.get("auto_read", false)):
        Tts.speak(text)
    _last_at = Time.get_ticks_msec() / 1000.0
    _gap = 1.0 + text.length() * 0.035  # time to read it
    if _i >= _msgs.size():
        _gap += 1.5  # linger on the last one
```

`profile_panel.gd` · scripts/ui/profile_panel.gd · 264 lines

```
extends "res://scripts/ui/modal.gd"
## Profil & Kedai: the player's card (display name, Player ID for adding
## friends, negeri for Papan Markah, motto, stats) and the skin shop. The
## profile picture is the head of the outfit being worn. Skins are Alif's
## outfits (data/skins.json), bought with syiling and worn in the world.

var start_tab: int = 0
var _tab: int = 0
var _body: VBoxContainer
var _tab_row: HBoxContainer
var _coins: Label
var _preview: TextureRect
var _preview_frames: Array[Texture2D] = []
var _t: float = 0.0


func _init() -> void:
    title_text = "PROFIL & KEDAI"
    panel_size = Vector2(1210, 650)


func _build() -> void:
    var top := UI.hbox(8)
    _tab_row = UI.hbox(8)
    for i in 2:
        var b := UI.button(["Profil", "Kedai Skin"][i], "", 200)
        b.pressed.connect(_show.bind(i))
```

```
        _tab_row.add_child(b)
    top.add_child(_tab_row)
    top.add_child(UI.spacer())
    top.add_child(UI.icon("res://assets/icons/syiling.png", 32))
    _coins = UI.label("", "Heading")
    top.add_child(_coins)
    content.add_child(top)
    _body = UI.vbox(12)
    _body.size_flags_vertical = Control.SIZE_EXPAND_FILL
    content.add_child(_body)
    Game.stats_changed.connect(_refresh_coins)
    _refresh_coins()
    _show(start_tab)


func _refresh_coins() -> void:
    if is_instance_valid(_coins):
        _coins.text = str(int(Game.save["coins"]))


func _process(delta: float) -> void:
    _t += delta
    if is_instance_valid(_preview) and not _preview_frames.is_empty():
        _preview.texture = _preview_frames[int(_t * 8.0) % _preview_frames.size()]


func _show(i: int) -> void:
    _tab = i
    for j in _tab_row.get_child_count():
        (_tab_row.get_child(j) as Button).theme_type_variation = "Primary" if j == i else ""
    for c in _body.get_children():
        c.queue_free()
    _preview = null
    if i == 0:
        _profile()
    else:
        _shop()


## walking-south frames of a skin, for previews
func _walk_frames(folder: String) -> Array[Texture2D]:
    var out: Array[Texture2D] = []
    var f := CharacterFrames.build(folder, true)
    for n in f.get_frame_count("walk_south"):
        out.append(f.get_frame_texture("walk_south", n))
    if out.is_empty():
        out.append(f.get_frame_texture("idle_south", 0))
    return out


# ---------------------------------------------------------------- profile
func _profile() -> void:
    var p: Dictionary = Game.save["profile"]
    var row := UI.hbox(24)
    row.size_flags_vertical = Control.SIZE_EXPAND_FILL

    # left: Alif in the equipped skin
    var left := UI.panel("Slot")
    left.custom_minimum_size = Vector2(300, 0)
    var lc := UI.vbox(6)
    _preview = TextureRect.new()
    _preview.custom_minimum_size = Vector2(260, 260)
    _preview.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    _preview.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    _preview.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    _preview_frames = _walk_frames(Game.skin_folder())
    lc.add_child(_preview)
    var sk := UI.label(str(Game.skin_def(str(Game.save["skin"]))["name"]), "Heading")
    sk.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    lc.add_child(sk)
    var change := UI.button("Tukar pakaian", "Blue")
    change.pressed.connect(_show.bind(1))
    lc.add_child(change)
    left.add_child(lc)
    row.add_child(left)

    # right: details
    var right := UI.vbox(12)
    right.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var grid := GridContainer.new()
    grid.columns = 2
    grid.add_theme_constant_override("h_separation", 18)
    grid.add_theme_constant_override("v_separation", 10)
    grid.add_child(UI.label("Nama paparan"))
    var name_e := LineEdit.new()
```

```
    name_e.text = str(p.get("display_name", p["username"]))
    name_e.max_length = 16
    name_e.custom_minimum_size.x = 360
    name_e.text_changed.connect(func(t: String):
        if t.strip_edges() != "":
            p["display_name"] = t.strip_edges()
            Game.persist()
            Game.stats_changed.emit())
    grid.add_child(name_e)
    grid.add_child(UI.label("ID Pemain"))
    var id_row := UI.hbox(8)
    var idl := UI.label(str(Game.save["player_id"]), "Heading")
    idl.add_theme_font_size_override("font_size", 30)
    id_row.add_child(idl)
    var copy := UI.button("Salin")
    copy.tooltip_text = tr("Beri ID ini kepada rakan supaya mereka boleh tambah kau.")
    copy.pressed.connect(func():
        DisplayServer.clipboard_set(str(Game.save["player_id"]))
        Game.toast(tr("ID disalin")))
    id_row.add_child(copy)
    grid.add_child(id_row)
    grid.add_child(UI.label("Moto"))
    var motto := LineEdit.new()
    motto.text = str(p.get("motto", ""))
    motto.placeholder_text = tr("Contoh: Sejarah? Boleh!")
    motto.max_length = 40
    motto.text_changed.connect(func(t: String):
        p["motto"] = t
        Game.persist())
    grid.add_child(motto)
    grid.add_child(UI.label("Negeri"))
    var neg := OptionButton.new()
    neg.custom_minimum_size = Vector2(360, 52)
    neg.add_item(tr("(Pilih negeri)"))
    for n in Game.NEGERI:
        neg.add_item(n)
    neg.selected = Game.NEGERI.find(str(p.get("negeri", ""))) + 1
    neg.item_selected.connect(func(i: int):
        p["negeri"] = Game.NEGERI[i - 1] if i > 0 else ""
        Game.persist()
        Sfx.play("select"))
    grid.add_child(neg)
    grid.add_child(UI.label("Gambar profil"))
    var pic_row := UI.hbox(10)
    var pic := UI.panel("Slot")
    pic.add_child(UI.head_icon(Game.skin_folder(), 64))
    pic_row.add_child(pic)
    var pn := UI.label("Ikut pakaian yang kau pakai.", "Small", true)
    pn.custom_minimum_size.x = 280
    pn.size_flags_vertical = Control.SIZE_SHRINK_CENTER
    pic_row.add_child(pn)
    grid.add_child(pic_row)
    right.add_child(grid)

    var stats := UI.hbox(10)
    var done := 0
    for z in Game.ZONES:
        if Game.zone_done(z):
            done += 1
    var battle: Dictionary = Game.save.get("battle", {})
    for s in [
        [tr("Tahap"), str(int(Game.save["level"]))],
        [tr("Misi siap"), "%d / %d" % [done, Game.ZONES.size()]],
        [tr("Pertarungan menang"), "%d / %d" % [int(battle.get("wins", 0)), int(battle.get("played", 0))]],
        [tr("Pakaian"), "%d / %d" % [Game.save["skins_owned"].size(), Game.skins().size()]],
    ]:
        var box := UI.panel("Slot")
        box.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        var bc := UI.vbox(0)
        var v := UI.label(s[1], "Heading")
        v.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        bc.add_child(v)
        var k := UI.label(s[0], "Small")
        k.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        bc.add_child(k)
        box.add_child(bc)
        stats.add_child(box)
    right.add_child(stats)
    row.add_child(right)
    _body.add_child(row)


# ---------------------------------------------------------------------- shop
func _shop() -> void:
    var hint := UI.label("Kumpul syiling dengan menyelesaikan misi dan menang pertarungan. Pakaian hanya untuk gaya, bu
```

```
kan kuasa.", "Small", true)
    _body.add_child(hint)
    var grid := GridContainer.new()
    grid.columns = 3
    grid.add_theme_constant_override("h_separation", 14)
    grid.add_theme_constant_override("v_separation", 14)
    for s in Game.skins():
        grid.add_child(_skin_card(s))
    var scroll := ScrollContainer.new()
    scroll.size_flags_vertical = Control.SIZE_EXPAND_FILL
    scroll.add_child(grid)
    _body.add_child(scroll)


func _skin_card(s: Dictionary) -> Control:
    var id := str(s["id"])
    var owned: bool = Game.save["skins_owned"].has(id)
    var worn: bool = str(Game.save["skin"]) == id
    var card := PanelContainer.new()
    card.theme_type_variation = "SlotGood" if worn else ("Slot" if owned else "SlotHot")
    card.custom_minimum_size = Vector2(366, 0)
    var row := UI.hbox(10)
    var pic := TextureRect.new()
    pic.custom_minimum_size = Vector2(96, 110)
    pic.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    pic.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    pic.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    var folder := str(s["folder"])
    var path := "res://assets/characters32/%s/%s_south.png" % [folder, folder]
    pic.texture = UI.tex(path) if ResourceLoader.exists(path) else UI.tex("res://assets/icons/kunci.png")
    row.add_child(pic)
    var col := UI.vbox(4)
    col.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    var nm := UI.label(str(s["name"]), "Heading", true)
    nm.custom_minimum_size.x = 220
    col.add_child(nm)
    var info := UI.label(str(s.get("info", "")), "Small", true)
    info.custom_minimum_size.x = 220
    col.add_child(info)
    var btn: Button
    if worn:
        btn = UI.button("Sedang dipakai")
        btn.disabled = true
    elif owned:
        btn = UI.button("Pakai", "Primary")
        btn.pressed.connect(func():
            Game.equip_skin(id)
            Sfx.play("pop")
            _show(1))
    else:
        var price := int(s["price"])
        btn = UI.button(tr("Beli: %d syiling") % price, "Blue")
        btn.icon = UI.tex("res://assets/icons/syiling.png")
        btn.expand_icon = false
        btn.pressed.connect(func():
            var err := Game.buy_skin(id)
            if err != "":
                Sfx.play("bump")
                Game.toast(err, Color("ff9a8a"))
                return
            MusicDirector.stinger("st_item")
            Game.toast(tr("Dapat pakaian baharu: %s!") % str(s["name"]))
            Game.equip_skin(id)
            _show(1))
    col.add_child(btn)
    row.add_child(col)
    card.add_child(row)
    return card
```

`settings_panel.gd` · scripts/ui/settings_panel.gd · 281 lines

```
extends "res://scripts/ui/modal.gd"
## Tetapan, in tabs: Paparan (Mod Mudah Baca: font, text size and speed,
## read-aloud), Audio (music, sound, ambience, voice), Kawalan (on-screen
## controller for phones), Bahasa (menus in BM or English) and Akaun.

const TABS := ["Paparan", "Audio", "Kawalan", "Bahasa", "Akaun"]

var _tab: int = 0
var _body: VBoxContainer
var _tab_row: HBoxContainer


```

```
func _init() -> void:
    title_text = "TETAPAN"
    panel_size = Vector2(860, 580)


func _build() -> void:
    _tab_row = UI.hbox(8)
    for i in TABS.size():
        var b := UI.button(TABS[i])
        b.size_flags_horizontal = Control.SIZE_EXPAND_FILL
        b.pressed.connect(_show.bind(i))
        _tab_row.add_child(b)
    content.add_child(_tab_row)
    _body = UI.vbox(14)
    _body.size_flags_vertical = Control.SIZE_EXPAND_FILL
    content.add_child(_body)
    _show(_tab)


func _show(i: int) -> void:
    _tab = i
    for j in _tab_row.get_child_count():
        (_tab_row.get_child(j) as Button).theme_type_variation = "Primary" if j == i else ""
    for c in _body.get_children():
        c.queue_free()
    match i:
        0: _paparan()
        1: _audio()
        2: _kawalan()
        3: _bahasa()
        4: _akaun()


func _grid() -> GridContainer:
    var grid := GridContainer.new()
    grid.columns = 2
    grid.add_theme_constant_override("h_separation", 24)
    grid.add_theme_constant_override("v_separation", 14)
    _body.add_child(grid)
    return grid


# ---------------------------------------------------------------- tabs
func _paparan() -> void:
    _body.add_child(UI.label("Mod Mudah Baca", "Heading"))
    var grid := _grid()
    grid.add_child(UI.label("Fon"))
    grid.add_child(_choice(["lexend", "atkinson", "opendyslexic"], ["Lexend", "Atkinson", "OpenDyslexic"], "font"))
    grid.add_child(UI.label("Saiz teks"))
    grid.add_child(_choice([0, 1, 2], ["Kecil", "Sederhana", "Besar"], "text_size"))
    grid.add_child(UI.label("Kelajuan teks"))
    grid.add_child(_choice([0, 1, 2], ["Perlahan", "Biasa", "Serta-merta"], "text_speed"))
    grid.add_child(UI.label("Baca dialog dengan suara"))
    grid.add_child(_toggle("auto_read"))
    var sample := UI.panel("Paper")
    var col := UI.vbox(4)
    col.add_child(UI.label("Tajuk 1 2 3 4 5 6 7 8 9 0", "InkHeading"))
    col.add_child(UI.label("Contoh: Kesultanan Melayu Melaka muncul pada abad ke-15.", "Ink", true))
    sample.add_child(col)
    _body.add_child(sample)


func _audio() -> void:
    _body.add_child(UI.label("Bunyi", "Heading"))
    var grid := _grid()
    for row in [["Muzik", "music"], ["Bunyi kesan", "sfx"], ["Suara watak", "voice"]]:
        grid.add_child(UI.label(row[0]))
        grid.add_child(_slider(row[1]))
    grid.add_child(UI.label("Senyapkan semua bunyi"))
    grid.add_child(_toggle("muted"))


func _kawalan() -> void:
    _body.add_child(UI.label("Kawalan skrin sentuh", "Heading"))
    var row := UI.hbox(20)
    var grid := GridContainer.new()
    grid.columns = 2
    grid.add_theme_constant_override("h_separation", 18)
    grid.add_theme_constant_override("v_separation", 12)
    row.add_child(grid)
    grid.add_child(UI.label("Jenis"))
    var kinds := UI.vbox(6)
    var r1 := _choice(["auto", "dpad"], ["Automatik", "D-pad"], "controls")
    var r2 := _choice(["joystick", "tap"], ["Joystick", "Ketik sahaja"], "controls")
    _join_choices(r1, r2)
```

```
    kinds.add_child(r1)
    kinds.add_child(r2)
    grid.add_child(kinds)
    grid.add_child(UI.label("Saiz butang"))
    grid.add_child(_choice([0, 1, 2], ["Kecil", "Sederhana", "Besar"], "pad_size"))
    grid.add_child(UI.label("Kelegapan"))
    grid.add_child(_slider("pad_alpha", 30, 240))
    grid.add_child(UI.label("Tangan kiri"))
    grid.add_child(_toggle("left_handed"))
    row.add_child(ControlsPreview.new())
    _body.add_child(row)
    var note := UI.label(tr(MODE_NOTES.get(str(Game.settings.get("controls", "auto")), "")), "Small", true)
    note.custom_minimum_size.x = 800
    _body.add_child(note)
    Game.settings_changed.connect(func():
        if is_instance_valid(note):
            note.text = tr(MODE_NOTES.get(str(Game.settings.get("controls", "auto")), "")))


const MODE_NOTES := {
    "auto": "Automatik: D-pad muncul di telefon dan tablet. Di komputer, guna papan kekunci (WASD / anak panah) atau klik peta.",
    "dpad": "D-pad: anak panah untuk berjalan, A untuk bercakap atau periksa, B (tekan lama) untuk berlari.",
    "joystick": "Joystick: letak ibu jari di mana-mana sebelah bawah kiri dan tolak. Tolak sampai hujung untuk berlari. A untuk bercakap.",
    "tap": "Ketik sahaja: tiada butang. Ketik tempat atau orang di peta, Alif akan jalan ke sana sendiri.",
}


## the two rows of controller choices act as one group
func _join_choices(a: HBoxContainer, b: HBoxContainer) -> void:
    var group: ButtonGroup = (a.get_child(0) as Button).button_group
    for c in b.get_children():
        (c as Button).button_group = group
    for c in a.get_children() + b.get_children():
        (c as Button).pressed.connect(func():
            for o in a.get_children() + b.get_children():
                (o as Button).theme_type_variation = "Primary" if o == c else "")


func _bahasa() -> void:
    _body.add_child(UI.label("Bahasa menu", "Heading"))
    var grid := _grid()
    grid.add_child(UI.label("Bahasa"))
    var row := _choice(["ms", "en"], ["Bahasa Melayu", "English"], "lang")
    for b in row.get_children():
        (b as Button).pressed.connect(func(): _show.call_deferred(3))  # redraw in the new language
    grid.add_child(row)
    var note := UI.label("Menu, butang dan peta ikut bahasa ini. Dialog, teka-teki dan nota sejarah kekal dalam Bahasa Melayu, sama seperti peperiksaan.", "Small", true)
    note.custom_minimum_size.x = 760
    _body.add_child(note)


func _akaun() -> void:
    var who := UI.label(tr("Pemain: %s") % Game.save["profile"]["username"], "Heading")
    _body.add_child(who)
    var acc := UI.hbox(12)
    var reset := UI.button("Mula semula permainan")
    reset.pressed.connect(_confirm_reset.bind(reset))
    acc.add_child(reset)
    var out := UI.button("Log keluar")
    out.pressed.connect(func():
        Backend.sign_out()
        close()
        Game.goto("login"))
    acc.add_child(out)
    _body.add_child(acc)


# ---------------------------------------------------------------- widgets
func _choice(values: Array, names: Array, key: String) -> HBoxContainer:
    var row := UI.hbox(8)
    var group := ButtonGroup.new()
    for i in values.size():
        var b := UI.button(str(names[i]))
        b.toggle_mode = true
        b.button_group = group
        b.button_pressed = str(Game.settings.get(key)) == str(values[i])
        if b.button_pressed:
            b.theme_type_variation = "Primary"
        var v = values[i]
        b.pressed.connect(func():
            Game.set_setting(key, v)
            for c in row.get_children():
```

```
                (c as Button).theme_type_variation = "Primary" if c == b else "")
        row.add_child(b)
    return row


func _toggle(key: String) -> CheckButton:
    var c := CheckButton.new()
    c.button_pressed = bool(Game.settings.get(key, false))
    c.text = tr("Ya") if c.button_pressed else tr("Tidak")
    c.toggled.connect(func(on: bool):
        c.text = tr("Ya") if on else tr("Tidak")
        Game.set_setting(key, on))
    return c


func _slider(key: String, min_v := 0, width := 380) -> HBoxContainer:
    var row := UI.hbox(10)
    var s := HSlider.new()
    s.min_value = min_v
    s.max_value = 100
    s.step = 5
    s.value = float(Game.settings.get(key, 0.8)) * 100.0
    s.custom_minimum_size = Vector2(width, 36)
    s.size_flags_vertical = Control.SIZE_SHRINK_CENTER
    var pct := UI.label("%d%%" % int(s.value), "Small")
    pct.custom_minimum_size.x = 60
    s.value_changed.connect(func(v: float):
        pct.text = "%d%%" % int(v)
        Game.set_setting(key, v / 100.0))
    s.drag_ended.connect(func(_c): Sfx.play("blip" if key == "voice" else "pop", 0.0, 1.2, "Voice" if key == "voice" el
se "SFX"))
    row.add_child(s)
    row.add_child(pct)
    return row


func _confirm_reset(btn: Button) -> void:
    if btn.text != tr("Pasti? Ketik lagi"):
        btn.text = tr("Pasti? Ketik lagi")
        return
    Game.reset_progress()
    close()
    Game.goto("peta")


## a little 16:9 phone screen showing where the controls will sit
class ControlsPreview extends Control:
    const W := 300.0

    func _init() -> void:
        custom_minimum_size = Vector2(W, W * 9.0 / 16.0)
        size_flags_vertical = Control.SIZE_SHRINK_CENTER
        Game.settings_changed.connect(queue_redraw)

    func _draw() -> void:
        var k := W / 1280.0
        draw_rect(Rect2(Vector2.ZERO, size), Color("4f8a4a"))
        draw_rect(Rect2(Vector2(0, size.y * 0.62), Vector2(size.x * 0.35, size.y * 0.38)), Color("3b8fc4"))
        draw_rect(Rect2(Vector2(size.x * 0.45, size.y * 0.25), Vector2(14, 20)), Color("f2efe6"))  # Alif
        draw_rect(Rect2(Vector2.ZERO, size), Color(0.17, 0.11, 0.06), false, 4.0)
        var m := Game.touch_mode()
        var s: float = [0.8, 1.0, 1.25][clampi(int(Game.settings.get("pad_size", 1)), 0, 2)]
        var a := clampf(float(Game.settings.get("pad_alpha", 0.7)), 0.3, 1.0)
        var lefty := bool(Game.settings.get("left_handed", false))
        var fx := func(x: float) -> float: return size.x - x if lefty else x
        var ink := Color(0.12, 0.08, 0.05, 0.6 * a)
        var cream := Color(1, 0.95, 0.8, a)
        var gold := Color("e8b04a", a)
        var blue := Color("7aa7d9", a)
        var pad := 96.0 * s * k
        var btn := 112.0 * s * k
        var mg := 36.0 * k
        # minimap + Pau move up when controls are on screen
        var mm := Rect2(Vector2(size.x - 12 - 60, size.y - 12 - 46), Vector2(60, 46))
        if m != "none":
            mm.position = Vector2(8, 166 * k) if lefty else Vector2(size.x - 8 - 60, 150 * k)
        draw_rect(mm, Color(0.95, 0.9, 0.75, 0.9))
        draw_rect(mm, Color(0.17, 0.11, 0.06), false, 2.0)
        match m:
            "dpad":
                var c := Vector2(mg + pad * 1.5, size.y - mg - pad * 1.5)
                c.x = fx.call(c.x)
                for d in [Vector2.UP, Vector2.DOWN, Vector2.LEFT, Vector2.RIGHT]:
                    draw_rect(Rect2(c + d * pad - Vector2(pad, pad) * 0.46, Vector2(pad, pad) * 0.92), ink)
                    var tip: Vector2 = c + d * pad * 1.25
```

```
                    var side: Vector2 = d.orthogonal() * pad * 0.2
                    draw_colored_polygon(PackedVector2Array([tip, c + d * pad * 0.8 + side, c + d * pad * 0.8 - side]), cream)
                draw_circle(Vector2(fx.call(size.x - mg - btn / 2), size.y - mg - btn * 0.95), btn / 2, gold)
                draw_circle(Vector2(fx.call(size.x - mg - btn * 1.6), size.y - mg - btn * 0.4), btn / 2, blue)
            "joystick":
                var r := 80.0 * s * k
                var c := Vector2(fx.call(mg + r * 1.6), size.y - mg - r * 1.6)
                draw_circle(c, r, ink)
                draw_arc(c, r, 0, TAU, 32, cream, 2.0, true)
                draw_circle(c + Vector2(r * 0.35, -r * 0.2), r * 0.45, gold)
                draw_circle(Vector2(fx.call(size.x - mg - btn * 0.7), size.y - mg - btn * 0.7), btn / 2, gold)
            _:
                # tap: a finger tapping the ground, with the dotted path Alif walks
                var to := Vector2(size.x * 0.7, size.y * 0.7)
                var from := Vector2(size.x * 0.45 + 7, size.y * 0.25 + 20)
                for i in 8:
                    draw_circle(from.lerp(to, i / 8.0), 2.5, Color(1, 1, 1, 0.8))
                draw_arc(to, 10, 0, TAU, 24, Color.WHITE, 3.0, true)
```

`start_prompt.gd` · scripts/ui/start_prompt.gd · 71 lines

```
extends Control
## "Do this first" indicator for the start of the game: a blinking key-cap
## card ("Tekan [E] untuk ambil beg") with a gold arrow bobbing over the
## thing to use. It follows that spot on screen and fades out with done()
## once the player has done it. The card says "Ketik" on touch screens.

const UI = preload("res://scripts/ui/ui.gd")

var target: Node2D          ## the thing to use (its position is the arrow tip)
var lift := 26.0            ## world px above target.position for the arrow
var key := "E"              ## keyboard key on the card
var action_text := ""       ## "untuk ambil beg"

var _t := 0.0
var _card: PanelContainer
var _done := false


func _ready() -> void:
    UI.full(self)
    theme = Game.theme
    mouse_filter = Control.MOUSE_FILTER_IGNORE
    _card = UI.panel("Paper")
    _card.mouse_filter = Control.MOUSE_FILTER_IGNORE
    var row := UI.hbox(10)
    if Game.controls_on_screen():
        row.add_child(UI.label(tr("Ketik"), "InkHeading"))
    else:
        row.add_child(UI.label(tr("Tekan"), "InkHeading"))
        var cap := UI.panel()  # the key, as a dark key-cap
        var k := UI.label(key, "Heading")
        k.add_theme_color_override("font_color", UI.GOLD)
        cap.add_child(k)
        row.add_child(cap)
    row.add_child(UI.label(tr(action_text), "InkHeading"))
    _card.add_child(row)
    add_child(_card)
    modulate.a = 0.0
    create_tween().tween_property(self, "modulate:a", 1.0, 0.3)
    Sfx.play("pop", -8.0, 1.2)


## the player did it: fade out and go
func done() -> void:
    if _done:
        return
    _done = true
    var tw := create_tween()
    tw.tween_property(self, "modulate:a", 0.0, 0.25)
    tw.tween_callback(queue_free)


func _process(delta: float) -> void:
    _t += delta
    visible = not Main.is_dialogue_open()  # never on top of a conversation
    if not is_instance_valid(target):
        return
    _card.modulate.a = 0.78 + 0.22 * sin(_t * 4.0)  # a gentle blink, always readable
    _card.reset_size()
    var tip := _tip()
    _card.position = Vector2(clampf(tip.x - _card.size.x / 2.0, 8, size.x - _card.size.x - 8), maxf(tip.y - 52 - _card.
```

```
size.y, 8))
    queue_redraw()


func _tip() -> Vector2:
    return get_viewport().get_canvas_transform() * (target.global_position + Vector2(0, -lift))


func _draw() -> void:
    if is_instance_valid(target):
        MapView.draw_nav_arrow(self, _tip() + Vector2(0, -22 + sin(_t * 5.0) * 5.0), PI / 2.0, 22)
```

`title_backdrop.gd` · scripts/ui/title_backdrop.gd · 98 lines

```
extends Control
## The game's opening backdrop (title, login, register). With the PixelLab
## layers in assets/backgrounds/tajuk/ it is a slow parallax scene: sky, far
## Melaka skyline, kampung hills, then the foreground (tree, Alif and Pau on
## the hill). Each layer is drawn at 4x with nearest filtering so the pixels
## stay sharp; the three back layers are 352x180 so they can drift sideways.
## Without the layers it shows the one-piece PixelLab scene (tajuk_penuh.png),
## else the old picture. Warm pollen drifts in front either way.

const OLD := "res://assets/backgrounds/bg_skrin_tajuk.png"
const FULL := "res://assets/backgrounds/tajuk/tajuk_penuh.png"
const LAYERS := [
    "res://assets/backgrounds/tajuk/tajuk_0_langit.png",
    "res://assets/backgrounds/tajuk/tajuk_1_jauh.png",
    "res://assets/backgrounds/tajuk/tajuk_2_tengah.png",
    "res://assets/backgrounds/tajuk/tajuk_3_depan.png",
]
## how far each layer sways (art pixels): the sky barely, the hills more
const DRIFT := [2.0, 5.0, 9.0, 0.0]
const PX := 4.0  ## art pixel -> screen pixel (320x180 art on 1280x720)

var tint := Color.WHITE  ## login / register darken it a little
var _layers: Array[TextureRect] = []
var _t := 0.0
var _motes: Control


static func layered() -> bool:
    for p in LAYERS:
        if not ResourceLoader.exists(p):
            return false
    return true


func _ready() -> void:
    set_anchors_and_offsets_preset(Control.PRESET_FULL_RECT)
    mouse_filter = Control.MOUSE_FILTER_IGNORE
    modulate = tint
    _motes = Motes.new()
    if not layered():
        var r := TextureRect.new()
        r.texture = load(FULL if ResourceLoader.exists(FULL) else OLD)
        r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
        r.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_COVERED
        r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
        r.set_anchors_and_offsets_preset(Control.PRESET_FULL_RECT)
        r.mouse_filter = Control.MOUSE_FILTER_IGNORE
        add_child(r)
        add_child(_motes)
        return
    for p in LAYERS:
        var r := TextureRect.new()
        r.texture = load(p)
        r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
        r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
        r.mouse_filter = Control.MOUSE_FILTER_IGNORE
        r.size = r.texture.get_size() * PX
        add_child(r)
        _layers.append(r)
    add_child(_motes)
    _place()


func _process(delta: float) -> void:
    if not _layers.is_empty():
        _t += delta
        _place()


```

```
## centred, swaying slowly and leaning a touch towards the mouse
func _place() -> void:
    var view := get_viewport_rect().size
    var lean := clampf((get_viewport().get_mouse_position().x - view.x / 2.0) / (view.x / 2.0), -1.0, 1.0)
    for i in _layers.size():
        var r := _layers[i]
        var sway := (sin(_t * 0.12 + i) * 0.7 + lean * 0.3) * float(DRIFT[i])
        r.position = Vector2(round((view.x - r.size.x) / 2.0 + sway * PX), view.y - r.size.y)


## golden pollen floating up through the morning light, in 4x4 "art pixels"
class Motes extends Control:
    var _t := 0.0

    func _ready() -> void:
        set_anchors_and_offsets_preset(Control.PRESET_FULL_RECT)
        mouse_filter = Control.MOUSE_FILTER_IGNORE

    func _process(delta: float) -> void:
        _t += delta
        queue_redraw()

    func _draw() -> void:
        for i in 18:
            var speed := 6.0 + (i * 7) % 9
            var x := fmod(i * 71.0 + sin(_t * 0.5 + i) * 6.0, 320.0)
            var y := 180.0 - fmod(_t * speed + i * 37.0, 200.0)
            var a := 0.25 + 0.35 * absf(sin(_t * 1.3 + i))
            draw_rect(Rect2(Vector2(floorf(x), floorf(y)) * 4.0, Vector2(4, 4)), Color(1, 0.92, 0.6, a))
```

`touch_controls.gd` · scripts/ui/touch_controls.gd · 267 lines

```
extends CanvasLayer
## On-screen controls for phones, picked in Tetapan > Kawalan:
##  - dpad: D-pad + A (act) / B (run)
##  - joystick: a stick that appears where the thumb lands (bottom-left) + A;
##    push it to the edge to run
##  - none ("Ketik sahaja", or Automatik on a laptop): nothing on screen, tap
##    the map to walk (world.gd _tap)
## Everything presses the same input actions as the keyboard, so player.gd
## does not care which one is used. Size, transparency and left-handed
## layout come from the settings too and apply straight away.

const SIZE := 96          ## D-pad arrow size (screen px, 1280x720 space)
const BTN := 112          ## A/B button diameter
const MARGIN := 36
const SCALES := [0.8, 1.0, 1.25]

var mode := ""
var brawl := false       ## Gelanggang: fight buttons instead of A/B
var suspended := false   ## the screen hides the controls (e.g. while answering)
var _built := ""
var _zones: Array[Rect2] = []  ## screen areas the controls own (taps there don't walk)
var _stick: Stick


func _ready() -> void:
    layer = 20
    Game.settings_changed.connect(_rebuild)
    _rebuild()


func _exit_tree() -> void:
    Input.emulate_touch_from_mouse = false
    if _stick:
        _stick.release()


func _process(_delta: float) -> void:
    # out of the way during dialogue, puzzles and menus
    var on := mode != "none" and not suspended and not Game.is_locked() and Game.active_puzzle == null
    if visible != on:
        visible = on
        if not on and _stick:
            _stick.release()


## true when a tap at this screen position belongs to the controls, so the
## world should not also walk there
func owns(pos: Vector2) -> bool:
    if not visible:
        return false
    for r in _zones:
```

```
        if r.has_point(pos):
            return true
    return false


# ---------------------------------------------------------------- layout
func _rebuild() -> void:
    var m := Game.touch_mode()
    var s: float = SCALES[clampi(int(Game.settings.get("pad_size", 1)), 0, 2)]
    var alpha := clampf(float(Game.settings.get("pad_alpha", 0.7)), 0.3, 1.0)
    var lefty := bool(Game.settings.get("left_handed", false))
    var key := "%s|%s|%s|%s" % [m, s, alpha, lefty]
    if key == _built:
        return
    _built = key
    mode = m
    if _stick:
        _stick.release()
    _stick = null
    _zones.clear()
    for c in get_children():
        remove_child(c)
        c.queue_free()
    # a laptop that picked D-pad/joystick on purpose: let the mouse drive them
    Input.emulate_touch_from_mouse = m != "none" and not DisplayServer.is_touchscreen_available()
    var pad := SIZE * s
    var btn := BTN * s
    match m:
        "dpad":
            var c := Vector2(MARGIN + pad * 1.5, 720 - MARGIN - pad * 1.5)
            _zones.append(_flip(Rect2(c - Vector2(pad, pad) * 1.5, Vector2(pad, pad) * 3), lefty))
            _add_button("move_up", _arrow_tex(0), c + Vector2(-pad / 2.0, -pad * 1.5), s, alpha, lefty)
            _add_button("move_down", _arrow_tex(2), c + Vector2(-pad / 2.0, pad * 0.5), s, alpha, lefty)
            _add_button("move_left", _arrow_tex(3 if not lefty else 1), c + Vector2(-pad * 1.5, -pad / 2.0), s, alpha,
lefty, "move_right" if lefty else "")
            _add_button("move_right", _arrow_tex(1 if not lefty else 3), c + Vector2(pad * 0.5, -pad / 2.0), s, alpha,
lefty, "move_left" if lefty else "")
            if not brawl:
                _add_button("interact", _round_tex(Color("e8b04a")), Vector2(1280 - MARGIN - btn, 720 - MARGIN - btn *
1.45), s, alpha, lefty, "", "A")
                _add_button("run", _round_tex(Color("7aa7d9")), Vector2(1280 - MARGIN - btn * 2.1, 720 - MARGIN - btn *
 0.9), s, alpha, lefty, "", "B")
        "joystick":
            _stick = Stick.new()
            _stick.radius = 80.0 * s
            _stick.zone = _flip(Rect2(0, 720 * 0.35, 1280 * 0.42, 720 * 0.65), lefty)
            var rest := Vector2(MARGIN + _stick.radius * 1.6, 720 - MARGIN - _stick.radius * 1.6)
            _stick.rest = Vector2(1280 - rest.x, rest.y) if lefty else rest
            _stick.base = _stick.rest
            _stick.modulate.a = alpha
            add_child(_stick)
            _zones.append(_stick.zone)
            if not brawl:
                _add_button("interact", _round_tex(Color("e8b04a")), Vector2(1280 - MARGIN - btn * 1.2, 720 - MARGIN -
btn * 1.2), s, alpha, lefty, "", "A")
    if brawl and m != "none":
        _fight_buttons(s, alpha, lefty)


## Gelanggang: Tumbuk (big), Sepak, Lompat, Ambil/Baling, Tangkis, Jurus
func _fight_buttons(s: float, alpha: float, lefty: bool) -> void:
    var d := BTN * s
    var base := Vector2(1280 - MARGIN - d, 720 - MARGIN - d)
    var k := 0.78   # the smaller buttons
    for b in [
        ["attack_punch", "Tumbuk", Color("e8b04a"), base, 1.0],
        ["attack_kick", "Sepak", Color("e0805a"), base + Vector2(-d * 1.1, d * 0.2), k],
        ["jump", "Lompat", Color("7aa7d9"), base + Vector2(d * 0.1, -d * 0.95), k],
        ["grab", "Ambil", Color("8ac27a"), base + Vector2(-d * 1.05, -d * 0.8), k],
        ["block", "Tangkis", Color("a8b0bc"), base + Vector2(-d * 2.05, d * 0.25), k],
        ["special", "Jurus", Color("c9a0ff"), base + Vector2(-d * 2.0, -d * 0.75), k],
    ]:
        _add_button(b[0], _round_tex(b[2]), b[3], s * b[4], alpha, lefty, "", tr(b[1]), int(22 / b[4]))


## mirror a rect to the other side of the screen for left-handed players
func _flip(r: Rect2, lefty: bool) -> Rect2:
    if lefty:
        r.position.x = 1280 - r.position.x - r.size.x
    return r


## action_override: left-handed mirrors the pad, so the button on the left side
## must still show and press "left" (and the right one "right")
func _add_button(action: String, tex: Texture2D, pos: Vector2, s: float, alpha: float, lefty: bool, action_override :=
```

```
"", label := "", font_size := 44) -> void:
    var b := TouchScreenButton.new()
    b.texture_normal = tex
    b.action = action_override if action_override != "" else action
    b.scale = Vector2(s, s)
    var w := tex.get_size().x * s
    b.position = Vector2(1280 - pos.x - w, pos.y) if lefty else pos
    b.passby_press = true  # slide a thumb across the D-pad without lifting
    b.visibility_mode = TouchScreenButton.VISIBILITY_ALWAYS
    b.modulate = Color(1, 1, 1, alpha)
    add_child(b)
    if label != "":
        _zones.append(Rect2(b.position, tex.get_size() * s))
        var l := Label.new()
        l.text = label
        l.size = tex.get_size()
        l.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        l.vertical_alignment = VERTICAL_ALIGNMENT_CENTER
        l.add_theme_font_size_override("font_size", font_size)
        l.add_theme_color_override("font_color", Color(0.17, 0.11, 0.06))
        b.add_child(l)


## rounded square with a triangle pointing up/right/down/left (0..3)
static func _arrow_tex(dir: int) -> Texture2D:
    var img := Image.create(SIZE, SIZE, false, Image.FORMAT_RGBA8)
    var bg := Color(0.12, 0.08, 0.05, 0.55)
    var fg := Color(1, 0.95, 0.8, 0.95)
    for y in SIZE:
        for x in SIZE:
            var p := Vector2(x, y) - Vector2(SIZE, SIZE) / 2.0
            if absf(p.x) < SIZE * 0.46 and absf(p.y) < SIZE * 0.46:
                img.set_pixel(x, y, bg)
            var q := p.rotated(-dir * PI / 2.0)  # draw an "up" triangle in rotated space
            if q.y > -SIZE * 0.22 and q.y < SIZE * 0.14 and absf(q.x) < (q.y + SIZE * 0.22) * 0.75:
                img.set_pixel(x, y, fg)
    return ImageTexture.create_from_image(img)


static func _round_tex(color: Color) -> Texture2D:
    var img := Image.create(BTN, BTN, false, Image.FORMAT_RGBA8)
    var r := BTN / 2.0
    for y in BTN:
        for x in BTN:
            var d := Vector2(x + 0.5, y + 0.5).distance_to(Vector2(r, r))
            if d < r - 1:
                img.set_pixel(x, y, color.darkened(0.25) if d > r - 7 else color)
    return ImageTexture.create_from_image(img)


# ---------------------------------------------------------------- joystick
## Floating stick: waits faintly at its rest spot, jumps to wherever the thumb
## lands inside its zone and follows the thumb if it slides too far. The
## strongest axis presses one of the four move actions (the world is a grid);
## pushing to the rim also presses "run".
class Stick extends Node2D:
    const DEAD := 0.3
    const RUN := 0.85
    const MOVES := ["move_up", "move_right", "move_down", "move_left"]

    var radius := 80.0
    var zone := Rect2()
    var rest := Vector2.ZERO
    var base := Vector2.ZERO
    var knob := Vector2.ZERO   ## thumb offset from base
    var touch := -1            ## finger index holding the stick, -1 = none
    var _held := {}            ## action -> pressed by us
    var _dir := ""

    func _input(e: InputEvent) -> void:
        if not is_visible_in_tree():
            return
        if e is InputEventScreenTouch:
            if e.pressed and touch < 0 and zone.has_point(e.position):
                touch = e.index
                base = e.position
                _move(e.position)
                get_viewport().set_input_as_handled()
            elif not e.pressed and e.index == touch:
                release()
                get_viewport().set_input_as_handled()
        elif e is InputEventScreenDrag and e.index == touch:
            _move(e.position)
            get_viewport().set_input_as_handled()

```

```
    func _move(pos: Vector2) -> void:
        var v := pos - base
        if v.length() > radius:
            base = pos - v.normalized() * radius
            v = pos - base
        knob = v
        var n := v / radius
        var want := ""
        if n.length() >= DEAD:
            # a little stickiness near the diagonals so the direction doesn't flicker
            var horiz := absf(n.x) > absf(n.y)
            if _dir in ["move_left", "move_right"]:
                horiz = absf(n.x) * 1.25 > absf(n.y)
            elif _dir in ["move_up", "move_down"]:
                horiz = absf(n.x) > absf(n.y) * 1.25
            if horiz:
                want = "move_right" if n.x > 0 else "move_left"
            else:
                want = "move_down" if n.y > 0 else "move_up"
        _apply(want, n.length() >= RUN)
        queue_redraw()

    func release() -> void:
        touch = -1
        knob = Vector2.ZERO
        base = rest
        _apply("", false)
        queue_redraw()

    func _apply(dir: String, run: bool) -> void:
        _dir = dir
        for a in MOVES + ["run"]:
            var on: bool = a == dir or (a == "run" and run and dir != "")
            if on != bool(_held.get(a, false)):
                _held[a] = on
                if on:
                    Input.action_press(a)
                else:
                    Input.action_release(a)

    func _draw() -> void:
        var active := touch >= 0
        draw_circle(base, radius, Color(0.12, 0.08, 0.05, 0.45 if active else 0.3))
        draw_arc(base, radius, 0, TAU, 48, Color(1, 0.95, 0.8, 0.8 if active else 0.45), 3.0, true)
        for i in 4:  # little arrows on the rim
            var d := Vector2.UP.rotated(i * PI / 2.0)
            var tip := base + d * (radius - 10)
            var side := d.orthogonal() * 9
            draw_colored_polygon(PackedVector2Array([tip, tip - d * 14 + side, tip - d * 14 - side]), Color(1, 0.95, 0.
8, 0.55))
        var k := base + knob
        draw_circle(k, radius * 0.45, Color("e8b04a") if active else Color(0.9, 0.85, 0.7, 0.7))
        draw_arc(k, radius * 0.45, 0, TAU, 32, Color(0.17, 0.11, 0.06, 0.8), 3.0, true)
```

`ui.gd` · scripts/ui/ui.gd · 400 lines

```
extends RefCounted
## Shared UI kit: colours, theme and small widget helpers.
## Look follows the GDS v0.2 mockups: navy panels, wooden borders,
## chunky pixel headings and green primary buttons.

const NAVY := Color("1e2433")
const NAVY2 := Color("2b3450")
const NAVY_T := Color(0.117, 0.141, 0.2, 0.94)
const WOOD := Color("8a5a34")
const WOOD_D := Color("5a381e")
const WOOD_L := Color("b07a48")
const CREAM := Color("f6efdc")
const CREAM_D := Color("e6dcc0")
const INK := Color("2b2420")
const GOLD := Color("f0c24a")
const GREEN := Color("4f9a45")
const GREEN_D := Color("2f6a2c")
const RED := Color("c94a4a")
const BLUE := Color("3f7fc4")
const GREY := Color("8a8a96")

const FONTS := {
    "lexend": ["res://assets/fonts/lexend-latin-400-normal.woff2", "res://assets/fonts/lexend-latin-700-normal.woff2"],
    "atkinson": ["res://assets/fonts/atkinson-hyperlegible-latin-400-normal.woff2", "res://assets/fonts/atkinson-hyperl
egible-latin-700-normal.woff2"],
    "opendyslexic": ["res://assets/fonts/opendyslexic-latin-400-normal.woff2", "res://assets/fonts/opendyslexic-latin-7
00-normal.woff2"],
```

```
}
const FONT_NAMES := {"lexend": "Lexend", "atkinson": "Atkinson Hyperlegible", "opendyslexic": "OpenDyslexic"}
const PIXEL_FONT := "res://assets/fonts/pixelify-sans-latin-700-normal.woff2"
const SIZES := [18, 22, 26]


static func sb(bg: Color, border: Color = Color(0, 0, 0, 0), bw: int = 0, radius: int = 6, margin: int = 12) -> StyleBoxFlat:
    var s := StyleBoxFlat.new()
    s.bg_color = bg
    s.border_color = border
    s.set_border_width_all(bw)
    s.set_corner_radius_all(radius)
    s.set_content_margin_all(margin)
    s.anti_aliasing = false
    return s


static func _btn_styles(theme: Theme, type: String, base: Color, dark: Color, light: Color) -> void:
    var normal := sb(base, dark, 3, 6, 10)
    normal.content_margin_left = 18
    normal.content_margin_right = 18
    normal.shadow_color = Color(0, 0, 0, 0.35)
    normal.shadow_size = 0
    normal.shadow_offset = Vector2(0, 3)
    normal.border_width_bottom = 6
    var hover := normal.duplicate() as StyleBoxFlat
    hover.bg_color = light
    var pressed := normal.duplicate() as StyleBoxFlat
    pressed.bg_color = dark
    pressed.border_width_bottom = 3
    pressed.border_width_top = 6
    var disabled := normal.duplicate() as StyleBoxFlat
    disabled.bg_color = Color(0.45, 0.45, 0.5)
    disabled.border_color = Color(0.3, 0.3, 0.34)
    var focus := sb(Color(0, 0, 0, 0), GOLD, 3, 8, 0)
    theme.set_stylebox("normal", type, normal)
    theme.set_stylebox("hover", type, hover)
    theme.set_stylebox("pressed", type, pressed)
    theme.set_stylebox("disabled", type, disabled)
    theme.set_stylebox("focus", type, focus)
    theme.set_stylebox("hover_pressed", type, pressed)
    theme.set_color("font_color", type, CREAM)
    theme.set_color("font_hover_color", type, Color.WHITE)
    theme.set_color("font_pressed_color", type, CREAM)
    theme.set_color("font_focus_color", type, CREAM)
    theme.set_color("font_disabled_color", type, Color(0.82, 0.82, 0.85))
    theme.set_color("icon_normal_color", type, CREAM)


static func build_theme(settings: Dictionary) -> Theme:
    var t := Theme.new()
    # Panels
    var dark := sb(NAVY_T, WOOD, 4, 8, 18)
    dark.shadow_color = Color(0, 0, 0, 0.4)
    dark.shadow_size = 6
    t.set_stylebox("panel", "PanelContainer", dark)
    t.set_stylebox("panel", "Panel", dark)
    t.set_type_variation("Paper", "PanelContainer")
    var paper := sb(CREAM, WOOD, 4, 8, 18)
    paper.shadow_color = Color(0, 0, 0, 0.35)
    paper.shadow_size = 6
    t.set_stylebox("panel", "Paper", paper)
    t.set_type_variation("Bar", "PanelContainer")
    var bar := sb(NAVY, WOOD_D, 0, 0, 10)
    bar.border_width_bottom = 4
    bar.content_margin_left = 24
    bar.content_margin_right = 24
    t.set_stylebox("panel", "Bar", bar)
    t.set_type_variation("Card", "PanelContainer")
    t.set_stylebox("panel", "Card", sb(CREAM, WOOD_D, 3, 6, 8))
    t.set_type_variation("CardGood", "PanelContainer")
    t.set_stylebox("panel", "CardGood", sb(Color("dff0d4"), GREEN_D, 3, 6, 8))
    t.set_type_variation("CardSel", "PanelContainer")
    t.set_stylebox("panel", "CardSel", sb(Color("fff3c4"), GOLD, 4, 6, 8))
    t.set_type_variation("CardLocked", "PanelContainer")
    t.set_stylebox("panel", "CardLocked", sb(Color(0.25, 0.25, 0.3, 0.9), Color(0.15, 0.15, 0.18), 3, 6, 8))
    t.set_type_variation("Slot", "PanelContainer")
    t.set_stylebox("panel", "Slot", sb(Color(0.17, 0.2, 0.3, 0.92), Color("7d6a4c"), 3, 8, 10))
    t.set_type_variation("SlotHot", "PanelContainer")
    t.set_stylebox("panel", "SlotHot", sb(Color(0.22, 0.27, 0.38, 0.95), GOLD, 4, 8, 10))
    t.set_type_variation("SlotBad", "PanelContainer")
    t.set_stylebox("panel", "SlotBad", sb(Color(0.35, 0.18, 0.2, 0.95), RED, 4, 8, 10))
    t.set_type_variation("SlotGood", "PanelContainer")
    t.set_stylebox("panel", "SlotGood", sb(Color(0.16, 0.3, 0.2, 0.95), GREEN, 4, 8, 10))
```

```
    t.set_type_variation("Glass", "PanelContainer")
    t.set_stylebox("panel", "Glass", sb(Color(0.96, 0.92, 0.86, 0.9), WOOD, 4, 10, 22))
    t.set_type_variation("Bubble", "PanelContainer")
    t.set_stylebox("panel", "Bubble", sb(Color(1, 1, 1, 0.97), INK, 3, 12, 12))
    # Buttons
    _btn_styles(t, "Button", WOOD, WOOD_D, WOOD_L)
    t.set_type_variation("Primary", "Button")
    _btn_styles(t, "Primary", GREEN, GREEN_D, Color("63b358"))
    t.set_type_variation("Blue", "Button")
    _btn_styles(t, "Blue", BLUE, Color("2a5a90"), Color("5596da"))
    t.set_type_variation("Flat", "Button")
    t.set_stylebox("normal", "Flat", sb(Color(0, 0, 0, 0), Color(0, 0, 0, 0), 0, 6, 6))
    t.set_stylebox("hover", "Flat", sb(Color(1, 1, 1, 0.1), Color(0, 0, 0, 0), 0, 6, 6))
    t.set_stylebox("pressed", "Flat", sb(Color(1, 1, 1, 0.2), Color(0, 0, 0, 0), 0, 6, 6))
    t.set_stylebox("focus", "Flat", StyleBoxEmpty.new())
    t.set_color("font_color", "Flat", Color("9fd0ff"))
    t.set_color("font_hover_color", "Flat", Color.WHITE)
    # Inputs
    var le := sb(CREAM, WOOD_D, 3, 6, 10)
    t.set_stylebox("normal", "LineEdit", le)
    var lef := sb(Color.WHITE, GOLD, 3, 6, 10)
    t.set_stylebox("focus", "LineEdit", lef)
    var led := sb(CREAM_D, GREY, 3, 6, 10)
    t.set_stylebox("read_only", "LineEdit", led)
    t.set_color("font_color", "LineEdit", INK)
    t.set_color("font_placeholder_color", "LineEdit", Color(0.4, 0.36, 0.32))
    t.set_color("caret_color", "LineEdit", INK)
    t.set_type_variation("Invalid", "LineEdit")
    t.set_stylebox("normal", "Invalid", sb(Color("fbe3e0"), RED, 3, 6, 10))
    t.set_stylebox("focus", "Invalid", sb(Color("fbe3e0"), RED, 4, 6, 10))
    # Progress
    t.set_stylebox("background", "ProgressBar", sb(Color(0.08, 0.1, 0.16), Color(0.05, 0.05, 0.08), 2, 4, 0))
    t.set_stylebox("fill", "ProgressBar", sb(Color("4aa3e8"), Color(0, 0, 0, 0), 0, 4, 0))
    t.set_type_variation("Good", "ProgressBar")
    t.set_stylebox("fill", "Good", sb(Color("62b85a"), Color(0, 0, 0, 0), 0, 4, 0))
    t.set_type_variation("Warn", "ProgressBar")
    t.set_stylebox("fill", "Warn", sb(Color("e0a040"), Color(0, 0, 0, 0), 0, 4, 0))
    # Labels
    t.set_color("font_color", "Label", CREAM)
    t.set_type_variation("Title", "Label")
    t.set_color("font_color", "Title", GOLD)
    t.set_color("font_outline_color", "Title", Color(0.08, 0.06, 0.1))
    t.set_constant("outline_size", "Title", 10)
    t.set_type_variation("Logo", "Label")
    t.set_color("font_color", "Logo", GOLD)
    t.set_color("font_outline_color", "Logo", Color(0.08, 0.06, 0.1))
    t.set_constant("outline_size", "Logo", 10)
    t.set_type_variation("Heading", "Label")
    t.set_color("font_color", "Heading", GOLD)
    t.set_type_variation("Ink", "Label")
    t.set_color("font_color", "Ink", INK)
    t.set_type_variation("InkSmall", "Label")
    t.set_color("font_color", "InkSmall", Color(0.3, 0.26, 0.22))
    t.set_type_variation("InkHeading", "Label")
    t.set_color("font_color", "InkHeading", WOOD_D)
    t.set_type_variation("Small", "Label")
    t.set_color("font_color", "Small", Color(0.85, 0.85, 0.88))
    t.set_type_variation("Good", "Label")
    t.set_color("font_color", "Good", Color("8fe08a"))
    t.set_type_variation("Bad", "Label")
    t.set_color("font_color", "Bad", Color("ff9a8a"))
    for typ in ["CheckBox", "CheckButton"]:
        t.set_color("font_color", typ, CREAM)
        t.set_color("font_hover_color", typ, Color.WHITE)
        t.set_color("font_pressed_color", typ, GOLD)
        t.set_stylebox("focus", typ, StyleBoxEmpty.new())
    t.set_color("font_color", "LinkButton", Color("9fd0ff"))
    t.set_color("font_hover_color", "LinkButton", Color.WHITE)
    t.set_stylebox("panel", "PopupMenu", sb(NAVY, WOOD, 3, 6, 8))
    t.set_color("font_color", "PopupMenu", CREAM)
    t.set_stylebox("panel", "TooltipPanel", sb(NAVY, GOLD, 2, 4, 8))
    apply_settings(t, settings)
    return t


static func apply_settings(t: Theme, settings: Dictionary) -> void:
    var key: String = settings.get("font", "lexend")
    if not FONTS.has(key):
        key = "lexend"
    var reg: Font = load(FONTS[key][0])
    var bold: Font = load(FONTS[key][1])
    var pixel: Font = load(PIXEL_FONT)
    var size_idx: int = clampi(int(settings.get("text_size", 1)), 0, 2)
    var base: int = SIZES[size_idx]
    t.default_font = reg
```

```
    t.default_font_size = base
    for typ in ["Button", "Primary", "Blue", "Flat", "CheckBox", "CheckButton", "OptionButton", "LinkButton"]:
        t.set_font("font", typ, bold)
        t.set_font_size("font_size", typ, base)
    # headings and big numbers use the chosen reading font: the pixel font's
    # digits (5 vs S) were hard to read; it stays only on the game logo
    t.set_font("font", "Logo", pixel)
    t.set_font_size("font_size", "Logo", int(base * 2.4))
    t.set_font("font", "Title", bold)
    t.set_font_size("font_size", "Title", int(base * 2.2))
    t.set_font("font", "Heading", bold)
    t.set_font_size("font_size", "Heading", int(base * 1.25))
    t.set_font("font", "InkHeading", bold)
    t.set_font_size("font_size", "InkHeading", int(base * 1.25))
    t.set_font_size("font_size", "Small", int(base * 0.8))
    t.set_font_size("font_size", "InkSmall", int(base * 0.8))
    t.set_font("font", "Good", bold)
    t.set_font("font", "Bad", bold)


# ------------------------------------------------------------------ widgets
static func label(text: String, variation: String = "", wrap: bool = false) -> Label:
    var l := Label.new()
    l.text = text
    if variation != "":
        l.theme_type_variation = variation
    if wrap:
        l.autowrap_mode = TextServer.AUTOWRAP_WORD_SMART
    return l


static func button(text: String, variation: String = "", min_w: int = 0) -> Button:
    var b := Button.new()
    b.text = text
    if variation != "":
        b.theme_type_variation = variation
    if min_w > 0:
        b.custom_minimum_size.x = min_w
    b.focus_mode = Control.FOCUS_ALL
    b.mouse_default_cursor_shape = Control.CURSOR_POINTING_HAND
    return b


static func tex(path: String) -> Texture2D:
    if path == "" or not ResourceLoader.exists(path):
        return null
    return load(path) as Texture2D


static var _heads := {}

## Profile picture: a square close-up of the head and shoulders of a
## character's front-facing sprite (assets/characters32/<folder>), so the
## picture always matches the outfit being worn.
static func head(folder: String) -> Texture2D:
    if _heads.has(folder):
        return _heads[folder]
    var src := tex("res://assets/characters32/%s/%s_south.png" % [folder, folder])
    if src == null:
        return null
    var used := src.get_image().get_used_rect()
    var side := maxf(used.size.x, used.size.y * 0.5)
    var a := AtlasTexture.new()
    a.atlas = src
    a.region = Rect2(used.position.x + used.size.x / 2.0 - side / 2.0, used.position.y - 1, side, side)
    _heads[folder] = a
    return a


static func head_icon(folder: String, size: float) -> TextureRect:
    var r := icon("", size)
    r.texture = head(folder)
    return r


static func icon(path: String, size: float) -> TextureRect:
    var r := TextureRect.new()
    r.texture = tex(path)
    r.custom_minimum_size = Vector2(size, size)
    r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    r.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
    r.mouse_filter = Control.MOUSE_FILTER_IGNORE
    return r

```

```
280
281 static func panel(variation: String = "") -> PanelContainer:
282     var p := PanelContainer.new()
283     if variation != "":
284         p.theme_type_variation = variation
285     return p
286
287
288 static func vbox(sep: int = 10) -> VBoxContainer:
289     var b := VBoxContainer.new()
290     b.add_theme_constant_override("separation", sep)
291     return b
292
293
294 static func hbox(sep: int = 10) -> HBoxContainer:
295     var b := HBoxContainer.new()
296     b.add_theme_constant_override("separation", sep)
297     return b
298
299
300 static func spacer(h: bool = true) -> Control:
301     var c := Control.new()
302     if h:
303         c.size_flags_horizontal = Control.SIZE_EXPAND_FILL
304     else:
305         c.size_flags_vertical = Control.SIZE_EXPAND_FILL
306     c.mouse_filter = Control.MOUSE_FILTER_IGNORE
307     return c
308
309
310 static func margin(child: Control, l: int, t: int, r: int, b: int) -> MarginContainer:
311     var m := MarginContainer.new()
312     m.add_theme_constant_override("margin_left", l)
313     m.add_theme_constant_override("margin_top", t)
314     m.add_theme_constant_override("margin_right", r)
315     m.add_theme_constant_override("margin_bottom", b)
316     m.add_child(child)
317     return m
318
319
320 static func full(c: Control) -> Control:
321     c.set_anchors_preset(Control.PRESET_FULL_RECT)
322     c.offset_left = 0
323     c.offset_top = 0
324     c.offset_right = 0
325     c.offset_bottom = 0
326     return c
327
328
329 static func center(c: Control) -> CenterContainer:
330     var cc := CenterContainer.new()
331     full(cc)
332     cc.add_child(c)
333     return cc
334
335
336 static func dim(alpha: float = 0.55) -> ColorRect:
337     var r := ColorRect.new()
338     r.color = Color(0.03, 0.04, 0.08, alpha)
339     full(r)
340     return r
341
342
343 static func bg_image(path: String, modulate_col: Color = Color.WHITE) -> TextureRect:
344     var r := TextureRect.new()
345     r.texture = tex(path)
346     r.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
347     r.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_COVERED
348     r.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST
349     r.modulate = modulate_col
350     r.mouse_filter = Control.MOUSE_FILTER_IGNORE
351     full(r)
352     return r
353
354
355 ## Top bar used on menu screens (mockup: "EXPLO-STORY | TITLE").
356 static func header_bar(title: String, subtitle: String = "") -> VBoxContainer:
357     var col := VBoxContainer.new()
358     col.add_theme_constant_override("separation", 0)
359     var bar := panel("Bar")
360     var row := hbox(24)
361     var brand := label("EXPLO-STORY", "Heading")
362     brand.add_theme_color_override("font_color", Color("f0a060"))
363     row.add_child(brand)
364     var t := label(title, "Heading")
```

```
    t.add_theme_color_override("font_color", CREAM)
    t.size_flags_horizontal = Control.SIZE_EXPAND_FILL
    t.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
    row.add_child(t)
    var pad := Control.new()
    pad.custom_minimum_size.x = brand.get_minimum_size().x + 60
    row.add_child(pad)
    bar.add_child(row)
    col.add_child(bar)
    if subtitle != "":
        var sub := panel("Bar")
        var sl := label(subtitle, "Small")
        sl.add_theme_color_override("font_color", GOLD)
        sub.add_child(sl)
        col.add_child(sub)
    return col


## Sprite-sheet frame (characters are 3x4 sheets of 16x24).
static func sheet_frame(sheet_path: String, col: int = 0, row: int = 0, w: int = 16, h: int = 24) -> Texture2D:
    var base := tex(sheet_path)
    if base == null:
        return null
    var a := AtlasTexture.new()
    a.atlas = base
    a.region = Rect2(col * w, row * h, w, h)
    return a


static func icon_button(icon_path: String, variation: String = "", size: Vector2 = Vector2(72, 64), text: String = "")
-> Button:
    var b := button(text, variation)
    b.icon = tex(icon_path)
    b.expand_icon = true
    b.icon_alignment = HORIZONTAL_ALIGNMENT_CENTER if text == "" else HORIZONTAL_ALIGNMENT_LEFT
    b.custom_minimum_size = size
    return b
```

scripts/world/

ambience.gd · scripts/world/ambience.gd · 259 lines

```
extends Node2D
## Makes the overworld feel alive: butterflies, drifting leaves, cloud
## shadows, passing birds, mist on locked gates and gold dust on open ones,
## kitchen smoke over the kampung houses, rings and jumping fish on the river.
## Everything is drawn or generated in code (no extra downloads).

const TILE := 16

var world: Node2D
var _butterflies: Array[Dictionary] = []
var _birds: Array[Dictionary] = []
var _t := 0.0
var _bird_timer := 4.0
var _river: Array[Vector2i] = []      ## river cells (not under bridges)
var _ripples: Array[Dictionary] = []  ## {pos, t}
var _fish: Array[Dictionary] = []     ## {from, t}
var _ripple_timer := 0.5

@onready var clouds: Node2D = $Clouds


func setup(w: Node2D, gates: Array) -> void:
    world = w
    var size := Vector2(w.terrain.width, w.terrain.height) * TILE
    # cloud shadows: big soft blobs drifting west -> east
    var tex := _soft_blob(96)
    for i in 7:
        var s := Sprite2D.new()
        s.texture = tex
        s.scale = Vector2(randf_range(2.5, 4.5), randf_range(1.6, 2.6))
        s.modulate = Color(0.05, 0.1, 0.2, 0.13)
        s.position = Vector2(randf() * size.x, randf() * size.y)
        clouds.add_child(s)
    # butterflies around the island
    for i in 40:
        _butterflies.append({
            "pos": Vector2(randf() * size.x, randf() * size.y),
            "vel": Vector2.from_angle(randf() * TAU) * 10.0,
            "col": [Color("ffd84a"), Color("ff8fb1"), Color("9fd8ff"), Color.WHITE][i % 4],
```

```
            "phase": randf() * TAU,
        })
    for g in gates:
        var at := Vector2((g["pos"][0] + 2) * TILE, g["pos"][1] * TILE + 8)
        add_child(_mist(at) if not g["open"] else _gold_dust(at))
    add_child(_leaves())
    for h in w.houses:  # a thin wisp of smoke from each kampung kitchen
        var top: float = h.texture.get_image().get_used_rect().size.y
        add_child(_smoke(h.position + Vector2(22, -top + 18)))
    for y in w.terrain.height:
        for x in w.terrain.width:
            var c := Vector2i(x, y)
            if w.terrain.terrain_at(c) == TerrainBuilder.RIVER and not w.walk_over.has(c):
                _river.append(c)


func _process(delta: float) -> void:
    _t += delta
    var size := Vector2(world.terrain.width, world.terrain.height) * TILE
    for c in clouds.get_children():
        c.position.x += delta * 6.0
        if c.position.x > size.x + 200:
            c.position.x = -200
            c.position.y = randf() * size.y
    for b in _butterflies:
        b["vel"] = (b["vel"] as Vector2).rotated(randf_range(-2.5, 2.5) * delta)
        b["pos"] += b["vel"] * delta
        b["pos"] = (b["pos"] as Vector2).clamp(Vector2.ZERO, size)
    _bird_timer -= delta
    if _bird_timer <= 0:
        _bird_timer = randf_range(9.0, 16.0)
        _spawn_flock()
    for b in _birds:
        b["pos"] += b["vel"] * delta
    _birds = _birds.filter(func(b): return b["pos"].x < size.x + 60 and b["pos"].x > -60)
    _water(delta)
    queue_redraw()


## Rings spread where a fish nibbles; now and then one jumps out in an arc.
func _water(delta: float) -> void:
    if _river.is_empty():
        return
    _ripple_timer -= delta
    if _ripple_timer <= 0:
        _ripple_timer = randf_range(0.25, 0.7)
        var c: Vector2i = _river.pick_random()
        var at := Vector2(c) * TILE + Vector2(randf_range(3, 13), randf_range(3, 13))
        if randf() < 0.12:
            _fish.append({"from": at, "t": 0.0, "dir": 1.0 if randf() < 0.5 else -1.0})
        else:
            _ripples.append({"pos": at, "t": 0.0})
    for r in _ripples:
        r["t"] += delta
    _ripples = _ripples.filter(func(r): return r["t"] < 1.6)
    for f in _fish:
        f["t"] += delta
        if f["t"] >= 0.7 and not f.has("splash"):
            f["splash"] = true
            _ripples.append({"pos": f["from"] + Vector2(10 * f["dir"], 0), "t": 0.0})
    _fish = _fish.filter(func(f): return f["t"] < 0.7)


func _draw() -> void:
    # ripples: squashed rings that grow and fade
    for r in _ripples:
        var k: float = r["t"] / 1.6
        draw_set_transform(r["pos"], 0.0, Vector2(1, 0.45))
        draw_arc(Vector2.ZERO, 2.0 + k * 7.0, 0, TAU, 14, Color(1, 1, 1, 0.55 * (1.0 - k)), 1.0)
    draw_set_transform(Vector2.ZERO)
    # jumping fish: a little orange arc
    for f in _fish:
        var k: float = f["t"] / 0.7
        var p: Vector2 = f["from"] + Vector2(10 * f["dir"] * k, -sin(k * PI) * 9)
        draw_rect(Rect2(p.floor() - Vector2(2, 1), Vector2(4, 2)), Color("f08a3c"))
        draw_rect(Rect2(p.floor() + Vector2(-3 if f["dir"] > 0 else 2, -1), Vector2(1, 2)), Color("c8642a"))
    # butterflies: two tiny wings that flap
    for b in _butterflies:
        var p: Vector2 = (b["pos"] as Vector2).floor()
        var flap := sin(_t * 14.0 + b["phase"]) > 0
        var col: Color = b["col"]
        var w := 2 if flap else 1
        draw_rect(Rect2(p + Vector2(-w, 0), Vector2(w, 2)), col)
        draw_rect(Rect2(p + Vector2(1, 0), Vector2(w, 2)), col)
        draw_rect(Rect2(p + Vector2(0, 0), Vector2(1, 2)), Color(0.15, 0.1, 0.05))
```

```
    # birds: little "v" shapes with their shadow far below
    for b in _birds:
        var p: Vector2 = (b["pos"] as Vector2).floor()
        var up := sin(_t * 9.0 + b["phase"]) > 0
        var dy := -1 if up else 1
        var shadow := Color(0, 0, 0, 0.18)
        draw_line(p + Vector2(-3, dy) + Vector2(0, 40), p + Vector2(0, 40), shadow)
        draw_line(p + Vector2(0, 40), p + Vector2(3, dy) + Vector2(0, 40), shadow)
        draw_line(p + Vector2(-3, dy), p, Color(0.15, 0.12, 0.1))
        draw_line(p, p + Vector2(3, dy), Color(0.15, 0.12, 0.1))


func _spawn_flock() -> void:
    var size := Vector2(world.terrain.width, world.terrain.height) * TILE
    var y := randf_range(80, size.y - 80)
    for i in randi_range(3, 6):
        _birds.append({
            "pos": Vector2(-20 - i * 9, y + (i % 2) * 7 - i * 2),
            "vel": Vector2(randf_range(38, 46), randf_range(-4, 4)),
            "phase": randf() * TAU,
        })


func _mist(at: Vector2) -> CPUParticles2D:
    var p := CPUParticles2D.new()
    p.position = at
    p.texture = _soft_blob(24)
    p.amount = 28
    p.lifetime = 5.0
    p.preprocess = 5.0
    p.emission_shape = CPUParticles2D.EMISSION_SHAPE_RECTANGLE
    p.emission_rect_extents = Vector2(40, 18)
    p.gravity = Vector2(4, -3)
    p.initial_velocity_min = 2.0
    p.initial_velocity_max = 8.0
    p.spread = 180.0
    p.scale_amount_min = 1.2
    p.scale_amount_max = 2.6
    var g := Gradient.new()
    g.set_color(0, Color(0.75, 0.65, 0.95, 0.0))
    g.set_color(1, Color(0.75, 0.65, 0.95, 0.0))
    g.add_point(0.4, Color(0.78, 0.7, 0.98, 0.55))
    p.color_ramp = g
    p.z_index = 2
    return p


func _gold_dust(at: Vector2) -> CPUParticles2D:
    var p := CPUParticles2D.new()
    p.position = at
    var img := Image.create(1, 1, false, Image.FORMAT_RGBA8)
    img.fill(Color.WHITE)
    p.texture = ImageTexture.create_from_image(img)
    p.amount = 22
    p.lifetime = 2.2
    p.preprocess = 2.0
    p.emission_shape = CPUParticles2D.EMISSION_SHAPE_RECTANGLE
    p.emission_rect_extents = Vector2(30, 6)
    p.direction = Vector2(0, -1)
    p.spread = 25.0
    p.gravity = Vector2(0, -6)
    p.initial_velocity_min = 6.0
    p.initial_velocity_max = 16.0
    var g := Gradient.new()
    g.set_color(0, Color(1, 0.95, 0.55, 1))
    g.set_color(1, Color(1, 0.8, 0.3, 0))
    p.color_ramp = g
    p.z_index = 2
    return p


func _smoke(at: Vector2) -> CPUParticles2D:
    var p := CPUParticles2D.new()
    p.position = at
    p.texture = _soft_blob(10)
    p.amount = 7
    p.lifetime = 3.5
    p.preprocess = 3.5
    p.direction = Vector2(0, -1)
    p.spread = 12.0
    p.gravity = Vector2(5, -4)
    p.initial_velocity_min = 5.0
    p.initial_velocity_max = 9.0
    p.scale_amount_min = 0.6
    p.scale_amount_max = 1.4
```

```
    var g := Gradient.new()
    g.set_color(0, Color(0.95, 0.95, 0.95, 0.0))
    g.set_color(1, Color(0.9, 0.9, 0.9, 0.0))
    g.add_point(0.2, Color(0.92, 0.92, 0.92, 0.4))
    p.color_ramp = g
    return p


func _leaves() -> CPUParticles2D:
    # follows the camera so leaves are always drifting on screen
    var p := CPUParticles2D.new()
    var img := Image.create(3, 2, false, Image.FORMAT_RGBA8)
    img.fill(Color("6fae3a"))
    img.set_pixel(0, 1, Color("4c8a2a"))
    p.texture = ImageTexture.create_from_image(img)
    p.amount = 10
    p.lifetime = 7.0
    p.preprocess = 7.0
    p.emission_shape = CPUParticles2D.EMISSION_SHAPE_RECTANGLE
    p.emission_rect_extents = Vector2(200, 10)
    p.gravity = Vector2(6, 12)
    p.initial_velocity_min = 4.0
    p.initial_velocity_max = 10.0
    p.angular_velocity_min = -90.0
    p.angular_velocity_max = 90.0
    p.local_coords = false
    p.z_index = 3
    p.set_meta("follow_camera", true)
    p.name = "Leaves"
    return p


func _physics_process(_d: float) -> void:
    var leaves := get_node_or_null("Leaves")
    if leaves and world:
        leaves.global_position = world.player.global_position + Vector2(0, -110)


func _soft_blob(size: int) -> Texture2D:
    var g := Gradient.new()
    g.set_color(0, Color(1, 1, 1, 1))
    g.set_color(1, Color(1, 1, 1, 0))
    var t := GradientTexture2D.new()
    t.gradient = g
    t.width = size
    t.height = size
    t.fill = GradientTexture2D.FILL_RADIAL
    t.fill_from = Vector2(0.5, 0.5)
    t.fill_to = Vector2(1.0, 0.5)
    return t
```

arena_yard.gd · scripts/world/arena_yard.gd · 152 lines

```
extends RefCounted
## The yard in front of an Arena Ulang Kaji (entities.json "arenas"): other
## players spar in pairs with the Gelanggang fight frames, a "Aina VS Irfan"
## tag over each match, next to wooden training dummies (Jurus Ingatan). Talking to a
## fighter offers the lobby; a dummy wobbles and opens Jurus Ingatan.

const TILE := 16
const DUMMY := "res://assets/objects/patung_latihan.png"
## shop outfits and classmates: every one has Gelanggang fight frames
const LOOKS := ["alif", "alif_sukan", "alif_pengakap", "alif_melayu", "alif_pahlawan", "alif_bomba",
    "rakan_aisyah", "rakan_daniel", "rakan_meiling", "rakan_ravi"]
const NAMES := ["Aina", "Irfan", "WeiJie", "Kavitha", "Danish", "Sofea", "Arjun", "Nurul",
    "Haziq", "MeiLing", "Farah", "Zul", "Priya", "Adam", "Hana", "Iman", "Syafiq", "Ravi"]
const ATTACKS := ["jab", "cross", "kick", "roundhouse"]


## spawn one yard's fighters and dummies into the world (cells blocked,
## interactions registered); returns the dummy sprites by cell
static func build(world: Node, def: Dictionary) -> Dictionary:
    var dummies := {}
    for p in def.get("pairs", []):
        var pair := Pair.new()
        world.objects.add_child(pair)
        var looks: Array = LOOKS.duplicate()
        looks.shuffle()
        for i in 2:
            var c := Vector2i(int(p[i][0]), int(p[i][1]))
            var f := Fighter.new()
            f.setup(looks[i], c, i == 1)
            world.objects.add_child(f)
```

```
            pair.fighters.append(f)
            world.blocked[c] = true
            world.interact[c] = {"type": "talk", "id": "pemain_arena"}
        pair.add_tag()
    for d in def.get("dummies", []):
        var c := Vector2i(int(d[0]), int(d[1]))
        var s := Sprite2D.new()
        s.texture = dummy_texture()
        s.offset = CharacterFrames.visible_bottom_offset(s.texture)
        s.position = Vector2((c.x + 0.5) * TILE, (c.y + 1) * TILE)
        world.objects.add_child(s)
        world.blocked[c] = true
        world.interact[c] = {"type": "talk", "id": "patung_latihan"}
        dummies[c] = s
    return dummies


## a hit on the dummy: it rocks on its base and settles
static func wobble(s: Sprite2D) -> void:
    var tw := s.create_tween()
    for a in [0.28, -0.2, 0.12, -0.06, 0.0]:
        tw.tween_property(s, "rotation", a, 0.09)


## assets/objects/patung_latihan.png when it exists, else a wooden silat
## dummy drawn in pixels (post, arms, padded head, base)
static func dummy_texture() -> Texture2D:
    if ResourceLoader.exists(DUMMY):
        return load(DUMMY)
    const ROWS := [
        "....####....", "...#hhhh#...", "...#hhhh#...", "...#hHhh#...", "....####....",
        ".....#w#....", "############.", "#aaaaaaaaa#.", "############.", ".....#w#....",
        "....#www#...", "....#wpw#...", "....#wpw#...", "....#www#...", ".....#w#....",
        ".....#w#....", ".....#w#....", "...#######..", "..#bbbbbbb#.", "..#########.",
    ]
    var cols := {"#": Color("3a2414"), "h": Color("c9a26b"), "H": Color("e8c890"), "w": Color("9c6a3c"),
        "a": Color("b5824c"), "p": Color("d24a3c"), "b": Color("6e4a2c")}
    var img := Image.create(12, 20, false, Image.FORMAT_RGBA8)
    for y in ROWS.size():
        for x in ROWS[y].length():
            if cols.has(ROWS[y][x]):
                img.set_pixel(x, y, cols[ROWS[y][x]])
    return ImageTexture.create_from_image(img)


## two fighters facing each other: one attacks (a random strike), the other
## takes it or steps back, then a pause; who attacks changes each round
class Pair extends Node2D:
    var fighters: Array = []
    var _wait := randf_range(0.4, 2.0)
    var _target: Fighter

    ## the match's name tag, centred over both fighters
    func add_tag() -> void:
        position = (fighters[0].position + fighters[1].position) / 2.0
        var names: Array = NAMES.duplicate()
        names.shuffle()
        var tag := Label.new()
        tag.text = "%s VS %s" % [names[0], names[1]]
        tag.theme = Game.theme
        tag.add_theme_font_size_override("font_size", 26)
        tag.add_theme_color_override("font_color", Color("fff3c4"))
        tag.add_theme_color_override("font_outline_color", Color("1b1408"))
        tag.add_theme_constant_override("outline_size", 10)
        tag.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        tag.size = Vector2(240, 40)
        tag.scale = Vector2(0.2, 0.2)   # drawn big, shown small: stays sharp under the zoomed camera
        tag.position = Vector2(-24, -40)
        tag.z_index = 6
        add_child(tag)

    func _process(delta: float) -> void:
        if fighters.size() < 2:
            return
        _wait -= delta
        if _wait > 0.0:
            return
        if _target:  # the strike lands
            if randf() < 0.65:
                _target.play("hurt")
            else:
                _target.step_back()
            _target = null
            _wait = randf_range(1.0, 2.4)
            return
```

```
        var i := randi() % 2
        fighters[i].play(ATTACKS.pick_random())
        _target = fighters[1 - i]
        _wait = 0.22


## one player on the yard: fight frames (drawn facing east, flipped when it
## faces west)
class Fighter extends Node2D:
    var _sprite: AnimatedSprite2D
    var _west := false

    func setup(look: String, at: Vector2i, west: bool) -> void:
        _west = west
        position = Player.cell_to_pos(at)
        _sprite = AnimatedSprite2D.new()
        _sprite.sprite_frames = CharacterFrames.build_fight(look)
        _sprite.offset = CharacterFrames.feet_offset(look)
        _sprite.flip_h = west
        _sprite.animation_finished.connect(func(): _sprite.play("stance"))
        add_child(_sprite)
        _sprite.play("stance")
        _sprite.frame = randi() % maxi(1, _sprite.sprite_frames.get_frame_count("stance"))

    func play(anim: String) -> void:
        _sprite.play(anim)

    ## a block: a quick hop back and in again
    func step_back() -> void:
        var tw := create_tween()
        var dx := 3.0 if _west else -3.0
        tw.tween_property(_sprite, "position:x", dx, 0.08).set_ease(Tween.EASE_OUT)
        tw.tween_property(_sprite, "position:x", 0.0, 0.2).set_ease(Tween.EASE_IN_OUT)

    func _draw() -> void:
        draw_set_transform(Vector2(0, -1), 0, Vector2(1, 0.45))
        draw_circle(Vector2.ZERO, 6, Color(0, 0, 0, 0.28))
```

bilik_alif.gd · scripts/world/bilik_alif.gd · 234 lines

```
extends "res://scripts/world/interior.gd"
## Bilik Alif in Dunia Explo-Story (inside Rumah Alif, Kampung Jejak). After
## the prologue Alif wakes up here: Pau explains the artifacts (the four
## Serpihan), the missions and the goal, then teaches walking and talking.
## The first thing to do is pick up the school bag (a blinking start prompt
## points at it; until then Alif wears no bag on his back); the bag holds
## every artifact from then on (Menu > Beg Alif).
## Then Pau shows the map and the gold arrow and Misi 1.1 starts: out the door
## and the arrow on the map points at Gerbang Masa. Later visits: just the room.
## Text: data/dialogue/prologue.json (bilik_*), data/interiors/bilik_alif.json.

const StartPrompt = preload("res://scripts/ui/start_prompt.gd")
const BG := "res://assets/backgrounds/bg_int_bilik_alif.png"
const BED_POS := Vector2(72, 84)   ## Alif asleep: head on the pillow
const UP_CELL := Vector2i(6, 5)    ## where he stands after jumping out of bed
const BAG_CELL := Vector2i(12, 7)
const NO_BAG := "alif_tanpa_beg"   ## school uniform without the backpack

signal bag_taken

var _pieces := {}           ## BilikArt piece id -> node
var _waiting_bag := false   ## tutorial: the door waits until the bag is taken
var _prompt: Control
var _emote: Label
var _blanket: Node2D


func _ready() -> void:
    if ResourceLoader.exists(BG):
        background = load(BG)
    _pieces = BilikArt.build(self, $Objects)
    if not Game.save["seen"].has("beg_alif"):
        player.set_look(NO_BAG)
    _emote = Label.new()
    _emote.add_theme_font_size_override("font_size", 40)
    _emote.add_theme_color_override("font_outline_color", Color(0.17, 0.11, 0.06))
    _emote.add_theme_constant_override("outline_size", 12)
    _emote.visible = false
    $HUD.add_child(_emote)
    super._ready()


func _process(d: float) -> void:
```

```
    super._process(d)
    if _emote.visible:  # the "z z z" / "!" above Alif's head
        _emote.reset_size()
        _emote.position = get_viewport().get_canvas_transform() * (player.global_position + Vector2(0, -36)) - _emote.size / 2.0


func _welcome() -> void:
    if Game.save["seen"].has("beg_alif"):
        _remove_bag(false)
    if not GameState.flags.get("bilik_intro", false):
        return  # a visit later on: just the room
    GameState.flags["bilik_intro"] = false
    await _wake_up()


# --- the wake-up ------------------------------------------------------------
func _wake_up() -> void:
    busy = true
    hud.visible = false
    player.can_move = false
    player.place_at(Vector2i(4, 4), "south")
    player.position = BED_POS
    player.z_index = 1
    _blanket = Blanket.new()  # tucked in up to the chest
    _blanket.room = background
    $Objects.add_child(_blanket)
    pau.visible = false
    _show_emote("z z z")
    await get_tree().create_timer(1.8).timeout

    # Pau bounces onto the floor beside the bed
    pau.place_at(UP_CELL + Vector2i(0, -1))
    pau.visible = true
    pau.hop()
    Sfx.play("pop")
    await get_tree().create_timer(0.6).timeout
    await Game.say(Dialogue.lines("pro_wake_1"))

    # Alif jolts awake and hops out of bed
    _show_emote("!")
    Sfx.play("pop", 0.0, 1.4)
    if _blanket:
        _blanket.queue_free()
    var t := create_tween()
    t.tween_property(player, "position", Player.cell_to_pos(UP_CELL) + Vector2(0, -8), 0.18).set_ease(Tween.EASE_OUT)
    t.tween_property(player, "position", Player.cell_to_pos(UP_CELL), 0.12).set_ease(Tween.EASE_IN)
    await t.finished
    player.z_index = 0
    player.place_at(UP_CELL, "north")
    await get_tree().create_timer(0.6).timeout
    _emote.visible = false

    await Game.say(Dialogue.lines("bilik_bangun"))
    var shards := _show_shards()
    await Game.say(Dialogue.lines("bilik_artifak"))
    _hide(shards)
    await Game.say(Dialogue.lines("bilik_misi"))
    await Game.say(Dialogue.lines("bilik_tujuan"))
    hud.visible = true
    await Game.say(Dialogue.lines("bilik_gerak_sentuh" if Game.controls_on_screen() else "bilik_gerak_papan"))

    # the very first thing to do: pick up the bag (start indicator on it)
    busy = false
    player.can_move = true
    _waiting_bag = true
    Game.context_hint = tr("Ambil beg sekolah kau atas lantai dulu.")
    _prompt = StartPrompt.new()
    _prompt.target = _pieces["beg"]
    _prompt.lift = 16.0
    _prompt.action_text = "untuk ambil beg"
    $HUD.add_child(_prompt)


func _show_emote(text: String) -> void:
    _emote.text = text
    _emote.visible = true


## the four scattered Serpihan, dim (none found yet), floating over Pau
func _show_shards() -> Node2D:
    var row := Node2D.new()
    row.z_index = 20
    add_child(row)
    var zones: Array = Game.chapter_zones(1)
```

```
    for i in zones.size():
        var g: Dictionary = Game.zone_data(zones[i]).get("ganjaran", {})
        var s := Sprite2D.new()
        s.texture = load("res://assets/icons/%s.png" % g.get("icon", "kunci"))
        s.scale = Vector2.ONE * (14.0 / s.texture.get_width())
        s.position = Vector2(104 + (i - (zones.size() - 1) / 2.0) * 22, 40)
        s.modulate = Color(0.75, 0.7, 0.9, 0.0)
        row.add_child(s)
        var tw := s.create_tween()
        tw.tween_interval(0.25 * i)
        tw.tween_property(s, "modulate:a", 0.8, 0.3)
        tw.tween_property(s, "position:y", 36.0, 0.9).set_trans(Tween.TRANS_SINE)
        tw.tween_property(s, "position:y", 40.0, 0.9).set_trans(Tween.TRANS_SINE)
        tw.set_loops()
    return row


func _hide(n: Node2D) -> void:
    var t := create_tween()
    t.tween_property(n, "modulate:a", 0.0, 0.4)
    t.tween_callback(n.queue_free)


# --- the school bag ----------------------------------------------------------
func _show_exhibit(id: String) -> void:
    if id == "beg":
        await _take_bag()
        return
    await super._show_exhibit(id)


func _take_bag() -> void:
    if _prompt:
        _prompt.done()
        _prompt = null
    MusicDirector.stinger("st_item")
    player.face_cell(BAG_CELL)
    _remove_bag(true)
    player.set_look("")  # the bag goes on his back
    if not Game.save["seen"].has("beg_alif"):
        Game.save["seen"].append("beg_alif")
        Game.persist()
    Game.toast(tr("Dapat: Beg Alif!"))
    await Game.say(Dialogue.lines("bilik_beg"))
    if _waiting_bag:
        _waiting_bag = false
        await Game.say(Dialogue.lines("bilik_peta"))
        _start_first_mission()
    bag_taken.emit()


## the bag leaves the floor (its tile becomes walkable)
func _remove_bag(animate: bool) -> void:
    _exhibits.erase("beg")
    _floor[BAG_CELL] = true
    var bag: Node2D = _pieces.get("beg")
    if bag == null or not is_instance_valid(bag):
        return
    _pieces.erase("beg")
    if not animate:
        bag.queue_free()
        return
    var t := create_tween().set_parallel()
    t.tween_property(bag, "position:y", bag.position.y - 10, 0.25)
    t.tween_property(bag, "modulate:a", 0.0, 0.25)
    t.chain().tween_callback(bag.queue_free)


## Misi 1.1 is on: out of the door, the map arrow points at Gerbang Masa
func _start_first_mission() -> void:
    if not Game.save["seen"].has("bilik_tutorial"):
        Game.save["seen"].append("bilik_tutorial")
        Game.persist()
    var zid := Game.next_zone()
    if zid == "":
        return
    MusicDirector.stinger("st_zone")
    Game.toast(tr("MISI %s BERMULA: %s") % [zid.replace("_", "."), str(Game.zone_data(zid).get("title", ""))], Color("8
fe08a"))
    Game.context_hint = tr("Keluar ikut pintu, lepas tu ikut anak panah emas ke misi pertama.")


## the tutorial keeps Alif in until he has his bag
func _leave() -> void:
    if _waiting_bag:
```

```
        Main.pau_say(tr("Eh, beg kau! Ambil dulu, baru kita keluar."), 4.0)
        return
    super._leave()


## the batik blanket over sleeping Alif: cut from the room picture when there
## is one, else drawn to match the code-drawn bed in BilikArt
class Blanket extends Node2D:
    const AREA := Rect2(49, 70, 46, 24)  ## the lower bed in room pixels
    var room: Texture2D

    func _ready() -> void:
        z_index = 2

    func _draw() -> void:
        if room:
            draw_texture_rect_region(room, AREA, AREA)
            return
        draw_rect(Rect2(50, 74, 44, 20), Color("3b7fa6"))
        draw_rect(Rect2(50, 74, 44, 3), Color("5a9cc0"))
        for y in range(80, 93, 6):
            for x in range(53, 92, 6):
                draw_rect(Rect2(x + (y / 6) % 2 * 3, y, 1, 1), Color("f3d36a"))
```

bilik_art.gd · scripts/world/bilik_art.gd · 367 lines

```
class_name BilikArt
extends RefCounted
## Bilik Alif, drawn: a Malaysian teenager's bedroom on the 20x11 interior
## grid (320x180, 16 px cells), morning light through a louvre window.
## The PixelLab picture assets/backgrounds/bg_int_bilik_alif.png is the whole
## furnished room (made over this layout, so the collision cells still fit);
## then only the bag stays a separate sprite, since Alif picks it up. Without
## that picture every piece is drawn here in native pixels (crisp at the 4x
## camera), and assets/objects/bilik/<id>.png can replace one piece of it
## (bottom-centre on the piece's footprint, see PIECES).
## Layout (cells): bed 3-5 x 3-5, side table 6,3, bookshelf 12-13,3, fan 14,3,
## wardrobe 15-16 x 3-4, desk 15-16 x 6-7, chair 14,7, bag 12,7, shoes 12,9,
## sejadah 3-4 x 7-8, rug 8-11 x 5-7, door 9-10,10.

const SHELL_ART := "res://assets/backgrounds/bg_int_bilik_alif.png"
const PIECE_ART := "res://assets/objects/bilik/%s.png"
## pieces that stay separate when the room picture already has the furniture
const LOOSE := ["beg"]
## id -> footprint Rect2i in cells (bottom row = where it stands for y-sorting)
const PIECES := {
    "katil": Rect2i(3, 3, 3, 3),
    "meja_sisi": Rect2i(6, 3, 1, 1),
    "rak_buku": Rect2i(12, 3, 2, 1),
    "kipas": Rect2i(14, 3, 1, 1),
    "almari": Rect2i(15, 3, 2, 2),
    "meja_belajar": Rect2i(15, 6, 2, 2),
    "kerusi": Rect2i(14, 7, 1, 1),
    "beg": Rect2i(12, 7, 1, 1),
    "kasut": Rect2i(12, 9, 1, 1),
}

const OUT := Color("1c2130")
const WALL := Color("a9d6c9")
const WALL_D := Color("8fc0b2")
const WALL_TOP := Color("efe6d2")
const WALL_TOP_D := Color("c9bea4")
const TRIM := Color("f4efe2")
const WOOD := Color("8a5a36")
const WOOD_L := Color("a8774a")
const WOOD_D := Color("5a3a20")
const TILE_A := Color("e8d8b8")
const TILE_B := Color("e2d0ad")
const GROUT := Color("c9b48f")
const INK := Color("2a2230")


## Adds the room to an interior scene: shell behind everything, furniture into
## the y-sorted Objects node, sunlight and dust in front. Returns id -> node.
static func build(room: Node2D, objects: Node2D) -> Dictionary:
    var painted := ResourceLoader.exists(SHELL_ART)
    var shell := Shell.new()
    shell.drawn = not painted
    room.add_child(shell)
    room.move_child(shell, 1)  # right after the Background sprite
    var out := {}
    for id in PIECES:
        if painted and not id in LOOSE:
```

```
            continue
        var r: Rect2i = PIECES[id]
        var bottom := Vector2((r.position.x + r.size.x / 2.0) * 16.0, r.end.y * 16.0)
        var n: Node2D
        if ResourceLoader.exists(PIECE_ART % id):
            var s := Sprite2D.new()
            s.texture = load(PIECE_ART % id)
            s.offset = CharacterFrames.visible_bottom_offset(s.texture)
            n = s
        else:
            var p := Piece.new()
            p.id = id
            n = p
        n.position = bottom
        n.name = "Bilik_" + id
        objects.add_child(n)
        out[id] = n
    room.add_child(Light.new())
    return out


# ---------------------------------------------------------------- room shell
class Shell extends Node2D:
    var drawn := true

    func _r(x: float, y: float, w: float, h: float, c: Color) -> void:
        draw_rect(Rect2(x, y, w, h), c)

    func _draw() -> void:
        if not drawn:
            return
        _r(0, 0, 320, 180, OUT)
        # back wall (rows 0-2) with ceiling edge and skirting
        _r(48, 0, 224, 48, WALL)
        _r(48, 0, 224, 3, WALL_D)
        for x in range(52, 272, 12):  # soft painted texture
            _r(x, 6 + (x / 12) % 3 * 11, 1, 1, WALL_D)
        _r(48, 43, 224, 5, WOOD)
        _r(48, 43, 224, 1, WOOD_L)
        # tiled floor (rows 3-9) and the door gap
        for y in range(3, 10):
            for x in range(3, 17):
                _r(x * 16, y * 16, 16, 16, TILE_A if (x + y) % 2 == 0 else TILE_B)
                _r(x * 16, y * 16, 16, 1, GROUT)
                _r(x * 16, y * 16, 1, 16, GROUT)
        _r(144, 160, 32, 16, TILE_A)
        _r(48, 48, 224, 3, Color(0, 0, 0, 0.12))  # shade under the wall
        # side and front walls, seen from above
        _r(40, 0, 8, 168, WALL_TOP)
        _r(272, 0, 8, 168, WALL_TOP)
        _r(40, 160, 104, 10, WALL_TOP)
        _r(176, 160, 104, 10, WALL_TOP)
        _r(40, 168, 104, 3, WALL_TOP_D)
        _r(176, 168, 104, 3, WALL_TOP_D)
        _r(47, 0, 1, 168, WALL_TOP_D)
        _r(272, 0, 1, 168, WALL_TOP_D)
        _r(140, 160, 4, 18, WOOD)   # door frame
        _r(176, 160, 4, 18, WOOD)
        _window()
        _wall_decor()
        _rug()
        _sejadah()
        # doormat (kain pengesat kaki)
        _r(148, 163, 24, 11, Color("8a6a4a"))
        for x in range(150, 171, 4):
            _r(x, 164, 2, 9, Color("a8865e"))

    ## glass louvres behind a white grille, floral curtains, a coconut tree outside
    func _window() -> void:
        for y in range(8, 40):  # morning sky
            _r(134, y, 52, 1, Color("8fd0f0").lerp(Color("fde7b5"), (y - 8) / 32.0))
        draw_circle(Vector2(174, 16), 5, Color("fff3c4"))
        draw_colored_polygon(PackedVector2Array([Vector2(134, 40), Vector2(134, 31), Vector2(150, 27), Vector2(168, 32)
, Vector2(186, 28), Vector2(186, 40)]), Color("6fae78"))
        _r(146, 18, 2, 22, Color("6b4a2e"))  # kelapa trunk
        for f in [Vector2(-9, 3), Vector2(-6, -3), Vector2(0, -5), Vector2(7, -2), Vector2(9, 4)]:
            draw_line(Vector2(147, 18), Vector2(147, 18) + f, Color("2f6b3a"), 2.0)
        for y in range(10, 40, 4):  # louvre slats
            _r(134, y, 52, 1, Color(1, 1, 1, 0.35))
        _r(132, 6, 56, 2, TRIM)
        _r(132, 40, 56, 3, TRIM)
        _r(132, 6, 2, 36, TRIM)
        _r(186, 6, 2, 36, TRIM)
        for x in range(140, 186, 9):  # jeriji (grille)
            _r(x, 8, 1, 32, Color("e9e4d8"))
```

```
        _r(134, 23, 52, 1, Color("e9e4d8"))
        # curtain rod and langsir
        _r(120, 3, 80, 2, WOOD_D)
        for sx in [122, 186]:
            _r(sx, 5, 14, 39, Color("e88aa0"))
            for y in range(8, 42, 6):
                for x in range(sx + 2, sx + 13, 5):
                    _r(x + (y / 6) % 2 * 2, y, 2, 2, Color("f6c6d1"))
            _r(sx, 5, 1, 39, Color("c96a82"))
            _r(sx, 30, 14, 2, Color("c9a24a"))  # tie-back

    func _wall_decor() -> void:
        # khat (calligraphy frame) over the bed
        _r(58, 9, 28, 18, Color("c9a24a"))
        _r(60, 11, 24, 14, Color("2f4f3a"))
        draw_arc(Vector2(66, 19), 3, PI, TAU, 6, Color("e6c96a"), 1.0)
        draw_line(Vector2(69, 19), Vector2(78, 15), Color("e6c96a"), 1.0)
        draw_arc(Vector2(76, 19), 3, 0, PI, 6, Color("e6c96a"), 1.0)
        # wall clock over the side table
        draw_circle(Vector2(104, 15), 6, INK)
        draw_circle(Vector2(104, 15), 5, Color("fbfaf5"))
        draw_line(Vector2(104, 15), Vector2(104, 11), INK, 1.0)
        draw_line(Vector2(104, 15), Vector2(107, 16), Color("c0392b"), 1.0)
        # calendar (the SPM date ringed in red)
        _r(113, 8, 14, 20, Color("fbfaf5"))
        _r(113, 8, 14, 4, Color("d9534f"))
        for y in range(14, 27, 3):
            for x in range(115, 126, 3):
                _r(x, y, 1, 1, Color("8a8a8a"))
        draw_arc(Vector2(121.5, 20.5), 2.5, 0, TAU, 8, Color("d9534f"), 1.0)
        # posters over the bookshelf: Harimau Malaya and a study timetable
        _r(194, 6, 15, 24, INK)
        _r(195, 7, 13, 22, Color("f0a030"))
        for y in range(9, 28, 4):
            _r(195, y, 4, 1, INK)
            _r(204, y + 2, 4, 1, INK)
        _r(199, 13, 5, 5, Color("fbe3b0"))
        _r(200, 14, 1, 1, INK)
        _r(202, 14, 1, 1, INK)
        _r(211, 9, 14, 19, Color("fbfaf5"))
        _r(211, 9, 14, 3, Color("3b6fb6"))
        for y in range(14, 27, 3):
            _r(212, y, 12, 1, Color("b9c4d6"))

    ## a songket-style rug under the middle of the room
    func _rug() -> void:
        _r(132, 84, 56, 40, Color("b8362e"))
        _r(135, 87, 50, 34, Color("e9c46a"))
        for y in range(91, 120, 8):
            for x in range(139, 182, 8):
                var c := Vector2(x + (y / 8) % 2 * 4, y)
                draw_colored_polygon(PackedVector2Array([c + Vector2(0, -3), c + Vector2(3, 0), c + Vector2(0, 3), c +
Vector2(-3, 0)]), Color("b8362e"))
        for x in range(133, 188, 3):
            _r(x, 82, 1, 2, TRIM)
            _r(x, 124, 1, 2, TRIM)

    ## sejadah laid in the corner (Alif walks around it)
    func _sejadah() -> void:
        _r(52, 114, 24, 30, Color("c9a24a"))
        _r(54, 116, 20, 26, Color("2f6b4a"))
        draw_colored_polygon(PackedVector2Array([Vector2(56, 128), Vector2(56, 122), Vector2(64, 117), Vector2(72, 122)
, Vector2(72, 128)]), Color("7a1f2b"))
        _r(63, 120, 2, 3, Color("c9a24a"))
        for y in range(131, 141, 3):
            _r(57, y, 14, 1, Color("3f8a60"))
        for x in range(52, 76, 2):
            _r(x, 144, 1, 2, TRIM)


# ---------------------------------------------------------------- furniture
class Piece extends Node2D:
    var id := ""
    var _t := 0.0

    func _process(delta: float) -> void:
        if id == "kipas":  # the fan keeps turning
            _t += delta
            queue_redraw()

    func _draw() -> void:
        # draw in room pixels: this node sits at the piece's bottom-centre
        draw_set_transform(-position)
        match id:
            "katil": _bed()
```

```
            "meja_sisi": _side_table()
            "rak_buku": _bookshelf()
            "kipas": _fan()
            "almari": _wardrobe()
            "meja_belajar": _desk()
            "kerusi": _chair()
            "beg": _bag()
            "kasut": _shoes()

    func _r(x: float, y: float, w: float, h: float, c: Color) -> void:
        draw_rect(Rect2(x, y, w, h), c)

    func _bed() -> void:
        _r(52, 94, 44, 4, Color(0, 0, 0, 0.18))  # shadow
        _r(48, 34, 48, 18, WOOD)  # headboard
        _r(48, 34, 48, 3, WOOD_L)
        _r(54, 39, 1, 10, WOOD_D)
        _r(89, 39, 1, 10, WOOD_D)
        _r(49, 50, 46, 46, WOOD_D)  # frame
        _r(50, 50, 44, 44, Color("f3f1ea"))  # sheet
        _r(56, 52, 28, 12, Color("cfcac0"))  # pillow
        _r(57, 52, 26, 11, Color("fbfaf5"))
        _r(57, 57, 26, 1, Color("d9e6f2"))
        _r(86, 56, 7, 28, Color("cdbf9e"))  # bantal peluk
        _r(87, 57, 5, 26, Color("f5e9d0"))
        _r(50, 66, 44, 28, Color("3b7fa6"))  # batik selimut
        _r(50, 66, 44, 4, Color("5a9cc0"))
        for y in range(73, 93, 6):
            for x in range(53, 92, 6):
                var c := Vector2(x + (y / 6) % 2 * 3, y)
                draw_colored_polygon(PackedVector2Array([c + Vector2(0, -2), c + Vector2(2, 0), c + Vector2(0, 2), c +
Vector2(-2, 0)]), Color("8ec7e0"))
                _r(c.x, c.y, 1, 1, Color("f3d36a"))
        _r(49, 94, 46, 2, WOOD)

    func _side_table() -> void:
        _r(98, 50, 13, 4, WOOD_L)
        _r(98, 54, 13, 10, WOOD)
        _r(99, 58, 11, 1, WOOD_D)
        _r(104, 56, 1, 1, Color("e6c96a"))
        _r(99, 44, 5, 7, Color("c9a24a"))  # family photo
        _r(100, 45, 3, 5, Color("9fd0e0"))
        _r(105, 46, 5, 5, Color("d9534f"))  # alarm clock
        _r(106, 47, 3, 3, Color("fbfaf5"))

    func _bookshelf() -> void:
        _r(194, 40, 28, 24, WOOD)
        _r(194, 40, 28, 3, WOOD_L)
        var cols := [Color("d9534f"), Color("3b6fb6"), Color("f0c24a"), Color("5fae6a"), Color("8e6cc0"), Color("e88aa0
"), Color("4aa3a3")]
        for row in 2:
            var x := 196
            var i := row * 3
            while x < 220:
                var w := 2 + (i * 7) % 3
                _r(x, 45 + row * 9, w, 7 - (i % 2), cols[i % cols.size()])
                x += w + 1
                i += 1
        _r(213, 34, 5, 6, Color("e6c96a"))  # a little trophy
        _r(214, 33, 3, 2, Color("e6c96a"))

    func _fan() -> void:
        draw_circle(Vector2(232, 62), 5, Color("9aa4ab"))
        draw_circle(Vector2(232, 61), 4, Color("bfc6cc"))
        _r(231, 42, 2, 19, Color("dfe3e6"))
        draw_circle(Vector2(232, 35), 8, Color("9aa4ab"))
        draw_circle(Vector2(232, 35), 7, Color("e6eef2"))
        for k in 3:
            var a := _t * 14.0 + k * TAU / 3.0
            draw_line(Vector2(232, 35), Vector2(232, 35) + Vector2.from_angle(a) * 6.0, Color("7fb2e0"), 3.0)
        draw_circle(Vector2(232, 35), 2, Color("4a7fc0"))
        draw_arc(Vector2(232, 35), 7, 0, TAU, 16, Color("b5bfc6"), 1.0)

    func _wardrobe() -> void:
        _r(246, 21, 16, 6, Color("c84a3a"))  # a suitcase on top
        _r(246, 21, 16, 1, Color("e06a5a"))
        _r(242, 26, 30, 54, WOOD_D)
        _r(243, 27, 28, 4, WOOD_L)
        _r(243, 31, 28, 48, Color("9c6a3c"))
        _r(256, 31, 1, 48, WOOD_D)
        _r(245, 34, 9, 42, Color("bfe0ea"))  # mirror
        draw_line(Vector2(247, 40), Vector2(252, 34), Color(1, 1, 1, 0.7), 1.0)
        draw_line(Vector2(247, 48), Vector2(253, 40), Color(1, 1, 1, 0.5), 1.0)
        _r(254, 52, 1, 5, Color("e6c96a"))
        _r(259, 52, 1, 5, Color("e6c96a"))
```

```
        _r(244, 80, 28, 2, Color(0, 0, 0, 0.18))

    func _desk() -> void:
        _r(240, 96, 32, 32, WOOD_D)
        _r(241, 97, 30, 30, Color("c99a63"))
        _r(245, 104, 15, 10, Color("6b4a2e"))  # open textbook
        _r(246, 104, 6, 9, Color("fbfaf5"))
        _r(253, 104, 6, 9, Color("fbfaf5"))
        for y in range(106, 112, 2):
            _r(247, y, 4, 1, Color("9aa0aa"))
            _r(254, y, 4, 1, Color("9aa0aa"))
        _r(262, 100, 7, 3, Color("3b6fb6"))  # book pile
        _r(262, 103, 7, 3, Color("d9534f"))
        _r(263, 106, 6, 3, Color("5fae6a"))
        _r(245, 118, 11, 4, Color("e05a5a"))  # pencil case
        _r(258, 118, 4, 5, Color("fbfaf5"))  # mug
        _r(262, 120, 1, 2, Color("fbfaf5"))
        draw_circle(Vector2(266, 118), 3, Color("4a7fc0"))  # desk lamp
        draw_circle(Vector2(266, 118), 1, Color("ffe9a0"))

    func _chair() -> void:
        _r(226, 114, 12, 12, WOOD)
        _r(227, 115, 10, 10, WOOD_L)
        _r(225, 110, 3, 18, WOOD_D)

    func _bag() -> void:
        _r(196, 126, 12, 2, Color(0, 0, 0, 0.2))
        _r(195, 113, 13, 14, Color("1b2a44"))
        _r(196, 114, 11, 12, Color("3b6fb6"))
        _r(196, 114, 11, 5, Color("5a8fd6"))
        _r(200, 118, 3, 2, Color("ffd24a"))
        _r(198, 111, 7, 2, Color("1b2a44"))  # handle
        draw_circle(Vector2(206, 123), 2, Color("f5e9d0"))  # a Pau keychain

    func _shoes() -> void:
        _r(194, 152, 14, 7, WOOD)
        _r(194, 152, 14, 2, WOOD_L)
        for x in [195, 201]:
            _r(x, 146, 5, 7, Color("9a9a92"))
            _r(x, 146, 4, 6, Color("f7f7f2"))
            _r(x + 1, 147, 2, 1, Color("c9c9c0"))


## morning sun through the louvres, with dust floating in it
class Light extends Node2D:
    var _t := 0.0

    func _ready() -> void:
        z_index = 10

    func _process(delta: float) -> void:
        _t += delta
        queue_redraw()

    func _draw() -> void:
        draw_colored_polygon(PackedVector2Array([Vector2(134, 44), Vector2(186, 44), Vector2(168, 122), Vector2(108, 12
2)]), Color(1, 0.94, 0.7, 0.10))
        for i in 9:
            var p := Vector2(140 + fmod(i * 23.0 + _t * 3.0, 40.0), 50 + fmod(i * 17.0 + _t * 5.0, 66.0))
            p.x -= (p.y - 44) * 0.3
            draw_rect(Rect2(p.floor(), Vector2.ONE), Color(1, 0.97, 0.85, 0.35 + 0.3 * sin(_t * 2.0 + i)))
```

`character_frames.gd` · scripts/world/character_frames.gd · 91 lines

```
class_name CharacterFrames
extends RefCounted
## Builds SpriteFrames from the PixelLab downloads in assets/characters32/:
##   <name>/<name>_<dir>.png                 idle, one per direction
##   <name>_walk/<name>_walk_<dir>_<i>.png   walk cycle (optional)
##   <name>_hover/<name>_hover_<dir>_<i>.png  looping idle instead of the
##                                           still pose (Nirbana floats)

const DIRS := ["south", "east", "north", "west"]
const BASE := "res://assets/characters32/"

static var _cache := {}


static func build(name: String, with_walk := false) -> SpriteFrames:
    var key := "%s:%s" % [name, with_walk]
    if _cache.has(key):
        return _cache[key]
    var frames := SpriteFrames.new()
```

```
    frames.remove_animation("default")
    for d in DIRS:
        var idle: String = "idle_" + d
        frames.add_animation(idle)
        var h := 0
        while ResourceLoader.exists("%s%s_hover/%s_hover_%s_%d.png" % [BASE, name, name, d, h]):
            frames.add_frame(idle, load("%s%s_hover/%s_hover_%s_%d.png" % [BASE, name, name, d, h]))
            h += 1
        if h == 0:
            frames.add_frame(idle, load("%s%s/%s_%s.png" % [BASE, name, name, d]))
        else:
            frames.set_animation_speed(idle, 7.0)
        if with_walk:
            var walk: String = "walk_" + d
            frames.add_animation(walk)
            frames.set_animation_speed(walk, 13.0)
            var i := 0
            while true:
                var path := "%s%s_walk/%s_walk_%s_%d.png" % [BASE, name, name, d, i]
                if not ResourceLoader.exists(path):
                    break
                frames.add_frame(walk, load(path))
                i += 1
    _cache[key] = frames
    return frames


## Offset that puts the bottom-centre of the visible pixels on the node origin.
static func feet_offset(name: String) -> Vector2:
    return visible_bottom_offset(load("%s%s/%s_south.png" % [BASE, name, name]))


static func visible_bottom_offset(tex: Texture2D) -> Vector2:
    var img := tex.get_image()
    var used := img.get_used_rect()
    var size := Vector2(img.get_size())
    return (size / 2.0 - Vector2(used.position.x + used.size.x / 2.0, used.end.y)).round()


## Gelanggang (side-view brawl): animations drawn facing east (flip for west).
##   <name>_<anim>/<name>_<anim>_east_<i>.png   (tools/asset_manifest.json)
## Missing ones fall back to the east idle / walk, so a fighter always works.
const FIGHT_ANIMS := ["stance", "jab", "cross", "kick", "roundhouse", "flykick", "hurt", "fall", "getup", "pickup", "th
row", "special"]


static func build_fight(name: String) -> SpriteFrames:
    var key := "%s:fight" % name
    if _cache.has(key):
        return _cache[key]
    var frames := SpriteFrames.new()
    frames.remove_animation("default")
    var idle: Texture2D = load("%s%s/%s_east.png" % [BASE, name, name])
    for anim in ["walk"] + FIGHT_ANIMS:
        frames.add_animation(anim)
        frames.set_animation_loop(anim, anim in ["walk", "stance"])
        frames.set_animation_speed(anim, 13.0 if anim == "walk" else 10.0)
        var i := 0
        while true:
            var path := "%s%s_%s/%s_%s_east_%d.png" % [BASE, name, anim, name, anim, i]
            if not ResourceLoader.exists(path):
                break
            frames.add_frame(anim, load(path))
            i += 1
        if i == 0:
            frames.add_frame(anim, idle)
    _cache[key] = frames
    return frames


## true when PixelLab frames exist for this fight animation
static func has_fight_anim(name: String, anim: String) -> bool:
    return ResourceLoader.exists("%s%s_%s/%s_%s_east_0.png" % [BASE, name, anim, name, anim])
```

critters.gd · scripts/world/critters.gd · 116 lines

```
class_name Critter
extends Node2D
## Little animals that make the kampung feel lived in: ayam (chickens) that
## wander and peck around the houses and itik (ducks) paddling on the pond and
## river. Drawn in code, purely for looks: they never block anyone.

const TILE := 16
```

```

var kind := "ayam"
var world: Node2D
var home := Vector2.ZERO        ## they never stray far from here
var _target := Vector2.ZERO
var _wait := 0.0
var _peck := 0.0
var _t := randf() * TAU
var _flip := false


func setup(w: Node2D, what: String, cell: Vector2i) -> void:
    world = w
    kind = what
    position = Vector2(cell) * TILE + Vector2(8, 12)
    home = position
    _target = position
    _wait = randf_range(0.5, 3.0)


func _process(delta: float) -> void:
    _t += delta
    _peck = maxf(_peck - delta, 0.0)
    var to := _target - position
    if to.length() > 0.5:
        var speed := 7.0 if kind == "itik" else 12.0
        position += to.limit_length(speed * delta)
        _flip = to.x < 0
    else:
        _wait -= delta
        if _wait <= 0:
            _wait = randf_range(1.5, 4.5)
            if kind == "ayam" and randf() < 0.45:
                _peck = 0.9  # stays put, pecks the ground
            else:
                _pick_target()
    queue_redraw()


func _pick_target() -> void:
    for i in 8:
        var p := home + Vector2(randf_range(-40, 40), randf_range(-28, 28))
        var c := Vector2i((p / TILE).floor())
        if _ok(c) and _ok(Vector2i(((position + p) / 2.0 / TILE).floor())):
            _target = p
            return


func _ok(c: Vector2i) -> bool:
    var water: bool = world.terrain.is_water(c) and not world.walk_over.has(c)
    if kind == "itik":
        return water and world.terrain.terrain_at(c) == TerrainBuilder.RIVER
    return not water and not world.blocked.has(c)


func _draw() -> void:
    if _flip:
        draw_set_transform(Vector2.ZERO, 0.0, Vector2(-1, 1))
    if kind == "itik":
        _draw_duck()
    else:
        _draw_chicken()


func _px(x: float, y: float, w: float, h: float, c: Color) -> void:
    draw_rect(Rect2(x, y, w, h), c)


func _draw_chicken() -> void:
    var walking := (_target - position).length() > 0.5
    var bob := 1.0 if walking and sin(_t * 16.0) > 0 else 0.0
    var head_down := _peck > 0 and fmod(_peck, 0.3) < 0.15
    var white := Color("f6f1e4")
    var shade := Color("d9cfbb")
    var out := Color("4a3a2a")
    # shadow, legs
    draw_circle(Vector2(0, 0), 3.5, Color(0, 0, 0, 0.18))
    _px(-1, -2, 1, 2, Color("e0a030"))
    _px(1, -2, 1, 2, Color("e0a030"))
    # body and tail
    _px(-4, -7 - bob, 7, 5, out)
    _px(-3, -6 - bob, 5, 3, white)
    _px(-3, -4 - bob, 5, 1, shade)
    _px(-5, -9 - bob, 2, 3, out)
    _px(-4, -8 - bob, 1, 2, white)
```

```
    # head (dips when pecking)
    var hy := -6.0 if head_down else -10.0
    var hx := 3.0 if head_down else 2.0
    _px(hx - 1, hy - bob - 1, 4, 4, out)
    _px(hx, hy - bob, 2, 2, white)
    _px(hx, hy - bob - 2, 2, 1, Color("e23b3b"))   # comb
    _px(hx + 2, hy - bob + 1, 2, 1, Color("f0b030"))  # beak
    _px(hx + 1, hy - bob, 1, 1, Color("1d1a18"))   # eye


func _draw_duck() -> void:
    var bob := sin(_t * 3.0) * 0.5
    # ripple ring under the duck
    draw_set_transform(Vector2(0, 0), 0.0, Vector2(-1 if _flip else 1, 0.4))
    draw_arc(Vector2(0, 0), 7.0 + sin(_t * 2.0), 0, TAU, 16, Color(1, 1, 1, 0.35), 1.0)
    draw_set_transform(Vector2.ZERO, 0.0, Vector2(-1 if _flip else 1, 1))
    var out := Color("3a3028")
    _px(-5, -4 + bob, 9, 4, out)
    _px(-4, -3 + bob, 7, 2, Color("f4efe2"))
    _px(-5, -5 + bob, 2, 2, out)
    _px(2, -8 + bob, 4, 4, out)
    _px(3, -7 + bob, 2, 2, Color("3f9a52"))   # green head
    _px(5, -6 + bob, 2, 1, Color("f0a020"))   # beak
    _px(4, -7 + bob, 1, 1, Color("101010"))
```

interior.gd · scripts/world/interior.gd · 377 lines

```
extends Node2D
## A one-screen interior (Pokémon-style building) drawn from a 320x180
## illustrated background. Alif walks on a set of floor cells; stepping onto
## an exit cell fades back to the overworld. Zone steps run through ZoneRunner.

const TILE := 16
const UI = preload("res://scripts/ui/ui.gd")
const Hud = preload("res://scripts/ui/hud.gd")
const ZonePrep = preload("res://scripts/screens/zone_prep.gd")

@export var background: Texture2D
@export var title := ""
@export var zone_id := ""
@export var music := "balai"
## floor rows as "y:x0-x1" strings, e.g. ["9:5-14", "10:2-17"]
@export var floor_rows: PackedStringArray
@export var exit_cells: Array[Vector2i] = []
@export var enter_cell := Vector2i(10, 10)
@export var return_cell_world := Vector2i(55, 30)   ## overworld cell outside the door
@export var npc_id := ""
@export var npc_cell := Vector2i(10, 9)
## extra objects: "id:texture:x:y:w" (anchor = bottom-left cell, w = width in cells)
@export var props: PackedStringArray
## story rooms: data/interiors/<file>.json with an intro, the
## host's talk and exhibits (hotspots or icons on pedestals) that each tell
## a short story. Exhibits tied to an artifact stay dark until it is found.
@export var exhibit_file := ""

var _floor := {}
var busy := false
var npc: Npc
var runner := ZoneRunner.new()
var hud: Control
var _props := {}  ## id -> {sprite, cells}
var _room := {}   ## exhibit_file contents
var _exhibits := {}  ## id -> exhibit dict (with "cells" as Vector2i)

@onready var player: Player = $Objects/Player
@onready var pau: Pau = $Objects/Pau
@onready var prompt: Button = $HUD/Prompt


func _ready() -> void:
    $Background.texture = background
    for row in floor_rows:
        var parts := row.split(":")
        var span := parts[1].split("-")
        for x in range(int(span[0]), int(span[1]) + 1):
            _floor[Vector2i(x, int(parts[0]))] = true
    for c in exit_cells:
        _floor[c] = true
    for p in props:
        var f := p.split(":")
        var tex: Texture2D = load("res://assets/objects/%s.png" % f[1])
        var s := Sprite2D.new()
        s.texture = tex
```

```
        s.offset = CharacterFrames.visible_bottom_offset(tex)
        var at := Vector2i(int(f[2]), int(f[3]))
        var w := int(f[4])
        s.position = Vector2((at.x + w / 2.0) * TILE, (at.y + 1) * TILE)
        $Objects.add_child(s)
        var cells := []
        for dx in w:
            cells.append(at + Vector2i(dx, 0))
            _floor.erase(at + Vector2i(dx, 0))
        _props[f[0]] = {"sprite": s, "cells": cells}
    if exhibit_file != "":
        _build_exhibits()
    if npc_id != "":
        npc = Npc.new()
        npc.setup(npc_id, npc_cell, "south")
        $Objects.add_child(npc)

    hud = Hud.new()
    hud.exit_action = _leave
    $HUD.add_child(hud)
    $HUD.move_child(hud, 0)
    hud.set_location(title.to_upper(), str(Game.zone_data(zone_id).get("subtopik", "")) if zone_id != "" else "")
    prompt.theme = Game.theme
    prompt.pressed.connect(func(): _interact(player.facing_cell()))

    if not exit_cells.is_empty():
        var arrow := ExitArrow.new()
        var mid := Vector2.ZERO
        for c in exit_cells:
            mid += Vector2(c) * TILE + Vector2(TILE / 2.0, 0)
        mid /= exit_cells.size()
        arrow.position = Vector2(mid.x, minf(mid.y + 4, 166))  # stay on screen
        add_child(arrow)

    player.world = self
    player.get_node("Camera2D").enabled = false
    player.place_at(enter_cell, "north")
    pau.place_at(enter_cell + Vector2i(-1, 0))
    pau.follow(player)
    player.step_finished.connect(_on_step)
    player.bumped.connect(func(_c): Sfx.play_varied("bump", -4.0))
    Main.set_pau_visible(true)
    MusicDirector.play(music)
    add_child(runner)
    runner.setup(self)
    _welcome()


func _welcome() -> void:
    if zone_id == "" and not _room.is_empty():
        var key := "room_" + exhibit_file
        if not Game.save["seen"].has(key):
            Game.save["seen"].append(key)
            Game.persist()
            busy = true
            await get_tree().create_timer(0.5).timeout
            await Game.say(_room.get("intro", []))
            busy = false
        return
    if zone_id == "" or Game.zone_done(zone_id) or not Game.zone_state(zone_id)["steps"].is_empty():
        return
    if not Game.zone_unlocked(zone_id) or GameState.flags.get("welcomed_" + zone_id, false):
        return
    GameState.flags["welcomed_" + zone_id] = true
    busy = true
    await get_tree().create_timer(0.5).timeout
    var prep := ZonePrep.new()
    prep.set("params", {"zone": zone_id})
    Main.overlay_root().add_child(prep)
    await prep.closed
    await Game.say(str(Game.zone_data(zone_id).get("intro", "")))
    runner.refresh()
    busy = false


# --- ZoneRunner host API ------------------------------------------------------
func spot_ids() -> Array:
    var ids := _props.keys()
    if npc_id != "":
        ids.append(npc_id)
    return ids


func marker_pos(id: String) -> Vector2:
    if id == npc_id:
```

```
        return Player.cell_to_pos(npc_cell) + Vector2(0, -34)
    var p: Dictionary = _props.get(id, {})
    if p.is_empty():
        return Vector2.ZERO
    var s: Sprite2D = p["sprite"]
    return s.position + Vector2(0, -s.texture.get_image().get_used_rect().size.y - 4)


func apply_after(_zid: String, _s: Dictionary, _animate: bool) -> void:
    pass


func on_zone_complete(_zid: String) -> void:
    pass


func return_cell() -> Vector2i:
    return return_cell_world


# --- movement / interaction ------------------------------------------------------
func is_walkable(c: Vector2i) -> bool:
    return _floor.has(c) and not (npc and c == npc.cell)


func _on_step(c: Vector2i) -> void:
    Sfx.play_varied("step_wood", -6.0, 0.12)
    if c in exit_cells:
        _leave()


## back to the overworld, outside this building's door
func _leave() -> void:
    if busy or runner.busy:
        return
    busy = true
    player.can_move = false
    Sfx.play("whoosh", -10.0)
    GameState.spawn_cell = return_cell_world
    GameState.spawn_facing = "south"
    GameState.go_to("res://scenes/world/world.tscn")


func _process(_d: float) -> void:
    var c := player.facing_cell()
    var label := ""
    if not busy and not runner.busy and not Game.is_locked() and not player.moving:
        if npc and c == npc.cell:
            label = Game.char_name(npc_id)
        else:
            for id in _props:
                if c in _props[id]["cells"]:
                    label = _prop_label(id)
            var ex := _exhibit_at(c)
            if ex != "":
                var e: Dictionary = _exhibits[ex]
                var have: bool = not e.has("artifact") or Game.save["artifacts"].has(str(e["artifact"]))
                label = str(e.get("label", "Lihat")) if have else "???"
    prompt.visible = label != ""
    if prompt.visible:
        prompt.text = "E / Ketik: " + label
        prompt.reset_size()
        var screen := get_viewport().get_canvas_transform() * (player.global_position + Vector2(0, 14))
        var y := screen.y + 8
        if y + prompt.size.y > 600:  # near the door: above Alif's head, off the Keluar arrow
            y = screen.y - 150 - prompt.size.y
        prompt.position = Vector2(clampf(screen.x - prompt.size.x / 2.0, 10, 1270 - prompt.size.x), clampf(y, 120, 640)
)


func _build_exhibits() -> void:
    _room = Game.load_json("res://data/interiors/%s.json" % exhibit_file)
    for e in _room.get("exhibits", []):
        var cells: Array = []
        for c in e["cells"]:
            cells.append(Vector2i(int(c[0]), int(c[1])))
            _floor.erase(Vector2i(int(c[0]), int(c[1])))
        var ex: Dictionary = e.duplicate()
        ex["cells"] = cells
        _exhibits[str(e["id"])] = ex
        if e.has("icon"):  # an artifact on a little pedestal
            var at: Vector2i = cells[0]
            var base := ColorRect.new()
            base.color = Color("6b4a2e")
            base.size = Vector2(14, 7)
```

```
            base.position = Vector2(at.x * TILE + 1, at.y * TILE + 8)
            $Objects.add_child(base)
            var top := ColorRect.new()
            top.color = Color("a8794a")
            top.size = Vector2(14, 2)
            top.position = base.position
            $Objects.add_child(top)
            var icon := Sprite2D.new()
            icon.texture = load("res://assets/icons/%s.png" % e["icon"])
            icon.scale = Vector2.ONE * (14.0 / icon.texture.get_width())
            icon.position = Vector2(at.x * TILE + 8, at.y * TILE + 2)
            var have: bool = not e.has("artifact") or Game.save["artifacts"].has(str(e["artifact"]))
            if not have:
                icon.modulate = Color(0, 0, 0, 0.6)
            else:
                var t := create_tween().set_loops()
                t.tween_property(icon, "position:y", icon.position.y - 1.5, 0.9).set_trans(Tween.TRANS_SINE)
                t.tween_property(icon, "position:y", icon.position.y, 0.9).set_trans(Tween.TRANS_SINE)
            $Objects.add_child(icon)


func _exhibit_at(c: Vector2i) -> String:
    for id in _exhibits:
        if c in _exhibits[id]["cells"]:
            return id
    return ""


func _show_exhibit(id: String) -> void:
    var e: Dictionary = _exhibits[id]
    if e.has("artifact") and not Game.save["artifacts"].has(str(e["artifact"])):
        await Game.say([{"s": "pau", "t": str(e.get("locked", "Kosong lagi. Kita belum jumpa benda ni. Nanti kita isi!"
))}])
        return
    Sfx.play("chime", -8.0)
    if e.has("icon"):
        Game.toast(str(e.get("label", "")))
    await Game.say(e.get("lines", []))
    match str(e.get("open", "")):
        "beg": await Main.open_beg().closed
        "nota": await Main.open_nota().closed


func _prop_label(id: String) -> String:
    for o in Game.zone_data(zone_id).get("objects", []):
        if str(o["id"]) == id and o.has("label"):
            return str(o["label"])
    return "Lihat"


func _unhandled_input(event: InputEvent) -> void:
    if busy or runner.busy or Game.is_locked() or player.moving:
        return
    if event.is_action_pressed("interact"):
        get_viewport().set_input_as_handled()
        _interact(player.facing_cell())
    elif event.is_action_pressed("menu"):
        get_viewport().set_input_as_handled()
        Main.open_settings()
    elif event is InputEventMouseButton and event.pressed and event.button_index == MOUSE_BUTTON_LEFT:
        get_viewport().set_input_as_handled()
        var tc := get_node_or_null("TouchControls")
        if tc and tc.owns(event.position):
            return  # a thumb on the D-pad / joystick, not a tap on the floor
        var cell := Vector2i((get_global_mouse_position() / TILE).floor())
        if _near_exit(get_global_mouse_position()):
            _leave()
            return
        var target_id := ""
        if npc and (cell == npc.cell or cell == npc.cell + Vector2i(0, -1)):
            cell = npc.cell
            target_id = npc_id
        for id in _props:
            if cell in _props[id]["cells"]:
                target_id = id
        if _exhibit_at(cell) != "":
            target_id = _exhibit_at(cell)
        if target_id != "":
            var path := Pathfind.to_neighbour(self, player.cell, cell)
            var go := func():
                player.face_cell(cell)
                _interact(cell)
            if path.is_empty():
                go.call()
            else:
```

```
                player.walk_path(path, go)
        elif is_walkable(cell):
            player.walk_path(Pathfind.find(self, player.cell, cell))


## a tap on (or just around) the Keluar arrow at the door
func _near_exit(at: Vector2) -> bool:
    for c in exit_cells:
        var r := Rect2(Vector2(c) * TILE + Vector2(-6, -18), Vector2(TILE + 12, TILE + 24))
        if r.has_point(at):
            return true
    return false


func _interact(c: Vector2i) -> void:
    if busy or runner.busy or Game.is_locked():
        return
    var id := ""
    if npc and c == npc.cell:
        id = npc_id
        npc.face_towards(player.cell)
    elif c == pau.cell:
        busy = true
        pau.hop()
        await Game.say(Dialogue.lines("pau_tap"))
        busy = false
        return
    else:
        for pid in _props:
            if c in _props[pid]["cells"]:
                id = pid
        if _exhibit_at(c) != "":
            busy = true
            await _show_exhibit(_exhibit_at(c))
            busy = false
            return
    if id == "":
        return
    busy = true
    var handled: bool = await runner.handle(id)
    if not handled and id == npc_id and _room.has("npc_talk"):
        await Game.say(_room["npc_talk"])
    elif not handled and Dialogue.has(id):
        await Game.say(Dialogue.lines(id))
    busy = false


## Pulsing "Keluar" arrow over the door, drawn in pixels so it is crisp at 4x.
class ExitArrow extends Node2D:
    var _t := 0.0

    func _ready() -> void:
        z_index = 15

    func _process(delta: float) -> void:
        _t += delta
        queue_redraw()

    func _draw() -> void:
        var y := sin(_t * 4.0) * 1.5
        var ink := Color(0.17, 0.11, 0.06)
        var gold := Color(1, 0.86, 0.3)
        # down arrow: shaft + head, with a dark outline
        draw_rect(Rect2(-3, y - 9, 6, 7), ink)
        draw_rect(Rect2(-2, y - 8, 4, 6), gold)
        for i in 5:
            draw_rect(Rect2(-6 + i, y - 2 + i, 12 - i * 2, 1), ink if i == 4 else gold)
        draw_rect(Rect2(-7, y - 3, 14, 1), ink)
```

npc.gd · scripts/world/npc.gd · 96 lines

```
class_name Npc
extends Node2D
## A villager or historical figure standing on one tile. Turns to face Alif
## when spoken to, breathes gently while idle. With enable_wander() it strolls
## one tile at a time around its home spot (never further than the radius),
## asking the world first whether the tile is free.

var id := ""
var look := ""  ## sprite folder: id, or its "stand_in" art until its own sprite exists
var cell := Vector2i.ZERO
var facing := "south"

```

```
const STEP_TIME := 0.42

var home := Vector2i.ZERO
var wander_radius := 0
var moving := false

var _sprite: AnimatedSprite2D
var _t := randf() * TAU
var _world: Node
var _rest := 0.0


func setup(npc_id: String, at: Vector2i, face: String) -> void:
    id = npc_id
    look = npc_id
    if not ResourceLoader.exists("%s%s/%s_south.png" % [CharacterFrames.BASE, npc_id, npc_id]):
        look = str(Game.characters.get(npc_id, {}).get("stand_in", {}).get("sprite", npc_id))
    cell = at
    facing = face
    position = Player.cell_to_pos(at)
    _sprite = AnimatedSprite2D.new()
    _sprite.sprite_frames = CharacterFrames.build(look)
    _sprite.offset = CharacterFrames.feet_offset(look)
    add_child(_sprite)
    _sprite.play("idle_" + facing)


func enable_wander(world: Node, radius := 2) -> void:
    _world = world
    home = cell
    wander_radius = radius
    _sprite.sprite_frames = CharacterFrames.build(look, true)
    _sprite.play("idle_" + facing)
    _rest = randf_range(1.5, 5.0)


func face_towards(other: Vector2i) -> void:
    var d := other - cell
    if absi(d.x) > absi(d.y):
        facing = "east" if d.x > 0 else "west"
    else:
        facing = "south" if d.y > 0 else "north"
    _sprite.play("idle_" + facing)


func _process(delta: float) -> void:
    _t += delta
    _sprite.scale.y = 1.0 + sin(_t * 2.2) * 0.02  # breathing
    if wander_radius > 0 and not moving:
        _rest -= delta
        if _rest <= 0.0:
            _rest = randf_range(2.0, 5.0)
            _wander()


func _wander() -> void:
    if not _world.npc_may_wander(self):
        return
    var options: Array[Vector2i] = []
    for d in [Vector2i.UP, Vector2i.DOWN, Vector2i.LEFT, Vector2i.RIGHT]:
        var c: Vector2i = cell + d
        var off := c - home
        if absi(off.x) <= wander_radius and absi(off.y) <= wander_radius and _world.npc_cell_free(c):
            options.append(c)
    if options.is_empty():
        return
    var to: Vector2i = options.pick_random()
    face_towards(to)
    _world.move_npc(self, to)
    cell = to
    moving = true
    var walk := "walk_" + facing
    if _sprite.sprite_frames.has_animation(walk) and _sprite.sprite_frames.get_frame_count(walk) > 0:
        _sprite.play(walk)
    var tw := create_tween()
    tw.tween_property(self, "position", Player.cell_to_pos(to), STEP_TIME)
    await tw.finished
    moving = false
    _sprite.play("idle_" + facing)


func _draw() -> void:
    draw_set_transform(Vector2(0, -1), 0, Vector2(1, 0.45))
    draw_circle(Vector2.ZERO, 6, Color(0, 0, 0, 0.28))
```

pathfind.gd · scripts/world/pathfind.gd · 41 lines

```
class_name Pathfind
extends RefCounted
## Grid BFS for tap-to-walk. `world` must provide is_walkable(cell).

const DIRS := [Vector2i(0, 1), Vector2i(0, -1), Vector2i(1, 0), Vector2i(-1, 0)]
const LIMIT := 2500  ## max cells explored (keeps taps on far-away tiles cheap)


## Cells to step through from `from` to `to` (excluding `from`). [] if unreachable.
static func find(world: Node, from: Vector2i, to: Vector2i) -> Array:
    if from == to or not world.is_walkable(to):
        return []
    return _bfs(world, from, func(c): return c == to)


## Path to any walkable tile next to `target` (for talking to it).
static func to_neighbour(world: Node, from: Vector2i, target: Vector2i) -> Array:
    return _bfs(world, from, func(c): return (c - target).length_squared() == 1)


static func _bfs(world: Node, from: Vector2i, goal: Callable) -> Array:
    if goal.call(from):
        return []
    var prev := {from: from}
    var queue: Array[Vector2i] = [from]
    var head := 0
    while head < queue.size() and head < LIMIT:
        var c := queue[head]
        head += 1
        for d in DIRS:
            var n: Vector2i = c + d
            if prev.has(n) or not world.is_walkable(n):
                continue
            prev[n] = c
            if goal.call(n):
                var path := [n]
                while prev[path[0]] != from:
                    path.push_front(prev[path[0]])
                return path
            queue.append(n)
    return []
```

terrain_builder.gd · scripts/world/terrain_builder.gd · 138 lines

```
class_name TerrainBuilder
extends RefCounted
## Builds a TileSet from the PixelLab Wang tilesets and paints a TileMapLayer
## from data/world/terrain.txt.
##
## Terrain lives on cell *corners* (vertices). Each vertex takes the highest
## terrain of the 4 cells around it, then every cell picks the tile whose
## NW/NE/SW/SE corners match. tools/gen_world.py guarantees only legal pairs
## meet (sea-sand, sand-grass, river-grass, grass-meadow, grass-path, grass-jalan).

enum { SEA, SAND, RIVER, GRASS, MEADOW, PATH, JALAN }

const CHARS := {"~": SEA, "s": SAND, "r": RIVER, ".": GRASS, ",": MEADOW, "=": PATH, "j": JALAN}
const PAIRS := {  # [lower, upper] -> tileset name in assets/tiles/
    Vector2i(SEA, SAND): "sea_sand",
    Vector2i(SAND, GRASS): "sand_grass",
    Vector2i(RIVER, GRASS): "river_grass",
    Vector2i(GRASS, MEADOW): "grass_meadow",
    Vector2i(GRASS, PATH): "grass_path",
    Vector2i(GRASS, JALAN): "grass_jalan",   # Lembah Kebangkitan's gravel town roads
}
const TILE := 16

var tile_set := TileSet.new()
var cells: Array = []  ## cells[y][x] = terrain id
var width := 0
var height := 0

var _wang := {}  ## Vector2i pair -> {source_id, coords: Dictionary wang_index -> Vector2i}


func _init() -> void:
    tile_set.tile_size = Vector2i(TILE, TILE)
    var source_id := 0
    for pair in PAIRS:
        var name: String = PAIRS[pair]
        var meta: Dictionary = JSON.parse_string(FileAccess.get_file_as_string("res://assets/tiles/%s_metadata.json" %
```

```
name))
        var src := TileSetAtlasSource.new()
        src.texture = load("res://assets/tiles/%s_image.png" % name)
        src.texture_region_size = Vector2i(TILE, TILE)
        var coords := {}
        for t in meta["tileset_data"]["tiles"]:
            var box: Dictionary = t["bounding_box"]
            var at := Vector2i(int(box["x"]) / TILE, int(box["y"]) / TILE)
            if not src.has_tile(at):
                src.create_tile(at)
            var c: Dictionary = t["corners"]
            var idx := _bit(c["NW"]) * 8 + _bit(c["NE"]) * 4 + _bit(c["SW"]) * 2 + _bit(c["SE"])
            coords[idx] = at
        tile_set.add_source(src, source_id)
        _wang[pair] = {"source": source_id, "coords": coords}
        source_id += 1


func _bit(corner: String) -> int:
    return 1 if corner == "upper" else 0


func load_layout(path: String) -> void:
    cells.clear()
    for line in FileAccess.get_file_as_string(path).split("\n", false):
        line = line.strip_edges(false, true)  # Windows checkouts end lines with \r
        if line == "":
            continue
        var row: Array[int] = []
        for ch in line:
            row.append(CHARS.get(ch, GRASS))
        cells.append(row)
    height = cells.size()
    width = cells[0].size() if height > 0 else 0


func terrain_at(cell: Vector2i) -> int:
    if cell.x < 0 or cell.y < 0 or cell.x >= width or cell.y >= height:
        return SEA
    return cells[cell.y][cell.x]


func is_water(cell: Vector2i) -> bool:
    var t := terrain_at(cell)
    return t == SEA or t == RIVER


func paint(layer: TileMapLayer) -> void:
    layer.tile_set = tile_set
    layer.clear()
    # vertex terrain = highest of the 4 surrounding cells
    var v: Array = []
    for y in height + 1:
        var row: Array[int] = []
        for x in width + 1:
            var best := SEA
            for d in [Vector2i(-1, -1), Vector2i(0, -1), Vector2i(-1, 0), Vector2i(0, 0)]:
                var cx: int = clampi(x + d.x, 0, width - 1)
                var cy: int = clampi(y + d.y, 0, height - 1)
                best = maxi(best, cells[cy][cx])
            row.append(best)
        v.append(row)

    for y in height:
        for x in width:
            var corners := [v[y][x], v[y][x + 1], v[y + 1][x], v[y + 1][x + 1]]
            _place(layer, Vector2i(x, y), corners, cells[y][x])


func _place(layer: TileMapLayer, cell: Vector2i, corners: Array, own: int) -> void:
    var lo: int = corners.min()
    var hi: int = corners.max()
    if lo == hi:
        _place_base(layer, cell, lo)
        return
    var pair := Vector2i(lo, hi)
    if not _wang.has(pair) or _has_third(corners, lo, hi):
        _place_base(layer, cell, own)  # illegal mix; fall back to the cell's own terrain
        return
    var idx := 0
    for i in 4:
        if corners[i] == hi:
            idx |= 8 >> i
    var w: Dictionary = _wang[pair]
    layer.set_cell(cell, w["source"], w["coords"][idx])
```

```
122
123
124 func _has_third(corners: Array, lo: int, hi: int) -> bool:
125     for c in corners:
126         if c != lo and c != hi:
127             return true
128     return false
129
130
131 func _place_base(layer: TileMapLayer, cell: Vector2i, terrain: int) -> void:
132     for pair in _wang:
133         if pair.x == terrain:
134             layer.set_cell(cell, _wang[pair]["source"], _wang[pair]["coords"][0])
135             return
136         if pair.y == terrain:
137             layer.set_cell(cell, _wang[pair]["source"], _wang[pair]["coords"][15])
138             return
```

`world.gd` · scripts/world/world.gd · 1267 lines

```
1  extends Node2D
2  ## Dunia Explo-Story overworld. Builds itself from data/world/ (made by
3  ## tools/gen_world.py): terrain tiles, props with collision, NPCs, signs, flag
4  ## gates and the Bab 1 puzzle spots. Zone logic (from the first prototype)
5  ## runs through ZoneRunner; this script shows its results in the world.
6
7  const TILE := 16
8  const SWAY := preload("res://shaders/sway.gdshader")
9  const WATER := preload("res://shaders/water.gdshader")
10 const KABUS := preload("res://shaders/kabus_lupa.gdshader")
11 const UI = preload("res://scripts/ui/ui.gd")
12 const Hud = preload("res://scripts/ui/hud.gd")
13 const ZonePrep = preload("res://scripts/screens/zone_prep.gd")
14 const Minimap = preload("res://scripts/ui/minimap.gd")
15 const ObjectiveArrow = preload("res://scripts/ui/objective_arrow.gd")
16 const ArenaYard = preload("res://scripts/world/arena_yard.gd")
17 const HOLD_TIME := 0.6  ## long-press Pau = calm moment
18 const LEMBAH_X := 82   ## columns from here on are Lembah Kebangkitan (Bab 2)
19
20 ## char -> texture in assets/objects/ (an Array = variants, see
21 ## object_texture), footprint (w, h) in cells, sway = wind shader, flat = drawn
22 ## under characters with no collision, walk = footprint cells become walkable
23 ## (jetty, bridges), open = walkable columns
24 const OBJECTS := {
25     "P": {"tex": "palm", "foot": Vector2i(1, 1), "sway": 1.2},
26     "T": {"tex": ["tree", "tree_mempelam", "tree_ketapang", "tree_round", "tree", "tree_rambutan", "tree_round"], "foot
   ": Vector2i(2, 1), "sway": 1.0},
27     "B": {"tex": ["bush", "bush_bunga"], "foot": Vector2i(2, 1), "sway": 0.6},
28     "R": {"tex": "rocks", "foot": Vector2i(2, 1)},
29     "F": {"tex": ["bunga_kuning", "bunga_merah_jambu"], "foot": Vector2i(2, 1), "flat": true, "sway": 0.5},
30     "f": {"tex": "pagar", "foot": Vector2i(1, 1)},
31     "H": {"tex": ["rumah_biru", "rumah_merah", "rumah_hijau"], "foot": Vector2i(6, 3), "talk": "house", "order": true},
32     "U": {"tex": "rumah_alif", "fallback": "rumah_biru", "foot": Vector2i(6, 3)},  # Rumah Alif
33     "M": {"tex": ["gerai_buah", "gerai_rempah"], "foot": Vector2i(4, 2), "order": true},
34     "D": {"tex": "gudang", "foot": Vector2i(6, 3)},
35     "n": {"tex": "jambatan", "foot": Vector2i(5, 2), "flat": true, "walk": true},
36     "l": {"tex": "pelita", "foot": Vector2i(1, 1)},
37     "w": {"tex": "perigi", "foot": Vector2i(2, 1)},
38     "j": {"tex": "tempayan", "foot": Vector2i(1, 1)},
39     "k": {"tex": "tong", "foot": Vector2i(1, 1)},
40     "c": {"tex": "peti", "foot": Vector2i(2, 1)},
41     "g": {"tex": "kayu_balak", "foot": Vector2i(2, 1)},
42     "u": {"tex": "tunggul", "foot": Vector2i(1, 1)},
43     "e": {"tex": "bangku", "foot": Vector2i(2, 1)},
44     "x": {"tex": "kayu_api", "foot": Vector2i(2, 1)},
45     "p": {"tex": "teratai", "foot": Vector2i(1, 1), "flat": true},
46     "q": {"tex": "rumput_air", "foot": Vector2i(1, 1), "flat": true, "sway": 0.8},
47     "K": {"tex": "monument", "foot": Vector2i(4, 2)},
48     "Y": {"tex": "banyan", "foot": Vector2i(6, 2), "sway": 0.8},
49     "G": {"tex": "gate_open", "foot": Vector2i(4, 1), "open": [1, 2]},
50     "L": {"tex": "gate_locked", "foot": Vector2i(4, 1)},
51     "S": {"tex": "sign", "foot": Vector2i(1, 1)},
52     "I": {"tex": "istana", "foot": Vector2i(10, 4)},
53     "J": {"tex": "jetty", "foot": Vector2i(2, 6), "flat": true, "walk": true},
54     "X": {"tex": "ship", "foot": Vector2i(6, 4)},
55     # Lembah Kebangkitan (Bab 2)
56     "O": {"tex": "dewan", "foot": Vector2i(8, 4)},
57     "V": {"tex": "rumah_falsafah", "foot": Vector2i(6, 3)},
58     "E": {"tex": "balai_serantau", "foot": Vector2i(6, 3)},
59     "N": {"tex": "madrasah", "foot": Vector2i(6, 3)},
60     "C": {"tex": "jelutong_press", "foot": Vector2i(6, 3)},
61     "h": {"tex": ["kedai_merah", "kedai_biru"], "foot": Vector2i(6, 3), "order": true},
62     "m": {"tex": "sitc", "foot": Vector2i(10, 4)},
```

```
    "t": {"tex": "menara", "foot": Vector2i(6, 4)},
    "y": {"tex": "tugu_pahlawan", "foot": Vector2i(3, 2)},
    "a": {"tex": "papan_notis", "foot": Vector2i(2, 1)},
    "o": {"tex": "rak_buku", "foot": Vector2i(2, 1)},
    "d": {"tex": "meja_pengarang", "foot": Vector2i(2, 1)},
    "v": {"tex": "pintu_kabus", "foot": Vector2i(4, 1), "open": [1, 2]},
    "z": {"tex": "lampu_kolonial", "foot": Vector2i(1, 1)},
    # Lembah Kebangkitan, a 1920s town (data/world/lembah_layout.json)
    "6": {"tex": "kedai_deret_a", "foot": Vector2i(8, 3)},
    "7": {"tex": "kedai_deret_b", "foot": Vector2i(8, 3)},
    "8": {"tex": "menara_jam", "foot": Vector2i(3, 2)},
    "9": {"tex": "wakaf_padang", "foot": Vector2i(5, 3)},
    "@": {"tex": "kapal_wap", "foot": Vector2i(8, 3)},
    "$": {"tex": "gerai_akhbar", "foot": Vector2i(2, 1)},
    "!": {"tex": "papan_misi", "foot": Vector2i(2, 1)},
    "%": {"tex": "pokok_angsana", "foot": Vector2i(2, 1), "sway": 0.8},
    "&": {"tex": "pokok_getah", "foot": Vector2i(2, 1), "sway": 1.0},
    "+": {"tex": "tiang_telegraf", "foot": Vector2i(1, 1)},
    "0": {"tex": "basikal", "foot": Vector2i(2, 1)},
    "#": {"tex": "pagar_besi", "foot": Vector2i(2, 1)},
    "b": {"tex": "perahu", "foot": Vector2i(3, 1)},
    "1": {"tex": "gate_funan", "foot": Vector2i(3, 1)},
    "2": {"tex": "gate_champa", "foot": Vector2i(3, 1)},
    "3": {"tex": "gate_srivijaya", "foot": Vector2i(3, 1)},
    "4": {"tex": "gate_melaka", "foot": Vector2i(3, 1)},
    "5": {"tex": "gate_johor", "foot": Vector2i(3, 1)},
    "A": {"tex": "tiang_runtuh", "foot": Vector2i(4, 1)},
    "Q": {"tex": "panji_turun", "foot": Vector2i(1, 1)},
    "W": {"tex": "arena", "foot": Vector2i(6, 2)},
    "Z": {"tex": "batu_renungan", "foot": Vector2i(2, 1)},
}
const KINGDOM := {"1": "funan", "2": "champa", "3": "srivijaya", "4": "melaka", "5": "johor"}
## buildings you can walk into as story / museum rooms:
## id -> [scene, card title, card subtitle, Pau's line at the door]
const ROOMS := {
    "rumah_alif": ["res://scenes/interior/bilik_alif.tscn", "BILIK ALIF", "Rumah Alif, Kampung Jejak", "Bilik kau! Jom masuk kejap."],
    "sitc": ["res://scenes/interior/sitc.tscn", "MAKTAB SITC", "Tanjung Malim, 1922", "Ni Maktab Perguruan Sultan Idris. Jom jenguk kelas dia."],
    "jelutong_press": ["res://scenes/interior/jelutong.tscn", "JELUTONG PRESS", "Pulau Pinang, 1927", "Bunyi mesin cetak! Jom tengok dalam."],
}
const AREA_ZONE := {"gerbang": "1_1", "pelabuhan": "1_2", "siguntang": "1_4"}
const LABELS := {
    "house": "Rumah", "rumah_alif": "Rumah Alif", "monument": "Peta Jejak", "istana": "Istana Melaka",
    "sign_start": "Papan tanda", "sign_sea": "Papan tanda", "t5": "Papan tanda",
    "arena": "Arena Ulang Kaji", "gate_locked": "Gerbang berkunci", "gate_open_1": "Gerbang Bab 1",
    "gate_open_2": "Pintu Kabus Bab 2", "portal_balik": "Pintu Kabus", "dewan": "Dewan Kedaulatan Rakyat",
    "madrasah": "Madrasah", "sitc": "Maktab Perguruan Sultan Idris", "sign_lembah": "Papan tanda",
    "arena_lembah": "Arena Ulang Kaji", "papan_markah": "Papan Markah", "patung_latihan": "Patung Latihan",
    "pemain_arena": "Pemain lain",
}

var terrain := TerrainBuilder.new()
var blocked := {}      ## Vector2i -> true
var walk_over := {}    ## Vector2i -> true (jetty planks over water)
var interact := {}     ## Vector2i -> {type, id}
var npcs := {}         ## Vector2i -> Npc
var portals := {}      ## Vector2i -> {to, facing}: step in and you cross to the other island
var houses: Array[Sprite2D] = []   ## kampung houses (chickens and kitchen smoke)
var critters: Array[Node2D] = []   ## chickens and ducks (looks only)
var spots := {}        ## spot id -> {pos, size}
var spot_sprites := {} ## spot id -> Sprite2D
var gate_sprites := {} ## kingdom -> Sprite2D
var areas: Array = []
var current_area := ""
var busy := false
var runner := ZoneRunner.new()
var hud: Control
var minimap: Control
var map_texture: Texture2D  ## picture of the whole world for the maps (see _capture_map)
var map_sea := Color("3b8fc4")  ## sea colour around the map picture
var _ent: Dictionary
var _last_move := 0.0
var _thread: Line2D
var _pau_press := -1.0  ## time Pau was pressed (long-press), -1 = not pressed
var _ujian_cells := {}  ## chapter -> cell where its Ujian Bab boss waits
var _goal := {}         ## objective() of this frame (the maps and the arrow all ask)
var _goal_frame := -1
var _dummies := {}      ## cell -> training dummy sprite (Arena Ulang Kaji yards)

@onready var ground: TileMapLayer = $Ground
@onready var decor: Node2D = $Decor
@onready var objects: Node2D = $Objects
@onready var player: Player = $Objects/Player
```

```
@onready var pau: Pau = $Objects/Pau
@onready var camera: Camera2D = $Objects/Player/Camera2D
@onready var emote: Label = $HUD/Emote
@onready var title_card: Control = $HUD/TitleCard  ## area title card (_area_intro)
@onready var prompt: Button = $HUD/Prompt


func _ready() -> void:
    _ent = JSON.parse_string(FileAccess.get_file_as_string("res://data/world/entities.json"))
    terrain.load_layout("res://data/world/terrain.txt")
    terrain.paint(ground)
    var water := ShaderMaterial.new()
    water.shader = WATER
    ground.material = water

    spots = _ent["spots"]
    _build_objects()
    _build_entities()
    map_texture = MapView.build_texture(self)
    $Ambience.setup(self, _ent["gates"])
    _spawn_critters()
    _build_kabus()

    camera.limit_right = terrain.width * TILE
    camera.limit_bottom = terrain.height * TILE

    hud = Hud.new()
    hud.world = self
    $HUD.add_child(hud)
    $HUD.move_child(hud, 0)
    # the mini map takes the Pau button's corner (with on-screen controls it
    # moves up: the thumbs own the bottom corners)
    minimap = Minimap.new()
    minimap.world = self
    $HUD.add_child(minimap)
    _place_minimap()
    Game.settings_changed.connect(_place_minimap)
    minimap.open_requested.connect(open_map)
    var arrow := ObjectiveArrow.new()
    arrow.world = self
    $HUD.add_child(arrow)
    $HUD.move_child(arrow, 1)  # under the HUD panels and the mini map
    prompt.pressed.connect(func(): _interact(player.facing_cell()))
    prompt.theme = Game.theme

    player.world = self
    player.step_finished.connect(_on_step_finished)
    player.step_finished.connect(_footstep)
    player.step_started.connect(func(_a, _b, _c): _last_move = Time.get_ticks_msec() / 1000.0)
    player.bumped.connect(func(_c): Sfx.play_varied("bump", -4.0))
    pau.follow(player)
    var spawn := Vector2i(_ent["spawn"][0], _ent["spawn"][1])
    if GameState.spawn_cell != null:
        spawn = GameState.spawn_cell
    elif Game.save.get("map_pos", []).size() == 2:
        spawn = Vector2i(Game.save["map_pos"][0], Game.save["map_pos"][1])
    GameState.spawn_cell = null
    if not is_walkable(spawn):  # an old save inside something built since (arena yards)
        spawn = _nearest_walkable(spawn)
    player.place_at(spawn, GameState.spawn_facing)
    pau.place_at(_free_neighbour(spawn))
    camera.reset_smoothing()

    add_child(runner)
    runner.setup(self)
    _capture_map()
    if GameState.flags.get("just_crossed", false):
        GameState.flags["just_crossed"] = false
        _arrive_lembah.call_deferred()
    _on_step_finished(spawn)
    _controls_tip()


## once, on a phone: Pau points at Tetapan > Kawalan
func _controls_tip() -> void:
    if not DisplayServer.is_touchscreen_available() or "tip_kawalan" in Game.save["seen"]:
        return
    await get_tree().create_timer(4.0).timeout
    if busy or runner.busy or Game.is_locked():
        return
    Game.save["seen"].append("tip_kawalan")
    Main.pau_say("Susah nak gerak? Tukar kawalan dalam Tetapan > Kawalan. Ada joystick pun!", 8.0)


## bottom-right normally; under the player card with on-screen controls, or
```

```
## under the menu buttons for left-handed players (the pad is on the right)
func _place_minimap() -> void:
    if not Game.controls_on_screen():
        minimap.position = Vector2(1280 - 16 - minimap.size.x, 720 - 16 - minimap.size.y)
    elif Game.settings.get("left_handed", false):
        minimap.position = Vector2(16, 166)
    else:
        minimap.position = Vector2(1280 - 16 - minimap.size.x, 150)


func _process(_delta: float) -> void:
    # mini map in the corner while exploring; the Pau button comes back for
    # puzzle hints (puzzle_base.gd talks to it)
    var in_puzzle := Game.active_puzzle != null
    Main.pau_button.visible = in_puzzle
    minimap.visible = hud.visible and not in_puzzle
    if _pau_press >= 0.0 and Time.get_ticks_msec() / 1000.0 - _pau_press >= HOLD_TIME:
        _pau_press = -1.0
        if not (busy or runner.busy or Game.is_locked()):
            pau.hop()
            Main.open_calm()
    if emote.visible:  # keep the "!" / "z z" bubble above Alif's head on screen
        var screen := get_viewport().get_canvas_transform() * (player.global_position + Vector2(0, -38))
        emote.position = screen - emote.size / 2.0
    _update_prompt()
    # the melody layer breathes with you: up while exploring, down when still
    MusicDirector.set_melody(Time.get_ticks_msec() / 1000.0 - _last_move < 6.0)


# --- building ----------------------------------------------------------
func _build_objects() -> void:
    var rows := FileAccess.get_file_as_string("res://data/world/objects.txt").split("\n", false)
    var count := {}  ## char -> how many placed so far (for "order" variants)
    for y in rows.size():
        var row: String = rows[y]
        for x in row.length():
            var ch := row[x]
            if OBJECTS.has(ch):
                _spawn_object(ch, Vector2i(x, y), count.get(ch, 0))
                count[ch] = count.get(ch, 0) + 1


## Texture name for an object. Variants go in turn ("order": houses, stalls)
## or by a stable hash of the cell (trees, flowers), so the island looks the
## same every run.
static func object_texture(def: Dictionary, anchor: Vector2i, nth: int) -> String:
    var tex = def["tex"]
    if tex is String:
        return tex
    var i: int = nth if def.get("order", false) else absi((anchor.x * 73856093) ^ (anchor.y * 19349663)) % 1009
    return tex[i % tex.size()]


func _spawn_object(ch: String, anchor: Vector2i, nth := 0) -> void:
    var def: Dictionary = OBJECTS[ch]
    var foot: Vector2i = def["foot"]
    var path := "res://assets/objects/%s.png" % object_texture(def, anchor, nth)
    if not ResourceLoader.exists(path) and def.has("fallback"):  # new art not dropped in yet
        path = "res://assets/objects/%s.png" % def["fallback"]
    var tex: Texture2D = load(path)
    var s := Sprite2D.new()
    s.texture = tex
    s.offset = CharacterFrames.visible_bottom_offset(tex)
    s.position = Vector2((anchor.x + foot.x / 2.0) * TILE, (anchor.y + 1) * TILE)  # bottom-centre
    if def.has("sway"):
        var m := ShaderMaterial.new()
        m.shader = SWAY
        m.set_shader_parameter("strength", def["sway"])
        s.material = m
    if def.get("flat", false):
        decor.add_child(s)
    else:
        objects.add_child(s)
    if ch == "H":
        houses.append(s)
    if ch == "L":
        gate_sprites["L%d_%d" % [anchor.x, anchor.y]] = s
    if KINGDOM.has(ch):
        gate_sprites[KINGDOM[ch]] = s
    for id in spots:
        if Vector2i(spots[id]["pos"][0], spots[id]["pos"][1]) == anchor:
            spot_sprites[id] = s

    var open: Array = def.get("open", [])
    for dy in foot.y:
```

```
        for dx in foot.x:
            var c := Vector2i(anchor.x + dx, anchor.y - dy)
            if def.get("walk", false):
                walk_over[c] = true
            elif not def.get("flat", false) and not (dx in open):
                blocked[c] = true
            if def.has("talk"):
                interact[c] = {"type": "talk", "id": def["talk"]}


func _build_entities() -> void:
    for s in _ent["signs"]:
        var size: Array = s.get("size", [1, 1])
        for dy in size[1]:
            for dx in size[0]:
                interact[Vector2i(s["pos"][0] + dx, s["pos"][1] - dy)] = {"type": "talk", "id": s["id"]}
    for g in _ent["gates"]:
        var ch := int(g["chapter"])
        # a chapter gate opens once the chapter before it is finished
        var open: bool = g["open"] or (not Game.chapter_zones(ch).is_empty() and Game.chapter_done(ch - 1))
        g["open"] = open
        var id := "gate_open_%d" % ch if open else "gate_locked"
        for dx in 4:
            interact[Vector2i(g["pos"][0] + dx, g["pos"][1])] = {"type": "talk", "id": id}
        var s: Sprite2D = gate_sprites.get("L%d_%d" % [int(g["pos"][0]), int(g["pos"][1])])
        if open and s:  # the misty gate becomes the portal: walk through its middle
            s.texture = load("res://assets/objects/pintu_kabus.png")
            s.offset = CharacterFrames.visible_bottom_offset(s.texture)
            for dx in [1, 2]:
                blocked.erase(Vector2i(int(g["pos"][0]) + dx, int(g["pos"][1])))
    for pt in _ent.get("portals", []):
        if Game.chapter_zones(int(pt["chapter"])).is_empty() or not Game.chapter_done(int(pt["chapter"]) - 1):
            continue
        for c in pt["cells"]:
            portals[Vector2i(c[0], c[1])] = pt
    for id in spots:
        var sp: Dictionary = spots[id]
        for dy in sp["size"][1]:
            for dx in sp["size"][0]:
                interact[Vector2i(sp["pos"][0] + dx, sp["pos"][1] - dy)] = {"type": "spot", "id": id}
    for n in _ent["npcs"]:
        var npc := Npc.new()
        var at := Vector2i(n["pos"][0], n["pos"][1])
        npc.setup(n["id"], at, n["facing"])
        objects.add_child(npc)
        if n.get("wander", true) and not blocked.has(at):
            npc.enable_wander(self, int(n.get("radius", 2)))
        npcs[at] = npc
        blocked[at] = true
        interact[at] = {"type": "npc", "id": n["id"]}
    for a in _ent.get("arenas", []):
        _dummies.merge(ArenaYard.build(self, a))
    areas = _ent["areas"]
    _build_misi_signs()
    _spawn_ujian()
    _spawn_gabenor()


## Misi 2.8: Gabenor Sekatan 1941 guards the Menara until he is beaten.
const GABENOR_CELL := Vector2i(129, 30)
var _gabenor: Npc


func _spawn_gabenor() -> void:
    if not Game.zone_unlocked("2_8") or Game.zone_state("2_8")["steps"].has("gabenor"):
        return
    if not is_walkable(GABENOR_CELL):
        return
    _gabenor = Npc.new()
    _gabenor.setup("gabenor", GABENOR_CELL, "south")
    objects.add_child(_gabenor)
    npcs[GABENOR_CELL] = _gabenor
    blocked[GABENOR_CELL] = true
    interact[GABENOR_CELL] = {"type": "npc", "id": "gabenor"}


## Numbered mission signposts in Lembah ("2.1" ... "2.8"): the number is
## written on the board; talking to one says where that mission stands.
func _build_misi_signs() -> void:
    for m in _ent.get("misi_signs", []):
        var at := Vector2i(int(m["pos"][0]), int(m["pos"][1]))
        var lbl := Label.new()
        lbl.text = str(m["no"])
        lbl.theme = Game.theme
        lbl.add_theme_font_size_override("font_size", 40)
```

```
        lbl.add_theme_color_override("font_color", Color("4a2c14"))
        lbl.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
        lbl.size = Vector2(120, 52)
        lbl.scale = Vector2(0.2, 0.2)   # drawn big, shown small: stays sharp under the zoomed camera
        lbl.position = Vector2((at.x + 1) * TILE - 12, (at.y + 1) * TILE - 27)
        lbl.z_index = 5
        objects.add_child(lbl)
        for dx in 2:
            interact[at + Vector2i(dx, 0)] = {"type": "talk", "id": "misi_" + str(m["no"])}


## Ujian Bab: once a chapter's missions are done, its boss waits on the map
## (data/ujian.json) until the final test is passed.
func _spawn_ujian() -> void:
    for n in range(1, 11):
        if not Game.ujian_pending(n):
            continue
        var cfg := Game.ujian_def(n)
        var at := Vector2i(int(cfg["pos"][0]), int(cfg["pos"][1]))
        if not is_walkable(at):
            for r in range(1, 4):
                for d in [Vector2i(r, 0), Vector2i(-r, 0), Vector2i(0, r), Vector2i(0, -r)]:
                    if is_walkable(at + d):
                        at += d
                        break
                if is_walkable(at):
                    break
        var npc := Npc.new()
        npc.setup(str(cfg["npc"]), at, "south")
        objects.add_child(npc)
        npcs[at] = npc
        blocked[at] = true
        interact[at] = {"type": "ujian", "id": str(n)}
        _ujian_cells[n] = at
        var m := ZoneRunner.Marker.new()
        m.position = Player.cell_to_pos(at) + Vector2(0, -38)
        objects.add_child(m)


## Photographs the whole island once (every tile, tree, house and boat) for the
## mini map and the full map. People, Pau and quest marks are left out: the
## maps draw them as live icons. Headless runs keep the simple 1 px map.
func _capture_map() -> void:
    if DisplayServer.get_name() == "headless":
        return
    var sub := SubViewport.new()
    sub.size = Vector2i(terrain.width * TILE, terrain.height * TILE)
    sub.world_2d = get_viewport().world_2d  # same world, own camera
    sub.render_target_update_mode = SubViewport.UPDATE_ONCE
    sub.canvas_item_default_texture_filter = Viewport.DEFAULT_CANVAS_ITEM_TEXTURE_FILTER_NEAREST
    sub.canvas_cull_mask = 1  # characters live on layer 2 too: this camera skips them
    for n in [player, pau, $Ambience] + npcs.values() + runner.markers.values() + critters + ([_fog] if _fog else []):
        _to_layer2(n)
    var cam := Camera2D.new()
    cam.anchor_mode = Camera2D.ANCHOR_MODE_FIXED_TOP_LEFT
    sub.add_child(cam)
    add_child(sub)
    await RenderingServer.frame_post_draw
    await RenderingServer.frame_post_draw
    var img := sub.get_texture().get_image()
    sub.queue_free()
    if img == null or img.is_empty():
        return
    map_sea = img.get_pixel(2, 2)  # open sea in the corner
    img.generate_mipmaps()
    map_texture = ImageTexture.create_from_image(img)


## Kabus Lupa: every mission place starts grey and misty, on both islands, and
## gets its colour back when its mission is done (the town comes back to
## life). One fog over the whole world; the mini map and full map draw the
## same rects (fog_rects()).
const KABUS_AREAS := {"gerbang": "1_1", "pelabuhan": "1_2", "melaka": "1_3", "siguntang": "1_4"}
var _fog: ColorRect
var _fog_rects := {}   ## zid -> Vector4 (world px) still under the fog


func _build_kabus() -> void:
    for a in areas:
        var zid: String = str(a.get("zone", KABUS_AREAS.get(str(a["id"]), "")))
        if zid != "" and not Game.zone_done(zid) and not _fog_rects.has(zid):
            _fog_rects[zid] = _area_px(a)
    if _fog_rects.is_empty():
        return
    _fog = ColorRect.new()
```

```
    _fog.size = Vector2(terrain.width, terrain.height) * TILE
    _fog.mouse_filter = Control.MOUSE_FILTER_IGNORE
    _fog.z_index = 19   # under the "!" quest markers (20)
    var m := ShaderMaterial.new()
    m.shader = KABUS
    m.set_shader_parameter("origin", Vector2.ZERO)
    m.set_shader_parameter("size", _fog.size)
    _fog.material = m
    add_child(_fog)
    _push_fog()


## the fogged places, in world pixels (for the maps)
func fog_rects() -> Array:
    return _fog_rects.values()


func _area_px(a: Dictionary) -> Vector4:
    var r: Array = a["rect"]
    return Vector4(r[0] * TILE, r[1] * TILE, (r[2] - r[0] + 1) * TILE, (r[3] - r[1] + 1) * TILE)


func _push_fog() -> void:
    var arr: Array = _fog_rects.values().slice(0, 16)
    var count := arr.size()
    while arr.size() < 16:
        arr.append(Vector4.ZERO)
    _fog.material.set_shader_parameter("clear_rects", arr)
    _fog.material.set_shader_parameter("clear_count", count)


## the fog over a finished place shrinks into its centre and the colour floods back
func _clear_kabus(zid: String) -> void:
    if _fog == null or not _fog_rects.has(zid):
        return
    var full: Vector4 = _fog_rects[zid]
    var centre := Vector2(full.x + full.z / 2.0, full.y + full.w / 2.0)
    Sfx.play("sparkle")
    var t := create_tween()
    t.tween_method(func(v: float):
        _fog_rects[zid] = Vector4(centre.x - full.z * v / 2.0, centre.y - full.w * v / 2.0, full.z * v, full.w * v)
        _push_fog(), 1.0, 0.0, 2.2).set_trans(Tween.TRANS_SINE)
    await t.finished
    _fog_rects.erase(zid)
    _push_fog()


func _to_layer2(n: Node) -> void:
    if n is CanvasItem:
        (n as CanvasItem).visibility_layer = 2
    for c in n.get_children():
        _to_layer2(c)


func entities() -> Dictionary:
    return _ent


## Chickens peck around the house yards, ducks paddle where the water is deep
## (the spring pond and the wide river). Same places every run.
func _spawn_critters() -> void:
    var rng := RandomNumberGenerator.new()
    rng.seed = 11
    for h in houses:
        var base := Vector2i((h.position / TILE).floor())
        for i in 2:
            for tries in 12:
                var c := base + Vector2i(rng.randi_range(-4, 4), rng.randi_range(0, 3))
                if is_walkable(c) and not interact.has(c):
                    _add_critter("ayam", c)
                    break
    var deep: Array[Vector2i] = []
    for y in terrain.height:
        for x in terrain.width:
            var c := Vector2i(x, y)
            if terrain.terrain_at(c) != TerrainBuilder.RIVER or walk_over.has(c):
                continue
            var all_water := true
            for d in [Vector2i(-1, 0), Vector2i(1, 0), Vector2i(0, -1), Vector2i(0, 1)]:
                if terrain.terrain_at(c + d) != TerrainBuilder.RIVER:
                    all_water = false
            if all_water:
                deep.append(c)
    var placed: Array[Vector2i] = []
    for i in deep.size():
```

```
        var c: Vector2i = deep[rng.randi_range(0, deep.size() - 1)]
        if placed.all(func(p): return (p - c).length() > 5.0):
            placed.append(c)
            _add_critter("itik", c)
        if placed.size() >= 4:
            break


func _add_critter(kind: String, c: Vector2i) -> void:
    var k := Critter.new()
    k.setup(self, kind, c)
    objects.add_child(k)
    critters.append(k)


# --- ZoneRunner host API -----------------------------------------------------
func spot_ids() -> Array:
    var ids := spots.keys()
    for n in _ent["npcs"]:
        ids.append(n["id"])
    return ids


# --- wandering NPCs (Npc.enable_wander) --------------------------------------
## NPCs only stroll while nothing is happening and Alif isn't close by, so the
## one you're walking up to (or talking to) stays put.
func npc_may_wander(npc: Npc) -> bool:
    if busy or runner.busy or Game.is_locked():
        return false
    var d := npc.cell - player.cell
    return absi(d.x) + absi(d.y) > 4


func npc_cell_free(c: Vector2i) -> bool:
    if not is_walkable(c) or c == player.cell or c == pau.cell:
        return false
    # keep doors, signs, spots and portals clear
    return not interact.has(c) and not portals.has(c)


func move_npc(npc: Npc, to: Vector2i) -> void:
    var from := npc.cell
    var it: Dictionary = interact.get(from, {"type": "npc", "id": npc.id})
    npcs.erase(from)
    blocked.erase(from)
    interact.erase(from)
    npcs[to] = npc
    blocked[to] = true
    interact[to] = it


func npc_node(id: String) -> Npc:
    for at in npcs:
        if npcs[at].id == id:
            return npcs[at]
    return null


func marker_pos(id: String) -> Vector2:
    if spots.has(id):
        var sp: Dictionary = spots[id]
        var s: Sprite2D = spot_sprites.get(id)
        var h := 40.0
        if s:
            h = s.texture.get_image().get_used_rect().size.y + 4
        return Vector2((sp["pos"][0] + sp["size"][0] / 2.0) * TILE, (sp["pos"][1] + 1) * TILE - h)
    for at in npcs:
        if npcs[at].id == id:
            return Player.cell_to_pos(at) + Vector2(0, -34)
    return Vector2.ZERO


func return_cell() -> Vector2i:
    return player.cell


# --- navigation arrow (anak panah penunjuk) ----------------------------------
## Where Alif should go next, for the arrows on the maps and at the screen
## edge: {"tile": Vector2 (tile units, centre), "name": String}, or {} when
## there is nothing to do. In order: the current mission's nearest "!" (a step
## may set "target": [x, y] to point somewhere else), a waiting Ujian Bab boss,
## then the next mission's place (zone "target", else its map legend spot).
## A target on the other island points at the Pintu Kabus instead.
func objective() -> Dictionary:
    if _goal_frame != Engine.get_process_frames():
```

```
        _goal_frame = Engine.get_process_frames()
        _goal = _find_objective()
    return _goal


func _find_objective() -> Dictionary:
    var out := {}
    for zid in Game.ZONES:
        var best := Vector2(-1, -1)
        for id in runner.markers:
            if not runner.markers[id].visible:
                continue
            var f: Dictionary = runner.step_at(id)
            if f.is_empty() or f["zid"] != zid:
                continue
            var t: Array = f["step"].get("target", [])
            var tile := Vector2(t[0], t[1]) + Vector2(0.5, 0.5) if t.size() == 2 else _spot_tile(id)
            if tile.x >= 0 and (best.x < 0 or tile.distance_to(Vector2(player.cell)) < best.distance_to(Vector2(player.cell))):
                best = tile
        if best.x >= 0:
            out = {"tile": best, "name": _misi_name(zid)}
            break
    if out.is_empty():
        for n in _ujian_cells:
            out = {"tile": Vector2(_ujian_cells[n]) + Vector2(0.5, 0.5), "name": tr("Ujian Bab %d") % n}
            break
    if out.is_empty():
        var nz := Game.next_zone()
        if nz != "" and Game.zone_unlocked(nz):
            var t: Array = Game.zone_data(nz).get("target", [])
            if t.size() == 2:
                out = {"tile": Vector2(t[0], t[1]) + Vector2(0.5, 0.5), "name": _misi_name(nz)}
            for p in MapView.legend()["places"]:
                if out.is_empty() and str(p.get("zone", "")) == nz:
                    out = {"tile": Vector2(p["at"][0], p["at"][1]) + Vector2(0.5, 0.5), "name": _misi_name(nz)}
    if out.is_empty():
        return out
    # the other island: first the Pintu Kabus on this side
    var here_lembah := player.cell.x >= LEMBAH_X
    if (out["tile"].x >= LEMBAH_X) != here_lembah:
        var door := Vector2(-1, -1)
        for c in portals:
            if (c.x >= LEMBAH_X) == here_lembah and (door.x < 0 or Vector2(c).distance_to(Vector2(player.cell)) < door.distance_to(Vector2(player.cell))):
                door = Vector2(c) + Vector2(0.5, 0.5)
        if door.x >= 0:
            out = {"tile": door, "name": tr("Pintu Kabus"), "then": out["name"]}
    return out


func _misi_name(zid: String) -> String:
    return tr("Misi %s") % zid.replace("_", ".")


## centre of a spot (object) or an NPC, in tiles
func _spot_tile(id: String) -> Vector2:
    if spots.has(id):
        var sp: Dictionary = spots[id]
        return Vector2(sp["pos"][0] + sp["size"][0] / 2.0, sp["pos"][1] + 0.5)
    var n := npc_node(id)
    return Vector2(n.cell) + Vector2(0.5, 0.5) if n else Vector2(-1, -1)


func apply_after(zid: String, s: Dictionary, animate: bool) -> void:
    var pid: String = s["puzzle"]
    var at: String = s["at"]
    if pid == "susun_gerbang":
        _arrange_gates(animate)
    elif at == "tiang":
        _swap(spot_sprites.get("tiang"), "tiang_berdiri", animate)
    elif at.begins_with("p_"):
        _swap(spot_sprites.get(at), "panji_naik", animate)
    if zid == "1_4":
        _update_tree(animate)
    if pid == "gabenor" and _gabenor:  # beaten: he fades into the mist
        var g := _gabenor
        _gabenor = null
        npcs.erase(GABENOR_CELL)
        blocked.erase(GABENOR_CELL)
        interact.erase(GABENOR_CELL)
        if animate:
            var t := create_tween()
            t.tween_property(g, "modulate:a", 0.0, 1.2)
            await t.finished
```

```
        g.queue_free()


func on_zone_complete(zid: String) -> void:
    if _fog_rects.has(zid):
        await _clear_kabus(zid)
    if zid == "1_2":  # the envoy ship sails in and docks
        var ship: Sprite2D = spot_sprites.get("kapal")
        if ship:
            var t := create_tween()
            t.tween_property(ship, "position", ship.position + Vector2(-28, -36), 2.5).set_trans(Tween.TRANS_SINE)
            Sfx.play("whoosh", -6.0)
            await t.finished


func _swap(s: Sprite2D, tex_name: String, animate: bool) -> void:
    if s == null:
        return
    var tex: Texture2D = load("res://assets/objects/%s.png" % tex_name)
    s.texture = tex
    s.offset = CharacterFrames.visible_bottom_offset(tex)
    if animate:
        Sfx.play("sparkle")
        s.scale = Vector2(1.0, 0.6)
        var t := create_tween()
        t.tween_property(s, "scale", Vector2.ONE, 0.45).set_trans(Tween.TRANS_BACK).set_ease(Tween.EASE_OUT)
        t.parallel().tween_property(s, "modulate", Color(1.6, 1.5, 0.9), 0.2)
        t.tween_property(s, "modulate", Color.WHITE, 0.6)


## Misi 1.1: the five kingdom gates glide into time order and a golden
## thread of heritage runs through them.
func _arrange_gates(animate: bool) -> void:
    var kg: Dictionary = _ent["kingdom_gates"]
    var pts := PackedVector2Array()
    for i in kg["order"].size():
        var s: Sprite2D = gate_sprites.get(kg["order"][i])
        if s == null:
            continue
        var to := Vector2((kg["slots"][i] + 1.5) * TILE, (kg["row"] + 1) * TILE)
        pts.append(to + Vector2(0, -30))
        if animate:
            create_tween().tween_property(s, "position", to, 1.2).set_trans(Tween.TRANS_SINE).set_delay(i * 0.15)
        else:
            s.position = to
    _thread = Line2D.new()
    _thread.points = pts
    _thread.width = 3
    _thread.default_color = Color(1, 0.85, 0.3)
    _thread.z_index = 15
    var glow := Line2D.new()
    glow.points = pts
    glow.width = 8
    glow.default_color = Color(1, 0.8, 0.3, 0.3)
    _thread.add_child(glow)
    add_child(_thread)
    if animate:
        Sfx.play("chime")
        _thread.modulate.a = 0.0
        var t := create_tween()
        t.tween_interval(1.6)
        t.tween_property(_thread, "modulate:a", 1.0, 1.0)


## Misi 1.4: the banyan (pohon = raja, akar = rakyat) wilts in the storm and
## grows healthy again as the Waadat is understood.
func _update_tree(animate: bool) -> void:
    var tree: Sprite2D = spot_sprites.get("pokok")
    if tree == null:
        return
    var stage := 0
    for s in Game.zone_data("1_4").get("steps", []):
        if Game.zone_state("1_4")["steps"].has(s["puzzle"]) and s.has("tree"):
            stage = maxi(stage, int(s["tree"]))
    if Game.zone_done("1_4"):
        stage = 2
    var col: Color = [Color(0.72, 0.62, 0.42), Color(0.88, 0.85, 0.66), Color.WHITE][stage]
    if animate:
        var t := create_tween()
        t.tween_property(tree, "modulate", Color(1.5, 1.4, 0.8), 0.3)
        t.tween_property(tree, "modulate", col, 0.9)
        Sfx.play("sparkle")
    else:
        tree.modulate = col
    if tree.material:
```

```
        tree.material.set_shader_parameter("strength", [0.25, 0.5, 0.8][stage])


# --- queries used by Player ------------------------------------------------
func is_walkable(c: Vector2i) -> bool:
    if blocked.has(c):
        return false
    if walk_over.has(c):
        return true
    return not terrain.is_water(c)


func _free_neighbour(c: Vector2i) -> Vector2i:
    for d in [Vector2i(-1, 0), Vector2i(1, 0), Vector2i(0, 1), Vector2i(0, -1)]:
        if is_walkable(c + d):
            return c + d
    return c


# --- interaction -----------------------------------------------------------
func _input_blocked() -> bool:
    return busy or runner.busy or Game.is_locked() or player.moving


func _unhandled_input(event: InputEvent) -> void:
    if busy or runner.busy or Game.is_locked():
        return
    if event.is_action_pressed("interact") and not player.moving:
        get_viewport().set_input_as_handled()
        _interact(player.facing_cell())
    elif event.is_action_pressed("menu"):
        get_viewport().set_input_as_handled()
        Main.open_settings()
    elif event.is_action_pressed("map"):
        get_viewport().set_input_as_handled()
        open_map()
    elif event.is_action_pressed("pau"):
        get_viewport().set_input_as_handled()
        _talk_to_pau()
    elif event is InputEventMouseButton and event.button_index == MOUSE_BUTTON_LEFT:
        get_viewport().set_input_as_handled()
        if $TouchControls.owns(event.position):
            return  # a thumb on the D-pad / joystick, not a tap on the map
        var cell :=Vector2i((get_global_mouse_position() / TILE).floor())
        var on_pau := cell == pau.cell or cell == pau.cell + Vector2i(0, -1)
        if event.pressed and on_pau:
            _pau_press = Time.get_ticks_msec() / 1000.0  # tap = talk, hold = calm
        elif not event.pressed and _pau_press >= 0.0:
            _pau_press = -1.0
            _talk_to_pau()
        elif event.pressed:
            _tap(cell)


func open_map() -> void:
    if busy or runner.busy or Game.is_locked():
        return
    Sfx.play("select", -6.0)
    Main.open_map(self)


## tap-to-walk (phones): path to the tapped tile, or next to a tapped
## person / object and then talk to it
func _tap(cell: Vector2i) -> void:
    var target := cell
    if interact.has(cell + Vector2i(0, 1)) and not interact.has(cell):
        target = cell + Vector2i(0, 1)  # tapped the tall part of an object
    if interact.has(target) and not is_walkable(target):
        var path := Pathfind.to_neighbour(self, player.cell, target)
        if path.is_empty() and (player.cell - target).length() <= 1.01:
            player.face_cell(target)
            _interact(target)
        elif not path.is_empty():
            player.walk_path(path, func():
                player.face_cell(target)
                _interact(target))
    elif is_walkable(cell):
        player.walk_path(Pathfind.find(self, player.cell, cell))


func _interact(c: Vector2i) -> void:
    if _input_blocked():
        return
    if c == pau.cell:
        _talk_to_pau()
```

```
        return
    if not interact.has(c):
        return
    var it: Dictionary = interact[c]
    var id: String = it["id"]
    if it["type"] == "ujian":
        busy = true
        npcs[c].face_towards(player.cell)
        var cfg := Game.ujian_def(int(id))
        await Game.say(str(cfg.get("npc_talk", "")))
        _save_pos()
        Game.goto("ujian", {"chapter": int(id)})
        return
    if it["type"] == "npc":
        npcs[c].face_towards(player.cell)
    busy = true
    var handled := false
    if runner.owns(id):
        handled = await runner.handle(id)
    if not handled:
        await _talk_or_act(id)
    busy = false


func _talk_or_act(id: String) -> void:
    if id.begins_with("misi_"):
        var zid := id.substr(5).replace(".", "_")
        var z := Game.zone_data(zid)
        var state := "Dah selesai. Syabas!" if Game.zone_done(zid) else ("Tanda seru (!) tunggu kat sini." if Game.zone_unlocked(zid) else "Belum dibuka. Habiskan Misi %s dulu." % Game.prev_zone(zid).replace("_", "."))
        await Game.say([{"s": "papan", "t": "Misi %s: %s. %s" % [zid.replace("_", "."), z.get("title", ""), state]}])
        return
    if id == "gate_open_2" or id == "portal_balik":
        var g: Vector2i = player.cell
        for c in portals:
            if (c - g).length() <= 3.0:
                if id == "portal_balik":
                    await Game.say("bab2_back")
                _cross_portal(portals[c])
                return
    if ROOMS.has(id) and (id != "jelutong_press" or Game.zone_done("2_6")):
        var room: Array = ROOMS[id]
        await Game.say([{"s": "pau", "t": room[3]}])
        busy = true
        _save_pos()
        Sfx.play("step_wood", -2.0)
        GameState.go_to(room[0], Color.BLACK, tr(room[1]), tr(room[2]))
        return
    if id == "istana":
        if not Game.zone_unlocked("1_3"):
            await Game.say([{"s": "pau", "t": "Pintu balai masih tertutup. Habiskan Pelabuhan Melaka dulu."}])
            return
        await Game.say(Dialogue.lines("istana"))
        busy = true
        _save_pos()
        GameState.go_to("res://scenes/interior/balai.tscn", Color.BLACK, tr("BALAI PENGHADAPAN"), tr("Istana Melaka, abad ke-15"))
        return
    var key := id
    if id.begins_with("p_"):
        key = "panji"
    if Dialogue.has(key):
        await Game.say(Dialogue.lines(key))
    elif Game.story.has(key):
        await Game.say(key)
    match id:
        "monument":  # Peta Jejak: the Arka & Zon screen (Tingkatan / Bab)
            _leave_to("")
            Game.goto("arka")
        "arena", "arena_lembah":
            GameState.spawn_cell = player.cell
            Game.goto("lobby")
        "pemain_arena":
            if await Main.ask(tr("Masuk Arena dan cari lawan?"), "Jom!", "Nanti dulu"):
                GameState.spawn_cell = player.cell
                Game.goto("lobby")
        "papan_markah":
            Game.main.open_papan()
        "patung_latihan":
            var near: Array = _dummies.keys()
            near.sort_custom(func(x, y): return (x - player.cell).length() < (y - player.cell).length())
            if not near.is_empty():
                ArenaYard.wobble(_dummies[near[0]])
            Sfx.play("bump", -4.0)
            await _jurus_ingatan(player.cell.x < LEMBAH_X)
```

```
        "guru_silat":
            await _jurus_ingatan()


## Arena Ulang Kaji side task: Guru Jalal's Jurus Ingatan, three revision
## questions from the missions already finished (a reward the first time,
## then for practice). Opens once the first Serpihan is found. In Lembah
## (no Guru Jalal there) Pau has the last word.
func _jurus_ingatan(guru := true) -> void:
    var done_z: Array = Game.ZONES.filter(func(z): return Game.zone_done(z))
    if done_z.is_empty():
        await Game.say(Dialogue.lines("silat_jurus_belum"))
        return
    var first: bool = not Game.save["seen"].has("jurus_ingatan")
    var q := tr("Belajar Jurus Ingatan dengan Guru Jalal? Tiga soalan ulang kaji, tiada penalti.") if first         else tr("Ulang Jurus Ingatan? Untuk ulang kaji sahaja.")
    if not await Main.ask(q, "Jom!", "Nanti dulu"):
        return
    var qs: Array = []
    for n in range(1, Game.current_chapter() + 1):
        qs.append_array(Game.ujian_questions(n, 99, done_z))
    qs.shuffle()
    await runner.run_puzzle({
        "type": "objektif", "title": tr("Jurus Ingatan"),
        "instruction": tr("Jawab macam buka langkah: tenang, satu-satu."),
        "hint": "Ingat balik apa yang kau belajar dalam misi tadi.",
        "done_text": tr("Langkah demi langkah. Itulah Jurus Ingatan!"),
        "questions": qs.slice(0, 3),
    }, "silat_jurus")
    if first:
        Game.save["seen"].append("jurus_ingatan")
        if Game.add_reward(30, 15):
            Game.toast(tr("NAIK TAHAP! Tahap %d") % int(Game.save["level"]), Color("8fe08a"))
        Game.persist()
        Game.toast("+30 EXP   +15 Syiling")
        MusicDirector.stinger("st_item")
    await Game.say(Dialogue.lines("silat_jurus_lepas" if guru else "patung_lepas"))


## Pintu Kabus: Kampung Jejak <-> Lembah Kebangkitan (the world scene reloads
## with Alif on the other island)
func _cross_portal(pt: Dictionary) -> void:
    busy = true
    var to := Vector2i(int(pt["to"][0]), int(pt["to"][1]))
    var going_in := to.x >= LEMBAH_X
    if going_in:
        await Game.say("bab2_portal")
    Sfx.play("whoosh")
    GameState.spawn_cell = to
    GameState.spawn_facing = str(pt.get("facing", "south"))
    GameState.flags["just_crossed"] = going_in
    Game.save["map_pos"] = [to.x, to.y]
    Game.persist()
    if going_in:
        GameState.go_to(scene_file_path, Color(0.85, 0.9, 1.0), tr("LEMBAH KEBANGKITAN"), tr("Tanah Melayu, awal 1900-an hingga 1941"))
    else:
        GameState.go_to(scene_file_path, Color(0.85, 0.9, 1.0), tr("KAMPUNG JEJAK"), tr("Kembali ke Bab 1: Warisan Negara Bangsa"))


## Jalan Pintas (Beg Alif and the full map): every place of data/map_legend.json
## as {place, ok, why}. A Misi opens with its mission, Lembah once Bab 1 is
## done (like the Pintu Kabus); the mist gate itself is not a destination.
func travel_places() -> Array:
    var out: Array = []
    var lembah_open := not Game.chapter_zones(2).is_empty() and Game.chapter_done(1)
    for p in MapView.legend()["places"]:
        if str(p["id"]) == "kabus":
            continue
        var why := ""
        if int(p["at"][0]) >= LEMBAH_X and not lembah_open:
            why = tr("Selesaikan Bab 1 dulu")
        elif p.has("zone") and not Game.zone_unlocked(str(p["zone"])):
            why = tr("Selesaikan Misi %s dulu") % Game.prev_zone(str(p["zone"])).replace("_", ".")
        out.append({"place": p, "ok": why == "", "why": why})
    return out


## jump to a place from travel_places(): the world reloads with Alif on the
## nearest free tile there (crossing to Lembah plays its arrival, as the portal does)
func fast_travel(p: Dictionary) -> bool:
    if busy or runner.busy or Game.is_locked():
        return false
    var at := _nearest_walkable(Vector2i(int(p["at"][0]), int(p["at"][1])))
```

```
    busy = true
    Sfx.play("whoosh")
    GameState.spawn_cell = at
    GameState.spawn_facing = "south"
    GameState.flags["just_crossed"] = at.x >= LEMBAH_X and player.cell.x < LEMBAH_X
    Game.save["map_pos"] = [at.x, at.y]
    Game.persist()
    GameState.go_to(scene_file_path, Color(0.85, 0.9, 1.0), tr(str(p["name"])).to_upper(), tr("Jalan Pintas"))
    return true


## the walkable tile closest to c (rings outwards), away from doors and portals
func _nearest_walkable(c: Vector2i) -> Vector2i:
    for r in range(0, 8):
        for dy in range(-r, r + 1):
            for dx in range(-r, r + 1):
                var t := c + Vector2i(dx, dy)
                if maxi(absi(dx), absi(dy)) == r and is_walkable(t) and not interact.has(t) and not portals.has(t):
                    return t
    return c


func _arrive_lembah() -> void:
    busy = true
    await get_tree().create_timer(0.6).timeout
    await Game.say("bab2_arrive")
    busy = false
    _enter_area_at(player.cell)


func _enter_area_at(c: Vector2i) -> void:
    current_area = ""
    _on_step_finished(c)


func _leave_to(scene: String) -> void:
    busy = true
    Sfx.play("select")
    _save_pos()
    if scene != "":
        GameState.go_to(scene)


func _save_pos() -> void:
    Game.save["map_pos"] = [player.cell.x, player.cell.y]
    Game.persist()


func _talk_to_pau() -> void:
    if busy or Game.is_locked():
        return
    busy = true
    pau.hop()
    Sfx.play("squish", -6.0, 1.3)
    # a bit of Pau, then the hint the corner Pau button used to give
    var lines: Array = Dialogue.lines("pau_tap")
    if Game.context_hint != "":
        lines.append({"s": "pau", "t": Game.context_hint})
    await Game.say(lines)
    busy = false


func _update_prompt() -> void:
    var c := player.facing_cell()
    var label := ""
    if not _input_blocked() and not Main.is_dialogue_open():
        if c == pau.cell:
            label = "Pau"
        elif interact.has(c):
            var it: Dictionary = interact[c]
            label = _label_for(it)
    prompt.visible = label != ""
    if prompt.visible:
        prompt.text = tr("E / Ketik: ") + tr(label)
        prompt.reset_size()
        var screen := get_viewport().get_canvas_transform() * (player.global_position + Vector2(0, 14))
        prompt.position = Vector2(clampf(screen.x - prompt.size.x / 2.0, 10, 1270 - prompt.size.x), clampf(screen.y + 8
, 120, 640))


func _label_for(it: Dictionary) -> String:
    var id: String = it["id"]
    if it["type"] == "npc":
        return Game.char_name(id)
    if LABELS.has(id):
```

```
        return LABELS[id]
    if id.begins_with("misi_"):
        return "Papan Misi " + id.substr(5)
    for zid in Game.ZONES:
        for o in Game.zone_data(zid).get("objects", []):
            if str(o["id"]) == id and o.has("label"):
                return str(o["label"])
    return "Lihat"


const STEP_SOUNDS := {
    TerrainBuilder.SAND: "step_sand", TerrainBuilder.GRASS: "step_grass",
    TerrainBuilder.MEADOW: "step_grass", TerrainBuilder.PATH: "step_sand", TerrainBuilder.JALAN: "step_sand",
}


func _footstep(c: Vector2i) -> void:
    var sound: String = "step_wood" if walk_over.has(c) else STEP_SOUNDS.get(terrain.terrain_at(c), "step_grass")
    Sfx.play_varied(sound, -6.0, 0.12)


# --- areas ----------------------------------------------------------------
func _on_step_finished(c: Vector2i) -> void:
    if portals.has(c) and not busy:
        _cross_portal(portals[c])
        return
    for a in areas:
        var r: Array = a["rect"]
        if c.x >= r[0] and c.x <= r[2] and c.y >= r[1] and c.y <= r[3]:
            if a["id"] != current_area:
                current_area = a["id"]
                _enter_area(a)
            return


func _enter_area(a: Dictionary) -> void:
    var sub := ""
    var zid: String = str(a.get("zone", AREA_ZONE.get(a["id"], "")))
    if zid != "":
        sub = str(Game.zone_data(zid).get("subtopik", ""))
    hud.set_location(str(a["name"]).to_upper(), sub)
    var t := create_tween()
    t.tween_property(hud, "modulate", Color(1.4, 1.3, 0.8), 0.15)
    t.tween_property(hud, "modulate", Color.WHITE, 0.5)
    MusicDirector.play(a["id"])
    _save_pos()
    if busy:
        return
    if not Game.save["seen"].has("area_" + str(a["id"])):
        await _area_intro(a)
    if zid != "" and not busy:
        _zone_welcome(zid)


var _area_intros := {}


## First visit to a place: a big title card (name + era), then a line or two
## on why Alif and Pau are here, so arriving never feels random.
func _area_intro(a: Dictionary) -> void:
    if _area_intros.is_empty():
        _area_intros = Game.load_json("res://data/area_intro.json")
    var info: Dictionary = _area_intros.get(str(a["id"]), {})
    Game.save["seen"].append("area_" + str(a["id"]))
    Game.persist()
    if info.is_empty():
        return
    busy = true
    var title: Label = title_card.get_node("Title")
    var sub: Label = title_card.get_node("Subtitle")
    var old := [title.text, sub.text]
    var nm := str(a["name"])
    title.text = nm.substr(nm.find(": ") + 2) if nm.begins_with("Misi ") else nm
    title.add_theme_font_size_override("font_size", 84 if title.text.length() <= 22 else 64)
    sub.text = (nm.substr(0, nm.find(":")) + "  ·  " if nm.begins_with("Misi ") else "") + str(info.get("era", ""))
    Sfx.play("chime", -8.0, 0.8)
    var t := create_tween()
    t.tween_property(title_card, "modulate:a", 1.0, 0.5)
    t.tween_interval(1.6)
    t.tween_property(title_card, "modulate:a", 0.0, 0.6)
    await t.finished
    title.text = old[0]
    sub.text = old[1]
    title.add_theme_font_size_override("font_size", 84)
    var lines: Array = info.get("lines", [])
```

```
    if not lines.is_empty():
        await Game.say(lines)
    busy = false


## First visit to an open zone: the prototype's "Persiapan Misi" briefing,
## then the zone's intro conversation.
func _zone_welcome(zid: String) -> void:
    if not Game.zone_unlocked(zid):
        var prev := Game.prev_zone(zid)
        var need := ("Misi " + prev.replace("_", ".")) if prev != "" else ("Bab %d" % (Game.chapter_of(zid) - 1))
        Main.pau_say("Tempat ni untuk Misi %s. Kita kena habiskan %s dulu." % [zid.replace("_", "."), need], 5.0)
        return
    if Game.zone_done(zid) or not Game.zone_state(zid)["steps"].is_empty():
        return
    if GameState.flags.get("welcomed_" + zid, false):
        return
    GameState.flags["welcomed_" + zid] = true
    busy = true
    var prep := ZonePrep.new()
    prep.set("params", {"zone": zid})
    Main.overlay_root().add_child(prep)
    await prep.closed
    await Game.say(str(Game.zone_data(zid).get("intro", "")))
    runner.refresh()
    busy = false
```

zone_runner.gd · scripts/world/zone_runner.gd · 279 lines

```
class_name ZoneRunner
extends Node
## Runs the mission zones (data/zon_<bab>_<n>.json) inside
## the explorable world. Each step is: dialogue -> puzzle -> rewards ->
## dialogue -> a visible change in the world. Steps live at "spots" (NPC ids
## or objects), shown with bouncing "!" markers.
##
## A finished mission can be replayed any number of times for revision: talk
## to one of its spots and say yes. Zone state then gets a "replay" list of the
## steps done this time round; the world stays as it is and nothing is rewarded.
##
## The host (world.gd or interior.gd) provides:
##   spot_ids() -> Array        which step locations exist in this scene
##   marker_pos(id) -> Vector2  where to float the "!" marker
##   apply_after(zid, step, animate)  world reaction after a step
##   on_zone_complete(zid)      before the summary screen (e.g. ship docks)
##   return_cell() -> Vector2i  overworld cell to come back to after summary

const DragMatch = preload("res://scripts/puzzles/drag_match.gd")
const Board = preload("res://scripts/puzzles/board.gd")
const Choice = preload("res://scripts/puzzles/choice.gd")
const Objektif = preload("res://scripts/puzzles/objektif.gd")
const Suaikan = preload("res://scripts/puzzles/suaikan.gd")

signal changed

var host: Node
var markers := {}  ## spot id -> marker Node2D
var busy := false
var _puzzle_layer: CanvasLayer


func setup(h: Node) -> void:
    host = h
    _puzzle_layer = CanvasLayer.new()
    _puzzle_layer.layer = 15
    add_child(_puzzle_layer)
    for id in host.spot_ids():
        var m := Marker.new()
        m.position = host.marker_pos(id)
        if host.has_method("npc_node"):  # a wandering NPC takes its "!" along
            m.follow = host.npc_node(id)
        m.visible = false
        host.get_node("Objects").add_child(m)
        markers[id] = m
    # restore the world to match saved progress
    for zid in Game.ZONES:
        var done: Array = Game.zone_state(zid)["steps"]
        for s in Game.zone_data(zid).get("steps", []):
            if done.has(s["puzzle"]) and s["at"] in markers:
                host.apply_after(zid, s, false)
    refresh()


```

```
## Find the zone step that lives at this spot. {} if none.
## {"zid", "i", "step"}; the step may carry "target": [x, y] (world cell) for
## the navigation arrow when the place to walk to is not the spot itself.
func step_at(id: String) -> Dictionary:
    return _step_at(id)


func _step_at(id: String) -> Dictionary:
    for zid in Game.ZONES:
        var steps: Array = Game.zone_data(zid).get("steps", [])
        for i in steps.size():
            if str(steps[i]["at"]) == id:
                if not _done(zid, steps[i]):
                    return {"zid": zid, "i": i, "step": steps[i]}
    return {}


func owns(id: String) -> bool:
    for zid in Game.ZONES:
        for s in Game.zone_data(zid).get("steps", []):
            if str(s["at"]) == id:
                return true
    return false


func _done(zid: String, s: Dictionary) -> bool:
    var st := Game.zone_state(zid)
    if st.has("replay"):
        return st["replay"].has(s["puzzle"])
    return st["steps"].has(s["puzzle"])


static func replaying(zid: String) -> bool:
    return Game.zone_state(zid).has("replay")


## the finished (and not already replaying) mission that has a step at this spot
func _replayable_at(id: String) -> String:
    for zid in Game.ZONES:
        if not Game.zone_done(zid) or replaying(zid):
            continue
        for s in Game.zone_data(zid).get("steps", []):
            if str(s["at"]) == id:
                return zid
    return ""


func _available(zid: String, i: int) -> bool:
    if not Game.zone_unlocked(zid):
        return false
    var zone := Game.zone_data(zid)
    var steps: Array = zone.get("steps", [])
    if _done(zid, steps[i]):
        return false
    if zone.get("any_order", false):
        return true
    for j in i:
        if not _done(zid, steps[j]):
            return false
    return true


## Interact with a spot. Returns true if a zone step handled it.
func handle(id: String) -> bool:
    var found := _step_at(id)
    if found.is_empty():
        var again := _replayable_at(id)
        if again == "":
            return false
        var z := Game.zone_data(again)
        var yes: bool = await Game.main.ask(tr("Main semula Misi %s: %s?\nUntuk ulang kaji sahaja, tiada ganjaran.") % [again.replace("_", "."), z.get("title", "")], "Main semula", "Tak apa")
        if not yes:
            return false
        Game.zone_state(again)["replay"] = []
        Game.persist()
        refresh()
        found = _step_at(id)
        if found.is_empty() or not _available(again, found["i"]):
            Game.main.pau_say(tr("Jom! Ikut tanda seru (!) untuk mula semula."))
            return true
    var zid: String = found["zid"]
    if not Game.zone_unlocked(zid):
        var prev := Game.prev_zone(zid)
        if prev == "":
```

```
            Game.main.pau_say("Belum lagi. Habiskan Bab %d dulu, baru tempat ni terbuka." % (Game.chapter_of(zid) - 1))
        else:
            Game.main.pau_say("Belum lagi. Habiskan Misi %s dulu, baru tempat ni terbuka." % prev.replace("_", "."))
        return true
    if not _available(zid, found["i"]):
        Game.main.pau_say("Belum lagi. Selesaikan tugasan sebelumnya dulu, ikut tanda seru.")
        return true
    await run_step(zid, found["step"])
    return true


func run_step(zid: String, s: Dictionary) -> void:
    busy = true
    var zone := Game.zone_data(zid)
    var st := Game.zone_state(zid)
    await Game.say(str(s.get("before", "")))
    var pid: String = s["puzzle"]
    var stats := await run_puzzle(zone["puzzles"][pid], "%s_%s" % [zid, pid])
    if st.has("replay"):  # revision run: no rewards, the world is already changed
        st["replay"].append(pid)
        Game.persist()
        await Game.say(str(s.get("after", "")))
        refresh()
        busy = false
        if st["replay"].size() >= zone.get("steps", []).size():
            st.erase("replay")
            Game.persist()
            await complete(zid)
        return
    st["steps"].append(pid)
    for k in ["first_try", "retries", "hints", "total"]:
        st[k] = int(st.get(k, 0)) + int(stats.get(k, 0))
    var total := int(stats.get("total", 0))
    var ft := int(stats.get("first_try", 0))
    var exp_gain := ft * 10 + (total - ft) * 5 + 20
    var coins := total * 3 + 5
    var up := Game.add_reward(exp_gain, coins)
    Game.toast("+%d EXP   +%d Syiling" % [exp_gain, coins])
    MusicDirector.stinger("st_item")
    if up:
        Game.toast("NAIK TAHAP! Tahap %d" % int(Game.save["level"]), Color("8fe08a"))
        MusicDirector.stinger("st_levelup")
    Game.persist()
    host.apply_after(zid, s, true)
    await Game.say(str(s.get("after", "")))
    refresh()
    busy = false
    if st["steps"].size() >= zone.get("steps", []).size():
        await complete(zid)


func run_puzzle(def: Dictionary, puzzle_id: String) -> Dictionary:
    var p: Control
    match str(def.get("type", "drag_match")):
        "board": p = Board.new()
        "choice": p = Choice.new()
        "objektif": p = Objektif.new()
        "suaikan": p = Suaikan.new()
        _: p = DragMatch.new()
    p.theme = Game.theme
    p.set("def", def)
    p.set("puzzle_id", puzzle_id)
    Game.lock_input()
    Game.main.set_pau_visible(true)
    Game.main.bubble.visible = false
    MusicDirector.puzzle(true)
    _puzzle_layer.add_child(p)
    Game.active_puzzle = p
    var stats: Dictionary = await p.completed
    Game.active_puzzle = null
    MusicDirector.puzzle(false)
    Game.unlock_input()
    p.queue_free()
    return stats


func complete(zid: String) -> void:
    var zone := Game.zone_data(zid)
    var st := Game.zone_state(zid)
    var first_time := not bool(st.get("done", false))
    if first_time:
        await host.on_zone_complete(zid)
    if zone.has("outro"):
        await Game.say(str(zone["outro"]))
    if first_time and zone.has("taunt"):  # every two missions Nirbana mocks from the mist
```

```
        await Game.say(str(zone["taunt"]))
    var level_up := false
    if first_time:
        st["done"] = true
        Game.add_artifact(str(zone.get("ganjaran", {}).get("id", "")))
        level_up = Game.add_reward(100, 50)
        Game.persist()
    MusicDirector.stinger("st_zone")
    await get_tree().create_timer(1.2).timeout
    GameState.spawn_cell = host.return_cell()
    Game.goto("zone_summary", {"zone": zid, "first_time": first_time, "level_up": level_up})


func refresh() -> void:
    var hint := ""
    for id in markers:
        var f := _step_at(id)
        var on := not f.is_empty() and _available(f["zid"], f["i"])
        markers[id].visible = on
        if on and hint == "":
            hint = "Pergi ke tanda seru (!) di %s." % Game.zone_data(f["zid"])["title"]
    for n in range(1, 11):
        if hint == "" and Game.ujian_pending(n):
            hint = "Ujian Bab %d menanti di %s. Lawan %s untuk buka jalan seterusnya!" % [n, Game.ujian_def(n)["where"]
, Game.ujian_def(n)["boss"]]
    if hint == "":
        var next := Game.next_zone()
        if next == "":
            hint = "Semua bab yang dibuka sudah selesai! Buka Nota Alif untuk ulang kaji."
        else:
            hint = str(Game.zone_data(next).get("hint_go", "Ikut tanda seru (!) ke Misi %s." % next.replace("_", ".")))
    Game.context_hint = hint
    changed.emit()


## Bouncing "!" quest marker drawn in pixels (crisp at any zoom).
class Marker extends Node2D:
    var follow: Node2D
    var _t := randf() * TAU

    func _ready() -> void:
        z_index = 20
        y_sort_enabled = false

    func _process(delta: float) -> void:
        _t += delta
        if follow:
            position = follow.position + Vector2(0, -34)
        queue_redraw()

    func _draw() -> void:
        var y := -sin(_t * 4.0) * 2.0
        var o := Vector2(-3, y - 12)
        draw_rect(Rect2(o + Vector2(-1, -1), Vector2(8, 13)), Color(0.17, 0.11, 0.06))
        draw_rect(Rect2(o, Vector2(6, 11)), Color(1, 0.86, 0.3))
        draw_rect(Rect2(o + Vector2(2, 1), Vector2(2, 6)), Color(0.35, 0.2, 0.05))
        draw_rect(Rect2(o + Vector2(2, 8), Vector2(2, 2)), Color(0.35, 0.2, 0.05))
```

shaders/

kabus_lupa.gdshader · shaders/kabus_lupa.gdshader · 34 lines

```
shader_type canvas_item;
// Kabus Lupa: the forgotten past is grey. One rect over the whole world;
// fog_rects are the mission places not done yet (world pixels). Inside them
// the picture turns grey with a little light mist drifting across, with soft
// edges. The mini map and full map draw the same rects (world.fog_rects()).

uniform sampler2D screen_tex : hint_screen_texture, filter_nearest;
uniform vec2 origin;
uniform vec2 size;
uniform vec4 clear_rects[16];   // x, y, w, h of the fogged places
uniform int clear_count = 0;
uniform float softness = 48.0;

void fragment() {
    vec3 c = texture(screen_tex, SCREEN_UV).rgb;
    vec2 p = origin + UV * size;
    float k = 0.0;
    for (int i = 0; i < clear_count; i++) {
        vec4 r = clear_rects[i];
```

```
        if (r.z <= 0.0) {
            continue;
        }
        vec2 q = max(max(r.xy - p, p - (r.xy + r.zw)), vec2(0.0));
        k = max(k, 1.0 - smoothstep(0.0, softness, length(q)));   // 1 inside, fades out at the edge
    }
    float g = dot(c, vec3(0.3, 0.59, 0.11));
    float drift = 0.5 + 0.5 * sin(TIME * 0.5 + p.x * 0.02 + p.y * 0.013);
    vec3 grey = mix(vec3(g), vec3(0.8, 0.82, 0.86), 0.22 + 0.1 * drift);
    vec3 col = mix(c, grey, k * 0.62);   // grey, but the place still shows through
    // a little light mist: soft pale banks drifting slowly across
    float mist = 0.5 + 0.25 * sin(p.x * 0.011 + TIME * 0.35) + 0.25 * sin(p.y * 0.017 - TIME * 0.27 + p.x * 0.004);
    col = mix(col, vec3(0.94, 0.95, 0.98), k * (0.10 + 0.16 * mist));
    COLOR = vec4(col, 1.0);
}
```

`sway.gdshader` · shaders/sway.gdshader · 13 lines

```
shader_type canvas_item;
// Wind sway for trees and bushes: the top of the sprite moves, the base stays.
// Offset is snapped to whole pixels so the pixel art never smears.

uniform float strength = 1.5;
uniform float speed = 1.4;

void vertex() {
    float top = 1.0 - UV.y;
    float phase = MODEL_MATRIX[3].x * 0.05 + MODEL_MATRIX[3].y * 0.03;
    float gust = sin(TIME * speed + phase) + 0.4 * sin(TIME * speed * 2.3 + phase * 1.7);
    VERTEX.x += floor(gust * strength * top * top + 0.5);
}
```

`water.gdshader` · shaders/water.gdshader · 24 lines

```
shader_type canvas_item;
// Applied to the ground TileMapLayer: only bluish "water" pixels shimmer,
// with drifting light bands and occasional sparkles. Sand and grass untouched.

uniform float band_strength = 0.05;
uniform float sparkle_strength = 0.22;

varying vec2 world_pos;

void vertex() {
    world_pos = (MODEL_MATRIX * vec4(VERTEX, 0.0, 1.0)).xy;
}

void fragment() {
    vec4 c = texture(TEXTURE, UV);
    float water = step(c.r + 0.12, c.b) * step(c.r + 0.04, c.g);
    vec2 p = floor(world_pos); // stay on the pixel grid
    float w1 = sin(p.x * 0.30 + p.y * 0.18 + TIME * 1.6);
    float w2 = sin(p.y * 0.45 - p.x * 0.11 - TIME * 1.1);
    float band = w1 * w2;
    float sparkle = step(0.965, band) * step(0.5, fract(sin(dot(p, vec2(12.9898, 78.233))) * 43758.5453 + TIME * 0.4));
    c.rgb += water * (band * band_strength + sparkle * sparkle_strength);
    COLOR = c;
}
```

APPENDIX B

Game Data

All story text, dialogue, puzzles, questions and world layout are stored as data files read by the game at runtime. Files up to 4 KB are listed in full; larger files show their first 70 lines.

File	Size	Top-level content
data/area_intro.json	2.8 KB	_info, permulaan, kampung, gerbang, pelabuhan, melaka, siguntang, kabus …
data/brawl/moves.json	2.7 KB	_info, moves, weapons
data/chapters.json	1.3 KB	arcs
data/characters.json	7.1 KB	alif, pau, penjaga, syahbandar, chengho, tunperak, sapurba, demang …
data/dialogue/prologue.json	9.1 KB	pro_room, pro_obj_kalendar, pro_obj_kertas, pro_obj_telefon, pro_obj_tingkap, pro_obj_pau, pro_obj_buku, pro_read …
data/dialogue/world.json	17.6 KB	_note, sign_start, sign_sea, monument, istana, house, gate_open_1, gate_locked …
data/i18n/en.json	16.5 KB	_info, TETAPAN, Paparan, Audio, Bahasa, Akaun, Mod Mudah Baca, Fon …
data/interiors/bilik_alif.json	3.1 KB	_info, exhibits
data/interiors/jelutong.json	2.2 KB	_info, intro, npc_talk, exhibits
data/interiors/sitc.json	2.3 KB	_info, intro, npc_talk, exhibits
data/leaderboard_sample.json	2.1 KB	_info, players
data/map_legend.json	2.6 KB	_info, places, landmarks
data/nota.json	13.3 KB	title, pages
data/nota_bab2.json	6.6 KB	title, pages
data/peta_layout.json	0.8 KB	scale, flags, zones, special
data/prologue_room.json	2.1 KB	_info, chat, caption, room, paragraph, memory_size, read_words, word_every …
data/skins.json	1.0 KB	_info, skins
data/soalan/bab1.json	9.4 KB	_info, bab, questions
data/soalan/bab2.json	23.2 KB	_info, bab, questions
data/story.json	26.1 KB	_nota, peta_first, flag_locked, flag_1, flag_1_done, zone_locked, t5_locked, arena_locked …
data/ujian.json	0.9 KB	_info, 1, 2
data/world/entities.json	9.4 KB	spawn, npcs, signs, gates, areas, spots, kingdom_gates, arenas …
data/world/lembah_layout.json	4.6 KB	_info, terrain, bridges, landmarks, misi_signs, npcs, decor, areas
data/world/objects.txt	8.3 KB	
data/world/terrain.txt	8.3 KB	
data/zon_1_1.json	6.6 KB	id, no, title, subtopik, hint_go, persoalan, pemandu, ganjaran …
data/zon_1_2.json	14.2 KB	id, no, title, subtopik, hint_go, taunt, persoalan, pemandu …
data/zon_1_3.json	8.0 KB	id, no, title, subtopik, hint_go, persoalan, pemandu, ganjaran …
data/zon_1_4.json	9.9 KB	id, no, title, subtopik, hint_go, persoalan, pemandu, ganjaran …

File	Size	Top-level content
data/zon_2_1.json	6.5 KB	id, no, fakta_disemak, music, any_order, background, title, subtopik …
data/zon_2_2.json	6.6 KB	id, no, fakta_disemak, music, any_order, background, title, subtopik …
data/zon_2_3.json	6.0 KB	id, no, fakta_disemak, music, any_order, background, title, subtopik …
data/zon_2_4.json	6.8 KB	id, no, fakta_disemak, music, any_order, background, title, subtopik …
data/zon_2_5.json	4.7 KB	id, no, fakta_disemak, music, any_order, background, title, subtopik …
data/zon_2_6.json	6.2 KB	id, no, fakta_disemak, music, any_order, background, title, subtopik …
data/zon_2_7.json	8.8 KB	id, no, fakta_disemak, music, any_order, background, title, subtopik …
data/zon_2_8.json	6.8 KB	id, no, fakta_disemak, music, any_order, background, title, subtopik …

data/area_intro.json · 49 lines

```
{
  "_info": "Kad tajuk dan sebab datang, sekali sahaja pada lawatan pertama ke setiap kawasan peta. era = baris kecil di bawah nama. lines = dialog pendek (kosong = kad sahaja).",
  "permulaan": {"era": "Dunia Explo-Story", "lines": []},
  "kampung": {"era": "Dunia Explo-Story", "lines": [
    {"s": "pau", "t": "Ni Kampung Jejak. Rumah kita kat dunia ni."},
    {"s": "pau", "t": "Serpihan yang kita jumpa semua masuk beg kau. Batu bercahaya tu pula Peta Jejak."},
    {"s": "alif", "e": "confident", "t": "Serpihan pertama kat Gerbang Masa, sebelah utara. Jom."}
  ]},
  "gerbang": {"era": "Kerajaan awal Alam Melayu", "lines": [
    {"s": "alif", "t": "Serpihan pertama ada kat sini. Kita mula dari akar: kerajaan awal Alam Melayu."}
  ]},
  "pelabuhan": {"era": "Melaka, abad ke-15", "lines": [
    {"s": "alif", "t": "Serpihan kedua kat pelabuhan ni. Melaka kena buktikan dia negara bangsa yang lengkap."}
  ]},
  "melaka": {"era": "Kota Melaka, abad ke-15", "lines": [
    {"s": "alif", "t": "Kota Melaka. Serpihan ketiga ada dalam istana, kat Balai Penghadapan."}
  ]},
  "siguntang": {"era": "Kisah dalam Sulalatus Salatin", "lines": [
    {"s": "alif", "t": "Bukit Siguntang. Tempat bermulanya janji raja dan rakyat. Serpihan terakhir ada kat atas."}
  ]},
  "kabus": {"era": "Sempadan Bab 1", "lines": [
    {"s": "pau", "e": "scared", "t": "Pintu Kabus. Kat seberang sana ada bab lain..."},
    {"s": "alif", "t": "Kita habiskan Bab 1 dulu. Satu-satu."}
  ]},
  "dewan": {"era": "Tanah Melayu, awal abad ke-20", "lines": [
    {"s": "alif", "t": "Dewan Kedaulatan Rakyat. Mula-mula, kita kena faham apa maksud nasionalisme."}
  ]},
  "falsafah": {"era": "Eropah dan Amerika, 1688 hingga 1789", "lines": [
    {"s": "alif", "t": "Idea pasal hak rakyat ni datang dari jauh. Dari revolusi kat Barat."}
  ]},
  "dermaga": {"era": "Asia, akhir abad ke-19", "lines": [
    {"s": "alif", "t": "Dermaga Asia. Kapal-kapal ni bawa berita kebangkitan dari India, China dan Jepun."}
  ]},
  "serantau": {"era": "Asia Tenggara, 1890-an hingga 1930-an", "lines": [
    {"s": "alif", "t": "Jiran-jiran kita kat Asia Tenggara pun bangkit. Jom tengok macam mana."}
  ]},
  "tugu": {"era": "Tanah Melayu, abad ke-19", "lines": [
    {"s": "alif", "t": "Tugu Pahlawan. Macam mana semangat ni mula sampai ke negara kita?"}
  ]},
  "kompleks": {"era": "Tanah Melayu, 1900-an hingga 1930-an", "lines": [
    {"s": "alif", "t": "Sekolah, akhbar, buku. Inilah benda yang buat orang kita mula sedar."}
  ]},
  "sitc": {"era": "Tanjung Malim, dibuka 1922", "lines": [
    {"s": "alif", "t": "Maktab SITC. Ramai guru dan penulis hebat lahir kat sini."}
  ]},
  "menara": {"era": "Tanah Melayu, hingga 1941", "lines": [
    {"s": "alif", "t": "Menara Pentadbiran. Apa kesan semua kebangkitan ni pada negara kita?"}
  ]}
}
```

data/brawl/moves.json · 23 lines

```
{
  "_info": "Gelanggang Pesta Silat frame data. time = whole move (s); active = [start, end] as a fraction of time when it can hit; reach = [min, max] px in front (screen px, fighters are drawn 4x); kb = knockback px/s; down = knocks down; lunge = forward push px/s; radius = hits all around. Damage is multiplied by Semangat (1.0) or Lemah (0.3).",
  "moves": {
    "jab":       {"anim": "jab",       "time": 0.26, "active": [0.3, 0.65], "dmg": 4,  "reach": [-6, 78],  "kb": 60, "stun": 0.25, "sfx": "punch", "word": "PAK"},
    "jab2":      {"anim": "jab",       "time": 0.26, "active": [0.3, 0.65], "dmg": 4,  "reach": [-6, 78],  "kb": 60,
```

```
 "stun": 0.25, "sfx": "punch", "word": "PAK"},
    "straight":   {"anim": "cross",      "time": 0.36, "active": [0.35, 0.65], "dmg": 7, "reach": [-6, 86],  "kb": 260, "stun": 0.35, "sfx": "punch", "word": "DUSH"},
    "kick":       {"anim": "kick",       "time": 0.44, "active": [0.4, 0.7],  "dmg": 7,  "reach": [0, 104],  "kb": 220, "stun": 0.35, "sfx": "kick",  "word": "BAM"},
    "roundhouse": {"anim": "roundhouse", "time": 0.55, "active": [0.4, 0.75], "dmg": 12, "reach": [-10, 110], "kb": 420, "down": true, "sfx": "kick", "word": "POW!", "big": true},
    "flykick":    {"anim": "flykick",    "time": 0.5,  "active": [0.15, 0.85], "dmg": 10, "reach": [-6, 100], "kb": 380, "down": true, "lunge": 360, "air": true, "sfx": "kick", "word": "WHAM"},
    "dash":       {"anim": "cross",      "time": 0.4,  "active": [0.2, 0.6],  "dmg": 8,  "reach": [-6, 90],  "kb": 320, "stun": 0.4, "lunge": 420, "sfx": "punch", "word": "DUSH"},
    "special":    {"anim": "special",    "time": 0.85, "active": [0.15, 0.85], "dmg": 18, "radius": 140, "kb": 460, "down": true, "sfx": "kick", "word": "JURUS KILAT!", "big": true},
    "swing":      {"anim": "throw",      "time": 0.42, "active": [0.35, 0.7], "dmg": 0,  "reach": [-6, 0],   "kb": 280, "stun": 0.4, "sfx": "thwack", "word": "PAP!"},
    "throw":      {"anim": "throw",      "time": 0.4,  "release": 0.45}
  },
  "weapons": {
    "pembaris": {"name": "Pembaris Panjang", "dmg": 8,  "reach": 120, "uses": 4, "throw_dmg": 7, "color": "e8c25a"},
    "buku":     {"name": "Buku Teks Tebal",  "dmg": 10, "reach": 92,  "uses": 3, "throw_dmg": 9, "stun": 0.6, "color": "c0503c"},
    "penyapu":  {"name": "Penyapu",          "dmg": 7,  "reach": 130, "uses": 5, "throw_dmg": 6, "sweep": true, "color": "b88a4a"},
    "beg":      {"name": "Beg Sekolah",      "dmg": 9,  "reach": 98,  "uses": 3, "throw_dmg": 9, "kb": 360, "color": "3c5a9c"},
    "bola":     {"name": "Bola Keranjang",   "dmg": 6,  "reach": 84,  "uses": 6, "throw_dmg": 9, "bounce": true, "color": "e07a2a"},
    "bantal":   {"name": "Bantal",           "dmg": 5,  "reach": 96,  "uses": 6, "throw_dmg": 5, "down": true, "color": "f2efe6"}
  }
}
```

data/chapters.json · 29 lines

```
{
  "arcs": [
    {
      "id": "t4",
      "name": "Sejarah Tingkatan 4",
      "open": true,
      "chapters": [
        {"no": 1, "title": "Warisan Negara Bangsa", "icon": "mahkota", "open": true,
         "zones": ["1_1", "1_2", "1_3", "1_4"]},
        {"no": 2, "title": "Kebangkitan Nasionalisme", "icon": "bendera", "open": true,
         "zones": ["2_1", "2_2", "2_3", "2_4", "2_5", "2_6", "2_7", "2_8"]},
        {"no": 3, "title": "Konflik Dunia dan Pendudukan Jepun di Negara Kita", "icon": "pedang", "open": false},
        {"no": 4, "title": "Era Peralihan Kuasa British di Negara Kita", "icon": "kapal", "open": false},
        {"no": 5, "title": "Persekutuan Tanah Melayu 1948", "icon": "gulungan", "open": false},
        {"no": 6, "title": "Ancaman Komunis dan Perisytiharan Darurat", "icon": "kunci", "open": false},
        {"no": 7, "title": "Usaha ke Arah Kemerdekaan", "icon": "kumpulan", "open": false},
        {"no": 8, "title": "Pilihan Raya", "icon": "nota", "open": false},
        {"no": 9, "title": "Perlembagaan Persekutuan Tanah Melayu 1957", "icon": "kitab", "open": false},
        {"no": 10, "title": "Pemasyhuran Kemerdekaan", "icon": "bintang", "open": false}
      ]
    },
    {
      "id": "t5",
      "name": "Sejarah Tingkatan 5",
      "open": false,
      "chapters": []
    }
  ]
}
```

data/characters.json · 282 lines (first 70 shown)

```
{
  "alif": {
    "name": "Alif",
    "portrait": "alif_",
    "default_expr": "neutral",
    "sprite": "alif",
    "tokoh": false
  },
  "pau": {
    "name": "Pau",
    "portrait": "pau_",
    "default_expr": "neutral",
    "sprite": "pau",
    "tokoh": false
  },
```

```
  "penjaga": {
    "name": "Penjaga Gerbang Masa",
    "portrait": "penjaga",
    "sprite": "penjaga",
    "tokoh": true,
    "zon": "1_1",
    "gelaran": "Pemandu Misi 1.1",
    "lore": "Watak rekaan. Berpakaian gabungan lima zaman dan menjaga benang warisan kerajaan awal Alam Melayu.",
    "rekaan": true
  },
  "syahbandar": {
    "name": "Syahbandar",
    "portrait": "syahbandar",
    "sprite": "syahbandar",
    "tokoh": true,
    "zon": "1_2",
    "gelaran": "Pemandu Misi 1.2",
    "lore": "Watak rekaan. Pegawai pelabuhan Melaka yang sibuk mengurus kapal dan pedagang asing.",
    "rekaan": true
  },
  "chengho": {
    "name": "Laksamana Cheng Ho",
    "portrait": "chengho",
    "sprite": "chengho",
    "tokoh": true,
    "zon": "1_2",
    "gelaran": "Utusan Dinasti Ming",
    "lore": "Kunjungannya menjadi bukti kedaulatan Melaka diiktiraf oleh kuasa luar."
  },
  "tunperak": {
    "name": "Bendahara Tun Perak",
    "portrait": "tunperak",
    "sprite": "tunperak",
    "tokoh": true,
    "zon": "1_3",
    "gelaran": "Bendahara Melaka",
    "lore": "Pembesar yang bijaksana dan tenang. Membantu sultan mentadbir Melaka."
  },
  "sapurba": {
    "name": "Sang Sapurba",
    "portrait": "sapurba",
    "sprite": "sapurba",
    "tokoh": true,
    "zon": "1_4",
    "gelaran": "Pihak pemerintah dalam Waadat",
    "lore": "Tokoh dalam Sejarah Melayu. Berjanji memerintah dengan adil."
  },
  "demang": {
    "name": "Demang Lebar Daun",
    "portrait": "demang",
    "sprite": "demang",
    "tokoh": true,
    "zon": "1_4",
    "gelaran": "Pihak rakyat dalam Waadat",
    "lore": "Tokoh dalam Sejarah Melayu. Berjanji rakyat akan taat setia."
```

`data/dialogue/prologue.json` · 239 lines (first 70 shown)

```
{
  "pro_room": {
    "variants": [
      [
        {"who": "alif", "text": "Okey. Fokus, Alif. Bab 1.", "e": "worried"}
      ]
    ]
  },
  "pro_obj_kalendar": {
    "variants": [
      [
        {"who": "alif", "text": "SPM Sejarah. Tiga hari lagi.", "e": "worried"},
        {"who": "alif", "text": "Tiga hari untuk sepuluh bab. Tak panik. ...Panik sikit.", "e": "worried"}
      ]
    ]
  },
  "pro_obj_kertas": {
    "variants": [
      [
        {"who": "alif", "text": "Kertas Percubaan: 38 markah. Cikgu tulis, \"Fakta ada, tapi tak nampak kaitan.\"", "e"
: "confused"},
        {"who": "alif", "text": "Kaitan apa? Aku hafal semua. Lepas tu semua hilang.", "e": "worried"}
      ]
    ]
  },
```

```
  "pro_obj_telefon": {
    "variants": [
      [
        {"who": "mak", "text": "Lif, jangan tidur lewat sangat. Nasi ada dalam tudung saji. Mak sayang."},
        {"who": "alif", "text": "...Okey, Mak. Lepas satu bab.", "e": "happy"}
      ]
    ]
  },
  "pro_obj_tingkap": {
    "variants": [
      [
        {"who": "alif", "text": "Dulu Tok selalu cerita pasal Hang Tuah, pasal kapal dagang di Melaka. Best je dengar."
, "e": "happy"},
        {"who": "alif", "text": "Kenapa bila masuk buku teks, jadi macam senarai barang dapur?", "e": "confused"}
      ]
    ]
  },
  "pro_obj_pau": {
    "variants": [
      [
        {"who": "alif", "text": "Kau senang la, Pau. Squishy je. Tak payah ambil SPM.", "e": "worried"}
      ]
    ]
  },
  "pro_obj_buku": {
    "variants": [
      [
        {"who": "alif", "text": "Okey. Buka buku. Kali ni mesti masuk.", "e": "confident"}
      ]
    ]
  },
  "pro_read": {
    "variants": [
      [
        {"who": "alif", "text": "\"Kesultanan Melayu Melaka diasaskan oleh... Para... Parameswara.\"", "e": "worried"}
      ]
    ]
  },
  "pro_read_fikir_1": {
    "variants": [
      [
        {"who": "alif", "text": "Okey, masuk... Eh. Yang tadi tu apa?", "e": "confused"}
      ]
    ]
  },
  "pro_read_fikir_2": {
    "variants": [
```

`data/dialogue/world.json` · 763 lines (first 70 shown)

```
{
  "_note": "Each id has 'variants': a list of conversations. First talk plays variant 0, next talk variant 1, and so on
; after the last one it picks randomly. 'rekaan': true marks invented lines for historical figures (GDS rule).",
  "sign_start": {
    "variants": [
      [
        {
          "who": "papan",
          "text": "Dunia Explo-Story.\nKampung Jejak: ikut jalan ni ke atas."
        }
      ]
    ]
  },
  "sign_sea": {
    "variants": [
      [
        {
          "who": "papan",
          "text": "Bab 8, 9 dan 10: seberang laut."
        },
        {
          "who": "alif",
          "text": "Tak ada jambatan. Tak ada perahu."
        },
        {
          "who": "pau",
          "text": "Nanti ada la tu. Sabar."
        }
      ]
    ]
  },
  "monument": {
    "variants": [
```

```
      [
        {
          "who": "alif",
          "text": "Ni... peta ke? Ada sepuluh bendera."
        },
        {
          "who": "pau",
          "text": "Satu bendera, satu bab. Yang menyala tu Bab 1."
        },
        {
          "who": "alif",
          "text": "Sepuluh bab. Tiga hari lagi SPM."
        },
        {
          "who": "pau",
          "text": "Eh, jangan kira dulu. Kita buat satu-satu je."
        }
      ],
      [
        {
          "who": "pau",
          "text": "Bendera 1 masih tunggu kita. Gerbang dia kat utara kampung."
        }
      ]
    ]
  },
  "istana": {
    "variants": [
      [
        {
          "who": "alif",
          "text": "Besarnya... ni istana Melaka ke?"
        },
        {
          "who": "pau",
          "text": "Ni Balai Penghadapan. Bendahara ada kat dalam. Jom."
        }
      ],
```

data/i18n/en.json · 339 lines (first 70 shown)

```
{
  "_info": "English for menus only. Key = Bahasa Melayu text exactly as in the game, value = English. Dialogue, puzzles and history notes stay in BM.",
  "TETAPAN": "SETTINGS",
  "Paparan": "Display",
  "Audio": "Audio",
  "Bahasa": "Language",
  "Akaun": "Account",
  "Mod Mudah Baca": "Easy Reading Mode",
  "Fon": "Font",
  "Saiz teks": "Text size",
  "Kecil": "Small",
  "Sederhana": "Medium",
  "Besar": "Large",
  "Kelajuan teks": "Text speed",
  "Perlahan": "Slow",
  "Biasa": "Normal",
  "Serta-merta": "Instant",
  "Baca dialog dengan suara": "Read dialogue aloud",
  "Senyapkan semua bunyi": "Mute all sound",
  "Ya": "Yes",
  "Tidak": "No",
  "Bunyi": "Sound",
  "Muzik": "Music",
  "Bunyi kesan": "Sound effects",
  "Suara watak": "Character voices",
  "Bahasa menu": "Menu language",
  "Menu, butang dan peta ikut bahasa ini. Dialog, teka-teki dan nota sejarah kekal dalam Bahasa Melayu, sama seperti pe periksaan.": "Menus, buttons and the map use this language. Dialogue, puzzles and history notes stay in Bahasa Melayu, the same as the exam.",
  "Contoh: Kesultanan Melayu Melaka muncul pada abad ke-15.": "Contoh: Kesultanan Melayu Melaka muncul pada abad ke-15.",
  "Tajuk 1 2 3 4 5 6 7 8 9 0": "Heading 1 2 3 4 5 6 7 8 9 0",
  "Pemain: %s": "Player: %s",
  "Pasti? Ketik lagi": "Sure? Tap again",
  "Log keluar": "Log out",
  "Tutup": "Close",
  "Nota Alif": "Alif's Notes",
  "Beg Alif": "Alif's Bag",
  "Dapat: Beg Alif!": "Got: Alif's Bag!",
  "MISI %s BERMULA: %s": "MISSION %s STARTED: %s",
  "untuk ambil beg": "to pick up the bag",
```

```
  "Tekan": "Press",
  "Ketik": "Tap",
  "%d langkah": "%d steps",
  "Rumah Alif": "Alif's House",
  "Ketik untuk dengar ceritanya": "Tap to hear its story",
  "Tetapan": "Settings",
  "Peta (M)": "Map (M)",
  "Bertenang bersama Pau (atau tekan lama Pau)": "Calm down with Pau (or press and hold Pau)",
  "TAHAP %d": "LEVEL %d",
  "SYILING: %d": "COINS: %d",
  "EXP": "EXP",
  "E / Ketik: ": "E / Tap: ",
  "Lihat": "Look",
  "Rumah": "House",
  "Peta Jejak": "Trail Map",
  "Istana Melaka": "Melaka Palace",
  "Papan tanda": "Signboard",
  "Arena Ulang Kaji": "Revision Arena",
  "Gerbang berkunci": "Locked gate",
  "Gerbang Bab 1": "Chapter 1 gate",
  "LANGKAU »": "SKIP »",
  "PETA DUNIA EXPLO-STORY": "EXPLO-STORY WORLD MAP",
  "Tempat": "Places",
  "Kata Kunci": "Keywords",
  "Simbol": "Symbols",
  "-  Zum keluar": "-  Zoom out",
  "+  Zum masuk": "+  Zoom in",
  "Seluruh peta": "Whole map",
  "Cari Alif": "Find Alif",
  "Seret peta untuk gerak. Roda tetikus atau jepit untuk zum.": "Drag the map to move. Mouse wheel or pinch to zoom.",
  "Tunjuk di peta": "Show on map",
  "Ketik untuk buka peta penuh (M)": "Tap to open the full map (M)",
```

data/interiors/bilik_alif.json · 48 lines

```
{
 "_info": "Bilik Alif dalam Dunia Explo-Story (scripts/world/bilik_alif.gd, lukisan: scripts/world/bilik_art.gd). cells = petak [x, y] dalam bilik 20x11; petak barang tak boleh dipijak. Petak baris 2 ialah dinding (tengok dari lantai di bawahnya). 'beg' diambil semasa tutorial pertama.",
 "exhibits": [
  {"id": "katil", "label": "Katil", "cells": [[3, 3], [4, 3], [5, 3], [3, 4], [4, 4], [5, 4], [3, 5], [4, 5], [5, 5]], "lines": [
   {"s": "alif", "t": "Katil aku. Bantal peluk pun ada."},
   {"s": "pau", "t": "Jangan sambung tidur. Kabus Lupa tak tunggu orang."}
  ]},
  {"id": "meja_sisi", "label": "Gambar keluarga", "cells": [[6, 3]], "lines": [
   {"s": "alif", "e": "happy", "t": "Gambar aku dengan Mak dan Tok masa raya."},
   {"s": "alif", "t": "Tok la yang mula-mula cerita pasal Hang Tuah kat aku."}
  ]},
  {"id": "kalendar", "label": "Kalendar", "cells": [[7, 2]], "lines": [
   {"s": "alif", "e": "worried", "t": "Tarikh SPM Sejarah pun ada bulatan merah kat sini."},
   {"s": "pau", "t": "Relaks. Satu misi, satu langkah."}
  ]},
  {"id": "tingkap", "label": "Tingkap", "cells": [[8, 2], [9, 2], [10, 2], [11, 2]], "lines": [
   {"s": "alif", "e": "confused", "t": "Pokok kelapa, langit cerah... ni bukan pemandangan rumah aku."},
   {"s": "pau", "t": "Tu Kampung Jejak. Gerbang Masa kat utara sana. Misi pertama kita."}
  ]},
  {"id": "rak_buku", "label": "Rak buku", "cells": [[12, 3], [13, 3]], "lines": [
   {"s": "alif", "t": "Buku cerita, buku latihan, poster Harimau Malaya."},
   {"s": "alif", "e": "confused", "t": "Jadual belajar tu aku buat sendiri. Ikut? Tak pernah."},
   {"s": "pau", "e": "happy", "t": "Kali ni kita ikut sama-sama."}
  ]},
  {"id": "kipas", "label": "Kipas", "cells": [[14, 3]], "lines": [
   {"s": "alif", "t": "Kipas berdiri. Tanpa dia, aku tak boleh tidur."},
   {"s": "pau", "e": "happy", "t": "Sedapnya angin... aku nak duduk depan ni je."}
  ]},
  {"id": "almari", "label": "Almari baju", "cells": [[15, 3], [16, 3], [15, 4], [16, 4]], "lines": [
   {"s": "alif", "t": "Baju sekolah, baju Melayu raya..."},
   {"s": "pau", "t": "Pakaian lain boleh beli dengan syiling. Ketik kad kau kat bucu kanan atas, pilih Kedai."}
  ]},
  {"id": "meja_belajar", "label": "Meja belajar", "cells": [[15, 6], [16, 6], [15, 7], [16, 7], [14, 7]], "lines": [
   {"s": "alif", "t": "Buku teks Sejarah. Masih terbuka kat Bab 1."},
   {"s": "alif", "e": "worried", "t": "Lampu meja ni dah tiga malam menyala sampai pagi."},
   {"s": "pau", "t": "Kat sini kita belajar cara lain. Jalan, tengok, faham."}
  ]},
  {"id": "sejadah", "label": "Sejadah", "cells": [[3, 7], [4, 7], [3, 8], [4, 8]], "lines": [
   {"s": "alif", "t": "Sejadah hadiah Tok."},
   {"s": "alif", "t": "Mak selalu pesan, sesibuk mana pun, jangan tinggal solat."}
  ]},
  {"id": "beg", "label": "Beg sekolah", "cells": [[12, 7]], "lines": []},
  {"id": "kasut", "label": "Kasut sekolah", "cells": [[12, 9]], "lines": [
   {"s": "alif", "t": "Kasut sekolah. Baru sapu kapur semalam."},
   {"s": "pau", "t": "Putih berkilat! Tapi kat dunia ni banyak jalan kaki. Siap sedia."}
```

```
   ]}
  ]
}
```

data/interiors/jelutong.json · 115 lines

```
{
 "_info": "Jelutong Press, Pulau Pinang (1927), milik Syed Sheikh al-Hadi. Cerita akhbar dan novel sebagai faktor kebangkitan.",
 "intro": [
  {
   "s": "pencetak",
   "t": "Hati-hati, lantai licin dengan dakwat!"
  },
  {
   "s": "pencetak",
   "t": "Selamat datang ke Jelutong Press. Kat sini, kertas jadi senjata."
  },
  {
   "s": "pau",
   "e": "scared",
   "t": "Senjata? Kertas?"
  },
  {
   "s": "pencetak",
   "t": "Senjata fikiran. Tengoklah sendiri."
  }
 ],
 "npc_talk": [
  {
   "s": "pencetak",
   "t": "Satu akhbar boleh dibaca sepuluh orang kat kedai kopi. Itulah kuasa perkataan."
  },
  {
   "s": "pencetak",
   "t": "British boleh tutup mulut orang. Tapi susah nak tutup mata orang yang dah pandai membaca."
  }
 ],
 "exhibits": [
  {
   "id": "mesin_cetak",
   "label": "Mesin cetak",
   "cells": [
    [
     9,
     5
    ],
    [
     10,
     5
    ]
   ],
   "lines": [
    {
     "s": "narator",
     "t": "Mesin cetak berdentum. Kertas keluar satu demi satu."
    },
    {
     "s": "pencetak",
     "t": "Ni Jelutong Press, milik Syed Sheikh al-Hadi. Dibuka kat Pulau Pinang tahun 1927."
    },
    {
     "s": "pencetak",
     "t": "Akhbar Saudara pun kami cetak kat sini."
    }
   ]
  },
  {
   "id": "rak_akhbar",
   "label": "Rak akhbar lama",
   "cells": [
    [
     3,
     5
    ],
    [
     4,
     5
    ]
   ],
   "lines": [
    {
     "s": "pencetak",
```

```
      "t": "Akhbar Jawi Peranakan, 1876. Antara akhbar Melayu yang terawal, terbit kat Singapura."
     },
     {
      "s": "pencetak",
      "t": "Akhbar dan majalah sebarkan idea. Orang yang tak pernah jumpa pun boleh berkongsi semangat yang sama."
     },
     {
      "s": "alif",
      "e": "happy",
      "t": "Macam media sosial zaman dulu."
     }
    ]
   },
   {
    "id": "rak_novel",
    "label": "Rak novel",
    "cells": [
     [
      15,
      5
     ],
     [
      16,
      5
     ]
    ],
    "lines": [
     {
      "s": "pencetak",
      "t": "Ni novel tulisan Syed Sheikh al-Hadi, Hikayat Faridah Hanum."
     },
     {
      "s": "pencetak",
      "t": "Cerita dia pasal pendidikan dan hak wanita. Orang baca novel, tapi dalam diam, fikiran pun berubah."
     }
    ]
   }
  ]
}
```

data/interiors/sitc.json · 108 lines

```
{
 "_info": "Maktab Perguruan Sultan Idris (SITC), Tanjung Malim. Bilik darjah: cerita Pejabat Karang Mengarang, guru Melayu dan KMM.",
 "intro": [
  {
   "s": "pelatih",
   "t": "Eh, pelawat? Selamat datang ke SITC. Maktab Perguruan Sultan Idris."
  },
  {
   "s": "pau",
   "e": "happy",
   "t": "Wah, macam kelas sekolah kita! Cuma semua meja kayu."
  },
  {
   "s": "pelatih",
   "t": "Tengoklah sekeliling. Banyak cerita dalam bilik ni."
  }
 ],
 "npc_talk": [
  {
   "s": "pelatih",
   "t": "Cikgu kami selalu pesan: bangsa yang tak membaca, senang diperbodohkan."
  },
  {
   "s": "pelatih",
   "t": "Sebab tu kami belajar rajin-rajin. Nanti kami pula yang ajar anak-anak kampung."
  }
 ],
 "exhibits": [
  {
   "id": "meja_guru",
   "label": "Meja guru dan papan hitam",
   "cells": [
    [
     9,
     3
    ],
    [
     10,
     3
    ]
```

```
    ],
    "lines": [
     {
      "s": "narator",
      "t": "Papan hitam tertulis: Maktab Perguruan Sultan Idris, Tanjung Malim, 1922."
     },
     {
      "s": "pelatih",
      "t": "Kat sini kami belajar jadi guru. Lepas tamat, kami balik mengajar di sekolah Melayu."
     },
     {
      "s": "pelatih",
      "t": "Bila orang kampung pandai membaca, mereka mula sedar nasib bangsa sendiri."
     }
    ]
   },
   {
    "id": "rak_karang",
    "label": "Rak Pejabat Karang Mengarang",
    "cells": [
     [
      13,
      3
     ]
    ],
    "lines": [
     {
      "s": "pelatih",
      "t": "Ni buku-buku Pejabat Karang Mengarang. Ditubuhkan di SITC pada 1924."
     },
     {
      "s": "pelatih",
      "t": "Za'ba antara yang menterjemah dan menulis buku. Murid Melayu dapat baca buku dalam bahasa sendiri."
     },
     {
      "s": "alif",
      "e": "confident",
      "t": "Buku dalam bahasa sendiri... Tu pun satu cara bangkitkan semangat."
     }
    ]
   },
   {
    "id": "meja_pelatih",
    "label": "Meja pelatih",
    "cells": [
     [
      7,
      6
     ]
    ],
    "lines": [
     {
      "s": "narator",
      "t": "Ada nota terselit bawah meja: 'Ikatan Pemuda Pelajar. Jumpa malam ni.'"
     },
     {
      "s": "pelatih",
      "t": "Ibrahim Haji Yaakob pun belajar kat sini. Tahun 1938, dia tubuhkan Kesatuan Melayu Muda, KMM."
     },
     {
      "s": "alif",
      "e": "confident",
      "t": "Jadi SITC bukan sekadar maktab. Tempat lahirnya pejuang."
     }
    ]
   }
  ]
}
```

data/leaderboard_sample.json · 25 lines

```
{
  "_info": "Sample players for Papan Markah while the game is offline (shown with a 'Contoh' banner). 'friend': true = shown on the Rakan tab offline. Real rankings come from supabase/leaderboard.sql.",
  "players": [
    {"name": "Aisyah", "skin": "rakan_aisyah", "negeri": "Selangor", "exp": 1710, "wins": 9, "friend": true},
    {"name": "Ravi", "skin": "rakan_ravi", "negeri": "Perak", "exp": 1380, "wins": 14, "friend": true},
    {"name": "Mei Ling", "skin": "rakan_meiling", "negeri": "Pulau Pinang", "exp": 1620, "wins": 6, "friend": true},
    {"name": "Daniel", "skin": "rakan_daniel", "negeri": "Sabah", "exp": 990, "wins": 4, "friend": true},
    {"name": "Hakim", "skin": "alif_pahlawan", "negeri": "Melaka", "exp": 2280, "wins": 21},
    {"name": "Nurul Izzah", "skin": "rakan_aisyah", "negeri": "Kelantan", "exp": 2110, "wins": 12},
    {"name": "Arif", "skin": "alif_bomba", "negeri": "Johor", "exp": 1960, "wins": 17},
    {"name": "Siti Hajar", "skin": "rakan_aisyah", "negeri": "Terengganu", "exp": 1840, "wins": 8},
```

```
    {"name": "Kavitha", "skin": "rakan_meiling", "negeri": "Selangor", "exp": 1500, "wins": 10},
    {"name": "Wei Jie", "skin": "alif_sukan", "negeri": "WP Kuala Lumpur", "exp": 1440, "wins": 11},
    {"name": "Farhan", "skin": "alif_pengakap", "negeri": "Kedah", "exp": 1260, "wins": 5},
    {"name": "Jessica", "skin": "rakan_meiling", "negeri": "Sarawak", "exp": 1180, "wins": 7},
    {"name": "Aiman", "skin": "alif_melayu", "negeri": "Pahang", "exp": 1100, "wins": 3},
    {"name": "Balqis", "skin": "rakan_aisyah", "negeri": "Negeri Sembilan", "exp": 940, "wins": 2},
    {"name": "Darren", "skin": "rakan_daniel", "negeri": "Sabah", "exp": 860, "wins": 6},
    {"name": "Haziq", "skin": "alif", "negeri": "Perlis", "exp": 720, "wins": 1},
    {"name": "Syafiqah", "skin": "rakan_aisyah", "negeri": "WP Putrajaya", "exp": 640, "wins": 2},
    {"name": "Muthu", "skin": "rakan_ravi", "negeri": "Johor", "exp": 560, "wins": 4},
    {"name": "Amirul", "skin": "alif_sukan", "negeri": "Melaka", "exp": 410, "wins": 0},
    {"name": "Nadia", "skin": "rakan_meiling", "negeri": "WP Labuan", "exp": 300, "wins": 1}
  ]
}
```

data/map_legend.json · 29 lines

```
{
  "_info": "Legenda peta mini. 'at' = petak [x, y] dalam dunia (140x60: pulau Bab 1 di kiri, Lembah Kebangkitan Bab 2 di kanan). 'zone' = misi di tempat itu. Kata kunci diambil dari data/nota.json dan data/nota_bab2.json.",
  "places": [
    {"id": "permulaan", "name": "Bukit Permulaan", "icon": "res://assets/icons/bintang.png", "at": [19, 50], "info": "Rumah Alif: tempat Alif terjaga"},
    {"id": "kampung", "name": "Kampung Jejak", "icon": "res://assets/icons/kumpulan.png", "at": [33, 33], "info": "Peta Jejak, Arena Ulang Kaji (latihan & lawan pemain)"},
    {"id": "gerbang", "zone": "1_1", "name": "Misi 1.1: Gerbang Masa", "at": [34, 16]},
    {"id": "pelabuhan", "zone": "1_2", "name": "Misi 1.2: Pelabuhan Melaka", "at": [61, 41]},
    {"id": "melaka", "zone": "1_3", "name": "Misi 1.3: Balai Penghadapan", "at": [55, 29], "info": "Masuk ikut pintu Istana"},
    {"id": "siguntang", "zone": "1_4", "name": "Misi 1.4: Bukit Siguntang", "at": [57, 13]},
    {"id": "kabus", "name": "Pintu Kabus", "icon": "res://assets/icons/kunci.png", "at": [10, 26], "info": "Pintu ke Lembah Kebangkitan (Bab 2), terbuka selepas Bab 1"},
    {"id": "lembah", "name": "Lembah Kebangkitan", "icon": "res://assets/icons/bendera.png", "at": [90, 31], "info": "Bab 2: Kebangkitan Nasionalisme"},
    {"id": "arena_lembah", "name": "Arena Lembah", "icon": "res://assets/icons/pedang.png", "at": [130, 18], "info": "Arena Ulang Kaji tepi padang (latihan & lawan pemain)"},
    {"id": "dewan", "zone": "2_1", "name": "Misi 2.1: Dewan Kedaulatan Rakyat", "at": [99, 22]},
    {"id": "falsafah", "zone": "2_2", "name": "Misi 2.2: Rumah Falsafah Sejagat", "at": [99, 38]},
    {"id": "dermaga", "zone": "2_3", "name": "Misi 2.3: Dermaga Asia", "at": [112, 54]},
    {"id": "serantau", "zone": "2_4", "name": "Misi 2.4: Balai Serantau", "at": [128, 38]},
    {"id": "tugu", "zone": "2_5", "name": "Misi 2.5: Tugu Pahlawan", "at": [122, 17]},
    {"id": "kompleks", "zone": "2_6", "name": "Misi 2.6: Kompleks Intelektual", "at": [104, 6]},
    {"id": "sitc", "zone": "2_7", "name": "Misi 2.7: Maktab SITC", "at": [130, 5]},
    {"id": "menara", "zone": "2_8", "name": "Misi 2.8: Menara Pentadbiran", "at": [131, 24]}
  ],
  "landmarks": {
    "monument": {"name": "Peta Jejak", "icon": "res://assets/icons/peta.png"},
    "rumah_alif": {"name": "Rumah Alif", "icon": "res://assets/icons/sekolah.png"},
    "istana": {"name": "Istana Melaka", "icon": "res://assets/icons/mahkota.png"},
    "arena": {"name": "Arena Ulang Kaji", "icon": "res://assets/icons/pedang.png"},
    "arena_lembah": {"name": "Arena Ulang Kaji", "icon": "res://assets/icons/pedang.png"}
  }
}
```

data/nota.json · 91 lines (first 70 shown)

```
{
  "title": "Nota Alif: Bab 1 Warisan Negara Bangsa",
  "pages": [
    {"zone": "1_1", "title": "Garis Masa Kerajaan", "style": "timeline",
     "items": [
       {"icon": "res://assets/objects/gate_funan.png", "text": "Funan", "sub": "abad pertama"},
       {"icon": "res://assets/objects/gate_champa.png", "text": "Champa", "sub": "abad kedua"},
       {"icon": "res://assets/objects/gate_srivijaya.png", "text": "Srivijaya", "sub": "abad ke-7, Palembang"},
       {"icon": "res://assets/objects/gate_melaka.png", "text": "Kesultanan Melayu Melaka", "sub": "abad ke-15"},
       {"icon": "res://assets/objects/gate_johor.png", "text": "Kesultanan Johor-Riau", "sub": "abad ke-16"}
     ],
     "huraian": [
       {"h": "Ringkasnya", "t": ["Asas negara bangsa di Alam Melayu sudah wujud sebelum kedatangan Barat. Prosesnya ambil masa yang lama dan melalui beberapa kerajaan.", "Kerajaan awal yang hebat: Funan, Champa, Kedah Tua, Gangga Nagara, Srivijaya, Angkor dan Majapahit.", "Ciri-ciri negara bangsa Alam Melayu berbeza daripada Barat. Ciri-ciri ini berkembang mengikut peredaran masa dan keperluan."]},
       {"h": "Rantaian warisan", "t": ["Srivijaya (berpusat di Palembang) meneruskan ciri negara bangsa yang diasaskan oleh Funan dan Champa, lalu muncul sebagai empayar yang terkenal.", "Kesultanan Melayu Melaka mewarisi ciri-ciri ini daripada Srivijaya dan menambah ciri-ciri lain untuk memperkukuh kerajaannya.", "Kesultanan Johor-Riau meneruskan ciri-ciri ini dengan menekankan kepentingan Islam.", "Warisan inilah yang menjadikan kerajaan-kerajaan ini terkenal dan berkuasa di Alam Melayu."]},
       {"h": "Info", "t": ["Srivijaya dalam bahasa Sanskrit: \"sri\" bermaksud bercahaya, \"vijaya\" bermaksud kemenangan."]}
     ],
     "ingat": "Urutan kerajaan: Funan (abad pertama), Champa (abad kedua), Srivijaya (abad ke-7), Melaka (abad ke-15),
```

```
Johor-Riau (abad ke-16)."},
    {"zone": "1_1", "title": "Empat Tiang Negara Bangsa", "style": "grid",
     "items": [
       {"icon": "res://assets/icons/mahkota.png", "text": "Raja", "sub": "Raja Champa dikaitkan dengan Dewa Siva"},
       {"icon": "res://assets/icons/timbangan.png", "text": "Undang-undang", "sub": "Kutara Manawa (Majapahit)"},
       {"icon": "res://assets/icons/peta.png", "text": "Wilayah Pengaruh", "sub": "Funan: Sungai Mekong hingga Segentin
g Kra"},
       {"icon": "res://assets/icons/kumpulan.png", "text": "Rakyat", "sub": "Sumpah setia Prasasti Telaga Batu"}
     ],
     "huraian": [
       {"h": "1. Raja", "t": ["Raja ialah tonggak utama kerajaan. Kedudukannya diperkukuh oleh sistem pentadbiran yang
 dibantu oleh para pembesar.", "Raja sentiasa ditaati dan dianggap suci. Contoh: Raja Champa dianggap suci dan dikaitkan
 dengan Dewa Siva."]},
       {"h": "2. Undang-undang", "t": ["Undang-undang dilaksanakan untuk memastikan kesejahteraan kerajaan. Ia terpakai
 di seluruh wilayah pengaruh dan menjadi panduan pentadbiran.", "Contoh: kerajaan Majapahit mengamalkan undang-undang K
utara Manawa."]},
       {"h": "3. Wilayah pengaruh", "t": ["Kawasan yang rakyatnya menerima dan memperakui pemerintahan seseorang raja."
, "Contoh: wilayah Funan sangat luas, dari Sungai Mekong, Teluk Cam Ranh, selatan Myanmar hingga Segenting Kra."]},
       {"h": "4. Rakyat", "t": ["Penduduk di dalam sesebuah wilayah yang setia kepada raja dan tidak menderhaka.", "Con
toh: rakyat Srivijaya menunjukkan kesetiaan melalui upacara persetiaan dan meminum air sumpah.", "Caranya: air dituang
ke atas Inskripsi Telaga Batu, mengalir ke corong di bawahnya, lalu ditadah dan diminum oleh rakyat."]}
     ],
     "ingat": "4 ciri negara bangsa Alam Melayu: Raja, Undang-undang, Wilayah pengaruh, Rakyat."},
    {"zone": "1_2", "title": "Enam Panji Melaka", "style": "grid",
     "items": [
       {"icon": "res://assets/icons/mahkota.png", "text": "Kerajaan", "sub": "Sultan + Sistem Pembesar Empat Lipatan"},
       {"icon": "res://assets/icons/kumpulan.png", "text": "Rakyat", "sub": "Pemerintah dan diperintah, timbal balik"},
       {"icon": "res://assets/icons/bintang.png", "text": "Kedaulatan", "sub": "Diiktiraf Dinasti Ming"},
       {"icon": "res://assets/icons/peta.png", "text": "Wilayah Pengaruh", "sub": "Taklukan, naungan, perkahwinan"},
       {"icon": "res://assets/icons/gulungan.png", "text": "Undang-undang", "sub": "Hukum Kanun Melaka, Undang-Undang L
aut Melaka"},
       {"icon": "res://assets/icons/nobat.png", "text": "Lambang Kebesaran", "sub": "Adat istiadat, warna, nobat, regal
ia, cap mohor"}
     ],
     "huraian": [
       {"h": "Kerajaan", "t": ["Kerajaan wujud kerana adanya raja. Raja ialah pemerintah tertinggi dan dibantu oleh pem
besar.", "Sistem Pembesar Empat Lipatan menjadi tonggak pentadbiran Melaka.", "Selepas kedatangan Islam, pemerintah Mel
aka mula menggunakan gelaran sultan."]},
       {"h": "Rakyat", "t": ["Rakyat taat setia kepada sultan dan mematuhi undang-undang. Secara umum, rakyat terdiri d
aripada orang Melayu dan Orang Laut.", "Golongan diperintah terdiri daripada golongan merdeka dan hamba.", "Sultan menj
amin kesejahteraan rakyat; rakyat mengakui kekuasaan sultan. Inilah hubungan timbal balik."]},
       {"h": "Kedaulatan", "t": ["Lambang kekuasaan tertinggi sultan: hak mutlak untuk mengawal rakyat dan wilayah.", "
Kedaulatan Melaka diakui oleh kerajaan lain dan bebas daripada campur tangan kuasa asing.", "Diiktiraf oleh Dinasti Min
g, yang menghantar utusan termasuk Laksamana Cheng Ho."]},
       {"h": "Wilayah pengaruh", "t": ["Meliputi seluruh Semenanjung Tanah Melayu dan pantai timur Sumatera.", "Takluka
n (melalui perang): Pahang, Kuala Linggi, Kelantan, Rokan, Inderagiri, Kampar.", "Naungan (perkahwinan atau persahabata
n): Lingga, Patani, Kedah.", "Perkahwinan: Majapahit menghadiahkan Siantan kerana Sultan Mansur Shah berkahwin dengan P
uteri Majapahit."]},
       {"h": "Undang-undang", "t": ["Dua undang-undang bertulis: Hukum Kanun Melaka dan Undang-Undang Laut Melaka. Tuju
annya mengukuhkan kerajaan dan melindungi rakyat."]},
       {"h": "Lambang kebesaran", "t": ["Identiti kerajaan, keistimewaan sultan dan simbol kedaulatan.", "Contoh: adat
 istiadat, penggunaan warna, nobat, regalia dan cap mohor. Cap mohor digunakan untuk mengesahkan arahan sultan."]}
     ],
     "ingat": "6 ciri negara bangsa Melaka: Kerajaan, Rakyat, Kedaulatan, Wilayah pengaruh, Undang-undang, Lambang kebe
saran."},
    {"zone": "1_3", "title": "Pembesar Melaka", "style": "grid",
     "items": [
       {"icon": "res://assets/portraits/tunperak.png", "text": "Bendahara", "sub": "Memangku sultan"},
       {"icon": "res://assets/icons/syiling.png", "text": "Penghulu Bendahari", "sub": "Perbendaharaan dan cukai"},
       {"icon": "res://assets/portraits/syahbandar.png", "text": "Syahbandar", "sub": "Pelabuhan dan cukai kapal"},
       {"icon": "res://assets/icons/kunci.png", "text": "Temenggung", "sub": "Keamanan dalam kota"},
       {"icon": "res://assets/icons/kapal.png", "text": "Laksamana", "sub": "Keselamatan perairan"}
     ],
     "huraian": [
       {"h": "Sistem Pembesar Empat Lipatan", "t": ["Keunggulan bermaksud kesempurnaan dan keistimewaan sesebuah keraja
an. Melaka unggul kerana pentadbiran dan undang-undangnya tersusun.", "Sistem ini tersusun dan berhierarki. Sultan meng
etuai pemerintahan dan berkuasa dalam pemerintahan, peperangan, hubungan diplomatik dan agama.", "Pembesar Berempat: Be
ndahara, Penghulu Bendahari, Temenggung, Laksamana.", "Pembesar lain yang membantu: Syahbandar, Bentara, Orang Kaya, hu
lubalang dan penghulu."]},
       {"h": "Dua peringkat pentadbiran", "t": ["Pusat: dijalankan oleh Pembesar Berempat. Permuafakatan sultan dengan
 pembesar mengelakkan tindakan sewenang-wenangnya oleh pemerintah.", "Jajahan: pembesar yang dilantik oleh sultan dan Pe
mbesar Berempat mengurus daerah dan kampung, menjaga keamanan dan boleh mengutip cukai.", "Kawasan pegangan: wilayah, s
ungai atau pulau yang dikurniakan oleh sultan kepada Pembesar Berempat untuk ditadbir."]},
       {"h": "Tugas setiap pembesar", "t": ["Bendahara: ketua pentadbir, ketua turus angkatan tentera, penasihat sultan
 dan pemangku sultan.", "Penghulu Bendahari: mengurus perbendaharaan. Pendapatan Melaka datang daripada perdagangan, pe
lbagai cukai dan ufti daripada wilayah taklukan serta naungan.", "Syahbandar: mengutip cukai di pelabuhan. Pembesar lai
n mengutip cukai di kawasan pegangan. Hasilnya digunakan untuk membeli senjata dan membina istana.", "Temenggung: menja
ga keselamatan kota Melaka.", "Laksamana: panglima angkatan laut Melaka."]},
       {"h": "Info", "t": ["Bendahara Tun Perak ialah bendahara paling lama berkhidmat. Beliau berkhidmat untuk empat s
ultan: Sultan Muzaffar Shah, Sultan Mansur Shah, Sultan Alauddin Riayat Shah dan Sultan Mahmud Shah."]}
     ],
     "ingat": "Pembesar Berempat: Bendahara, Penghulu Bendahari, Temenggung, Laksamana. Syahbandar BUKAN Pembesar Berem
pat."},
    {"zone": "1_3", "title": "Undang-undang Melaka", "style": "list",
     "items": [
```

```
      {"icon": "res://assets/icons/gulungan.png", "text": "UULM Fasal 9", "sub": "Malim lalai » pelayaran selamat, pedagang yakin"},
      {"icon": "res://assets/icons/gulungan.png", "text": "HKM Fasal 30", "sub": "Riba diharamkan » perdagangan adil"},
      {"icon": "res://assets/icons/gulungan.png", "text": "UULM Fasal 13", "sub": "Nakhoda bermesyuarat » kurang pertikaian"},
      {"icon": "res://assets/icons/gulungan.png", "text": "HKM: bahasa istana", "sub": "Titah hanya untuk raja » kedalatan terpelihara"}
```

data/nota_bab2.json · 231 lines (first 70 shown)

```
{
  "title": "Nota Alif: Bab 2 Kebangkitan Nasionalisme",
  "pages": [
    {
      "zone": "2_1",
      "title": "Tafsiran Nasionalisme",
      "style": "list",
      "items": [
        {
          "icon": "res://assets/icons/kitab.png",
          "text": "Maksud",
          "sub": "Cinta mendalam pada bangsa dan negara; menentang penjajahan"
        },
        {
          "icon": "res://assets/icons/orang.png",
          "text": "Hans Kohn",
          "sub": "Kesetiaan ulung hanya kepada negara atau bangsa"
        },
        {
          "icon": "res://assets/icons/orang.png",
          "text": "William R. Roff",
          "sub": "Ancaman hak Melayu, ekonomi, sekularisme dan imperialisme Barat"
        },
        {
          "icon": "res://assets/icons/orang.png",
          "text": "Zainal Abidin Abdul Wahid",
          "sub": "Wujud sejak Kesultanan Melayu Melaka"
        },
        {
          "icon": "res://assets/icons/orang.png",
          "text": "Dr. Burhanuddin al-Helmi",
          "sub": "Persatuan dan perpaduan kuat (National Fronts)"
        }
      ]
    },
    {
      "zone": "2_2",
      "title": "Revolusi di Barat",
      "style": "timeline",
      "items": [
        {
          "icon": "res://assets/icons/mahkota.png",
          "text": "England 1688",
          "sub": "Revolusi Keagungan, Bill of Rights, John Locke"
        },
        {
          "icon": "res://assets/icons/gulungan.png",
          "text": "Amerika 1776",
          "sub": "\"Tiada cukai tanpa perwakilan\", Thomas Jefferson"
        },
        {
          "icon": "res://assets/icons/bendera.png",
          "text": "Perancis 1789",
          "sub": "Rousseau, The Social Contract, penjara Bastile"
        }
      ]
    },
    {
      "zone": "2_3",
      "title": "Nasionalisme di Asia",
      "style": "grid",
      "items": [
        {
          "icon": "res://assets/icons/orang.png",
          "text": "India",
          "sub": "Satyagraha (Gandhi); Tilak dan Pal"
        },
        {
          "icon": "res://assets/icons/bintang.png",
          "text": "China",
```

data/peta_layout.json · 81 lines

```
{
  "scale": 4,
  "flags": [
    [
      480,
      1080
    ],
    [
      860,
      928
    ],
    [
      1200,
      1048
    ],
    [
      1408,
      800
    ],
    [
      1200,
      600
    ],
    [
      1480,
      440
    ],
    [
      2048,
      472
    ],
    [
      2320,
      640
    ],
    [
      2416,
      384
    ],
    [
      2240,
      176
    ]
  ],
  "zones": {
    "1_1": [
      160,
      800
    ],
    "1_2": [
      240,
      1248
    ],
    "1_3": [
      600,
      784
    ],
    "1_4": [
      760,
      1272
    ]
  },
  "special": {
    "muzium": [
      1080,
      1272
    ],
    "arena": [
      1592,
      960
    ],
    "t5": [
      1880,
      136
    ],
    "spawn": [
      384,
      1320
    ]
  }
}
```

data/prologue_room.json · 30 lines

```
{
  "_info": "Prolog (scripts/prologue.gd), ditonton sahaja. chat = mesej kumpulan kelas di awal; room = barang yang Alif lihat satu demi satu, ikut susunan (rect = x, y, lebar, tinggi pada skrin 1280x720; say = kunci dialog dalam data/dialogue/prologue.json); paragraph = perenggan buku teks, {kata kunci} terbang ke Ingatan Alif; memory_size = berapa kata Alif boleh ingat; read_words = berapa kata dibaca; word_every = saat antara kata; thoughts = dialog selepas kata ke-n.",
  "chat": {
    "title": "5 Bestari",
    "time": "11:47 PM",
    "messages": [
      {"from": "daniel", "text": "korang dah habis Bab 1??"},
      {"from": "meiling", "text": "cikgu cakap ciri negara bangsa confirm keluar"},
      {"from": "ravi", "text": "aku dah baca 3 kali. satu pun tak masuk T_T"},
      {"from": "aisyah", "text": "tidur la weh. esok sekolah"},
      {"from": "daniel", "text": "Alif? hidup lagi ke"},
      {"from": "alif", "text": "tengah baca la ni..."},
      {"from": "ravi", "text": "kalau kau faham, esok ajar aku ok"}
    ]
  },
  "caption": "11:52 malam. 3 hari sebelum SPM Sejarah.",
  "room": [
    {"id": "kalendar", "label": "Kalendar", "rect": [950, 66, 112, 172], "say": "pro_obj_kalendar"},
    {"id": "kertas", "label": "Kertas Percubaan", "rect": [24, 304, 104, 158], "say": "pro_obj_kertas"},
    {"id": "telefon", "label": "Telefon", "rect": [300, 418, 96, 60], "say": "pro_obj_telefon"},
    {"id": "tingkap", "label": "Tingkap", "rect": [430, 84, 480, 280], "say": "pro_obj_tingkap"},
    {"id": "pau", "label": "Pau", "rect": [1040, 340, 172, 176], "say": "pro_obj_pau"},
    {"id": "buku", "label": "Buku Teks Sejarah", "rect": [706, 552, 308, 104], "say": "pro_obj_buku"}
  ],
  "paragraph": "Kesultanan Melayu Melaka diasaskan oleh {Parameswara} pada awal {abad ke-15}. Melaka mempunyai ciri-ciri negara bangsa iaitu {kerajaan}, {rakyat}, {kedaulatan}, {wilayah pengaruh}, {undang-undang} dan {lambang kebesaran}.",
  "memory_size": 2,
  "read_words": 6,
  "word_every": 1.3,
  "thoughts": {"2": "pro_read_fikir_1", "4": "pro_read_fikir_2"}
}
```

data/skins.json · 11 lines

```
{
  "_info": "Pakaian (skin) Alif. folder = assets/characters32/<folder>/ (dijana PixelLab, tools/asset_manifest.json). price 0 = percuma.",
  "skins": [
    {"id": "sekolah", "name": "Baju Sekolah", "folder": "alif", "price": 0, "info": "Kemeja putih, seluar biru. Alif yang biasa."},
    {"id": "melayu", "name": "Baju Melayu", "folder": "alif_melayu", "price": 0, "info": "Baju Melayu hijau, samping dan songkok."},
    {"id": "pengakap", "name": "Pengakap", "folder": "alif_pengakap", "price": 120, "info": "Sedia selalu! Lengkap dengan skarf dan beret."},
    {"id": "sukan", "name": "Baju Sukan", "folder": "alif_sukan", "price": 150, "info": "Jersi rumah sukan biru dan ikat kepala kuning."},
    {"id": "bomba", "name": "Kadet Bomba", "folder": "alif_bomba", "price": 200, "info": "Pakaian kadet bomba dengan jalur pemantul."},
    {"id": "pahlawan", "name": "Pahlawan Melaka", "folder": "alif_pahlawan", "price": 300, "info": "Tengkolok merah, samping songket dan keris kecil."}
  ]
}
```

data/soalan/bab1.json · 422 lines (first 70 shown)

```
{
 "_info": "Bank soalan objektif Bab 1 untuk Bilik Pertarungan. Semua fakta diambil daripada data/zon_1_*.json. 'answer' = indeks jawapan betul (0-3). 'zon' = misi/subtopik soalan ini (untuk maklum balas Ujian Bab).",
 "bab": 1,
 "questions": [
  {
   "id": 0,
   "zon": "1_1",
   "q": "Kerajaan manakah yang paling awal?",
   "options": [
    "Kesultanan Johor-Riau",
    "Srivijaya",
    "Funan",
    "Kesultanan Melayu Melaka"
   ],
   "answer": 2,
   "explain": "Funan muncul pada abad pertama."
  },
  {
   "id": 1,
```

```
    "zon": "1_1",
    "q": "Bilakah Kesultanan Melayu Melaka muncul?",
    "options": [
     "Abad pertama",
     "Abad ke-7",
     "Abad kedua",
     "Abad ke-15"
    ],
    "answer": 3,
    "explain": "Kesultanan Melayu Melaka muncul pada abad ke-15."
   },
   {
    "id": 2,
    "zon": "1_1",
    "q": "Srivijaya berpusat di...",
    "options": [
     "Melaka",
     "Palembang",
     "Kedah",
     "Sungai Mekong"
    ],
    "answer": 1,
    "explain": "Srivijaya muncul pada abad ke-7 dan berpusat di Palembang."
   },
   {
    "id": 3,
    "zon": "1_1",
    "q": "Kerajaan manakah yang muncul pada abad ke-16?",
    "options": [
     "Srivijaya",
     "Funan",
     "Kesultanan Johor-Riau",
     "Champa"
    ],
    "answer": 2,
    "explain": "Kesultanan Johor-Riau muncul pada abad ke-16."
   },
   {
    "id": 4,
    "zon": "1_1",
    "q": "Champa muncul pada abad ke berapa?",
    "options": [
     "Abad ke-16",
     "Abad kedua",
     "Abad ke-15",
     "Abad ke-7"
    ],
    "answer": 1,
    "explain": "Champa muncul pada abad kedua."
   },
   {
```

data/soalan/bab2.json · 624 lines (first 70 shown)

```
{
 "_info": "Bank soalan objektif Bab 2 (Kebangkitan Nasionalisme) untuk Bilik Pertarungan dan Gelanggang. Sumber: STORYTELLING/bab2_kebangkitan_nasionalisme.json (soalan mcq). 'answer' = indeks jawapan betul (0-3). 'semak' = fakta perlu disemak guru. 'zon' = misi/subtopik soalan ini (untuk maklum balas Ujian Bab).",
 "bab": 2,
 "questions": [
  {
   "id": 0,
   "zon": "2_1",
   "src": "B2_Q001",
   "sub": "2.1",
   "q": "Apakah maksud asas nasionalisme dalam konteks perjuangan sesebuah bangsa?",
   "options": [
    "Usaha meluaskan tanah jajahan ke benua lain melalui kekuatan tentera",
    "Dasar menutup sempadan negara daripada sebarang hubungan perdagangan luar",
    "Sistem pentadbiran yang menyerahkan kuasa mutlak sepenuhnya kepada golongan bangsawan",
    "Perasaan cinta yang mendalam terhadap bangsa dan negara serta perjuangan menentang penjajahan"
   ],
   "answer": 3,
   "explain": "Jawapan: Perasaan cinta yang mendalam terhadap bangsa dan negara serta perjuangan menentang penjajahan."
  },
  {
   "id": 1,
   "zon": "2_1",
   "src": "B2_Q002",
   "sub": "2.1",
   "q": "Siapakah tokoh sejarawan yang menyatakan bahawa nasionalisme merupakan satu gerakan yang meletakkan kesetiaan seseorang hanya kepada negara atau bangsanya melalui karya 'The Idea of Nationalism'?",
   "options": [
```

```
    "Profesor Emeritus Tan Sri Dr. Khoo Kay Kim",
    "Hans Kohn",
    "Profesor William R. Roff",
    "Dr. R. Suntharalingam"
   ],
   "answer": 1,
   "explain": "Jawapan: Hans Kohn."
  },
  {
   "id": 2,
   "zon": "2_1",
   "src": "B2_Q003",
   "sub": "2.1",
   "q": "Menurut Profesor William R. Roff, apakah faktor yang mencetuskan kebangkitan nasionalisme Melayu pada dekad terakhir sebelum Perang Dunia Kedua?",
   "options": [
    "Ancaman terhadap hak kepentingan orang Melayu, faktor ekonomi, serta kebimbangan terhadap sekularisme dan imperialisme Barat",
    "Keinginan untuk menghapuskan sistem kesultanan tradisional",
    "Kejayaan kuasa Perancis menakluki seluruh kepulauan Asia Tenggara",
    "Gesaan daripada pihak British supaya orang Melayu menubuhkan parti komunis"
   ],
   "answer": 0,
   "explain": "Jawapan: Ancaman terhadap hak kepentingan orang Melayu, faktor ekonomi, serta kebimbangan terhadap sekularisme dan imperialisme Barat."
  },
  {
   "id": 3,
   "zon": "2_1",
   "src": "B2_Q004",
   "sub": "2.1",
   "q": "Tokoh manakah yang menegaskan bahawa perjuangan menentang penjajahan hendaklah mempunyai persatuan dan perpaduan yang kuat (National Fronts) dengan tetap berpegang pada tatatertib bangsa Melayu dan agama Islam?",
   "options": [
    "Dr. Burhanuddin al-Helmi",
    "Hans Kohn",
    "Profesor Datuk Zainal Abidin Abdul Wahid",
    "Dato' Onn Jaafar"
   ],
   "answer": 0,
   "explain": "Jawapan: Dr. Burhanuddin al-Helmi."
  },
  {
   "id": 4,
   "zon": "2_1",
   "src": "B2_Q005",
   "sub": "2.1",
   "q": "Apakah pandangan Profesor Datuk Zainal Abidin Abdul Wahid berkaitan kewujudan nasionalisme Melayu?",
```

`data/story.json` · 450 lines (first 70 shown)

```
{
  "_nota": "Semua dialog Bab 1 dan Bab 2. 's' = penutur (lihat characters.json), 'e' = ekspresi, 't' = teks. Dialog tokoh sejarah yang direka ditanda 'rekaan': true.",
  "peta_first": [
    {"s": "pau", "t": "Nirbana pecahkan Inskripsi Telaga Batu jadi empat serpihan. Satu serpihan, satu misi."},
    {"s": "pau", "t": "Pergi ke bangunan yang ada tanda seru. Bila cukup empat, kita boleh lawan penjaga pintu."},
    {"s": "pau", "t": "Guna WASD atau anak panah untuk berjalan. Atau ketik je ke mana kau nak pergi."}
  ],
  "flag_locked": [
    {"s": "pau", "t": "Belum lagi, kita habiskan yang ini dulu."}
  ],
  "flag_1": [
    {"s": "pau", "t": "Bendera 1: Warisan Negara Bangsa. Empat misi, empat subtopik."}
  ],
  "flag_1_done": [
    {"s": "pau", "e": "happy", "t": "Bendera 1 sudah berkibar! Kau faham Warisan Negara Bangsa."}
  ],
  "zone_locked": [
    {"s": "pau", "t": "Misi ini masih berkunci. Selesaikan misi sebelumnya dulu, ya."}
  ],
  "t5_locked": [
    {"s": "pau", "t": "Itu laluan ke Sejarah Tingkatan 5. Masih berkunci. Satu langkah demi satu langkah."}
  ],
  "arena_locked": [
    {"s": "pau", "t": "Arena Ulang Kaji! Di sini nanti kau boleh berlawan kuiz dengan rakan."},
    {"s": "pau", "t": "Ia dibuka selepas Bab 1 selesai. Jom tengok lobinya."}
  ],
  "z11_intro": [
    {"s": "pau", "e": "scared", "t": "Gerbang-gerbang ni bersepah. Seram sikit..."},
    {"s": "penjaga", "t": "Selamat datang ke Gerbang Masa, anak muda.", "rekaan": true}
  ],
  "z11_a_before": [
```

```
    {"s": "penjaga", "t": "Benang warisan yang menyambung gerbang-gerbang ini telah putus.", "rekaan": true},
    {"s": "penjaga", "t": "Hanya orang yang faham asal-usul negara bangsa boleh menyambungnya semula.", "rekaan": true}
,
    {"s": "penjaga", "t": "Susun lima gerbang ini ikut urutan masa.", "rekaan": true}
  ],
  "z11_a_after": [
    {"s": "penjaga", "t": "Lihat! Benang emas mengalir dari gerbang pertama ke gerbang terakhir.", "rekaan": true},
    {"s": "penjaga", "t": "Setiap kerajaan mewarisi ciri kerajaan sebelumnya.", "rekaan": true},
    {"s": "alif", "e": "happy", "t": "Jadi negara bangsa kita tak muncul tiba-tiba. Ia bersambung!"}
  ],
  "z11_b_before": [
    {"s": "pau", "e": "scared", "t": "Alif! Empat tiang ni runtuh!"},
    {"s": "penjaga", "t": "Itulah empat ciri negara bangsa: Raja, Undang-undang, Wilayah Pengaruh dan Rakyat.", "rekaan
": true},
    {"s": "penjaga", "t": "Letakkan setiap artifak bukti pada tiang yang betul.", "rekaan": true}
  ],
  "z11_b_after": [
    {"s": "penjaga", "t": "Keempat-empat tiang sudah berdiri. Gerbang ke Melaka kini terbuka.", "rekaan": true},
    {"s": "penjaga", "t": "Ini serpihan pertama Inskripsi Telaga Batu. Simpan baik-baik, anak muda.", "rekaan": true},
    {"s": "pau", "e": "happy", "t": "Satu serpihan dah dapat! Tinggal tiga lagi."}
  ],
  "z12_intro": [
    {"s": "pau", "e": "happy", "t": "Wah, Melaka abad ke-15! Bau rempah. Aku lapar lagi."},
    {"s": "syahbandar", "t": "Cepat, cepat! Kapal utusan sedang datang!", "rekaan": true},
    {"s": "syahbandar", "t": "Melaka mesti buktikan dirinya negara bangsa yang lengkap. Naikkan keenam-enam panji!", "r
ekaan": true},
    {"s": "pau", "t": "Ada enam tiang panji. Pergi ke setiap satu. Ikut mana-mana urutan pun boleh."}
  ],
  "z12_kerajaan_before": [
    {"s": "syahbandar", "t": "Panji Kerajaan! Susun pemerintah dan pembesar ikut lapisan.", "rekaan": true}
  ],
  "z12_kerajaan_after": [
    {"s": "pau", "t": "Raja pemerintah tertinggi, dibantu Sistem Pembesar Empat Lipatan. Panji Kerajaan naik!"}
  ],
  "z12_rakyat_before": [
    {"s": "syahbandar", "t": "Panji Rakyat! Kenal pasti golongan pemerintah dan golongan diperintah.", "rekaan": true}
  ],
  "z12_rakyat_after": [
    {"s": "pau", "t": "Pemerintah dan rakyat ada hubungan timbal balik. Panji Rakyat naik!"}
  ],
  "z12_kedaulatan_before": [
    {"s": "syahbandar", "t": "Kapal utusan sudah tiba! Sambut Laksamana Cheng Ho dengan hormat.", "rekaan": true},
```

data/ujian.json · 21 lines

```
{
  "_info": "Ujian Bab: ujian akhir setiap bab. Mesti lulus (tumbangkan bos) sebelum bab seterusnya dibuka. Soalan diamb
il secara rawak daripada data/soalan/bab<n>.json. Tiada had masa. hearts = nyawa Alif, need = jawapan betul untuk menan
g.",
  "1": {
    "boss": "Panglima Derhaka",
    "npc": "panglima",
    "pos": [10, 28],
    "where": "Pintu Kabus di barat Kampung Jejak",
    "background": "res://assets/backgrounds/bg_zon1_4_bukit_siguntang.png",
    "questions": 12, "hearts": 3, "need": 7,
    "before": "ujian_1_before", "win": "ujian_1_win", "fail": "ujian_fail", "npc_talk": "ujian_1_npc"
  },
  "2": {
    "boss": "Bayang Nirbana",
    "npc": "nirbana",
    "pos": [129, 31],
    "where": "depan Menara Pentadbiran",
    "background": "res://assets/backgrounds/bg_zon2_8.png",
    "questions": 12, "hearts": 3, "need": 7,
    "before": "ujian_2_before", "win": "ujian_2_win", "fail": "ujian_fail", "npc_talk": "ujian_2_npc"
  }
}
```

data/world/entities.json · 919 lines (first 70 shown)

```
{
 "spawn": [
  19,
  50
 ],
 "npcs": [
  {
   "id": "makcik",
   "pos": [
    31,
```

```
    33
   ],
   "facing": "south"
  },
  {
   "id": "nelayan",
   "pos": [
    64,
    43
   ],
   "facing": "east"
  },
  {
   "id": "penjaga",
   "pos": [
    34,
    17
   ],
   "facing": "south"
  },
  {
   "id": "syahbandar",
   "pos": [
    61,
    40
   ],
   "facing": "south"
  },
  {
   "id": "chengho",
   "pos": [
    65,
    45
   ],
   "facing": "south"
  },
  {
   "id": "sapurba",
   "pos": [
    53,
    14
   ],
   "facing": "east"
  },
  {
   "id": "demang",
   "pos": [
    61,
    14
   ],
   "facing": "west"
  },
  {
   "id": "guru_silat",
   "pos": [
    43,
    39
   ],
   "facing": "east",
   "wander": false
```

`data/world/lembah_layout.json` · 95 lines (first 70 shown)

```
{
 "_info": "Lembah Kebangkitan (Bab 2), pekan kolonial 1920-an. Disusun di PixelLab Map Workshop (terrain ops sama) dan dibaca oleh tools/gen_world.py. Koordinat global (x 84-139). Bangunan menghadap ke bawah, ke jalan di hadapannya. Objek : anchor = petak bawah-kiri.",
 "terrain": [
  ["rect", ".", 87, 4, 52, 48, "tanah pulau"],
  ["rect", "~", 87, 4, 7, 6, "teluk barat laut"],
  ["rect", "~", 131, 49, 8, 3, "teluk tenggara"],
  ["rect", ",", 118, 17, 9, 10, "padang"],
  ["rect", "j", 89, 31, 46, 2, "Jalan Besar"],
  ["rect", "j", 94, 13, 2, 34, "jalan barat"],
  ["rect", "j", 94, 13, 42, 2, "jalan utara"],
  ["rect", "j", 114, 13, 2, 34, "Jalan Raja"],
  ["rect", "j", 94, 45, 42, 2, "jalan selatan"],
  ["rect", "j", 134, 13, 2, 34, "jalan timur"],
  ["rect", "j", 99, 30, 2, 1, "tangga dewan"],
  ["rect", "j", 99, 44, 2, 1, "tangga falsafah"],
  ["rect", "j", 111, 47, 2, 2, "ke jeti"],
  ["rect", "j", 128, 44, 2, 1, "tangga balai serantau"],
  ["rect", "j", 99, 12, 2, 1, "tangga madrasah"],
```

```
  ["rect", "j", 111, 12, 2, 1, "tangga jelutong"],
  ["rect", "j", 129, 12, 2, 1, "tangga sitc"],
  ["rect", "j", 130, 30, 2, 1, "tangga menara"],
  ["rect", "r", 106, 7, 3, 42, "sungai"]
 ],
 "bridges": [
  [105, 14],
  [105, 32],
  [105, 46]
 ],
 "landmarks": [
  ["dewan", "O", 96, 29, {"sign": "dewan"}],
  ["papan_tokoh", "a", 103, 30, {"spot": "papan_tokoh"}],
  ["rumah_falsafah", "V", 97, 43, {"spot": "rumah_falsafah"}],
  ["rak_buku", "o", 103, 43, {"spot": "rak_buku"}],
  ["papan_slogan", "a", 97, 37, {"spot": "papan_slogan"}],
  ["jeti", "J", 111, 57, {"ground": "~s"}],
  ["peti_warkah", "c", 108, 49, {"spot": "peti_warkah"}],
  ["jeti_serantau", "b", 113, 55, {"spot": "jeti_serantau", "ground": "~"}],
  ["kapal_wap", "@", 116, 57, {"ground": "~"}],
  ["balai_serantau", "E", 126, 43, {"spot": "balai_serantau"}],
  ["kedai_deret_b", "7", 116, 43, {}],
  ["menara_jam", "8", 111, 29, {}],
  ["wakaf_padang", "9", 120, 23, {}],
  ["tugu_pahlawan", "y", 121, 29, {"spot": "tugu_pahlawan"}],
  ["menara", "t", 128, 29, {"spot": "menara"}],
  ["arena_lembah", "W", 128, 17, {"spot": "arena_lembah"}],
  ["madrasah", "N", 97, 11, {"sign": "madrasah"}],
  ["jelutong_press", "C", 109, 11, {"spot": "jelutong_press"}],
  ["kedai_deret_a", "6", 116, 10, {}],
  ["meja_pengarang", "d", 116, 12, {"spot": "meja_pengarang"}],
  ["gerai_akhbar", "$", 118, 12, {}],
  ["sitc", "m", 125, 11, {"sign": "sitc"}],
  ["papan_persatuan", "a", 131, 12, {"spot": "papan_persatuan"}]
 ],
 "misi_signs": [
  ["2.1", 101, 30],
  ["2.2", 97, 44],
  ["2.3", 109, 47],
  ["2.4", 126, 44],
  ["2.5", 117, 29],
  ["2.6", 113, 12],
  ["2.7", 127, 12],
  ["2.8", 132, 30]
 ],
 "npcs": [
  {"id": "profesor", "pos": [98, 30], "facing": "south"},
  {"id": "nakhoda", "pos": [113, 49], "facing": "south"},
  {"id": "pengarang", "pos": [120, 30], "facing": "south"},
  {"id": "syed_sheikh", "pos": [103, 12], "facing": "south"},
  {"id": "zaba", "pos": [126, 12], "facing": "south"},
  {"id": "eunos", "pos": [124, 12], "facing": "south"},
```

data/world/objects.txt · 60 lines (first 60 shown)

```
1
2
3
4
5                                                                                                P    P    P    P    P
P    P    P    P
6                                                                                                 P    P    P    P    P
 P    P    P    P
7                                                                                                     &    q    q       &
8                                                  P       P                                              &
   &             P
9                          T
                  P
10                         T     P           P            L                                    P           p
11                       T       T           q q              P                    P    P                  6
12                     P P PT T             q       Y          P                    P    N          C
     m
13                               T    P    P    p                                P                p    !  d $
       !   a    P
14                        P T                                  T                  P
                  P
```

```
                      5  1  3  4  2                    T                  &              n
                  P                                    T   P                          +        +
+
               P  T                                 Z     T                  &              p
                  T  T           A       n                                P              p
      a W       P
            P    T P T  T                                                  P              p
                 P
                 T   T T               q    q   T                         &              q
        P         T  T                 q   q    T  T
                T T T T T    T        T    p      T   T                      &           q p
                 T  T T T      G     T T T   q    T  T                     P        %   %      q%
                P
                 T  T T  T           T T       T   uT T                    P
9                P
        T    T    T       T          T          T                         &
            T              T                                                             q
        L         B   j            j        q               PP                          q
                B     F H     F      F H                  P                 P                          #
# # #          P
                                          l I        l      u               P
                 P
      P       u     fff    fff     fff    f   q             uT    P                 O         p q 8     !
 y     t
            T              K                                T S  P            v           ! a
          !
             T                          n               T  T       P
          P  uT T    F                                    uT    P P P         P              n
                P
           P  T          F                     l     l    T uT T P                S       z       z      z
     z     & z    P
                        F                          T            T                          q p
                 j                           q  T  T        T T P                      %   %       %
           PPT T F H     F                     q T      Q      T T              &               p q      %
 %   %   %
           T   T           l     a W           q T           uT  P             P        a
                P
          T T T  fff    fff                    T    Q     Q   u              P  &
           &      P
         PT T                  l          q    uT T
            T                             q p
        P   T  T             l     l                                         &
            T   U                         n        Q            b           P                  q
                P
                              T             p                                P       V    o   p       7
      E          P
         T    T          l  T                                               &     !
0     !
        T T                     T  kM   M
         T T T  l                          qq   q      Q   Q                                  n
          T           T          T      P  P   D           J                P &         +    q p !
+           P    P
         T T              T T        T                                      P
            P    P
         T P         S    T           P       P               X                          c
         P
           P           T   T T    P               P                          P    P    P    P
P    P    P
                       T uP       P                    b                       P    P    P    P        P
 P    P
                       T
                       P     T
            S
                                                                                                  b
```

```
                                                
```

```
..............jj.ss~
~~~~~~~~~ssss.....,..=.....===.=...,,,,,,..rrrr.........=.......ssss~~~~~~~~~~~~~~ss.....jj.........rrr.....jj...
..............jj.ss~
~~~~~~~~~sss.....,,...=.....===..==....,,,,..rrrr..,.......=.......ssss~~~~~~~~~~~~~~ss.....jj.........rrr.....jj...
..............jj.ss~
~~~~~~~sss.....,,,....=...===....==.........rrr.........==.....ssss~~~~~~~~~~~~~~~~ss.....jj.........rrr.....jj...
..............jj.ss~
~~~~~~ssss.....,,,....=.====.....===........rrr.........===....ssss~~~~~~~~~~~~~~~~ss.....jj.........rrr.....jj...
..............jj.ss~
~~~~~~sss.................===........========.rrr.====..========.ssss~~~~~~~~~~~~~~~~ss.....jj.........rrr.....jj...
..............jj.ss~
~~~~~~ss.................====.................rrr....=========..sss~~~~~~~~~~~~~~~~~~ss.....jj.........rrr.....jj...
..............jj.ss~
~~~~~~ss.............========.................rrr.......=====..sss~~~~~~~~~~~~~~~~~~ss.....jj.........rrr.....jj...
..............jj.ss~
~~~~~~sss............======.............,,,,,,...rrr.........==..ssss~~~~~~~~~~~~~~~~~~ss.....jj...jj.....rrr.....jj...
.........jj....jj.ss~
~~~~~~sss............=====.............,,,,,,..rrrrrr.........ssss~~~~~~~~~~~~~~~~~~ss.....jjjjjjjjjjj.rrr.jjjjjjjjj
jjjjjjjjjjjjjjjjj.ss~
~~~~~~~sss........======.................,,.......rr...........ssss~~~~~~~~~~~~~~~~~~ss.....jjjjjjjjjjj.rrr.jjjjjjjjj
jjjjjjjjj.........ss~
~~~~~~~sss........=====.............,.........................ssss~~~~~~~~~~~~~~~~~~~~ss.................rrr..jj......
..........sssssssss~
~~~~~~~~ss.........................,,,,,,.....sssssssssss..sssssss~~~~~~~~~~~~~~~~~~~~ss.................rrr..jj......
..........sssssssss~
~~~~~~~sss.........................,,,.....sssssssssssssssssssss~~~~~~~~~~~~~~~~~~~~ss.............................
..........ss~~~~~~~~~
~~~~~~~sss.........................,,.....ssssss~~~~~~~ssssss~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
sssssssssss~~~~~~~~~
~~~~~~~ssss.........................sssss~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
sssssssssss~~~~~~~~~
~~~~~~~~~sssss.....................sssss~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~ssssss..sssssssssss.....sssss~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~sssssssssssssssssss.sssssss~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~ssssss~~~~~~~ssssssssss~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~sssss~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~
```

data/zon_1_1.json · 257 lines (first 70 shown)

```
{
  "id": "1_1",
  "no": "1.1",
  "title": "Gerbang Masa",
  "subtopik": "1.1 Negara Bangsa Kerajaan Alam Melayu",
  "hint_go": "Gerbang Masa ada di utara Kampung Jejak. Ikut jalan ke atas.",
  "persoalan": "Dari mana asal negara bangsa kita?",
  "pemandu": "penjaga",
  "ganjaran": {
    "id": "serpihan1",
    "nama": "Serpihan 1: Batu Asas Telaga Batu",
    "icon": "prasasti",
    "info": "Serpihan pertama Inskripsi Telaga Batu. Rakyat Srivijaya meminum air sumpah di sini sebagai tanda setia kepada raja.",
    "cerita": [
      {
        "s": "pau",
        "t": "Serpihan pertama! Inskripsi Telaga Batu dari Srivijaya."
      },
      {
        "s": "pau",
        "t": "Dulu rakyat minum air sumpah yang dituang atas batu ni. Tanda setia, tak akan derhaka pada raja."
      },
      {
        "s": "alif",
        "t": "Jadi dari dulu lagi, rakyat dah jadi tonggak negara bangsa.",
        "e": "confident"
      }
    ]
  },
  "fakta_disemak": false,
  "background": "res://assets/backgrounds/bg_zon1_1_gerbang_masa.png",
  "music": "",
```

```
  "spawn": [
    640,
    660
  ],
  "walk_rect": [
    80,
    430,
    1120,
    270
  ],
  "intro": "z11_intro",
  "any_order": false,
  "objects": [
    {
      "id": "g_johor",
      "texture": "res://assets/objects/gate_johor.png",
      "pos": [
        190,
        440
      ],
      "scale": 4
    },
    {
      "id": "g_funan",
      "texture": "res://assets/objects/gate_funan.png",
      "pos": [
        400,
        420
      ],
      "scale": 4
    },
    {
      "id": "g_melaka",
      "texture": "res://assets/objects/gate_melaka.png",
      "pos": [
        880,
        420
      ],
```

data/zon_1_2.json · 494 lines (first 70 shown)

```
{
  "id": "1_2",
  "no": "1.2",
  "title": "Pelabuhan Melaka",
  "subtopik": "1.2 Ciri-ciri Negara Bangsa Kesultanan Melayu Melaka",
  "hint_go": "Pelabuhan Melaka ada di pantai timur. Cari Syahbandar.",
  "taunt": "nirbana_1_2",
  "persoalan": "Apa yang menjadikan Melaka sebuah negara bangsa yang lengkap?",
  "pemandu": "syahbandar",
  "ganjaran": {
    "id": "serpihan2",
    "nama": "Serpihan 2: Cap Mohor",
    "icon": "capmohor",
    "info": "Cap mohor ialah cop rasmi raja, satu lambang kebesaran Kesultanan Melayu Melaka yang mengesahkan titah.",
    "cerita": [
      {
        "s": "pau",
        "t": "Cap Mohor. Ni cop rasmi raja Melaka."
      },
      {
        "s": "pau",
        "t": "Surat atau titah yang ada cap ni, barulah sah. Ia lambang kebesaran, sama macam nobat, regalia dan warna kuning."
      }
    ]
  },
  "fakta_disemak": false,
  "background": "res://assets/backgrounds/bg_zon1_2_pelabuhan_melaka.png",
  "music": "",
  "spawn": [
    640,
    680
  ],
  "walk_rect": [
    40,
    400,
    1200,
    300
  ],
  "intro": "z12_intro",
  "any_order": true,
  "objects": [
```

```
    {
      "id": "kapal",
      "texture": "res://assets/proto/objects/kapal_utusan.png",
      "pos": [
        1010,
        250
      ],
      "scale": 4
    },
    {
      "id": "syahbandar",
      "sprite": "syahbandar",
      "pos": [
        560,
        520
      ],
      "label": "Syahbandar",
      "talk": "z12_intro"
    },
    {
      "id": "chengho",
      "sprite": "chengho",
      "pos": [
        760,
        400
      ],
      "label": "Laksamana Cheng Ho"
    },
    {
```

data/zon_1_3.json · 283 lines (first 70 shown)

```
{
  "id": "1_3",
  "no": "1.3",
  "title": "Balai Penghadapan",
  "subtopik": "1.3 Keunggulan Sistem Pentadbiran dan Perundangan",
  "hint_go": "Balai Penghadapan ada dalam Istana di Kota Melaka.",
  "persoalan": "Bagaimana Melaka kekal aman dan makmur?",
  "pemandu": "tunperak",
  "ganjaran": {
    "id": "serpihan3",
    "nama": "Serpihan 3: Naskhah Hukum Kanun",
    "icon": "kitab",
    "info": "Hukum Kanun Melaka (44 fasal) dan Undang-Undang Laut Melaka (25 fasal) menjaga ketenteraman di darat dan di laut.",
    "cerita": [
      {
        "s": "pau",
        "t": "Naskhah Hukum Kanun Melaka. Ada 44 fasal."
      },
      {
        "s": "pau",
        "t": "Undang-Undang Laut Melaka pula 25 fasal. Kat laut, nakhoda diibaratkan macam raja dalam kapal."
      },
      {
        "s": "alif",
        "t": "Patutlah pedagang suka datang Melaka. Ada peraturan, rasa selamat."
      }
    ]
  },
  "fakta_disemak": false,
  "background": "res://assets/backgrounds/bg_zon1_3_balai_penghadapan.png",
  "music": "",
  "spawn": [
    640,
    680
  ],
  "walk_rect": [
    120,
    380,
    1040,
    320
  ],
  "intro": "z13_intro",
  "any_order": false,
  "objects": [
    {
      "id": "sultan",
      "texture": "res://assets/portraits/sultan.png",
      "pos": [
        640,
        210
```

```
      ],
      "scale": 2.4
    },
    {
      "id": "tunperak",
      "sprite": "tunperak",
      "pos": [
        470,
        480
      ],
      "label": "Bendahara Tun Perak"
    },
    {
      "id": "mahkamah",
      "texture": "res://assets/icons/timbangan.png",
      "pos": [
        1110,
        600
      ],
      "scale": 4,
```

data/zon_1_4.json · 347 lines (first 70 shown)

```
{
  "id": "1_4",
  "no": "1.4",
  "title": "Bukit Siguntang",
  "subtopik": "1.4 Peranan Pemerintah dan Rakyat dalam Pembentukan Negara Bangsa",
  "hint_go": "Bukit Siguntang ada di timur laut. Ikut jalan dari Gerbang Masa.",
  "persoalan": "Apa yang berlaku jika pemerintah atau rakyat mungkir janji?",
  "pemandu": "demang",
  "ganjaran": {
    "id": "serpihan4",
    "nama": "Serpihan 4: Warkah Waadat",
    "icon": "gulungan",
    "info": "Waadat: janji Sang Sapurba dengan Demang Lebar Daun. Raja berlaku adil, rakyat kekal setia.",
    "cerita": [
      {
        "s": "pau",
        "t": "Warkah Waadat. Janji Sang Sapurba dengan Demang Lebar Daun."
      },
      {
        "s": "pau",
        "t": "Raja tak boleh zalim, rakyat tak boleh derhaka. Kalau dua-dua pegang janji, negeri aman."
      },
      {
        "s": "alif",
        "t": "Janji, setia, adil. Itulah teras Bab 1.",
        "e": "confident"
      }
    ]
  },
  "fakta_disemak": false,
  "background": "res://assets/backgrounds/bg_zon1_4_bukit_siguntang.png",
  "music": "",
  "spawn": [
    640,
    680
  ],
  "walk_rect": [
    60,
    470,
    1160,
    230
  ],
  "intro": "z14_intro",
  "any_order": false,
  "objects": [
    {
      "id": "pokok",
      "tree": true,
      "pos": [
        880,
        560
      ],
      "scale": 4,
      "label": "Pohon dan Akar"
    },
    {
      "id": "sapurba",
      "sprite": "sapurba",
      "pos": [
        330,
```

```
        600
      ],
      "label": "Sang Sapurba"
    },
    {
      "id": "demang",
      "sprite": "demang",
      "pos": [
        480,
        600
```

data/zon_2_1.json · 153 lines (first 70 shown)

```
{
  "id": "2_1",
  "no": "2.1",
  "fakta_disemak": false,
  "music": "",
  "any_order": false,
  "background": "res://assets/backgrounds/bg_zon2_1.png",
  "title": "Dewan Kedaulatan Rakyat",
  "subtopik": "2.1 Maksud Nasionalisme",
  "hint_go": "Dewan Kedaulatan Rakyat dekat je dengan Pintu Kabus. Ikut Jalan Besar, cari papan Misi 2.1.",
  "persoalan": "Apakah maksud nasionalisme?",
  "pemandu": "profesor",
  "ganjaran": {
    "id": "kitab_tafsiran",
    "nama": "Kitab Tafsiran Nasionalisme",
    "icon": "kitab",
    "info": "Himpunan tafsiran tokoh: Hans Kohn, William R. Roff, Dato' Onn Jaafar, Dr. Burhanuddin al-Helmi dan ramai lagi.",
    "cerita": [
      {
        "s": "pau",
        "t": "Kitab Tafsiran Nasionalisme dari Lembah Kebangkitan."
      },
      {
        "s": "pau",
        "t": "Hans Kohn kata, nasionalisme ialah fahaman yang meletakkan kesetiaan tertinggi individu kepada negara bangsa."
      },
      {
        "s": "pau",
        "t": "Kat Tanah Melayu, semangat ni mula subur awal abad ke-20."
      }
    ]
  },
  "intro": "z21_intro",
  "objects": [
    {
      "id": "profesor",
      "label": "Profesor Sejarawan"
    },
    {
      "id": "papan_tokoh",
      "label": "Papan Tafsiran Tokoh"
    }
  ],
  "steps": [
    {
      "at": "profesor",
      "before": "z21_a_before",
      "puzzle": "maksud",
      "after": "z21_a_after"
    },
    {
      "at": "papan_tokoh",
      "before": "z21_b_before",
      "puzzle": "tafsiran",
      "after": "z21_b_after"
    }
  ],
  "puzzles": {
    "maksud": {
      "type": "objektif",
      "title": "Apa Itu Nasionalisme?",
      "instruction": "Jawab soalan tentang maksud nasionalisme.",
      "hint": "Nasionalisme = cinta pada bangsa dan negara, serta berjuang menentang penjajahan.",
      "questions": [
        {
          "q": "Apakah maksud asas nasionalisme dalam konteks perjuangan sesebuah bangsa?",
          "options": [
            "Perasaan cinta yang mendalam terhadap bangsa dan negara serta perjuangan menentang penjajahan",
```

```
            "Usaha meluaskan tanah jajahan ke benua lain melalui kekuatan tentera",
            "Sistem pentadbiran yang menyerahkan kuasa mutlak sepenuhnya kepada golongan bangsawan",
```

data/zon_2_2.json · 196 lines (first 70 shown)

```
{
  "id": "2_2",
  "no": "2.2",
  "fakta_disemak": false,
  "music": "",
  "any_order": false,
  "background": "res://assets/backgrounds/bg_zon2_2.png",
  "title": "Rumah Falsafah Sejagat",
  "subtopik": "2.2 Perkembangan Idea Nasionalisme di Barat",
  "hint_go": "Rumah Falsafah Sejagat di selatan Dewan. Ikut jalan barat ke bawah, cari papan Misi 2.2.",
  "taunt": "nirbana_2_2",
  "persoalan": "Dari mana datangnya idea kedaulatan rakyat?",
  "pemandu": "profesor",
  "ganjaran": {
    "id": "bill_of_rights",
    "nama": "Bill of Rights & The Social Contract",
    "icon": "gulungan",
    "info": "Bill of Rights lahir daripada Revolusi Keagungan 1688. The Social Contract tulisan Rousseau menggerakkan R
evolusi Perancis 1789.",
    "cerita": [
      {
        "s": "pau",
        "t": "Bill of Rights, hasil Revolusi Keagungan 1688 di England. Kuasa raja mula dihadkan."
      },
      {
        "s": "pau",
        "t": "Sebelah dia, The Social Contract tulisan Rousseau. Idea ni yang bakar semangat Revolusi Perancis 1789."
      }
    ]
  },
  "intro": "z22_intro",
  "objects": [
    {
      "id": "rumah_falsafah",
      "label": "Rumah Falsafah Sejagat"
    },
    {
      "id": "rak_buku",
      "label": "Rak Buku Falsafah"
    },
    {
      "id": "papan_slogan",
      "label": "Papan Slogan"
    }
  ],
  "steps": [
    {
      "at": "rumah_falsafah",
      "before": "z22_a_before",
      "puzzle": "tiga_revolusi",
      "after": "z22_a_after"
    },
    {
      "at": "rak_buku",
      "before": "z22_b_before",
      "puzzle": "karya_tokoh",
      "after": "z22_b_after"
    },
    {
      "at": "papan_slogan",
      "before": "z22_c_before",
      "puzzle": "slogan",
      "after": "z22_c_after"
    }
  ],
  "puzzles": {
    "tiga_revolusi": {
      "type": "drag_match",
      "title": "Tiga Revolusi",
      "instruction": "Seret setiap revolusi ke kedudukan yang betul ikut urutan masa.",
      "hint": "Revolusi Keagungan di England berlaku tanpa pertumpahan darah, paling awal.",
```

data/zon_2_3.json · 173 lines (first 70 shown)

```
{
  "id": "2_3",
  "no": "2.3",
  "fakta_disemak": false,
  "music": "",
  "any_order": false,
  "background": "res://assets/backgrounds/bg_zon2_3.png",
  "title": "Dermaga Asia",
  "subtopik": "2.3 Perkembangan Nasionalisme di Asia",
  "hint_go": "Dermaga Asia di pantai selatan. Ikut jalan selatan, lintas jambatan, cari papan Misi 2.3.",
  "persoalan": "Bagaimana bangsa Asia bangkit menentang kuasa Barat?",
  "pemandu": "nakhoda",
  "ganjaran": {
    "id": "warkah_urwah",
    "nama": "Warkah al-Urwah al-Wuthqa",
    "icon": "nota",
    "info": "Akhbar 'Ikatan yang Kuat' (1883) oleh Sayyid Jamal al-Din al-Afghani menyebarkan idea Pan-Islamisme."
  },
  "intro": "z23_intro",
  "objects": [
    {
      "id": "nakhoda",
      "label": "Nakhoda Warkah"
    },
    {
      "id": "peti_warkah",
      "label": "Peti Warkah"
    }
  ],
  "steps": [
    {
      "at": "nakhoda",
      "before": "z23_a_before",
      "puzzle": "kapal_asia",
      "after": "z23_a_after"
    },
    {
      "at": "peti_warkah",
      "before": "z23_b_before",
      "puzzle": "peti_asia",
      "after": "z23_b_after"
    }
  ],
  "puzzles": {
    "kapal_asia": {
      "type": "drag_match",
      "title": "Kapal-kapal Asia",
      "instruction": "Seret setiap gerakan ke negara asalnya.",
      "hint": "San Min Chu-i dari China. Bushido dari Jepun. Satyagraha dari India.",
      "targets": [
        {
          "id": "india",
          "label": "India"
        },
        {
          "id": "china",
          "label": "China"
        },
        {
          "id": "mesir",
          "label": "Mesir & Uthmaniyah"
        },
        {
          "id": "jepun",
          "label": "Jepun"
        }
      ],
      "items": [
        {
          "id": "satya",
```

data/zon_2_4.json · 191 lines (first 70 shown)

```
{
  "id": "2_4",
  "no": "2.4",
  "fakta_disemak": false,
  "music": "",
  "any_order": false,
  "background": "res://assets/backgrounds/bg_zon2_4.png",
  "title": "Balai Serantau",
```

```
  "subtopik": "2.4 Perkembangan Nasionalisme di Asia Tenggara",
  "hint_go": "Balai Serantau di hujung timur jalan selatan. Cari papan Misi 2.4.",
  "taunt": "nirbana_2_4",
  "persoalan": "Bagaimana jiran-jiran kita memperjuangkan kemerdekaan?",
  "pemandu": "nakhoda",
  "ganjaran": {
    "id": "akhbar_kalayan",
    "nama": "Akhbar Kalayan",
    "icon": "nota",
    "info": "Akhbar Katipunan pimpinan Andres Bonifacio yang menyebarkan idea Revolusi Filipina 1896."
  },
  "intro": "z24_intro",
  "objects": [
    {
      "id": "jeti_serantau",
      "label": "Perahu Serantau"
    },
    {
      "id": "balai_serantau",
      "label": "Balai Serantau"
    }
  ],
  "steps": [
    {
      "at": "jeti_serantau",
      "before": "z24_a_before",
      "puzzle": "perahu_serantau",
      "after": "z24_a_after"
    },
    {
      "at": "balai_serantau",
      "before": "z24_b_before",
      "puzzle": "uji_serantau",
      "after": "z24_b_after"
    }
  ],
  "puzzles": {
    "perahu_serantau": {
      "type": "drag_match",
      "title": "Perahu Serantau",
      "instruction": "Seret setiap muatan ke perahu negara yang betul.",
      "hint": "Isu Kasut berlaku di kuil Buddha di Burma. Sumpah Pemuda di Indonesia.",
      "targets": [
        {
          "id": "fil",
          "label": "Filipina"
        },
        {
          "id": "bur",
          "label": "Burma"
        },
        {
          "id": "vie",
          "label": "Vietnam"
        },
        {
          "id": "ind",
          "label": "Indonesia"
        },
        {
          "id": "sia",
          "label": "Siam"
```

data/zon_2_5.json · 142 lines (first 70 shown)

```
{
  "id": "2_5",
  "no": "2.5",
  "fakta_disemak": false,
  "music": "",
  "any_order": false,
  "background": "res://assets/backgrounds/bg_zon2_5.png",
  "title": "Tugu Pahlawan",
  "subtopik": "2.5 Kesedaran Nasionalisme di Negara Kita",
  "hint_go": "Tugu Pahlawan di tepi padang, sebelah menara jam. Papan Misi 2.5 ada di Jalan Besar.",
  "persoalan": "Bagaimana kesedaran nasionalisme bermula di negara kita?",
  "pemandu": "pengarang",
  "ganjaran": {
    "id": "pena_kebangkitan",
    "nama": "Pena Kebangkitan",
    "icon": "nota",
    "info": "Lambang peralihan perjuangan: daripada penentangan bersenjata kepada penulisan dan persatuan."
  },
```

```
  "intro": "z25_intro",
  "objects": [
    {
      "id": "tugu_pahlawan",
      "label": "Tugu Pahlawan"
    },
    {
      "id": "pengarang",
      "label": "Ketua Pengarang"
    }
  ],
  "steps": [
    {
      "at": "tugu_pahlawan",
      "before": "z25_a_before",
      "puzzle": "pahlawan",
      "after": "z25_a_after"
    },
    {
      "at": "pengarang",
      "before": "z25_b_before",
      "puzzle": "senjata_minda",
      "after": "z25_b_after"
    }
  ],
  "puzzles": {
    "pahlawan": {
      "type": "drag_match",
      "title": "Tugu Pahlawan",
      "instruction": "Seret setiap pahlawan ke negeri tempat dia berjuang.",
      "hint": "Mat Salleh di Sabah, Rentap di Sarawak, Tok Janggut di Kelantan.",
      "targets": [
        {
          "id": "semenanjung_barat",
          "label": "Naning dan Perak"
        },
        {
          "id": "pantai_timur",
          "label": "Pahang, Kelantan, Terengganu"
        },
        {
          "id": "borneo",
          "label": "Sarawak dan Sabah"
        }
      ],
      "items": [
        {
          "id": "dol",
          "label": "Penghulu Dol Said",
          "target": "semenanjung_barat",
          "fact": "Naning menentang British."
        },
```

`data/zon_2_6.json` · 201 lines (first 70 shown)

```
{
  "id": "2_6",
  "no": "2.6",
  "fakta_disemak": false,
  "music": "",
  "any_order": false,
  "background": "res://assets/backgrounds/bg_zon2_6.png",
  "title": "Kompleks Intelektual & Jelutong Press",
  "subtopik": "2.6 Faktor Kemunculan Gerakan Nasionalisme",
  "hint_go": "Kompleks Intelektual di utara: madrasah dan Jelutong Press di jalan utara. Cari papan Misi 2.6.",
  "taunt": "nirbana_2_6",
  "persoalan": "Apakah faktor yang mencetuskan gerakan nasionalisme?",
  "pemandu": "syed_sheikh",
  "ganjaran": {
    "id": "jawi_peranakan",
    "nama": "Akhbar Jawi Peranakan",
    "icon": "nota",
    "info": "Antara akhbar terawal. Menegur campur tangan British dalam soal agama Islam di Singapura dan membangkitkan isu bahasa Melayu."
  },
  "intro": "z26_intro",
  "objects": [
    {
      "id": "syed_sheikh",
      "label": "Syed Sheikh al-Hadi"
    },
    {
      "id": "jelutong_press",
```

```
      "label": "Jelutong Press"
    },
    {
      "id": "meja_pengarang",
      "label": "Meja Pengarang"
    }
  ],
  "steps": [
    {
      "at": "syed_sheikh",
      "before": "z26_a_before",
      "puzzle": "madrasah",
      "after": "z26_a_after"
    },
    {
      "at": "jelutong_press",
      "before": "z26_b_before",
      "puzzle": "enam_faktor",
      "after": "z26_b_after"
    },
    {
      "at": "meja_pengarang",
      "before": "z26_c_before",
      "puzzle": "meja",
      "after": "z26_c_after"
    }
  ],
  "puzzles": {
    "madrasah": {
      "type": "drag_match",
      "title": "Madrasah Moden",
      "instruction": "Seret setiap madrasah ke tahun penubuhannya.",
      "hint": "Madrasah al-Attas di Johor Bahru paling awal.",
      "targets": [
        {
          "id": "y1914",
          "label": "1914"
        },
        {
          "id": "y1916",
          "label": "1916"
        },
        {
```

data/zon_2_7.json · 258 lines (first 70 shown)

```
{
  "id": "2_7",
  "no": "2.7",
  "fakta_disemak": false,
  "music": "",
  "any_order": false,
  "background": "res://assets/backgrounds/bg_zon2_7.png",
  "title": "Maktab Perguruan Sultan Idris",
  "subtopik": "2.7 Perkembangan Nasionalisme",
  "hint_go": "Maktab SITC di hujung timur jalan utara. Cari papan Misi 2.7.",
  "persoalan": "Bagaimana nasionalisme berkembang di negara kita?",
  "pemandu": "zaba",
  "ganjaran": {
    "id": "al_imam",
    "nama": "Naskhah Al-Imam (1906)",
    "icon": "kitab",
    "info": "Majalah Kaum Muda yang terbit pada 23 Julai 1906 di Singapura, menyeru umat Islam kembali kepada al-Quran dan Hadis.",
    "cerita": [
      {
        "s": "pau",
        "t": "Al-Imam. Terbit 23 Julai 1906 di Singapura."
      },
      {
        "s": "pau",
        "t": "Majalah Kaum Muda ni ajak orang Melayu maju melalui pendidikan, dan kembali kepada al-Quran dan Hadis."
      }
    ]
  },
  "intro": "z27_intro",
  "objects": [
    {
      "id": "zaba",
      "label": "Za'ba"
    },
    {
      "id": "eunos",
```

```
      "label": "Mohammad Eunos Abdullah"
    },
    {
      "id": "ibrahim",
      "label": "Ibrahim Haji Yaakob"
    },
    {
      "id": "papan_persatuan",
      "label": "Papan Persatuan"
    }
  ],
  "steps": [
    {
      "at": "zaba",
      "before": "z27_a_before",
      "puzzle": "intelektual",
      "after": "z27_a_after"
    },
    {
      "at": "eunos",
      "before": "z27_b_before",
      "puzzle": "garis_persatuan",
      "after": "z27_b_after"
    },
    {
      "at": "ibrahim",
      "before": "z27_c_before",
      "puzzle": "karya_penulis",
      "after": "z27_c_after"
    },
    {
      "at": "papan_persatuan",
      "before": "z27_d_before",
      "puzzle": "persatuan_kaum",
```

data/zon_2_8.json · 196 lines (first 70 shown)

```
{
  "id": "2_8",
  "no": "2.8",
  "fakta_disemak": false,
  "music": "",
  "any_order": false,
  "background": "res://assets/backgrounds/bg_zon2_8.png",
  "title": "Menara Pentadbiran Kolonial",
  "subtopik": "2.8 Kesan Perkembangan Nasionalisme",
  "hint_go": "Menara Pentadbiran di hujung timur Jalan Besar. Cari papan Misi 2.8.",
  "persoalan": "Apakah kesan perkembangan nasionalisme?",
  "pemandu": "ibu_zain",
  "ganjaran": {
    "id": "bulan_melayu",
    "nama": "Majalah Bulan Melayu",
    "icon": "kitab",
    "info": "Diterbitkan pada 1930 oleh guru wanita di Johor Bahru yang digerakkan oleh Ibu Zain.",
    "cerita": [
      {
        "s": "pau",
        "t": "Majalah Bulan Melayu, 1930."
      },
      {
        "s": "pau",
        "t": "Diterbitkan oleh guru-guru perempuan Melayu Johor, digerakkan oleh Ibu Zain. Perempuan pun bangkit!"
      },
      {
        "s": "alif",
        "t": "Ibu Zain memang power.",
        "e": "happy"
      }
    ]
  },
  "intro": "z28_intro",
  "objects": [
    {
      "id": "ibu_zain",
      "label": "Ibu Zain"
    },
    {
      "id": "menara",
      "label": "Menara Pentadbiran Kolonial"
    }
  ],
  "steps": [
    {
```

```
      "at": "ibu_zain",
      "before": "z28_a_before",
      "puzzle": "tiga_kesan",
      "after": "z28_a_after"
    },
    {
      "at": "menara",
      "before": "z28_b_before",
      "puzzle": "gabenor",
      "after": "z28_b_after"
    }
  ],
  "puzzles": {
    "tiga_kesan": {
      "type": "drag_match",
      "title": "Tiga Kesan",
      "instruction": "Seret setiap kesan ke aspek yang betul.",
      "hint": "Bilangan pegawai Melayu dalam MCS ialah kesan politik.",
      "targets": [
        {
          "id": "pol",
          "label": "Politik"
        },
        {
```

APPENDIX C

Asset Catalogue

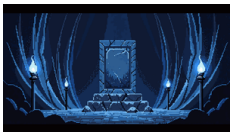
Every image in the game (1753 PNG files), grouped by folder. Character sprites are shown once per sprite set (first frame) with the number of frames in the set. Captions give the file name and pixel size; a chapter tag is shown where the asset belongs to one chapter.

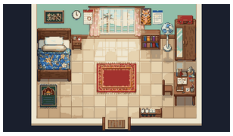
assets/backgrounds/ (22)


bg_bilik_alif_malam.png
320×180


bg_bilik_alif_malam_v2.png
320×180


bg_gelanggang.png
1280×720


bg_gua_warisan.png
640×360


bg_int_bilik_alif.png
320×180

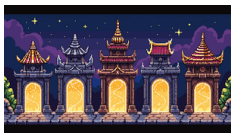

bg_int_jelutong.png
320×180 · Bab 2


bg_int_sitc.png
320×180 · Bab 2


bg_pelabuhan_huru.png
640×360


bg_peta_jejak_pembelajaran.png
320×180


bg_skrin_tajuk.png
320×180

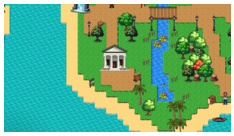

bg_zon1_1_gerbang_masa.png
320×180 · Bab 1


bg_zon1_2_pelabuhan_melaka.png
320×180 · Bab 1


bg_zon1_3_balai_penghadapan.png
320×180 · Bab 1


bg_zon1_4_bukit_siguntang.png
320×180 · Bab 1


bg_zon2_1.png
640×360 · Bab 2


bg_zon2_2.png
640×360 · Bab 2


bg_zon2_3.png
640×360 · Bab 2


bg_zon2_4.png
640×360 · Bab 2


bg_zon2_5.png
640×360 · Bab 2


bg_zon2_6.png
640×360 · Bab 2


bg_zon2_7.png
640×360 · Bab 2


bg_zon2_8.png
640×360 · Bab 2

assets/backgrounds/tajuk/ (1)


tajuk_penuh.png
320×180

assets/characters/ (1)



pau.png
64×64

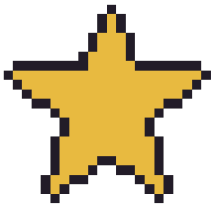
assets/icons/ (27)



beg.png
24×24



bendera.png
24×24



bintang.png
24×24



capmohor.png
64×64



gulungan.png
64×64



kain.png
64×64



kapal.png
24×24



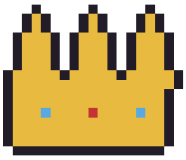
kitab.png
64×64



kumpulan.png
24×24



kunci.png
24×24



mahkota.png
24×24



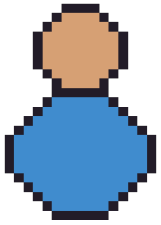
muzium.png
24×24



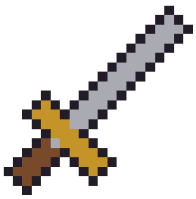
nobat.png
64×64



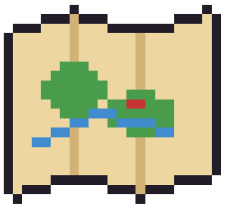
nota.png
24×24



orang.png
24×24



pedang.png
24×24



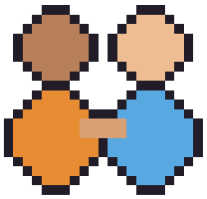
peta.png
24×24



pohon.png
24×24

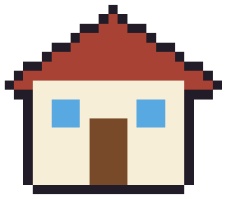


prasasti.png
64×64

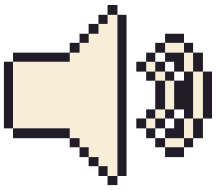


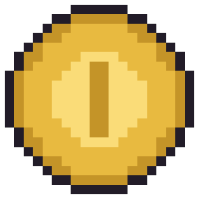
rakan.png
24×24

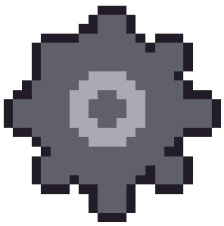

regalia.png
24×24


sekolah.png
24×24


siva.png
64×64


suara.png
24×24


syiling.png
24×24


tetapan.png
24×24


timbangan.png
24×24

assets/items/ (10)


item_cap_mohor.png
64×64


item_gulungan_undang_undang.png
64×64


item_kain_kuning_diraja.png
64×64


item_kitab_kutara_manawa.png
64×64


item_nobat.png
64×64


item_patung_dewa_siva.png
64×64


item_peta_funan.png
64×64


item_prasasti_telaga_batu.png
64×64 · Bab 1


sheet_artifak_zon1_1.png
128×128 · Bab 1


sheet_regalia_zon1_2.png
128×128 · Bab 1

assets/objects/ (80)


arena.png
96×80


balai_serantau.png
96×80


bangku.png
32×32


banyan.png
128×128


basikal.png
32×32


batu_renungan.png
32×32

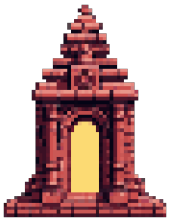

bunga_kuning.png
32×32


bunga_merah_jambu.png
32×32


bush.png
32×32


bush_bunga.png
32×32


dewan.png
128×112

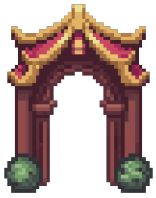

gate_champa.png
48×64


gate_funan.png
48×64


gate_johor.png
48×64


gate_locked.png
64×64


gate_melaka.png
48×64 · Bab 1


gate_open.png
64×64


gate_srivijaya.png
48×64


gerai_akhbar.png
32×32


gerai_buah.png
64×64


gerai_rempah.png
64×64

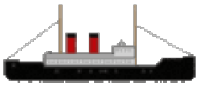

gudang.png
96×80


istana.png
160×128


jambatan.png
80×40


jelutong_press.png
96×96 · Bab 2


jetty.png
32×96


kapal_wap.png
128×80


kayu_api.png
32×32


kayu_balak.png
32×32


kedai_biru.png
96×96


kedai_deret_a.png
128×96


kedai_deret_b.png
128×96


kedai_merah.png
96×96


lampu_kolonial.png
32×48


madrasah.png
96×96

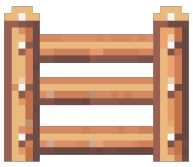

meja_mahkamah.png
48×32


meja_pengarang.png
32×32


menara.png
96×128 · Bab 2


menara_jam.png
48×96 · Bab 2


monument.png
64×64

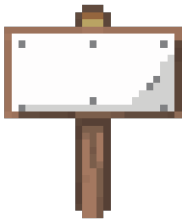

pagar.png
32×32


pagar_besi.png
32×32


palm.png
48×64


panji_naik.png
32×64


panji_turun.png
32×64


papan_misi.png
32×32


papan_notis.png
32×32


patung_latihan.png
32×48


pelita.png
32×48


perahu.png
48×32


perigi.png
32×48


peti.png
32×32


pintu_kabus.png
64×64


pokok_angsana.png
48×56


pokok_getah.png
48×56


rak_buku.png
32×32

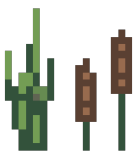

rocks.png
32×32


rumah_biru.png
96×96


rumah_falsafah.png
96×96


rumah_hijau.png
96×96


rumah_merah.png
96×96


rumput_air.png
32×32


ship.png
96×96


sign.png
32×32


sitc.png
160×112 · Bab 2


sultan_siluet.png
32×48


tempayan.png
32×32


teratai.png
32×32


tiang_berdiri.png
64×64


tiang_runtuh.png
64×48


tiang_telegraf.png
32×64


tong.png
32×32


tree.png
48×56


tree_ketapang.png
48×56


tree_mempelam.png
48×56


tree_rambutan.png
48×56


tree_round.png
48×56


tugu_pahlawan.png
64×80


tunggul.png
32×32


wakaf_padang.png
80×80

assets/objects/bilik/ (1)


beg.png
15×16

assets/portraits/ (63)


alif_bomba_confident.png
64×64


alif_bomba_confused.png
64×64


alif_bomba_happy.png
64×64


alif_bomba_neutral.png
64×64


alif_bomba_worried.png
64×64


alif_confident.png
64×64


alif_confused.png
64×64


alif_happy.png
64×64


alif_melayu_confident.png
64×64


alif_melayu_confused.png
64×64


alif_melayu_happy.png
64×64


alif_melayu_neutral.png
64×64


alif_melayu_worried.png
64×64


alif_neutral.png
64×64


alif_pahlawan_confident.png
64×64


alif_pahlawan_confused.png
64×64


alif_pahlawan_happy.png
64×64


alif_pahlawan_neutral.png
64×64


alif_pahlawan_worried.png
64×64


alif_pengakap_confident.png
64×64


alif_pengakap_confused.png
64×64


alif_pengakap_happy.png
64×64


alif_pengakap_neutral.png
64×64


alif_pengakap_worried.png
64×64


alif_sukan_confident.png
64×64


alif_sukan_confused.png
64×64


alif_sukan_happy.png
64×64


alif_sukan_neutral.png
64×64


alif_sukan_worried.png
64×64


alif_worried.png
64×64

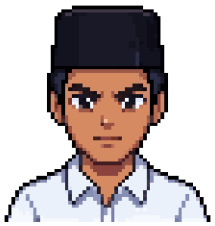

chengho.png
64×64


demang.png
64×64


eunos.png
64×64


gabenor.png
64×64


guru_silat.png
64×64


ibrahim.png
64×64


ibu_zain.png
64×64


makcik.png
64×64


nakhoda.png
64×64


nelayan.png
64×64


nirbana.png
64×64

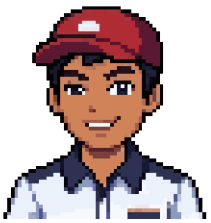

panglima.png
64×64


pau_happy.png
48×48


pau_neutral.png
48×48


pau_scared.png
48×48

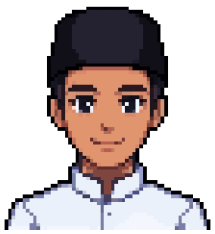

pedagang.png
64×64


pelajar_1.png
64×64


pelajar_2.png
64×64


pelajar_3.png
64×64


pelajar_4.png
64×64


pelatih.png
64×64


pencetak.png
64×64


pengarang.png
64×64


penjaga.png
64×64


pesilat_amir.png
64×64


pesilat_hafiz.png
64×64


profesor.png
64×64

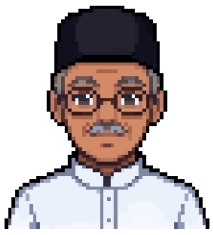

sapurba.png
64×64


sultan.png
64×64


syahbandar.png
64×64


syed_sheikh.png
64×64

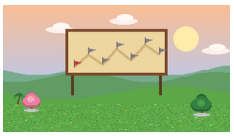

tunperak.png
64×64


zaba.png
64×64

assets/proto/bg/ (8)



bilik_alif.png
320×180



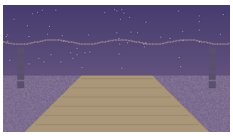
bukit_peta.png
320×180



papan_siasatan.png
320×160



peta_jejak.png
640×360



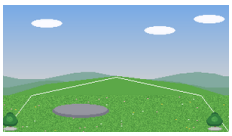
zon_1_1.png
320×180 · Bab 1



zon_1_2.png
320×180 · Bab 1



zon_1_3.png
320×180 · Bab 1

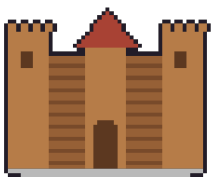


zon_1_4.png
320×180 · Bab 1

assets/proto/objects/ (21)



bangunan_1_1.png
56×50



bangunan_1_2.png
56×50



bangunan_1_3.png
56×50



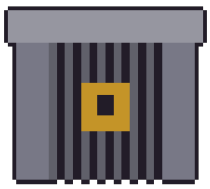
bangunan_1_4.png
56×50



bangunan_arena.png
56×50



bangunan_muzium.png
56×50



bangunan_t5.png
56×50



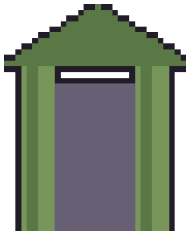
bendera_aktif.png
16×32



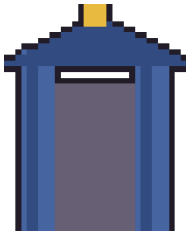
bendera_kunci.png
16×32



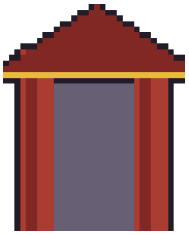
gerbang_champa.png
32×40 · Bab 1



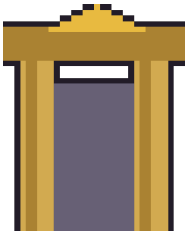
gerbang_funan.png
32×40 · Bab 1



gerbang_johor.png
32×40 · Bab 1



gerbang_melaka.png
32×40 · Bab 1



gerbang_srivijaya.png
32×40 · Bab 1



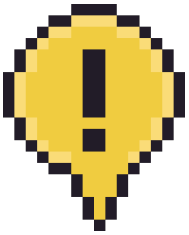
kapal_utusan.png
64×48



panji_naik.png
16×48



panji_turun.png
16×48



penanda.png
16×20



pokok_0.png
64×72



pokok_1.png
64×72


pokok_2.png
64×72

assets/proto/sprites/ (13)


alif.png
48×96


chengho.png
48×96

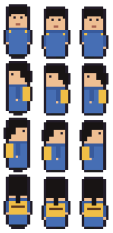

demang.png
48×96


pau.png
64×16


pedagang.png
48×96


pelajar_1.png
48×96


pelajar_2.png
48×96


pelajar_3.png
48×96


pelajar_4.png
48×96

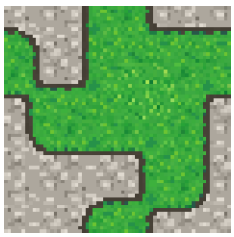

penjaga.png
48×96

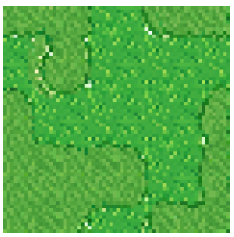

sapurba.png
48×96

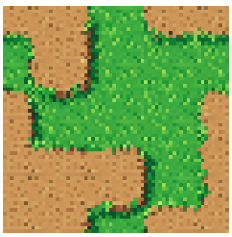

syahbandar.png
48×96

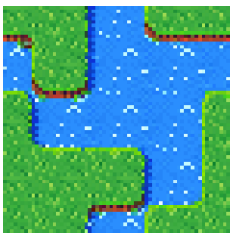

tunperak.png
48×96

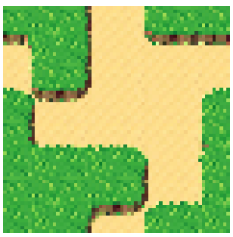
assets/tiles/ (6)

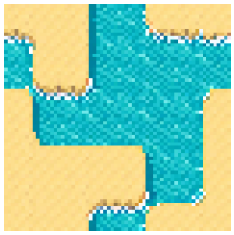

grass_jalan_image.png
64×64


grass_meadow_image.png
64×64


grass_path_image.png
64×64


river_grass_image.png
64×64


sand_grass_image.png
64×64


sea_sand_image.png
64×64

assets/ui/ (4)


bendera_berkibar.png
48×64


bendera_berkunci.png
48×64


bg_papan_siasatan.png
320×180


sheet_bendera_kunci_buka.png
96×64

assets/characters32/ (182 sprite sets, 1496 frames)


alif
4 frames · 36×36


alif_bomba
4 frames · 36×36


alif_bomba_cross
6 frames · 36×36


alif_bomba_fall
7 frames · 36×36


alif_bomba_flykick
6 frames · 36×36


alif_bomba_getup
5 frames · 36×36


alif_bomba_hurt
6 frames · 36×36


alif_bomba_jab
3 frames · 36×36


alif_bomba_kick
7 frames · 36×36


alif_bomba_pickup
5 frames · 36×36


alif_bomba_roundhouse
7 frames · 36×36


alif_bomba_special
4 frames · 36×36


alif_bomba_stance
8 frames · 36×36


alif_bomba_throw
7 frames · 36×36


alif_bomba_walk
24 frames · 36×36


alif_cross
6 frames · 36×36


alif_fall
7 frames · 36×36


alif_flykick
6 frames · 36×36


alif_getup
5 frames · 36×36


alif_hurt
6 frames · 36×36


alif_jab
3 frames · 36×36


alif_kick
7 frames · 36×36


alif_melayu
4 frames · 36×36


alif_melayu_cross
6 frames · 36×36


alif_melayu_fall
7 frames · 36×36


alif_melayu_flykick
6 frames · 36×36


alif_melayu_getup
5 frames · 36×36


alif_melayu_hurt
6 frames · 36×36


alif_melayu_jab
3 frames · 36×36


alif_melayu_kick
7 frames · 36×36


alif_melayu_pickup
5 frames · 36×36


alif_melayu_roundhouse
7 frames · 36×36


alif_melayu_special
4 frames · 36×36


alif_melayu_stance
8 frames · 36×36


alif_melayu_throw
7 frames · 36×36


alif_melayu_walk
24 frames · 36×36


alif_pahlawan
4 frames · 36×36


alif_pahlawan_cross
6 frames · 36×36


alif_pahlawan_fall
7 frames · 36×36


alif_pahlawan_flykick
6 frames · 36×36


alif_pahlawan_getup
5 frames · 36×36


alif_pahlawan_hurt
6 frames · 36×36


alif_pahlawan_jab
3 frames · 36×36


alif_pahlawan_kick
7 frames · 36×36


alif_pahlawan_pickup
5 frames · 36×36


alif_pahlawan_roundhouse
7 frames · 36×36


alif_pahlawan_special
4 frames · 36×36


alif_pahlawan_stance
8 frames · 36×36


alif_pahlawan_throw
7 frames · 36×36


alif_pahlawan_walk
24 frames · 36×36


alif_pengakap
4 frames · 36×36


alif_pengakap_cross
6 frames · 36×36


alif_pengakap_fall
7 frames · 36×36


alif_pengakap_flykick
6 frames · 36×36


alif_pengakap_getup
5 frames · 36×36


alif_pengakap_hurt
6 frames · 36×36


alif_pengakap_jab
3 frames · 36×36


alif_pengakap_kick
7 frames · 36×36


alif_pengakap_pickup
5 frames · 36×36


alif_pengakap_roundhouse
7 frames · 36×36


alif_pengakap_special
4 frames · 36×36


alif_pengakap_stance
8 frames · 36×36


alif_pengakap_throw
7 frames · 36×36


alif_pengakap_walk
24 frames · 36×36


alif_pickup
5 frames · 36×36


alif_roundhouse
7 frames · 36×36


alif_special
4 frames · 36×36


alif_stance
8 frames · 36×36


alif_sukan
4 frames · 36×36


alif_sukan_cross
6 frames · 36×36


alif_sukan_fall
7 frames · 36×36


alif_sukan_flykick
6 frames · 36×36


alif_sukan_getup
5 frames · 36×36


alif_sukan_hurt
6 frames · 36×36


alif_sukan_jab
3 frames · 36×36


alif_sukan_kick
7 frames · 36×36


alif_sukan_pickup
5 frames · 36×36


alif_sukan_roundhouse
7 frames · 36×36


alif_sukan_special
4 frames · 36×36


alif_sukan_stance
8 frames · 36×36


alif_sukan_throw
7 frames · 36×36


alif_sukan_walk
24 frames · 36×36


alif_tanpa_beg
4 frames · 36×36


alif_tanpa_beg_walk
24 frames · 36×36


alif_throw
7 frames · 36×36


alif_walk
24 frames · 36×36


chengho
4 frames · 36×36 · Bab 1


chengho_walk
24 frames · 36×36 · Bab 1


demang
4 frames · 36×36 · Bab 1


demang_walk
24 frames · 36×36 · Bab 1


eunos
4 frames · 36×36 · Bab 2


eunos_walk
24 frames · 36×36 · Bab 2


gabenor
4 frames · 36×36


ibrahim
4 frames · 36×36 · Bab 2


ibrahim_walk
24 frames · 36×36 · Bab 2


ibu_zain
4 frames · 36×36 · Bab 2


ibu_zain_walk
24 frames · 36×36


makcik
4 frames · 36×36


makcik_walk
24 frames · 36×36


nakhoda
4 frames · 36×36 · Bab 2


nakhoda_walk
24 frames · 36×36 · Bab 2


nelayan
4 frames · 36×36


nelayan_walk
24 frames · 36×36


nirbana
4 frames · 36×36


nirbana_hover
9 frames · 36×36


panglima
4 frames · 36×36


pau
1 frames · 32×32


pelatih
4 frames · 36×36


pencetak
4 frames · 36×36


pengarang
4 frames · 36×36 · Bab 2


pengarang_walk
24 frames · 36×36 · Bab 2


penjaga
4 frames · 36×36 · Bab 1


penjaga_walk
24 frames · 36×36 · Bab 1


profesor
4 frames · 36×36 · Bab 2


profesor_walk
24 frames · 36×36 · Bab 2


rakan_aisyah
4 frames · 36×36


rakan_aisyah_cross
6 frames · 36×36


rakan_aisyah_fall
7 frames · 36×36


rakan_aisyah_flykick
6 frames · 36×36


rakan_aisyah_getup
5 frames · 36×36


rakan_aisyah_hurt
6 frames · 36×36


rakan_aisyah_jab
3 frames · 36×36


rakan_aisyah_kick
7 frames · 36×36


rakan_aisyah_pickup
5 frames · 36×36


rakan_aisyah_roundhouse
7 frames · 36×36


rakan_aisyah_special
4 frames · 36×36


rakan_aisyah_stance
8 frames · 36×36


rakan_aisyah_throw
7 frames · 36×36


rakan_aisyah_walk
24 frames · 36×36


rakan_daniel
4 frames · 36×36


rakan_daniel_cross
6 frames · 36×36


rakan_daniel_fall
7 frames · 36×36


rakan_daniel_flykick
6 frames · 36×36


rakan_daniel_getup
5 frames · 36×36


rakan_daniel_hurt
6 frames · 36×36


rakan_daniel_jab
3 frames · 36×36


rakan_daniel_kick
7 frames · 36×36


rakan_daniel_pickup
5 frames · 36×36


rakan_daniel_roundhouse
7 frames · 36×36


rakan_daniel_special
4 frames · 36×36


rakan_daniel_stance
8 frames · 36×36


rakan_daniel_throw
7 frames · 36×36


rakan_daniel_walk
24 frames · 36×36


rakan_meiling
4 frames · 36×36


rakan_meiling_cross
6 frames · 36×36


rakan_meiling_fall
7 frames · 36×36


rakan_meiling_flykick
6 frames · 36×36


rakan_meiling_getup
5 frames · 36×36


rakan_meiling_hurt
6 frames · 36×36


rakan_meiling_jab
3 frames · 36×36


rakan_meiling_kick
7 frames · 36×36


rakan_meiling_pickup
5 frames · 36×36


rakan_meiling_roundhouse
7 frames · 36×36


rakan_meiling_special
4 frames · 36×36


rakan_meiling_stance
8 frames · 36×36


rakan_meiling_throw
7 frames · 36×36


rakan_meiling_walk
24 frames · 36×36


rakan_ravi
4 frames · 36×36


rakan_ravi_cross
6 frames · 36×36


rakan_ravi_fall
7 frames · 36×36


rakan_ravi_flykick
6 frames · 36×36


rakan_ravi_getup
5 frames · 36×36


rakan_ravi_hurt
6 frames · 36×36


rakan_ravi_jab
3 frames · 36×36


rakan_ravi_kick
7 frames · 36×36


rakan_ravi_pickup
5 frames · 36×36


rakan_ravi_roundhouse
7 frames · 36×36


rakan_ravi_special
4 frames · 36×36


rakan_ravi_stance
8 frames · 36×36


rakan_ravi_throw
7 frames · 36×36


rakan_ravi_walk
24 frames · 36×36


sapurba
4 frames · 36×36 · Bab 1


sapurba_walk
24 frames · 36×36 · Bab 1


syahbandar
4 frames · 36×36 · Bab 1


syahbandar_walk
24 frames · 36×36 · Bab 1


syed_sheikh
4 frames · 36×36 · Bab 2


syed_sheikh_walk
24 frames · 36×36


tunperak
4 frames · 36×36 · Bab 1


tunperak_walk
24 frames · 36×36 · Bab 1


zaba
4 frames · 36×36 · Bab 2


zaba_walk
24 frames · 36×36 · Bab 2

Audio (21)

File	Length	Format
assets/audio/music/bilik_base.wav	30.0 s	WAV 22050 Hz, 1 ch
assets/audio/music/bilik_melody.wav	30.0 s	WAV 22050 Hz, 1 ch
assets/audio/music/fokus.wav	10.7 s	WAV 22050 Hz, 1 ch
assets/audio/music/gerbang_base.wav	27.4 s	WAV 22050 Hz, 1 ch
assets/audio/music/gerbang_melody.wav	27.4 s	WAV 22050 Hz, 1 ch
assets/audio/music/kampung_base.wav	20.9 s	WAV 22050 Hz, 1 ch
assets/audio/music/kampung_melody.wav	20.9 s	WAV 22050 Hz, 1 ch
assets/audio/music/melaka_base.wav	24.0 s	WAV 22050 Hz, 1 ch
assets/audio/music/melaka_melody.wav	24.0 s	WAV 22050 Hz, 1 ch
assets/audio/music/pelabuhan_base.wav	17.8 s	WAV 22050 Hz, 1 ch
assets/audio/music/pelabuhan_melody.wav	17.8 s	WAV 22050 Hz, 1 ch
assets/audio/music/permulaan_base.wav	22.9 s	WAV 22050 Hz, 1 ch
assets/audio/music/permulaan_melody.wav	22.9 s	WAV 22050 Hz, 1 ch
assets/audio/music/siguntang_base.wav	29.1 s	WAV 22050 Hz, 1 ch
assets/audio/music/siguntang_melody.wav	29.1 s	WAV 22050 Hz, 1 ch
assets/audio/music/st_correct.wav	3.2 s	WAV 22050 Hz, 1 ch
assets/audio/music/st_item.wav	3.6 s	WAV 22050 Hz, 1 ch
assets/audio/music/st_levelup.wav	3.4 s	WAV 22050 Hz, 1 ch
assets/audio/music/st_zone.wav	4.1 s	WAV 22050 Hz, 1 ch
assets/audio/music/tajuk_base.wav	25.3 s	WAV 22050 Hz, 1 ch
assets/audio/music/tajuk_melody.wav	25.3 s	WAV 22050 Hz, 1 ch

Fonts and other media (8)

File	Size
assets/fonts/atkinson-hyperlegible-latin-400-normal.woff2	17 KB
assets/fonts/atkinson-hyperlegible-latin-700-normal.woff2	17 KB
assets/fonts/lexend-latin-400-normal.woff2	14 KB
assets/fonts/lexend-latin-700-normal.woff2	14 KB
assets/fonts/opendyslexic-latin-400-normal.woff2	113 KB
assets/fonts/opendyslexic-latin-700-normal.woff2	118 KB
assets/fonts/pixelify-sans-latin-500-normal.woff2	8 KB
assets/fonts/pixelify-sans-latin-700-normal.woff2	8 KB